TABLE OF CONTENTS

1. Introduction and Objectives....3
1.1 Title of the Project....3
1.2 Introduction....3
1.3 Evolution of the Project since the Synopsis....4
1.4 Objectives of the Project....5
1.5 Project Category....6
1.6 Tools, Platform and Environment Used....6
1.7 Hardware Requirements....7
1.8 Software Requirements....7
2. System Analysis....9
2.1 Identification of Need....9
2.1.1 Problem Definition....9
2.1.2 The Existing System....9
2.1.3 The Proposed System....10
2.1.4 Feasibility Study....11
2.2 Project Planning and Project Scheduling....12
2.2.1 Project Plan....12
2.2.2 Activity Estimates and Critical Path (PERT Analysis)....13
2.2.3 PERT Chart....14
2.2.4 Gantt Chart....16
2.3 Software Requirement Specification (SRS)....18
1. Introduction....18
2. General Description....19
3. Specific Requirements....22
2.4 Software Engineering Paradigm Applied....26
2.5 Data Models and Analysis Diagrams....27
2.5.1 Data Flow Diagrams (DFD)....27
2.5.2 Control Flow Diagram....33
2.5.3 State Diagrams....34
2.5.4 Sequence Diagrams....39
2.5.5 Entity Relationship Model....41
2.5.6 Class Diagram....43
2.5.7 Use Case Diagram....45
2.5.8 Activity Diagrams....47
3. System Design....49
3.1 Architecture Overview....49
3.2 Modularization Details....51
Module descriptions....51
Source code organisation....54
3.3 Data Integrity and Constraints....55
3.4 Database Design....56
3.5 Procedural Design (Process Logic)....64
Object-Oriented Design....67
3.6 User Interface Design....67
3.7 Test Case Design....73
4. Coding....74
4.1 Complete Project Coding....74
4.1.1 Project Structure....74

4.1.2 Running the Project....76
4.1.3 Running and Deployment....83
4.1.4 Backend: Application and Core....89
4.1.5 Backend: Models....109
4.1.6 Backend: Schemas....123
4.1.7 Backend: Services....135
4.1.8 Backend: API Routes....190
4.1.9 Backend: Background Jobs....237
4.1.10 Frontend: Application Shell....242
4.1.11 Frontend: Layouts and Shared Components....254
4.1.12 Frontend: Project Components....280
4.1.13 Frontend: Pages....302
5. Standardization of the Coding....363
5.1 Naming and Layout Conventions....363
5.2 Code Efficiency....363
5.3 Error Handling....364
5.4 Parameters Calling and Passing....365
5.5 Validation Checks....366
6. Testing....367
6.1 Testing Techniques and Testing Strategies Used....367
6.1.1 Testing Techniques....367
6.1.2 Testing Strategies....368
6.2 Testing Plan Used....369
6.3 Test Reports....369
6.3.1 Unit Test Cases....369
6.3.2 System Test Cases....373
6.4 Debugging and Code Improvement....375
7. System Security Measures....377
7.1 Database and Data Security....377
7.2 Creation of User Profiles and Access Rights....379
8. Reports....381
8.1 Project Report (CSV Download)....381
8.2 Project Progress Document and Report Document....382
8.3 Task Progress Document....383
8.4 Gantt Chart....383
8.5 Dashboard Statistics....384
8.6 Notification E-mail....384
9. Output....385
10. Future Scope and Further Enhancement of the Project....420
10.1 Limitations of the Present System....420
10.2 Future Enhancements....420
11. Bibliography....422
Appendices....424
Appendix A — API Endpoint Reference....424
Appendix B — Installation and Running....425
Appendix C — Scheduled Jobs....427
Glossary....428

1. INTRODUCTION AND OBJECTIVES

1.1 Title of the Project

TRACE — Task and Resource Attendance and Comprehensive Evaluation System

An AI-driven project and task management platform with GitHub integration.

1.2 Introduction

Every software project begins as a pile of requirement documents — a client brief, a feature list, meeting notes, a scope statement — and ends, if it succeeds, as a set of finished pieces of work that someone can point to. Between those two ends lies the part that usually goes wrong: turning the documents into a plan, splitting the plan into tasks, deciding who does what, keeping a schedule honest while work slips, and telling management how far along things really are. In most organisations this is done by hand, with spreadsheets, a chat group and a Gantt chart that stops being true the week after it is drawn.

TRACE is a web-based platform that automates that middle part. A Manager creates a project and uploads its raw requirement documents (PDF, Word, Markdown or plain text). With one action — **Evaluate Project** — an AI pipeline reads the documents, writes a structured project document, splits the work into task groups (modules) with duration estimates and dependencies, and draws the first version of a Gantt chart. From there the project is handed to people: the Manager appoints a Project Manager (with AI-ranked suggestions), the Project Manager creates tasks (each of which automatically becomes an issue in the project's own GitHub repository), assigns them to Members (again with AI-ranked suggestions), and the Members do the work and mark their tasks complete.

The system then keeps itself up to date. Every completion, every missed deadline and every change to the project document re-plans the Gantt chart as a new, auditable version. Every week the AI reads each task's progress document and rewrites the project's progress report for the Manager. Notifications reach the right people in real time over a WebSocket and by e-mail, and each task has its own discussion thread between the person doing it and the person who assigned it.

TRACE also keeps a complete **workforce record set** — employees, departments, designations, leaves, holidays, attendance and work shifts — owned and maintained by an Administrator, because planning and assignment need to know who exists and

who is available. The Administrator additionally has a raw control panel over every table, a live service monitor and a configuration screen for GitHub and mail settings.

The platform is built with a **React** single-page frontend, a **FastAPI** (Python) backend on **PostgreSQL**, **Celery** workers on **Redis** for background and AI jobs, **MinIO** object storage for uploaded files, **GitHub** for issues and generated documents, and an **OpenAI-compatible** large language model for the AI steps.

1.3 Evolution of the Project since the Synopsis

The approved synopsis described TRACE as an AI-powered *attendance* and productivity system centred on face recognition, GPS geo-fencing, IP tracking and a weighted productivity score. During analysis and the first build iterations it became clear that the part of the problem worth automating was not attendance but **project delivery**: attendance data is easy to record and hard to act on, while the path from requirement documents to a tracked, scheduled set of tasks is where teams lose the most time. The scope was therefore re-specified as follows, keeping the name and the underlying technology stack of the synopsis:

Table 1: Synopsis scope versus delivered scope

Area	In the synopsis	As delivered
Core feature	Attendance with face recognition and geo-fencing	AI-driven project evaluation, task planning and Gantt scheduling
Attendance	Employee clock-in / clock-out with GPS validation	Retained as Admin-maintained records (bulk upload); no self-service capture
Tasks	Task CRUD with in-browser timer	Project → Task Group → Task hierarchy; every task is a GitHub issue
AI	Weighted productivity score, anomaly flags	LLM pipeline: project document, task groups, Gantt, people ranking, task plans, weekly progress reports
Productivity score	Attendance + tasks + active time	Re-based on task signals only (throughput and on-time rate); used as an input to AI ranking
Reporting	Weekly e-mailed PDF report	AI-written progress and report documents; CSV report download for a date range
Real time	WebSocket live attendance	WebSocket notifications and task discussions
Roles	Admin, Manager, Employee	Same three platform roles, plus per-project roles (Project Manager, Member)

The technology choices of the synopsis — React, FastAPI, PostgreSQL, SQLAlchemy with Alembic, JWT, Celery with Redis, WebSockets, SMTP mail, Git/GitHub and Docker Compose — were all kept.

1.4 Objectives of the Project

The objectives of TRACE are:

- **Automate project planning from raw documents.** Convert uploaded requirement files into a structured project document, a set of task groups with realistic durations and dependencies, and a first Gantt chart — without manual data entry.
- **Keep the schedule honest automatically.** Regenerate the Gantt chart as a new version whenever a task is completed, becomes overdue or the project document changes, recording the reason for every change.
- **Make every task a real work item where the code lives.** Create one GitHub repository per project and one GitHub issue per task, and keep the task and the issue in step (assignment, completion, cancellation, deletion).
- **Assist people decisions without taking them.** Rank candidate Project Managers and task assignees with AI, using live performance signals, but always leave the decision to a human.
- **Put completion in the hands of the person doing the work.** Let the assignee mark their own task complete, let the Project Manager set any status, and enforce this rule on the server.
- **Report progress without meetings.** Maintain per-task progress documents and a weekly AI-written project progress report, and allow a CSV report of the project tree to be downloaded for any date range.
- **Deliver the right information to the right person in real time**, through in-app notifications, e-mail and per-task discussion threads.
- **Separate the three audiences cleanly** — Administrator, Manager, Employee — each with its own panel, with authorisation enforced at two layers on the server.
- **Retain a complete, Admin-owned workforce record set** with CRUD and bulk upload, as the basis for planning and availability.
- **Be secure, testable and maintainable**: hashed passwords, JWT authentication, encrypted secrets, schema-validated AI output, database migrations as the only schema owner, and an automated test suite over every route.

1.5 Project Category

TRACE belongs to the following categories:

- **Web Technology / Cloud Computing (SaaS)** — a browser-based single-page application backed by a REST API and WebSockets, deployable to a cloud server and published through a secure tunnel.
- **Artificial Intelligence** — it uses a large language model (LLM) for requirement analysis, planning, ranking and report writing, with strict schema validation of every AI response.
- **RDBMS** — all business data lives in a normalised PostgreSQL database with primary keys, foreign keys, unique and not-null constraints, managed through versioned migrations.
- **Object-Oriented Programming** — the backend is built from Python classes (ORM models, service classes, access-control objects, Pydantic schemas) and the frontend from reusable React components.

1.6 Tools, Platform and Environment Used

Table 2: Software stack

Component	Technology	Purpose
Frontend	React 19 (JSX) with Vite	Single-page user interface for the three panels
UI library	shadcn/ui, Tailwind CSS v4, lucide-react icons	Consistent components, light and dark themes
Graph / chart	@xyflow/react (tree graph), hand-built SVG Gantt	Project tree and Gantt visualisation
Backend framework	FastAPI (Python 3)	REST API, validation, dependency injection
ASGI server	Uvicorn	Serves the FastAPI application
Database	PostgreSQL	Relational data storage
ORM and migrations	SQLAlchemy 2.0 (async) + Alembic	Models, queries and versioned schema changes
Validation	Pydantic V2	Request, response and AI-output schemas
Authentication	JWT (PyJWT) + bcrypt	Stateless tokens and password hashing
Secret encryption	Fernet (cryptography)	Encrypts stored tokens and passwords at rest
Background jobs	Celery 5 (worker + beat) on Redis	AI jobs, scheduled sweeps, e-mail
Real time	WebSocket + Redis publish/subscribe	Live notification and discussion events
Object storage	MinIO (S3 compatible)	Uploaded project source files

Component	Technology	Purpose
Document storage	GitHub repository files	Generated Markdown documents with history
Issue tracking	GitHub REST API	One issue per task
AI	OpenAI SDK (any OpenAI-compatible model)	Evaluation, planning, ranking, reports
Text extraction	pypdf, python-docx	Reading uploaded PDF and Word files
Mail	SMTP (Mailpit in development)	Notification e-mails
Containers	Docker Compose	PostgreSQL, Redis, MinIO, Mailpit, pgAdmin
Testing	pytest, pytest-asyncio	Automated backend tests
Version control	Git + GitHub	Source control
Editor / tools	VS Code, pgAdmin, browser developer tools	Development and inspection

1.7 Hardware Requirements

Table 3: Hardware requirements

Requirement	Specification
Developer machine	Intel Core i5 (8th generation) or equivalent, 8 GB RAM (16 GB recommended), 20 GB free disk space
Server (deployment)	2 vCPU, 4 GB RAM, 40 GB SSD (for example a cloud virtual machine)
Client machine	Any computer able to run a current web browser; 4 GB RAM
Display	1366 × 768 or higher (the interface is designed for desktop screens)
Network	Broadband Internet connection (needed for GitHub and the AI model)

1.8 Software Requirements

Table 4: Software requirements

Requirement	Specification
Operating system (server)	Linux (Ubuntu 22.04 or later) or any OS that runs Docker
Operating system (client)	Platform independent — Windows, Linux or macOS
Runtime	Python 3.11 or later, Node.js 20 or later
Containers	Docker Engine with Docker Compose
Browser	Google Chrome, Mozilla Firefox, Microsoft Edge (current versions)

Requirement	Specification
External accounts	A GitHub Organization with a fine-grained access token; an OpenAI-compatible API key

2. SYSTEM ANALYSIS

System analysis is the study of the problem the software must solve, the environment it will work in and the requirements it must meet, before any design decision is taken. This chapter identifies the need for TRACE, lays out the project plan and schedule, specifies the requirements in the IEEE 830 format, names the software engineering paradigm followed, and presents the analysis models of the system.

2.1 Identification of Need

2.1.1 Problem Definition

A study of how small and medium software teams plan and track their projects showed the following recurring problems:

- **Requirements are read, not processed.** Requirement documents arrive as PDFs, Word files and notes. Someone has to read all of them and re-type the scope into a plan. The effort is large, and the result depends on who did it.
- **Plans go stale immediately.** A Gantt chart drawn at kick-off is not updated when tasks finish early, slip or are added. Within weeks it describes a project that no longer exists, and nobody can say *why* a date moved.
- **Task trackers and code live apart.** Tasks are kept in a spreadsheet or a separate tool while the code and its issues live on GitHub. The two drift apart: a task closed in one place stays open in the other.
- **Assignment is guesswork.** Choosing a Project Manager or an assignee depends on memory and impressions rather than on the person's actual delivery record.
- **Completion is unclear.** It is often not recorded who declared a task done, and when, and the rules for who may do so are enforced only socially.
- **Progress reporting is manual.** Managers receive progress by asking in meetings or reading long chat histories; weekly status reports take hours to compile.
- **Communication is scattered.** Discussion about one task is buried in a project-wide chat, and people learn about assignments or deadlines late.
- **Workforce data is disconnected from planning.** Employee records, leaves, holidays and shifts are kept in HR spreadsheets that the planning process never sees.

2.1.2 The Existing System

In the organisations studied, project delivery was managed with a combination of general-purpose tools:

- requirement documents in shared folders;
- a spreadsheet or a generic board tool for the task list;
- a manually drawn Gantt chart in a spreadsheet or presentation;
- GitHub (or a similar service) for the code and, sometimes, issues;
- a group chat for all communication;
- HR records in separate spreadsheets.

Limitations of the existing system

- No single source of truth: the same task exists in two or three places.
- High manual effort to create and maintain the plan; no automation at all.
- No history of schedule changes and no reason recorded for them.
- No enforcement of who may change a task's status.
- No objective basis for choosing people.
- Reports must be written by hand and are out of date when read.
- Security depends on each tool separately; there is no role-based view of the project.

2.1.3 The Proposed System

TRACE replaces the collection of tools with one platform built around the project:

- The Manager uploads the raw requirement documents; **the AI turns them into a project document, task groups and a Gantt chart**.
- Every task **is** a GitHub issue in a repository TRACE creates for the project, so the task list and the code's issue list cannot drift apart.
- The Gantt chart is **regenerated as a new version** whenever something changes, with the reason recorded.
- **AI ranks candidates** for Project Manager and for each task from live delivery data; a human decides.
- **Status authority is enforced by the server**: the assignee sets their own task's status, including *Completed*; the Project Manager may set any status; nobody else can.
- **The AI writes the weekly progress report**, and any user can download a CSV report of the project tree for a date range.
- Notifications and per-task discussions keep communication attached to the work it is about.
- The Administrator maintains the workforce record set in the same system, with bulk upload.

Advantages of the proposed system

- Large reduction of manual planning effort; a plan exists minutes after the documents are uploaded.
- One source of truth for tasks, shared with the code repository.
- An auditable schedule history.
- Clear, enforced accountability for task completion.
- Fair, data-based suggestions for people decisions.
- Up-to-date reporting with no meetings needed.
- Clean separation of Administrator, Manager and Employee concerns.

2.1.4 Feasibility Study

A feasibility study checks whether the proposed system can be built and used with the available resources. TRACE was examined from five angles.

Technical feasibility

All components of TRACE are mature, well-documented open-source technologies: React, FastAPI, PostgreSQL, Redis, Celery, MinIO and Docker. GitHub provides a stable REST API for repositories, issues, collaborators and file commits. Large language models are available through a standard, OpenAI-compatible API that returns JSON on request. The main technical risk — that an AI model returns malformed or invalid output — is controlled by validating every response against a declared schema and retrying with the validation errors. The system is therefore **technically feasible**.

Economic feasibility

All software used is free and open source. GitHub Organizations with public repositories are free. The only running costs are a small cloud server (or an existing machine) and the AI model's usage charges, which are incurred only when an AI action runs (evaluation, suggestions, task plans, weekly reports) and not on every page view. Compared with the hours of manual planning and reporting it saves, the system is **economically feasible**.

Operational feasibility

Users work in a web browser and need no installation. Each role sees only its own panel with a small number of clear actions: a Manager creates a project and presses *Evaluate Project*; a Project Manager creates and assigns tasks; a Member updates the status of their own tasks. Developers keep working on GitHub, where their tasks already appear as issues. The system fits the way teams already work and is **operationally feasible**.

Schedule feasibility

The work was divided into eleven activities with a critical path of 84 days (Section 2.2), within the time available for the MCA project. The phased build allowed each part to be finished and tested before the next depended on it. The project is **feasible within schedule**.

Legal and ethical feasibility

All libraries are used under their open-source licences. Repositories created by TRACE are public by design (an educational choice), and the interface states this clearly where requirement documents are uploaded; uploaded source files themselves stay private in MinIO. The AI only *suggests* people and never assigns them, so accountability for people decisions stays with a human. Attendance data is recorded but deliberately not used to rank people.

2.2 Project Planning and Project Scheduling

2.2.1 Project Plan

Project planning divides the work into activities, estimates each one, and orders them by their dependencies. TRACE was planned as a sequence of build phases, each ending in something that can be demonstrated and tested, grouped into the activities below.

Table 5: Work breakdown of the project

Id	Activity	Work included
A	Requirement analysis and feasibility study	Study of the problem, existing tools, user roles; feasibility study
B	Synopsis and SRS	Project synopsis, Software Requirement Specification (IEEE 830)
C	System design	Database schema, architecture, API surface, UI wireframes, AI pipeline design
D	Foundation (build phases 0–2)	Clean-up, schema and migrations, JWT authentication, three roles, three panels, route guards, landing page
E	Projects, tasks and GitHub (phases 3, 3a)	Projects, source files, text extraction, task groups, tasks as GitHub issues, status authority, repository creation, collaborator invitations, encrypted settings
F	Visualisation (phase 4)	Project tree graph, Gantt chart data schema, Gantt renderer, version history
G	AI pipeline (phase 5)	AI service layer, Evaluate Project, suggestions, task progress documents, Gantt re-planning, weekly progress report, re-based scoring
H	Notifications and discussion (phase 7)	Notification centre, e-mail, WebSocket live events, per-task discussions, due and overdue reminders

Id	Activity	Work included
I	Administration (phase 8)	Mail configuration, Workforce with bulk upload, Control Panel, Monitor, report download
J	Testing and debugging	Unit, integration and system testing; role matrix; migration tests; fixes
K	Documentation and deployment	Project report, README, production configuration, secure tunnel deployment

2.2.2 Activity Estimates and Critical Path (PERT Analysis)

The Programme Evaluation and Review Technique (PERT) models the project as a network of activities. For each activity the **earliest start (ES)**, **earliest finish (EF)**, **latest start (LS)** and **latest finish (LF)** are computed by a forward and a backward pass; the difference LS − ES is the **slack**. Activities with zero slack form the **critical path** — any delay on them delays the whole project.

In the table, *t* is the expected duration and *Pred.* the predecessor activities. The durations are expected times $t = (o + 4m + p) / 6$ computed from optimistic (o), most likely (m) and pessimistic (p) estimates, rounded to whole days.

Table 6: Activity estimates, predecessors and PERT times (days)

Id	Activity	o	m	p	t	Pred.	ES	EF	LS	LF	Slack
A	Requirement analysis and feasibility	5	7	9	7	—	0	7	0	7	0
B	Synopsis and SRS	5	7	9	7	A	7	14	7	14	0
C	System design	7	10	13	10	B	14	24	14	24	0
D	Foundation (phases 0–2)	7	10	13	10	C	24	34	24	34	0
E	Projects, tasks and GitHub (3, 3a)	10	14	18	14	D	34	48	34	48	0
F	Visualisation (phase 4)	5	7	9	7	E	48	55	55	62	7
G	AI pipeline (phase 5)	10	14	18	14	E	48	62	48	62	0
H	Notifications and discussion (phase 7)	5	7	9	7	F, G	62	69	62	69	0
I	Administration (phase 8)	7	10	13	10	D	34	44	59	69	25
J	Testing and debugging	7	10	13	10	H, I	69	79	69	79	0
K	Documentation and deployment	3	5	7	5	J	79	84	79	84	0

Critical path: A → B → C → D → E → G → H → J → K, with a total expected duration of **84 days**. Visualisation (F) has 7 days of slack because it runs in parallel

with the longer AI pipeline (G); Administration (I) has 25 days of slack because it depends only on the foundation and can proceed alongside the core work.

2.2.3 PERT Chart

In the PERT chart each node is an event, numbered in order, and each arrow is an activity labelled with its id and expected duration. Dashed arrows are dummy activities that carry a dependency but take no time. The critical path is drawn in bold.

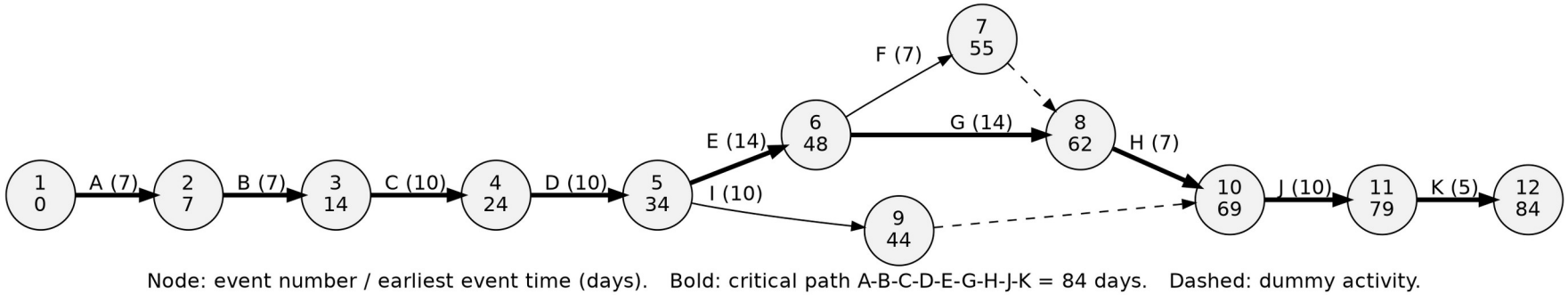


Figure 1: PERT chart of the TRACE project (critical path in bold)

2.2.4 Gantt Chart

The Gantt chart places each activity on a calendar. The project was scheduled to start on **5 July 2026** and, following the critical path, to finish on **26 September 2026**.

Table 7: Project schedule (calendar dates)

Id	Activity	Start	Finish	Duration
A	Requirement analysis and feasibility	05 Jul 2026	11 Jul 2026	7 days
B	Synopsis and SRS	12 Jul 2026	18 Jul 2026	7 days
C	System design	19 Jul 2026	28 Jul 2026	10 days
D	Foundation (phases 0–2)	29 Jul 2026	07 Aug 2026	10 days
E	Projects, tasks and GitHub (3, 3a)	08 Aug 2026	21 Aug 2026	14 days
F	Visualisation (phase 4)	22 Aug 2026	28 Aug 2026	7 days
G	AI pipeline (phase 5)	22 Aug 2026	04 Sep 2026	14 days
H	Notifications and discussion (phase 7)	05 Sep 2026	11 Sep 2026	7 days
I	Administration (phase 8)	08 Aug 2026	17 Aug 2026	10 days
J	Testing and debugging	12 Sep 2026	21 Sep 2026	10 days
K	Documentation and deployment	22 Sep 2026	26 Sep 2026	5 days
	Total (critical path)	**05 Jul 2026**	**26 Sep 2026**	**84 days**

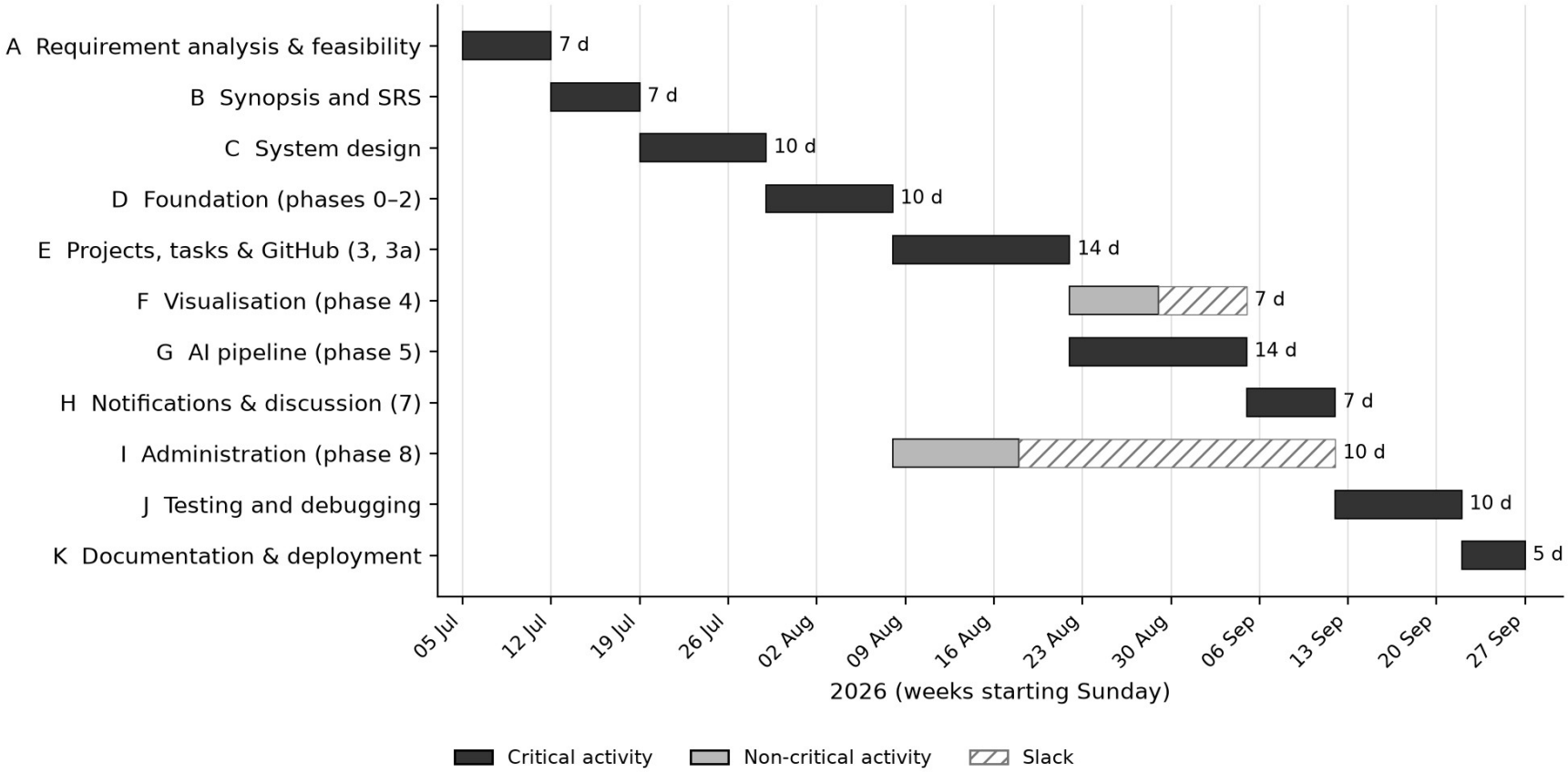


Figure 2: Gantt chart of the TRACE project

2.3 Software Requirement Specification (SRS)

The Software Requirement Specification below follows the structure of **IEEE Standard 830-1998**, *IEEE Recommended Practice for Software Requirements Specifications*.

1. Introduction

1.1 Purpose of the Software Requirement Specification

This document describes the features, behaviour and constraints of TRACE. It serves as a guide for the developers on one hand and as a validation reference for the users and evaluators on the other. It covers what the system must do for each of its three kinds of user — Administrator, Manager and Employee — how the AI pipeline behaves, and the quality attributes the system must have.

1.2 Scope of the Product

TRACE is a web application that takes a software project from its raw requirement documents to a delivered, tracked set of GitHub issues, with automatically maintained documentation, a self-updating Gantt chart and progress reports. In scope:

- projects, their uploaded source documents and their AI-generated living documentation;
- task groups (Gantt modules) and tasks, where each task is a GitHub issue;
- AI automation: project evaluation, task suggestion, assignee and Project Manager suggestion, Gantt generation and re-planning, task progress documents and weekly progress reports;
- manual task completion by the assignee, with server-enforced status authority;
- per-task discussion, notifications, progress and completion reporting;
- an Admin-owned workforce record set (employees, departments, designations, leaves, holidays, attendance, work shifts).

Out of scope: payroll, recruitment, performance appraisal as an HR process, and employee self-service HR workflows such as clock-in/out or leave application.

1.3 Definitions, Acronyms and Abbreviations

Table 8: Definitions and acronyms

Term	Meaning
TRACE	Task and Resource Attendance and Comprehensive Evaluation System
SRS	Software Requirement Specification
AI / LLM	Artificial Intelligence / Large Language Model
API / REST	Application Programming Interface / Representational State Transfer

Term	Meaning
JWT	JSON Web Token — a signed token that proves the user's identity
ORM	Object-Relational Mapping (SQLAlchemy)
Task Group	A module of work within a project; one bar of the Gantt chart
Project Manager (PM)	An Employee appointed to run one project
Member	An Employee assigned at least one task in a project
Gantt version	One stored, immutable snapshot of a project's Gantt chart
Evaluate Project	The AI action that turns source documents into a plan
MinIO	S3-compatible object store holding uploaded files
Celery	Background job queue used for AI and scheduled work
DFD / ERD	Data Flow Diagram / Entity Relationship Diagram

1.4 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- The approved project synopsis of TRACE.
- Official documentation of FastAPI, SQLAlchemy, PostgreSQL, Celery, React and the GitHub REST API (see the Bibliography).

1.5 Overview of the Rest of the SRS

Section 2 gives a general description of the product — its perspective, interfaces, functions, users, constraints and assumptions. Section 3 lists the specific requirements — external interfaces, functional requirements, performance requirements, design constraints, logical database requirements and software system attributes.

2. General Description

2.1 Product Perspective

TRACE is a new, self-contained system. It depends on three external services: **GitHub** (repositories, issues, collaborators and document files), an **OpenAI-compatible AI model** (planning and writing), and an **SMTP mail server** (notification e-mail). Internally it consists of a browser client, an API server, background workers, a relational database, a message broker and an object store.

2.1.1 System Interface. The system is accessed through a web browser on any device with an Internet connection. The browser talks to the API over HTTPS using JSON, and holds one WebSocket connection for live events.

2.1.2 User Interface. The interface is divided into three panels:

- *Landing and Login pages* — a public landing page describing the product, and a login page accepting e-mail and password.
- *Admin panel* (sidebar layout) — Control Panel (CRUD over every table), Workforce (employees, managers, leaves, holidays, attendance, shifts, departments, designations, each with bulk upload), Monitor (service health and recent jobs), Configuration (GitHub and mail settings, each with a connection test).
- *Manager panel* (top bar with a project selector) — a combined dashboard of all the Manager's projects, a create-project wizard, and a project page with tabs Overview, Tree, Gantt, Progress Report and Team.
- *Employee panel* (top bar with a project selector) — a combined dashboard with the Employee's tasks, a project page whose tabs depend on the project role (Project Manager: Overview, Tree, Gantt, Tasks, Project Doc, Team; Member: Overview, Tree, Gantt, My Tasks) and a task page with the progress document and the discussion.
- *Notification bell* on every panel, with the unread count and deep links.

2.1.3 Hardware Interface. Any computer able to run a modern browser; the server needs 2 vCPU, 4 GB RAM and 40 GB disk (Section 1.7).

2.1.4 Software Interface. PostgreSQL (via SQLAlchemy/asyncpg), Redis (Celery broker and publish/subscribe), MinIO (S3 API), GitHub REST API (HTTPS, token authentication), the AI model (OpenAI chat-completions API in JSON mode) and SMTP.

2.1.5 Communication Interface. HTTPS for all client–server traffic, WebSocket (`/api/v1/ws`) for live events, HTTPS to GitHub and the AI provider, SMTP for mail. In production the application is published through a Cloudflare tunnel, so the server opens no inbound port.

2.2 Product Functions

- **Authentication and roles** — login, token refresh, three fixed platform roles and per-project roles.
- **Project management** — create a project (and its GitHub repository), upload source files, appoint and remove Project Managers, edit and delete projects.
- **AI evaluation** — generate the project document, task groups and the first Gantt chart.
- **Task management** — create, edit, assign, re-assign and delete tasks, each mirrored as a GitHub issue; set status under the status-authority rule.

- **AI assistance** — suggest Project Managers, assignees and tasks; write task progress documents; revise the plan when the project document changes; write the weekly progress report.
- **Visualisation** — project tree graph and versioned Gantt chart.
- **Communication** — notifications (in-app, live and e-mail) and per-task discussions.
- **Reporting** — AI progress and report documents; CSV report download for a date range.
- **Administration** — table CRUD, workforce records with bulk upload, service monitoring, GitHub and mail configuration.

2.3 User Characteristics

Table 9: User classes

User	Characteristics
Administrator	Operates the platform. Technically skilled; maintains data, configuration and the workforce records; does not take part in project delivery.
Manager	Brings projects in. Needs no technical skill beyond uploading documents and reading reports; creates projects, appoints Project Managers and watches progress.
Employee as Project Manager	Plans and runs a project: edits the project document, creates and assigns tasks, sets any task's status. Familiar with software delivery.
Employee as Member	Does the work: sees the project, works on their own tasks on GitHub, discusses them and keeps their status current, including marking them complete.

2.4 General Constraints

- The system runs in current versions of Chrome, Firefox and Edge; it is designed for desktop screens.
- A GitHub Organization and a fine-grained token are required before any project can be created (a project's repository is created with it).
- AI features require an API key configured on the server; without it, AI actions are refused with a clear message and the rest of the system keeps working.
- Every AI call must run in the background; no web request may wait for the model.
- Repositories created by TRACE are public; confidential material must not be placed in requirement documents.

2.5 Assumptions and Dependencies

- GitHub, the AI provider and the mail server are reachable from the server.

- Each Employee who works on tasks has a GitHub account and records its username in their profile, and accepts the repository invitation GitHub e-mails them.
- The Celery worker and beat processes are running; otherwise background work waits in the queue.
- The AI model can return JSON output on request (OpenAI-compatible "JSON mode").

3. Specific Requirements

3.1 External Interface Requirements

The system provides a consistent graphical interface built from one component library, in light and dark themes. All data exchange with the server uses JSON over HTTPS; uploads use multipart form data; the report download is a CSV file. The REST API is versioned under `/api/v1` and documented automatically (OpenAPI) at `/docs`.

3.2 Functional Requirements

Table 10: Functional requirements

Id	Requirement	Actor
FR-01	The system shall authenticate users by e-mail and password and issue a JWT access token and a refresh token.	All users
FR-02	The system shall support exactly three platform roles — Admin, Manager, Employee — and route each to its own panel; a user opening another role's panel shall see a 403 page.	System
FR-03	Public self-registration shall only create Employee accounts; Admin and Manager accounts shall be created by an Admin.	System / Admin
FR-04	A Manager shall create a project with a name and description; the system shall create the project's public GitHub repository under the configured Organization in the same operation, and refuse the project if the repository cannot be created.	Manager
FR-05	A Manager shall upload one or more source files (PDF, DOCX, MD, TXT, up to 20 MB each) to a project; the system shall store them in MinIO, extract their text and report rejected files.	Manager
FR-06	A Manager shall press *Evaluate Project*; the system shall, in the background, write the project document to the repository (twice: initial and living copy), create task groups with durations and dependencies, and save Gantt version 1.	Manager / AI
FR-07	Evaluation shall be refused while running, without extracted text, or once the project has tasks; a failed evaluation shall be shown with the step that failed and shall keep no partial data.	System
FR-08	A Manager shall appoint and remove Project Managers from the active Employees, optionally from an AI-ranked list with score and reasoning; the appointee shall be invited to the repository.	Manager / AI

Id	Requirement	Actor
FR-09	A Project Manager shall create, edit and delete task groups; a group holding tasks shall not be deleted.	Project Manager
FR-10	A Project Manager shall create a task in a group; the system shall create the matching GitHub issue in the same request and roll the task back if the issue cannot be created.	Project Manager
FR-11	A Project Manager shall assign or re-assign a task to an Employee; the system shall make them a Member, invite them to the repository, assign the GitHub issue and notify them.	Project Manager
FR-12	The system shall suggest tasks for a task group and rank assignees for a draft task using AI; suggestions shall only fill the form, never create or assign.	Project Manager / AI
FR-13	After a task is created, the system shall write its step-by-step plan and checklist (task progress document) to the repository in the background.	AI
FR-14	The assignee shall set their own task's status to Pending, In Progress, In Review, Blocked or Completed; the Project Manager shall set any status except Overdue; everyone else shall be refused.	Member / Project Manager
FR-15	Completing or cancelling a task shall close its GitHub issue; re-opening shall re-open it; deleting shall close it as "not planned".	System
FR-16	Each task group's completion percentage and status shall be recomputed from its tasks on every change.	System
FR-17	The system shall regenerate the Gantt chart as a new version when a task is completed or re-opened, when tasks become overdue, when the project document changes, and on the Project Manager's request; bursts of triggers shall produce one version.	System / AI
FR-18	A Project Manager shall edit the living project document; a changed document shall trigger a re-plan in which the AI may revise durations and add groups.	Project Manager / AI
FR-19	The system shall show a project tree graph (Project → Task Groups → Tasks → assignees) and a Gantt chart with version history to everyone who can see the project.	All project users
FR-20	Every Monday the system shall rewrite each active project's progress document and report document with AI; the owner or a Project Manager may request it at any time.	AI / Manager / PM
FR-21	A nightly job shall mark open past-due tasks Overdue and notify; a daily job shall remind about tasks due tomorrow.	System
FR-22	The system shall notify the assignee on assignment; the Project Managers on completion (and the assignee if someone else completed it); the owner when evaluation finishes or fails — in-app, live and by e-mail.	System
FR-23	Each task shall have a discussion thread visible to its assignee, its assigner, the project's Project Managers and the owning Manager, with one level of replies.	Project users
FR-24	Managers and Employees shall download a CSV report of the project → task group → task tree for a date range.	Manager / Employee

Id	Requirement	Actor
FR-25	The Admin shall perform CRUD on every table except the secret settings table through the Control Panel.	Admin
FR-26	The Admin shall maintain employees, managers, leaves, holidays, attendance, shifts, departments and designations, each with CSV bulk upload that adds good rows and reports rejected ones.	Admin
FR-27	The Admin shall configure the GitHub Organization, token and committer, and the mail server, and test each connection; secrets shall be write-only.	Admin
FR-28	The Admin shall monitor the health of the API, database, Redis, Celery worker and beat, MinIO, mail, WebSocket and AI, and see recent job outcomes.	Admin
FR-29	Deleting a project shall be refused while it has open tasks and shall archive its repository first.	Manager
FR-30	Every project and task endpoint shall check the caller's project role in addition to the platform role.	System

3.3 Performance Requirements

- Standard API requests shall respond within 2 seconds; AI actions return a job id immediately and never block a request.
- Dashboard pages shall load within 4 seconds on a broadband connection.
- A burst of status changes shall produce a single Gantt re-plan (30-second debounce).
- Live notifications shall reach an open browser within a few seconds of the event.
- The system shall support at least 200 concurrent users on the recommended server.

3.4 Design Constraints

- The frontend uses React with the shadcn/ui component library, Tailwind design tokens and lucide-react icons only.
- The backend uses FastAPI with asynchronous SQLAlchemy; the database schema is created and changed only by Alembic migrations.
- Every AI call runs in a Celery job and every AI response is validated against a declared JSON schema.
- All GitHub calls go through one service module; all credentials are resolved through one settings function.
- One configuration file (`.env`) holds all environment settings.

3.5 Logical Database Requirements

The data is stored in a relational PostgreSQL database. Every table has a surrogate `BIGINT` primary key, `created_at` / `updated_at` timestamps and a `deleted_at` column for soft deletion. Foreign keys enforce every relationship; unique constraints protect e-mails, usernames, role names, (project, user) assignments and (category, key) settings. Generated documents are stored as GitHub file URLs, never as text in the database. The database design is given in Chapter 3.

3.6 Software System Attributes

3.6.1 Reliability. A task and its GitHub issue are created together or not at all; a failed AI job leaves a visible error and no partial data; every Gantt version is validated before it is saved. The system produces correct output for valid input and meaningful error messages for invalid input.

3.6.2 Availability. The system is designed to be available around the clock. Background work is queued in Redis, so a restarted worker continues where it stopped. Health checks for every service are visible on the Admin Monitor page.

3.6.3 Security.

- Passwords are hashed with bcrypt (12 rounds); plain passwords are never stored.
- Every request is authenticated with a signed JWT; every route declares a role guard, and every project route also checks the project role.
- Stored secrets (GitHub token, SMTP password) are encrypted with Fernet and are never returned by any API.
- All input is validated server-side with Pydantic schemas; SQL is issued only through the ORM with bound parameters.
- Production refuses to start with default passwords, weak JWT secrets or a missing encryption key.

3.6.4 Maintainability. The code follows fixed conventions (one service per concern, one way to call the AI, one way to call GitHub). Every change ships with automated tests, including an audit that fails if any route lacks an authorisation guard and a test that builds the whole schema from migrations. The design, decisions and status of every feature are documented alongside the code.

3.6.5 Portability. The client runs in any modern browser on any operating system. The supporting services run in Docker containers, and the application runs on any system with Python and Node.js.

2.4 Software Engineering Paradigm Applied

TRACE was developed using the **Iterative and Incremental model**, organised as a sequence of numbered build phases, each of which delivers a working, tested increment.

Why this paradigm. A pure Waterfall model was unsuitable because the requirements changed materially during development: after the first increments the core of the product moved from attendance to AI-driven project delivery, and later decisions (for example making task completion manual instead of CI/CD-driven, or fixing the AI configuration in the environment file) changed parts of the design. A prototyping model alone would not have produced a maintainable system. The iterative and incremental model allowed the requirements to be revised between increments while every increment was still specified, built, tested and documented properly.

How it was applied.

1. *Analysis and design* produced the specification (roles, panels, schema, AI pipeline) and a dependency graph of build phases.
2. Each *increment* (phase) had written steps and a "done when" condition — for example, *"a Manager can create a project and assign a Project Manager, and that Project Manager can create tasks that appear as GitHub issues, with no AI involved at all."*
3. Each increment was *tested* with automated tests before the next began; the test suite grew with every phase and was always run in full.
4. After each increment the *documentation was updated in the same change*, and any decision that was not obvious was recorded with its reason.
5. When requirements changed, the *specification was changed first*, then the code.

Table 11: Build increments

Phase	Increment	Delivered
0	Clear the ground	Removal of retired features (clock-in, geo-fencing, dynamic roles, group chat)
1	Schema	New project tables, backfill migrations, three roles
2	Panels	Three panel layouts, route guards, landing page, 403 page
3	Projects without AI	Projects, source files, task groups, tasks as GitHub issues, status authority

Phase	Increment	Delivered
3a	GitHub owner model	Repository creation, collaborator invitations, encrypted settings
4	Visualisation	Tree graph and versioned Gantt chart
5	AI pipeline	Evaluation, suggestions, task documents, re-planning, weekly reports
7	Notifications and discussion	Notification centre, e-mail, live events, discussions
8	Administration	Mail configuration, Workforce, Monitor, report download
9	Polish	Pagination, search, empty and error states, revision sweep

(Phase 6, a CI/CD-driven automatic completion feature, was removed from the design before it was built; the phase numbers were kept so that existing references stayed valid.)

2.5 Data Models and Analysis Diagrams

2.5.1 Data Flow Diagrams (DFD)

A Data Flow Diagram shows how data moves between external entities, processes and data stores. The context diagram (Level 0) shows TRACE as a single process; Level 1 breaks it into its main processes; Level 2 refines the two most important processes — project evaluation and task management.

Notation: rectangles are external entities, circles (rounded boxes) are processes, open-ended boxes are data stores, and arrows are data flows.

Level 0 DFD (Context Diagram)

The external entities are the three users (Admin, Manager, Employee) and the three external systems (GitHub, the AI model and the mail server).

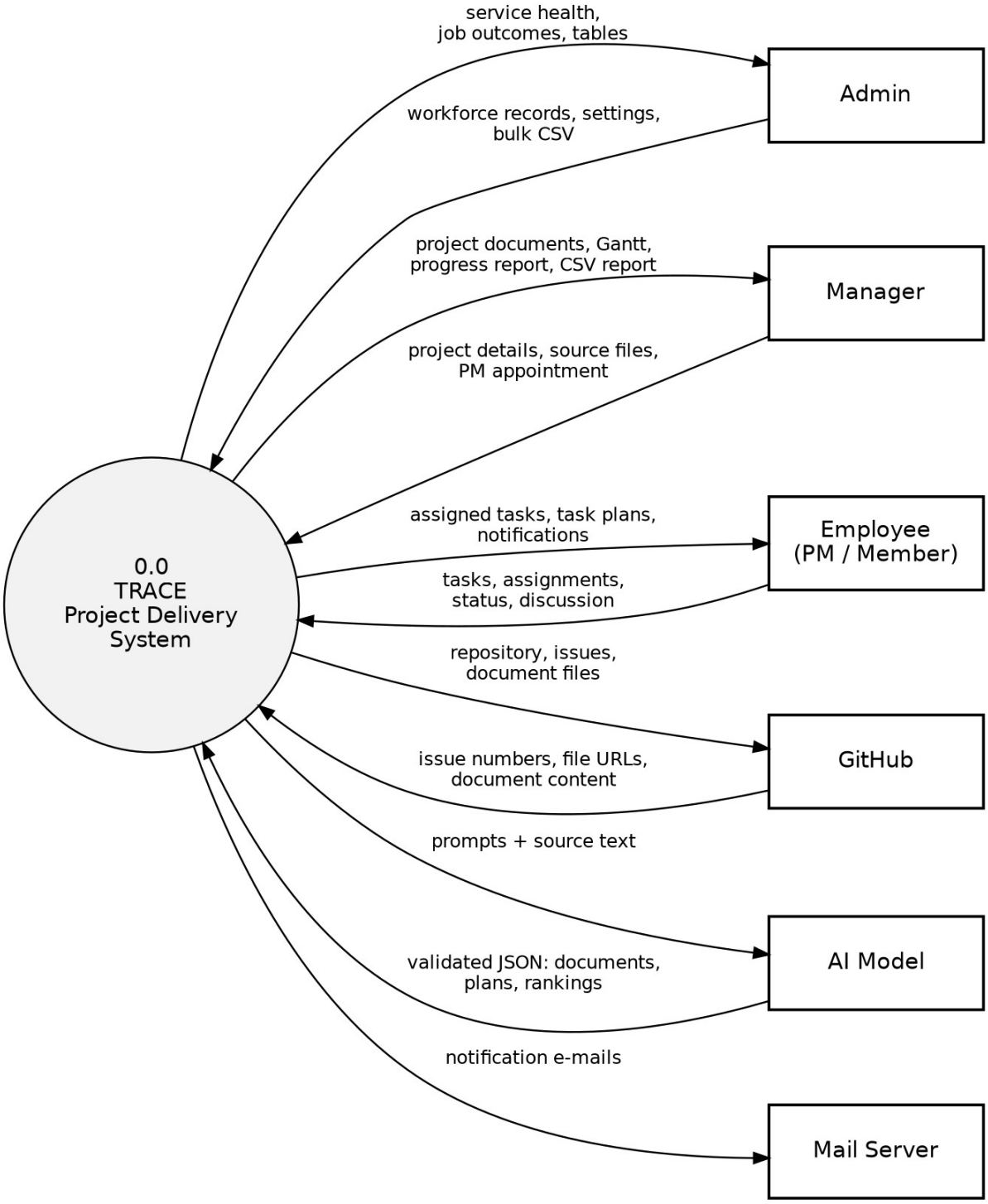


Figure 3: Level 0 DFD — context diagram

Level 1 DFD

Table 12: Level 1 DFD processes

No.	Process	Description
1.0	Authenticate user	Checks credentials, issues tokens, resolves the platform role
2.0	Manage projects	Creates projects and repositories, stores source files, manages the Project Manager roster

No.	Process	Description
3.0	Evaluate project	Runs the AI pipeline: project document, task groups, first Gantt version
4.0	Manage tasks	Task groups, tasks, GitHub issues, assignment, status, progress documents
5.0	Plan and report	Gantt re-planning, weekly progress and report documents, CSV report
6.0	Notify and discuss	Notifications, e-mail, live events, task discussions
7.0	Administer	Workforce records, table CRUD, configuration, monitoring

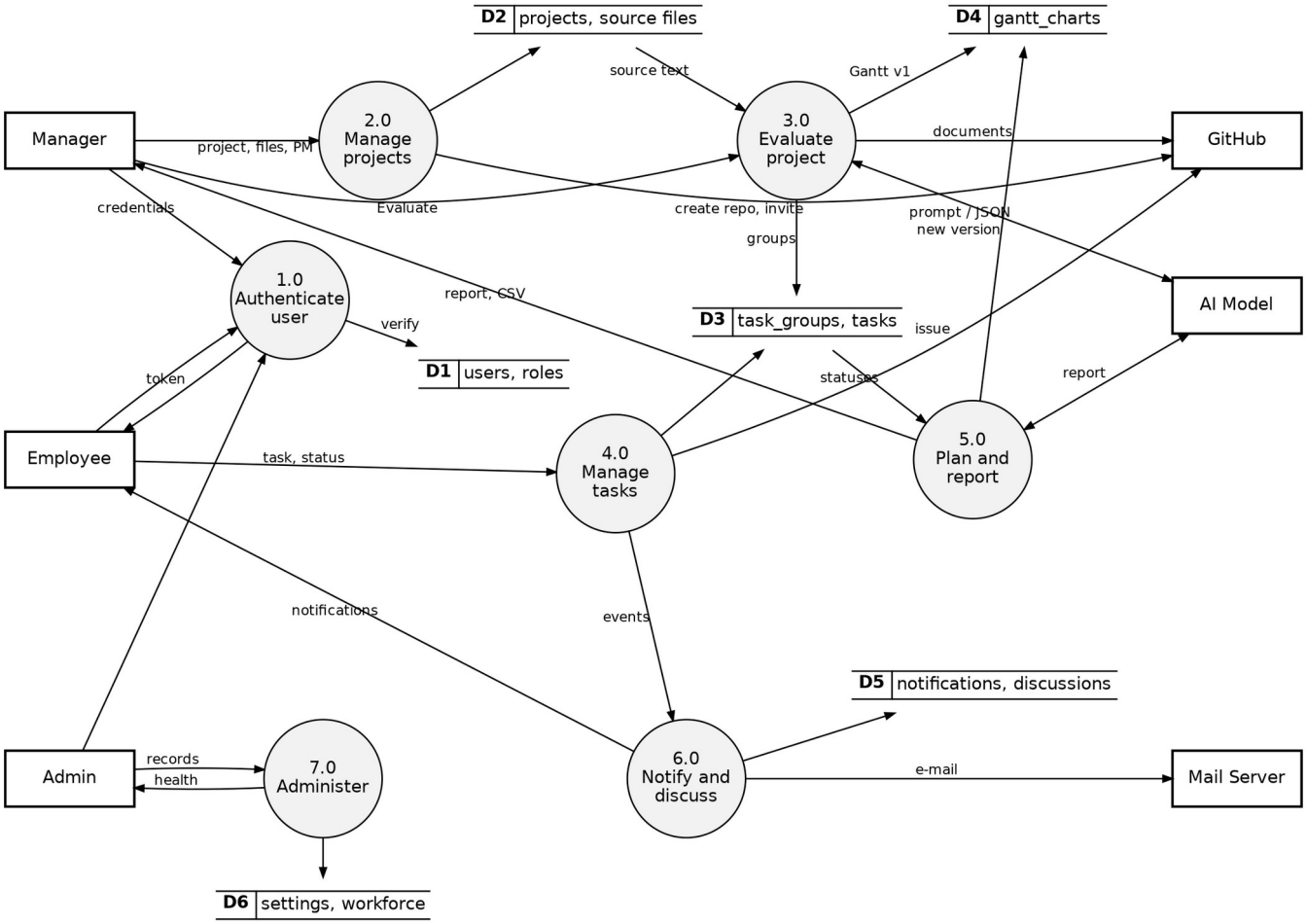


Figure 4: Level 1 DFD

Level 2 DFD — Process 3.0 Evaluate Project

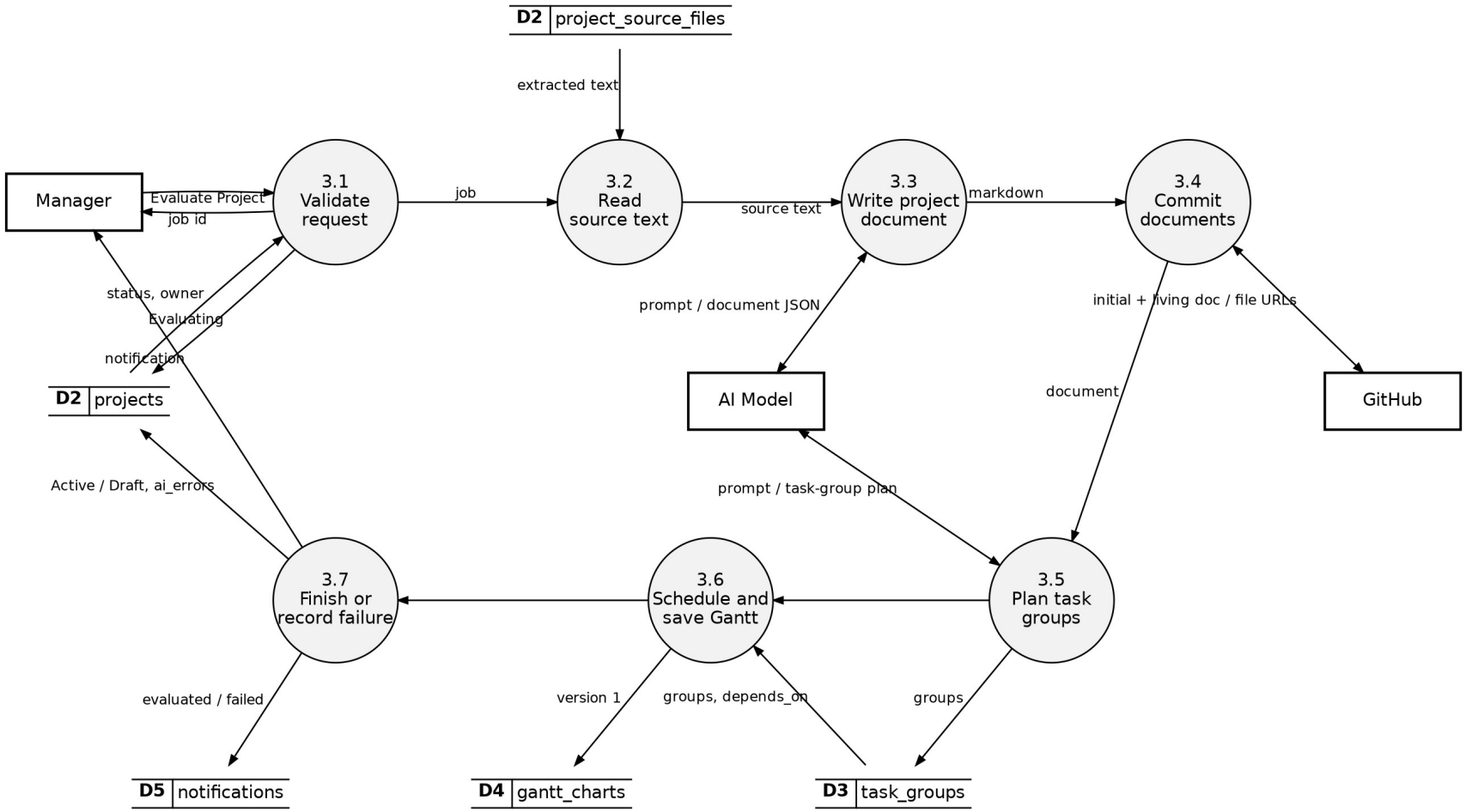


Figure 5: Level 2 DFD — Evaluate Project (process 3.0)

Level 2 DFD — Process 4.0 Manage Tasks

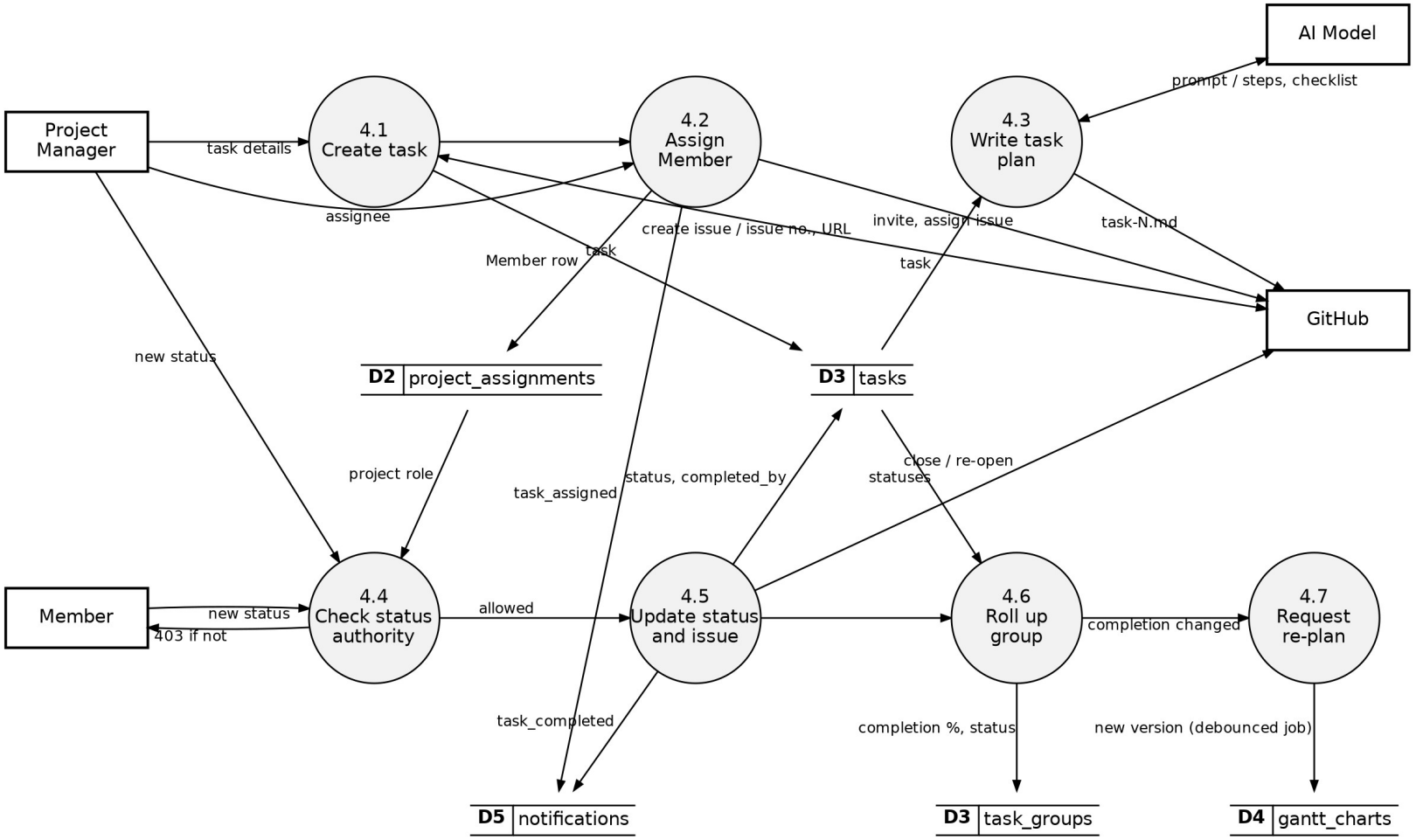


Figure 6: Level 2 DFD — Manage Tasks (process 4.0)

2.5.2 Control Flow Diagram

The control flow diagram shows how control passes through the layers of the system for every request: the browser sends a request with its token; the API authenticates it, checks the platform role on the route and the project role in the service; the service validates the input, performs the change and, where needed, calls GitHub before committing; after the commit, background jobs are queued and live events published. AI work never runs inside the request — the route returns a job id, and the browser polls for the result.

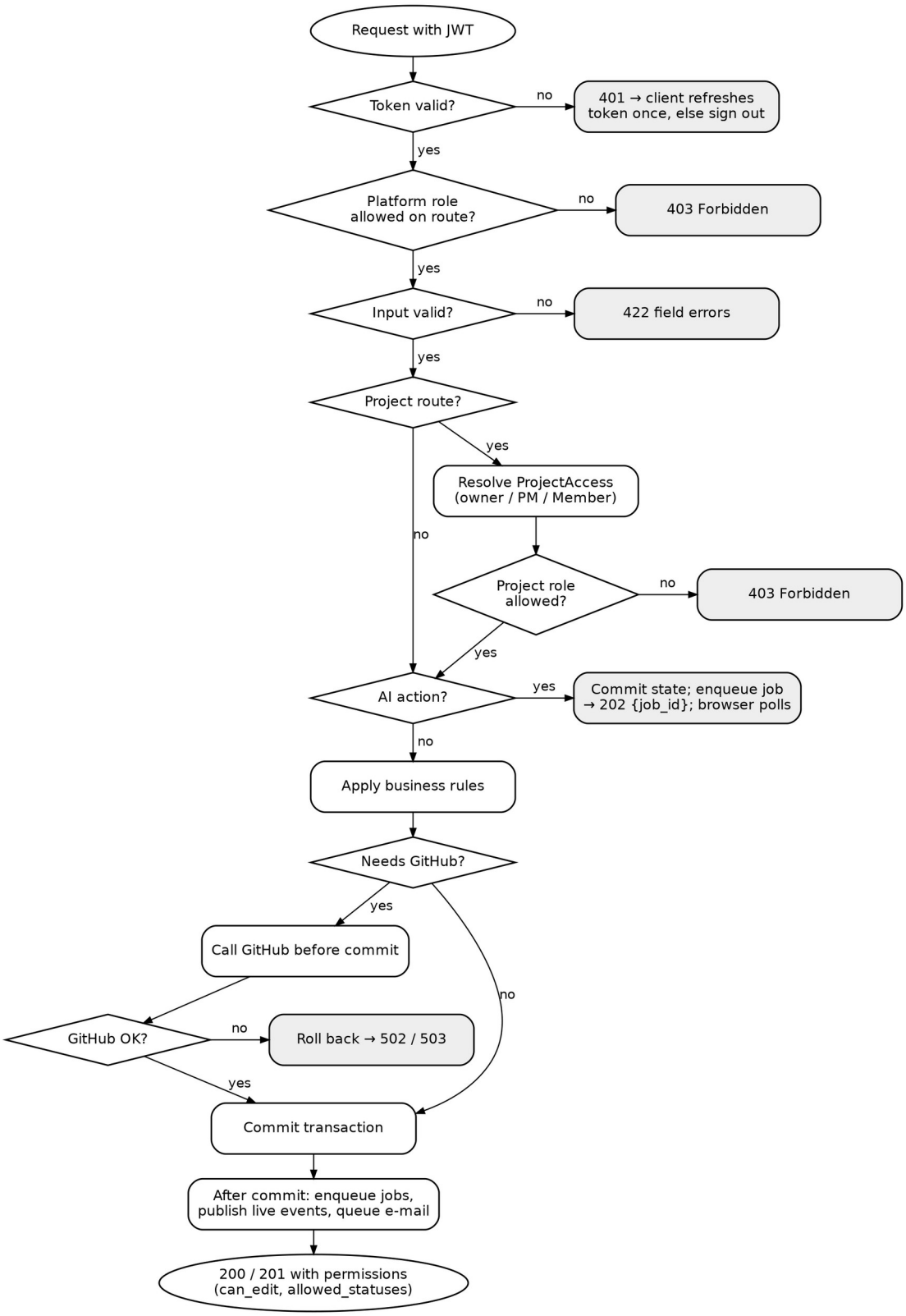


Figure 7: Control flow of a request through TRACE

2.5.3 State Diagrams

State diagrams show the states an object can be in and the events that move it between them.

Task

A task starts *Pending*. The assignee moves it through *In Progress*, *In Review* and *Blocked* and marks it *Completed*; the Project Manager can set any of these and can also *Cancel* it. The nightly sweep moves open past-due tasks to *Overdue*. Completing or cancelling closes the GitHub issue; any move back to an open state re-opens it.

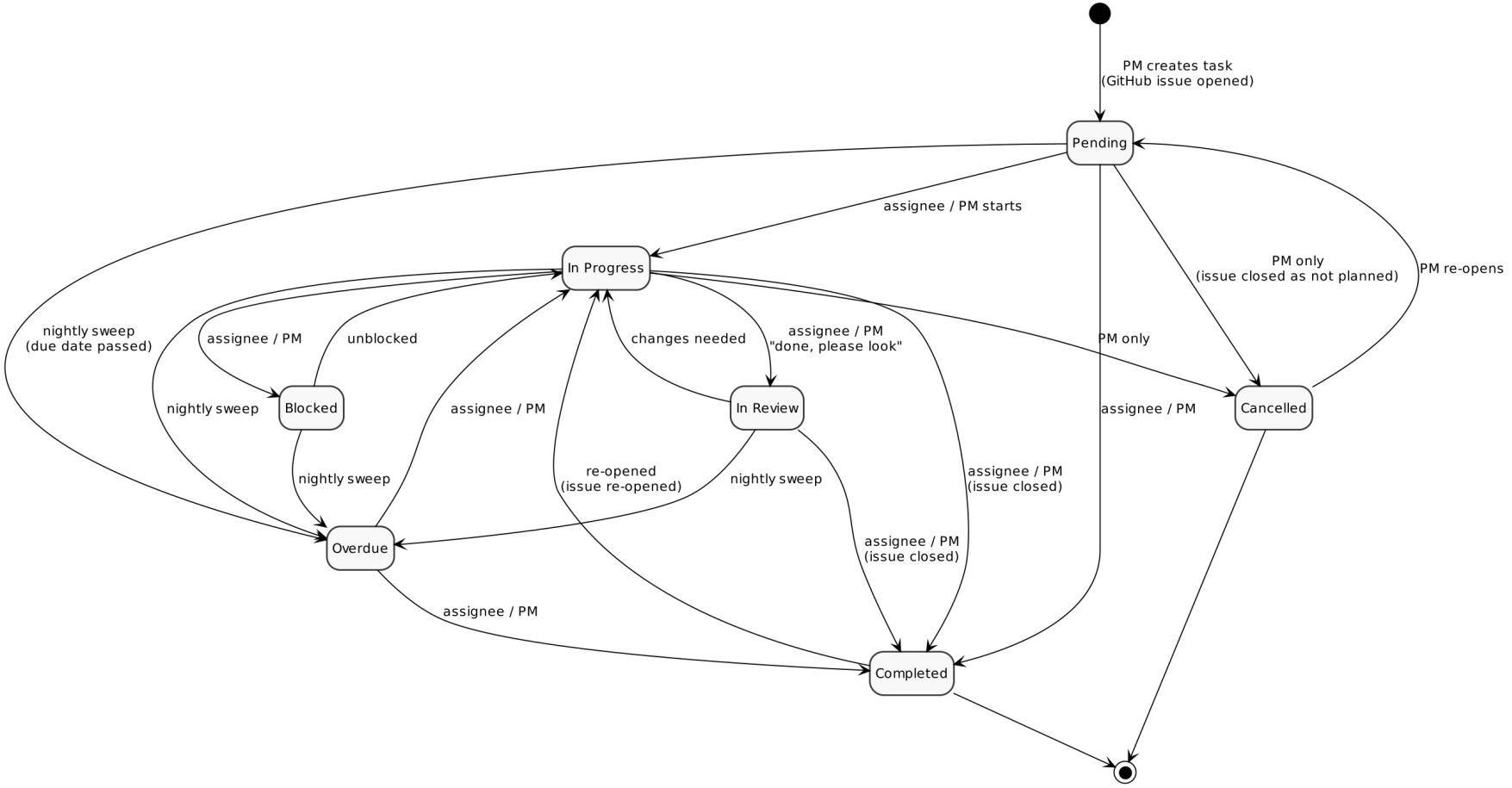


Figure 8: State diagram — Task

Project

A project is created as *Draft*. Pressing *Evaluate Project* moves it to *Evaluating*; success makes it *Active*; failure returns it to *Draft* (or *Active* if it had been evaluated before). An active project can be marked *Completed* or *Archived* by its owner.

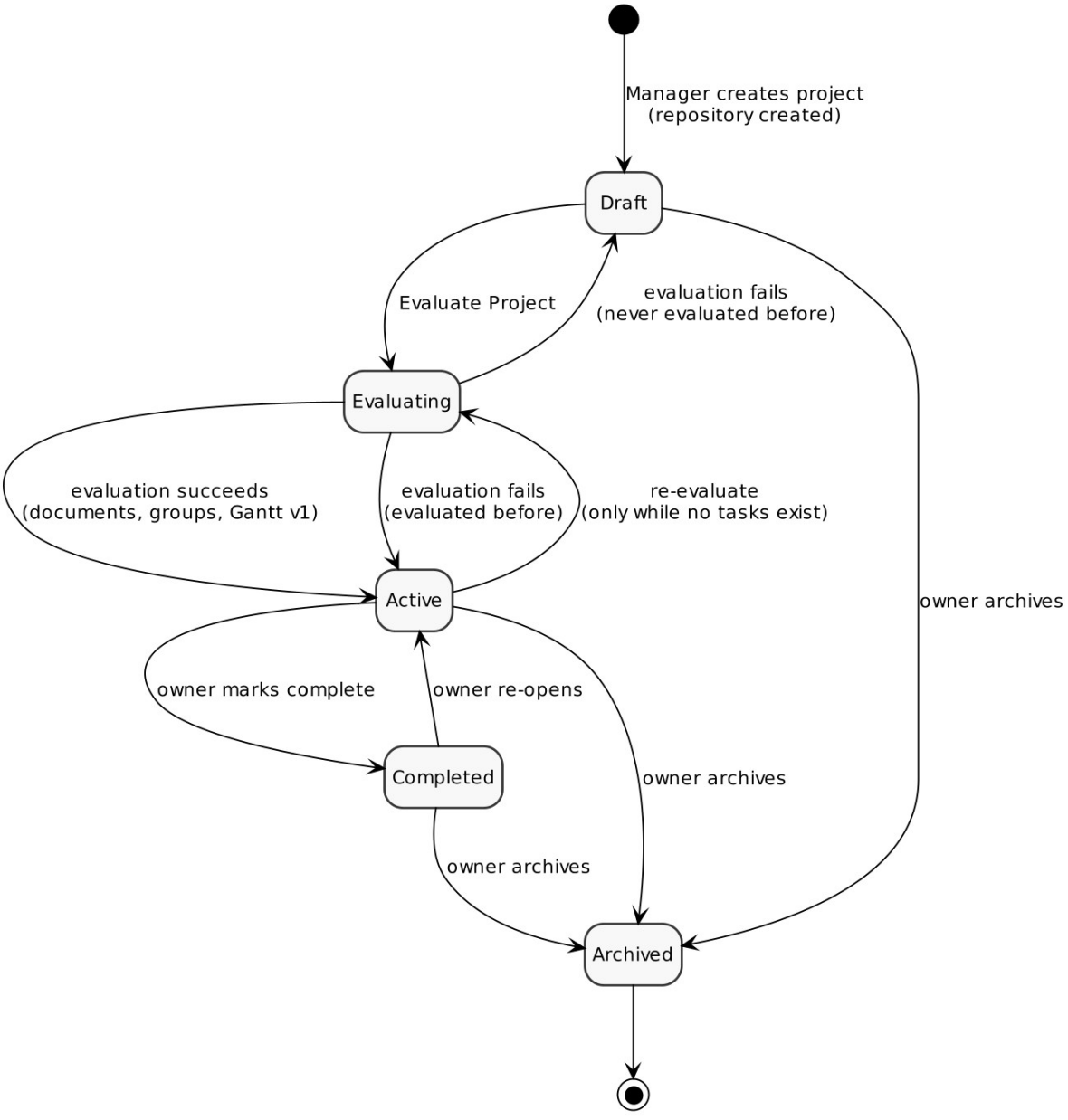


Figure 9: State diagram — Project

Task Group

A task group's status is rolled up from its (non-cancelled) tasks after every change. It is *Pending* while no task has started; it becomes *In Progress* as soon as any task is In Progress, In Review or Completed (the actual start date is stamped the first time); it becomes *Completed* when every task is completed (the actual end date is stamped)

and returns to *In Progress* if one is re-opened; it shows *Blocked* when every task in it is blocked.

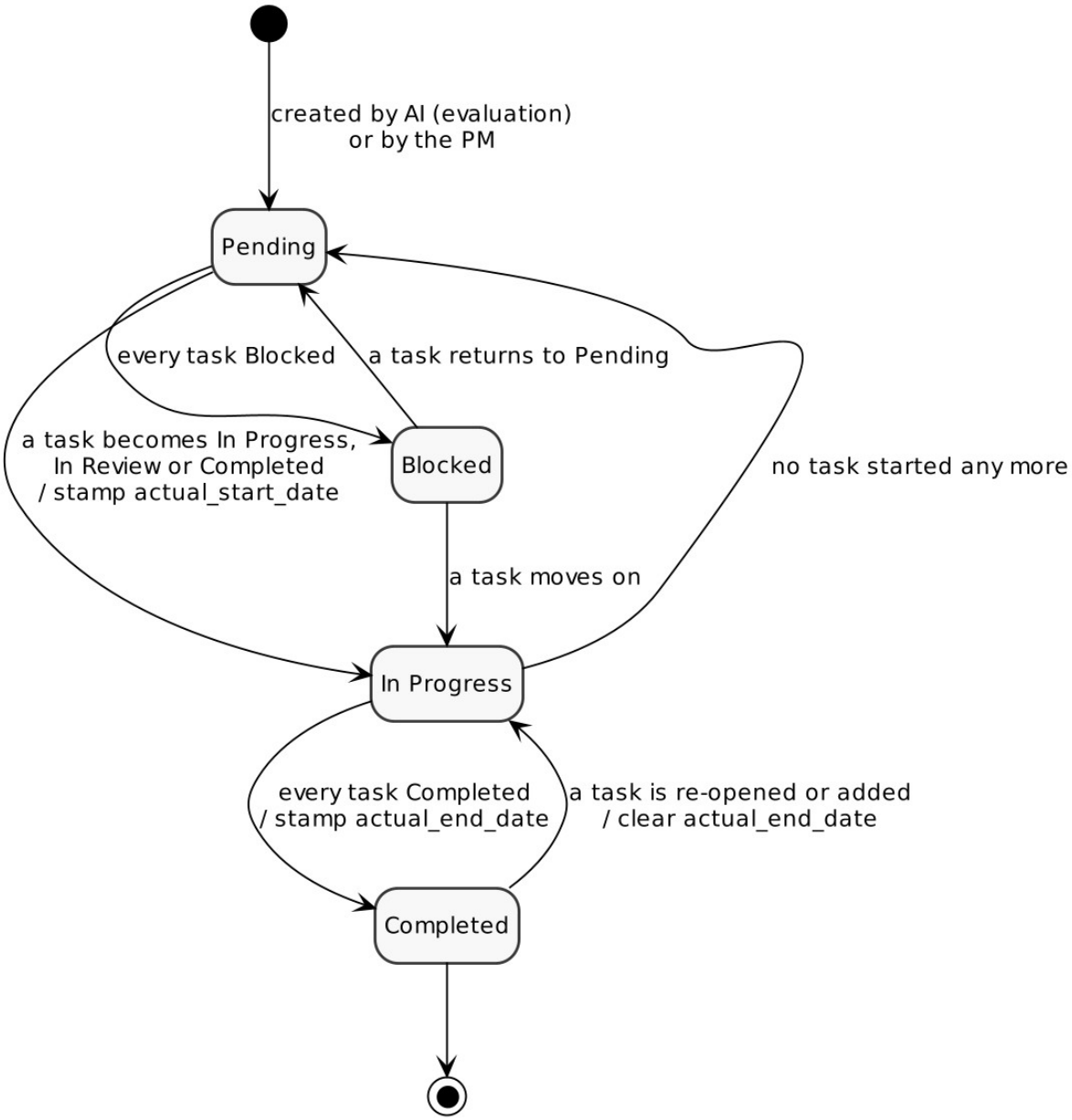


Figure 10: State diagram — Task Group

2.5.4 Sequence Diagrams

Evaluate Project

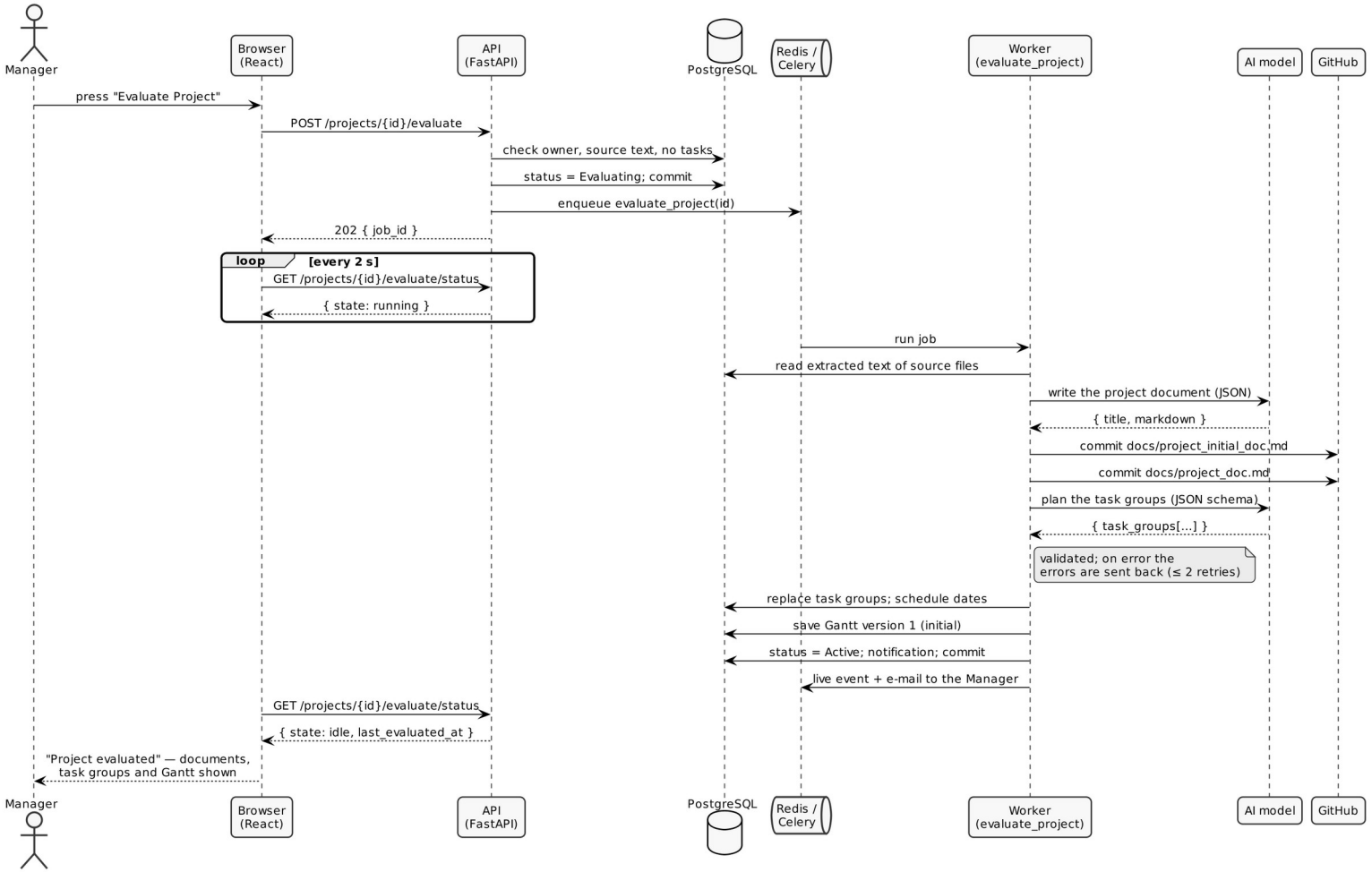


Figure 11: Sequence diagram — Evaluate Project

Create and complete a task

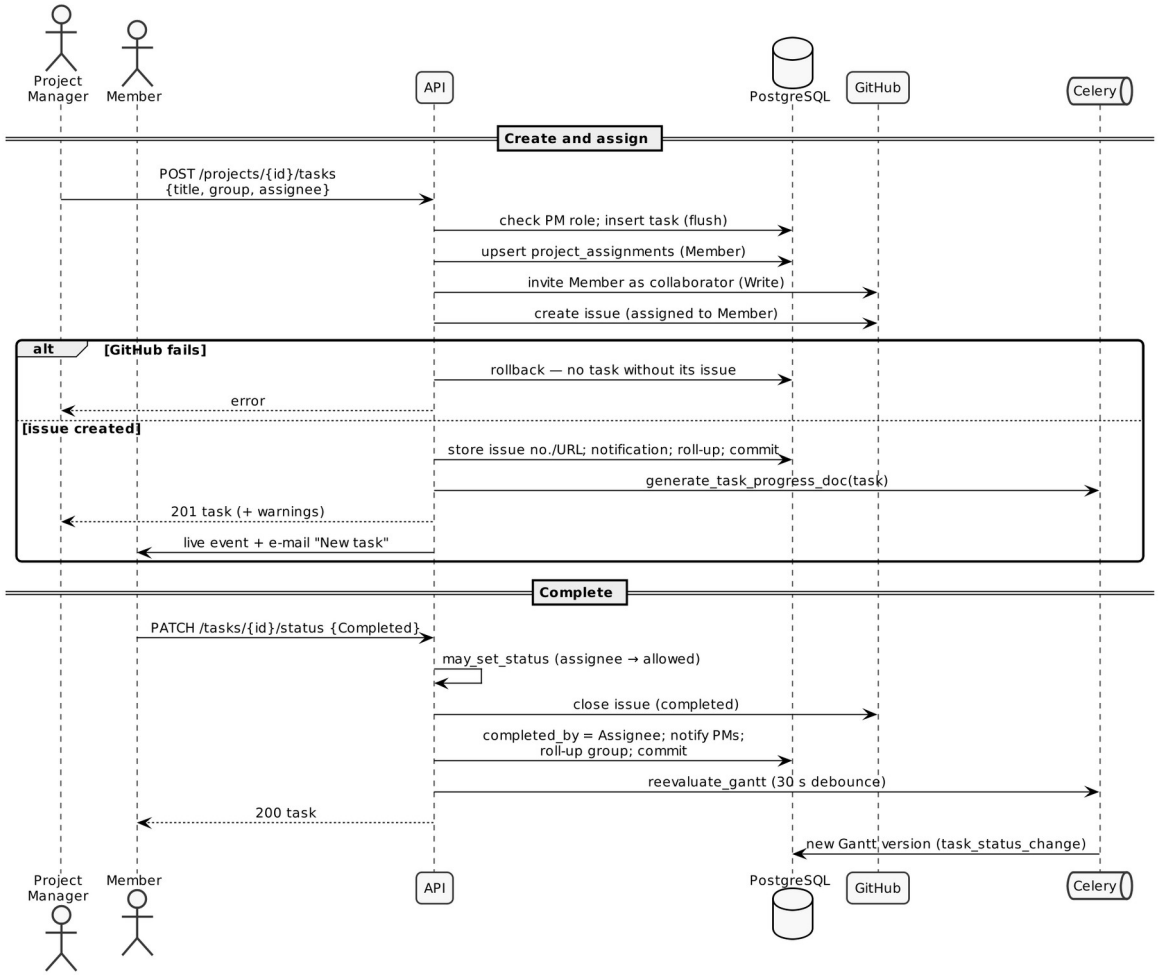


Figure 12: Sequence diagram — creating and completing a task

2.5.5 Entity Relationship Model

The ER model shows the main entities of TRACE and the relationships between them. Attributes are listed in full in the database design (Section 3.4).

Table 13: Relationships between the main entities

Entity	Entity	Cardinality	Relationship
User	Role	M : N (through user_roles)	holds
User	Employee	1 : 1	is recorded as
User (Manager)	Project	1 : N	owns
Project	Project Source File	1 : N	has
Project	User	M : N (through project_assignments)	staffs
Project	Task Group	1 : N	is divided into
Task Group	Task	1 : N	contains
Task Group	Task Group	M : N (depends_on)	waits for
User	Task	1 : N	is assigned
Task	Task Discussion	1 : N	is discussed in
Task Discussion	Task Discussion	1 : N	is replied to by
Project	Gantt Chart	1 : N (versions)	is scheduled by
User	Notification	1 : N	receives
Employee	Productivity Score	1 : N	is scored by

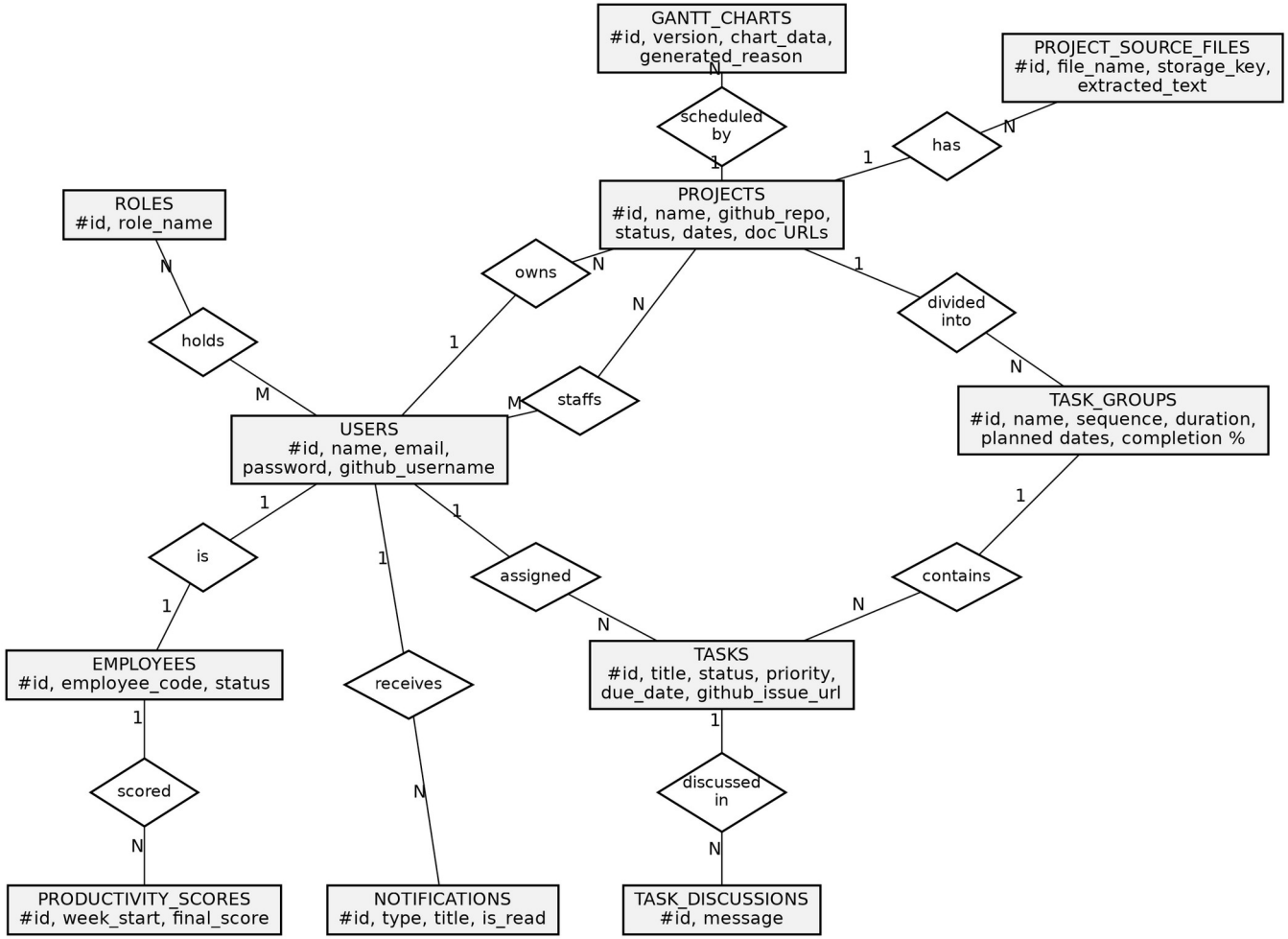


Figure 13: Entity Relationship Diagram

2.5.6 Class Diagram

The class diagram shows the main classes of the backend: the ORM entity classes with their attributes and relationships, and the service and access-control classes that operate on them.

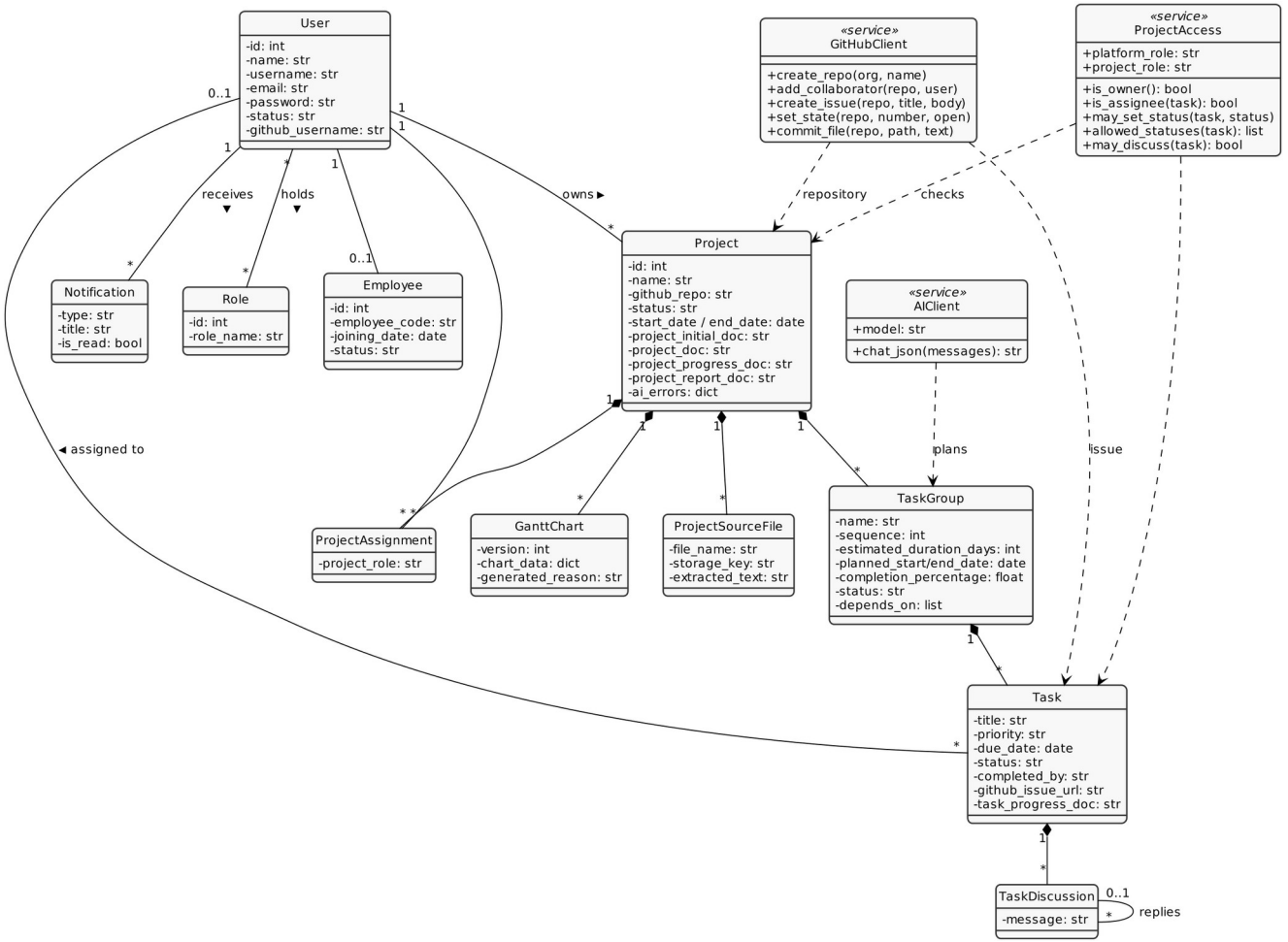


Figure 14: Class diagram

2.5.7 Use Case Diagram

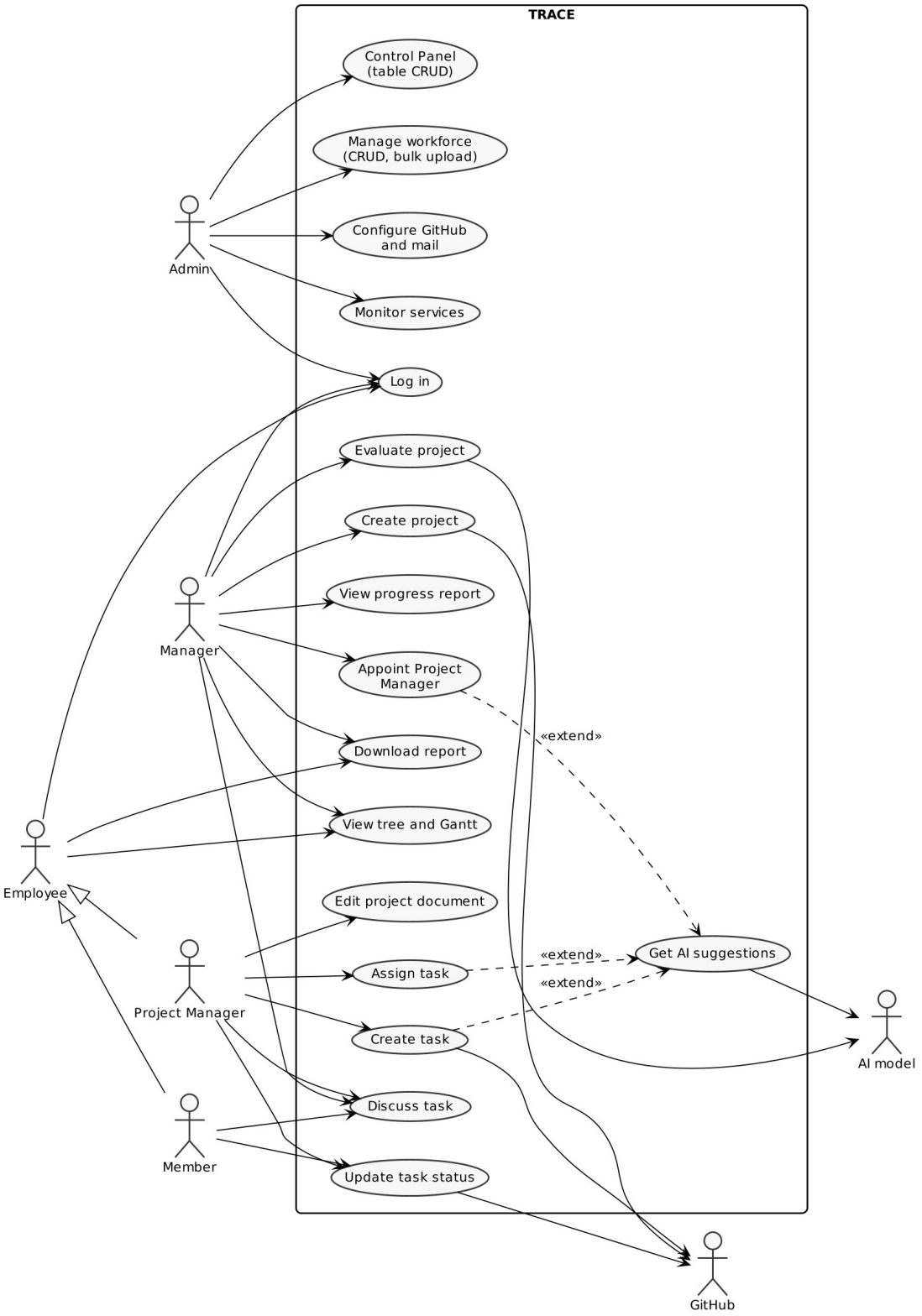


Figure 15: Use case diagram

Table 14: Principal use cases

Id	Use case	Actor	Summary
UC-01	Log in	All	Enter e-mail and password; land on the role's panel
UC-02	Create project	Manager	Name, description, source files; repository created
UC-03	Evaluate project	Manager	AI writes the project document, task groups, Gantt v1
UC-04	Appoint Project Manager	Manager	Pick an Employee, optionally from AI suggestions
UC-05	View progress report	Manager	Read the AI-written progress and report documents
UC-06	Edit project document	Project Manager	Save the living document; the Gantt re-plans
UC-07	Create task	Project Manager	Task and GitHub issue created; AI may suggest content and assignee
UC-08	Assign task	Project Manager	Member invited and notified; issue assigned
UC-09	Update task status	Member / PM	Server enforces who may set what; completion closes the issue
UC-10	Discuss task	Assignee, assigner, PMs, Manager	Post and reply in the task's thread
UC-11	View tree and Gantt	All project users	Graph of the project; Gantt with version history
UC-12	Download report	Manager / Employee	CSV of the project tree for a date range
UC-13	Manage workforce	Admin	CRUD and bulk upload of workforce records
UC-14	Configure GitHub and mail	Admin	Save settings; test the connection
UC-15	Monitor services	Admin	Health of every service and recent jobs

2.5.8 Activity Diagrams

Evaluate Project

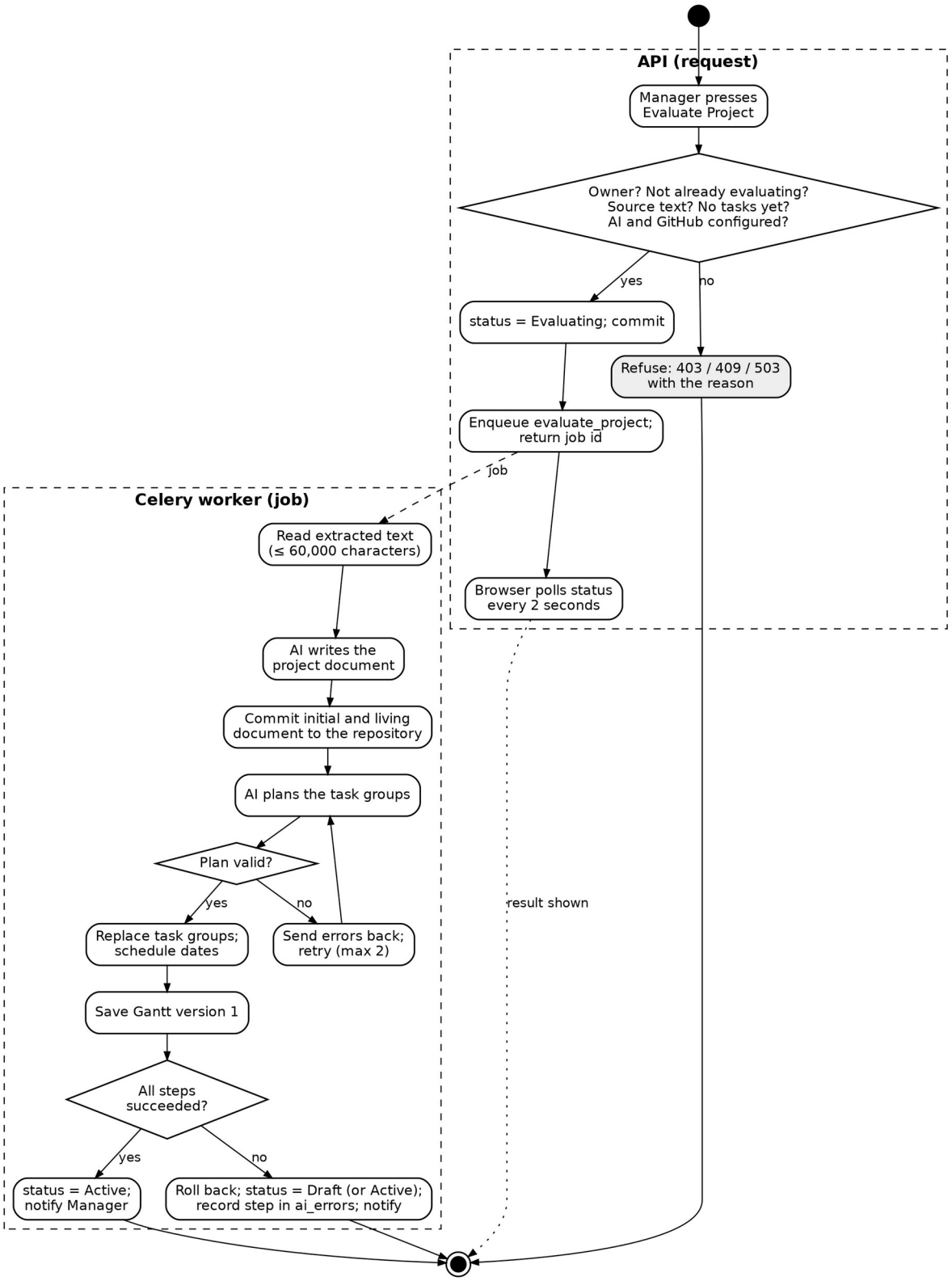


Figure 16: Activity diagram — Evaluate Project

Task life cycle

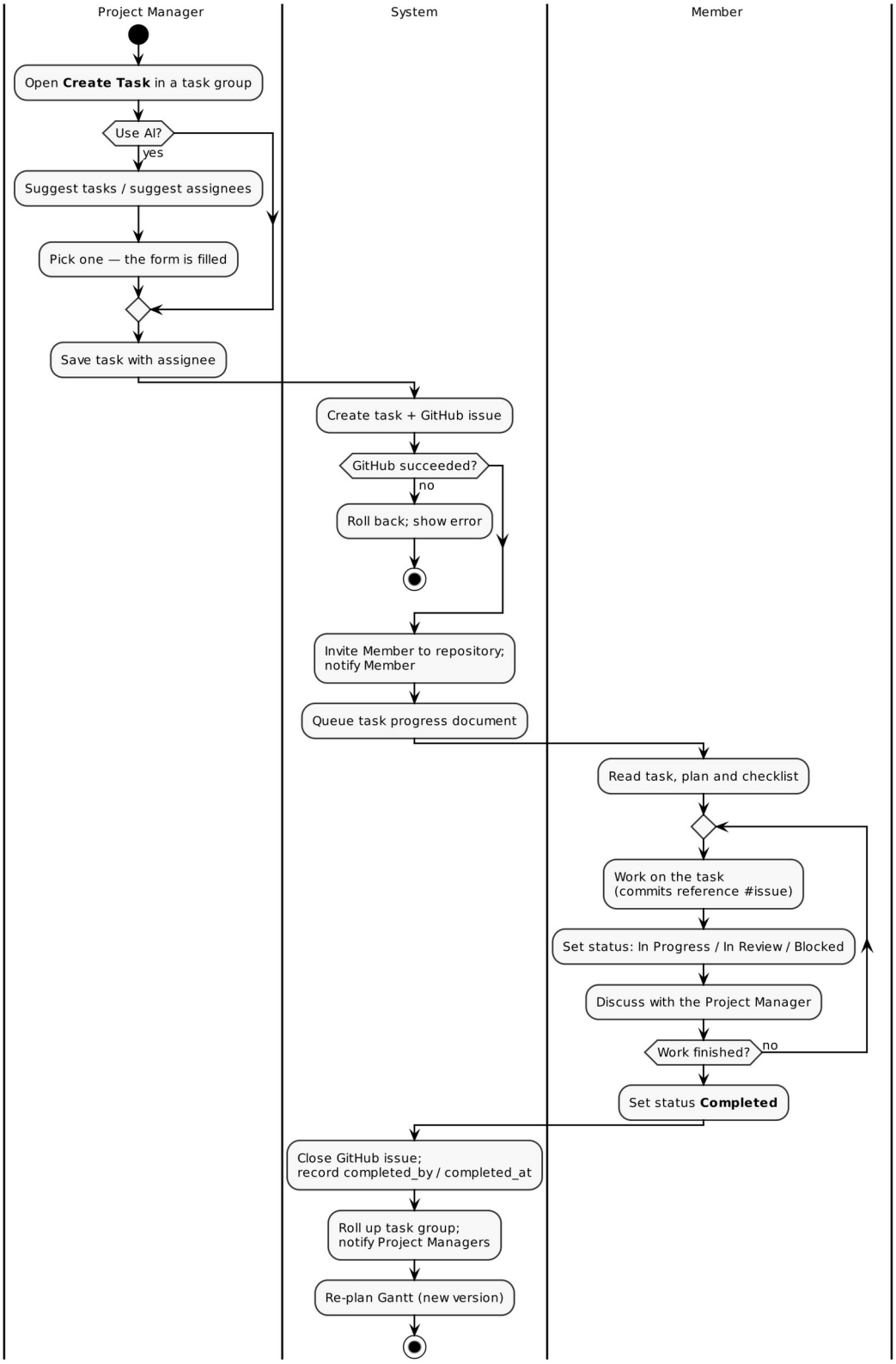


Figure 17: Activity diagram — task life cycle from creation to completion

3. SYSTEM DESIGN

System design turns the requirements of Chapter 2 into a blueprint for the software: how it is split into modules, how data is stored and kept consistent, how each process works step by step, what the user sees, and how the result will be tested.

3.1 Architecture Overview

TRACE follows a **three-tier architecture** with a separate background tier:

Table 15: Tiers of the system

Tier	Components	Role
Presentation	React single-page application (Vite build) in the browser	Three role panels, landing page, live updates over one WebSocket
Application	FastAPI on Uvicorn (ASGI)	REST API under `/api/v1`, authentication, two-layer authorisation, validation, GitHub calls, WebSocket relay
Background	Celery worker and Celery beat on Redis	AI jobs, Gantt re-planning, scheduled sweeps and reports, e-mail
Data	PostgreSQL, MinIO, GitHub repository files	Relational data, uploaded source files, generated Markdown documents
External	GitHub REST API, OpenAI-compatible model, SMTP server	Issues and repositories, AI, e-mail

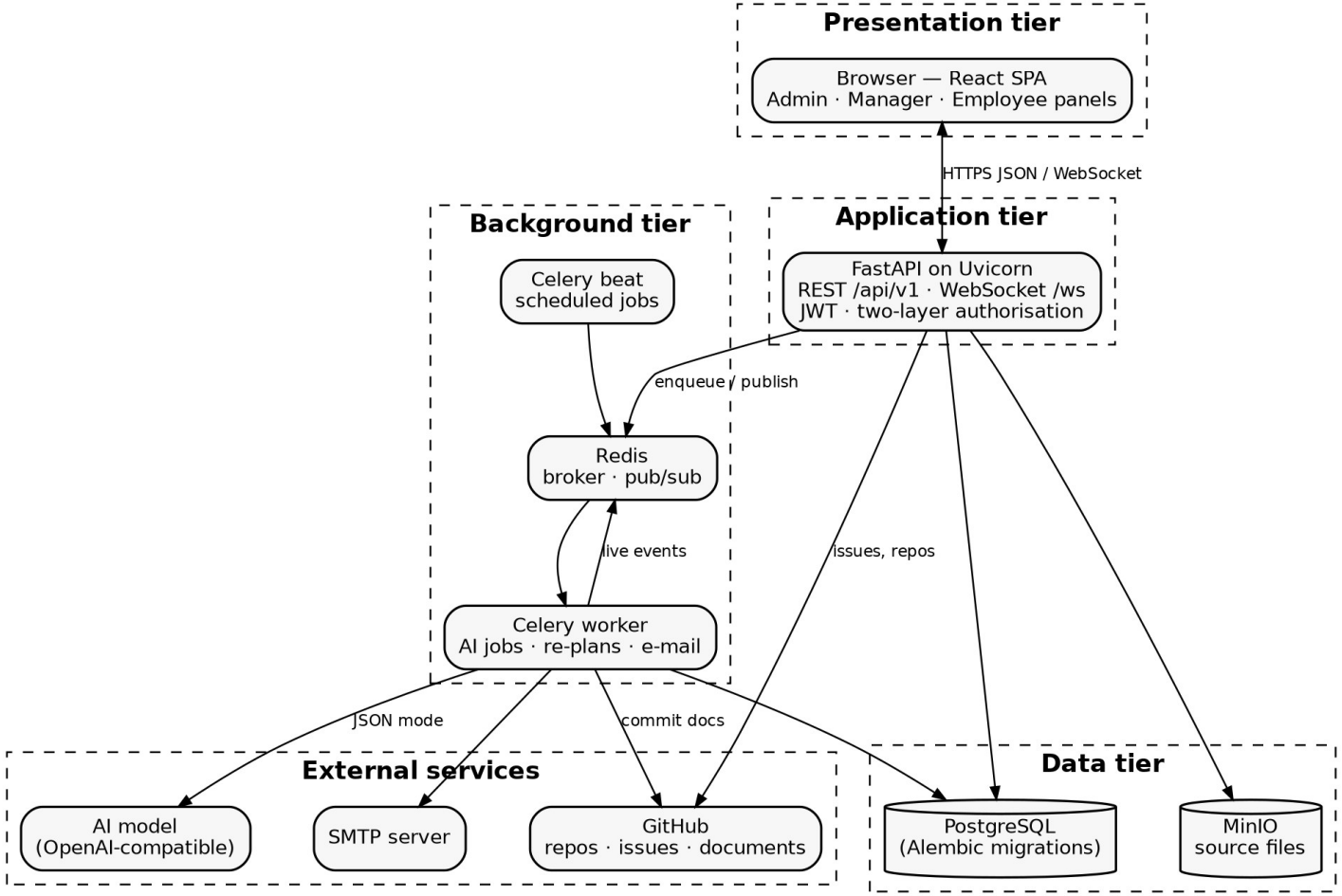


Figure 18: System architecture of TRACE

3.2 Modularization Details

The system is divided into the following modules. Each backend module is a FastAPI router (the API surface) backed by one or more service modules (the rules); the frontend mirrors them with one API client module per router and one page or component per screen.

Table 16: Modules of TRACE

No.	Module	Responsibility
M1	Authentication and Users	Login, token refresh, registration (Employee only for the public), password hashing, profile and GitHub username
M2	Access Control	Platform-role guards on every route; project-role resolution (`ProjectAccess`) for every project and task operation; status authority
M3	Project Management	Projects and their GitHub repositories, source-file upload and text extraction, Project Manager roster, candidates, project deletion with repository archiving
M4	Task Management	Task groups, tasks as GitHub issues, assignment with repository invitation, status changes with issue open/close, task-group roll-up
M5	AI Pipeline	The AI service layer (schema-validated calls with retry), Evaluate Project, people and task suggestions, task progress documents, project-document revision, weekly progress report
M6	Planning and Visualisation	Gantt scheduling rule, chart data builder and validator, version history, debounced re-evaluation, project tree graph
M7	Notifications and Discussion	Notification rows, live events through Redis publish/subscribe and the WebSocket, e-mail, per-task discussion threads
M8	Reporting	AI progress and report documents; CSV report of the project tree for a date range; performance scores used by AI ranking
M9	Administration	Control Panel (generic CRUD), Workforce with bulk upload, Configuration (GitHub, mail) with encrypted secrets, Monitor (service health, recent jobs)
M10	Scheduler	Celery beat: nightly overdue sweep, daily due reminders, weekly progress reports, weekly performance scores

Module descriptions

M1 — Authentication and Users

The user signs in with e-mail and password. The password is compared against its bcrypt hash; on success the API returns a short-lived access token and a longer-lived refresh token, both signed JWTs, together with the user's roles. Every later request carries the access token in the `Authorization` header. Anyone may register publicly, but a public registration always creates an **Employee**; only an Admin can create

Admin or Manager accounts. Users edit their own profile, including their GitHub username, which TRACE needs to invite them to repositories and assign issues to them.

M2 — Access Control

Authorisation happens in **two layers**. The first layer is the **platform role** on the route: every route declares a guard (`require_admin`, `require_roles(...)` or `require_any_role`), and an automated test fails the build if any route lacks one. The second layer is the **project role**: every project and task operation resolves a `ProjectAccess` object, which answers *who is the caller in this project?* — Admin (reads everything, changes nothing), owning Manager, Project Manager, Member or nobody (403). The same object decides status authority and discussion visibility, and the API responses carry `can_edit`, `can_discuss` and `allowed_statuses` derived from it, so the interface never has to guess.

M3 — Project Management

A Manager creates a project; TRACE derives a repository name (slug) from the project name and creates a public repository under the company's GitHub Organization **before** saving the project, so that no project exists without its repository. Source files are uploaded to MinIO and their text is extracted immediately (PDF, DOCX, Markdown, text) so the AI never has to parse them again. The Manager appoints Project Managers from the active Employees; each appointee is invited to the repository with write access. A project can be deleted only when it has no open tasks, and its repository is archived first.

M4 — Task Management

A Project Manager creates task groups and tasks. **Task creation is issue creation**: the task row and the GitHub issue are created in one request, and if GitHub refuses, the task is rolled back. Assigning a task makes the assignee a Member of the project, invites them to the repository and assigns the issue to them (or, while their invitation is still pending, leaves the issue unassigned with a warning). Status changes go through a single endpoint that enforces status authority and keeps the issue in step: completing or cancelling closes it, re-opening re-opens it. After every change the task group's completion percentage and status are recomputed.

M5 — AI Pipeline

Every AI call goes through one function, `complete(ai, Schema, system, user, context)`, which sends the prompt in JSON mode, validates the reply against a Pydantic schema, and on failure hands the validation errors back to the model and asks again (up to two retries). The jobs built on it are: **Evaluate Project** (project

document, task groups with dependencies, Gantt version 1), **suggestions** (Project Managers, assignees, tasks — returned as the job result and never stored), **task progress document** (steps and acceptance checklist, committed to the repository), **project-document revision** (revised durations and new groups when the living document changes) and the **weekly progress report**. Every job that writes something records its failure where users will see it.

M6 — Planning and Visualisation

The Gantt chart is **never edited**; it is regenerated as a new, validated version. A deterministic scheduling rule lays the task groups out by sequence and dependencies, keeps finished groups at their actual end date, and stretches unfinished groups whose end has passed up to today so that a slip pushes everything that depends on it. Triggers (completion, overdue sweep, document change, manual button) call one function that enqueues the re-plan with a 30-second delay; a job that finds a newer version already exists does nothing, so a burst of triggers produces one version. The project tree graph is a live read of Project → Task Groups → Tasks → assignees.

M7 — Notifications and Discussion

Notifications are written through one function that never notifies the person who acted. After the database commit, a live event is published on a Redis channel and every API process relays it to that user's open WebSocket; the event only says "refetch", so the browser reloads the data through the normal authorised REST routes. E-mails are queued as background jobs. Each task has a discussion thread visible to its assignee, its assigner, the project's Project Managers and the owning Manager.

M8 — Reporting

Every Monday the AI reads each task's progress document and the current task statuses of every active project and rewrites the project's **progress document** and **report document** in its repository. Managers and Employees can also download a **CSV report** of the project → task group → task tree for any date range. Weekly performance scores (completed ÷ assigned tasks due that week, and on-time rate) are computed for use as evidence in AI ranking.

M9 — Administration

The Admin has a **Control Panel** with generic create/read/update/delete over every table except the secret settings table; **Workforce**, a curated interface over employees, managers, leaves, holidays, attendance, shifts, departments and designations with CSV bulk upload that reports rejected rows; **Configuration** for the GitHub Organization, token and committer identity and for the mail server, each with a

connection test and write-only secrets; and **Monitor**, showing the health of every service and the outcome of recent background jobs.

M10 — Scheduler

Celery beat runs four scheduled jobs: the overdue sweep every night at 23:00, due-tomorrow reminders every day at 08:00, the progress-report roll-up every Monday at 07:00 and the performance scores every Sunday at 23:30.

Source code organisation

Table 17: Backend package layout

Path	Contents
`app/main.py`	Application factory, middleware, router registration, WebSocket endpoint
`app/config.py`	Settings loaded from the single `.env` file; production pre-flight checks
`app/models/`	SQLAlchemy ORM classes (users, employees, projects, tasks, workforce)
`app/schemas/`	Pydantic request/response schemas, AI output schemas, Gantt chart schema
`app/api/v1/`	Routers: auth, users, employees, projects, tasks, notifications, settings, workforce, admin, services, attendance, hrms
`app/core/`	Security (JWT, bcrypt), platform-role guards, project access
`app/services/`	Business rules: project, GitHub, AI, AI jobs, Gantt, notifications, realtime, storage, extraction, scoring, reports, settings
`app/tasks.py`, `app/celery_app.py`	Celery task wrappers and the beat schedule
`alembic/versions/`	Database migrations — the only thing that creates tables
`tests/`	Automated tests with fake GitHub, fake AI and fake storage

Table 18: Frontend layout

Path	Contents
`src/App.jsx`	Route table with guest-only and role guards
`src/layouts/`	AdminLayout (sidebar), ManagerLayout and EmployeeLayout (top bar with project selector)
`src/pages/`	Landing, login, 403; admin, manager and employee pages
`src/components/project/`	Project overview, evaluation card, task board, team panel, suggestions, progress report, document editor, task discussion
`src/components/`	Gantt chart, project tree graph, notification bell, service monitor, Markdown renderer
`src/contexts/`	Authentication, theme, live-event (WebSocket) contexts

Path	Contents
`src/api/`	One API client module per backend router
`src/lib/`	Shared helpers — error messages, dates, status colours, job polling

3.3 Data Integrity and Constraints

Data integrity means the data stays correct and consistent however the system is used. TRACE enforces it at three levels — the database, the API schemas and the business rules.

Database-level constraints

- **Entity integrity** — every table has a surrogate primary key `id BIGSERIAL`.
- **Referential integrity** — every relationship is a `FOREIGN KEY`; a task cannot point to a project, group or user that does not exist.
- **Uniqueness** — `users.email`, `users.username`, `roles.role_name`, `employees.employee_code`; the pairs (`user_id`, `role_id`) in `user_roles`, (`project_id`, `user_id`) in `project_assignments` and (`category`, `key`) in `system_settings`.
- **NOT NULL** — every mandatory column (names, titles, statuses, foreign keys that must exist, `created_at`).
- **Defaults** — `status` columns default to their starting state (`Draft`, `Pending`, `Active`), `completion_percentage` to 0, `is_read` to false, `created_at` to the current time.
- **Soft delete** — rows are marked with `deleted_at` instead of being removed, so history and references survive; every query filters out deleted rows.

Domain constraints (API level)

Every request body is validated by a Pydantic schema before any code runs:

- statuses are restricted to fixed sets — task: *Pending, In Progress, In Review, Completed, Blocked, Overdue, Cancelled*; editable project status: *Draft, Active, Completed, Archived*; priority: *Low, Medium, High, Critical*;
- lengths and ranges — project and group names 2–255 / 1–255 characters, task titles 2–255, estimated hours 0–999, group durations at least 1 day, discussion messages 1–5000 characters;
- uploads — only PDF, DOCX, MD and TXT files, at most 20 MB each;
- every AI response is validated against its schema, including rules that need data (for example, a suggested user id must be one of the candidates, a task

group may only depend on an *earlier* group, 1–20 groups with unique names and 1–365 days each);

- every Gantt version is validated against the `GanttChartData` schema (dates in range, bars ordered, dependencies between existing bars, completion 0–100).

Business-rule constraints

- **Status authority** — only the assignee (Pending, In Progress, In Review, Blocked, Completed) or the Project Manager (any status except Overdue) may change a task's status; Overdue is set only by the system.
- **All-or-nothing records** — a project is not saved without its repository; a task is not saved without its GitHub issue.
- **Write-once document** — the initial project document is never rewritten after evaluation; only the living project document is edited.
- **Append-only Gantt** — Gantt versions are only ever added, never updated.
- **Safe deletion** — a task group holding tasks cannot be deleted; a project with open tasks cannot be deleted.
- **Evaluation guard** — evaluation is refused while one is running, without extracted text, or once tasks exist.
- **Transactions** — each request runs in one database transaction; failures roll back explicitly.

3.4 Database Design

The database is PostgreSQL. It contains the main tables described below and a set of supporting (auxiliary) tables that are not detailed here: `profiles`, `employee_details`, `departments`, `designations`, `work_schedules`, `leaves`, `holidays`, `attendance`, `time_zones`, `task_types`, `task_details`, `task_comments` and `task_time_logs`.

Common columns. Every table inherits four columns from a common base class: `id BIGSERIAL PRIMARY KEY`, `created_at TIMESTAMPTZ NOT NULL DEFAULT now()`, `updated_at TIMESTAMPTZ` and `deleted_at TIMESTAMPTZ` (soft delete). They are listed once in each table below for completeness. Defaults such as `status = 'Draft'` or `completion_percentage = 0` are applied by the application (ORM) when a row is inserted; `created_at` is a database default.

Table 19: List of main tables

No.	Table	Purpose
1	users	Every person who can log in
2	roles	The three platform roles

No.	Table	Purpose
3	user_roles	Which user holds which role
4	employees	Workforce record linking a user to the organisation
5	projects	The core project entity
6	project_source_files	Uploaded requirement documents and their extracted text
7	project_assignments	Who is on a project and in which project role
8	task_groups	Modules of work — one Gantt bar each
9	tasks	Units of work — one GitHub issue each
10	gantt_charts	Versioned Gantt chart snapshots
11	task_discussions	Per-task discussion messages
12	notifications	In-app notifications
13	system_settings	Runtime configuration, secrets encrypted
14	productivity_scores	Weekly task-based performance scores

Table 20: Data dictionary — users

Column	Data Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Unique user id
name	VARCHAR(150)	NOT NULL	Full name
username	VARCHAR(100)	UNIQUE, NOT NULL	Login name
email	VARCHAR(255)	UNIQUE, NOT NULL	Login e-mail
password	TEXT	NOT NULL	bcrypt hash — never plain text
phone	VARCHAR(20)		Phone number
gender	VARCHAR(20)		Gender
dob	DATE		Date of birth
profile_id	BIGINT	FK → profiles.id	Extended profile
is_verified	BOOLEAN	NOT NULL, DEFAULT FALSE	E-mail verified
status	VARCHAR(20)	NOT NULL, DEFAULT 'Active'	Active, Inactive, Blocked, Password Changed
github_username	VARCHAR(100)		Used for repository invitations and issue assignment
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Creation time
updated_at	TIMESTAMPTZ		Last update
deleted_at	TIMESTAMPTZ		Soft-delete time

Table 21: Data dictionary — roles

Column	Data Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Role id
role_name	VARCHAR(50)	UNIQUE, NOT NULL	Admin, Manager or Employee
created_at / updated_at / deleted_at	TIMESTAMPTZ	as common columns	Audit and soft delete

Table 22: Data dictionary — user_roles

Column	Data Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Row id
user_id	BIGINT	FK → users.id, NOT NULL	The user
role_id	BIGINT	FK → roles.id, NOT NULL	The role held
—	—	UNIQUE (user_id, role_id)	A role is held once
created_at / updated_at / deleted_at	TIMESTAMPTZ	as common columns	Audit and soft delete

Table 23: Data dictionary — employees

Column	Data Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Employee id
user_id	BIGINT	FK → users.id, NOT NULL	Login account
employee_code	VARCHAR(50)	UNIQUE, NOT NULL	Organisation's employee code
manager_id	BIGINT	FK → employees.id	Reporting manager (self reference)
department_id	BIGINT	FK → departments.id	Department
designation_id	BIGINT	FK → designations.id	Designation
work_schedule_id	BIGINT	FK → work_schedules.id	Shift
employee_details_id	BIGINT	FK → employee_details.id	Personal details
joining_date	DATE		Date of joining
status	VARCHAR(20)	NOT NULL, DEFAULT 'Active'	Active, Inactive, Blocked
created_at / updated_at / deleted_at	TIMESTAMPTZ	as common columns	Audit and soft delete

Table 24: Data dictionary — projects

Column	Data Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Project id
name	VARCHAR(255)	NOT NULL	Project name
description	TEXT		Short description
created_by_user_id	BIGINT	FK → users.id, NOT NULL	Owning Manager
github_repo	VARCHAR(255)		`organization/repository`, created by TRACE
github_default_branch	VARCHAR(100)	NOT NULL, DEFAULT 'main'	Branch for documents
status	VARCHAR(20)	NOT NULL, DEFAULT 'Draft'	Draft, Evaluating, Active, Completed, Archived
start_date	DATE		Planned start
end_date	DATE		Planned end
project_initial_doc	VARCHAR(500)		URL — AI project document, written once
project_doc	VARCHAR(500)		URL — living project document
project_progress_doc	VARCHAR(500)		URL — weekly AI progress document
project_report_doc	VARCHAR(500)		URL — weekly AI report
last_evaluated_at	TIMESTAMPTZ		Last successful evaluation
ai_errors	JSONB		`{job: {message, at}}` — last failure of each AI job
created_at / updated_at / deleted_at	TIMESTAMPTZ	as common columns	Audit and soft delete

Table 25: Data dictionary — project_source_files

Column	Data Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	File id
project_id	BIGINT	FK → projects.id, NOT NULL	Owning project
file_name	VARCHAR(255)	NOT NULL	Original file name
storage_key	VARCHAR(500)	NOT NULL	Object key in MinIO
content_type	VARCHAR(100)		MIME type
size_bytes	BIGINT		File size
extracted_text	TEXT		Text extracted at upload, read by the AI
uploaded_by_user_id	BIGINT	FK → users.id	Uploader

Column	Data Type	Constraints	Description
created_at / updated_at / deleted_at	TIMESTAMPTZ	as common columns	Audit and soft delete

Table 26: Data dictionary — project_assignments

Column	Data Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Row id
project_id	BIGINT	FK → projects.id, NOT NULL	Project
user_id	BIGINT	FK → users.id, NOT NULL	Employee
project_role	VARCHAR(30)	NOT NULL	Project Manager or Member
assigned_by_user_id	BIGINT	FK → users.id	Who appointed / assigned
—	—	UNIQUE (project_id, user_id)	One role per person per project
created_at / updated_at / deleted_at	TIMESTAMPTZ	as common columns	Audit and soft delete

Table 27: Data dictionary — task_groups

Column	Data Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Group id
project_id	BIGINT	FK → projects.id, NOT NULL	Project
name	VARCHAR(255)	NOT NULL	Module name
description	TEXT		What the module delivers
sequence	INTEGER	NOT NULL, DEFAULT 0	Order in the plan
estimated_duration_days	INTEGER		AI estimate
planned_start_date	DATE		Gantt bar start (current forecast)
planned_end_date	DATE		Gantt bar end (current forecast)
actual_start_date	DATE		Stamped when work starts
actual_end_date	DATE		Stamped when completed
completion_percentage	NUMERIC(5,2)	NOT NULL, DEFAULT 0	Rolled up from tasks
status	VARCHAR(20)	NOT NULL, DEFAULT 'Pending'	Pending, In Progress, Completed, Blocked
created_by	VARCHAR(20)	NOT NULL, DEFAULT 'Human'	AI or Human
depends_on	JSONB		Ids of the groups this one waits for

Column	Data Type	Constraints	Description
created_at / updated_at / deleted_at	TIMESTAMPTZ	as common columns	Audit and soft delete

Table 28: Data dictionary — tasks

Column	Data Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Task id
project_id	BIGINT	FK → projects.id	Project
task_group_id	BIGINT	FK → task_groups.id	Task group
title	VARCHAR(255)	NOT NULL	Title (also the issue title)
description	TEXT		Details
type_id	BIGINT	FK → task_types.id	Task type
priority	VARCHAR(20)	NOT NULL, DEFAULT 'Medium'	Low, Medium, High, Critical
impact_type	VARCHAR(50)		Impact category
estimated_hours	NUMERIC(5,2)		Estimate
due_date	DATE		Deadline
status	VARCHAR(20)	NOT NULL, DEFAULT 'Pending'	Pending, In Progress, In Review, Completed, Blocked, Overdue, Cancelled
assigned_to_user_id	BIGINT	FK → users.id	Assignee (a Member)
assigned_by_user_id	BIGINT	FK → users.id	Assigning Project Manager
completed_by	VARCHAR(20)		Assignee or Project Manager
completed_at	TIMESTAMPTZ		Completion time
github_issue_id	VARCHAR(100)		Issue number
github_issue_url	VARCHAR(255)		Issue link
task_progress_doc	VARCHAR(500)		URL — AI step plan and checklist
progress_doc_error	TEXT		Why the AI could not write it
health_score	VARCHAR(20)	NOT NULL, DEFAULT 'Green'	Green, Yellow, Red
health_reason	TEXT		Explanation of the health score
last_health_check	TIMESTAMPTZ		Last health computation
commit_hash, commit_verified, llm_verification_notes	VARCHAR(100), BOOLEAN, TEXT	commit_verified NOT NULL DEFAULT FALSE	Retained legacy values, no longer written
created_at / updated_at / deleted_at	TIMESTAMPTZ	as common columns	Audit and soft delete

Table 29: Data dictionary — gantt_charts

Column	Data Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Row id
project_id	BIGINT	FK → projects.id, NOT NULL	Project
version	INTEGER	NOT NULL, DEFAULT 1	Incremented per regeneration
chart_data	JSONB		Validated render payload: range, bars, tasks, dependencies, milestones
generated_reason	VARCHAR(100)		initial, task_status_change, project_doc_updated, manual
generated_at	TIMESTAMPTZ		Generation time
generated_by_job	VARCHAR(64)		Background job id that wrote it
created_at / updated_at / deleted_at	TIMESTAMPTZ	as common columns	Audit and soft delete

Table 30: Data dictionary — task_discussions

Column	Data Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Message id
task_id	BIGINT	FK → tasks.id, NOT NULL	Task discussed
user_id	BIGINT	FK → users.id, NOT NULL	Author
parent_id	BIGINT	FK → task_discussions.id	Thread root (self reference)
message	TEXT	NOT NULL	Message text
created_at / updated_at / deleted_at	TIMESTAMPTZ	as common columns	Audit and soft delete

Table 31: Data dictionary — notifications

Column	Data Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Notification id
user_id	BIGINT	FK → users.id, NOT NULL	Recipient
type	VARCHAR(50)	NOT NULL	task_assigned, task_due, task_completed, discussion_reply, project_evaluated
title	VARCHAR(255)	NOT NULL	Headline
body	TEXT		Detail

Column	Data Type	Constraints	Description
link	VARCHAR(500)		Deep link into the recipient's panel
is_read	BOOLEAN	NOT NULL, DEFAULT FALSE	Read flag
related_task_id	BIGINT	FK → tasks.id	Related task
related_project_id	BIGINT	FK → projects.id	Related project
created_at / updated_at / deleted_at	TIMESTAMPTZ	as common columns	Audit and soft delete

Table 32: Data dictionary — system_settings

Column	Data Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Row id
category	VARCHAR(50)	NOT NULL	github or mail
key	VARCHAR(100)	NOT NULL	Setting name, e.g. token, owner, host
value	TEXT		Plain value, or a Fernet-encrypted token for secrets
is_secret	BOOLEAN	NOT NULL, DEFAULT FALSE	Secret values are never returned by any API
—	—	UNIQUE (category, key)	One value per setting
created_at / updated_at / deleted_at	TIMESTAMPTZ	as common columns	Audit and soft delete

Table 33: Data dictionary — productivity_scores

Column	Data Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Row id
employee_id	BIGINT	FK → employees.id, NOT NULL	Employee scored
week_start	DATE	NOT NULL	Monday of the week
task_score	NUMERIC(5,2)		Completed ÷ assigned tasks due that week × 100
deadline_score	NUMERIC(5,2)		Completed on time ÷ completed × 100
final_score	NUMERIC(5,2)		Mean of the two scores
rating	VARCHAR(15)		Excellent, Good, Average, Risk
att_score, active_time_score	NUMERIC(5,2)		Retained, left empty — attendance is not a scoring input
anomaly_flags	JSONB		Flags raised for the week
generated_at	TIMESTAMPTZ		Computation time

Column	Data Type	Constraints	Description
created_at / updated_at / deleted_at	TIMESTAMPTZ	as common columns	Audit and soft delete

Relationships Table

Table 34: Relationships between tables

Parent table	Child table	Relationship	Through column
users	user_roles	One-to-Many	user_roles.user_id
roles	user_roles	One-to-Many	user_roles.role_id
users	employees	One-to-One	employees.user_id
employees	employees	One-to-Many (self join)	employees.manager_id
users	projects	One-to-Many	projects.created_by_user_id
projects	project_source_files	One-to-Many	project_id
projects	project_assignments	One-to-Many	project_id
users	project_assignments	One-to-Many	user_id
projects	task_groups	One-to-Many	project_id
task_groups	tasks	One-to-Many	task_group_id
projects	tasks	One-to-Many	project_id
users	tasks	One-to-Many	assigned_to_user_id, assigned_by_user_id
projects	gantt_charts	One-to-Many	project_id
tasks	task_discussions	One-to-Many	task_id
task_discussions	task_discussions	One-to-Many (self join)	parent_id
users	notifications	One-to-Many	user_id
employees	productivity_scores	One-to-Many	employee_id

3.5 Procedural Design (Process Logic)

The process logic of the main operations is given below as numbered steps.

Login

1. The user enters e-mail and password on the login page and presses *Sign in*.
2. The API finds the active user with that e-mail; if none exists, it answers "Invalid credentials".
3. The password is checked against the stored bcrypt hash; on mismatch, "Invalid credentials".

4. The API issues an access token and a refresh token (JWT, signed with the server secret) and returns them with the user's roles.
5. The browser stores the token and routes the user to `/admin`, `/manager` or `/employee` by role.

Create project

1. The Manager enters the name and description, adds source files and reviews the repository name shown.
2. The API resolves the GitHub Organization and token from Configuration (database value, else `.env`); if either is missing, it answers 503.
3. The name is turned into a slug (lower-case, other characters to `-`, at most 100 characters).
4. The API creates the public repository under the Organization. A name clash answers 409; any other GitHub failure answers 502 and nothing is saved.
5. The project row is saved with `status = Draft` and the repository name.
6. Each source file is stored in MinIO and its text is extracted and saved; unsupported or oversized files are reported back.

Evaluate project

1. The owner presses *Evaluate Project*. The API refuses (409) if the project is already evaluating, has no extracted text or already has tasks, and (503) if the AI or GitHub is not configured.
2. The project is set to *Evaluating*, committed, and the `evaluate_project` job is queued; the API returns the job id.
3. The job reads the extracted text of all source files (at most 60,000 characters).
4. The AI writes the project document; it is committed to `docs/project_initial_doc.md` and `docs/project_doc.md`.
5. The AI plans the task groups (name, description, duration, dependencies); the plan is validated.
6. Old groups are soft-deleted and the new groups saved; the scheduling rule assigns dates; Gantt version 1 is saved with reason *initial*.
7. The project becomes *Active*, the owner is notified; on any failure the transaction is rolled back, the project returns to its previous state and the failed step is recorded in `ai_errors`.

Create task

1. The Project Manager fills in the task (optionally using *Suggest tasks* and *Suggest assignees*) and saves.

2. The API checks the caller is a Project Manager of the project and that the group belongs to it.
3. The task row is flushed. If an assignee is chosen, they become a Member and are invited to the repository.
4. The GitHub issue is created (assigned to the assignee's GitHub account if possible). If GitHub fails, the transaction is rolled back and nothing is kept.
5. The issue number and URL are stored, the assignee is notified, the group is rolled up and the transaction committed.
6. The `generate_task_progress_doc` job is queued to write the task's plan and checklist.

Update task status

1. The user chooses a new status (the buttons shown are exactly the server's `allowed_statuses`).
2. The API checks status authority; a caller without it receives 403.
3. If the task moves between open and closed, the GitHub issue is closed (completed / not planned) or re-opened.
4. On *Completed*, `completed_by` and `completed_at` are set and the Project Managers (and the assignee, if someone else completed it) are notified; otherwise they are cleared.
5. The group is rolled up and the transaction committed; if completion changed, a Gantt re-plan is requested.

Gantt re-plan

1. A trigger calls `request_reevaluation(project, reason)`, which queues the job with a 30-second delay (none for the manual button).
2. When it runs, the job exits if a version newer than its request time already exists.
3. For *project_doc_updated*, the AI proposes revised durations for unfinished groups and new groups for uncovered features.
4. The scheduling rule re-computes every group's planned dates.
5. A new chart is built, validated and saved as the next version with its reason and job id.

Weekly progress report

1. Every Monday at 07:00, one job is queued per active project.
2. The job reads every task's progress document from the repository and the current task statuses.
3. The AI writes the progress document and the report document.

4. Both are committed to the repository and their URLs stored on the project.

Object-Oriented Design

The backend is object-oriented. The main classes and their responsibilities are:

Table 35: Main classes and responsibilities

Class	Responsibility
`BaseModelMixin`	Common id, timestamps and soft-delete columns for every entity
`User`, `Role`, `UserRole`	Identity and platform roles
`Employee`	Workforce record of a user
`Project`, `ProjectSourceFile`, `ProjectAssignment`	Project, its documents and its team
`TaskGroup`, `Task`	Modules and units of work
`GanttChart`	One immutable version of a project's chart
`TaskDiscussion`, `Notification`	Communication records
`SystemSetting`	A configuration value, encrypted when secret
`ProjectAccess`	The caller's rights in one project: owner, Project Manager, assignee; `may_set_status`, `allowed_statuses`, `may_discuss`
`GitHubClient`	Every GitHub call: repositories, collaborators, issues, files
`AIClient`	The model connection in JSON mode
`GanttChartData`	Schema and validator of a Gantt version
`Outbox`	Notifications collected in a transaction, delivered after the commit
`Settings`	Configuration from `.env`, with production checks

3.6 User Interface Design

The user interface was designed around the following principles:

- **One audience per panel.** Admin, Manager and Employee each have their own layout and base route; nothing a role cannot use is shown to it.
- **The project is the unit of navigation.** Manager and Employee panels have a project selector in the top bar, defaulting to "All" (a combined dashboard); the URL always identifies the selected project.
- **The server decides what the user may do.** Buttons such as status changes and *Edit* are rendered from the permissions in the API response.

- **Long work never blocks the screen.** AI actions show a running state, poll for the result and report failures with their reason.
- **Consistency.** One component library, design tokens for every colour, one icon set, the same page shell everywhere, and light and dark themes.
- **Feedback.** Every action ends in a toast message; lists and AI surfaces have empty and error states.

Table 36: Screens of the system

Screen	Panel	Purpose
Landing page	Public	Product overview, how it works, features, sign-in link
Login	Public	E-mail and password sign-in
403 page	All	Shown when a user opens another role's panel
Control Panel	Admin	Table picker and generic data grid with create, edit, delete
Workforce	Admin	Tabs for employees, managers, leaves, holidays, attendance, shifts, departments, designations; bulk upload
Monitor	Admin	Health cards for every service; recent job outcomes
Configuration	Admin	GitHub and mail settings with connection tests
Manager dashboard	Manager	Project counts, progress, at-risk projects, activity, report download, *New Project*
Create project wizard	Manager	Details → source files → review → create
Project page	Manager	Overview (evaluation card, documents, files), Tree, Gantt, Progress Report, Team
Employee dashboard	Employee	Progress across projects, my tasks by due state, overdue callout, report download
Project page (PM)	Employee	Overview, Tree, Gantt, Tasks (board with *Create Task*), Project Doc editor, Team
Project page (Member)	Employee	Overview, Tree, Gantt, My Tasks
Task detail	Employee / Manager	Description, GitHub issue, progress document with checklist, status buttons, discussion
Notification bell	All	Unread count, latest notifications, deep links, mark all read
Settings	All	Own profile, GitHub username, password

The main screen of each type of user follows, one to a page and turned to fit it. The other screens are shown in Chapter 9 (Output).

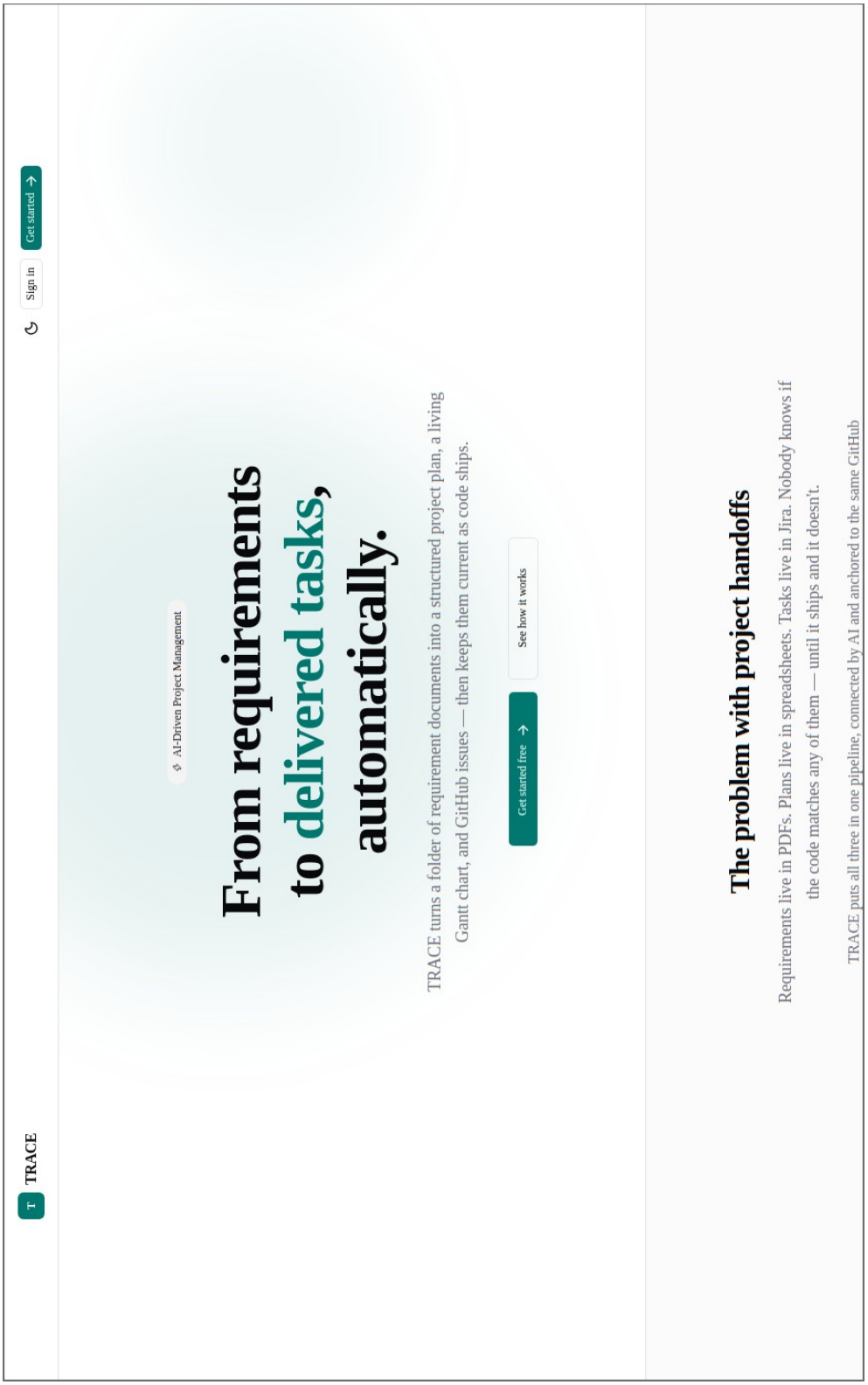


Figure 19: Visitor — Landing Page

The public page: what TRACE does — from requirement documents to delivered tasks — and the way to sign in.

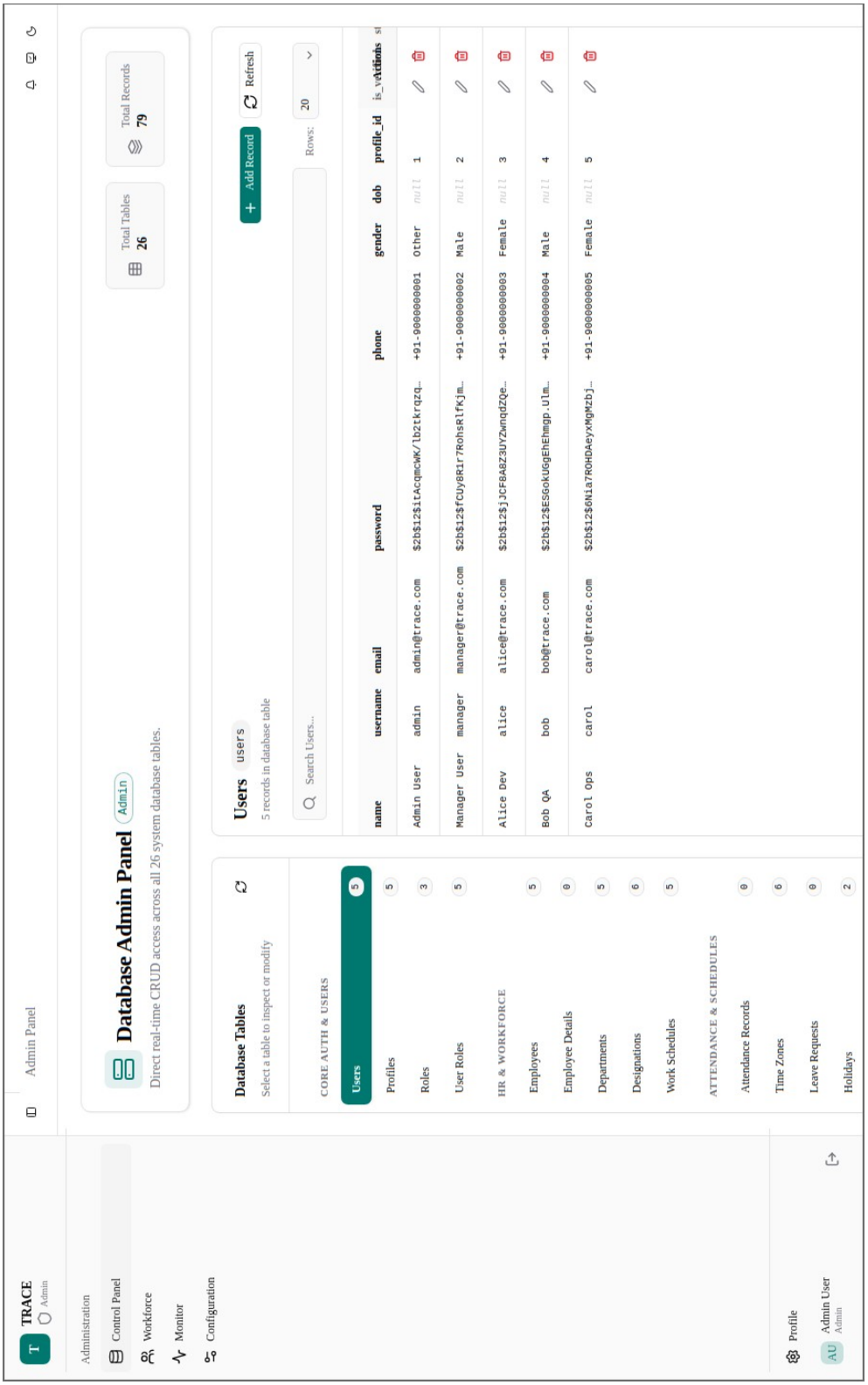


Figure 20: Admin — Control Panel

Direct create, read, update and delete access to every table of the database, grouped by area.

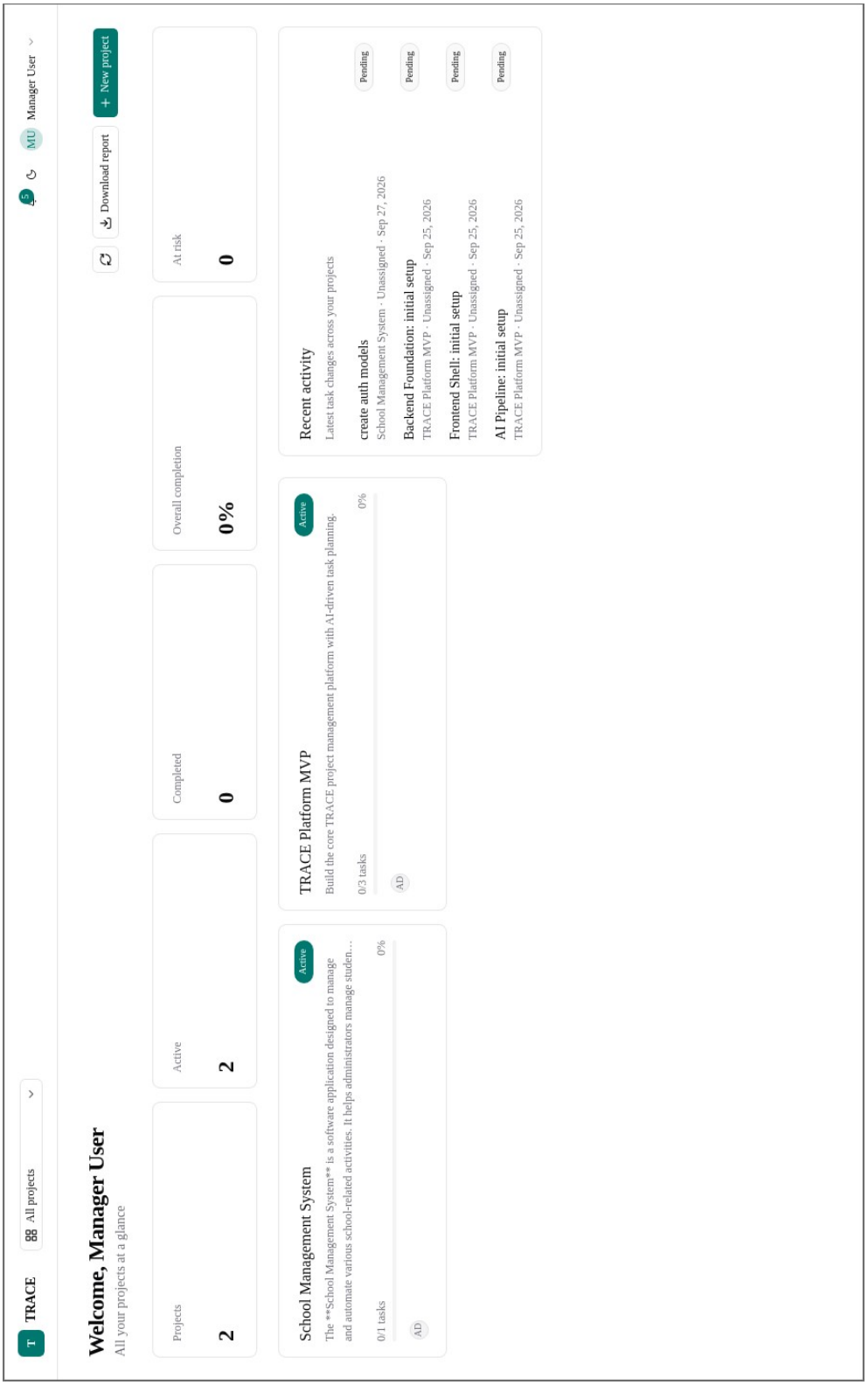


Figure 21: Manager — Dashboard

All the Manager's projects at a glance: counts, completion, projects at risk and the latest task activity.

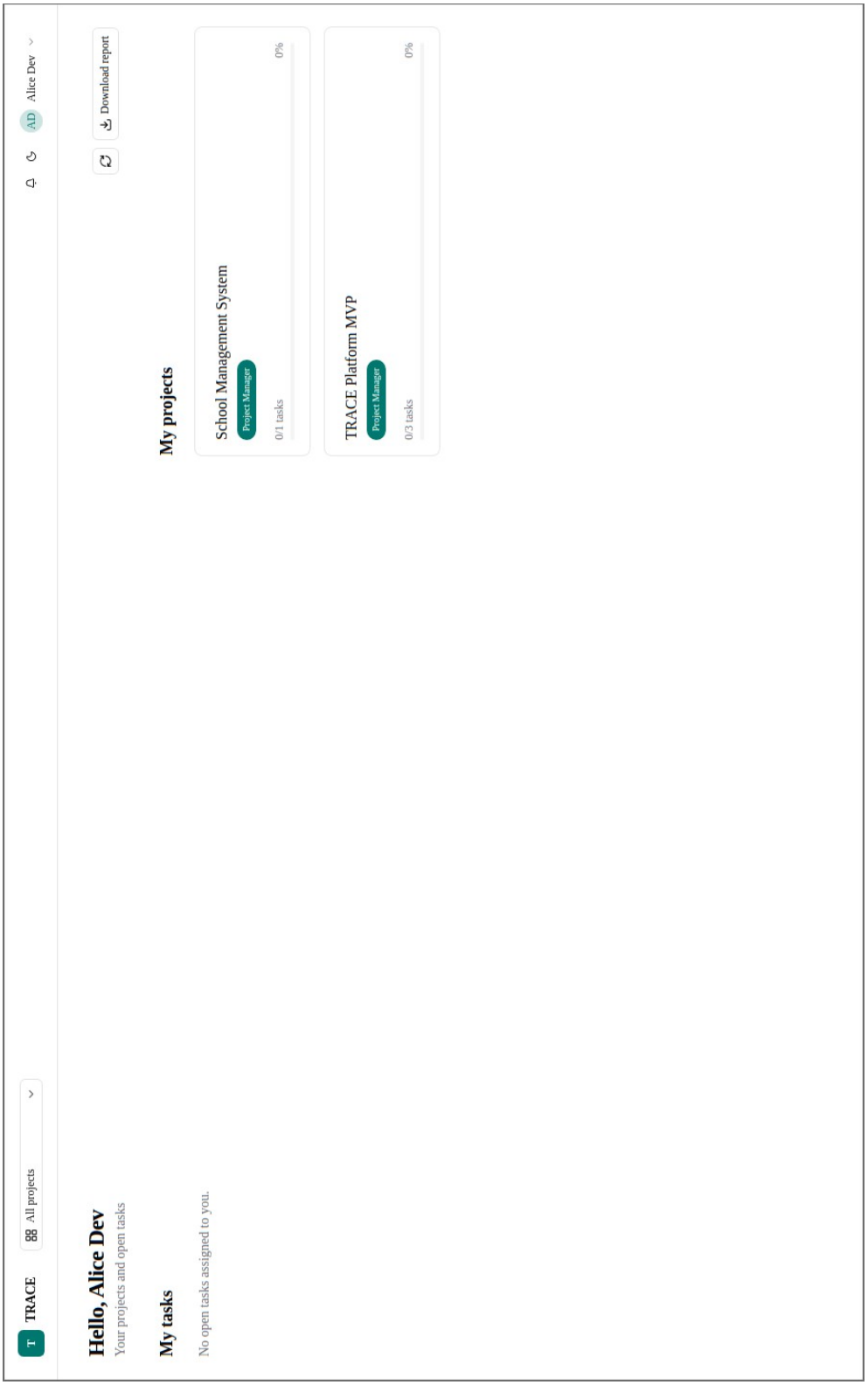


Figure 22: Employee — Dashboard

The employee's open tasks and the projects they work on, with their role and progress in each.

3.7 Test Case Design

Test cases were designed before and during implementation, at two levels.

Unit test cases exercise one function or endpoint in isolation, with the external services (GitHub, the AI model, MinIO, Redis) replaced by fakes that behave like the real services. They cover each module's rules — validation, authorisation, the task–issue rule, the scheduling rule, schema validation of AI output.

System test cases exercise complete user flows through the API across modules — for example, *a Manager creates a project, appoints a Project Manager, who creates a task that becomes a GitHub issue*, or *an evaluation writes documents, task groups and Gantt version 1*.

Techniques used in designing them:

- **Equivalence partitioning** — valid and invalid classes of input (supported and unsupported file types; statuses the caller may and may not set).
- **Boundary value analysis** — limits such as 20 MB uploads, 1–20 task groups, 1–365 days, title lengths, the 60,000-character source limit.
- **Decision tables** — who may do what: every role (owner Manager, other Manager, Admin, Project Manager, Member, outsider, anonymous) against every project route.
- **State transition testing** — task and project status changes, including re-opening.
- **Error guessing** — GitHub refusing an invitation, a pending invitee, an AI that keeps returning invalid JSON, a changed encryption key.

The designed test cases and their results are reported in Chapter 6.

4. CODING

4.1 Complete Project Coding

4.1.1 Project Structure

```
trace/
├── backend/
│   ├── alembic/
│   │   ├── versions/  (9 files: Alembic revisions)
│   │   ├── env.py
│   │   └── script.py.mako
│   ├── app/
│   │   ├── api/
│   │   │   └── v1/
│   │   │       ├── admin.py
│   │   │       ├── attendance.py
│   │   │       ├── auth.py
│   │   │       ├── employees.py
│   │   │       ├── hrms.py
│   │   │       ├── notifications.py
│   │   │       ├── productivity.py
│   │   │       ├── projects.py
│   │   │       ├── router.py
│   │   │       ├── services.py
│   │   │       ├── settings.py
│   │   │       ├── tasks.py
│   │   │       ├── users.py
│   │   │       └── workforce.py
│   │   ├── core/
│   │   │   ├── permissions.py
│   │   │   ├── project_access.py
│   │   │   └── security.py
│   │   ├── models/
│   │   │   ├── attendance.py
│   │   │   ├── base.py
│   │   │   ├── employee.py
│   │   │   ├── hrms.py
│   │   │   ├── productivity.py
│   │   │   ├── project.py
│   │   │   ├── task.py
│   │   │   └── user.py
│   │   ├── schemas/
│   │   │   ├── ai.py
│   │   │   ├── attendance.py
│   │   │   ├── auth.py
│   │   │   ├── employee.py
│   │   │   ├── gantt.py
│   │   │   ├── productivity.py
│   │   │   ├── project.py
│   │   │   └── user.py
│   │   ├── services/
│   │   │   ├── ai_jobs.py
│   │   │   ├── ai_service.py
│   │   │   ├── auth_service.py
│   │   │   ├── employee_service.py
│   │   │   ├── extraction.py
│   │   │   ├── gantt_service.py
│   │   │   ├── github_service.py
│   │   │   ├── jobs.py
│   │   │   ├── mail_service.py
│   │   │   ├── notification_service.py
│   │   │   ├── project_report.py
│   │   │   ├── project_service.py
│   │   │   ├── realtime.py
│   │   │   ├── report_service.py
│   │   │   ├── scoring_service.py
│   │   │   ├── storage.py
│   │   │   ├── system_settings.py
```

```
│   │   │   ├── user_service.py
│   │   │   └── workforce.py
│   │   ├── utils/
│   │   │   └── helpers.py
│   │   ├── celery_app.py
│   │   ├── cli.py
│   │   ├── config.py
│   │   ├── database.py
│   │   ├── dependencies.py
│   │   ├── exceptions.py
│   │   ├── main.py
│   │   ├── middleware.py
│   │   ├── seeds.py
│   │   └── tasks.py
│   ├── .gitignore
│   ├── alembic.ini
│   ├── Dockerfile
│   ├── main.py
│   ├── requirements.txt
│   ├── schema.py
│   └── setup.cfg
├── frontend/
│   ├── public/  (2 files: icons)
│   ├── src/
│   │   ├── api/
│   │   │   ├── admin.js
│   │   │   ├── auth.js
│   │   │   ├── client.js
│   │   │   ├── index.js
│   │   │   ├── notifications.js
│   │   │   ├── projects.js
│   │   │   ├── services.js
│   │   │   ├── settings.js
│   │   │   ├── tasks.js
│   │   │   └── workforce.js
│   │   ├── app/
│   │   │   └── dashboard/
│   │   │       └── data.json
│   │   ├── assets/  (3 files: images)
│   │   ├── components/
│   │   │   ├── project/
│   │   │   │   ├── EvaluationCard.jsx
│   │   │   │   ├── NotYetAvailable.jsx
│   │   │   │   ├── PeopleSuggestions.jsx
│   │   │   │   ├── ProgressReport.jsx
│   │   │   │   ├── ProjectDocEditor.jsx
│   │   │   │   ├── ProjectOverview.jsx
│   │   │   │   ├── ReportDownload.jsx
│   │   │   │   ├── status.jsx
│   │   │   │   ├── TaskBoard.jsx
│   │   │   │   ├── TaskDiscussion.jsx
│   │   │   │   └── TeamPanel.jsx
│   │   │   ├── ui/  (62 files: shadcn/ui components, installed by its CLI)
│   │   │   ├── GanttChart.jsx
│   │   │   ├── GuestOnlyRoute.jsx
│   │   │   ├── Markdown.jsx
│   │   │   ├── NotificationBell.jsx
│   │   │   ├── ProjectSelector.jsx
│   │   │   ├── ProjectTreeGraph.jsx
│   │   │   ├── ProtectedRoute.jsx
│   │   │   ├── RoleRoute.jsx
│   │   │   └── ServiceMonitor.jsx
│   │   ├── contexts/
│   │   │   ├── AuthContext.jsx
│   │   │   ├── LiveContext.jsx
│   │   │   └── ThemeContext.jsx
│   │   ├── hooks/
│   │   │   └── use-mobile.js
│   │   ├── layouts/
│   │   │   ├── AdminHeader.jsx
│   │   │   ├── AdminLayout.jsx
│   │   │   ├── EmployeeHeader.jsx
```

```
│   │   │   ├── EmployeeLayout.jsx
│   │   │   ├── ManagerHeader.jsx
│   │   │   ├── ManagerLayout.jsx
│   │   │   └── Sidebar.jsx
│   │   ├── lib/
│   │   │   ├── jobs.js
│   │   │   ├── project.js
│   │   │   └── utils.js
│   │   ├── pages/
│   │   │   ├── admin/
│   │   │   │   ├── AdminSettingsPage.jsx
│   │   │   │   ├── ConfigurationPage.jsx
│   │   │   │   ├── ControlPanelPage.jsx
│   │   │   │   ├── MonitorPage.jsx
│   │   │   │   └── WorkforcePage.jsx
│   │   │   ├── employee/
│   │   │   │   ├── EmployeeDashboardPage.jsx
│   │   │   │   ├── EmployeeSettingsPage.jsx
│   │   │   │   ├── ProjectDetailPage.jsx
│   │   │   │   └── TaskDetailPage.jsx
│   │   │   ├── manager/
│   │   │   │   ├── ManagerDashboardPage.jsx
│   │   │   │   ├── ManagerSettingsPage.jsx
│   │   │   │   ├── ProjectCreatePage.jsx
│   │   │   │   └── ProjectDetailPage.jsx
│   │   │   ├── ForbiddenPage.jsx
│   │   │   ├── LandingPage.jsx
│   │   │   ├── LoginPage.jsx
│   │   │   └── NotFoundPage.jsx
│   │   ├── App.jsx
│   │   └── main.jsx
│   ├── .gitignore
│   ├── .oxlintrc.json
│   ├── components.json
│   ├── Dockerfile
│   ├── index.html
│   ├── jsconfig.json
│   ├── package.json
│   └── vite.config.js
├── pgadmin/
│   └── servers.json
├── .env
├── .env.example
├── .gitignore
├── docker-compose.yml
├── migrate.sh
├── start.sh
└── stop.sh
```

4.1.2 Running the Project

TRACE runs in Docker containers, and one script, **start.sh**, whose code follows, starts, stops and migrates the whole system. The machine needs Docker with Docker Compose and git. To run the project:

1. Clone the repository and enter it: **git clone <repository-url> trace** and **cd trace**.
2. Create the environment file from the example: **cp .env.example .env**, and fill in the database login, the JWT secret, the encryption key and the LLM provider's key.
3. Start everything: **./start.sh start**. The first run builds the images; the database migrations run before the application starts.
4. Apply the migrations again whenever the schema changes: **./start.sh migrate**.

5. Open **http://localhost:5173** and sign in as the administrator with **admin@trace.com** / **Admin@123**.
6. Stop with **./start.sh stop**; **./start.sh status** shows what is running.

start.sh

```
#!/usr/bin/env bash

set -e

ROOT_DIR="$(cd "$(dirname "$0")" && pwd)"
BACKEND_DIR="$ROOT_DIR/backend"
FRONTEND_DIR="$ROOT_DIR/frontend"
VENV_DIR="$BACKEND_DIR/.venv"
PID_FILE="$ROOT_DIR/.trace_pids"

GREEN='\033[0;32m'
CYAN='\033[0;36m'
YELLOW='\033[1;33m'
RED='\033[0;31m'
BOLD='\033[1m'
NC='\033[0m'

env_value() {
    [ -f "$ROOT_DIR/.env" ] || return 0
    grep -E "^$1=" "$ROOT_DIR/.env" | tail -1 | cut -d= -f2- | sed -e
     's/^["'\'']//' -e 's/["'\'']$//'
}

load_app_env() {
    APP_ENV="${APP_ENV:-$(env_value APP_ENV)}"
    APP_ENV="${APP_ENV:-local}"
    case "$APP_ENV" in
        local|production) ;;
        *) echo -e "${RED}❌ APP_ENV must be 'local' or 'production', got
     '$APP_ENV'${NC}"; exit 1 ;;
    esac
    PRODUCTION_URL="$(env_value PRODUCTION_URL)"
    PRODUCTION_URL="${PRODUCTION_URL#https://}"; PRODUCTION_URL="$
     {PRODUCTION_URL#http://}"; PRODUCTION_URL="${PRODUCTION_URL%/}"
    export APP_ENV
}

is_production() { [ "$APP_ENV" = "production" ]; }

compose() {
    if is_production; then
        docker compose -f "$ROOT_DIR/docker-compose.yml" --profile
     production "$@"
    else
        docker compose -f "$ROOT_DIR/docker-compose.yml" "$@"
    fi
}

preflight_production() {
    echo -e "${CYAN}🔒 Production preflight...${NC}"
    local failed=0
    if [ -z "$(env_value CLOUDFLARE_TUNNEL_TOKEN)" ]; then
        echo -e "${RED}  ✗ CLOUDFLARE_TUNNEL_TOKEN is empty in .env${NC}";
     failed=1
    fi
    if [ -z "$PRODUCTION_URL" ]; then
        echo -e "${RED}  ✗ PRODUCTION_URL is empty in .env${NC}"; failed=1
    fi
```

```
    if ! (cd "$BACKEND_DIR" && "$VENV_DIR/bin/python" -c "from app.config
     import get_settings; get_settings()" 2>&1 \
            | grep -E "^RuntimeError: |^  - " | sed "s/^/  /"; exit "$
     {PIPESTATUS[0]}"); then
        failed=1
    fi
    if [ "$failed" -ne 0 ]; then
        echo -e "${RED}❌ Not starting in production. Fix the above, or set
     APP_ENV=local.${NC}"
        exit 1
    fi
    echo -e "${GREEN}  ✓ passed${NC}"
}

load_app_env

print_banner() {
    local title="$1"
    echo ""
    echo -e "${CYAN}╔═══════════════════════════════════════════════════════════╗$
     {NC}"
    echo -e "${CYAN}║          ${GREEN}TRACE — ${title}${CYAN}            ║$
     {NC}"
    echo -e "${CYAN}╚═══════════════════════════════════════════════════════════╝$
     {NC}"
    echo ""
}

wait_for_db() {
    echo -e "${YELLOW}⏳ Waiting for PostgreSQL to be ready...${NC}"
    for i in $(seq 1 30); do
        if docker exec trace_db pg_isready -U trace_user -d trace_db >
     /dev/null 2>&1; then
            echo -e "${GREEN}✅ PostgreSQL is ready!${NC}"
            return 0
        fi
        if [ "$i" -eq 30 ]; then
            echo -e "${RED}❌ PostgreSQL failed to start. Check: docker
     compose logs db${NC}"
            exit 1
        fi
        sleep 1
    done
}

wait_for_redis() {
    echo -e "${YELLOW}⏳ Waiting for Redis to be ready...${NC}"
    for i in $(seq 1 15); do
        if docker exec trace_redis redis-cli ping > /dev/null 2>&1; then
            echo -e "${GREEN}✅ Redis is ready!${NC}"
            return 0
        fi
        if [ "$i" -eq 15 ]; then
            echo -e "${RED}❌ Redis failed to start. Check: docker compose
     logs redis${NC}"
            exit 1
        fi
        sleep 1
    done
}

ensure_env() {
    if [ ! -f "$VENV_DIR/bin/python" ]; then
        echo -e "${YELLOW}📦 Creating Python virtual environment...${NC}"
        python3 -m venv "$VENV_DIR"
    fi
    if [ -f "$BACKEND_DIR/requirements.txt" ]; then
        echo -e "${YELLOW}📦 Checking Python dependencies...${NC}"
```

```
        "$VENV_DIR/bin/pip" install -q -r "$BACKEND_DIR/requirements.txt"
    fi
    if [ ! -d "$FRONTEND_DIR/node_modules" ]; then
        echo -e "${YELLOW}📦 Installing frontend dependencies...${NC}"
        (cd "$FRONTEND_DIR" && npm ci)
    fi
}

do_start() {
    print_banner "Starting Up ($APP_ENV)..."

    ensure_env
    if is_production; then
        preflight_production
    fi

    local infra="db redis pgadmin mailpit minio"
    if is_production; then infra="$infra cloudflared"; fi
    echo -e "${CYAN}🐳 Starting Docker services: $infra${NC}"
    compose up -d $infra 2>&1 | tail -8

    wait_for_db
    wait_for_redis

    echo -e "${GREEN}✅ pgAdmin  → http://localhost:5050${NC}"
    echo -e "${GREEN}✅ Mailpit  → http://localhost:8025${NC}"
    echo -e "${GREEN}✅ MinIO    → http://localhost:9001${NC}"

    echo -e "${CYAN}⚡ Running Alembic migration (upgrade head)...${NC}"
    (cd "$BACKEND_DIR" && "$VENV_DIR/bin/alembic" upgrade head)

    cleanup_fg() {
        echo ""
        echo -e "${YELLOW}🛑  Shutting down TRACE local processes...${NC}"
        if [ -f "$PID_FILE" ]; then
            while read -r pid; do
                if kill -0 "$pid" 2>/dev/null; then
                    kill "$pid" 2>/dev/null || true
                fi
            done < "$PID_FILE"
            rm -f "$PID_FILE"
        fi
        kill 0 2>/dev/null || true
        wait 2>/dev/null || true
        echo -e "${GREEN}✅ Local processes stopped.${NC}"
    }
    trap cleanup_fg EXIT INT TERM

    > "$PID_FILE"

    echo ""
    echo -e "${CYAN}🚀 Starting Backend (FastAPI) on http://localhost:8000$
     {NC}"
    local uvicorn_args="--reload --host 0.0.0.0 --port 8000"
    if is_production; then uvicorn_args="--host 127.0.0.1 --port 8000"; fi
    (
        cd "$BACKEND_DIR"
        exec "$VENV_DIR/bin/uvicorn" app.main:app $uvicorn_args 2>&1 | \
            sed "s/^/  [backend] /"
    ) &
    BACKEND_PID=$!
    echo "$BACKEND_PID" >> "$PID_FILE"

    sleep 3

    echo -e "${CYAN}⚙️  Starting Celery Worker...${NC}"
    (
        cd "$BACKEND_DIR"
```

```
    exec "$VENV_DIR/bin/celery" -A app.celery_app worker --loglevel=info
 2>&1 | \
        sed "s/^/  [celery-w] /"
) &
CELERY_W_PID=$!
echo "$CELERY_W_PID" >> "$PID_FILE"

echo -e "${CYAN}⏰ Starting Celery Beat...${NC}"
(
    cd "$BACKEND_DIR"
    exec "$VENV_DIR/bin/celery" -A app.celery_app beat --loglevel=info
 2>&1 | \
        sed "s/^/  [celery-b] /"
) &
CELERY_B_PID=$!
echo "$CELERY_B_PID" >> "$PID_FILE"

if is_production; then
    echo -e "${CYAN}🏗️  Building frontend for production...${NC}"
    (cd "$FRONTEND_DIR" && npm run build 2>&1 | tail -3)
    echo -e "${CYAN}🚀 Serving built frontend (vite preview) on
 http://127.0.0.1:5173${NC}"
    (
        cd "$FRONTEND_DIR"
        exec npx vite preview 2>&1 | sed "s/^/  [frontend] /"
    ) &
else
    echo -e "${CYAN}🚀 Starting Frontend (Vite) on
 http://localhost:5173${NC}"
    (
        cd "$FRONTEND_DIR"
        exec npm run dev 2>&1 | sed "s/^/  [frontend] /"
    ) &
fi
FRONTEND_PID=$!
echo "$FRONTEND_PID" >> "$PID_FILE"

sleep 3
echo ""
echo -e "$
 {GREEN}╔═══════════════════════════════════════════════════════════════╗${NC}"
echo -e "${GREEN}║          TRACE is running successfully! 🎉
 ║${NC}"
echo -e "$
 {GREEN}╠═══════════════════════════════════════════════════════════════╣${NC}"
if is_production; then
echo -e "${GREEN}║  Public   → ${CYAN}https://${PRODUCTION_URL}${NC}"
fi
echo -e "${GREEN}║  Frontend → ${CYAN}http://localhost:5173${GREEN}
 ║${NC}"
echo -e "${GREEN}║  Backend  → ${CYAN}http://localhost:8000${GREEN}
 ║${NC}"
echo -e "${GREEN}║  API Docs → ${CYAN}http://localhost:8000/docs${GREEN}
 ║${NC}"
echo -e "${GREEN}║  pgAdmin  → ${CYAN}http://localhost:5050${GREEN}
 ║${NC}"
echo -e "${GREEN}║  Mailpit  → ${CYAN}http://localhost:8025${GREEN}
 ║${NC}"
echo -e "${GREEN}║  MinIO    → ${CYAN}http://localhost:9001${GREEN}
 ║${NC}"
echo -e "$
 {GREEN}╠═══════════════════════════════════════════════════════════════╣${NC}"
if ! is_production; then
echo -e "${GREEN}║  Login:
 ║${NC}"
echo -e "${GREEN}║    Email:    ${YELLOW}admin@trace.com${GREEN}
 ║${NC}"
```

```
    echo -e "${GREEN}║    Password: ${YELLOW}Admin@123${GREEN}
     ║${NC}"
    echo -e "$
     {GREEN}╠═══════════════════════════════════════════════════════════╣${NC}"
    fi
    echo -e "${GREEN}║  Press ${RED}Ctrl+C${GREEN} or run ${CYAN}./start.sh
     stop${GREEN} to stop.  ║${NC}"
    echo -e "$
     {GREEN}╚═══════════════════════════════════════════════════════════╝${NC}"
    echo ""

    wait
}

do_stop() {
    print_banner "Stopping Environment..."

    if [ -f "$PID_FILE" ]; then
        echo -e "${YELLOW}🛑 Stopping background application processes...$
     {NC}"
        while read -r pid; do
            if kill -0 "$pid" 2>/dev/null; then
                kill "$pid" 2>/dev/null || true
            fi
        done < "$PID_FILE"
        rm -f "$PID_FILE"
    fi

    pkill -f "uvicorn app.main:app" 2>/dev/null || true
    pkill -f "celery -A app.celery_app" 2>/dev/null || true
    pkill -f "vite" 2>/dev/null || true

    echo -e "${CYAN}🐳 Stopping Docker containers...${NC}"
    docker compose -f "$ROOT_DIR/docker-compose.yml" --profile production
     down --remove-orphans

    echo ""
    echo -e "${GREEN}✅ All Docker containers and TRACE services have been
     stopped successfully.${NC}"
    echo ""
}

do_migrate() {
    print_banner "Running Database Migrations..."

    echo -e "${CYAN}🐳 Ensuring PostgreSQL container is running...${NC}"
    docker compose -f "$ROOT_DIR/docker-compose.yml" up -d db

    wait_for_db

    if [ ! -f "$VENV_DIR/bin/python" ]; then
        echo -e "${YELLOW}📦 Creating Python virtual environment...${NC}"
        python3 -m venv "$VENV_DIR"
        "$VENV_DIR/bin/pip" install -q -r "$BACKEND_DIR/requirements.txt"
    fi

    echo -e "${CYAN}⚡ Running Alembic migration (upgrade head)...${NC}"
    (
        cd "$BACKEND_DIR"
        "$VENV_DIR/bin/alembic" upgrade head
    )

    echo ""
    echo -e "${GREEN}✅ Database migrations completed successfully!${NC}"
    echo ""
}

do_status() {
```

```
    print_banner "Service Status"

    echo -e "${BOLD}Environment:${NC} APP_ENV=${APP_ENV}"
    if is_production; then
        echo -e "  Public URL: https://${PRODUCTION_URL}  →  $(curl -s -o
     /dev/null -w '%{http_code}' --max-time 10 "https://${PRODUCTION_URL}/"
     || echo unreachable)"
    fi
    echo ""
    echo -e "${BOLD}Docker Containers:${NC}"
    compose ps

    echo ""
    echo -e "${BOLD}Local Services:${NC}"
    if pgrep -f "uvicorn app.main:app" > /dev/null; then
        echo -e "  Backend (FastAPI):  ${GREEN}RUNNING${NC}"
    else
        echo -e "  Backend (FastAPI):  ${RED}STOPPED${NC}"
    fi

    if pgrep -f "celery -A app.celery_app worker" > /dev/null; then
        echo -e "  Celery Worker:      ${GREEN}RUNNING${NC}"
    else
        echo -e "  Celery Worker:      ${RED}STOPPED${NC}"
    fi

    if pgrep -f "celery -A app.celery_app beat" > /dev/null; then
        echo -e "  Celery Beat:        ${GREEN}RUNNING${NC}"
    else
        echo -e "  Celery Beat:        ${RED}STOPPED${NC}"
    fi

    if pgrep -f "vite" > /dev/null; then
        echo -e "  Frontend (Vite):    ${GREEN}RUNNING${NC}"
    else
        echo -e "  Frontend (Vite):    ${RED}STOPPED${NC}"
    fi
    echo ""
}

show_help() {
    echo "TRACE Control Script"
    echo ""
    echo "Usage:"
    echo "  ./start.sh [command]"
    echo ""
    echo "Commands:"
    echo "  start    - Start Docker infrastructure and local services
     (default)"
    echo "  stop     - Stop Docker containers and application processes"
    echo "  migrate  - Run Alembic database migrations"
    echo "  restart  - Restart all services (stop + start)"
    echo "  status   - Show status of Docker containers and app services"
    echo "  set-password <email> - Set a user's password (prompts for it)"
    echo "  help     - Show this help message"
    echo ""
}

COMMAND="${1:-start}"

case "$COMMAND" in
    start)
        do_start
        ;;
    stop)
        do_stop
        ;;
    migrate)
```

```
        do_migrate
        ;;
    restart)
        do_stop
        sleep 2
        do_start
        ;;
    status)
        do_status
        ;;
    set-password)
        [ -n "$2" ] || { echo -e "${RED}Usage: ./start.sh set-password
     <email>${NC}"; exit 1; }
        (cd "$BACKEND_DIR" && "$VENV_DIR/bin/python" -m app.cli set-password
     "$2")
        ;;
    help|-h|--help)
        show_help
        ;;
    *)
        echo -e "${RED}Unknown command: $COMMAND${NC}"
        echo ""
        show_help
        exit 1
        ;;
esac
```

4.1.3 Running and Deployment

`stop.sh`

```
#!/usr/bin/env bash
ROOT_DIR="$(cd "$(dirname "$0")" && pwd)"
exec "$ROOT_DIR/start.sh" stop "$@"
```

`migrate.sh`

```
#!/usr/bin/env bash
ROOT_DIR="$(cd "$(dirname "$0")" && pwd)"
exec "$ROOT_DIR/start.sh" migrate "$@"
```

`docker-compose.yml`

```
services:
  db:
    image: postgres:16-alpine
    container_name: trace_db
    restart: unless-stopped
    environment:
      POSTGRES_DB: ${POSTGRES_DB:-trace_db}
      POSTGRES_USER: ${POSTGRES_USER:-trace_user}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-trace_password}
    ports:
      - "${DB_PORT:-5433}:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER:-trace_user} -d $
     {POSTGRES_DB:-trace_db}"]
      interval: 10s
      timeout: 5s
      retries: 5

  redis:
    image: redis:7-alpine
    container_name: trace_redis
    restart: unless-stopped
    ports:
```

```
    - "${REDIS_PORT:-6379}:6379"
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
    interval: 10s
    timeout: 5s
    retries: 5

api:
  build:
    context: ./backend
    dockerfile: Dockerfile
  container_name: trace_api
  restart: unless-stopped
  command: uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
  ports:
    - "${API_PORT:-8000}:8000"
  volumes:
    - ./backend:/app
  env_file:
    - ./.env
  environment:
    DATABASE_URL: postgresql+asyncpg://${POSTGRES_USER:-trace_user}:$
   {POSTGRES_PASSWORD:-trace_password}@db:5432/${POSTGRES_DB:-trace_db}
    REDIS_URL: redis://redis:6379/0
  depends_on:
    db:
      condition: service_healthy
    redis:
      condition: service_healthy

celery_worker:
  build:
    context: ./backend
    dockerfile: Dockerfile
  container_name: trace_celery_worker
  restart: unless-stopped
  command: celery -A app.celery_app worker --loglevel=info
  volumes:
    - ./backend:/app
  env_file:
    - ./.env
  environment:
    DATABASE_URL: postgresql+asyncpg://${POSTGRES_USER:-trace_user}:$
   {POSTGRES_PASSWORD:-trace_password}@db:5432/${POSTGRES_DB:-trace_db}
    REDIS_URL: redis://redis:6379/0
  depends_on:
    db:
      condition: service_healthy
    redis:
      condition: service_healthy

celery_beat:
  build:
    context: ./backend
    dockerfile: Dockerfile
  container_name: trace_celery_beat
  restart: unless-stopped
  command: celery -A app.celery_app beat --loglevel=info
  volumes:
    - ./backend:/app
  env_file:
    - ./.env
  environment:
    DATABASE_URL: postgresql+asyncpg://${POSTGRES_USER:-trace_user}:$
   {POSTGRES_PASSWORD:-trace_password}@db:5432/${POSTGRES_DB:-trace_db}
    REDIS_URL: redis://redis:6379/0
  depends_on:
    db:
```

```
      condition: service_healthy
    redis:
      condition: service_healthy

frontend:
  build:
    context: ./frontend
    dockerfile: Dockerfile
  container_name: trace_frontend
  restart: unless-stopped
  command: npm run dev -- --host 0.0.0.0 --port 5173
  ports:
    - "${FRONTEND_PORT:-5173}:5173"
  volumes:
    - ./frontend:/app
    - /app/node_modules
  environment:
    VITE_API_TARGET: http://api:8000
  depends_on:
    - api

pgadmin:
  image: dpage/pgadmin4:latest
  container_name: trace_pgadmin
  restart: unless-stopped
  environment:
    PGADMIN_DEFAULT_EMAIL: ${PGADMIN_EMAIL:-admin@trace.com}
    PGADMIN_DEFAULT_PASSWORD: ${PGADMIN_PASSWORD:-admin123}
    PGADMIN_CONFIG_SERVER_MODE: "False"
    PGADMIN_CONFIG_MASTER_PASSWORD_REQUIRED: "False"
  ports:
    - "${PGADMIN_PORT:-5050}:80"
  volumes:
    - ./pgadmin/servers.json:/pgadmin4/servers.json:ro
  depends_on:
    db:
      condition: service_healthy

mailpit:
  image: axllent/mailpit:latest
  container_name: trace_mailpit
  restart: unless-stopped
  ports:
    - "${MAILPIT_SMTP_PORT:-1025}:1025"
    - "${MAILPIT_UI_PORT:-8025}:8025"
  volumes:
    - mailpit_data:/data
  environment:
    MP_DATABASE: /data/mailpit.db
    MP_SMTP_AUTH_ACCEPT_ANY: 1
    MP_SMTP_AUTH_ALLOW_INSECURE: 1

minio:
  image: minio/minio:latest
  container_name: trace_minio
  restart: unless-stopped
  command: server /data --console-address ":9001"
  ports:
    - "${MINIO_API_PORT:-9000}:9000"
    - "${MINIO_CONSOLE_PORT:-9001}:9001"
  volumes:
    - minio_data:/data
  environment:
    MINIO_ROOT_USER: ${MINIO_ROOT_USER:-trace_minio}
    MINIO_ROOT_PASSWORD: ${MINIO_ROOT_PASSWORD:-trace_minio_secret}

cloudflared:
  image: cloudflare/cloudflared:latest
```

```
    container_name: trace_cloudflared
    profiles: ["production"]
    restart: unless-stopped
    network_mode: host
    command: tunnel --no-autoupdate run
    environment:
      TUNNEL_TOKEN: ${CLOUDFLARE_TUNNEL_TOKEN:-}

volumes:
  postgres_data:
  mailpit_data:
  minio_data:
```

backend/Dockerfile

```
FROM python:3.12-slim AS base

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends \
    gcc \
    libpq-dev \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

backend/requirements.txt

```
fastapi==0.136.0
uvicorn[standard]==0.46.0
pydantic[email]==2.13.3
pydantic-settings==2.9.1

sqlalchemy[asyncio]==2.0.41
asyncpg==0.31.0
alembic==1.16.2

PyJWT==2.10.1
cryptography==50.0.1
bcrypt==4.3.0

celery[redis]==5.5.3
redis==5.2.1

python-dotenv==1.1.0
python-multipart==0.0.20
httpx==0.28.1

pytest==8.4.1
pytest-asyncio==1.0.0

sendgrid==6.11.0

aiosmtplib==3.0.2

minio==7.2.15
```

```
pypdf==6.1.1
python-docx==1.2.0

openai==1.82.0

websockets==15.0.1
```

frontend/Dockerfile

```
FROM node:22-alpine

WORKDIR /app

COPY package.json package-lock.json ./
RUN npm ci

COPY . .

EXPOSE 5173

CMD ["npm", "run", "dev", "--", "--host", "0.0.0.0", "--port", "5173"]
```

frontend/package.json

```
{
  "name": "frontend",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "oxlint",
    "preview": "vite preview"
  },
  "dependencies": {
    "@base-ui/react": "^1.7.0",
    "@dnd-kit/core": "^6.3.1",
    "@dnd-kit/modifiers": "^9.0.0",
    "@dnd-kit/sortable": "^10.0.0",
    "@dnd-kit/utilities": "^3.2.2",
    "@fontsource-variable/raleway": "^5.3.0",
    "@radix-ui/react-avatar": "^1.2.2",
    "@radix-ui/react-checkbox": "^1.3.7",
    "@radix-ui/react-dialog": "^1.1.19",
    "@radix-ui/react-dropdown-menu": "^2.1.20",
    "@radix-ui/react-label": "^2.1.11",
    "@radix-ui/react-popover": "^1.1.19",
    "@radix-ui/react-progress": "^1.1.12",
    "@radix-ui/react-scroll-area": "^1.2.14",
    "@radix-ui/react-select": "^2.3.3",
    "@radix-ui/react-separator": "^1.1.11",
    "@radix-ui/react-slot": "^1.3.0",
    "@radix-ui/react-tabs": "^1.1.17",
    "@radix-ui/react-tooltip": "^1.2.12",
    "@shadcn/react": "^0.3.1",
    "@tailwindcss/vite": "^4.3.3",
    "@tanstack/react-table": "^9.2.4",
    "@xyflow/react": "^12.12.0",
    "axios": "^1.18.1",
    "class-variance-authority": "^0.7.1",
    "clsx": "^2.1.1",
    "cmdk": "^1.1.1",
    "cn": "^0.2.6",
    "date-fns": "^4.4.0",
    "embla-carousel-react": "^8.6.0",
    "input-otp": "^1.5.0",
    "lucide-react": "^1.24.0",
```

```
    "next-themes": "^0.4.6",
    "react": "^19.2.7",
    "react-day-picker": "^10.0.1",
    "react-dom": "^19.2.7",
    "react-icons": "^5.7.0",
    "react-markdown": "^10.1.0",
    "react-resizable-panels": "^4.12.3",
    "react-router-dom": "^7.18.1",
    "recharts": "^3.8.0",
    "remark-gfm": "^4.0.1",
    "shadcn": "^4.19.1",
    "sonner": "^2.0.8",
    "tailwind-merge": "^3.6.0",
    "tailwindcss": "^4.3.3",
    "tw-animate-css": "^1.4.0",
    "zod": "^4.5.4"
  },
  "devDependencies": {
    "@types/react": "^19.2.17",
    "@types/react-dom": "^19.2.3",
    "@vitejs/plugin-react": "^6.0.3",
    "oxlint": "^1.71.0",
    "vite": "^8.1.1"
  }
}
```

frontend/vite.config.js

```
import { defineConfig, loadEnv } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'
import path from 'path'

export default defineConfig(({ mode }) => {
  const rootEnv = loadEnv(mode, path.resolve(__dirname, '..'), '')
  const domain = (rootEnv.PRODUCTION_URL || '').replace(/^https?:\/\//,
     '').replace(/\/+$/, '')
  const allowedHosts = domain ? [domain, `www.${domain}`] : []

  const apiProxy = {
    '/api': {
      target: process.env.VITE_API_TARGET || 'http://localhost:8000',
      changeOrigin: true,
      ws: true,
    },
  }

  return {
    plugins: [react(), tailwindcss()],
    resolve: {
      alias: {
        '@': path.resolve(__dirname, './src'),
      },
    },
    server: {
      port: 5173,
      host: true,
      proxy: apiProxy,
    },
    preview: {
      port: 5173,
      strictPort: true,
      host: '127.0.0.1',
      allowedHosts,
      proxy: apiProxy,
    },
  }
})
```

frontend/index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta name="description" content="TRACE — AI-powered workforce
     management with geo-fenced attendance, task tracking, and productivity
     scoring." />
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
    <link href="https://fonts.googleapis.com/css2?
     family=Inter:wght@300;400;500;600;700;800&display=swap" rel="stylesheet"
     />
    <title>TRACE — Workforce Management</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

4.1.4 Backend: Application and Core

backend/main.py

```
from fastapi import FastAPI
from schema import CategoryCreateSchema, UserCreateSchema

app = FastAPI()

@app.get("/")
async def index():
    return {"status": 200, "Text": "Welcome to my app"}

@app.get("/user")
async def get_user():
    return {"id": 1, "first_name": "Bipasha", "last_name": "Bagchi", "age":
     22}

@app.get("/products")
async def get_product():
    return {"id": 1, "name": "iphone 17 pro max", "brand": "apple",
     "colour": "Red"}

@app.get("/categories")
async def get_categories(q: str | None = None):
    return [
        {"id": 1, "name": "Phone"},
        {"id": 2, "name": "Laptop"},
    ]

@app.get("/categories/{id}")
async def get_category(id: int):
    return {"id": id, "name": "Phone"}

@app.post("/categories")
async def create_category(data: CategoryCreateSchema):
    return data

@app.post("/users")
async def create_user(data: UserCreateSchema):
    return data

@app.get("/users/{user_id}")
async def get_user_detail(user_id: int):
```

```
    return {
        "id": user_id,
        "name": "Bipasha",
        "email": "bipasha@example.com",
        "addresses": [
            {"id": 1, "street": "123 Main St", "city": "Metropolis",
     "user_id": user_id}
        ],
    }

from schema import CategoryCreateSchema, UserCreateSchema

app = FastAPI()

@app.get("/")
async def index():
    return {"status": 200, "Text": "Welcome to my app"}

@app.get("/user")
async def get_user():
    return {"id": 1, "first_name": "Bipasha", "last_name": "Bagchi", "age":
     22}

@app.get("/products")
async def get_product():
    return {"id": 1, "name": "iphone 17 pro max", "brand": "apple",
     "colour": "Red"}

@app.get("/categories")
async def get_categories(q: str | None = None):
    return [
        {"id": 1, "name": "Phone"},
        {"id": 2, "name": "Laptop"},
    ]

@app.get("/categories/{id}")
async def get_category(id: int):
    return {"id": 1, "name": "Phone"}

@app.post("/categories")
async def create_category(data: CategoryCreateSchema):
    return data

@app.post("/users")
async def create_user(data: UserCreateSchema):
    return data

@app.get("/users")
async def get_user_detail():
    return {
        "id": 1,
        "name": "Bipasha",
        "email": "bipasha@example.com",
        "addresses": [
            {"id": 1, "street": "123 Main St", "city": "Metropolis",
     "user_id": 1}
        ],
    }
```

backend/schema.py

```
from pydantic import BaseModel

class CategoryCreateSchema(BaseModel):
    id : int
    name : str
    description : str | None = None
```

```
class AddressSchema(BaseModel):
    id: int
    street: str
    city: str
    user_id: int

class UserCreateSchema(BaseModel):
    id: int
    name: str
    email: str | None = None
    addresses: list[AddressSchema] | None = None
```

backend/app/main.py

```
import logging

from contextlib import asynccontextmanager

from fastapi import FastAPI

from app.api.v1.router import router as v1_router
from app.config import get_settings
from app.middleware import setup_middleware

settings = get_settings()

logging.basicConfig(
    level=logging.DEBUG if settings.DEBUG else logging.INFO,
    format="%(asctime)s | %(levelname)-8s | %(name)s | %(message)s",
)
logger = logging.getLogger("trace")

@asynccontextmanager
async def lifespan(app: FastAPI):
    from app.database import async_session_factory
    from app.seeds import accounts_with_seed_passwords, run_seeds

    try:
        await run_seeds()
    except Exception as e:
        logger.warning(f"Seed script encountered an issue: {e}")

    if settings.is_production:
        async with async_session_factory() as db:
            weak = await accounts_with_seed_passwords(db)
        if weak:
            raise RuntimeError(
                "APP_ENV=production refused: these accounts still use their
     published seed "
                f"password: {', '.join(weak)}. Change each with `./start.sh
     set-password <email>`."
            )

    yield

    logger.info("TRACE shutting down.")

def create_app() -> FastAPI:
    app = FastAPI(
        title=settings.APP_NAME,
        version=settings.APP_VERSION,
        description=(
            "TRACE — Task and Resource Attendance and Comprehensive
     Evaluation System. "
            "AI-powered workforce management with geo-fenced attendance, "
            "task tracking, and productivity scoring."
        ),
        docs_url="/docs",
```

```
        redoc_url="/redoc",
        lifespan=lifespan,
    )

    setup_middleware(app)

    app.include_router(v1_router)

    @app.get("/health", tags=["System"])
    async def health_check():
        return {
            "status": "healthy",
            "app": settings.APP_NAME,
            "version": settings.APP_VERSION,
        }

    return app

app = create_app()
```

backend/app/config.py

```
from functools import lru_cache
from pathlib import Path
from typing import Literal

from pydantic import model_validator
from pydantic_settings import BaseSettings

BACKEND_DIR = Path(__file__).resolve().parents[1]
ROOT_DIR = BACKEND_DIR.parent

INSECURE_JWT_SECRETS = {
    "change-me-in-production",
    "your-super-secret-key-change-in-production",
}

def valid_fernet_key(key: str) -> bool:
    from cryptography.fernet import Fernet

    try:
        Fernet(key.encode())
    except (ValueError, TypeError):
        return False
    return True

class Settings(BaseSettings):
    POSTGRES_USER: str = "trace_user"
    POSTGRES_PASSWORD: str = "trace_password"
    POSTGRES_DB: str = "trace_db"
    DB_HOST: str = "localhost"
    DB_PORT: int = 5433
    REDIS_PORT: int = 6379
    MINIO_ROOT_USER: str = "trace_minio"
    MINIO_ROOT_PASSWORD: str = "trace_minio_secret"
    MINIO_API_PORT: int = 9000
    MAILPIT_SMTP_PORT: int = 1025

    APP_NAME: str = "TRACE"
    APP_VERSION: str = "1.0.0"
    DEBUG: bool = False

    APP_ENV: Literal["local", "production"] = "local"
    PRODUCTION_URL: str = ""

    DATABASE_URL: str = ""
    DATABASE_ECHO: bool = False
```

```
JWT_SECRET_KEY: str = "change-me-in-production"
JWT_ALGORITHM: str = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES: int = 480
REFRESH_TOKEN_EXPIRE_DAYS: int = 7

CORS_ORIGINS: list[str] = ["http://localhost:3000",
 "http://localhost:5173"]

REDIS_URL: str = ""

SENDGRID_API_KEY: str = ""
ADMIN_EMAIL: str = "admin@trace.com"

MINIO_ENDPOINT: str = ""
MINIO_ACCESS_KEY: str = ""
MINIO_SECRET_KEY: str = ""
MINIO_BUCKET: str = "trace-uploads"
MINIO_SECURE: bool = False

SMTP_HOST: str = "localhost"
SMTP_PORT: int | None = None
SMTP_FROM: str = "noreply@trace.com"

SETTINGS_ENCRYPTION_KEY: str = ""

GITHUB_OWNER: str = ""
GITHUB_TOKEN: str = ""
GITHUB_API_URL: str = "https://api.github.com"
GITHUB_COMMITTER_NAME: str = ""
GITHUB_COMMITTER_EMAIL: str = ""

OPENAI_API_KEY: str = ""
OPENAI_BASE_URL: str = ""
OPENAI_MODEL: str = "gpt-4o-mini"

@model_validator(mode="after")
def _derive_connection_settings(self) -> "Settings":
    if not self.DATABASE_URL:
        self.DATABASE_URL = (
            f"postgresql+asyncpg://{self.POSTGRES_USER}:
 {self.POSTGRES_PASSWORD}"
            f"@{self.DB_HOST}:{self.DB_PORT}/{self.POSTGRES_DB}"
        )
    if not self.REDIS_URL:
        self.REDIS_URL = f"redis://localhost:{self.REDIS_PORT}/0"
    if not self.MINIO_ENDPOINT:
        self.MINIO_ENDPOINT = f"localhost:{self.MINIO_API_PORT}"
    self.MINIO_ACCESS_KEY = self.MINIO_ACCESS_KEY or
 self.MINIO_ROOT_USER
    self.MINIO_SECRET_KEY = self.MINIO_SECRET_KEY or
 self.MINIO_ROOT_PASSWORD
    if self.SMTP_PORT is None:
        self.SMTP_PORT = self.MAILPIT_SMTP_PORT
    return self

@property
def is_production(self) -> bool:
    return self.APP_ENV == "production"

@property
def production_domain(self) -> str:
    return
 self.PRODUCTION_URL.strip().removeprefix("https://").removeprefix("http:
 //").rstrip("/")

@property
def app_url(self) -> str:
    if self.is_production and self.production_domain:
```

```
            return f"https://{self.production_domain}"
        return "http://localhost:5173"

    @property
    def cors_origins(self) -> list[str]:
        if self.is_production:
            domain = self.production_domain
            return [f"https://{domain}", f"https://www.{domain}"]
        return self.CORS_ORIGINS

    def production_problems(self) -> list[str]:
        if not self.is_production:
            return []
        problems = []
        if not self.production_domain:
            problems.append("PRODUCTION_URL is empty (.env)")
        if self.DEBUG:
            problems.append("DEBUG must be false — it leaks exception
     details to clients (.env)")
        if self.JWT_SECRET_KEY in INSECURE_JWT_SECRETS or
     len(self.JWT_SECRET_KEY) < 32:
            problems.append(
                "JWT_SECRET_KEY is a placeholder or shorter than 32
     characters (.env) — "
                'generate one with: python -c "import secrets;
     print(secrets.token_urlsafe(48))"'
            )
        if not self.SETTINGS_ENCRYPTION_KEY:
            problems.append(
                "SETTINGS_ENCRYPTION_KEY is empty (.env) — generate one
     with: python -c "
                '"from cryptography.fernet import Fernet;
     print(Fernet.generate_key().decode())"'
            )
        elif not valid_fernet_key(self.SETTINGS_ENCRYPTION_KEY):
            problems.append("SETTINGS_ENCRYPTION_KEY is not a valid Fernet
     key (.env)")
        return problems

    model_config = {
        "env_file": ROOT_DIR / ".env",
        "env_file_encoding": "utf-8",
        "case_sensitive": True,
        "extra": "ignore",
    }

@lru_cache()
def get_settings() -> Settings:
    settings = Settings()
    if problems := settings.production_problems():
        raise RuntimeError("APP_ENV=production refused:\n  - " + "\n  -
     ".join(problems))
    return settings
```

backend/app/database.py

```
from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker,
     create_async_engine
from sqlalchemy.orm import DeclarativeBase

from app.config import get_settings

settings = get_settings()

engine = create_async_engine(
    settings.DATABASE_URL,
    echo=settings.DATABASE_ECHO,
    pool_pre_ping=True,
```

```
    pool_size=10,
    max_overflow=20,
)

async_session_factory = async_sessionmaker(
    bind=engine,
    class_=AsyncSession,
    expire_on_commit=False,
)

class Base(DeclarativeBase):
    pass

async def get_db() -> AsyncSession:
    async with async_session_factory() as session:
        try:
            yield session
            await session.commit()
        except Exception:
            await session.rollback()
            raise
        finally:
            await session.close()
```

backend/app/middleware.py

```
import time
import logging
import traceback

from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse

from app.config import get_settings

settings = get_settings()
logger = logging.getLogger("trace")

def setup_middleware(app: FastAPI) -> None:
    app.add_middleware(
        CORSMiddleware,
        allow_origins=settings.cors_origins,
        allow_credentials=True,
        allow_methods=["*"],
        allow_headers=["*"],
    )

    @app.exception_handler(Exception)
    async def global_exception_handler(request: Request, exc: Exception):
        logger.error(
            f"Unhandled exception on {request.method} {request.url.path}: "
            f"{exc.__class__.__name__}: {exc}",
            exc_info=True,
        )

        detail = str(exc) if settings.DEBUG else "An internal error
     occurred."

        return JSONResponse(
            status_code=500,
            content={
                "error": True,
                "type": exc.__class__.__name__,
                "detail": detail,
                "path": request.url.path,
            },
        )
```

```
    from fastapi.exceptions import RequestValidationError

    @app.exception_handler(RequestValidationError)
    async def validation_exception_handler(
        request: Request, exc: RequestValidationError
    ):
        errors = []
        for err in exc.errors():
            field = " → ".join(str(loc) for loc in err.get("loc", []))
            errors.append({
                "field": field,
                "message": err.get("msg", ""),
                "type": err.get("type", ""),
            })
        return JSONResponse(
            status_code=422,
            content={
                "error": True,
                "detail": "Validation failed",
                "errors": errors,
            },
        )

    @app.middleware("http")
    async def log_requests(request: Request, call_next):
        start_time = time.time()
        response = await call_next(request)
        process_time = time.time() - start_time

        logger.info(
            f"{request.method} {request.url.path} "
            f"→ {response.status_code} "
            f"({process_time:.3f}s)"
        )

        response.headers["X-Process-Time"] = f"{process_time:.3f}"
        response.headers["X-Request-ID"] = f"{id(request):x}"
        return response
```

backend/app/exceptions.py

```
from fastapi import HTTPException, status

class TraceException(HTTPException):
    def __init__(self, status_code: int, detail: str):
        super().__init__(status_code=status_code, detail=detail)

class InvalidCredentialsException(TraceException):
    def __init__(self):
        super().__init__(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid email or password.",
        )

class TokenExpiredException(TraceException):
    def __init__(self):
        super().__init__(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Token has expired.",
        )

class InvalidTokenException(TraceException):
    def __init__(self):
        super().__init__(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid or malformed token.",
        )
```

```
class InsufficientPermissionsException(TraceException):
    def __init__(self):
        super().__init__(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="You do not have permission to perform this action.",
        )

class NotFoundException(TraceException):
    def __init__(self, resource: str = "Resource"):
        super().__init__(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"{resource} not found.",
        )

class AlreadyExistsException(TraceException):
    def __init__(self, resource: str = "Resource"):
        super().__init__(
            status_code=status.HTTP_409_CONFLICT,
            detail=f"{resource} already exists.",
        )
```

backend/app/dependencies.py

```
from fastapi import Depends, Header
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy.orm import selectinload

import jwt as pyjwt

from app.config import get_settings
from app.database import get_db
from app.exceptions import InvalidTokenException, NotFoundException,
     TokenExpiredException
from app.models.user import User, UserRole

settings = get_settings()

async def get_current_user(
    authorization: str = Header(..., description="Bearer <token>"),
    db: AsyncSession = Depends(get_db),
) -> User:
    if not authorization.startswith("Bearer "):
        raise InvalidTokenException()

    token = authorization.removeprefix("Bearer ").strip()

    try:
        payload = pyjwt.decode(
            token,
            settings.JWT_SECRET_KEY,
            algorithms=[settings.JWT_ALGORITHM],
        )
    except pyjwt.ExpiredSignatureError:
        raise TokenExpiredException()
    except pyjwt.InvalidTokenError:
        raise InvalidTokenException()

    user_id_raw = payload.get("sub")
    if user_id_raw is None:
        raise InvalidTokenException()

    try:
        user_id = int(user_id_raw)
    except (ValueError, TypeError):
        raise InvalidTokenException()
```

```
    stmt = (
        select(User)
        .where(User.id == user_id, User.deleted_at.is_(None))
        .options(selectinload(User.user_roles).selectinload(UserRole.role))
    )
    result = await db.execute(stmt)
    user = result.scalar_one_or_none()

    if user is None:
        raise NotFoundException("User")

    if user.status == "Blocked":
        raise InvalidTokenException()

    return user

async def get_optional_current_user(
    authorization: str | None = Header(None, description="Bearer <token>"),
    db: AsyncSession = Depends(get_db),
) -> User | None:
    if authorization is None:
        return None
    return await get_current_user(authorization=authorization, db=db)
```

backend/app/core/security.py

```
from datetime import datetime, timedelta, timezone

import bcrypt
import jwt

from app.config import get_settings

settings = get_settings()

def hash_password(password: str) -> str:
    salt = bcrypt.gensalt(rounds=12)
    hashed = bcrypt.hashpw(password.encode("utf-8"), salt)
    return hashed.decode("utf-8")

def verify_password(plain_password: str, hashed_password: str) -> bool:
    return bcrypt.checkpw(
        plain_password.encode("utf-8"),
        hashed_password.encode("utf-8"),
    )

def create_access_token(data: dict, expires_delta: timedelta | None = None)
     -> str:
    to_encode = data.copy()
    if "sub" in to_encode:
        to_encode["sub"] = str(to_encode["sub"])
    expire = datetime.now(timezone.utc) + (
        expires_delta or
     timedelta(minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES)
    )
    to_encode.update({"exp": expire, "iat": datetime.now(timezone.utc),
     "type": "access"})
    return jwt.encode(to_encode, settings.JWT_SECRET_KEY,
     algorithm=settings.JWT_ALGORITHM)

def create_refresh_token(data: dict) -> str:
    to_encode = data.copy()
    if "sub" in to_encode:
        to_encode["sub"] = str(to_encode["sub"])
    expire = datetime.now(timezone.utc) +
     timedelta(days=settings.REFRESH_TOKEN_EXPIRE_DAYS)
    to_encode.update({"exp": expire, "iat": datetime.now(timezone.utc),
     "type": "refresh"})
```

```
    return jwt.encode(to_encode, settings.JWT_SECRET_KEY,
     algorithm=settings.JWT_ALGORITHM)

def decode_token(token: str) -> dict:
    payload = jwt.decode(
        token,
        settings.JWT_SECRET_KEY,
        algorithms=[settings.JWT_ALGORITHM],
    )
    if "sub" in payload and payload["sub"] is not None:
        payload["sub"] = int(payload["sub"])
    return payload
```

backend/app/core/permissions.py

```
from enum import Enum

from fastapi import Depends

from app.dependencies import get_current_user
from app.exceptions import InsufficientPermissionsException
from app.models.user import User

class RoleEnum(str, Enum):
    ADMIN = "Admin"
    MANAGER = "Manager"
    EMPLOYEE = "Employee"

def require_roles(*allowed_roles: RoleEnum):
    async def role_checker(
        current_user: User = Depends(get_current_user),
    ) -> User:
        user_roles = set()
        if current_user.user_roles:
            for ur in current_user.user_roles:
                if ur.role:
                    user_roles.add(ur.role.role_name)

        allowed = {r.value for r in allowed_roles}
        if not user_roles.intersection(allowed):
            raise InsufficientPermissionsException()
        return current_user

    return role_checker

require_admin = require_roles(RoleEnum.ADMIN)
require_manager_or_admin = require_roles(RoleEnum.ADMIN, RoleEnum.MANAGER)
require_any_role = require_roles(RoleEnum.ADMIN, RoleEnum.MANAGER,
     RoleEnum.EMPLOYEE)
```

backend/app/core/project_access.py

```
from dataclasses import dataclass

from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from app.exceptions import InsufficientPermissionsException,
     NotFoundException, TraceException
from app.models.project import Project, ProjectAssignment
from app.models.task import Task
from app.models.user import User

PROJECT_MANAGER = "Project Manager"
MEMBER = "Member"

ASSIGNEE_SETTABLE_STATUSES = {"Pending", "In Progress", "In Review",
     "Blocked", "Completed"}
```

```
SYSTEM_ONLY_STATUSES = {"Overdue"}
TASK_STATUSES = {"Pending", "In Progress", "In Review", "Completed",
     "Blocked", "Overdue", "Cancelled"}

def platform_roles(user: User) -> set[str]:
    return {ur.role.role_name for ur in (user.user_roles or []) if ur.role}

@dataclass
class ProjectAccess:
    project: Project
    user: User
    platform_role: str
    project_role: str | None

    @property
    def is_owner(self) -> bool:
        return self.platform_role == "Manager" and
     self.project.created_by_user_id == self.user.id

    @property
    def is_project_manager(self) -> bool:
        return self.project_role == PROJECT_MANAGER

    def require_owner(self) -> None:
        if not self.is_owner:
            raise InsufficientPermissionsException()

    def require_project_manager(self) -> None:
        if not self.is_project_manager:
            raise InsufficientPermissionsException()

    def is_assignee(self, task: Task) -> bool:
        return self.platform_role == "Employee" and task.assigned_to_user_id
     == self.user.id

    def may_discuss(self, task: Task) -> bool:
        return (self.is_owner or self.is_project_manager
                or (self.platform_role == "Employee"
                    and self.user.id in (task.assigned_to_user_id,
     task.assigned_by_user_id)))

    def may_set_status(self, task: Task, new_status: str) -> None:
        if new_status in SYSTEM_ONLY_STATUSES:
            raise TraceException(403, f"'{new_status}' is set by the system,
     not by hand")
        if self.is_project_manager:
            return
        if self.is_assignee(task):
            if new_status in ASSIGNEE_SETTABLE_STATUSES:
                return
            raise TraceException(403, f"Only the Project Manager can set
     '{new_status}'")
        raise InsufficientPermissionsException()

    def allowed_statuses(self, task: Task) -> list[str]:
        allowed = []
        for status in sorted(TASK_STATUSES):
            if status == task.status:
                continue
            try:
                self.may_set_status(task, status)
            except TraceException:
                continue
            allowed.append(status)
        return allowed

async def resolve_project_access(db: AsyncSession, user: User, project_id:
     int) -> ProjectAccess:
```

```
    project = (await db.execute(
        select(Project).where(Project.id == project_id,
     Project.deleted_at.is_(None))
    )).scalar_one_or_none()
    if project is None:
        raise NotFoundException("Project")

    roles = platform_roles(user)
    if "Admin" in roles:
        return ProjectAccess(project, user, "Admin", None)
    if "Manager" in roles:
        if project.created_by_user_id != user.id:
            raise InsufficientPermissionsException()
        return ProjectAccess(project, user, "Manager", None)

    assignment = (await db.execute(
        select(ProjectAssignment).where(
            ProjectAssignment.project_id == project_id,
            ProjectAssignment.user_id == user.id,
            ProjectAssignment.deleted_at.is_(None),
        )
    )).scalar_one_or_none()
    if assignment is None:
        raise InsufficientPermissionsException()
    return ProjectAccess(project, user, "Employee", assignment.project_role)

async def resolve_task_access(db: AsyncSession, user: User, task_id: int) ->
     tuple[Task, ProjectAccess]:
    task = (await db.execute(
        select(Task).where(Task.id == task_id, Task.deleted_at.is_(None))
    )).scalar_one_or_none()
    if task is None or task.project_id is None:
        raise NotFoundException("Task")
    return task, await resolve_project_access(db, user, task.project_id)
```

backend/app/utils/helpers.py

```
import uuid
from datetime import date

def generate_employee_code(prefix: str = "EMP") -> str:
    short_uuid = uuid.uuid4().hex[:6].upper()
    return f"{prefix}-{short_uuid}"

def get_current_week_start() -> date:
    today = date.today()
    return today - __import__("datetime").timedelta(days=today.weekday())
```

backend/app/cli.py

```
import argparse
import asyncio
import getpass
import os
import sys

from sqlalchemy import select

from app.core.security import hash_password
from app.database import async_session_factory
from app.models.user import User

MIN_PASSWORD_LENGTH = 12

def _read_password() -> str:
    if env_password := os.environ.get("TRACE_NEW_PASSWORD"):
        return env_password
    if not sys.stdin.isatty():
```

```
        sys.exit("No TTY: pass the new password in TRACE_NEW_PASSWORD.")
    first = getpass.getpass("New password: ")
    if first != getpass.getpass("Repeat it:    "):
        sys.exit("Passwords do not match.")
    return first

async def set_password(email: str, password: str) -> None:
    if len(password) < MIN_PASSWORD_LENGTH:
        sys.exit(f"Password must be at least {MIN_PASSWORD_LENGTH}
     characters.")
    async with async_session_factory() as db:
        user = (await db.execute(
            select(User).where(User.email == email,
     User.deleted_at.is_(None))
        )).scalar_one_or_none()
        if user is None:
            sys.exit(f"No active user with email {email}.")
        user.password = hash_password(password)
        await db.commit()
    print(f"Password updated for {email}.")

def main() -> None:
    parser = argparse.ArgumentParser(prog="python -m app.cli")
    commands = parser.add_subparsers(dest="command", required=True)
    set_pw = commands.add_parser("set-password", help="Set a user's
     password")
    set_pw.add_argument("email")
    args = parser.parse_args()

    if args.command == "set-password":
        asyncio.run(set_password(args.email, _read_password()))

if __name__ == "__main__":
    main()
```

backend/app/seeds.py

```
import asyncio
import logging
from datetime import date, time as time_type

from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from app.config import get_settings
from app.database import async_session_factory
from app.models.attendance import TimeZone
from app.models.employee import Department, Employee, WorkSchedule,
     Designation
from app.models.task import TaskType
from app.models.user import Profile, Role, User, UserRole

logger = logging.getLogger("trace.seeds")

ROLES = ["Admin", "Manager", "Employee"]

DEPARTMENTS = [
    {"name": "Design", "description": "UI/UX and graphic design team"},
    {"name": "Development", "description": "Software development and
     engineering"},
    {"name": "QA", "description": "Quality assurance and testing"},
    {"name": "DevOps", "description": "Infrastructure and deployment
     operations"},
    {"name": "HR", "description": "Human resources and people operations"},
]

DESIGNATIONS = [
```

```
    {"name": "Software Engineer", "description": "Full-stack application
     developer"},
    {"name": "Senior Developer", "description": "Senior software engineer"},
    {"name": "Team Lead", "description": "Engineering team lead"},
    {"name": "Product Manager", "description": "Product strategy and task
     manager"},
    {"name": "QA Specialist", "description": "Quality assurance engineer"},
    {"name": "HR Specialist", "description": "People operations and HR
     manager"},
]

TASK_TYPES = [
    {"name": "Bug Fix", "description": "Fixing reported bugs and issues"},
    {"name": "Feature Development", "description": "Building new features"},
    {"name": "Enhancement", "description": "Improving existing
     functionality"},
    {"name": "Research", "description": "Research and investigation tasks"},
    {"name": "Documentation", "description": "Writing or updating
     documentation"},
]

WORK_SCHEDULES = [
    {"schedule_name": "Standard (9-6)", "start_time": "09:00", "end_time":
     "18:00", "break_minutes": 60},
    {"schedule_name": "Early (7-4)", "start_time": "07:00", "end_time":
     "16:00", "break_minutes": 60},
    {"schedule_name": "Late (11-8)", "start_time": "11:00", "end_time":
     "20:00", "break_minutes": 60},
    {"schedule_name": "Night (22-7)", "start_time": "22:00", "end_time":
     "07:00", "break_minutes": 60},
    {"schedule_name": "Flexi (10-7)", "start_time": "10:00", "end_time":
     "19:00", "break_minutes": 60},
]

TIME_ZONES = [
    {"timezone_name": "Asia/Kolkata", "utc_offset": "+05:30"},
    {"timezone_name": "America/New_York", "utc_offset": "-05:00"},
    {"timezone_name": "America/Los_Angeles", "utc_offset": "-08:00"},
    {"timezone_name": "Europe/London", "utc_offset": "+00:00"},
    {"timezone_name": "Asia/Tokyo", "utc_offset": "+09:00"},
    {"timezone_name": "Australia/Sydney", "utc_offset": "+11:00"},
]

SEED_USERS = [
    {
        "name": "Admin User",
        "username": "admin",
        "email": "admin@trace.com",
        "password": "Admin@123",
        "phone": "+91-9000000001",
        "gender": "Other",
        "role": "Admin",
        "employee_code": "TRACE-ADM-001",
    },
    {
        "name": "Manager User",
        "username": "manager",
        "email": "manager@trace.com",
        "password": "Manager@123",
        "phone": "+91-9000000002",
        "gender": "Male",
        "role": "Manager",
        "employee_code": "TRACE-MGR-001",
    },
    {
        "name": "Alice Dev",
        "username": "alice",
        "email": "alice@trace.com",
```

```
        "password": "Alice@123",
        "phone": "+91-9000000003",
        "gender": "Female",
        "role": "Employee",
        "employee_code": "TRACE-EMP-001",
    },
    {
        "name": "Bob QA",
        "username": "bob",
        "email": "bob@trace.com",
        "password": "Bob@123",
        "phone": "+91-9000000004",
        "gender": "Male",
        "role": "Employee",
        "employee_code": "TRACE-EMP-002",
    },
    {
        "name": "Carol Ops",
        "username": "carol",
        "email": "carol@trace.com",
        "password": "Carol@123",
        "phone": "+91-9000000005",
        "gender": "Female",
        "role": "Employee",
        "employee_code": "TRACE-EMP-003",
    },
]

async def seed_roles(db: AsyncSession) -> None:
    for role_name in ROLES:
        existing = await db.execute(
            select(Role).where(Role.role_name == role_name)
        )
        if existing.scalar_one_or_none() is None:
            db.add(Role(role_name=role_name))
            print(f"  ✅ Role: {role_name}")
        else:
            print(f"  ⏭️  Role: {role_name} (exists)")

async def seed_departments(db: AsyncSession) -> None:
    for dept in DEPARTMENTS:
        existing = await db.execute(
            select(Department).where(Department.name == dept["name"])
        )
        if existing.scalar_one_or_none() is None:
            db.add(Department(**dept))
            print(f"  ✅ Department: {dept['name']}")
        else:
            print(f"  ⏭️  Department: {dept['name']} (exists)")

async def seed_designations(db: AsyncSession) -> None:
    for desig in DESIGNATIONS:
        existing = await db.execute(
            select(Designation).where(Designation.name == desig["name"])
        )
        if existing.scalar_one_or_none() is None:
            db.add(Designation(**desig))
            print(f"  ✅ Designation: {desig['name']}")
        else:
            print(f"  ⏭️  Designation: {desig['name']} (exists)")

async def seed_task_types(db: AsyncSession) -> None:
    for tt in TASK_TYPES:
        existing = await db.execute(
            select(TaskType).where(TaskType.name == tt["name"])
        )
        if existing.scalar_one_or_none() is None:
            db.add(TaskType(**tt))
```

```
            print(f"  ✅ Task Type: {tt['name']}")
        else:
            print(f"  ⏭️  Task Type: {tt['name']} (exists)")

async def seed_work_schedules(db: AsyncSession) -> None:
    for ws in WORK_SCHEDULES:
        existing = await db.execute(
            select(WorkSchedule).where(
                WorkSchedule.schedule_name == ws["schedule_name"]
            )
        )
        if existing.scalar_one_or_none() is None:
            h1, m1 = map(int, ws["start_time"].split(":"))
            h2, m2 = map(int, ws["end_time"].split(":"))
            db.add(WorkSchedule(
                schedule_name=ws["schedule_name"],
                start_time=time_type(h1, m1),
                end_time=time_type(h2, m2),
                break_minutes=ws["break_minutes"],
            ))
            print(f"  ✅ Work Schedule: {ws['schedule_name']}")
        else:
            print(f"  ⏭️  Work Schedule: {ws['schedule_name']} (exists)")

async def seed_time_zones(db: AsyncSession) -> None:
    for tz in TIME_ZONES:
        existing = await db.execute(
            select(TimeZone).where(TimeZone.timezone_name ==
     tz["timezone_name"])
        )
        if existing.scalar_one_or_none() is None:
            db.add(TimeZone(**tz))
            print(f"  ✅ TimeZone: {tz['timezone_name']}")
        else:
            print(f"  ⏭️  TimeZone: {tz['timezone_name']} (exists)")

async def seed_users(db: AsyncSession) -> None:
    from app.core.security import hash_password

    dept_result = await db.execute(
        select(Department).where(Department.name == "Development")
    )
    dept = dept_result.scalar_one_or_none()

    ws_result = await db.execute(
        select(WorkSchedule).where(WorkSchedule.schedule_name == "Standard
     (9-6)")
    )
    work_schedule = ws_result.scalar_one_or_none()

    for u_data in SEED_USERS:
        existing = await db.execute(
            select(User).where(User.email == u_data["email"])
        )
        if existing.scalar_one_or_none() is not None:
            print(f"  ⏭️  User: {u_data['email']} (exists)")
            continue

        role_result = await db.execute(
            select(Role).where(Role.role_name == u_data["role"])
        )
        role = role_result.scalar_one_or_none()
        if role is None:
            print(f"  ❌ Role '{u_data['role']}' not found — skipping
     {u_data['email']}")
            continue

        profile = Profile()
```

```
        db.add(profile)
        await db.flush()

        user = User(
            name=u_data["name"],
            username=u_data["username"],
            email=u_data["email"],
            password=hash_password(u_data["password"]),
            phone=u_data["phone"],
            gender=u_data["gender"],
            profile_id=profile.id,
            is_verified=True,
            status="Active",
        )
        db.add(user)
        await db.flush()

        db.add(UserRole(user_id=user.id, role_id=role.id))
        await db.flush()

        db.add(Employee(
            user_id=user.id,
            employee_code=u_data["employee_code"],
            department_id=dept.id if dept else None,
            work_schedule_id=work_schedule.id if work_schedule else None,
            joining_date=date.today(),
            status="Active",
        ))
        await db.flush()

        print(f"  ✅ User: {u_data['email']} ({u_data['role']})
     [{u_data['employee_code']}]")

async def seed_sample_hrms(db: AsyncSession) -> None:
    from app.models.hrms import Holiday

    h_exist = await db.execute(select(Holiday).where(Holiday.name == "New
     Year's Day"))
    if not h_exist.scalar_one_or_none():
        db.add(Holiday(name="New Year's Day", holiday_date=date(2026, 1, 1),
     description="National holiday", type="Public"))
        db.add(Holiday(name="Independence Day", holiday_date=date(2026, 8,
     15), description="National holiday", type="Public"))
        print("  ✅ Holidays seeded")
    else:
        print("  ⏭️  Holidays (exist)")

async def seed_demo_project(db: AsyncSession) -> None:
    from app.models.project import Project, ProjectAssignment, TaskGroup
    from app.models.task import Task

    existing_count = (await db.execute(
        select(Project).limit(1)
    )).scalar_one_or_none()
    if existing_count is not None:
        print("  ⏭️  Demo project (exists)")
        return

    manager_result = await db.execute(
        select(User).where(User.email == "manager@trace.com")
    )
    manager = manager_result.scalar_one_or_none()

    alice_result = await db.execute(
        select(User).where(User.email == "alice@trace.com")
    )
    alice = alice_result.scalar_one_or_none()
```

```
if manager is None:
    print("  ❌ Manager user not found — skipping demo project")
    return

project = Project(
    name="TRACE Platform MVP",
    description="Build the core TRACE project management platform with
 AI-driven task planning.",
    created_by_user_id=manager.id,
    github_repo=f"{get_settings().GITHUB_OWNER or 'org'}/trace-
 platform",
    github_default_branch="main",
    status="Active",
    start_date=date(2026, 9, 1),
    end_date=date(2026, 12, 31),
)
db.add(project)
await db.flush()

if alice:
    db.add(ProjectAssignment(
        project_id=project.id,
        user_id=alice.id,
        project_role="Project Manager",
        assigned_by_user_id=manager.id,
    ))

groups = [
    ("Backend Foundation", "Database schema, API routes,
 authentication", 1, date(2026, 9, 1), date(2026, 10, 15)),
    ("Frontend Shell", "Panel layouts, routing, landing page", 2,
 date(2026, 9, 15), date(2026, 10, 31)),
    ("AI Pipeline", "Evaluate project, Gantt generation, task docs", 3,
 date(2026, 11, 1), date(2026, 12, 20)),
]

group_ids = []
for name, desc, seq, start, end in groups:
    group = TaskGroup(
        project_id=project.id,
        name=name,
        description=desc,
        sequence=seq,
        planned_start_date=start,
        planned_end_date=end,
        estimated_duration_days=(end - start).days + 1,
        status="Pending",
        created_by="Human",
    )
    db.add(group)
    await db.flush()
    group_ids.append(group.id)

    task = Task(
        title=f"{name}: initial setup",
        description=f"First task for the '{name}' group.",
        project_id=project.id,
        task_group_id=group.id,
        status="Pending",
        priority="High",
    )
    db.add(task)

await db.flush()
from app.services import gantt_service
backend, frontend, ai = group_ids
(await db.get(TaskGroup, ai)).depends_on = [backend, frontend]
await gantt_service.snapshot(db, project, "initial")
```

```
    print(f"  ✅ Demo project: '{project.name}' (id={project.id})")

async def run_seeds() -> None:
    print("\n🌱 TRACE Database Seeding\n" + "=" * 40)

    async with async_session_factory() as db:
        try:
            print("\n📋 Roles:")
            await seed_roles(db)

            print("\n🏢 Departments:")
            await seed_departments(db)

            print("\n💼 Designations:")
            await seed_designations(db)

            print("\n📝 Task Types:")
            await seed_task_types(db)

            print("\n⏰ Work Schedules:")
            await seed_work_schedules(db)

            print("\n🌍 Time Zones:")
            await seed_time_zones(db)

            print("\n🌿 HRMS Reference Data:")
            await seed_sample_hrms(db)

            demo_data = not get_settings().is_production
            if demo_data:
                print("\n👥 Users (Admin · Manager · Employees):")
                await seed_users(db)

                print("\n📁 Demo Project:")
                await seed_demo_project(db)

            await db.commit()
            print("\n" + "=" * 40)
            print("✅ All seeds completed successfully!\n")
            if not demo_data:
                print("Production: demo users and demo project were not
     seeded.")
                return
            print("Default credentials:")
            print("  Admin:   admin@trace.com   / Admin@123")
            print("  Manager: manager@trace.com / Manager@123")
            print("  Employee: alice@trace.com  / Alice@123")
        except Exception as e:
            await db.rollback()
            print(f"\n❌ Seeding failed: {e}")
            raise

async def accounts_with_seed_passwords(db: AsyncSession) -> list[str]:
    from app.core.security import verify_password

    weak = []
    for u_data in SEED_USERS:
        user = (await db.execute(
            select(User).where(User.email == u_data["email"],
     User.deleted_at.is_(None))
        )).scalar_one_or_none()
        if user is not None and verify_password(u_data["password"],
     user.password):
            weak.append(u_data["email"])
    return weak

if __name__ == "__main__":
    asyncio.run(run_seeds())
```

4.1.5 Backend: Models

backend/app/models/__init__.py

```
from app.models.base import BaseModelMixin, SoftDeleteMixin, TimestampMixin
from app.models.user import Profile, Role, User, UserRole
from app.models.employee import Department, Designation, Employee,
     EmployeeDetail, WorkSchedule
from app.models.attendance import Attendance, TimeZone
from app.models.hrms import Leave, Holiday
from app.models.task import Task, TaskComment, TaskDetail, TaskTimeLog,
     TaskType
from app.models.productivity import ProductivityScore
from app.models.project import (
    Project,
    ProjectSourceFile,
    ProjectAssignment,
    TaskGroup,
    GanttChart,
    TaskDiscussion,
    Notification,
    SystemSetting,
)

__all__ = [
    "BaseModelMixin",
    "TimestampMixin",
    "SoftDeleteMixin",
    "User",
    "Profile",
    "Role",
    "UserRole",
    "Employee",
    "EmployeeDetail",
    "Department",
    "Designation",
    "WorkSchedule",
    "Attendance",
    "TimeZone",
    "Leave",
    "Holiday",
    "Task",
    "TaskType",
    "TaskDetail",
    "TaskComment",
    "TaskTimeLog",
    "ProductivityScore",
    "Project",
    "ProjectSourceFile",
    "ProjectAssignment",
    "TaskGroup",
    "GanttChart",
    "TaskDiscussion",
    "Notification",
    "SystemSetting",
]
```

backend/app/models/base.py

```
from datetime import datetime

from sqlalchemy import BigInteger, DateTime, func
from sqlalchemy.orm import Mapped, mapped_column

class TimestampMixin:
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        server_default=func.now(),
```

```
        nullable=False,
    )
    updated_at: Mapped[datetime | None] = mapped_column(
        DateTime(timezone=True),
        nullable=True,
    )

class SoftDeleteMixin:
    deleted_at: Mapped[datetime | None] = mapped_column(
        DateTime(timezone=True),
        nullable=True,
        default=None,
    )

class BaseModelMixin(TimestampMixin, SoftDeleteMixin):
    id: Mapped[int] = mapped_column(
        BigInteger,
        primary_key=True,
        autoincrement=True,
    )
```

backend/app/models/user.py

```
from datetime import date, datetime

from sqlalchemy import (
    BigInteger,
    Boolean,
    Date,
    ForeignKey,
    String,
    Text,
    UniqueConstraint,
)
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database import Base
from app.models.base import BaseModelMixin

class Profile(BaseModelMixin, Base):
    __tablename__ = "profiles"

    avatar: Mapped[str | None] = mapped_column(Text, nullable=True)
    bio: Mapped[str | None] = mapped_column(Text, nullable=True)
    address: Mapped[str | None] = mapped_column(Text, nullable=True)
    emergency_contact: Mapped[str | None] = mapped_column(String(20),
     nullable=True)
    linkedin_url: Mapped[str | None] = mapped_column(String(255),
     nullable=True)

    user: Mapped["User"] = relationship("User", back_populates="profile",
     uselist=False)

class User(BaseModelMixin, Base):
    __tablename__ = "users"

    name: Mapped[str] = mapped_column(String(150), nullable=False)
    username: Mapped[str] = mapped_column(String(100), unique=True,
     nullable=False)
    email: Mapped[str] = mapped_column(String(255), unique=True,
     nullable=False)
    password: Mapped[str] = mapped_column(Text, nullable=False)
    phone: Mapped[str | None] = mapped_column(String(20), nullable=True)
    gender: Mapped[str | None] = mapped_column(String(20), nullable=True)
    dob: Mapped[date | None] = mapped_column(Date, nullable=True)
    profile_id: Mapped[int | None] = mapped_column(
        BigInteger, ForeignKey("profiles.id"), nullable=True
    )
```

```
    is_verified: Mapped[bool] = mapped_column(Boolean, default=False,
     nullable=False)
    status: Mapped[str] = mapped_column(
        String(20), default="Active", nullable=False
    )
    github_username: Mapped[str | None] = mapped_column(String(100),
     nullable=True)

    profile: Mapped[Profile | None] = relationship(
        "Profile", back_populates="user", uselist=False, lazy="joined"
    )
    user_roles: Mapped[list["UserRole"]] = relationship(
        "UserRole", back_populates="user", lazy="selectin"
    )
    employee: Mapped["Employee | None"] = relationship(
        "Employee", back_populates="user", uselist=False
    )

    def __repr__(self) -> str:
        return f"<User(id={self.id}, username={self.username},
     email={self.email})>"

class Role(BaseModelMixin, Base):
    __tablename__ = "roles"

    role_name: Mapped[str] = mapped_column(String(50), unique=True,
     nullable=False)

    user_roles: Mapped[list["UserRole"]] = relationship(
        "UserRole", back_populates="role", lazy="selectin"
    )

    def __repr__(self) -> str:
        return f"<Role(id={self.id}, role_name={self.role_name})>"

class UserRole(BaseModelMixin, Base):
    __tablename__ = "user_roles"

    user_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("users.id"), nullable=False
    )
    role_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("roles.id"), nullable=False
    )

    user: Mapped[User] = relationship("User", back_populates="user_roles")
    role: Mapped[Role] = relationship("Role", back_populates="user_roles")

    __table_args__ = (
        UniqueConstraint("user_id", "role_id", name="uq_user_role"),
    )

    def __repr__(self) -> str:
        return f"<UserRole(user_id={self.user_id}, role_id={self.role_id})>"

from app.models.employee import Employee
```

backend/app/models/employee.py

```
from datetime import date, time

from sqlalchemy import BigInteger, Date, ForeignKey, Integer, String, Text,
     Time
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database import Base
from app.models.base import BaseModelMixin
```

```
class Designation(BaseModelMixin, Base):
    __tablename__ = "designations"

    name: Mapped[str] = mapped_column(String(100), unique=True,
     nullable=False)
    description: Mapped[str | None] = mapped_column(Text, nullable=True)

    employees: Mapped[list["Employee"]] = relationship(
        "Employee", back_populates="designation"
    )

    def __repr__(self) -> str:
        return f"<Designation(id={self.id}, name={self.name})>"

class Department(BaseModelMixin, Base):
    __tablename__ = "departments"

    name: Mapped[str] = mapped_column(String(100), unique=True,
     nullable=False)
    description: Mapped[str | None] = mapped_column(Text, nullable=True)

    employees: Mapped[list["Employee"]] = relationship(
        "Employee", back_populates="department"
    )

    def __repr__(self) -> str:
        return f"<Department(id={self.id}, name={self.name})>"

class WorkSchedule(BaseModelMixin, Base):
    __tablename__ = "work_schedules"

    schedule_name: Mapped[str] = mapped_column(String(100), nullable=False)
    start_time: Mapped[time] = mapped_column(Time, nullable=False)
    end_time: Mapped[time] = mapped_column(Time, nullable=False)
    break_minutes: Mapped[int] = mapped_column(Integer, default=0,
     nullable=False)

    employees: Mapped[list["Employee"]] = relationship(
        "Employee", back_populates="work_schedule"
    )

    def __repr__(self) -> str:
        return f"<WorkSchedule(id={self.id}, name={self.schedule_name})>"

class EmployeeDetail(BaseModelMixin, Base):
    __tablename__ = "employee_details"

    address: Mapped[str | None] = mapped_column(Text, nullable=True)
    city: Mapped[str | None] = mapped_column(String(100), nullable=True)
    state: Mapped[str | None] = mapped_column(String(100), nullable=True)
    country: Mapped[str | None] = mapped_column(String(100), nullable=True)
    postal_code: Mapped[str | None] = mapped_column(String(20),
     nullable=True)
    blood_group: Mapped[str | None] = mapped_column(String(10),
     nullable=True)
    marital_status: Mapped[str | None] = mapped_column(String(20),
     nullable=True)

    employee: Mapped["Employee | None"] = relationship(
        "Employee", back_populates="employee_detail", uselist=False
    )

    def __repr__(self) -> str:
        return f"<EmployeeDetail(id={self.id}, city={self.city})>"

class Employee(BaseModelMixin, Base):
    __tablename__ = "employees"
```

```
    user_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("users.id"), nullable=False
    )
    employee_code: Mapped[str] = mapped_column(
        String(50), unique=True, nullable=False
    )
    manager_id: Mapped[int | None] = mapped_column(
        BigInteger, ForeignKey("employees.id"), nullable=True
    )
    department_id: Mapped[int | None] = mapped_column(
        BigInteger, ForeignKey("departments.id"), nullable=True
    )
    designation_id: Mapped[int | None] = mapped_column(
        BigInteger, ForeignKey("designations.id"), nullable=True
    )
    work_schedule_id: Mapped[int | None] = mapped_column(
        BigInteger, ForeignKey("work_schedules.id"), nullable=True
    )
    employee_details_id: Mapped[int | None] = mapped_column(
        BigInteger, ForeignKey("employee_details.id"), nullable=True
    )
    joining_date: Mapped[date | None] = mapped_column(Date, nullable=True)
    status: Mapped[str] = mapped_column(
        String(20), default="Active", nullable=False
    )

    user: Mapped["User"] = relationship("User", back_populates="employee")
    manager: Mapped["Employee | None"] = relationship(
        "Employee", remote_side="Employee.id", back_populates="subordinates"
    )
    subordinates: Mapped[list["Employee"]] = relationship(
        "Employee", back_populates="manager"
    )
    department: Mapped[Department | None] = relationship(
        "Department", back_populates="employees"
    )
    designation: Mapped[Designation | None] = relationship(
        "Designation", back_populates="employees"
    )
    work_schedule: Mapped[WorkSchedule | None] = relationship(
        "WorkSchedule", back_populates="employees"
    )
    employee_detail: Mapped[EmployeeDetail | None] = relationship(
        "EmployeeDetail", back_populates="employee", lazy="joined"
    )
    attendance_records: Mapped[list["Attendance"]] = relationship(
        "Attendance", back_populates="employee"
    )
    task_time_logs: Mapped[list["TaskTimeLog"]] = relationship(
        "TaskTimeLog", back_populates="employee"
    )
    task_comments: Mapped[list["TaskComment"]] = relationship(
        "TaskComment", back_populates="employee"
    )
    productivity_scores: Mapped[list["ProductivityScore"]] = relationship(
        "ProductivityScore", back_populates="employee"
    )

    def __repr__(self) -> str:
        return f"<Employee(id={self.id}, code={self.employee_code})>"

from app.models.user import User
from app.models.attendance import Attendance
from app.models.task import Task, TaskTimeLog, TaskComment
from app.models.productivity import ProductivityScore
```

backend/app/models/project.py

```
from datetime import date, datetime
from decimal import Decimal

from sqlalchemy import (
    BigInteger,
    Boolean,
    Date,
    DateTime,
    ForeignKey,
    Integer,
    Numeric,
    String,
    Text,
    UniqueConstraint,
)
from sqlalchemy.dialects.postgresql import JSONB
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database import Base
from app.models.base import BaseModelMixin

class Project(BaseModelMixin, Base):
    __tablename__ = "projects"

    name: Mapped[str] = mapped_column(String(255), nullable=False)
    description: Mapped[str | None] = mapped_column(Text, nullable=True)
    created_by_user_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("users.id"), nullable=False
    )
    github_repo: Mapped[str | None] = mapped_column(String(255),
     nullable=True)
    github_default_branch: Mapped[str] = mapped_column(
        String(100), default="main", nullable=False
    )
    status: Mapped[str] = mapped_column(
        String(20), default="Draft", nullable=False
    )
    start_date: Mapped[date | None] = mapped_column(Date, nullable=True)
    end_date: Mapped[date | None] = mapped_column(Date, nullable=True)
    project_initial_doc: Mapped[str | None] = mapped_column(String(500),
     nullable=True)
    project_doc: Mapped[str | None] = mapped_column(String(500),
     nullable=True)
    project_progress_doc: Mapped[str | None] = mapped_column(String(500),
     nullable=True)
    project_report_doc: Mapped[str | None] = mapped_column(String(500),
     nullable=True)
    last_evaluated_at: Mapped[datetime | None] = mapped_column(
        DateTime(timezone=True), nullable=True
    )
    ai_errors: Mapped[dict | None] = mapped_column(JSONB, nullable=True)

    created_by: Mapped["User"] = relationship(
        "User", foreign_keys=[created_by_user_id]
    )
    source_files: Mapped[list["ProjectSourceFile"]] = relationship(
        "ProjectSourceFile", back_populates="project", cascade="all, delete-
     orphan"
    )
    assignments: Mapped[list["ProjectAssignment"]] = relationship(
        "ProjectAssignment", back_populates="project", cascade="all, delete-
     orphan"
    )
    task_groups: Mapped[list["TaskGroup"]] = relationship(
        "TaskGroup", back_populates="project", cascade="all, delete-orphan"
    )
```

```
    gantt_charts: Mapped[list["GanttChart"]] = relationship(
        "GanttChart", back_populates="project", cascade="all, delete-orphan"
    )

    def __repr__(self) -> str:
        return f"<Project(id={self.id}, name={self.name},
     status={self.status})>"

class ProjectSourceFile(BaseModelMixin, Base):
    __tablename__ = "project_source_files"

    project_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("projects.id"), nullable=False
    )
    file_name: Mapped[str] = mapped_column(String(255), nullable=False)
    storage_key: Mapped[str] = mapped_column(String(500), nullable=False)
    content_type: Mapped[str | None] = mapped_column(String(100),
     nullable=True)
    size_bytes: Mapped[int | None] = mapped_column(BigInteger,
     nullable=True)
    extracted_text: Mapped[str | None] = mapped_column(Text, nullable=True)
    uploaded_by_user_id: Mapped[int | None] = mapped_column(
        BigInteger, ForeignKey("users.id"), nullable=True
    )

    project: Mapped[Project] = relationship("Project",
     back_populates="source_files")
    uploaded_by: Mapped["User | None"] = relationship(
        "User", foreign_keys=[uploaded_by_user_id]
    )

    def __repr__(self) -> str:
        return f"<ProjectSourceFile(id={self.id}, file={self.file_name})>"

class ProjectAssignment(BaseModelMixin, Base):
    __tablename__ = "project_assignments"

    project_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("projects.id"), nullable=False
    )
    user_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("users.id"), nullable=False
    )
    project_role: Mapped[str] = mapped_column(
        String(30), nullable=False
    )
    assigned_by_user_id: Mapped[int | None] = mapped_column(
        BigInteger, ForeignKey("users.id"), nullable=True
    )

    __table_args__ = (
        UniqueConstraint("project_id", "user_id", name="uq_project_user"),
    )

    project: Mapped[Project] = relationship("Project",
     back_populates="assignments")
    user: Mapped["User"] = relationship("User", foreign_keys=[user_id])
    assigned_by: Mapped["User | None"] = relationship(
        "User", foreign_keys=[assigned_by_user_id]
    )

    def __repr__(self) -> str:
        return f"<ProjectAssignment(project={self.project_id},
     user={self.user_id}, role={self.project_role})>"

class TaskGroup(BaseModelMixin, Base):
    __tablename__ = "task_groups"
```

```
    project_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("projects.id"), nullable=False
    )
    name: Mapped[str] = mapped_column(String(255), nullable=False)
    description: Mapped[str | None] = mapped_column(Text, nullable=True)
    sequence: Mapped[int] = mapped_column(Integer, default=0,
     nullable=False)
    estimated_duration_days: Mapped[int | None] = mapped_column(Integer,
     nullable=True)
    planned_start_date: Mapped[date | None] = mapped_column(Date,
     nullable=True)
    planned_end_date: Mapped[date | None] = mapped_column(Date,
     nullable=True)
    actual_start_date: Mapped[date | None] = mapped_column(Date,
     nullable=True)
    actual_end_date: Mapped[date | None] = mapped_column(Date,
     nullable=True)
    completion_percentage: Mapped[Decimal] = mapped_column(
        Numeric(5, 2), default=0.0, nullable=False
    )
    status: Mapped[str] = mapped_column(
        String(20), default="Pending", nullable=False
    )
    created_by: Mapped[str] = mapped_column(
        String(20), default="Human", nullable=False
    )
    depends_on: Mapped[list | None] = mapped_column(JSONB, nullable=True)

    project: Mapped[Project] = relationship("Project",
     back_populates="task_groups")
    tasks: Mapped[list["Task"]] = relationship(
        "Task", back_populates="task_group", cascade="all, delete-orphan"
    )

    def __repr__(self) -> str:
        return f"<TaskGroup(id={self.id}, name={self.name},
     project={self.project_id})>"

class GanttChart(BaseModelMixin, Base):
    __tablename__ = "gantt_charts"

    project_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("projects.id"), nullable=False
    )
    version: Mapped[int] = mapped_column(Integer, default=1, nullable=False)
    chart_data: Mapped[dict | None] = mapped_column(JSONB, nullable=True)
    generated_reason: Mapped[str | None] = mapped_column(String(100),
     nullable=True)
    generated_at: Mapped[datetime | None] = mapped_column(
        DateTime(timezone=True), nullable=True
    )
    generated_by_job: Mapped[str | None] = mapped_column(String(64),
     nullable=True)

    project: Mapped[Project] = relationship("Project",
     back_populates="gantt_charts")

    def __repr__(self) -> str:
        return f"<GanttChart(id={self.id}, project={self.project_id},
     v={self.version})>"

class TaskDiscussion(BaseModelMixin, Base):
    __tablename__ = "task_discussions"

    task_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("tasks.id"), nullable=False
    )
    user_id: Mapped[int] = mapped_column(
```

```
        BigInteger, ForeignKey("users.id"), nullable=False
    )
    parent_id: Mapped[int | None] = mapped_column(
        BigInteger, ForeignKey("task_discussions.id"), nullable=True
    )
    message: Mapped[str] = mapped_column(Text, nullable=False)

    task: Mapped["Task"] = relationship("Task",
     back_populates="discussions")
    user: Mapped["User"] = relationship("User", foreign_keys=[user_id])
    parent: Mapped["TaskDiscussion | None"] = relationship(
        "TaskDiscussion", remote_side="TaskDiscussion.id",
     back_populates="replies"
    )
    replies: Mapped[list["TaskDiscussion"]] = relationship(
        "TaskDiscussion", back_populates="parent"
    )

    def __repr__(self) -> str:
        return f"<TaskDiscussion(id={self.id}, task={self.task_id})>"

class Notification(BaseModelMixin, Base):
    __tablename__ = "notifications"

    user_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("users.id"), nullable=False
    )
    type: Mapped[str] = mapped_column(String(50), nullable=False)
    title: Mapped[str] = mapped_column(String(255), nullable=False)
    body: Mapped[str | None] = mapped_column(Text, nullable=True)
    link: Mapped[str | None] = mapped_column(String(500), nullable=True)
    is_read: Mapped[bool] = mapped_column(Boolean, default=False,
     nullable=False)
    related_task_id: Mapped[int | None] = mapped_column(
        BigInteger, ForeignKey("tasks.id"), nullable=True
    )
    related_project_id: Mapped[int | None] = mapped_column(
        BigInteger, ForeignKey("projects.id"), nullable=True
    )

    user: Mapped["User"] = relationship("User", foreign_keys=[user_id])
    related_task: Mapped["Task | None"] = relationship(
        "Task", foreign_keys=[related_task_id]
    )
    related_project: Mapped[Project | None] = relationship(
        "Project", foreign_keys=[related_project_id]
    )

    def __repr__(self) -> str:
        return f"<Notification(id={self.id}, user={self.user_id},
     type={self.type})>"

class SystemSetting(BaseModelMixin, Base):
    __tablename__ = "system_settings"

    category: Mapped[str] = mapped_column(String(50), nullable=False)
    key: Mapped[str] = mapped_column(String(100), nullable=False)
    value: Mapped[str | None] = mapped_column(Text, nullable=True)
    is_secret: Mapped[bool] = mapped_column(Boolean, default=False,
     nullable=False)

    __table_args__ = (
        UniqueConstraint("category", "key", name="uq_setting_category_key"),
    )

    def __repr__(self) -> str:
        return f"<SystemSetting(category={self.category}, key={self.key})>"
```

```
from app.models.user import User
from app.models.task import Task
```

backend/app/models/task.py

```
from datetime import date, datetime
from decimal import Decimal

from sqlalchemy import BigInteger, Date, DateTime, ForeignKey, Integer,
     Numeric, String, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database import Base
from app.models.base import BaseModelMixin

class TaskType(BaseModelMixin, Base):
    __tablename__ = "task_types"

    name: Mapped[str] = mapped_column(String(100), unique=True,
     nullable=False)
    description: Mapped[str | None] = mapped_column(Text, nullable=True)

    tasks: Mapped[list["Task"]] = relationship("Task",
     back_populates="task_type")

    def __repr__(self) -> str:
        return f"<TaskType(id={self.id}, name={self.name})>"

class Task(BaseModelMixin, Base):
    __tablename__ = "tasks"

    project_id: Mapped[int | None] = mapped_column(
        BigInteger, ForeignKey("projects.id"), nullable=True
    )
    task_group_id: Mapped[int | None] = mapped_column(
        BigInteger, ForeignKey("task_groups.id"), nullable=True
    )
    task_progress_doc: Mapped[str | None] = mapped_column(String(500),
     nullable=True)
    progress_doc_error: Mapped[str | None] = mapped_column(Text,
     nullable=True)
    assigned_to_user_id: Mapped[int | None] = mapped_column(
        BigInteger, ForeignKey("users.id"), nullable=True
    )
    assigned_by_user_id: Mapped[int | None] = mapped_column(
        BigInteger, ForeignKey("users.id"), nullable=True
    )
    completed_by: Mapped[str | None] = mapped_column(
        String(20), nullable=True
    )
    completed_at: Mapped[datetime | None] = mapped_column(
        DateTime(timezone=True), nullable=True
    )

    title: Mapped[str] = mapped_column(String(255), nullable=False)
    description: Mapped[str | None] = mapped_column(Text, nullable=True)
    type_id: Mapped[int | None] = mapped_column(
        BigInteger, ForeignKey("task_types.id"), nullable=True
    )
    priority: Mapped[str] = mapped_column(
        String(20), default="Medium", nullable=False
    )
    impact_type: Mapped[str | None] = mapped_column(String(50),
     nullable=True)
    estimated_hours: Mapped[Decimal | None] = mapped_column(Numeric(5, 2),
     nullable=True)
    due_date: Mapped[date | None] = mapped_column(Date, nullable=True)
    status: Mapped[str] = mapped_column(
```

```
            String(20), default="Pending", nullable=False
        )
        github_issue_id: Mapped[str | None] = mapped_column(String(100),
         nullable=True)
        github_issue_url: Mapped[str | None] = mapped_column(String(255),
         nullable=True)
        commit_verified: Mapped[bool] = mapped_column(default=False,
         nullable=False)
        commit_hash: Mapped[str | None] = mapped_column(String(100),
         nullable=True)
        llm_verification_notes: Mapped[str | None] = mapped_column(Text,
         nullable=True)
        health_score: Mapped[str] = mapped_column(String(20), default="Green",
         nullable=False)
        health_reason: Mapped[str | None] = mapped_column(Text, nullable=True)
        last_health_check: Mapped[datetime | None] =
         mapped_column(DateTime(timezone=True), nullable=True)

        task_type: Mapped[TaskType | None] = relationship("TaskType",
         back_populates="tasks")
        project: Mapped["Project | None"] = relationship(
            "Project", foreign_keys=[project_id], back_populates=None
        )
        task_group: Mapped["TaskGroup | None"] = relationship(
            "TaskGroup", foreign_keys=[task_group_id], back_populates="tasks"
        )
        assigned_to_user: Mapped["User | None"] = relationship(
            "User", foreign_keys=[assigned_to_user_id]
        )
        assigned_by_user: Mapped["User | None"] = relationship(
            "User", foreign_keys=[assigned_by_user_id]
        )
        discussions: Mapped[list["TaskDiscussion"]] = relationship(
            "TaskDiscussion", back_populates="task", cascade="all, delete-
         orphan"
        )
        task_detail: Mapped["TaskDetail | None"] = relationship(
            "TaskDetail", back_populates="task", uselist=False
        )
        comments: Mapped[list["TaskComment"]] = relationship(
            "TaskComment", back_populates="task"
        )
        time_logs: Mapped[list["TaskTimeLog"]] = relationship(
            "TaskTimeLog", back_populates="task"
        )

        def __repr__(self) -> str:
            return f"<Task(id={self.id}, title={self.title},
         status={self.status})>"

    class TaskDetail(BaseModelMixin, Base):
        __tablename__ = "task_details"

        task_id: Mapped[int] = mapped_column(
            BigInteger, ForeignKey("tasks.id"), nullable=False
        )
        requirements: Mapped[str | None] = mapped_column(Text, nullable=True)
        acceptance_criteria: Mapped[str | None] = mapped_column(Text,
         nullable=True)
        remarks: Mapped[str | None] = mapped_column(Text, nullable=True)

        task: Mapped[Task] = relationship("Task", back_populates="task_detail")

        def __repr__(self) -> str:
            return f"<TaskDetail(id={self.id}, task_id={self.task_id})>"

    class TaskComment(BaseModelMixin, Base):
        __tablename__ = "task_comments"
```

```
    task_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("tasks.id"), nullable=False
    )
    employee_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("employees.id"), nullable=False
    )
    comment: Mapped[str] = mapped_column(Text, nullable=False)

    task: Mapped[Task] = relationship("Task", back_populates="comments")
    employee: Mapped["Employee"] = relationship("Employee",
     back_populates="task_comments")

    def __repr__(self) -> str:
        return f"<TaskComment(id={self.id}, task_id={self.task_id})>"

class TaskTimeLog(BaseModelMixin, Base):
    __tablename__ = "task_time_logs"

    task_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("tasks.id"), nullable=False
    )
    employee_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("employees.id"), nullable=False
    )
    start_time: Mapped[datetime | None] = mapped_column(
        DateTime(timezone=True), nullable=True
    )
    end_time: Mapped[datetime | None] = mapped_column(
        DateTime(timezone=True), nullable=True
    )
    active_minutes: Mapped[int] = mapped_column(Integer, default=0,
     nullable=False)
    idle_minutes: Mapped[int] = mapped_column(Integer, default=0,
     nullable=False)

    task: Mapped[Task] = relationship("Task", back_populates="time_logs")
    employee: Mapped["Employee"] = relationship("Employee",
     back_populates="task_time_logs")

    def __repr__(self) -> str:
        return f"<TaskTimeLog(id={self.id}, task_id={self.task_id},
     active_min={self.active_minutes})>"

from app.models.employee import Employee
from app.models.project import Project, TaskGroup, TaskDiscussion
from app.models.user import User
```

backend/app/models/hrms.py

```
from datetime import date
from sqlalchemy import BigInteger, Date, ForeignKey, String, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database import Base
from app.models.base import BaseModelMixin

class Leave(BaseModelMixin, Base):
    __tablename__ = "leaves"

    employee_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("employees.id"), nullable=False
    )
    leave_type: Mapped[str] = mapped_column(
        String(50), default="Casual", nullable=False
    )
    start_date: Mapped[date] = mapped_column(Date, nullable=False)
    end_date: Mapped[date] = mapped_column(Date, nullable=False)
```

```
    reason: Mapped[str | None] = mapped_column(Text, nullable=True)
    status: Mapped[str] = mapped_column(
        String(20), default="Pending", nullable=False
    )
    approved_by: Mapped[int | None] = mapped_column(
        BigInteger, ForeignKey("employees.id"), nullable=True
    )

    employee: Mapped["Employee"] = relationship(
        "Employee", foreign_keys=[employee_id]
    )
    approver: Mapped["Employee | None"] = relationship(
        "Employee", foreign_keys=[approved_by]
    )

    def __repr__(self) -> str:
        return f"<Leave(id={self.id}, employee_id={self.employee_id},
     type={self.leave_type}, status={self.status})>"

class Holiday(BaseModelMixin, Base):
    __tablename__ = "holidays"

    name: Mapped[str] = mapped_column(String(150), nullable=False)
    holiday_date: Mapped[date] = mapped_column(Date, nullable=False)
    description: Mapped[str | None] = mapped_column(Text, nullable=True)
    type: Mapped[str] = mapped_column(
        String(50), default="Public", nullable=False
    )

    def __repr__(self) -> str:
        return f"<Holiday(id={self.id}, name={self.name},
     date={self.holiday_date})>"

from app.models.employee import Employee
```

backend/app/models/attendance.py

```
from datetime import date, datetime
from decimal import Decimal

from sqlalchemy import BigInteger, Date, DateTime, ForeignKey, Numeric,
     String, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database import Base
from app.models.base import BaseModelMixin

class TimeZone(BaseModelMixin, Base):
    __tablename__ = "time_zones"

    timezone_name: Mapped[str] = mapped_column(String(100), nullable=False)
    utc_offset: Mapped[str] = mapped_column(String(20), nullable=False)

    attendance_records: Mapped[list["Attendance"]] = relationship(
        "Attendance", back_populates="time_zone"
    )

    def __repr__(self) -> str:
        return f"<TimeZone(id={self.id}, name={self.timezone_name})>"

class Attendance(BaseModelMixin, Base):
    __tablename__ = "attendance"

    employee_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("employees.id"), nullable=False
    )
    time_zone_id: Mapped[int | None] = mapped_column(
        BigInteger, ForeignKey("time_zones.id"), nullable=True
```

```
    )
    attendance_date: Mapped[date] = mapped_column(Date, nullable=False)
    time_in: Mapped[datetime | None] =
     mapped_column(DateTime(timezone=True), nullable=True)
    time_out: Mapped[datetime | None] =
     mapped_column(DateTime(timezone=True), nullable=True)
    total_hours: Mapped[Decimal | None] = mapped_column(Numeric(5, 2),
     nullable=True)
    latitude: Mapped[Decimal | None] = mapped_column(Numeric(10, 7),
     nullable=True)
    longitude: Mapped[Decimal | None] = mapped_column(Numeric(10, 7),
     nullable=True)
    location: Mapped[str | None] = mapped_column(Text, nullable=True)
    ip_address: Mapped[str | None] = mapped_column(String(45),
     nullable=True)
    location_status: Mapped[str | None] = mapped_column(
        String(20), nullable=True
    )

    employee: Mapped["Employee"] = relationship(
        "Employee", back_populates="attendance_records"
    )
    time_zone: Mapped[TimeZone | None] = relationship(
        "TimeZone", back_populates="attendance_records"
    )

    def __repr__(self) -> str:
        return f"<Attendance(id={self.id}, employee_id={self.employee_id},
     date={self.attendance_date})>"

from app.models.employee import Employee
```

backend/app/models/productivity.py

```
from datetime import date, datetime
from decimal import Decimal
from typing import Any

from sqlalchemy import BigInteger, Date, DateTime, ForeignKey, Numeric,
     String
from sqlalchemy.dialects.postgresql import JSONB
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database import Base
from app.models.base import BaseModelMixin

class ProductivityScore(BaseModelMixin, Base):
    __tablename__ = "productivity_scores"

    employee_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("employees.id"), nullable=False
    )
    week_start: Mapped[date] = mapped_column(Date, nullable=False)
    att_score: Mapped[Decimal | None] = mapped_column(Numeric(5, 2),
     nullable=True)
    task_score: Mapped[Decimal | None] = mapped_column(Numeric(5, 2),
     nullable=True)
    deadline_score: Mapped[Decimal | None] = mapped_column(Numeric(5, 2),
     nullable=True)
    active_time_score: Mapped[Decimal | None] = mapped_column(Numeric(5, 2),
     nullable=True)
    final_score: Mapped[Decimal | None] = mapped_column(Numeric(5, 2),
     nullable=True)
    rating: Mapped[str | None] = mapped_column(
        String(15), nullable=True
    )
    anomaly_flags: Mapped[dict[str, Any] | None] = mapped_column(JSONB,
     nullable=True)
```

```
    generated_at: Mapped[datetime | None] = mapped_column(
        DateTime(timezone=True), nullable=True
    )

    employee: Mapped["Employee"] = relationship(
        "Employee", back_populates="productivity_scores"
    )

    def __repr__(self) -> str:
        return (
            f"<ProductivityScore(id={self.id},
     employee_id={self.employee_id}, "
            f"week={self.week_start}, final={self.final_score},
     rating={self.rating})>"
        )

from app.models.employee import Employee
```

4.1.6 Backend: Schemas

backend/app/schemas/auth.py

```
from typing import Literal

from pydantic import BaseModel, EmailStr, Field

class RegisterRequest(BaseModel):
    name: str = Field(..., min_length=2, max_length=150, examples=["Bipasha
     Bagchi"])
    username: str = Field(..., min_length=3, max_length=100,
     examples=["bipasha"])
    email: EmailStr = Field(..., examples=["bipasha@trace.com"])
    password: str = Field(..., min_length=8, max_length=128,
     examples=["SecurePass123!"])
    phone: str | None = Field(None, max_length=20, examples=["+91-
     9876543210"])
    gender: str | None = Field(None, max_length=20, examples=["Female"])
    role: Literal["Admin", "Manager", "Employee"] =
     Field(default="Employee", examples=["Employee"])

class LoginRequest(BaseModel):
    email: EmailStr = Field(..., examples=["bipasha@trace.com"])
    password: str = Field(..., examples=["SecurePass123!"])

class RefreshTokenRequest(BaseModel):
    refresh_token: str

class TokenResponse(BaseModel):
    access_token: str
    refresh_token: str
    token_type: str = "Bearer"

class AuthUserResponse(BaseModel):
    id: int
    name: str
    username: str
    email: str
    roles: list[str]
    is_verified: bool
    status: str

    model_config = {"from_attributes": True}

class LoginResponse(BaseModel):
    tokens: TokenResponse
    user: AuthUserResponse
    message: str = "Login successful"
```

```
class RegisterResponse(BaseModel):
    user: AuthUserResponse
    tokens: TokenResponse
    message: str = "Registration successful"
```

backend/app/schemas/user.py

```
from datetime import date, datetime

from pydantic import BaseModel, EmailStr, Field

class ProfileBase(BaseModel):
    avatar: str | None = None
    bio: str | None = None
    address: str | None = None
    emergency_contact: str | None = None
    linkedin_url: str | None = None

class ProfileCreate(ProfileBase):
    pass

class ProfileResponse(ProfileBase):
    id: int
    created_at: datetime | None = None

    model_config = {"from_attributes": True}

class UserBase(BaseModel):
    name: str = Field(..., min_length=2, max_length=150)
    username: str = Field(..., min_length=3, max_length=100)
    email: EmailStr
    phone: str | None = None
    gender: str | None = None
    dob: date | None = None

class UserUpdate(BaseModel):
    name: str | None = Field(None, min_length=2, max_length=150)
    phone: str | None = None
    gender: str | None = None
    dob: date | None = None
    github_username: str | None = Field(None, max_length=39, pattern=r"^$|
     ^[A-Za-z0-9](?:[A-Za-z0-9-]*[A-Za-z0-9])?$")

class UserResponse(UserBase):
    id: int
    is_verified: bool
    status: str
    profile: ProfileResponse | None = None
    roles: list[str] = []
    github_username: str | None = None
    created_at: datetime | None = None

    model_config = {"from_attributes": True}

class UserListResponse(BaseModel):
    users: list[UserResponse]
    total: int
    page: int
    per_page: int

class RoleResponse(BaseModel):
    id: int
    role_name: str

    model_config = {"from_attributes": True}
```

backend/app/schemas/employee.py

```
from datetime import date, datetime, time
from pydantic import BaseModel, EmailStr, Field

class DesignationCreate(BaseModel):
    name: str = Field(..., max_length=100)
    description: str | None = None

class DesignationResponse(BaseModel):
    id: int
    name: str
    description: str | None = None
    created_at: datetime | None = None

    model_config = {"from_attributes": True}

class DepartmentCreate(BaseModel):
    name: str = Field(..., max_length=100)
    description: str | None = None

class DepartmentResponse(BaseModel):
    id: int
    name: str
    description: str | None = None
    created_at: datetime | None = None

    model_config = {"from_attributes": True}

class WorkScheduleCreate(BaseModel):
    schedule_name: str = Field(..., max_length=100)
    start_time: time
    end_time: time
    break_minutes: int = 0

class WorkScheduleResponse(BaseModel):
    id: int
    schedule_name: str
    start_time: time
    end_time: time
    break_minutes: int
    created_at: datetime | None = None

    model_config = {"from_attributes": True}

class EmployeeDetailCreate(BaseModel):
    address: str | None = None
    city: str | None = None
    state: str | None = None
    country: str | None = None
    postal_code: str | None = None
    blood_group: str | None = None
    marital_status: str | None = None

class EmployeeDetailResponse(EmployeeDetailCreate):
    id: int
    created_at: datetime | None = None

    model_config = {"from_attributes": True}

class UserBriefResponse(BaseModel):
    id: int
    name: str
    username: str
    email: str
    role: str = "Employee"

    model_config = {"from_attributes": True}
```

```
class EmployeeCreate(BaseModel):
    user_id: int | None = None
    name: str | None = None
    email: EmailStr | None = None
    password: str | None = None
    role: str = "Employee"

    employee_code: str = Field(..., max_length=50)
    manager_id: int | None = None
    department_id: int | None = None
    designation_id: int | None = None
    work_schedule_id: int | None = None
    joining_date: date | None = None
    employee_detail: EmployeeDetailCreate | None = None

class EmployeeUpdate(BaseModel):
    manager_id: int | None = None
    department_id: int | None = None
    designation_id: int | None = None
    work_schedule_id: int | None = None
    status: str | None = None

class EmployeeResponse(BaseModel):
    id: int
    user_id: int
    employee_code: str
    manager_id: int | None = None
    department_id: int | None = None
    designation_id: int | None = None
    work_schedule_id: int | None = None
    joining_date: date | None = None
    status: str
    employee_detail: EmployeeDetailResponse | None = None
    department: DepartmentResponse | None = None
    designation: DesignationResponse | None = None
    work_schedule: WorkScheduleResponse | None = None
    user: UserBriefResponse | None = None
    created_at: datetime | None = None

    model_config = {"from_attributes": True}

class EmployeeListResponse(BaseModel):
    employees: list[EmployeeResponse]
    total: int
    page: int
    per_page: int
```

backend/app/schemas/project.py

```
from datetime import date, datetime
from decimal import Decimal
from typing import Literal

from pydantic import BaseModel, ConfigDict, Field, field_validator

from app.services.github_service import REPO_PATTERN

Priority = Literal["Low", "Medium", "High", "Critical"]
TaskStatus = Literal["Pending", "In Progress", "In Review", "Completed",
     "Blocked", "Overdue", "Cancelled"]
EditableProjectStatus = Literal["Draft", "Active", "Completed", "Archived"]

def _validate_repo(value: str | None) -> str | None:
    if value is None:
        return value
```

```
    value =
     value.strip().removeprefix("https://github.com/").removesuffix(".git").s
     trip("/")
    if not REPO_PATTERN.match(value):
        raise ValueError("github_repo must look like 'owner/repo'")
    return value

class PersonBrief(BaseModel):
    id: int
    name: str
    email: str
    github_username: str | None = None

class ProjectCreate(BaseModel):
    name: str = Field(..., min_length=2, max_length=255)
    description: str | None = None
    start_date: date | None = None
    end_date: date | None = None

class ProjectUpdate(BaseModel):
    name: str | None = Field(None, min_length=2, max_length=255)
    description: str | None = None
    github_repo: str | None = None
    github_default_branch: str | None = Field(None, min_length=1,
     max_length=100)
    status: EditableProjectStatus | None = None
    start_date: date | None = None
    end_date: date | None = None

    _repo = field_validator("github_repo")(_validate_repo)

class ProjectStats(BaseModel):
    task_groups: int = 0
    total_tasks: int = 0
    completed_tasks: int = 0
    overdue_tasks: int = 0
    completion_percentage: float = 0.0

class ProjectSummary(BaseModel):
    id: int
    name: str
    description: str | None
    status: str
    github_repo: str | None
    start_date: date | None
    end_date: date | None
    created_at: datetime | None
    owner: PersonBrief
    project_managers: list[PersonBrief]
    my_role: str | None = None
    stats: ProjectStats
    at_risk: bool

class SourceFileResponse(BaseModel):
    id: int
    file_name: str
    content_type: str | None
    size_bytes: int | None
    extracted_chars: int
    uploaded_at: datetime | None

class ProjectDocuments(BaseModel):
    project_initial_doc: str | None
    project_doc: str | None
    project_progress_doc: str | None
    project_report_doc: str | None

class ProjectDetail(ProjectSummary):
```

```
    github_default_branch: str
    documents: ProjectDocuments
    last_evaluated_at: datetime | None
    ai_errors: dict[str, dict] = {}
    source_files: list[SourceFileResponse]

class UploadRejection(BaseModel):
    file_name: str
    reason: str

class SourceFileUploadResponse(BaseModel):
    uploaded: list[SourceFileResponse]
    rejected: list[UploadRejection]

class AssignProjectManager(BaseModel):
    user_id: int

class AssignmentResponse(BaseModel):
    user: PersonBrief
    project_role: str
    open_tasks: int
    assigned_at: datetime | None
    warnings: list[str] = []

class RemovalResponse(BaseModel):
    project_role: str | None
    warnings: list[str] = []

class TaskGroupCreate(BaseModel):
    name: str = Field(..., min_length=1, max_length=255)
    description: str | None = None
    sequence: int | None = None
    estimated_duration_days: int | None = Field(None, ge=1)
    planned_start_date: date | None = None
    planned_end_date: date | None = None

class TaskGroupUpdate(BaseModel):
    name: str | None = Field(None, min_length=1, max_length=255)
    description: str | None = None
    sequence: int | None = None
    estimated_duration_days: int | None = Field(None, ge=1)
    planned_start_date: date | None = None
    planned_end_date: date | None = None

class TaskGroupResponse(BaseModel):
    id: int
    project_id: int
    name: str
    description: str | None
    sequence: int
    estimated_duration_days: int | None
    planned_start_date: date | None
    planned_end_date: date | None
    completion_percentage: float
    status: str
    created_by: str
    total_tasks: int
    completed_tasks: int

class TaskCreate(BaseModel):
    title: str = Field(..., min_length=2, max_length=255)
    description: str | None = None
    task_group_id: int
    priority: Priority = "Medium"
    due_date: date | None = None
    estimated_hours: Decimal | None = Field(None, ge=0, le=999)
    assigned_to_user_id: int | None = None
```

```
class TaskUpdate(BaseModel):
    title: str | None = Field(None, min_length=2, max_length=255)
    description: str | None = None
    task_group_id: int | None = None
    priority: Priority | None = None
    due_date: date | None = None
    estimated_hours: Decimal | None = Field(None, ge=0, le=999)

class TaskAssign(BaseModel):
    assigned_to_user_id: int | None

class TaskStatusUpdate(BaseModel):
    status: TaskStatus

class TaskResponse(BaseModel):
    id: int
    project_id: int
    project_name: str
    task_group_id: int | None
    task_group_name: str | None
    title: str
    description: str | None
    priority: str
    status: str
    due_date: date | None
    estimated_hours: float | None
    github_issue_id: str | None
    github_issue_url: str | None
    task_progress_doc: str | None
    progress_doc_error: str | None = None
    assigned_to: PersonBrief | None
    assigned_by: PersonBrief | None
    completed_by: str | None
    completed_at: datetime | None
    health_score: str
    health_reason: str | None
    created_at: datetime | None
    updated_at: datetime | None
    can_edit: bool = False
    can_discuss: bool = False
    allowed_statuses: list[str] = []

class TaskMutationResponse(BaseModel):
    task: TaskResponse
    warnings: list[str] = []

class ActivityItem(BaseModel):
    task_id: int
    task_title: str
    project_id: int
    project_name: str
    status: str
    assigned_to: PersonBrief | None
    updated_at: datetime | None

DocumentKind = Literal["initial", "doc", "progress", "report"]

class JobQueued(BaseModel):
    job_id: str

class EvaluationStatus(BaseModel):
    state: Literal["idle", "running", "failed"]
    error: str | None
    project_status: str
    last_evaluated_at: datetime | None

class DocumentResponse(BaseModel):
    kind: str
```

```
    url: str | None
    content: str | None

class DocumentUpdate(BaseModel):
    content: str = Field(..., min_length=1, max_length=200_000)

class DocumentSaved(DocumentResponse):
    changed: bool

class AssigneeSuggestionRequest(BaseModel):
    title: str = Field(..., min_length=2, max_length=255)
    description: str | None = None

class DiscussionCreate(BaseModel):
    message: str = Field(min_length=1, max_length=5000)
    parent_id: int | None = None

class DiscussionResponse(BaseModel):
    id: int
    task_id: int
    parent_id: int | None
    message: str
    author: PersonBrief | None
    created_at: datetime | None

class NotificationResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    type: str
    title: str
    body: str | None
    link: str | None
    is_read: bool
    related_task_id: int | None
    related_project_id: int | None
    created_at: datetime | None

class NotificationList(BaseModel):
    items: list[NotificationResponse]
    unread: int
```

backend/app/schemas/gantt.py

```
from datetime import date, datetime
from typing import Literal

from pydantic import BaseModel, Field, model_validator

from app.schemas.project import PersonBrief, TaskStatus

GroupStatus = Literal["Pending", "In Progress", "Completed", "Blocked"]
GanttReason = Literal["initial", "task_status_change",
     "project_doc_updated", "manual"]

class GanttRange(BaseModel):
    start: date
    end: date

class GanttTask(BaseModel):
    task_id: int
    title: str
    status: TaskStatus
    due: date | None = None
    assignee: str | None = None

class GanttBar(BaseModel):
    task_group_id: int
```

```
    name: str
    sequence: int
    start: date
    end: date
    completion: float = Field(ge=0, le=100)
    status: GroupStatus
    tasks: list[GanttTask] = []

    @model_validator(mode="after")
    def _ordered(self):
        if self.start > self.end:
            raise ValueError(f"bar '{self.name}' ends before it starts")
        return self

class GanttDependency(BaseModel):
    from_: int = Field(alias="from")
    to: int

    model_config = {"populate_by_name": True}

class GanttMilestone(BaseModel):
    name: str
    date: date

class GanttChartData(BaseModel):
    schema_: Literal[1] = Field(1, alias="schema")
    range: GanttRange
    bars: list[GanttBar]
    dependencies: list[GanttDependency] = []
    milestones: list[GanttMilestone] = []

    model_config = {"populate_by_name": True}

    @model_validator(mode="after")
    def _consistent(self):
        ids = [b.task_group_id for b in self.bars]
        if len(ids) != len(set(ids)):
            raise ValueError("each task group may have only one bar")
        for dep in self.dependencies:
            if dep.from_ == dep.to:
                raise ValueError("a task group cannot depend on itself")
            if dep.from_ not in ids or dep.to not in ids:
                raise ValueError("dependencies must reference bars in this
     chart")
        dates = [d for b in self.bars for d in (b.start, b.end)] + [m.date
     for m in self.milestones]
        if self.range.start > self.range.end or any(not self.range.start <=
     d <= self.range.end for d in dates):
            raise ValueError("range must cover every bar and milestone")
        return self

    def dump(self) -> dict:
        return self.model_dump(mode="json", by_alias=True)

class GanttVersionBrief(BaseModel):
    version: int
    generated_reason: str | None
    generated_at: datetime | None

class GanttResponse(GanttVersionBrief):
    chart_data: dict
    latest_version: int

class TreeTask(BaseModel):
    id: int
    title: str
    status: str
    priority: str | None
```

```
    due_date: date | None
    overdue: bool
    assignee: PersonBrief | None
    github_issue_url: str | None

class TreeGroup(BaseModel):
    id: int
    name: str
    sequence: int
    status: str
    completion_percentage: float
    tasks: list[TreeTask]

class TreeProject(BaseModel):
    id: int
    name: str
    status: str
    completion_percentage: float

class ProjectTree(BaseModel):
    project: TreeProject
    groups: list[TreeGroup]
```

backend/app/schemas/ai.py

```
from typing import Literal

from pydantic import BaseModel, Field, ValidationInfo, field_validator,
     model_validator

from app.schemas.project import Priority

def _context(info: ValidationInfo, key: str):
    return (info.context or {}).get(key)

class ProjectDocumentOut(BaseModel):
    title: str = Field(..., min_length=1, max_length=255)
    markdown: str = Field(..., min_length=50)

class TaskGroupOut(BaseModel):
    name: str = Field(..., min_length=1, max_length=255)
    description: str = ""
    sequence: int = Field(..., ge=1)
    estimated_duration_days: int = Field(..., ge=1, le=365)
    depends_on: list[str] = []

class TaskGroupPlanOut(BaseModel):
    task_groups: list[TaskGroupOut] = Field(..., min_length=1,
     max_length=20)

    @model_validator(mode="after")
    def _consistent(self):
        self.task_groups.sort(key=lambda g: g.sequence)
        seen: set[str] = set()
        for g in self.task_groups:
            key = g.name.strip().lower()
            if key in seen:
                raise ValueError(f"task group names must be unique —
     '{g.name}' repeats")
            for dep in g.depends_on:
                if dep.strip().lower() not in seen:
                    raise ValueError(f"'{g.name}' depends on '{dep}', which
     is not an earlier task group")
            seen.add(key)
        return self

class PersonSuggestionOut(BaseModel):
    user_id: int
```

```
    score: float = Field(..., ge=0, le=1)
    reasoning: str = Field(..., min_length=1)
    evidence: list[str] = []

    @field_validator("user_id")
    @classmethod
    def _is_candidate(cls, value: int, info: ValidationInfo) -> int:
        candidates = _context(info, "candidates")
        if candidates is not None and value not in candidates:
            raise ValueError(f"user_id {value} is not one of the
     candidates")
        return value

class PeopleSuggestionsOut(BaseModel):
    suggestions: list[PersonSuggestionOut] = Field(..., max_length=25)

    @model_validator(mode="after")
    def _unique(self):
        ids = [s.user_id for s in self.suggestions]
        if len(ids) != len(set(ids)):
            raise ValueError("each candidate may appear once")
        self.suggestions.sort(key=lambda s: s.score, reverse=True)
        return self

class TaskSuggestionOut(BaseModel):
    title: str = Field(..., min_length=2, max_length=255)
    description: str = ""
    priority: Priority = "Medium"
    estimated_hours: float | None = Field(None, ge=0, le=999)

class TaskSuggestionsOut(BaseModel):
    tasks: list[TaskSuggestionOut] = Field(..., min_length=1, max_length=15)

class ProgressStepOut(BaseModel):
    order: int = Field(..., ge=1)
    title: str = Field(..., min_length=1)
    detail: str = ""
    done: bool = False

class TaskProgressPlanOut(BaseModel):
    steps: list[ProgressStepOut] = Field(..., min_length=1, max_length=20)
    acceptance_criteria: list[str] = Field(..., min_length=1)
    completion_status: Literal["not_started", "in_progress", "done"] =
     "not_started"

class GroupRevisionOut(BaseModel):
    task_group_id: int
    estimated_duration_days: int = Field(..., ge=1, le=365)

    @field_validator("task_group_id")
    @classmethod
    def _is_open_group(cls, value: int, info: ValidationInfo) -> int:
        groups = _context(info, "open_groups")
        if groups is not None and value not in groups:
            raise ValueError(f"task_group_id {value} is not an unfinished
     group of this project")
        return value

class NewGroupOut(BaseModel):
    name: str = Field(..., min_length=1, max_length=255)
    description: str = ""
    estimated_duration_days: int = Field(..., ge=1, le=365)
    depends_on: list[int] = []

    @field_validator("depends_on")
    @classmethod
    def _existing(cls, value: list[int], info: ValidationInfo) -> list[int]:
        groups = _context(info, "all_groups")
```

```
        if groups is not None and any(g not in groups for g in value):
            raise ValueError("depends_on may only name existing task group
     ids")
        return value

class GanttRevisionOut(BaseModel):
    revisions: list[GroupRevisionOut] = []
    new_groups: list[NewGroupOut] = Field([], max_length=10)

class ProgressReportOut(BaseModel):
    progress_markdown: str = Field(..., min_length=20)
    report_markdown: str = Field(..., min_length=20)
```

backend/app/schemas/attendance.py

```
from datetime import date, datetime
from decimal import Decimal

from pydantic import BaseModel

class AttendanceResponse(BaseModel):
    id: int
    employee_id: int
    attendance_date: date
    time_in: datetime | None = None
    time_out: datetime | None = None
    total_hours: Decimal | None = None
    latitude: Decimal | None = None
    longitude: Decimal | None = None
    location: str | None = None
    location_status: str | None = None
    ip_address: str | None = None
    created_at: datetime | None = None

    model_config = {"from_attributes": True}

class AttendanceListResponse(BaseModel):
    records: list[AttendanceResponse]
    total: int
    page: int
    per_page: int
```

backend/app/schemas/productivity.py

```
from datetime import date, datetime
from decimal import Decimal
from typing import Any

from pydantic import BaseModel

class ProductivityScoreResponse(BaseModel):
    id: int
    employee_id: int
    week_start: date
    att_score: Decimal | None = None
    task_score: Decimal | None = None
    deadline_score: Decimal | None = None
    active_time_score: Decimal | None = None
    final_score: Decimal | None = None
    rating: str | None = None
    anomaly_flags: dict[str, Any] | None = None
    generated_at: datetime | None = None
    created_at: datetime | None = None

    model_config = {"from_attributes": True}

class ProductivityScoreListResponse(BaseModel):
    scores: list[ProductivityScoreResponse]
```

```
    total: int
    page: int
    per_page: int

class ProductivitySummary(BaseModel):
    total_employees: int
    average_score: Decimal | None = None
    top_performer: ProductivityScoreResponse | None = None
    at_risk_employees: list[ProductivityScoreResponse] = []
    excellent_count: int = 0
    good_count: int = 0
    average_count: int = 0
    risk_count: int = 0
```

4.1.7 Backend: Services

backend/app/services/auth_service.py

```
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy.orm import selectinload

from app.core.security import (
    create_access_token,
    create_refresh_token,
    decode_token,
    hash_password,
    verify_password,
)
from app.exceptions import (
    AlreadyExistsException,
    InvalidCredentialsException,
    InvalidTokenException,
)
from app.models.user import Profile, Role, User, UserRole
from app.schemas.auth import (
    AuthUserResponse,
    LoginRequest,
    LoginResponse,
    RegisterRequest,
    RegisterResponse,
    TokenResponse,
)

class AuthService:
    def __init__(self, db: AsyncSession):
        self.db = db

    async def register(self, data: RegisterRequest) -> RegisterResponse:
        existing = await self.db.execute(
            select(User).where(
                (User.email == data.email) | (User.username ==
     data.username),
                User.deleted_at.is_(None),
            )
        )
        if existing.scalar_one_or_none():
            raise AlreadyExistsException("User with this email or username")

        profile = Profile()
        self.db.add(profile)
        await self.db.flush()

        user = User(
            name=data.name,
            username=data.username,
            email=data.email,
            password=hash_password(data.password),
```

```
            phone=data.phone,
            gender=data.gender,
            profile_id=profile.id,
            is_verified=False,
            status="Active",
        )
        self.db.add(user)
        await self.db.flush()

        role = await self.db.execute(
            select(Role).where(Role.role_name == data.role)
        )
        role_obj = role.scalar_one_or_none()
        if role_obj is None:
            role_obj = Role(role_name=data.role)
            self.db.add(role_obj)
            await self.db.flush()

        user_role = UserRole(user_id=user.id, role_id=role_obj.id)
        self.db.add(user_role)
        await self.db.flush()

        token_data = {
            "sub": user.id,
            "email": user.email,
            "roles": [data.role],
        }
        tokens = TokenResponse(
            access_token=create_access_token(token_data),
            refresh_token=create_refresh_token(token_data),
        )

        user_response = AuthUserResponse(
            id=user.id,
            name=user.name,
            username=user.username,
            email=user.email,
            roles=[data.role],
            is_verified=user.is_verified,
            status=user.status,
        )

        return RegisterResponse(user=user_response, tokens=tokens)

    async def login(self, data: LoginRequest) -> LoginResponse:
        stmt = (
            select(User)
            .where(User.email == data.email, User.deleted_at.is_(None))
            .options(selectinload(User.user_roles).selectinload(UserRole.rol
 e))
        )
        result = await self.db.execute(stmt)
        user = result.scalar_one_or_none()

        if user is None:
            raise InvalidCredentialsException()

        if not verify_password(data.password, user.password):
            raise InvalidCredentialsException()

        if user.status == "Blocked":
            raise InvalidCredentialsException()

        roles = [ur.role.role_name for ur in user.user_roles]

        token_data = {
            "sub": user.id,
            "email": user.email,
```

```
            "roles": roles,
        }
        tokens = TokenResponse(
            access_token=create_access_token(token_data),
            refresh_token=create_refresh_token(token_data),
        )

        user_response = AuthUserResponse(
            id=user.id,
            name=user.name,
            username=user.username,
            email=user.email,
            roles=roles,
            is_verified=user.is_verified,
            status=user.status,
        )

        return LoginResponse(tokens=tokens, user=user_response)

    async def refresh_token(self, refresh_token: str) -> TokenResponse:
        try:
            payload = decode_token(refresh_token)
        except Exception:
            raise InvalidTokenException()

        if payload.get("type") != "refresh":
            raise InvalidTokenException()

        user = await self.db.execute(
            select(User).where(User.id == payload["sub"],
     User.deleted_at.is_(None))
        )
        user_obj = user.scalar_one_or_none()
        if user_obj is None or user_obj.status == "Blocked":
            raise InvalidTokenException()

        token_data = {
            "sub": payload["sub"],
            "email": payload["email"],
            "roles": payload.get("roles", []),
        }

        return TokenResponse(
            access_token=create_access_token(token_data),
            refresh_token=create_refresh_token(token_data),
        )
```

backend/app/services/user_service.py

```
from sqlalchemy import func, select
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy.orm import selectinload

from app.exceptions import NotFoundException
from app.models.user import User, UserRole
from app.schemas.user import UserListResponse, UserResponse, UserUpdate

class UserService:
    def __init__(self, db: AsyncSession):
        self.db = db

    async def get_user_by_id(self, user_id: int) -> UserResponse:
        stmt = (
            select(User)
            .where(User.id == user_id, User.deleted_at.is_(None))
            .options(selectinload(User.user_roles).selectinload(UserRole.rol
     e))
        )
```

```
        result = await self.db.execute(stmt)
        user = result.scalar_one_or_none()

        if user is None:
            raise NotFoundException("User")

        return self._to_response(user)

    async def list_users(self, page: int = 1, per_page: int = 20) ->
     UserListResponse:
        offset = (page - 1) * per_page

        count_stmt =
     select(func.count(User.id)).where(User.deleted_at.is_(None))
        total = (await self.db.execute(count_stmt)).scalar() or 0

        stmt = (
            select(User)
            .where(User.deleted_at.is_(None))
            .options(selectinload(User.user_roles).selectinload(UserRole.rol
    e))
            .order_by(User.created_at.desc())
            .offset(offset)
            .limit(per_page)
        )
        result = await self.db.execute(stmt)
        users = result.scalars().all()

        return UserListResponse(
            users=[self._to_response(u) for u in users],
            total=total,
            page=page,
            per_page=per_page,
        )

    async def update_user(self, user_id: int, data: UserUpdate) ->
     UserResponse:
        stmt = select(User).where(User.id == user_id,
     User.deleted_at.is_(None))
        result = await self.db.execute(stmt)
        user = result.scalar_one_or_none()

        if user is None:
            raise NotFoundException("User")

        update_data = data.model_dump(exclude_unset=True)
        if update_data.get("github_username") == "":
            update_data["github_username"] = None
        for field, value in update_data.items():
            setattr(user, field, value)

        await self.db.flush()
        return await self.get_user_by_id(user_id)

    async def soft_delete_user(self, user_id: int) -> dict:
        stmt = select(User).where(User.id == user_id,
     User.deleted_at.is_(None))
        result = await self.db.execute(stmt)
        user = result.scalar_one_or_none()

        if user is None:
            raise NotFoundException("User")

        from datetime import datetime, timezone
        user.deleted_at = datetime.now(timezone.utc)
        user.status = "Inactive"
        await self.db.flush()
```

```
        return {"message": "User deleted successfully", "user_id": user_id}

    @staticmethod
    def _to_response(user: User) -> UserResponse:
        roles = [ur.role.role_name for ur in user.user_roles] if
     user.user_roles else []
        return UserResponse(
            id=user.id,
            name=user.name,
            username=user.username,
            email=user.email,
            phone=user.phone,
            gender=user.gender,
            dob=user.dob,
            is_verified=user.is_verified,
            status=user.status,
            profile=user.profile,
            roles=roles,
            github_username=user.github_username,
            created_at=user.created_at,
        )
```

backend/app/services/employee_service.py

```
from datetime import date, datetime, timezone

from sqlalchemy import func, select
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy.orm import selectinload

from app.core.security import hash_password
from app.exceptions import AlreadyExistsException, NotFoundException
from app.models.employee import Department, Designation, Employee,
     EmployeeDetail, WorkSchedule
from app.models.user import User, Role, UserRole
from app.schemas.employee import (
    DepartmentCreate,
    DepartmentResponse,
    DesignationCreate,
    DesignationResponse,
    EmployeeCreate,
    EmployeeDetailCreate,
    EmployeeListResponse,
    EmployeeResponse,
    EmployeeUpdate,
    WorkScheduleCreate,
    WorkScheduleResponse,
)

class DesignationService:
    def __init__(self, db: AsyncSession):
        self.db = db

    async def create(self, data: DesignationCreate) -> DesignationResponse:
        existing = await self.db.execute(
            select(Designation).where(
                Designation.name == data.name,
     Designation.deleted_at.is_(None)
            )
        )
        if existing.scalar_one_or_none():
            raise AlreadyExistsException("Designation")

        desig = Designation(name=data.name, description=data.description)
        self.db.add(desig)
        await self.db.flush()
        return DesignationResponse.model_validate(desig)
```

```
    async def list_all(self) -> list[DesignationResponse]:
        result = await self.db.execute(
            select(Designation)
            .where(Designation.deleted_at.is_(None))
            .order_by(Designation.name)
        )
        return [DesignationResponse.model_validate(d) for d in
     result.scalars().all()]

    async def get_by_id(self, desig_id: int) -> DesignationResponse:
        result = await self.db.execute(
            select(Designation).where(
                Designation.id == desig_id, Designation.deleted_at.is_(None)
            )
        )
        desig = result.scalar_one_or_none()
        if desig is None:
            raise NotFoundException("Designation")
        return DesignationResponse.model_validate(desig)

    async def delete(self, desig_id: int) -> dict:
        result = await self.db.execute(
            select(Designation).where(
                Designation.id == desig_id, Designation.deleted_at.is_(None)
            )
        )
        desig = result.scalar_one_or_none()
        if desig is None:
            raise NotFoundException("Designation")
        desig.deleted_at = datetime.now(timezone.utc)
        await self.db.flush()
        return {"message": "Designation deleted", "id": desig_id}

class DepartmentService:
    def __init__(self, db: AsyncSession):
        self.db = db

    async def create(self, data: DepartmentCreate) -> DepartmentResponse:
        existing = await self.db.execute(
            select(Department).where(
                Department.name == data.name,
     Department.deleted_at.is_(None)
            )
        )
        if existing.scalar_one_or_none():
            raise AlreadyExistsException("Department")

        dept = Department(name=data.name, description=data.description)
        self.db.add(dept)
        await self.db.flush()
        return DepartmentResponse.model_validate(dept)

    async def list_all(self) -> list[DepartmentResponse]:
        result = await self.db.execute(
            select(Department)
            .where(Department.deleted_at.is_(None))
            .order_by(Department.name)
        )
        return [DepartmentResponse.model_validate(d) for d in
     result.scalars().all()]

    async def get_by_id(self, dept_id: int) -> DepartmentResponse:
        result = await self.db.execute(
            select(Department).where(
                Department.id == dept_id, Department.deleted_at.is_(None)
            )
        )
        dept = result.scalar_one_or_none()
```

```
        if dept is None:
            raise NotFoundException("Department")
        return DepartmentResponse.model_validate(dept)

    async def delete(self, dept_id: int) -> dict:
        result = await self.db.execute(
            select(Department).where(
                Department.id == dept_id, Department.deleted_at.is_(None)
            )
        )
        dept = result.scalar_one_or_none()
        if dept is None:
            raise NotFoundException("Department")
        dept.deleted_at = datetime.now(timezone.utc)
        await self.db.flush()
        return {"message": "Department deleted", "id": dept_id}

class WorkScheduleService:
    def __init__(self, db: AsyncSession):
        self.db = db

    async def create(self, data: WorkScheduleCreate) ->
     WorkScheduleResponse:
        schedule = WorkSchedule(
            schedule_name=data.schedule_name,
            start_time=data.start_time,
            end_time=data.end_time,
            break_minutes=data.break_minutes,
        )
        self.db.add(schedule)
        await self.db.flush()
        return WorkScheduleResponse.model_validate(schedule)

    async def list_all(self) -> list[WorkScheduleResponse]:
        result = await self.db.execute(
            select(WorkSchedule)
            .where(WorkSchedule.deleted_at.is_(None))
            .order_by(WorkSchedule.schedule_name)
        )
        return [WorkScheduleResponse.model_validate(s) for s in
     result.scalars().all()]

    async def get_by_id(self, schedule_id: int) -> WorkScheduleResponse:
        result = await self.db.execute(
            select(WorkSchedule).where(
                WorkSchedule.id == schedule_id,
     WorkSchedule.deleted_at.is_(None)
            )
        )
        schedule = result.scalar_one_or_none()
        if schedule is None:
            raise NotFoundException("Work Schedule")
        return WorkScheduleResponse.model_validate(schedule)

class EmployeeService:
    def __init__(self, db: AsyncSession):
        self.db = db

    async def create(self, data: EmployeeCreate) -> EmployeeResponse:
        existing = await self.db.execute(
            select(Employee).where(
                Employee.employee_code == data.employee_code,
                Employee.deleted_at.is_(None),
            )
        )
        if existing.scalar_one_or_none():
            raise AlreadyExistsException("Employee with this code")
```

```
    target_user_id = data.user_id
    if not target_user_id:
        if not data.email or not data.password:
            raise ValueError("Either user_id or email & password must be
provided to create an employee.")

        user_res = await self.db.execute(select(User).where(User.email
== data.email))
        existing_user = user_res.scalar_one_or_none()
        if existing_user:
            target_user_id = existing_user.id
        else:
            uname = data.email.split("@")[0]
            user_name = data.name or uname
            new_user = User(
                name=user_name,
                username=uname,
                email=data.email,
                password=hash_password(data.password),
                is_verified=True,
                status="Active",
            )
            self.db.add(new_user)
            await self.db.flush()
            target_user_id = new_user.id

            role_name = data.role or "Employee"
            role_res = await
self.db.execute(select(Role).where(Role.role_name == role_name))
            role_obj = role_res.scalar_one_or_none()
            if not role_obj:
                role_obj = Role(role_name=role_name)
                self.db.add(role_obj)
                await self.db.flush()

            user_role = UserRole(user_id=new_user.id,
role_id=role_obj.id)
            self.db.add(user_role)
            await self.db.flush()

    detail_id = None
    if data.employee_detail:
        detail = EmployeeDetail(**data.employee_detail.model_dump())
        self.db.add(detail)
        await self.db.flush()
        detail_id = detail.id

    employee = Employee(
        user_id=target_user_id,
        employee_code=data.employee_code,
        manager_id=data.manager_id,
        department_id=data.department_id,
        designation_id=data.designation_id,
        work_schedule_id=data.work_schedule_id,
        employee_details_id=detail_id,
        joining_date=data.joining_date or date.today(),
        status="Active",
    )
    self.db.add(employee)
    await self.db.flush()
    return await self.get_by_id(employee.id)

async def get_by_id(self, employee_id: int) -> EmployeeResponse:
    stmt = (
        select(Employee)
        .where(Employee.id == employee_id,
Employee.deleted_at.is_(None))
        .options(
```

```
            selectinload(Employee.employee_detail),
            selectinload(Employee.department),
            selectinload(Employee.designation),
            selectinload(Employee.work_schedule),

selectinload(Employee.user).selectinload(User.user_roles).selectinload(U
serRole.role),
        )
    )
    result = await self.db.execute(stmt)
    employee = result.scalar_one_or_none()
    if employee is None:
        raise NotFoundException("Employee")

    resp = EmployeeResponse.model_validate(employee)
    if employee.user:
        role_name = "Employee"
        if employee.user.user_roles:
            r = employee.user.user_roles[0]
            if hasattr(r, 'role') and r.role:
                role_name = r.role.role_name
        resp.user = {
            "id": employee.user.id,
            "name": employee.user.name,
            "username": employee.user.username,
            "email": employee.user.email,
            "role": role_name,
        }
    return resp

async def get_by_user_id(self, user_id: int) -> EmployeeResponse:
    stmt = (
        select(Employee)
        .where(Employee.user_id == user_id,
Employee.deleted_at.is_(None))
        .options(
            selectinload(Employee.employee_detail),
            selectinload(Employee.department),
            selectinload(Employee.designation),
            selectinload(Employee.work_schedule),

selectinload(Employee.user).selectinload(User.user_roles).selectinload(U
serRole.role),
        )
    )
    result = await self.db.execute(stmt)
    employee = result.scalar_one_or_none()
    if employee is None:
        raise NotFoundException("Employee")
    return await self.get_by_id(employee.id)

async def list_employees(
    self,
    page: int = 1,
    per_page: int = 20,
    department_id: int | None = None,
    status: str | None = None,
) -> EmployeeListResponse:
    offset = (page - 1) * per_page
    conditions = [Employee.deleted_at.is_(None)]
    if department_id:
        conditions.append(Employee.department_id == department_id)
    if status:
        conditions.append(Employee.status == status)

    count_stmt = select(func.count(Employee.id)).where(*conditions)
    total = (await self.db.execute(count_stmt)).scalar() or 0
```

```
        stmt = (
            select(Employee)
            .where(*conditions)
            .options(
                selectinload(Employee.employee_detail),
                selectinload(Employee.department),
                selectinload(Employee.designation),
                selectinload(Employee.work_schedule),

    selectinload(Employee.user).selectinload(User.user_roles).selectinload(U
    serRole.role),
            )
            .order_by(Employee.created_at.desc())
            .offset(offset)
            .limit(per_page)
        )
        result = await self.db.execute(stmt)
        employees = result.scalars().all()

        emp_responses = []
        for e in employees:
            resp = EmployeeResponse.model_validate(e)
            if e.user:
                role_name = "Employee"
                if e.user.user_roles:
                    r = e.user.user_roles[0]
                    if hasattr(r, 'role') and r.role:
                        role_name = r.role.role_name
                resp.user = {
                    "id": e.user.id,
                    "name": e.user.name,
                    "username": e.user.username,
                    "email": e.user.email,
                    "role": role_name,
                }
            emp_responses.append(resp)

        return EmployeeListResponse(
            employees=emp_responses,
            total=total,
            page=page,
            per_page=per_page,
        )

    async def update(self, employee_id: int, data: EmployeeUpdate) ->
     EmployeeResponse:
        stmt = select(Employee).where(
            Employee.id == employee_id, Employee.deleted_at.is_(None)
        )
        result = await self.db.execute(stmt)
        employee = result.scalar_one_or_none()
        if employee is None:
            raise NotFoundException("Employee")

        for field, value in data.model_dump(exclude_unset=True).items():
            setattr(employee, field, value)

        await self.db.flush()
        return await self.get_by_id(employee_id)

    async def deactivate(self, employee_id: int) -> dict:
        stmt = select(Employee).where(
            Employee.id == employee_id, Employee.deleted_at.is_(None)
        )
        result = await self.db.execute(stmt)
        employee = result.scalar_one_or_none()
        if employee is None:
            raise NotFoundException("Employee")
```

```
        employee.status = "Inactive"
        employee.deleted_at = datetime.now(timezone.utc)
        await self.db.flush()
        return {"message": "Employee deactivated", "id": employee_id}
```

backend/app/services/project_service.py

```
from datetime import date, datetime, timedelta, timezone

from sqlalchemy import case, func, select
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy.orm import selectinload

from app.core.project_access import MEMBER, PROJECT_MANAGER, ProjectAccess,
     platform_roles
from app.exceptions import NotFoundException, TraceException
from app.models.project import Project, ProjectAssignment,
     ProjectSourceFile, TaskDiscussion, TaskGroup
from app.models.task import Task
from app.models.user import User, UserRole
from app.services import jobs
from app.services import notification_service as notifications
from app.services.github_service import GitHubClient, GitHubError, repo_slug

OPEN_STATUSES = ("Pending", "In Progress", "In Review", "Blocked",
     "Overdue")

def person(user: User | None) -> dict | None:
    if user is None:
        return None
    return {"id": user.id, "name": user.name, "email": user.email,
            "github_username": user.github_username}

async def create_project(db: AsyncSession, user: User, data, github:
     GitHubClient, owner: str) -> Project:
    slug = repo_slug(data.name)
    if not slug:
        raise TraceException(422, "The project name needs at least one
     letter or digit to name its repository")
    repo = await github.create_repo(owner, slug, data.description)
    project = Project(created_by_user_id=user.id, status="Draft",
     name=data.name,
                      description=data.description,
     start_date=data.start_date, end_date=data.end_date,
                      github_repo=repo.full_name,
     github_default_branch=repo.default_branch)
    db.add(project)
    await db.commit()
    return project

async def delete_project(db: AsyncSession, project: Project, github:
     GitHubClient | None, owner: str) -> None:
    repo = project.github_repo
    if repo and owner and repo.split("/")[0].lower() == owner.lower():
        from app.services.github_service import require_github
        await require_github(github).archive_repo(repo)
    project.deleted_at = datetime.now(timezone.utc)
    await db.commit()

async def project_stats(db: AsyncSession, project_ids: list[int]) ->
     dict[int, dict]:
    if not project_ids:
        return {}
    today = date.today()
    live = (Task.deleted_at.is_(None), Task.status != "Cancelled")
    rows = (await db.execute(
        select(
```

```
            Task.project_id,
            func.count(Task.id),
            func.count(case((Task.status == "Completed", 1))),
            func.count(case((
                (Task.status == "Overdue")
                | ((Task.due_date < today) &
     Task.status.in_(OPEN_STATUSES)), 1,
            ))),
        ).where(Task.project_id.in_(project_ids),
     *live).group_by(Task.project_id)
    )).all()
    groups = dict((await db.execute(
        select(TaskGroup.project_id, func.count(TaskGroup.id))
        .where(TaskGroup.project_id.in_(project_ids),
     TaskGroup.deleted_at.is_(None))
        .group_by(TaskGroup.project_id)
    )).all())
    stats = {pid: {"task_groups": groups.get(pid, 0), "total_tasks": 0,
     "completed_tasks": 0,
                   "overdue_tasks": 0, "completion_percentage": 0.0} for pid
     in project_ids}
    for pid, total, completed, overdue in rows:
        stats[pid].update(total_tasks=total, completed_tasks=completed,
     overdue_tasks=overdue,
                          completion_percentage=round(completed / total *
     100, 1) if total else 0.0)
    return stats

async def project_managers(db: AsyncSession, project_ids: list[int]) ->
     dict[int, list[dict]]:
    result = {pid: [] for pid in project_ids}
    if not project_ids:
        return result
    rows = (await db.execute(
        select(ProjectAssignment.project_id, User)
        .join(User, User.id == ProjectAssignment.user_id)
        .where(ProjectAssignment.project_id.in_(project_ids),
               ProjectAssignment.project_role == PROJECT_MANAGER,
               ProjectAssignment.deleted_at.is_(None))
        .order_by(User.name)
    )).all()
    for pid, user in rows:
        result[pid].append(person(user))
    return result

def is_at_risk(project: Project, stats: dict) -> bool:
    if project.status in ("Completed", "Archived"):
        return False
    past_end = project.end_date is not None and project.end_date <
     date.today()
    return stats["overdue_tasks"] > 0 or (past_end and
     stats["completion_percentage"] < 100)

async def visible_projects(db: AsyncSession, user: User) ->
     tuple[list[Project], dict[int, str]]:
    roles = platform_roles(user)
    stmt =
     select(Project).where(Project.deleted_at.is_(None)).options(selectinload
     (Project.created_by))
    my_roles: dict[int, str] = {}
    if "Admin" in roles:
        pass
    elif "Manager" in roles:
        stmt = stmt.where(Project.created_by_user_id == user.id)
    else:
        assignments = (await db.execute(
            select(ProjectAssignment.project_id,
     ProjectAssignment.project_role).where(
```

```
                ProjectAssignment.user_id == user.id,
     ProjectAssignment.deleted_at.is_(None))
        )).all()
        my_roles = dict(assignments)
        stmt = stmt.where(Project.id.in_(list(my_roles) or [-1]))
    projects = (await
     db.execute(stmt.order_by(Project.created_at.desc()))).scalars().all()
    return list(projects), my_roles

async def summarize(db: AsyncSession, projects: list[Project], my_roles:
     dict[int, str]) -> list[dict]:
    ids = [p.id for p in projects]
    stats, pms = await project_stats(db, ids), await project_managers(db,
     ids)
    return [{
        "id": p.id, "name": p.name, "description": p.description, "status":
     p.status,
        "github_repo": p.github_repo, "start_date": p.start_date,
     "end_date": p.end_date,
        "created_at": p.created_at, "owner": person(p.created_by),
        "project_managers": pms[p.id], "my_role": my_roles.get(p.id),
        "stats": stats[p.id], "at_risk": is_at_risk(p, stats[p.id]),
    } for p in projects]

def source_file_dict(f: ProjectSourceFile) -> dict:
    return {"id": f.id, "file_name": f.file_name, "content_type":
     f.content_type,
            "size_bytes": f.size_bytes, "extracted_chars":
     len(f.extracted_text or ""),
            "uploaded_at": f.created_at}

async def project_detail(db: AsyncSession, access: ProjectAccess) -> dict:
    project = (await db.execute(
        select(Project).where(Project.id == access.project.id)
        .options(selectinload(Project.created_by),
     selectinload(Project.source_files))
    )).scalar_one()
    my_role = {project.id: access.project_role} if access.project_role else
     {}
    summary = (await summarize(db, [project], my_role))[0]
    return {
        **summary,
        "github_default_branch": project.github_default_branch,
        "last_evaluated_at": project.last_evaluated_at,
        "ai_errors": project.ai_errors or {},
        "documents": {k: getattr(project, k) for k in (
            "project_initial_doc", "project_doc", "project_progress_doc",
     "project_report_doc")},
        "source_files": [source_file_dict(f) for f in sorted(
            (f for f in project.source_files if f.deleted_at is None),
     key=lambda f: f.id)],
    }

async def active_employees(db: AsyncSession) -> list[User]:
    from app.models.user import Role
    return list((await db.execute(
        select(User).join(UserRole, UserRole.user_id == User.id).join(Role,
     Role.id == UserRole.role_id)
        .where(Role.role_name == "Employee", User.deleted_at.is_(None),
     User.status == "Active")
        .order_by(User.name)
    )).scalars().unique().all())

async def active_employee(db: AsyncSession, user_id: int) -> User:
    user = (await db.execute(
        select(User).where(User.id == user_id, User.deleted_at.is_(None))
        .options(selectinload(User.user_roles).selectinload(UserRole.role))
    )).scalar_one_or_none()
```

```
    if user is None:
        raise NotFoundException("User")
    if "Employee" not in platform_roles(user) or user.status != "Active":
        raise TraceException(422, "Only an active Employee can hold a
     project role")
    return user

async def grant_repo_access(github: GitHubClient | None, project: Project,
     user: User) -> list[str]:
    if not project.github_repo:
        return []
    if not user.github_username:
        return [f"{user.name} has no GitHub username, so they were not given
     access to {project.github_repo}"]
    if github is None:
        return [f"GitHub is not configured, so {user.name} was not given
     access to {project.github_repo}"]
    try:
        await github.add_collaborator(project.github_repo,
     user.github_username)
    except GitHubError as exc:
        return [f"{user.name} was not given access to {project.github_repo}:
     {exc.detail}"]
    return []

async def revoke_repo_access(github: GitHubClient | None, project: Project,
     user: User) -> list[str]:
    if not project.github_repo or not user.github_username:
        return []
    if github is None:
        return [f"GitHub is not configured, so {user.name}'s access to
     {project.github_repo} was not removed"]
    try:
        await github.remove_collaborator(project.github_repo,
     user.github_username)
    except GitHubError as exc:
        return [f"{user.name}'s access to {project.github_repo} was not
     removed: {exc.detail}"]
    return []

async def ensure_member(db: AsyncSession, project: Project, user: User,
     assigned_by: int,
                        github: GitHubClient | None) -> list[str]:
    project_id, user_id = project.id, user.id
    existing = (await db.execute(
        select(ProjectAssignment).where(ProjectAssignment.project_id ==
     project_id,
                                        ProjectAssignment.user_id ==
     user_id)
    )).scalar_one_or_none()
    if existing is None:
        db.add(ProjectAssignment(project_id=project_id, user_id=user_id,
                                 project_role=MEMBER,
     assigned_by_user_id=assigned_by))
    elif existing.deleted_at is not None:
        existing.deleted_at = None
        existing.project_role = MEMBER
        existing.assigned_by_user_id = assigned_by
    await db.flush()
    return await grant_repo_access(github, project, user)

async def notify_assignment(db: AsyncSession, task: Task, project: Project,
     assignee_id: int,
                            actor_id: int) -> notifications.Outbox:
    return await notifications.notify(
        db, [assignee_id], "task_assigned", f"New task: {task.title}",
        f"You were assigned '{task.title}' in {project.name}.", task=task,
     exclude=actor_id)
```

```
def issue_body(task: Task, project: Project) -> str:
    parts = [task.description.strip()] if task.description else []
    parts.append(
        "---\n"
        f"Managed by TRACE · project **{project.name}**.\n\n"
        "Update the task's status in TRACE — marking it Completed there
     closes this issue."
    )
    return "\n\n".join(parts)

async def load_task(db: AsyncSession, task_id: int) -> Task:
    return (await db.execute(
        select(Task).where(Task.id == task_id).options(
            selectinload(Task.project), selectinload(Task.task_group),
            selectinload(Task.assigned_to_user),
     selectinload(Task.assigned_by_user),
        ).execution_options(populate_existing=True)
    )).scalar_one()

def task_dict(task: Task, access: ProjectAccess | None = None) -> dict:
    return {
        "id": task.id, "project_id": task.project_id,
        "project_name": task.project.name if task.project else "",
        "task_group_id": task.task_group_id,
        "task_group_name": task.task_group.name if task.task_group else
     None,
        "title": task.title, "description": task.description, "priority":
     task.priority,
        "status": task.status, "due_date": task.due_date,
        "estimated_hours": float(task.estimated_hours) if
     task.estimated_hours is not None else None,
        "github_issue_id": task.github_issue_id, "github_issue_url":
     task.github_issue_url,
        "task_progress_doc": task.task_progress_doc,
        "progress_doc_error": task.progress_doc_error,
        "assigned_to": person(task.assigned_to_user), "assigned_by":
     person(task.assigned_by_user),
        "completed_by": task.completed_by, "completed_at":
     task.completed_at,
        "health_score": task.health_score, "health_reason":
     task.health_reason,
        "created_at": task.created_at, "updated_at": task.updated_at,
        "can_discuss": bool(access and access.may_discuss(task)),
        "can_edit": bool(access and access.is_project_manager),
        "allowed_statuses": access.allowed_statuses(task) if access else [],
    }

async def group_in_project(db: AsyncSession, project_id: int, group_id: int)
     -> TaskGroup:
    group = (await db.execute(
        select(TaskGroup).where(TaskGroup.id == group_id,
     TaskGroup.project_id == project_id,
                                TaskGroup.deleted_at.is_(None))
    )).scalar_one_or_none()
    if group is None:
        raise TraceException(422, "task_group_id does not belong to this
     project")
    return group

async def recompute_group(db: AsyncSession, group_id: int | None) -> None:
    if group_id is None:
        return
    await db.flush()
    group = await db.get(TaskGroup, group_id)
    statuses = (await db.execute(
        select(Task.status).where(Task.task_group_id == group_id,
     Task.deleted_at.is_(None),
```

```
                                    Task.status != "Cancelled")
    )).scalars().all()
    done = sum(1 for s in statuses if s == "Completed")
    group.completion_percentage = round(done / len(statuses) * 100, 2) if
     statuses else 0
    if statuses and done == len(statuses):
        group.status = "Completed"
    elif any(s in ("In Progress", "In Review", "Completed") for s in
     statuses):
        group.status = "In Progress"
    elif statuses and all(s == "Blocked" for s in statuses):
        group.status = "Blocked"
    else:
        group.status = "Pending"
    today = date.today()
    if group.status != "Pending" and group.actual_start_date is None:
        group.actual_start_date = today
    if group.status == "Completed":
        group.actual_end_date = group.actual_end_date or today
    else:
        group.actual_end_date = None

def _pending_invitation(assignee: User, project: Project) -> str:
    return (f"GitHub would not assign the issue to
     {assignee.github_username} — their invitation to "
            f"{project.github_repo} is probably not accepted yet — so it was
     left unassigned. "
            f"Assign the task again once they have accepted")

async def _open_issue(github: GitHubClient, project: Project, task: Task,
                      assignee: User | None) -> tuple[object, list[str]]:
    assignees = [assignee.github_username] if assignee and
     assignee.github_username else []
    try:
        return await github.create_issue(project.github_repo, task.title,
     issue_body(task, project), assignees), []
    except GitHubError as exc:
        if not (assignees and exc.github_status == 422):
            raise
    issue = await github.create_issue(project.github_repo, task.title,
     issue_body(task, project), [])
    return issue, [_pending_invitation(assignee, project)]

def _issue_number(task: Task) -> int | None:
    return int(task.github_issue_id) if task.github_issue_id and
     task.github_issue_id.isdigit() else None

async def create_task(db: AsyncSession, access: ProjectAccess, data, github:
     GitHubClient) -> tuple[Task, list[str]]:
    project, warnings = access.project, []
    if not project.github_repo:
        raise TraceException(409, "This project has no GitHub repository;
     set one before creating tasks")
    group = await group_in_project(db, project.id, data.task_group_id)
    assignee = await active_employee(db, data.assigned_to_user_id) if
     data.assigned_to_user_id else None

    task = Task(
        project_id=project.id, task_group_id=group.id, title=data.title,
        description=data.description, priority=data.priority,
     due_date=data.due_date,
        estimated_hours=data.estimated_hours, status="Pending",
        assigned_to_user_id=assignee.id if assignee else None,
        assigned_by_user_id=access.user.id if assignee else None,
    )
    db.add(task)
    await db.flush()
```

```
    try:
        if assignee:
            warnings += await ensure_member(db, project, assignee,
     access.user.id, github)
            if not assignee.github_username:
                warnings = [f"{assignee.name} has no GitHub username, so the
     issue was left unassigned"]
        issue, issue_warnings = await _open_issue(github, project, task,
     assignee)
    except Exception:
        await db.rollback()
        raise
    warnings += issue_warnings
    task.github_issue_id, task.github_issue_url = str(issue.number),
     issue.url

    outbox = await notify_assignment(db, task, project, assignee.id,
     access.user.id) if assignee else None
    await recompute_group(db, group.id)
    await db.commit()
    jobs.enqueue("generate_task_progress_doc", task.id)
    if outbox:
        notifications.deliver(outbox)
    return await load_task(db, task.id), warnings

async def update_task(db: AsyncSession, access: ProjectAccess, task: Task,
     data) -> Task:
    changes = data.model_dump(exclude_unset=True)
    old_group = task.task_group_id
    if "task_group_id" in changes:
        await group_in_project(db, access.project.id,
     changes["task_group_id"])
    for field, value in changes.items():
        setattr(task, field, value)
    await recompute_group(db, old_group)
    if task.task_group_id != old_group:
        await recompute_group(db, task.task_group_id)
    await db.commit()
    return await load_task(db, task.id)

async def assign_task(db: AsyncSession, access: ProjectAccess, task: Task,
     user_id: int | None,
                      github: GitHubClient) -> tuple[Task, list[str]]:
    warnings, assignees, project = [], [], access.project
    assignee = await active_employee(db, user_id) if user_id else None
    if assignee:
        warnings += await ensure_member(db, project, assignee,
     access.user.id, github)
        if assignee.github_username:
            assignees = [assignee.github_username]
        else:
            warnings = [f"{assignee.name} has no GitHub username, so the
     issue was left unassigned"]
    number = _issue_number(task)
    if number is not None:
        try:
            await github.set_assignees(project.github_repo, number,
     assignees)
        except GitHubError as exc:
            if not (assignees and exc.github_status == 422):
                raise
            await github.set_assignees(project.github_repo, number, [])
            warnings.append(_pending_invitation(assignee, project))

    task.assigned_to_user_id = assignee.id if assignee else None
    task.assigned_by_user_id = access.user.id if assignee else None
    outbox = await notify_assignment(db, task, project, assignee.id,
     access.user.id) if assignee else None
```

```
    await db.commit()
    if outbox:
        notifications.deliver(outbox)
    return await load_task(db, task.id), warnings

async def set_status(db: AsyncSession, access: ProjectAccess, task: Task,
     new_status: str,
                     github: GitHubClient | None) -> Task:
    access.may_set_status(task, new_status)
    completion_changed = (task.status == "Completed") != (new_status ==
     "Completed")
    was_closed = task.status in ("Completed", "Cancelled")
    closes = new_status in ("Completed", "Cancelled")
    number = _issue_number(task)
    if number is not None and was_closed != closes:
        from app.services.github_service import require_github
        await require_github(github).set_state(
            access.project.github_repo, number, open_=not closes,
     not_planned=new_status == "Cancelled")

    task.status = new_status
    outbox = None
    if new_status == "Completed":
        task.completed_by = "Assignee" if access.is_assignee(task) else
     "Project Manager"
        task.completed_at = datetime.now(timezone.utc)
        outbox = await notifications.notify(
            db, [*await notifications.project_managers(db, task.project_id),
     task.assigned_to_user_id],
            "task_completed", f"Completed: {task.title}",
            f"'{task.title}' in {access.project.name} was marked complete by
     {access.user.name}.",
            task=task, exclude=access.user.id)
    else:
        task.completed_by = None
        task.completed_at = None
    await recompute_group(db, task.task_group_id)
    await db.commit()
    if outbox:
        notifications.deliver(outbox)
    if completion_changed:
        from app.services.gantt_service import request_reevaluation
        request_reevaluation(access.project.id, "task_status_change")
    return await load_task(db, task.id)

async def delete_task(db: AsyncSession, access: ProjectAccess, task: Task,
     github: GitHubClient | None) -> None:
    number = _issue_number(task)
    if number is not None and task.status not in ("Completed", "Cancelled"):
        from app.services.github_service import require_github
        await require_github(github).set_state(access.project.github_repo,
     number, open_=False, not_planned=True)
    task.deleted_at = datetime.now(timezone.utc)
    await recompute_group(db, task.task_group_id)
    await db.commit()

async def _notify_due(db: AsyncSession, tasks: list[Task], title: str, body:
     str) -> notifications.Outbox:
    outbox = notifications.Outbox()
    for task in tasks:
        recipients = [task.assigned_to_user_id, *await
     notifications.project_managers(db, task.project_id)]
        outbox += await notifications.notify(
            db, recipients, "task_due", title.format(title=task.title),
            body.format(title=task.title, project=task.project.name,
     due=task.due_date), task=task)
    return outbox
```

```
async def mark_overdue_tasks(db: AsyncSession, today: date) -> dict:
    from app.services.gantt_service import request_reevaluation

    tasks = (await db.execute(
        select(Task).where(
            Task.due_date < today, Task.status.in_(("Pending", "In
     Progress", "In Review", "Blocked")),
            Task.deleted_at.is_(None), Task.project_id.is_not(None),
        ).options(selectinload(Task.project)).order_by(Task.id)
    )).scalars().all()
    for task in tasks:
        task.status = "Overdue"
    outbox = await _notify_due(db, tasks, "Overdue: {title}",
                               "'{title}' in {project} was due {due} and is
     not complete.")
    await db.commit()
    notifications.deliver(outbox)
    projects = sorted({t.project_id for t in tasks})
    for project_id in projects:
        request_reevaluation(project_id, "task_status_change")
    return {"status": "success", "date": today.isoformat(), "tasks": [t.id
     for t in tasks], "projects": projects}

async def task_due_reminders(db: AsyncSession, today: date) -> dict:
    tomorrow = today + timedelta(days=1)
    tasks = (await db.execute(
        select(Task).where(
            Task.due_date == tomorrow, Task.status.in_(OPEN_STATUSES),
            Task.deleted_at.is_(None), Task.project_id.is_not(None),
        ).options(selectinload(Task.project)).order_by(Task.id)
    )).scalars().all()
    outbox = await _notify_due(db, tasks, "Due tomorrow: {title}",
     "'{title}' in {project} is due {due}.")
    await db.commit()
    notifications.deliver(outbox)
    return {"status": "success", "date": today.isoformat(), "tasks": [t.id
     for t in tasks]}

def discussion_dict(d: TaskDiscussion) -> dict:
    return {"id": d.id, "task_id": d.task_id, "parent_id": d.parent_id,
     "message": d.message,
            "author": person(d.user), "created_at": d.created_at}

async def list_discussions(db: AsyncSession, task: Task) -> list[dict]:
    rows = (await db.execute(
        select(TaskDiscussion).where(TaskDiscussion.task_id == task.id,
     TaskDiscussion.deleted_at.is_(None))
        .options(selectinload(TaskDiscussion.user)).order_by(TaskDiscussion.
     id)
    )).scalars().all()
    return [discussion_dict(d) for d in rows]

async def post_discussion(db: AsyncSession, access: ProjectAccess, task:
     Task, message: str,
                          parent_id: int | None) -> dict:
    if parent_id is not None:
        parent = await db.get(TaskDiscussion, parent_id)
        if parent is None or parent.task_id != task.id or parent.deleted_at
     is not None:
            raise TraceException(422, "The message you are replying to is
     not on this task")
        parent_id = parent.parent_id or parent.id
    entry = TaskDiscussion(task_id=task.id, user_id=access.user.id,
     parent_id=parent_id, message=message.strip())
    db.add(entry)
    await db.flush()

    authors = (await db.execute(
```

```
        select(TaskDiscussion.user_id).where(TaskDiscussion.task_id ==
    task.id,

    TaskDiscussion.deleted_at.is_(None)).distinct()
    )).scalars().all()
    outbox = await notifications.notify(
        db, [*authors, task.assigned_to_user_id, task.assigned_by_user_id],
    "discussion_reply",
        f"{access.user.name} on {task.title}", message.strip()[:280],
    task=task,
        email=False, exclude=access.user.id)
    await db.commit()
    notifications.deliver(outbox)
    viewers = [access.project.created_by_user_id, task.assigned_to_user_id,
    task.assigned_by_user_id,
               *await notifications.project_managers(db, task.project_id)]
    notifications.discussion_event(viewers, task.id)
    await db.refresh(entry, ["user"])
    return discussion_dict(entry)
```

backend/app/services/github_service.py

```
import base64
import re
from dataclasses import dataclass

import httpx
from fastapi import Depends
from sqlalchemy.ext.asyncio import AsyncSession

from app.database import get_db
from app.exceptions import TraceException
from app.services.system_settings import resolve_setting

REPO_PATTERN = re.compile(r"^[A-Za-z0-9](?:[A-Za-z0-9-]{0,38})/[A-Za-z0-
    9._-]{1,100}$")
OWNER_PATTERN = re.compile(r"^[A-Za-z0-9](?:[A-Za-z0-9-]{0,38})$")

def repo_slug(name: str) -> str:
    slug = re.sub(r"[^a-z0-9]+", "-", name.lower()).strip("-")
    return slug[:100].rstrip("-")

class GitHubNotConfigured(TraceException):
    def __init__(self):
        super().__init__(
            status_code=503,
            detail=(
                "GitHub is not configured, so no issue can be created. "
                "Set github.token in Admin → Configuration, or GITHUB_TOKEN
    in .env."
            ),
        )

class GitHubError(TraceException):
    def __init__(self, action: str, response: httpx.Response | None = None,
    reason: str | None = None):
        if response is not None:
            try:
                message = response.json().get("message",
    response.text[:200])
            except ValueError:
                message = response.text[:200]
            reason = f"{response.status_code} {message}"
        self.github_status = response.status_code if response is not None
    else None
        self.github_body = response.text if response is not None else ""
        super().__init__(status_code=502, detail=f"GitHub refused to
    {action}: {reason}")
```

```
class GitHubOwnerNotConfigured(TraceException):
    def __init__(self):
        super().__init__(
            status_code=503,
            detail=(
                "No GitHub organization is configured, so no repository can
     be created. "
                "Set it in Admin → Configuration → GitHub, or GITHUB_OWNER
     in .env."
            ),
        )

@dataclass
class Issue:
    number: int
    url: str

@dataclass
class Repo:
    full_name: str
    url: str
    default_branch: str

class GitHubClient:
    def __init__(self, token: str, api_url: str = "https://api.github.com",
                 transport: httpx.AsyncBaseTransport | None = None,
                 committer: dict | None = None):
        self._token = token
        self._api_url = api_url.rstrip("/")
        self._transport = transport
        self._committer = committer

    def _client(self) -> httpx.AsyncClient:
        return httpx.AsyncClient(
            base_url=self._api_url,
            transport=self._transport,
            timeout=15.0,
            headers={
                "Authorization": f"Bearer {self._token}",
                "Accept": "application/vnd.github+json",
                "X-GitHub-Api-Version": "2022-11-28",
            },
        )

    async def _request(self, action: str, method: str, path: str, **kwargs)
     -> httpx.Response:
        try:
            async with self._client() as client:
                response = await client.request(method, path, **kwargs)
        except httpx.HTTPError as exc:
            raise GitHubError(action, reason=f"network error
     ({exc.__class__.__name__})") from exc
        if response.status_code >= 400:
            raise GitHubError(action, response)
        return response

    async def authenticated_login(self) -> str:
        return (await self._request("authenticate the token", "GET",
     "/user")).json()["login"]

    async def organization(self, org: str) -> dict:
        return (await self._request(f"show the organization '{org}'", "GET",
     f"/orgs/{org}")).json()

    async def create_repo(self, org: str, name: str, description: str |
     None) -> Repo:
```

```
    payload = {"name": name, "private": False, "auto_init": True,
"has_issues": True}
    if description:
        payload["description"] = " ".join(description.split())[:350]
    try:
        data = (await self._request("create the repository", "POST",
f"/orgs/{org}/repos",
                                    json=payload)).json()
    except GitHubError as exc:
        if exc.github_status == 422 and "already exists" in
exc.github_body:
            raise TraceException(
                409, f"A repository named '{org}/{name}' already exists
— rename the project",
            ) from None
        if exc.github_status in (403, 404):
            raise TraceException(
                502,
                f"The GitHub token cannot create repositories in '{org}'
({exc.github_status}). "
                "Give it Administration: read and write on that
organization.",
            ) from None
        raise
    return Repo(full_name=data["full_name"], url=data["html_url"],
                default_branch=data.get("default_branch") or "main")

async def archive_repo(self, repo: str) -> bool:
    try:
        await self._request("archive the repository", "PATCH",
f"/repos/{repo}", json={"archived": True})
    except GitHubError as exc:
        if exc.github_status == 404:
            return False
        raise
    return True

async def add_collaborator(self, repo: str, username: str) -> bool:
    response = await self._request(f"invite '{username}' to the
repository", "PUT",

f"/repos/{repo}/collaborators/{username}",
                                   json={"permission": "push"})
    return response.status_code == 201

async def remove_collaborator(self, repo: str, username: str) -> None:
    try:
        await self._request(f"remove '{username}' from the repository",
"DELETE",
                            f"/repos/{repo}/collaborators/{username}")
    except GitHubError as exc:
        if exc.github_status != 404:
            raise
    invitations = (await self._request("list the repository
invitations", "GET",

f"/repos/{repo}/invitations")).json()
    for invitation in invitations:
        if (invitation.get("invitee") or {}).get("login", "").lower() ==
username.lower():
            await self._request(f"withdraw the invitation for
'{username}'", "DELETE",

f"/repos/{repo}/invitations/{invitation['id']}")

async def create_issue(self, repo: str, title: str, body: str,
                       assignees: list[str] | None = None) -> Issue:
    payload = {"title": title, "body": body}
```

```
    if assignees:
        payload["assignees"] = assignees
    data = (await self._request("create the issue", "POST",
f"/repos/{repo}/issues", json=payload)).json()
    return Issue(number=data["number"], url=data["html_url"])

async def set_assignees(self, repo: str, number: int, assignees:
list[str]) -> None:
    await self._request("assign the issue", "PATCH",
f"/repos/{repo}/issues/{number}",
                        json={"assignees": assignees})

async def set_state(self, repo: str, number: int, open_: bool,
not_planned: bool = False) -> None:
    payload = {"state": "open"} if open_ else {
        "state": "closed", "state_reason": "not_planned" if not_planned
else "completed",
    }
    await self._request("update the issue state", "PATCH",
f"/repos/{repo}/issues/{number}", json=payload)

async def read_file(self, repo: str, path: str, branch: str = "main") ->
str | None:
    try:
        async with self._client() as client:
            response = await
client.get(f"/repos/{repo}/contents/{path}", params={"ref": branch})
    except httpx.HTTPError as exc:
        raise GitHubError("read the file", reason=f"network error
({exc.__class__.__name__})") from exc
    if response.status_code == 404:
        return None
    if response.status_code >= 400:
        raise GitHubError("read the file", response)
    return base64.b64decode(response.json().get("content",
"")).decode("utf-8", errors="replace")

async def commit_file(self, repo: str, path: str, content: str, message:
str,
                      branch: str = "main") -> str:
    existing_sha = None
    try:
        async with self._client() as client:
            current = await client.get(f"/repos/{repo}/contents/{path}",
params={"ref": branch})
    except httpx.HTTPError as exc:
        raise GitHubError("read the file", reason=f"network error
({exc.__class__.__name__})") from exc
    if current.status_code == 200:
        existing_sha = current.json().get("sha")
    elif current.status_code != 404:
        raise GitHubError("read the file", current)

    payload = {
        "message": message,
        "content": base64.b64encode(content.encode()).decode(),
        "branch": branch,
    }
    if existing_sha:
        payload["sha"] = existing_sha
    if self._committer:
        payload["committer"] = self._committer
        payload["author"] = self._committer
    data = (await self._request("commit the file", "PUT",
f"/repos/{repo}/contents/{path}", json=payload)).json()
    return data["content"]["html_url"]
```

```
async def get_github_client(db: AsyncSession = Depends(get_db)) ->
     GitHubClient | None:
    token = await resolve_setting(db, "github", "token")
    if not token:
        return None
    name = await resolve_setting(db, "github", "committer_name")
    email = await resolve_setting(db, "github", "committer_email")
    return GitHubClient(
        token,
        await resolve_setting(db, "github", "api_url") or
     "https://api.github.com",
        committer={"name": name, "email": email} if name and email else
     None,
    )

async def resolve_owner(db: AsyncSession) -> str:
    owner = await resolve_setting(db, "github", "owner")
    if not owner:
        raise GitHubOwnerNotConfigured()
    return owner

def require_github(client: GitHubClient | None) -> GitHubClient:
    if client is None:
        raise GitHubNotConfigured()
    return client
```

backend/app/services/ai_service.py

```
import json
import logging

from pydantic import BaseModel, ValidationError
from app.exceptions import TraceException
from app.config import get_settings

logger = logging.getLogger("trace.ai")

MAX_RETRIES = 2

class AINotConfigured(TraceException):
    def __init__(self):
        super().__init__(
            status_code=503,
            detail="AI is not configured. Set OPENAI_API_KEY in .env.",
        )

class AIOutputInvalid(Exception):
    def __init__(self, schema: str, errors: str):
        super().__init__(f"The AI returned invalid {schema} after
     {MAX_RETRIES + 1} attempts: {errors}")

class AIClient:
    def __init__(self, api_key: str, model: str, base_url: str | None =
     None):
        from openai import AsyncOpenAI

        self.model = model
        self._client = AsyncOpenAI(api_key=api_key, base_url=base_url or
     None, timeout=120)

    async def chat_json(self, messages: list[dict], schema_name: str) ->
     str:
        response = await self._client.chat.completions.create(
            model=self.model, messages=messages, response_format={"type":
     "json_object"},
        )
        return response.choices[0].message.content or ""
```

```
def ai_configured() -> bool:
    return bool(get_settings().OPENAI_API_KEY)

def client_from_settings() -> AIClient:
    settings = get_settings()
    if not settings.OPENAI_API_KEY:
        raise AINotConfigured()
    return AIClient(settings.OPENAI_API_KEY, settings.OPENAI_MODEL,
     settings.OPENAI_BASE_URL)

def _errors(exc: ValidationError) -> str:
    return "; ".join(
        f"{'.'.join(str(p) for p in e['loc']) or 'root'}: {e['msg']}" for e
     in exc.errors()[:10]
    )

async def complete(ai, schema: type[BaseModel], system: str, user: str,
                   context: dict | None = None) -> BaseModel:
    messages = [
        {"role": "system", "content": (
            f"{system}\n\nReply with a single JSON object that validates
     against this JSON Schema. "
            f"No prose outside the JSON.\
     n{json.dumps(schema.model_json_schema())}"
        )},
        {"role": "user", "content": user},
    ]
    last = ""
    for attempt in range(MAX_RETRIES + 1):
        raw = await ai.chat_json(messages, schema.__name__)
        try:
            return schema.model_validate_json(raw, context=context)
        except ValidationError as exc:
            last = _errors(exc)
            logger.warning("[ai] %s attempt %d invalid: %s",
     schema.__name__, attempt + 1, last)
            messages += [
                {"role": "assistant", "content": raw},
                {"role": "user", "content": f"That JSON is invalid: {last}.
     Return the corrected JSON object only."},
            ]
    raise AIOutputInvalid(schema.__name__, last)
```

backend/app/services/ai_jobs.py

```
import json
import logging
from datetime import date, datetime, timezone

from sqlalchemy import func, select
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy.orm import selectinload

from app.exceptions import TraceException
from app.models.project import Project, ProjectAssignment,
     ProjectSourceFile, TaskGroup
from app.models.task import Task
from app.schemas.ai import (
    GanttRevisionOut, PeopleSuggestionsOut, ProgressReportOut,
     ProjectDocumentOut,
    TaskGroupPlanOut, TaskProgressPlanOut, TaskSuggestionsOut,
)
from app.services import gantt_service
from app.services import notification_service as notifications
from app.services.ai_service import AINotConfigured, complete
from app.services.github_service import GitHubClient, GitHubNotConfigured
from app.services.project_service import active_employees, person
from app.services.scoring_service import candidate_stats
```

```
logger = logging.getLogger("trace.ai_jobs")

DOC_PATHS = {
    "initial": "docs/project_initial_doc.md",
    "doc": "docs/project_doc.md",
    "progress": "docs/project_progress_doc.md",
    "report": "docs/project_report_doc.md",
}
DOC_COLUMNS = {
    "initial": "project_initial_doc", "doc": "project_doc",
    "progress": "project_progress_doc", "report": "project_report_doc",
}
SOURCE_CHAR_LIMIT = 60_000
TASK_DOC_CHAR_LIMIT = 4_000
MAX_CANDIDATES = 25

def task_doc_path(task_id: int) -> str:
    return f"docs/tasks/task-{task_id}.md"

def describe(exc: Exception) -> str:
    if isinstance(exc, TraceException):
        return str(exc.detail)
    return str(exc) or exc.__class__.__name__

def set_ai_error(project: Project, job: str, message: str | None) -> None:
    errors = dict(project.ai_errors or {})
    if message:
        errors[job] = {"message": message[:1000], "at":
     datetime.now(timezone.utc).isoformat()}
    else:
        errors.pop(job, None)
    project.ai_errors = errors or None

def _require(ai, github) -> None:
    if ai is None:
        raise AINotConfigured()
    if github is None:
        raise GitHubNotConfigured()

async def _commit_doc(github: GitHubClient, project: Project, path: str,
     content: str,
                      job: str, job_id: str | None) -> str:
    return await github.commit_file(
        project.github_repo, path, content, f"TRACE: {path} ({job} {job_id
     or 'manual'})",
        project.github_default_branch,
    )

async def read_document(github: GitHubClient, project: Project, path: str) -
     > str | None:
    return await github.read_file(project.github_repo, path,
     project.github_default_branch)

DOC_SYSTEM = (
    "You are the requirements analyst of TRACE, a project management system.
     From the source "
    "documents, write the project document the team will plan from: purpose,
     scope, features "
    "(one heading each, with acceptance points), non-functional
     requirements, open questions. "
    "Markdown, factual, no invented requirements."
)
GROUPS_SYSTEM = (
    "You are the delivery planner of TRACE. Split the project into 3–10 task
     groups (modules of "
    "work that each end in something demonstrable). Give each a realistic
     duration in working-"
```

```
    "calendar days for a small team, a sequence starting at 1, and
     depends_on: names of earlier "
    "groups that must finish first. Only add a dependency when the work
     truly waits for it."
)

def _sources_text(files: list[ProjectSourceFile]) -> str:
    parts, used = [], 0
    for f in files:
        room = SOURCE_CHAR_LIMIT - used
        if room <= 0:
            break
        text = (f.extracted_text or "")[:room]
        used += len(text)
        parts.append(f"## Source file: {f.file_name}\n\n{text}")
    return "\n\n".join(parts)

async def run_evaluate_project(db: AsyncSession, project_id: int, *, job_id:
     str | None,
                               ai, github: GitHubClient | None) -> dict:
    project = await db.get(Project, project_id)
    if project is None or project.deleted_at is not None:
        return {"status": "skipped", "reason": "project not found"}
    step = "starting"
    try:
        _require(ai, github)
        step = "reading the source files"
        files = (await db.execute(select(ProjectSourceFile).where(
            ProjectSourceFile.project_id == project_id,
     ProjectSourceFile.deleted_at.is_(None),
            func.length(func.coalesce(ProjectSourceFile.extracted_text, ""))
     > 0,
        ).order_by(ProjectSourceFile.id))).scalars().all()
        if not files:
            raise TraceException(409, "no source file has any extracted
     text")

        step = "writing the project document"
        doc = await complete(ai, ProjectDocumentOut, DOC_SYSTEM,
                             f"Project: {project.name}\n{project.description
     or ''}\n\n{_sources_text(files)}")
        markdown = doc.markdown if doc.markdown.lstrip().startswith("#")
     else f"# {doc.title}\n\n{doc.markdown}"

        step = "committing the project documents"
        initial_url = await _commit_doc(github, project,
     DOC_PATHS["initial"], markdown, "evaluate_project", job_id)
        doc_url = await _commit_doc(github, project, DOC_PATHS["doc"],
     markdown, "evaluate_project", job_id)

        step = "planning the task groups"
        plan = await complete(ai, TaskGroupPlanOut, GROUPS_SYSTEM, markdown)

        step = "building the Gantt chart"
        if (await db.execute(select(func.count(Task.id)).where(
                Task.project_id == project_id,
     Task.deleted_at.is_(None)))).scalar():
            raise TraceException(409, "the project has tasks now, so its
     task groups can no longer be replaced")
        now = datetime.now(timezone.utc)
        for old in (await db.execute(select(TaskGroup).where(
                TaskGroup.project_id == project_id,
     TaskGroup.deleted_at.is_(None)))).scalars():
            old.deleted_at = now

        today = date.today()
        project.start_date = project.start_date or today
        ids: dict[str, int] = {}
```

```
            groups = []
            for sequence, g in enumerate(plan.task_groups, start=1):
                row = TaskGroup(project_id=project_id, name=g.name.strip(),
        description=g.description or None,
                                sequence=sequence,
        estimated_duration_days=g.estimated_duration_days,
                                status="Pending", created_by="AI",
        completion_percentage=0)
                db.add(row)
                await db.flush()
                row.depends_on = [ids[d.strip().lower()] for d in g.depends_on]
        or None
                ids[row.name.lower()] = row.id
                groups.append(row)
            gantt_service.schedule(groups, project.start_date, today)
            project.end_date = project.end_date or max(g.planned_end_date for g
        in groups)
            project.project_initial_doc, project.project_doc = initial_url,
        doc_url
            await db.flush()
            await gantt_service.snapshot(db, project, "initial", job_id)

            project.status = "Active"
            project.last_evaluated_at = now
            set_ai_error(project, "evaluate_project", None)
            outbox = await notifications.notify(
                db, [project.created_by_user_id], "project_evaluated",
        f"Evaluated: {project.name}",
                f"{project.name} has its project document, {len(groups)} task
        groups and a Gantt chart.",
                project_id=project.id)
            await db.commit()
            notifications.deliver(outbox)
            return {"status": "success", "task_groups": len(groups)}
        except Exception as exc:
            await db.rollback()
            logger.exception("[evaluate_project] project %s failed while %s",
        project_id, step)
            project = await db.get(Project, project_id)
            project.status = "Active" if project.last_evaluated_at else "Draft"
            message = f"Failed while {step}: {describe(exc)}"
            set_ai_error(project, "evaluate_project", message)
            outbox = await notifications.notify(
                db, [project.created_by_user_id], "project_evaluated",
        f"Evaluation failed: {project.name}",
                message, project_id=project.id)
            await db.commit()
            notifications.deliver(outbox)
            return {"status": "failed", "error": message}

    PM_SYSTEM = (
        "You rank candidates to be the Project Manager of a project. Rank on
         projects delivered, "
        "on-time delivery rate, and average task completion across the projects
         they manage; "
        "a strong task record counts when they have not managed a project yet.
         Score 0–1. "
        "Reason briefly from the numbers given; never invent facts. Return at
         most 10."
    )
    ASSIGNEE_SYSTEM = (
        "You rank candidates to be assigned one task. Rank on tasks completed,
         on-time rate, current "
        "open workload, and how similar their recently completed task titles are
         to this task. "
        "Score 0–1. Reason briefly from the data given; never invent facts.
         Return at most 10."
    )
```

```
TASKS_SYSTEM = (
    "You break one task group of a project into concrete tasks a single
     person can finish in "
    "1–5 days. Do not repeat tasks that already exist. Give each an estimate
     in hours."
)

async def _rank_people(db: AsyncSession, ai, system: str, brief: str, users:
     list) -> dict:
    if ai is None:
        raise AINotConfigured()
    if not users:
        return {"suggestions": []}
    stats = await candidate_stats(db, [u.id for u in users])
    users = sorted(users, key=lambda u: stats[u.id]["tasks_completed"],
     reverse=True)[:MAX_CANDIDATES]
    by_id = {u.id: u for u in users}
    candidates = [{"user_id": u.id, "name": u.name, **stats[u.id]} for u in
     users]
    out = await complete(ai, PeopleSuggestionsOut, system,
                         f"{brief}\n\nCandidates:\n{json.dumps(candidates,
     default=str)}",
                         context={"candidates": set(by_id)})
    return {"suggestions": [{
        "user": person(by_id[s.user_id]), "score": s.score, "reasoning":
     s.reasoning,
        "evidence": s.evidence, "stats": stats[s.user_id],
    } for s in out.suggestions]}

async def _project_brief(db: AsyncSession, project: Project) -> str:
    groups = (await db.execute(select(TaskGroup.name).where(
        TaskGroup.project_id == project.id,
     TaskGroup.deleted_at.is_(None)).order_by(TaskGroup.sequence)
    )).scalars().all()
    return (f"Project: {project.name}\n{project.description or ''}\n"
            f"Task groups: {', '.join(groups) or 'none yet'}")

async def run_suggest_managers(db: AsyncSession, project_id: int, *, ai,
     **_) -> dict:
    project = await db.get(Project, project_id)
    current = set((await db.execute(select(ProjectAssignment.user_id).where(
        ProjectAssignment.project_id == project_id,
     ProjectAssignment.project_role == "Project Manager",
        ProjectAssignment.deleted_at.is_(None)))).scalars())
    users = [u for u in await active_employees(db) if u.id not in current]
    return await _rank_people(db, ai, PM_SYSTEM, await _project_brief(db,
     project), users)

async def run_suggest_assignees(db: AsyncSession, project_id: int, title:
     str, description: str | None,
                                *, ai, **_) -> dict:
    project = await db.get(Project, project_id)
    brief = f"{await _project_brief(db, project)}\n\nTask: {title}\
     n{description or ''}"
    return await _rank_people(db, ai, ASSIGNEE_SYSTEM, brief, await
     active_employees(db))

async def run_suggest_tasks(db: AsyncSession, group_id: int, *, ai, **_) ->
     dict:
    if ai is None:
        raise AINotConfigured()
    group = await db.get(TaskGroup, group_id)
    project = await db.get(Project, group.project_id)
    existing = (await db.execute(select(Task.title).where(
        Task.task_group_id == group_id,
     Task.deleted_at.is_(None)))).scalars().all()
    out = await complete(ai, TaskSuggestionsOut, TASKS_SYSTEM, (
        f"Project: {project.name}\n{project.description or ''}\n\n"
```

```
        f"Task group: {group.name}\n{group.description or ''}\n\n"
        f"Existing tasks: {json.dumps(existing)}"))
    return {"task_group_id": group_id, **out.model_dump()}

TASK_DOC_SYSTEM = (
    "You write the working plan for one task in a software project: ordered,
     concrete steps "
    "and testable acceptance criteria. Keep it proportionate to the task."
)

def render_task_doc(task: Task, plan: TaskProgressPlanOut) -> str:
    lines = [f"# {task.title}", "", task.description or "", "", "## Steps",
     ""]
    for step in sorted(plan.steps, key=lambda s: s.order):
        lines.append(f"- [{'x' if step.done else ' '}] **{step.title}**" +
     (f" — {step.detail}" if step.detail else ""))
    lines += ["", "## Acceptance criteria", ""] + [f"- [ ] {c}" for c in
     plan.acceptance_criteria]
    if task.github_issue_url:
        lines += ["", f"GitHub issue: {task.github_issue_url}"]
    return "\n".join(lines).replace("\n\n\n", "\n\n") + "\n"

async def run_task_progress_doc(db: AsyncSession, task_id: int, *, job_id:
     str | None,
                                ai, github: GitHubClient | None) -> dict:
    task = (await db.execute(select(Task).where(Task.id == task_id).options(
        selectinload(Task.task_group),
     selectinload(Task.project)))).scalar_one_or_none()
    if task is None or task.deleted_at is not None:
        return {"status": "skipped", "reason": "task not found"}
    try:
        _require(ai, github)
        plan = await complete(ai, TaskProgressPlanOut, TASK_DOC_SYSTEM, (
            f"Project: {task.project.name}\nTask group:
     {task.task_group.name if task.task_group else '—'}\n"
            f"Task: {task.title}\nPriority: {task.priority}\nDue:
     {task.due_date or 'not set'}\n\n"
            f"{task.description or ''}"))
        url = await _commit_doc(github, task.project,
     task_doc_path(task.id), render_task_doc(task, plan),
                                "generate_task_progress_doc", job_id)
        task.task_progress_doc, task.progress_doc_error = url, None
        await db.commit()
        return {"status": "success", "url": url}
    except Exception as exc:
        await db.rollback()
        logger.exception("[generate_task_progress_doc] task %s failed",
     task_id)
        task = await db.get(Task, task_id)
        task.progress_doc_error = describe(exc)[:1000]
        await db.commit()
        return {"status": "failed", "error": task.progress_doc_error}

REVISE_SYSTEM = (
    "The project document changed. Given it and every task group's progress,
     re-estimate the "
    "remaining duration (in days, from its start) of unfinished groups whose
     scope changed, and "
    "add new groups only for features that no existing group covers. Leave
     everything else out."
)

async def _revise_from_document(db: AsyncSession, project: Project, groups:
     list[TaskGroup],
                                ai, github: GitHubClient | None, job_id: str
     | None) -> list[TaskGroup]:
    _require(ai, github)
    document = await read_document(github, project, DOC_PATHS["doc"])
```

```
    if not document:
        raise TraceException(409, "the project document could not be read
     from the repository")
    open_ids = {g.id for g in groups if g.status != "Completed"}
    summary = [{"task_group_id": g.id, "name": g.name, "status": g.status,
                "completion_percentage": float(g.completion_percentage or
     0),
                "estimated_duration_days": g.estimated_duration_days} for g
     in groups]
    out = await complete(ai, GanttRevisionOut, REVISE_SYSTEM,
                         f"Task groups:\n{json.dumps(summary)}\n\nProject
     document:\n{document[:SOURCE_CHAR_LIMIT]}",
                         context={"open_groups": open_ids, "all_groups":
     {g.id for g in groups}})
    by_id = {g.id: g for g in groups}
    for r in out.revisions:
        by_id[r.task_group_id].estimated_duration_days =
     r.estimated_duration_days
    added = []
    sequence = max((g.sequence for g in groups), default=0)
    for n in out.new_groups:
        sequence += 1
        row = TaskGroup(project_id=project.id, name=n.name.strip(),
     description=n.description or None,
                        sequence=sequence,
     estimated_duration_days=n.estimated_duration_days,
                        depends_on=n.depends_on or None, status="Pending",
     created_by="AI",
                        completion_percentage=0)
        db.add(row)
        added.append(row)
    await db.flush()
    return added

async def run_reevaluate_gantt(db: AsyncSession, project_id: int, reason:
     str, requested_at: str, *,
                               job_id: str | None, ai, github: GitHubClient
     | None) -> dict:
    project = await db.get(Project, project_id)
    if project is None or project.deleted_at is not None:
        return {"status": "skipped", "reason": "project not found"}
    latest = next(iter(await gantt_service.versions(db, project_id)), None)
    if latest and latest.generated_at and latest.generated_at >
     datetime.fromisoformat(requested_at):
        return {"status": "skipped", "reason": f"already covered by version
     {latest.version}"}
    groups = [g for g, _ in await gantt_service.groups_with_tasks(db,
     project_id)]
    if not groups:
        return {"status": "skipped", "reason": "the project has no task
     groups"}

    if reason == "project_doc_updated":
        try:
            groups += await _revise_from_document(db, project, groups, ai,
     github, job_id)
            set_ai_error(project, "reevaluate_gantt", None)
        except Exception as exc:
            logger.exception("[reevaluate_gantt] AI revision failed for
     project %s", project_id)
            set_ai_error(project, "reevaluate_gantt",
                         f"The AI could not re-read the project document
     ({describe(exc)}); "
                         "the chart was re-planned from the current task
     groups without it")

    today = date.today()
    gantt_service.schedule(groups, project.start_date or today, today)
```

```
    await db.flush()
    version = await gantt_service.snapshot(db, project, reason, job_id)
    await db.commit()
    return {"status": "success", "version": version.version}

REPORT_SYSTEM = (
    "You maintain two documents for a software project. progress_markdown:
     the team's progress "
    "document — per task group, what is done, in progress, blocked or
     overdue, and what is next. "
    "report_markdown: a one-page report for the Manager — overall status,
     risks, slipped dates, "
    "decisions needed. Use only the data given."
)

async def run_progress_report(db: AsyncSession, project_id: int, *, job_id:
     str | None,
                              ai, github: GitHubClient | None) -> dict:
    project = await db.get(Project, project_id)
    if project is None or project.deleted_at is not None:
        return {"status": "skipped", "reason": "project not found"}
    try:
        _require(ai, github)
        groups = await gantt_service.groups_with_tasks(db, project_id)
        today = date.today()
        data, docs, used = [], [], 0
        for g, tasks in groups:
            data.append({"task_group": g.name, "status": g.status,
                         "completion_percentage":
     float(g.completion_percentage or 0),
                         "planned": [str(g.planned_start_date),
     str(g.planned_end_date)],
                         "tasks": [{"title": t.title, "status": t.status,
     "due": str(t.due_date),
                                    "overdue": gantt_service.is_overdue(t,
     today),
                                    "assignee": t.assigned_to_user.name if
     t.assigned_to_user else None,
                                    "completed_by": t.completed_by} for t in
     tasks]})
            for t in tasks:
                if t.task_progress_doc and used < SOURCE_CHAR_LIMIT:
                    text = await read_document(github, project,
     task_doc_path(t.id))
                    if text:
                        text = text[:TASK_DOC_CHAR_LIMIT]
                        used += len(text)
                        docs.append(f"### {t.title}\n{text}")
        out = await complete(ai, ProgressReportOut, REPORT_SYSTEM, (
            f"Project: {project.name} (status {project.status},
     {project.start_date} → {project.end_date}, "
            f"today {today})\n\nTask groups and tasks:\n{json.dumps(data)}\
     n\n"
            f"Task progress documents:\n\n" + "\n\n".join(docs)))
        project.project_progress_doc = await _commit_doc(
            github, project, DOC_PATHS["progress"], out.progress_markdown,
     "update_project_progress_doc", job_id)
        project.project_report_doc = await _commit_doc(
            github, project, DOC_PATHS["report"], out.report_markdown,
     "update_project_progress_doc", job_id)
        set_ai_error(project, "update_project_progress_doc", None)
        await db.commit()
        return {"status": "success"}
    except Exception as exc:
        await db.rollback()
        logger.exception("[update_project_progress_doc] project %s failed",
     project_id)
        project = await db.get(Project, project_id)
```

```
        set_ai_error(project, "update_project_progress_doc", describe(exc))
        await db.commit()
        return {"status": "failed", "error": describe(exc)}
```

backend/app/services/jobs.py

```
import uuid
from contextlib import asynccontextmanager

from sqlalchemy.ext.asyncio import AsyncSession, create_async_engine
from sqlalchemy.pool import NullPool

from app.config import get_settings

def enqueue(name: str, *args, countdown: int | None = None) -> str:
    from app.celery_app import celery_app

    job_id = str(uuid.uuid4())
    celery_app.send_task(f"app.tasks.{name}", args=list(args),
     countdown=countdown, task_id=job_id)
    return job_id

@asynccontextmanager
async def job_session():
    engine = create_async_engine(get_settings().DATABASE_URL,
     poolclass=NullPool)
    try:
        async with AsyncSession(engine, expire_on_commit=False) as session:
            yield session
    finally:
        await engine.dispose()
```

backend/app/services/gantt_service.py

```
from datetime import date, datetime, timedelta, timezone

from sqlalchemy import func, select
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy.orm import selectinload

from app.models.project import GanttChart, Project, TaskGroup
from app.models.task import Task
from app.schemas.gantt import GanttChartData
from app.services import jobs
from app.services.project_service import OPEN_STATUSES, person

DEFAULT_GROUP_DAYS = 7
DEBOUNCE_SECONDS = 30

def is_overdue(task: Task, today: date | None = None) -> bool:
    today = today or date.today()
    return task.status == "Overdue" or (
        task.due_date is not None and task.due_date < today and task.status
     in OPEN_STATUSES)

async def groups_with_tasks(db: AsyncSession, project_id: int) ->
     list[tuple[TaskGroup, list[Task]]]:
    groups = (await db.execute(
        select(TaskGroup).where(TaskGroup.project_id == project_id,
     TaskGroup.deleted_at.is_(None))
        .order_by(TaskGroup.sequence, TaskGroup.id)
    )).scalars().all()
    tasks = (await db.execute(
        select(Task).where(Task.project_id == project_id,
     Task.deleted_at.is_(None))
        .options(selectinload(Task.assigned_to_user)).order_by(Task.id)
    )).scalars().all()
    by_group: dict[int, list[Task]] = {}
```

```
    for task in tasks:
        by_group.setdefault(task.task_group_id, []).append(task)
    return [(g, by_group.get(g.id, [])) for g in groups]

async def project_tree(db: AsyncSession, project: Project) -> dict:
    groups = await groups_with_tasks(db, project.id)
    today = date.today()
    live = [t for _, tasks in groups for t in tasks if t.status !=
     "Cancelled"]
    done = sum(1 for t in live if t.status == "Completed")
    return {
        "project": {"id": project.id, "name": project.name, "status":
     project.status,
                    "completion_percentage": round(done / len(live) * 100,
     1) if live else 0.0},
        "groups": [{
            "id": g.id, "name": g.name, "sequence": g.sequence, "status":
     g.status,
            "completion_percentage": float(g.completion_percentage or 0),
            "tasks": [{
                "id": t.id, "title": t.title, "status": t.status,
     "priority": t.priority,
                "due_date": t.due_date, "overdue": is_overdue(t, today),
                "assignee": person(t.assigned_to_user), "github_issue_url":
     t.github_issue_url,
            } for t in tasks],
        } for g, tasks in groups],
    }

def build_chart_data(project: Project, groups: list[tuple[TaskGroup,
     list[Task]]],
                     dependencies: list[tuple[int, int]] | None = None,
                     today: date | None = None) -> GanttChartData:
    cursor = project.start_date or today or date.today()
    bars = []
    for g, tasks in groups:
        start = g.planned_start_date or cursor
        end = g.planned_end_date or start +
     timedelta(days=(g.estimated_duration_days or DEFAULT_GROUP_DAYS) - 1)
        end = max(end, start)
        cursor = end + timedelta(days=1)
        bars.append({
            "task_group_id": g.id, "name": g.name, "sequence": g.sequence,
            "start": start, "end": end, "completion":
     float(g.completion_percentage or 0),
            "status": g.status,
            "tasks": [{"task_id": t.id, "title": t.title, "status":
     t.status, "due": t.due_date,
                       "assignee": t.assigned_to_user.name if
     t.assigned_to_user else None}
                      for t in tasks if t.status != "Cancelled"],
        })
    milestones = [{"name": "Project end", "date": project.end_date}] if
     project.end_date else []
    dates = [d for b in bars for d in (b["start"], b["end"])] + [m["date"]
     for m in milestones]
    dates += [t["due"] for b in bars for t in b["tasks"] if t["due"]]
    start = min(dates, default=cursor)
    return GanttChartData.model_validate({
        "schema": 1,
        "range": {"start": start, "end": max(dates, default=start)},
        "bars": bars,
        "dependencies": [{"from": a, "to": b} for a, b in (dependencies or
     [])],
        "milestones": milestones,
    })
```

```
def schedule(groups: list[TaskGroup], start: date, today: date | None =
     None) -> None:
    today = today or date.today()
    ends: dict[int, date] = {}
    for g in sorted(groups, key=lambda g: (g.sequence, g.id or 0)):
        begin = g.planned_start_date or start
        after = max((ends[d] for d in (g.depends_on or []) if d in ends),
     default=None)
        if after is not None and begin <= after:
            begin = after + timedelta(days=1)
        if g.estimated_duration_days:
            days = g.estimated_duration_days
        elif g.planned_start_date and g.planned_end_date:
            days = (g.planned_end_date - g.planned_start_date).days + 1
        else:
            days = DEFAULT_GROUP_DAYS
        end = begin + timedelta(days=max(days, 1) - 1)
        if g.status == "Completed" and g.actual_end_date:
            end = max(begin, g.actual_end_date)
        elif g.status != "Completed" and end < today:
            end = today
        g.planned_start_date, g.planned_end_date = begin, end
        ends[g.id] = end

def dependencies_of(groups: list[TaskGroup]) -> list[tuple[int, int]]:
    ids = {g.id for g in groups}
    return [(d, g.id) for g in groups for d in (g.depends_on or []) if d in
     ids and d != g.id]

async def save_version(db: AsyncSession, project_id: int, data:
     GanttChartData | dict, reason: str,
                       job_id: str | None = None) -> GanttChart:
    chart_data = (data if isinstance(data, GanttChartData) else
     GanttChartData.model_validate(data)).dump()
    latest = (await db.execute(
        select(func.max(GanttChart.version)).where(GanttChart.project_id ==
     project_id,

     GanttChart.deleted_at.is_(None))
    )).scalar() or 0
    row = GanttChart(project_id=project_id, version=latest + 1,
     chart_data=chart_data,
                     generated_reason=reason,
     generated_at=datetime.now(timezone.utc),
                     generated_by_job=job_id)
    db.add(row)
    await db.flush()
    return row

async def snapshot(db: AsyncSession, project: Project, reason: str, job_id:
     str | None = None) -> GanttChart:
    groups = await groups_with_tasks(db, project.id)
    deps = dependencies_of([g for g, _ in groups])
    return await save_version(db, project.id, build_chart_data(project,
     groups, deps), reason, job_id)

async def versions(db: AsyncSession, project_id: int) -> list[GanttChart]:
    return list((await db.execute(
        select(GanttChart).where(GanttChart.project_id == project_id,
     GanttChart.deleted_at.is_(None))
        .order_by(GanttChart.version.desc())
    )).scalars().all())

def request_reevaluation(project_id: int, reason: str, manual: bool = False)
     -> str:
    requested_at = datetime.now(timezone.utc).isoformat()
    return jobs.enqueue("reevaluate_gantt", project_id, reason,
     requested_at,
```

```
                        countdown=0 if manual else DEBOUNCE_SECONDS)
```

backend/app/services/scoring_service.py

```
from datetime import date, datetime, timedelta, timezone
from decimal import Decimal

from sqlalchemy import func, select
from sqlalchemy.ext.asyncio import AsyncSession

from app.exceptions import NotFoundException
from app.models.employee import Employee
from app.models.productivity import ProductivityScore
from app.models.project import Project, ProjectAssignment
from app.models.task import Task

OPEN_STATUSES = ("Pending", "In Progress", "In Review", "Blocked",
     "Overdue")

def _on_time(task: Task) -> bool:
    return task.due_date is None or (task.completed_at is not None and
     task.completed_at.date() <= task.due_date)

def _pct(part: int, whole: int) -> Decimal:
    return Decimal(str(round(part / whole * 100, 2))) if whole else
     Decimal("100")

class ScoringService:
    def __init__(self, db: AsyncSession):
        self.db = db

    async def compute_weekly_score(self, employee_id: int, week_start: date)
     -> ProductivityScore:
        employee = await self.db.get(Employee, employee_id)
        if employee is None or employee.deleted_at is not None:
            raise NotFoundException("Employee")
        week_end = week_start + timedelta(days=6)
        tasks = (await self.db.execute(select(Task).where(
            Task.assigned_to_user_id == employee.user_id,
     Task.deleted_at.is_(None),
            Task.status != "Cancelled", Task.due_date >= week_start,
     Task.due_date <= week_end,
        ))).scalars().all()
        completed = [t for t in tasks if t.status == "Completed"]
        task_score = _pct(len(completed), len(tasks))
        deadline_score = _pct(sum(1 for t in completed if _on_time(t)),
     len(completed))
        final = ((task_score + deadline_score) /
     2).quantize(Decimal("0.01"))

        overdue = (await self.db.execute(select(func.count(Task.id)).where(
            Task.assigned_to_user_id == employee.user_id,
     Task.deleted_at.is_(None),
            Task.status.in_(OPEN_STATUSES), Task.due_date < date.today(),
        ))).scalar() or 0
        flags = {}
        if final < 60:
            flags["low_score"] = True
        if overdue >= 3:
            flags["overdue_tasks"] = overdue

        score = ProductivityScore(
            employee_id=employee_id, week_start=week_start,
     task_score=task_score,
            deadline_score=deadline_score, final_score=final,
     rating=self._get_rating(final),
            anomaly_flags=flags, generated_at=datetime.now(timezone.utc),
        )
```

```
        self.db.add(score)
        await self.db.flush()
        return score

    @staticmethod
    def _get_rating(score: Decimal) -> str:
        if score >= 90:
            return "Excellent"
        if score >= 75:
            return "Good"
        if score >= 60:
            return "Average"
        return "Risk"

async def candidate_stats(db: AsyncSession, user_ids: list[int]) ->
     dict[int, dict]:
    stats = {uid: {"tasks_completed": 0, "on_time_rate": None, "open_tasks":
     0,
                   "projects_managed": 0, "projects_delivered": 0,
     "managed_projects_completion": None,
                   "latest_weekly_score": None, "recent_completed_titles":
     []} for uid in user_ids}
    if not user_ids:
        return stats

    tasks = (await db.execute(select(Task).where(
        Task.assigned_to_user_id.in_(user_ids), Task.deleted_at.is_(None),
     Task.status != "Cancelled",
    ).order_by(Task.completed_at.desc().nullslast()))).scalars().all()
    on_time: dict[int, int] = {}
    for t in tasks:
        s = stats[t.assigned_to_user_id]
        if t.status == "Completed":
            s["tasks_completed"] += 1
            on_time[t.assigned_to_user_id] =
     on_time.get(t.assigned_to_user_id, 0) + _on_time(t)
            if len(s["recent_completed_titles"]) < 10:
                s["recent_completed_titles"].append(t.title)
        elif t.status in OPEN_STATUSES:
            s["open_tasks"] += 1
    for uid, n in on_time.items():
        stats[uid]["on_time_rate"] = round(n / stats[uid]
     ["tasks_completed"], 2)

    managed = (await db.execute(
        select(ProjectAssignment.user_id, Project.id, Project.status)
        .join(Project, Project.id == ProjectAssignment.project_id)
        .where(ProjectAssignment.user_id.in_(user_ids),
     ProjectAssignment.project_role == "Project Manager",
               ProjectAssignment.deleted_at.is_(None),
     Project.deleted_at.is_(None))
    )).all()
    project_ids = {pid for _, pid, _ in managed}
    completion: dict[int, float] = {}
    if project_ids:
        rows = (await db.execute(
            select(Task.project_id, func.count(Task.id),
                   func.count(Task.id).filter(Task.status == "Completed"))
            .where(Task.project_id.in_(project_ids),
     Task.deleted_at.is_(None), Task.status != "Cancelled")
            .group_by(Task.project_id)
        )).all()
        completion = {pid: done / total for pid, total, done in rows if
     total}
    per_user: dict[int, list[float]] = {}
    for uid, pid, status in managed:
        stats[uid]["projects_managed"] += 1
        stats[uid]["projects_delivered"] += status == "Completed"
```

```
        if pid in completion:
            per_user.setdefault(uid, []).append(completion[pid])
    for uid, values in per_user.items():
        stats[uid]["managed_projects_completion"] = round(sum(values) /
     len(values), 2)

    latest = (await db.execute(
        select(Employee.user_id, ProductivityScore.final_score)
        .join(ProductivityScore, ProductivityScore.employee_id ==
     Employee.id)
        .where(Employee.user_id.in_(user_ids),
     ProductivityScore.deleted_at.is_(None))
        .order_by(ProductivityScore.week_start.desc())
    )).all()
    for uid, score in latest:
        if stats[uid]["latest_weekly_score"] is None and score is not None:
            stats[uid]["latest_weekly_score"] = float(score)
    return stats
```

backend/app/services/extraction.py

```
import io
from pathlib import PurePath

SUPPORTED_EXTENSIONS = {".pdf", ".docx", ".md", ".markdown", ".txt"}
MAX_SOURCE_FILE_BYTES = 20 * 1024 * 1024

class UnsupportedFile(ValueError):
    pass

def extension_of(file_name: str) -> str:
    return PurePath(file_name).suffix.lower()

def extract_text(file_name: str, data: bytes) -> str:
    ext = extension_of(file_name)
    if ext in {".md", ".markdown", ".txt"}:
        return data.decode("utf-8", errors="replace")
    if ext == ".pdf":
        from pypdf import PdfReader

        try:
            reader = PdfReader(io.BytesIO(data))
            return "\n\n".join((page.extract_text() or "") for page in
     reader.pages).strip()
        except Exception as exc:
            raise UnsupportedFile(f"{file_name}: not a readable PDF
     ({exc.__class__.__name__})") from exc
    if ext == ".docx":
        import docx

        try:
            document = docx.Document(io.BytesIO(data))
        except Exception as exc:
            raise UnsupportedFile(f"{file_name}: not a readable .docx
     ({exc.__class__.__name__})") from exc
        parts = [p.text for p in document.paragraphs]
        for table in document.tables:
            for row in table.rows:
                parts.append(" | ".join(cell.text for cell in row.cells))
        return "\n".join(parts).strip()
    raise UnsupportedFile(
        f"{file_name}: unsupported type — upload one of {',
     '.join(sorted(SUPPORTED_EXTENSIONS))}"
    )
```

backend/app/services/storage.py

```
import asyncio
```

```
import io

from app.config import get_settings

class ObjectStorage:
    def __init__(self):
        from minio import Minio

        s = get_settings()
        self._bucket = s.MINIO_BUCKET
        self._client = Minio(s.MINIO_ENDPOINT,
     access_key=s.MINIO_ACCESS_KEY,
                             secret_key=s.MINIO_SECRET_KEY,
     secure=s.MINIO_SECURE)
        self._bucket_checked = False

    def _ensure_bucket(self) -> None:
        if not self._bucket_checked:
            if not self._client.bucket_exists(self._bucket):
                self._client.make_bucket(self._bucket)
            self._bucket_checked = True

    def _put(self, key: str, data: bytes, content_type: str) -> None:
        self._ensure_bucket()
        self._client.put_object(self._bucket, key, io.BytesIO(data),
     len(data), content_type=content_type)

    def _get(self, key: str) -> bytes:
        response = self._client.get_object(self._bucket, key)
        try:
            return response.read()
        finally:
            response.close()
            response.release_conn()

    async def put(self, key: str, data: bytes, content_type: str) -> None:
        await asyncio.to_thread(self._put, key, data, content_type)

    async def get(self, key: str) -> bytes:
        return await asyncio.to_thread(self._get, key)

    async def delete(self, key: str) -> None:
        await asyncio.to_thread(self._client.remove_object, self._bucket,
     key)

_storage: ObjectStorage | None = None

def get_storage() -> ObjectStorage:
    global _storage
    if _storage is None:
        _storage = ObjectStorage()
    return _storage
```

backend/app/services/notification_service.py

```
from dataclasses import dataclass, field

from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy.orm import selectinload

from app.config import get_settings
from app.core.project_access import PROJECT_MANAGER, platform_roles
from app.models.project import Notification, ProjectAssignment
from app.models.task import Task
from app.models.user import User, UserRole
from app.services import jobs, realtime
```

```
@dataclass
class Outbox:
    user_ids: set[int] = field(default_factory=set)
    emails: list[tuple[str, str, str]] = field(default_factory=list)

    def __add__(self, other: "Outbox") -> "Outbox":
        return Outbox(self.user_ids | other.user_ids, self.emails +
     other.emails)

def task_link(user: User, task_id: int) -> str:
    panel = "manager" if "Manager" in platform_roles(user) else "employee"
    return f"/{panel}/tasks/{task_id}"

def project_link(user: User, project_id: int) -> str:
    panel = "manager" if "Manager" in platform_roles(user) else "employee"
    return f"/{panel}/projects/{project_id}"

async def project_managers(db: AsyncSession, project_id: int) -> list[int]:
    return list((await db.execute(
        select(ProjectAssignment.user_id).where(
            ProjectAssignment.project_id == project_id,
            ProjectAssignment.project_role == PROJECT_MANAGER,
            ProjectAssignment.deleted_at.is_(None),
        )
    )).scalars())

async def notify(db: AsyncSession, user_ids, type_: str, title: str, body:
     str, *,
                 task: Task | None = None, project_id: int | None = None,
                 email: bool = True, exclude: int | None = None) -> Outbox:
    ids = {u for u in user_ids if u and u != exclude}
    outbox = Outbox()
    if not ids:
        return outbox
    users = (await db.execute(
        select(User).where(User.id.in_(ids), User.deleted_at.is_(None))
        .options(selectinload(User.user_roles).selectinload(UserRole.role))
    )).scalars().all()
    project_id = project_id or (task.project_id if task else None)
    for user in users:
        if task is not None:
            link = task_link(user, task.id)
        elif project_id is not None:
            link = project_link(user, project_id)
        else:
            link = None
        db.add(Notification(
            user_id=user.id, type=type_, title=title[:255], body=body,
     link=link,
            related_task_id=task.id if task else None,
     related_project_id=project_id,
        ))
        outbox.user_ids.add(user.id)
        if email and user.email:
            url = f"{get_settings().app_url}{link}" if link else
     get_settings().app_url
            outbox.emails.append((user.email, title[:255], f"{body}\n\nOpen
     in TRACE: {url}"))
    return outbox

def deliver(outbox: Outbox) -> None:
    realtime.publish(outbox.user_ids, {"type": "notification"})
    for to, subject, body in outbox.emails:
        jobs.enqueue("send_mail_task", to, subject, body)

def discussion_event(user_ids, task_id: int) -> None:
    realtime.publish(user_ids, {"type": "discussion", "task_id": task_id})
```

backend/app/services/mail_service.py

```
import smtplib
from dataclasses import dataclass
from email.mime.multipart import MIMEMultipart
from email.mime.text import MIMEText
from html import escape

from sqlalchemy.ext.asyncio import AsyncSession

from app.services.system_settings import resolve_setting

SECURITY = ("none", "starttls", "ssl")

@dataclass
class MailConfig:
    host: str
    port: int
    from_address: str
    username: str = ""
    password: str = ""
    security: str = "none"

async def mail_config(db: AsyncSession) -> MailConfig:
    port = await resolve_setting(db, "mail", "port")
    security = await resolve_setting(db, "mail", "security")
    return MailConfig(
        host=await resolve_setting(db, "mail", "host"),
        port=int(port) if port.isdigit() else 25,
        from_address=await resolve_setting(db, "mail", "from_address"),
        username=await resolve_setting(db, "mail", "username"),
        password=await resolve_setting(db, "mail", "password"),
        security=security if security in SECURITY else "none",
    )

def _connect(config: MailConfig, timeout: int = 10) -> smtplib.SMTP:
    if config.security == "ssl":
        smtp = smtplib.SMTP_SSL(config.host, config.port, timeout=timeout)
    else:
        smtp = smtplib.SMTP(config.host, config.port, timeout=timeout)
        if config.security == "starttls":
            smtp.starttls()
    if config.username:
        smtp.login(config.username, config.password)
    return smtp

def render(subject: str, body: str) -> tuple[str, str]:
    html = (
        '<html><body style="font-family: sans-serif; line-height: 1.6;">'
        f"<h3>{escape(subject)}</h3><p>{escape(body).replace(chr(10),
     '<br>')}</p>"
        '<p style="color: #888; font-size: 12px;">Sent by
     TRACE</p></body></html>'
    )
    return body, html

def send(config: MailConfig, to: str, subject: str, body: str) -> None:
    text, html = render(subject, body)
    msg = MIMEMultipart("alternative")
    msg["From"], msg["To"], msg["Subject"] = config.from_address, to,
     subject
    msg.attach(MIMEText(text, "plain"))
    msg.attach(MIMEText(html, "html"))
    with _connect(config) as smtp:
        smtp.sendmail(config.from_address, [to], msg.as_string())

def check(config: MailConfig) -> list[dict]:
    if not config.host:
```

```
        return [{"name": "Server", "ok": False, "detail": "No SMTP host is
     set"}]
    checks = []
    try:
        with _connect(config, timeout=8) as smtp:
            code, _ = smtp.noop()
            checks.append({"name": "Server", "ok": code == 250,
                           "detail": f"{config.host}:{config.port} answered
     ({config.security})"})
            if config.username:
                checks.append({"name": "Login", "ok": True, "detail":
     f"Signed in as {config.username}"})
    except smtplib.SMTPAuthenticationError:
        checks += [{"name": "Server", "ok": True, "detail": f"{config.host}:
     {config.port} answered"},
                   {"name": "Login", "ok": False, "detail": "The username or
     password was refused"}]
    except (OSError, smtplib.SMTPException) as exc:
        checks.append({"name": "Server", "ok": False, "detail":
     f"{config.host}:{config.port} — {exc}"})
    return checks
```

backend/app/services/realtime.py

```
import asyncio
import json
import logging
from collections import defaultdict

from fastapi import WebSocket

from app.config import get_settings

logger = logging.getLogger("trace.realtime")

CHANNEL = "trace:events"

def publish(user_ids, event: dict) -> None:
    ids = sorted({int(u) for u in user_ids if u})
    if not ids:
        return
    try:
        import redis
        client = redis.Redis.from_url(get_settings().REDIS_URL,
     socket_timeout=2)
        try:
            client.publish(CHANNEL, json.dumps({"users": ids, "event":
     event}))
        finally:
            client.close()
    except Exception as exc:
        logger.warning("[realtime] publish failed: %s", exc)

class Hub:
    def __init__(self):
        self.sockets: dict[int, set[WebSocket]] = defaultdict(set)
        self._listener: asyncio.Task | None = None

    async def connect(self, user_id: int, ws: WebSocket) -> None:
        await ws.accept()
        self.sockets[user_id].add(ws)
        if self._listener is None or self._listener.done():
            self._listener = asyncio.create_task(self._listen())

    def disconnect(self, user_id: int, ws: WebSocket) -> None:
        self.sockets[user_id].discard(ws)
        if not self.sockets[user_id]:
            del self.sockets[user_id]
```

```
    def online(self) -> int:
        return len(self.sockets)

    async def deliver(self, user_ids: list[int], event: dict) -> None:
        text = json.dumps(event)
        for uid in user_ids:
            for ws in list(self.sockets.get(uid, ())):
                try:
                    await ws.send_text(text)
                except Exception:
                    self.disconnect(uid, ws)

    async def _listen(self) -> None:
        import redis.asyncio as aioredis
        while True:
            try:
                client = aioredis.Redis.from_url(get_settings().REDIS_URL)
                async with client.pubsub() as pubsub:
                    await pubsub.subscribe(CHANNEL)
                    async for message in pubsub.listen():
                        if message.get("type") != "message":
                            continue
                        data = json.loads(message["data"])
                        await self.deliver(data.get("users", []),
     data.get("event", {}))
            except asyncio.CancelledError:
                raise
            except Exception as exc:
                logger.warning("[realtime] subscriber dropped, retrying:
     %s", exc)
                await asyncio.sleep(5)

hub = Hub()
```

backend/app/services/report_service.py

```
from datetime import date, datetime, timedelta, timezone
from decimal import Decimal
from typing import Any

from sqlalchemy import and_, func, select
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy.orm import selectinload

from app.models.employee import Employee
from app.models.productivity import ProductivityScore
from app.models.user import User

class ReportService:
    def __init__(self, db: AsyncSession):
        self.db = db

    async def generate_weekly_summary(
        self, week_start: date | None = None
    ) -> dict[str, Any]:
        if week_start is None:
            today = date.today()
            week_start = today - timedelta(days=today.weekday())

        stmt = (
            select(ProductivityScore)
            .where(
                ProductivityScore.week_start == week_start,
                ProductivityScore.deleted_at.is_(None),
            )
            .options(
                selectinload(ProductivityScore.employee)
```

```
                .selectinload(Employee.user)
            )
            .order_by(ProductivityScore.final_score.desc())
        )
        result = await self.db.execute(stmt)
        scores = result.scalars().all()

        if not scores:
            return {
                "week_start": week_start.isoformat(),
                "total_employees": 0,
                "average_score": None,
                "top_performer": None,
                "at_risk_employees": [],
                "rating_distribution": {
                    "Excellent": 0,
                    "Good": 0,
                    "Average": 0,
                    "Risk": 0,
                },
                "generated_at": datetime.now(timezone.utc).isoformat(),
            }

        total = len(scores)
        avg_score = sum(
            float(s.final_score) for s in scores if s.final_score
        ) / total

        distribution = {"Excellent": 0, "Good": 0, "Average": 0, "Risk": 0}
        for s in scores:
            if s.rating and s.rating in distribution:
                distribution[s.rating] += 1

        top = scores[0] if scores else None
        top_performer = None
        if top and top.employee and top.employee.user:
            top_performer = {
                "employee_id": top.employee_id,
                "name": top.employee.user.name,
                "employee_code": top.employee.employee_code,
                "final_score": float(top.final_score) if top.final_score
else 0,
                "rating": top.rating,
            }

        at_risk = []
        for s in scores:
            if s.final_score and float(s.final_score) < 60:
                emp_name = s.employee.user.name if s.employee and
s.employee.user else "Unknown"
                emp_code = s.employee.employee_code if s.employee else "N/A"
                at_risk.append({
                    "employee_id": s.employee_id,
                    "name": emp_name,
                    "employee_code": emp_code,
                    "final_score": float(s.final_score),
                    "rating": s.rating,
                    "anomaly_flags": s.anomaly_flags or {},
                })

        return {
            "week_start": week_start.isoformat(),
            "total_employees": total,
            "average_score": round(avg_score, 2),
            "top_performer": top_performer,
            "at_risk_employees": at_risk,
            "at_risk_count": len(at_risk),
            "rating_distribution": distribution,
```

```
            "generated_at": datetime.now(timezone.utc).isoformat(),
        }

    async def generate_attendance_summary(
        self, target_date: date | None = None
    ) -> dict[str, Any]:
        from app.models.attendance import Attendance

        if target_date is None:
            target_date = date.today()

        total_emp = (
            await self.db.execute(
                select(func.count(Employee.id)).where(
                    Employee.status == "Active",
     Employee.deleted_at.is_(None)
                )
            )
        ).scalar() or 0

        present = (
            await self.db.execute(
                select(func.count(Attendance.id)).where(
                    Attendance.attendance_date == target_date,
                    Attendance.deleted_at.is_(None),
                )
            )
        ).scalar() or 0

        office_count = (
            await self.db.execute(
                select(func.count(Attendance.id)).where(
                    Attendance.attendance_date == target_date,
                    Attendance.location_status == "office",
                    Attendance.deleted_at.is_(None),
                )
            )
        ).scalar() or 0

        remote_count = (
            await self.db.execute(
                select(func.count(Attendance.id)).where(
                    Attendance.attendance_date == target_date,
                    Attendance.location_status == "remote",
                    Attendance.deleted_at.is_(None),
                )
            )
        ).scalar() or 0

        suspicious_count = (
            await self.db.execute(
                select(func.count(Attendance.id)).where(
                    Attendance.attendance_date == target_date,
                    Attendance.location_status == "suspicious",
                    Attendance.deleted_at.is_(None),
                )
            )
        ).scalar() or 0

        absent = total_emp - present

        return {
            "date": target_date.isoformat(),
            "total_employees": total_emp,
            "present": present,
            "absent": absent,
            "attendance_rate": round((present / total_emp * 100), 1) if
     total_emp > 0 else 0,
```

```
            "location_breakdown": {
                "office": office_count,
                "remote": remote_count,
                "suspicious": suspicious_count,
            },
            "generated_at": datetime.now(timezone.utc).isoformat(),
        }
```

backend/app/services/project_report.py

```
import csv
import io
from datetime import date

from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy.orm import selectinload

from app.models.project import Project
from app.models.task import Task

COLUMNS = [
    "Project", "Project status", "Task group", "Group completion %", "Task",
     "Status", "Priority",
    "Assignee", "Created", "Due", "Completed", "Completed by", "GitHub
     issue",
]

def _in_range(task: Task, start: date, end: date) -> bool:
    days = [
        task.created_at.date() if task.created_at else None,
        task.due_date,
        task.completed_at.date() if task.completed_at else None,
    ]
    return any(d is not None and start <= d <= end for d in days)

async def report_rows(db: AsyncSession, projects: list[Project], start:
     date, end: date) -> list[list]:
    if not projects:
        return []
    by_id = {p.id: p for p in projects}
    tasks = (await db.execute(
        select(Task).where(Task.project_id.in_(list(by_id)),
     Task.deleted_at.is_(None))
        .options(selectinload(Task.task_group),
     selectinload(Task.assigned_to_user))
    )).scalars().all()
    tasks = [t for t in tasks if _in_range(t, start, end)]

    def order(t: Task):
        group = t.task_group
        return (by_id[t.project_id].name.lower(), group.sequence if group
     else 1_000_000,
                group.name.lower() if group else "", t.id)

    rows = []
    for t in sorted(tasks, key=order):
        project, group = by_id[t.project_id], t.task_group
        rows.append([
            project.name, project.status,
            group.name if group else "",
            f"{float(group.completion_percentage or 0):.0f}" if group else
     "",
            t.title, t.status, t.priority,
            t.assigned_to_user.name if t.assigned_to_user else "",
            t.created_at.date().isoformat() if t.created_at else "",
            t.due_date.isoformat() if t.due_date else "",
            t.completed_at.date().isoformat() if t.completed_at else "",
```

```
            t.completed_by or "",
            t.github_issue_url or "",
        ])
    return rows

def to_csv(rows: list[list]) -> str:
    out = io.StringIO()
    writer = csv.writer(out)
    writer.writerow(COLUMNS)
    writer.writerows(rows)
    return out.getvalue()
```

backend/app/services/system_settings.py

```
from cryptography.fernet import Fernet, InvalidToken
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from app.config import get_settings
from app.exceptions import TraceException
from app.models.project import SystemSetting

ENV_FALLBACKS = {
    ("github", "owner"): "GITHUB_OWNER",
    ("github", "token"): "GITHUB_TOKEN",
    ("github", "api_url"): "GITHUB_API_URL",
    ("github", "committer_name"): "GITHUB_COMMITTER_NAME",
    ("github", "committer_email"): "GITHUB_COMMITTER_EMAIL",
    ("mail", "host"): "SMTP_HOST",
    ("mail", "port"): "SMTP_PORT",
    ("mail", "from_address"): "SMTP_FROM",
}

SECRET_SETTINGS = {("github", "token"), ("mail", "password")}

class EncryptionKeyMissing(TraceException):
    def __init__(self):
        super().__init__(
            status_code=503,
            detail="SETTINGS_ENCRYPTION_KEY is not set in .env, so secrets
     cannot be stored or read",
        )

def _fernet() -> Fernet:
    key = get_settings().SETTINGS_ENCRYPTION_KEY
    if not key:
        raise EncryptionKeyMissing()
    try:
        return Fernet(key.encode())
    except ValueError:
        raise TraceException(503, "SETTINGS_ENCRYPTION_KEY in .env is not a
     valid Fernet key") from None

def encrypt_secret(value: str) -> str:
    return _fernet().encrypt(value.encode()).decode()

def decrypt_secret(token: str, label: str) -> str:
    try:
        return _fernet().decrypt(token.encode()).decode()
    except InvalidToken:
        raise TraceException(
            503,
            f"{label} cannot be decrypted — SETTINGS_ENCRYPTION_KEY has
     changed since it was saved. "
            "Enter it again in Admin → Configuration.",
        ) from None
```

```
async def _row(db: AsyncSession, category: str, key: str) -> SystemSetting |
     None:
    return (await db.execute(
        select(SystemSetting).where(
            SystemSetting.category == category,
            SystemSetting.key == key,
            SystemSetting.deleted_at.is_(None),
        )
    )).scalar_one_or_none()

def _env_value(category: str, key: str) -> str:
    attr = ENV_FALLBACKS.get((category, key))
    value = getattr(get_settings(), attr, "") if attr else ""
    return "" if value is None else str(value)

async def resolve_setting(db: AsyncSession, category: str, key: str) -> str:
    row = await _row(db, category, key)
    if row is not None and row.value:
        return decrypt_secret(row.value, f"{category}.{key}") if
     row.is_secret else row.value
    return _env_value(category, key)

async def setting_source(db: AsyncSession, category: str, key: str) -> str |
     None:
    row = await _row(db, category, key)
    if row is not None and row.value:
        return "database"
    return "env" if _env_value(category, key) else None

async def store_setting(db: AsyncSession, category: str, key: str, value:
     str | None) -> None:
    secret = (category, key) in SECRET_SETTINGS
    stored = encrypt_secret(value) if secret and value else (value or None)
    row = (await db.execute(
        select(SystemSetting).where(SystemSetting.category == category,
     SystemSetting.key == key)
    )).scalar_one_or_none()
    if row is None:
        db.add(SystemSetting(category=category, key=key, value=stored,
     is_secret=secret))
    else:
        row.value, row.is_secret, row.deleted_at = stored, secret, None
    await db.flush()
```

backend/app/services/workforce.py

```
import csv
import io
from dataclasses import dataclass, field
from datetime import date, datetime, time, timezone
from decimal import Decimal
from typing import Any, Awaitable, Callable

from pydantic import ValidationError
from sqlalchemy import exists, func, select
from sqlalchemy.exc import IntegrityError
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy.orm import selectinload

from app.exceptions import NotFoundException, TraceException
from app.models.attendance import Attendance
from app.models.employee import Department, Designation, Employee,
     WorkSchedule
from app.models.hrms import Holiday, Leave
from app.models.project import Project
from app.models.task import Task
from app.models.user import Role, User, UserRole
```

```
OPEN_TASK_STATUSES = ("Pending", "In Progress", "In Review", "Blocked",
     "Overdue")

@dataclass
class F:
    name: str
    label: str
    type: str = "text"
    required: bool = False
    options: list[str] = field(default_factory=list)
    create_only: bool = False
    list: bool = True
    read_only: bool = False

    def meta(self) -> dict:
        return {"name": self.name, "label": self.label, "type": self.type,
     "required": self.required,
                "options": self.options, "create_only": self.create_only,
     "list": self.list,
                "read_only": self.read_only}

def _coerce(f: F, raw: Any) -> Any:
    if raw is None or (isinstance(raw, str) and raw.strip() == ""):
        return None
    value = raw.strip() if isinstance(raw, str) else raw
    try:
        if f.type == "int":
            return int(value)
        if f.type == "date":
            return value if isinstance(value, date) else
     date.fromisoformat(str(value)[:10])
        if f.type == "time":
            return value if isinstance(value, time) else
     time.fromisoformat(str(value))
    except ValueError:
        kind = {"int": "a whole number", "date": "a date (YYYY-MM-DD)",
     "time": "a time (HH:MM)"}[f.type]
        raise TraceException(422, f"{f.label}: must be {kind}") from None
    if f.type == "email" and ("@" not in str(value) or
     str(value).startswith("@")):
        raise TraceException(422, f"{f.label}: must be an email address")
    if f.type == "select" and value not in f.options:
        raise TraceException(422, f"{f.label}: must be one of {',
     '.join(f.options)}")
    if f.type == "password" and len(str(value)) < 8:
        raise TraceException(422, f"{f.label}: must be at least 8
     characters")
    return value

def clean(fields: list[F], data: dict, *, partial: bool) -> dict:
    out = {}
    for f in fields:
        if f.create_only and partial:
            continue
        if f.name not in data:
            if f.required and not partial:
                raise TraceException(422, f"{f.label}: is required")
            continue
        value = _coerce(f, data[f.name])
        if value is None and f.required:
            raise TraceException(422, f"{f.label}: is required")
        out[f.name] = value
    return out

async def _by_name(db: AsyncSession, model, column, name: str | None, label:
     str):
    if not name:
        return None
```

```
    row = (await db.execute(select(model).where(func.lower(column) ==
     name.lower(),

     model.deleted_at.is_(None)))).scalar_one_or_none()
    if row is None:
        raise TraceException(422, f"{label}: no {label.lower()} called
     '{name}'")
    return row

async def _employee_by_code(db: AsyncSession, code: str) -> Employee:
    emp = (await db.execute(select(Employee).where(Employee.employee_code ==
     code,

     Employee.deleted_at.is_(None)))).scalar_one_or_none()
    if emp is None:
        raise TraceException(422, f"Employee code: no employee with code
     '{code}'")
    return emp

async def _get(db: AsyncSession, model, row_id: int, label: str):
    row = await db.get(model, row_id)
    if row is None or row.deleted_at is not None:
        raise NotFoundException(label)
    return row

def _iso(value) -> str | None:
    return value.isoformat() if value is not None else None

def _people_fields(role: str) -> list[F]:
    fields = [
        F("name", "Name", required=True),
        F("username", "Username", required=True),
        F("email", "Email", "email", required=True),
        F("password", "Password", "password", required=True,
     create_only=True, list=False),
        F("phone", "Phone", list=False),
    ]
    if role == "Employee":
        fields += [
            F("employee_code", "Employee code", required=True),
            F("github_username", "GitHub username", list=False),
            F("department", "Department"),
            F("designation", "Designation"),
            F("shift", "Shift", list=False),
            F("joining_date", "Joining date", "date", list=False),
        ]
    return fields + [F("status", "Status", "select", options=["Active",
     "Blocked"])]

async def _people(db: AsyncSession, role: str) -> list[User]:
    return list((await db.execute(
        select(User).join(UserRole, UserRole.user_id == User.id).join(Role,
     Role.id == UserRole.role_id)
        .where(Role.role_name == role, User.deleted_at.is_(None))
        .options(selectinload(User.employee).selectinload(Employee.departmen
     t),

     selectinload(User.employee).selectinload(Employee.designation),

     selectinload(User.employee).selectinload(Employee.work_schedule))
        .order_by(User.name)
    )).scalars().unique())

def _person_row(user: User) -> dict:
    emp = user.employee
    return {
        "id": user.id, "name": user.name, "username": user.username,
     "email": user.email,
```

```
        "phone": user.phone, "status": user.status, "github_username":
    user.github_username,
        "employee_code": emp.employee_code if emp else None,
        "department": emp.department.name if emp and emp.department else
    None,
        "designation": emp.designation.name if emp and emp.designation else
    None,
        "shift": emp.work_schedule.schedule_name if emp and
    emp.work_schedule else None,
        "joining_date": _iso(emp.joining_date) if emp else None,
    }

async def _apply_employee(db: AsyncSession, user: User, data: dict) -> None:
    keys = {"employee_code", "department", "designation", "shift",
    "joining_date"}
    if not keys & data.keys():
        return
    emp = (await db.execute(select(Employee).where(Employee.user_id ==
    user.id))).scalar_one_or_none()
    if emp is None:
        emp = Employee(user_id=user.id,
    employee_code=data.get("employee_code") or f"EMP-{user.id}")
        db.add(emp)
    if data.get("employee_code"):
        emp.employee_code = data["employee_code"]
    if "department" in data:
        dept = await _by_name(db, Department, Department.name,
    data["department"], "Department")
        emp.department_id = dept.id if dept else None
    if "designation" in data:
        des = await _by_name(db, Designation, Designation.name,
    data["designation"], "Designation")
        emp.designation_id = des.id if des else None
    if "shift" in data:
        shift = await _by_name(db, WorkSchedule, WorkSchedule.schedule_name,
    data["shift"], "Shift")
        emp.work_schedule_id = shift.id if shift else None
    if "joining_date" in data:
        emp.joining_date = data["joining_date"]
    if "status" in data:
        emp.status = "Active" if data["status"] == "Active" else "Blocked"

def people_entity(role: str):
    fields = _people_fields(role)

    async def list_(db):
        return [_person_row(u) for u in await _people(db, role)]

    async def create(db, data):
        from app.schemas.auth import RegisterRequest
        from app.services.auth_service import AuthService
        try:
            request = RegisterRequest(name=data["name"],
    username=data["username"], email=data["email"],
                                      password=data["password"],
    phone=data.get("phone"), role=role)
        except ValidationError as exc:
            raise TraceException(422, "; ".join(e["msg"] for e in
    exc.errors())) from None
        result = await AuthService(db).register(request)
        user = await db.get(User, result.user.id)
        user.github_username = data.get("github_username")
        user.status = data.get("status") or "Active"
        await _apply_employee(db, user, data)
        return user.id

    async def update(db, row_id, data):
        user = await _get(db, User, row_id, role)
```

```
            for key in ("name", "username", "email", "phone", "github_username",
         "status"):
                if key in data and (data[key] is not None or key in ("phone",
         "github_username")):
                    setattr(user, key, data[key])
            if role == "Employee":
                await _apply_employee(db, user, data)

        async def delete(db, row_id):
            user = await _get(db, User, row_id, role)
            if role == "Employee" and (await db.execute(select(exists().where(
                    Task.assigned_to_user_id == user.id,
         Task.deleted_at.is_(None),
                    Task.status.in_(OPEN_TASK_STATUSES))))).scalar():
                raise TraceException(409, f"{user.name} still has open tasks —
         reassign them first")
            if role == "Manager" and (await db.execute(select(exists().where(
                    Project.created_by_user_id == user.id,
         Project.deleted_at.is_(None))))).scalar():
                raise TraceException(409, f"{user.name} still owns projects")
            user.deleted_at, user.status = datetime.now(timezone.utc), "Blocked"

        return fields, list_, create, update, delete

    def simple_entity(model, fields: list[F], label: str, order, to_row:
         Callable[[Any], dict],
                      build: Callable[[AsyncSession, Any, dict],
         Awaitable[None]] | None = None,
                      used_by=None):
        async def list_(db):
            rows = (await
         db.execute(select(model).where(model.deleted_at.is_(None)).order_by(*ord
         er))).scalars()
            return [to_row(r) for r in rows]

        async def assign(db, row, data):
            if build:
                await build(db, row, data)
            else:
                for key, value in data.items():
                    setattr(row, key, value)

        async def create(db, data):
            row = model()
            await assign(db, row, data)
            db.add(row)
            await db.flush()
            return row.id

        async def update(db, row_id, data):
            await assign(db, await _get(db, model, row_id, label), data)

        async def delete(db, row_id):
            row = await _get(db, model, row_id, label)
            if used_by is not None and (await db.execute(select(exists().where(
                    used_by == row.id,
         Employee.deleted_at.is_(None))))).scalar():
                raise TraceException(409, f"This {label.lower()} is still
         assigned to employees")
            await db.delete(row)

        return fields, list_, create, update, delete

    def _named_row(r) -> dict:
        return {"id": r.id, "name": r.name, "description": r.description}

    def _shift_row(r: WorkSchedule) -> dict:
```

```
    return {"id": r.id, "schedule_name": r.schedule_name, "start_time":
     r.start_time.strftime("%H:%M"),
            "end_time": r.end_time.strftime("%H:%M"), "break_minutes":
     r.break_minutes}

def _holiday_row(r: Holiday) -> dict:
    return {"id": r.id, "name": r.name, "holiday_date":
     _iso(r.holiday_date), "type": r.type,
            "description": r.description}

async def _codes(db: AsyncSession, employee_ids) -> dict[int, tuple[str,
     str]]:
    rows = (await db.execute(select(Employee.id, Employee.employee_code,
     User.name)
                             .join(User, User.id == Employee.user_id)
                             .where(Employee.id.in_(list(employee_ids) or [-
     1])))).all()
    return {r[0]: (r[1], r[2]) for r in rows}

LEAVE_FIELDS = [
    F("employee_code", "Employee code", required=True),
    F("employee_name", "Employee", read_only=True),
    F("leave_type", "Type", "select", required=True, options=["Casual",
     "Sick", "Earned", "Unpaid"]),
    F("start_date", "From", "date", required=True),
    F("end_date", "To", "date", required=True),
    F("reason", "Reason", list=False),
    F("status", "Status", "select", options=["Pending", "Approved",
     "Rejected"]),
]

ATTENDANCE_FIELDS = [
    F("employee_code", "Employee code", required=True),
    F("employee_name", "Employee", read_only=True),
    F("attendance_date", "Date", "date", required=True),
    F("time_in", "In", "time"),
    F("time_out", "Out", "time"),
    F("location_status", "Where", "select", options=["office", "remote"]),
    F("location", "Location", list=False),
]

async def _build_leave(db, row: Leave, data: dict) -> None:
    if data.get("employee_code"):
        row.employee_id = (await _employee_by_code(db,
     data["employee_code"])).id
    for key in ("leave_type", "start_date", "end_date", "reason", "status"):
        if key in data and (data[key] is not None or key == "reason"):
            setattr(row, key, data[key])
    row.status = row.status or "Pending"
    if row.start_date and row.end_date and row.end_date < row.start_date:
        raise TraceException(422, "To: is before From")

async def _build_attendance(db, row: Attendance, data: dict) -> None:
    if data.get("employee_code"):
        row.employee_id = (await _employee_by_code(db,
     data["employee_code"])).id
    if data.get("attendance_date"):
        row.attendance_date = data["attendance_date"]
    for key in ("time_in", "time_out"):
        if key in data:
            value = data[key]
            setattr(row, key, datetime.combine(row.attendance_date, value,
     tzinfo=timezone.utc) if value else None)
    for key in ("location", "location_status"):
        if key in data:
            setattr(row, key, data[key])
    if row.time_in and row.time_out:
        if row.time_out < row.time_in:
```

```
            raise TraceException(422, "Out: is before In")
        row.total_hours = Decimal(str(round((row.time_out -
     row.time_in).total_seconds() / 3600, 2)))
    else:
        row.total_hours = None

async def _list_with_codes(db, model, order, to_row):
    rows = list((await
     db.execute(select(model).where(model.deleted_at.is_(None)).order_by(*ord
     er))).scalars())
    codes = await _codes(db, {r.employee_id for r in rows})
    return [to_row(r, *codes.get(r.employee_id, ("", ""))) for r in rows]

def _leave_row(r: Leave, code: str, name: str) -> dict:
    return {"id": r.id, "employee_code": code, "employee_name": name,
     "leave_type": r.leave_type,
            "start_date": _iso(r.start_date), "end_date": _iso(r.end_date),
     "reason": r.reason,
            "status": r.status}

def _attendance_row(r: Attendance, code: str, name: str) -> dict:
    return {"id": r.id, "employee_code": code, "employee_name": name,
            "attendance_date": _iso(r.attendance_date),
            "time_in": r.time_in.astimezone(timezone.utc).strftime("%H:%M")
     if r.time_in else None,
            "time_out": r.time_out.astimezone(timezone.utc).strftime("%H:
     %M") if r.time_out else None,
            "location_status": r.location_status, "location": r.location}

def _with_codes(entity, model, order, to_row):
    fields, _, create, update, delete = entity

    async def list_(db):
        return await _list_with_codes(db, model, order, to_row)
    return fields, list_, create, update, delete

ENTITIES: dict[str, tuple[str, tuple]] = {
    "employees": ("Employees", people_entity("Employee")),
    "managers": ("Managers", people_entity("Manager")),
    "leaves": ("Leaves", _with_codes(
        simple_entity(Leave, LEAVE_FIELDS, "Leave", [], lambda r: r,
     _build_leave),
        Leave, [Leave.start_date.desc(), Leave.id.desc()], _leave_row)),
    "holidays": ("Holidays", simple_entity(
        Holiday, [F("name", "Name", required=True), F("holiday_date",
     "Date", "date", required=True),
                  F("type", "Type", "select", options=["Public", "Optional",
     "Company"]),
                  F("description", "Description")],
        "Holiday", [Holiday.holiday_date], _holiday_row)),
    "attendance": ("Attendance", _with_codes(
        simple_entity(Attendance, ATTENDANCE_FIELDS, "Attendance record",
     [], lambda r: r, _build_attendance),
        Attendance, [Attendance.attendance_date.desc(),
     Attendance.id.desc()], _attendance_row)),
    "shifts": ("Shifts", simple_entity(
        WorkSchedule, [F("schedule_name", "Name", required=True),
     F("start_time", "Starts", "time", required=True),
                       F("end_time", "Ends", "time", required=True),
     F("break_minutes", "Break (min)", "int")],
        "Shift", [WorkSchedule.start_time], _shift_row,
     used_by=Employee.work_schedule_id)),
    "departments": ("Departments", simple_entity(
        Department, [F("name", "Name", required=True), F("description",
     "Description")],
        "Department", [Department.name], _named_row,
     used_by=Employee.department_id)),
    "designations": ("Designations", simple_entity(
```

```
        Designation, [F("name", "Name", required=True), F("description",
     "Description")],
        "Designation", [Designation.name], _named_row,
     used_by=Employee.designation_id)),
}

def entity(name: str):
    if name not in ENTITIES:
        raise NotFoundException("Workforce tab")
    return ENTITIES[name]

def writable(fields: list[F]) -> list[F]:
    return [f for f in fields if not f.read_only]

def catalogue() -> list[dict]:
    return [{"key": key, "label": label, "fields": [f.meta() for f in
     spec[0]]}
            for key, (label, spec) in ENTITIES.items()]

async def list_rows(db: AsyncSession, name: str) -> list[dict]:
    return await entity(name)[1][1](db)

async def _write(db: AsyncSession, action) -> Any:
    try:
        result = await action()
        await db.flush()
        return result
    except IntegrityError as exc:
        text = str(exc.orig).lower()
        if "unique" in text or "duplicate" in text:
            raise TraceException(409, "A record with that value already
     exists") from None
        if "foreign key" in text:
            raise TraceException(409, "It is still referenced by other
     records") from None
        raise TraceException(409, "The database refused the change") from
     None

async def create_row(db: AsyncSession, name: str, data: dict) -> int:
    fields, _, create, _, _ = entity(name)[1]
    values = clean(writable(fields), data, partial=False)
    return await _write(db, lambda: create(db, values))

async def update_row(db: AsyncSession, name: str, row_id: int, data: dict) -
     > None:
    fields, _, _, update, _ = entity(name)[1]
    values = clean(writable(fields), data, partial=True)
    await _write(db, lambda: update(db, row_id, values))

async def delete_row(db: AsyncSession, name: str, row_id: int) -> None:
    await _write(db, lambda: entity(name)[1][4](db, row_id))

def template(name: str) -> str:
    fields = writable(entity(name)[1][0])
    out = io.StringIO()
    csv.writer(out).writerow([f.name for f in fields])
    return out.getvalue()

async def bulk_upload(db: AsyncSession, name: str, content: bytes) -> dict:
    entity(name)
    try:
        text = content.decode("utf-8-sig")
    except UnicodeDecodeError:
        raise TraceException(422, "The file is not UTF-8 text — save it as
     CSV (UTF-8)") from None
    reader = csv.DictReader(io.StringIO(text))
    if not reader.fieldnames:
        raise TraceException(422, "The file is empty")
```

```
    known = {f.name for f in writable(entity(name)[1][0])}
    unknown = [h for h in reader.fieldnames if h and h.strip() not in known]
    created, rejected = 0, []
    for line, raw in enumerate(reader, start=2):
        row = {(k or "").strip(): v for k, v in raw.items() if k}
        if not any((v or "").strip() for v in row.values()):
            continue
        try:
            async with db.begin_nested():
                await create_row(db, name, row)
            created += 1
        except TraceException as exc:
            rejected.append({"line": line, "error": exc.detail})
    await db.commit()
    return {"created": created, "rejected": rejected, "ignored_columns":
     unknown}
```

4.1.8 Backend: API Routes

backend/app/api/v1/router.py

```
from fastapi import APIRouter

from app.api.v1.auth import router as auth_router
from app.api.v1.users import router as users_router
from app.api.v1.employees import router as employees_router
from app.api.v1.attendance import router as attendance_router
from app.api.v1.projects import router as projects_router
from app.api.v1.tasks import router as tasks_router
from app.api.v1.productivity import router as productivity_router
from app.api.v1.services import router as services_router
from app.api.v1.hrms import router as hrms_router
from app.api.v1.admin import router as admin_router
from app.api.v1.settings import router as settings_router
from app.api.v1.notifications import router as notifications_router,
     ws_router
from app.api.v1.workforce import router as workforce_router

router = APIRouter(prefix="/api/v1")

router.include_router(auth_router)
router.include_router(users_router)
router.include_router(employees_router)
router.include_router(attendance_router)
router.include_router(hrms_router)
router.include_router(projects_router)
router.include_router(tasks_router)
router.include_router(productivity_router)
router.include_router(services_router)
router.include_router(settings_router)
router.include_router(notifications_router)
router.include_router(ws_router)
router.include_router(workforce_router)
router.include_router(admin_router, prefix="/admin", tags=["Admin Panel"])
```

backend/app/api/v1/auth.py

```
from fastapi import APIRouter, Depends, status
from sqlalchemy.ext.asyncio import AsyncSession

from app.database import get_db
from app.dependencies import get_optional_current_user
from app.exceptions import InsufficientPermissionsException
from app.models.user import User
from app.schemas.auth import (
    LoginRequest, LoginResponse,
    RefreshTokenRequest, RegisterRequest, RegisterResponse, TokenResponse,
)
```

```
from app.services.auth_service import AuthService

router = APIRouter(prefix="/auth", tags=["Authentication"])

@router.post(
    "/register",
    response_model=RegisterResponse,
    status_code=status.HTTP_201_CREATED,
    summary="Register a new user",
    description=(
        "Create a new user account. Self-registration always creates an
     Employee; "
        "only an authenticated Admin may create Admin or Manager accounts."
    ),
)
async def register(
    data: RegisterRequest,
    caller: User | None = Depends(get_optional_current_user),
    db: AsyncSession = Depends(get_db),
):
    if data.role != "Employee":
        caller_roles = {ur.role.role_name for ur in (caller.user_roles if
     caller else []) if ur.role}
        if "Admin" not in caller_roles:
            raise InsufficientPermissionsException()
    service = AuthService(db)
    return await service.register(data)

@router.post(
    "/login",
    response_model=LoginResponse,
    summary="User login",
    description="Authenticate with email and password. Returns JWT token
     pair.",
)
async def login(data: LoginRequest, db: AsyncSession = Depends(get_db)):
    service = AuthService(db)
    return await service.login(data)

@router.post(
    "/refresh",
    response_model=TokenResponse,
    summary="Refresh access token",
    description="Exchange a valid refresh token for a new token pair.",
)
async def refresh_token(data: RefreshTokenRequest, db: AsyncSession =
     Depends(get_db)):
    service = AuthService(db)
    return await service.refresh_token(data.refresh_token)
```

backend/app/api/v1/users.py

```
from fastapi import APIRouter, Depends, Query
from sqlalchemy.ext.asyncio import AsyncSession

from app.core.permissions import require_admin, require_manager_or_admin
from app.database import get_db
from app.dependencies import get_current_user
from app.models.user import User
from app.schemas.user import UserListResponse, UserResponse, UserUpdate
from app.services.user_service import UserService

router = APIRouter(prefix="/users", tags=["Users"])

@router.get("/me", response_model=UserResponse, summary="Get current user")
async def get_me(
    current_user: User = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),
```

```
):
    service = UserService(db)
    return await service.get_user_by_id(current_user.id)

@router.get(
    "",
    response_model=UserListResponse,
    summary="List all users",
    description="Admin/Manager only.",
)
async def list_users(
    page: int = Query(1, ge=1),
    per_page: int = Query(20, ge=1, le=100),
    current_user: User = Depends(require_manager_or_admin),
    db: AsyncSession = Depends(get_db),
):
    service = UserService(db)
    return await service.list_users(page=page, per_page=per_page)

@router.get("/{user_id}", response_model=UserResponse, summary="Get user by
     ID")
async def get_user(
    user_id: int,
    current_user: User = Depends(require_manager_or_admin),
    db: AsyncSession = Depends(get_db),
):
    service = UserService(db)
    return await service.get_user_by_id(user_id)

@router.patch(
    "/{user_id}",
    response_model=UserResponse,
    summary="Update user",
    description="Users can update their own profile. Admin can update any
     user.",
)
async def update_user(
    user_id: int,
    data: UserUpdate,
    current_user: User = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),
):
    if current_user.id != user_id:
        from app.exceptions import InsufficientPermissionsException

        user_roles = set()
        if current_user.user_roles:
            for ur in current_user.user_roles:
                if ur.role:
                    user_roles.add(ur.role.role_name)
        if "Admin" not in user_roles:
            raise InsufficientPermissionsException()

    service = UserService(db)
    return await service.update_user(user_id, data)

@router.delete(
    "/{user_id}",
    summary="Soft-delete user",
    description="Admin only.",
)
async def delete_user(
    user_id: int,
    current_user: User = Depends(require_admin),
    db: AsyncSession = Depends(get_db),
):
    service = UserService(db)
    return await service.soft_delete_user(user_id)
```

backend/app/api/v1/employees.py

```
from fastapi import APIRouter, Depends, Query, status
from sqlalchemy.ext.asyncio import AsyncSession

from app.core.permissions import require_admin
from app.database import get_db
from app.dependencies import get_current_user
from app.models.user import User
from app.schemas.employee import (
    DepartmentCreate,
    DepartmentResponse,
    DesignationCreate,
    DesignationResponse,
    EmployeeCreate,
    EmployeeListResponse,
    EmployeeResponse,
    EmployeeUpdate,
    WorkScheduleCreate,
    WorkScheduleResponse,
)
from app.services.employee_service import (
    DepartmentService,
    DesignationService,
    EmployeeService,
    WorkScheduleService,
)

router = APIRouter(tags=["Employees"])

@router.post(
    "/designations",
    response_model=DesignationResponse,
    status_code=status.HTTP_201_CREATED,
    summary="Create designation",
    description="Admin/Manager only.",
)
async def create_designation(
    data: DesignationCreate,
    current_user: User = Depends(require_admin),
    db: AsyncSession = Depends(get_db),
):
    service = DesignationService(db)
    return await service.create(data)

@router.get(
    "/designations",
    response_model=list[DesignationResponse],
    summary="List all designations",
)
async def list_designations(
    current_user: User = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),
):
    service = DesignationService(db)
    return await service.list_all()

@router.delete(
    "/designations/{desig_id}",
    summary="Delete designation",
    description="Admin only.",
)
async def delete_designation(
    desig_id: int,
    current_user: User = Depends(require_admin),
    db: AsyncSession = Depends(get_db),
):
    service = DesignationService(db)
```

```
    return await service.delete(desig_id)

@router.post(
    "/departments",
    response_model=DepartmentResponse,
    status_code=status.HTTP_201_CREATED,
    summary="Create department",
    description="Admin/Manager only.",
)
async def create_department(
    data: DepartmentCreate,
    current_user: User = Depends(require_admin),
    db: AsyncSession = Depends(get_db),
):
    service = DepartmentService(db)
    return await service.create(data)

@router.get(
    "/departments",
    response_model=list[DepartmentResponse],
    summary="List all departments",
)
async def list_departments(
    current_user: User = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),
):
    service = DepartmentService(db)
    return await service.list_all()

@router.get(
    "/departments/{dept_id}",
    response_model=DepartmentResponse,
    summary="Get department by ID",
)
async def get_department(
    dept_id: int,
    current_user: User = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),
):
    service = DepartmentService(db)
    return await service.get_by_id(dept_id)

@router.delete(
    "/departments/{dept_id}",
    summary="Delete department (soft)",
    description="Admin only.",
)
async def delete_department(
    dept_id: int,
    current_user: User = Depends(require_admin),
    db: AsyncSession = Depends(get_db),
):
    service = DepartmentService(db)
    return await service.delete(dept_id)

@router.post(
    "/work-schedules",
    response_model=WorkScheduleResponse,
    status_code=status.HTTP_201_CREATED,
    summary="Create work schedule",
    description="Admin/Manager only.",
)
async def create_work_schedule(
    data: WorkScheduleCreate,
    current_user: User = Depends(require_admin),
    db: AsyncSession = Depends(get_db),
):
    service = WorkScheduleService(db)
```

```
    return await service.create(data)

@router.get(
    "/work-schedules",
    response_model=list[WorkScheduleResponse],
    summary="List all work schedules",
)
async def list_work_schedules(
    current_user: User = Depends(require_admin),
    db: AsyncSession = Depends(get_db),
):
    service = WorkScheduleService(db)
    return await service.list_all()

@router.get(
    "/work-schedules/{schedule_id}",
    response_model=WorkScheduleResponse,
    summary="Get work schedule by ID",
)
async def get_work_schedule(
    schedule_id: int,
    current_user: User = Depends(require_admin),
    db: AsyncSession = Depends(get_db),
):
    service = WorkScheduleService(db)
    return await service.get_by_id(schedule_id)

@router.post(
    "/employees",
    response_model=EmployeeResponse,
    status_code=status.HTTP_201_CREATED,
    summary="Create employee record",
    description="Admin/Manager only. Links a user to the company
     structure.",
)
async def create_employee(
    data: EmployeeCreate,
    current_user: User = Depends(require_admin),
    db: AsyncSession = Depends(get_db),
):
    service = EmployeeService(db)
    return await service.create(data)

@router.get(
    "/employees",
    response_model=EmployeeListResponse,
    summary="List employees",
    description="Filter by department and status. Admin/Manager only.",
)
async def list_employees(
    page: int = Query(1, ge=1),
    per_page: int = Query(20, ge=1, le=100),
    department_id: int | None = None,
    emp_status: str | None = Query(None, alias="status"),
    current_user: User = Depends(require_admin),
    db: AsyncSession = Depends(get_db),
):
    service = EmployeeService(db)
    return await service.list_employees(
        page=page, per_page=per_page,
        department_id=department_id, status=emp_status,
    )

@router.get(
    "/employees/me",
    response_model=EmployeeResponse,
    summary="Get my employee record",
)
```

```
async def get_my_employee_record(
    current_user: User = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),
):
    service = EmployeeService(db)
    return await service.get_by_user_id(current_user.id)

@router.get(
    "/employees/{employee_id}",
    response_model=EmployeeResponse,
    summary="Get employee by ID",
)
async def get_employee(
    employee_id: int,
    current_user: User = Depends(require_admin),
    db: AsyncSession = Depends(get_db),
):
    service = EmployeeService(db)
    return await service.get_by_id(employee_id)

@router.patch(
    "/employees/{employee_id}",
    response_model=EmployeeResponse,
    summary="Update employee",
    description="Admin/Manager only.",
)
async def update_employee(
    employee_id: int,
    data: EmployeeUpdate,
    current_user: User = Depends(require_admin),
    db: AsyncSession = Depends(get_db),
):
    service = EmployeeService(db)
    return await service.update(employee_id, data)

@router.delete(
    "/employees/{employee_id}",
    summary="Deactivate employee (soft delete)",
    description="Admin only.",
)
async def deactivate_employee(
    employee_id: int,
    current_user: User = Depends(require_admin),
    db: AsyncSession = Depends(get_db),
):
    service = EmployeeService(db)
    return await service.deactivate(employee_id)
```

backend/app/api/v1/projects.py

```
import uuid
from datetime import date, datetime, timezone

from fastapi import APIRouter, Depends, File, Response, UploadFile, status
from sqlalchemy import func, select
from sqlalchemy.ext.asyncio import AsyncSession

from app.core.permissions import RoleEnum, require_any_role, require_roles
from app.core.project_access import (
    MEMBER, PROJECT_MANAGER, ProjectAccess, resolve_project_access,
)
from app.database import get_db
from app.exceptions import NotFoundException, TraceException
from app.models.project import ProjectAssignment, ProjectSourceFile,
     TaskGroup
from app.models.task import Task
from app.models.user import User
from app.schemas.project import (
```

```
    ActivityItem, AssigneeSuggestionRequest, AssignmentResponse,
     AssignProjectManager, DocumentKind,
    DocumentResponse, DocumentSaved, DocumentUpdate, EvaluationStatus,
     JobQueued, PersonBrief, ProjectCreate,
    ProjectDetail, ProjectSummary, ProjectUpdate, RemovalResponse,
     SourceFileUploadResponse, TaskCreate,
    TaskGroupCreate, TaskGroupResponse, TaskGroupUpdate,
     TaskMutationResponse, TaskResponse,
)
from app.schemas.gantt import GanttResponse, GanttVersionBrief, ProjectTree
from app.services import ai_jobs, ai_service, gantt_service, jobs,
     project_report
from app.services import project_service as svc
from app.services.extraction import (
    MAX_SOURCE_FILE_BYTES, UnsupportedFile, extension_of, extract_text,
)
from app.services.github_service import GitHubClient, get_github_client,
     require_github, resolve_owner
from app.services.storage import ObjectStorage, get_storage

router = APIRouter(prefix="/projects", tags=["Projects"],
     dependencies=[Depends(require_any_role)])

async def project_access(
    project_id: int,
    user: User = Depends(require_any_role),
    db: AsyncSession = Depends(get_db),
) -> ProjectAccess:
    return await resolve_project_access(db, user, project_id)

@router.get("", response_model=list[ProjectSummary], summary="Projects
     visible to the caller")
async def list_projects(user: User = Depends(require_any_role), db:
     AsyncSession = Depends(get_db)):
    projects, my_roles = await svc.visible_projects(db, user)
    return await svc.summarize(db, projects, my_roles)

@router.post("", response_model=ProjectDetail,
     status_code=status.HTTP_201_CREATED,
              summary="Create a project and its GitHub repository (Manager)")
async def create_project(
    data: ProjectCreate,
    user: User = Depends(require_roles(RoleEnum.MANAGER)),
    github: GitHubClient | None = Depends(get_github_client),
    db: AsyncSession = Depends(get_db),
):
    github = require_github(github)
    project = await svc.create_project(db, user, data, github, await
     resolve_owner(db))
    return await svc.project_detail(db, await resolve_project_access(db,
     user, project.id))

@router.get("/activity", response_model=list[ActivityItem], summary="Recent
     task activity")
async def recent_activity(
    limit: int = 15,
    user: User = Depends(require_any_role),
    db: AsyncSession = Depends(get_db),
):
    projects, _ = await svc.visible_projects(db, user)
    if not projects:
        return []
    names = {p.id: p.name for p in projects}
    rows = (await db.execute(
        select(Task).where(Task.project_id.in_(list(names)),
     Task.deleted_at.is_(None))
        .order_by(func.coalesce(Task.updated_at,
     Task.created_at).desc()).limit(min(limit, 50))
```

```
    )).scalars().all()
    out = []
    for t in rows:
        t = await svc.load_task(db, t.id)
        out.append({"task_id": t.id, "task_title": t.title, "project_id":
     t.project_id,
                    "project_name": names[t.project_id], "status": t.status,
                    "assigned_to": svc.person(t.assigned_to_user),
                    "updated_at": t.updated_at or t.created_at})
    return out

@router.get("/report", summary="Download the project → task group → task
     report for a date range (CSV)",
            response_class=Response, responses={200: {"content":
     {"text/csv": {}}}})
async def download_report(
    start: date,
    end: date,
    project_id: int | None = None,
    user: User = Depends(require_roles(RoleEnum.MANAGER,
     RoleEnum.EMPLOYEE)),
    db: AsyncSession = Depends(get_db),
):
    if end < start:
        raise TraceException(422, "The end date is before the start date")
    if project_id is not None:
        projects = [(await resolve_project_access(db, user,
     project_id)).project]
    else:
        projects, _ = await svc.visible_projects(db, user)
    body = project_report.to_csv(await project_report.report_rows(db,
     projects, start, end))
    name = f"trace-report-{start.isoformat()}-to-{end.isoformat()}.csv"
    return Response(body, media_type="text/csv; charset=utf-8",
                    headers={"Content-Disposition": f'attachment;
     filename="{name}"'})

@router.get("/{project_id}", response_model=ProjectDetail)
async def get_project(access: ProjectAccess = Depends(project_access), db:
     AsyncSession = Depends(get_db)):
    return await svc.project_detail(db, access)

@router.patch("/{project_id}", response_model=ProjectDetail, summary="Edit
     project details (owner)")
async def update_project(
    data: ProjectUpdate,
    access: ProjectAccess = Depends(project_access),
    db: AsyncSession = Depends(get_db),
):
    access.require_owner()
    for field, value in data.model_dump(exclude_unset=True).items():
        setattr(access.project, field, value)
    await db.commit()
    return await svc.project_detail(db, access)

@router.delete("/{project_id}", status_code=status.HTTP_204_NO_CONTENT,
               summary="Delete a project and archive its repository
     (owner)")
async def delete_project(
    access: ProjectAccess = Depends(project_access),
    github: GitHubClient | None = Depends(get_github_client),
    db: AsyncSession = Depends(get_db),
):
    access.require_owner()
    open_tasks = (await db.execute(
        select(func.count(Task.id)).where(Task.project_id ==
     access.project.id, Task.deleted_at.is_(None),
```

```
                                            Task.status.notin_(["Completed",
    "Cancelled"]))
    )).scalar()
    if open_tasks:
        raise TraceException(409, f"{open_tasks} open task(s) — archive the
    project instead, or close them first")
    from app.services.system_settings import resolve_setting
    await svc.delete_project(db, access.project, github, await
    resolve_setting(db, "github", "owner"))

@router.post("/{project_id}/source-files",
    response_model=SourceFileUploadResponse,
             status_code=status.HTTP_201_CREATED, summary="Upload
    requirement documents (owner)")
async def upload_source_files(
    files: list[UploadFile] = File(...),
    access: ProjectAccess = Depends(project_access),
    storage: ObjectStorage = Depends(get_storage),
    db: AsyncSession = Depends(get_db),
):
    import asyncio

    access.require_owner()
    uploaded, rejected = [], []
    for upload in files:
        name = upload.filename or "unnamed"
        data = await upload.read()
        if not data:
            rejected.append({"file_name": name, "reason": "empty file"})
            continue
        if len(data) > MAX_SOURCE_FILE_BYTES:
            rejected.append({"file_name": name, "reason": f"larger than
    {MAX_SOURCE_FILE_BYTES // 2**20} MB"})
            continue
        try:
            text = await asyncio.to_thread(extract_text, name, data)
        except UnsupportedFile as exc:
            rejected.append({"file_name": name, "reason": str(exc).split(":
    ", 1)[-1]})
            continue
        key = f"projects/{access.project.id}/source/{uuid.uuid4().hex}
    {extension_of(name)}"
        await storage.put(key, data, upload.content_type or
    "application/octet-stream")
        row = ProjectSourceFile(
            project_id=access.project.id, file_name=name, storage_key=key,
            content_type=upload.content_type, size_bytes=len(data),
    extracted_text=text,
            uploaded_by_user_id=access.user.id,
        )
        db.add(row)
        await db.flush()
        await db.refresh(row)
        uploaded.append(svc.source_file_dict(row))
    await db.commit()
    return {"uploaded": uploaded, "rejected": rejected}

@router.delete("/{project_id}/source-files/{file_id}",
    status_code=status.HTTP_204_NO_CONTENT)
async def delete_source_file(
    file_id: int,
    access: ProjectAccess = Depends(project_access),
    storage: ObjectStorage = Depends(get_storage),
    db: AsyncSession = Depends(get_db),
):
    access.require_owner()
    row = (await db.execute(
        select(ProjectSourceFile).where(ProjectSourceFile.id == file_id,
```

```
                                        ProjectSourceFile.project_id ==
    access.project.id,

    ProjectSourceFile.deleted_at.is_(None))
    )).scalar_one_or_none()
    if row is None:
        raise NotFoundException("Source file")
    row.deleted_at = datetime.now(timezone.utc)
    await db.commit()
    await storage.delete(row.storage_key)

@router.get("/{project_id}/assignments",
    response_model=list[AssignmentResponse])
async def list_assignments(access: ProjectAccess = Depends(project_access),
    db: AsyncSession = Depends(get_db)):
    rows = (await db.execute(
        select(ProjectAssignment, User).join(User, User.id ==
    ProjectAssignment.user_id)
        .where(ProjectAssignment.project_id == access.project.id,
    ProjectAssignment.deleted_at.is_(None))
        .order_by(ProjectAssignment.project_role, User.name)
    )).all()
    open_counts = dict((await db.execute(
        select(Task.assigned_to_user_id, func.count(Task.id))
        .where(Task.project_id == access.project.id,
    Task.deleted_at.is_(None),
               Task.status.in_(svc.OPEN_STATUSES))
        .group_by(Task.assigned_to_user_id)
    )).all())
    return [{"user": svc.person(u), "project_role": a.project_role,
             "open_tasks": open_counts.get(u.id, 0), "assigned_at":
    a.created_at} for a, u in rows]

@router.post("/{project_id}/assignments", response_model=AssignmentResponse,
             status_code=status.HTTP_201_CREATED, summary="Assign a Project
    Manager (owner)")
async def assign_project_manager(
    data: AssignProjectManager,
    access: ProjectAccess = Depends(project_access),
    github: GitHubClient | None = Depends(get_github_client),
    db: AsyncSession = Depends(get_db),
):
    access.require_owner()
    employee = await svc.active_employee(db, data.user_id)
    row = (await db.execute(
        select(ProjectAssignment).where(ProjectAssignment.project_id ==
    access.project.id,
                                        ProjectAssignment.user_id ==
    employee.id)
    )).scalar_one_or_none()
    if row is None:
        row = ProjectAssignment(project_id=access.project.id,
    user_id=employee.id)
        db.add(row)
    elif row.project_role == PROJECT_MANAGER and row.deleted_at is None:
        raise TraceException(409, f"{employee.name} is already a Project
    Manager on this project")
    row.project_role, row.deleted_at, row.assigned_by_user_id =
    PROJECT_MANAGER, None, access.user.id
    warnings = await svc.grant_repo_access(github, access.project, employee)
    await db.commit()
    await db.refresh(row)
    return {"user": svc.person(employee), "project_role": PROJECT_MANAGER,
    "open_tasks": 0,
            "assigned_at": row.created_at, "warnings": warnings}

@router.delete("/{project_id}/assignments/{user_id}",
    response_model=RemovalResponse,
```

```
              summary="Remove a Project Manager (owner)")
async def remove_project_manager(
    user_id: int,
    access: ProjectAccess = Depends(project_access),
    github: GitHubClient | None = Depends(get_github_client),
    db: AsyncSession = Depends(get_db),
):
    access.require_owner()
    row = (await db.execute(
        select(ProjectAssignment).where(ProjectAssignment.project_id ==
     access.project.id,
                                        ProjectAssignment.user_id ==
     user_id,
                                        ProjectAssignment.project_role ==
     PROJECT_MANAGER,

     ProjectAssignment.deleted_at.is_(None))
    )).scalar_one_or_none()
    if row is None:
        raise NotFoundException("Project Manager assignment")
    holds_tasks = (await db.execute(
        select(func.count(Task.id)).where(Task.project_id ==
     access.project.id,
                                          Task.assigned_to_user_id ==
     user_id, Task.deleted_at.is_(None))
    )).scalar()
    warnings = []
    if holds_tasks:
        row.project_role = MEMBER
    else:
        row.deleted_at = datetime.now(timezone.utc)
        warnings = await svc.revoke_repo_access(github, access.project,
     await db.get(User, user_id))
    await db.commit()
    return {"project_role": MEMBER if holds_tasks else None, "warnings":
     warnings}

@router.get("/{project_id}/candidates", response_model=list[PersonBrief],
            summary="Employees who can be given a role or a task here")
async def list_candidates(access: ProjectAccess = Depends(project_access),
     db: AsyncSession = Depends(get_db)):
    if not (access.is_owner or access.is_project_manager):
        from app.exceptions import InsufficientPermissionsException
        raise InsufficientPermissionsException()
    return [svc.person(u) for u in await svc.active_employees(db)]

async def _group_dicts(db: AsyncSession, groups: list[TaskGroup]) ->
     list[dict]:
    ids = [g.id for g in groups]
    counts = {gid: (0, 0) for gid in ids}
    if ids:
        from sqlalchemy import case
        for gid, total, done in (await db.execute(
            select(Task.task_group_id, func.count(Task.id),
     func.count(case((Task.status == "Completed", 1))))
            .where(Task.task_group_id.in_(ids), Task.deleted_at.is_(None),
     Task.status != "Cancelled")
            .group_by(Task.task_group_id)
        )).all():
            counts[gid] = (total, done)
    return [{
        "id": g.id, "project_id": g.project_id, "name": g.name,
     "description": g.description,
        "sequence": g.sequence, "estimated_duration_days":
     g.estimated_duration_days,
        "planned_start_date": g.planned_start_date, "planned_end_date":
     g.planned_end_date,
```

```
        "completion_percentage": float(g.completion_percentage or 0),
     "status": g.status,
        "created_by": g.created_by, "total_tasks": counts[g.id][0],
     "completed_tasks": counts[g.id][1],
    } for g in groups]

@router.get("/{project_id}/task-groups",
     response_model=list[TaskGroupResponse])
async def list_task_groups(access: ProjectAccess = Depends(project_access),
     db: AsyncSession = Depends(get_db)):
    groups = (await db.execute(
        select(TaskGroup).where(TaskGroup.project_id == access.project.id,
     TaskGroup.deleted_at.is_(None))
        .order_by(TaskGroup.sequence, TaskGroup.id)
    )).scalars().all()
    return await _group_dicts(db, list(groups))

@router.post("/{project_id}/task-groups", response_model=TaskGroupResponse,
     status_code=status.HTTP_201_CREATED)
async def create_task_group(
    data: TaskGroupCreate,
    access: ProjectAccess = Depends(project_access),
    db: AsyncSession = Depends(get_db),
):
    access.require_project_manager()
    values = data.model_dump()
    if values["sequence"] is None:
        values["sequence"] = ((await db.execute(
            select(func.max(TaskGroup.sequence)).where(TaskGroup.project_id
     == access.project.id,

     TaskGroup.deleted_at.is_(None))
        )).scalar() or 0) + 1
    group = TaskGroup(project_id=access.project.id, created_by="Human",
     status="Pending",
                      completion_percentage=0, **values)
    db.add(group)
    await db.commit()
    return (await _group_dicts(db, [group]))[0]

@router.patch("/{project_id}/task-groups/{group_id}",
     response_model=TaskGroupResponse)
async def update_task_group(
    group_id: int,
    data: TaskGroupUpdate,
    access: ProjectAccess = Depends(project_access),
    db: AsyncSession = Depends(get_db),
):
    access.require_project_manager()
    group = await svc.group_in_project(db, access.project.id, group_id)
    for field, value in data.model_dump(exclude_unset=True).items():
        setattr(group, field, value)
    await db.commit()
    return (await _group_dicts(db, [group]))[0]

@router.delete("/{project_id}/task-groups/{group_id}",
     status_code=status.HTTP_204_NO_CONTENT)
async def delete_task_group(
    group_id: int,
    access: ProjectAccess = Depends(project_access),
    db: AsyncSession = Depends(get_db),
):
    access.require_project_manager()
    group = await svc.group_in_project(db, access.project.id, group_id)
    has_tasks = (await db.execute(
        select(func.count(Task.id)).where(Task.task_group_id == group.id,
     Task.deleted_at.is_(None))
    )).scalar()
```

```
    if has_tasks:
        raise TraceException(409, "Move or delete this group's tasks first")
    group.deleted_at = datetime.now(timezone.utc)
    await db.commit()

@router.get("/{project_id}/tasks", response_model=list[TaskResponse])
async def list_project_tasks(
    task_group_id: int | None = None,
    mine: bool = False,
    access: ProjectAccess = Depends(project_access),
    db: AsyncSession = Depends(get_db),
):
    stmt = select(Task.id).where(Task.project_id == access.project.id,
     Task.deleted_at.is_(None))
    if task_group_id is not None:
        stmt = stmt.where(Task.task_group_id == task_group_id)
    if mine:
        stmt = stmt.where(Task.assigned_to_user_id == access.user.id)
    ids = (await db.execute(stmt.order_by(Task.due_date.asc().nulls_last(),
     Task.id))).scalars().all()
    return [svc.task_dict(await svc.load_task(db, i), access) for i in ids]

@router.post("/{project_id}/tasks", response_model=TaskMutationResponse,
     status_code=status.HTTP_201_CREATED,
            summary="Create a task — and its GitHub issue (Project
     Manager)")
async def create_project_task(
    data: TaskCreate,
    access: ProjectAccess = Depends(project_access),
    github: GitHubClient | None = Depends(get_github_client),
    db: AsyncSession = Depends(get_db),
):
    access.require_project_manager()
    task, warnings = await svc.create_task(db, access, data,
     require_github(github))
    return {"task": svc.task_dict(task, access), "warnings": warnings}

@router.get("/{project_id}/tree", response_model=ProjectTree,
            summary="Project → task groups → tasks → assignees, for the
     graph view")
async def get_project_tree(access: ProjectAccess = Depends(project_access),
     db: AsyncSession = Depends(get_db)):
    return await gantt_service.project_tree(db, access.project)

@router.get("/{project_id}/gantt", response_model=GanttResponse | None,
            summary="The latest Gantt version (or ?version=N); null before
     the first one")
async def get_gantt(
    version: int | None = None,
    access: ProjectAccess = Depends(project_access),
    db: AsyncSession = Depends(get_db),
):
    rows = await gantt_service.versions(db, access.project.id)
    if not rows:
        if version is not None:
            raise NotFoundException("Gantt version")
        return None
    row = rows[0] if version is None else next((r for r in rows if r.version
     == version), None)
    if row is None:
        raise NotFoundException("Gantt version")
    return {"version": row.version, "generated_reason":
     row.generated_reason, "generated_at": row.generated_at,
            "chart_data": row.chart_data, "latest_version": rows[0].version}

@router.get("/{project_id}/gantt/versions",
     response_model=list[GanttVersionBrief],
            summary="Gantt version history, newest first")
```

```
async def list_gantt_versions(access: ProjectAccess =
     Depends(project_access), db: AsyncSession = Depends(get_db)):
    return [{"version": r.version, "generated_reason": r.generated_reason,
     "generated_at": r.generated_at}
            for r in await gantt_service.versions(db, access.project.id)]

async def _require_ai(db: AsyncSession) -> None:
    if not ai_service.ai_configured():
        raise ai_service.AINotConfigured()

def _enqueue_or_503(name: str, *args) -> str:
    try:
        return jobs.enqueue(name, *args)
    except Exception as exc:
        raise TraceException(503, f"The job queue is unavailable
     ({exc.__class__.__name__}); try again shortly") from exc

@router.post("/{project_id}/evaluate", response_model=JobQueued,
     status_code=status.HTTP_202_ACCEPTED,
             summary="Evaluate Project — source files → documents, task
     groups, Gantt v1 (owner)")
async def evaluate_project(
    access: ProjectAccess = Depends(project_access),
    github: GitHubClient | None = Depends(get_github_client),
    db: AsyncSession = Depends(get_db),
):
    access.require_owner()
    await _require_ai(db)
    require_github(github)
    project = access.project
    if project.status == "Evaluating":
        raise TraceException(409, "This project is already being evaluated")
    has_text = (await
     db.execute(select(func.count(ProjectSourceFile.id)).where(
        ProjectSourceFile.project_id == project.id,
     ProjectSourceFile.deleted_at.is_(None),
        func.length(func.coalesce(ProjectSourceFile.extracted_text, "")) >
     0,
    ))).scalar()
    if not has_text:
        raise TraceException(409, "Upload at least one requirement document
     with readable text first")
    if (await db.execute(select(func.count(Task.id)).where(
            Task.project_id == project.id,
     Task.deleted_at.is_(None)))).scalar():
        raise TraceException(409, "This project already has tasks, so
     evaluating it again would replace task "
                                  "groups that hold work. Edit the project
     document instead — it re-plans the Gantt")

    previous = project.status
    project.status = "Evaluating"
    ai_jobs.set_ai_error(project, "evaluate_project", None)
    await db.commit()
    try:
        return {"job_id": jobs.enqueue("evaluate_project", project.id)}
    except Exception as exc:
        project.status = previous
        ai_jobs.set_ai_error(project, "evaluate_project", f"Could not queue
     the evaluation ({exc.__class__.__name__})")
        await db.commit()
        raise TraceException(503, "The job queue is unavailable; try again
     shortly") from exc

@router.get("/{project_id}/evaluate/status",
     response_model=EvaluationStatus,
            summary="Evaluation state: idle, running or failed (with the
     reason)")
```

```
async def evaluation_status(access: ProjectAccess =
     Depends(project_access)):
    project = access.project
    error = (project.ai_errors or {}).get("evaluate_project")
    state = "running" if project.status == "Evaluating" else "failed" if
     error else "idle"
    return {"state": state, "error": error["message"] if error else None,
            "project_status": project.status, "last_evaluated_at":
     project.last_evaluated_at}

@router.post("/{project_id}/suggest-managers", response_model=JobQueued,
     status_code=status.HTTP_202_ACCEPTED,
             summary="AI-ranked Project Manager candidates (owner) — poll
     /services/task/{job_id}/result")
async def suggest_managers(access: ProjectAccess = Depends(project_access),
     db: AsyncSession = Depends(get_db)):
    access.require_owner()
    await _require_ai(db)
    return {"job_id": _enqueue_or_503("suggest_managers",
     access.project.id)}

@router.post("/{project_id}/tasks/suggest-assignees",
     response_model=JobQueued,
             status_code=status.HTTP_202_ACCEPTED,
             summary="AI-ranked assignees for a draft task (Project
     Manager)")
async def suggest_assignees(
    data: AssigneeSuggestionRequest,
    access: ProjectAccess = Depends(project_access),
    db: AsyncSession = Depends(get_db),
):
    access.require_project_manager()
    await _require_ai(db)
    return {"job_id": _enqueue_or_503("suggest_assignees",
     access.project.id, data.title, data.description)}

@router.post("/{project_id}/task-groups/{group_id}/suggest-tasks",
     response_model=JobQueued,
             status_code=status.HTTP_202_ACCEPTED, summary="AI-suggested
     tasks for a task group (Project Manager)")
async def suggest_tasks(
    group_id: int,
    access: ProjectAccess = Depends(project_access),
    db: AsyncSession = Depends(get_db),
):
    access.require_project_manager()
    await svc.group_in_project(db, access.project.id, group_id)
    await _require_ai(db)
    return {"job_id": _enqueue_or_503("suggest_tasks", group_id)}

@router.post("/{project_id}/gantt/reevaluate", response_model=JobQueued,
     status_code=status.HTTP_202_ACCEPTED,
             summary="Re-plan the Gantt now (Project Manager)")
async def reevaluate_gantt(access: ProjectAccess = Depends(project_access)):
    access.require_project_manager()
    try:
        return {"job_id":
     gantt_service.request_reevaluation(access.project.id, "manual",
     manual=True)}
    except Exception as exc:
        raise TraceException(503, "The job queue is unavailable; try again
     shortly") from exc

@router.post("/{project_id}/progress-report", response_model=JobQueued,
     status_code=status.HTTP_202_ACCEPTED,
             summary="Rewrite the progress document and report now (owner or
     Project Manager)")
async def refresh_progress_report(
```

```
    access: ProjectAccess = Depends(project_access),
    github: GitHubClient | None = Depends(get_github_client),
    db: AsyncSession = Depends(get_db),
):
    if not (access.is_owner or access.is_project_manager):
        from app.exceptions import InsufficientPermissionsException
        raise InsufficientPermissionsException()
    await _require_ai(db)
    require_github(github)
    return {"job_id": _enqueue_or_503("update_project_progress_doc",
     access.project.id)}

@router.get("/{project_id}/documents/{kind}",
     response_model=DocumentResponse,
            summary="A project document's markdown, read from the
     repository")
async def get_document(
    kind: DocumentKind,
    access: ProjectAccess = Depends(project_access),
    github: GitHubClient | None = Depends(get_github_client),
):
    project = access.project
    url = getattr(project, ai_jobs.DOC_COLUMNS[kind])
    if not url:
        return {"kind": kind, "url": None, "content": None}
    content = await ai_jobs.read_document(require_github(github), project,
     ai_jobs.DOC_PATHS[kind])
    return {"kind": kind, "url": url, "content": content}

@router.put("/{project_id}/documents/doc", response_model=DocumentSaved,
            summary="Save the project document (Project Manager) — re-plans
     the Gantt when it changed")
async def save_project_doc(
    data: DocumentUpdate,
    access: ProjectAccess = Depends(project_access),
    github: GitHubClient | None = Depends(get_github_client),
    db: AsyncSession = Depends(get_db),
):
    access.require_project_manager()
    project, github = access.project, require_github(github)
    if not project.project_doc:
        raise TraceException(409, "There is no project document yet — the
     project has to be evaluated first")
    current = await ai_jobs.read_document(github, project,
     ai_jobs.DOC_PATHS["doc"])
    content = data.content if data.content.endswith("\n") else data.content
     + "\n"
    if current is not None and current.strip() == content.strip():
        return {"kind": "doc", "url": project.project_doc, "content":
     current, "changed": False}
    project.project_doc = await github.commit_file(
        project.github_repo, ai_jobs.DOC_PATHS["doc"], content,
        f"TRACE: docs/project_doc.md (edited by {access.user.name})",
     project.github_default_branch)
    await db.commit()
    gantt_service.request_reevaluation(project.id, "project_doc_updated")
    return {"kind": "doc", "url": project.project_doc, "content": content,
     "changed": True}
```

backend/app/api/v1/tasks.py

```
from fastapi import APIRouter, Depends, status
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from app.core.permissions import RoleEnum, require_any_role, require_roles
from app.core.project_access import resolve_project_access,
     resolve_task_access
```

```
from app.database import get_db
from app.models.task import Task
from app.models.user import User
from app.exceptions import InsufficientPermissionsException
from app.schemas.project import (
    DiscussionCreate, DiscussionResponse, DocumentResponse, TaskAssign,
     TaskMutationResponse, TaskResponse, TaskStatusUpdate, TaskUpdate,
)
from app.services import project_service as svc
from app.services.github_service import GitHubClient, get_github_client,
     require_github

router = APIRouter(prefix="/tasks", tags=["Tasks"],
     dependencies=[Depends(require_any_role)])

@router.get("/mine", response_model=list[TaskResponse], summary="Tasks
     assigned to me, across projects")
async def my_tasks(
    include_closed: bool = False,
    user: User = Depends(require_roles(RoleEnum.EMPLOYEE)),
    db: AsyncSession = Depends(get_db),
):
    stmt = select(Task).where(Task.assigned_to_user_id == user.id,
     Task.deleted_at.is_(None),
                              Task.project_id.is_not(None))
    if not include_closed:
        stmt = stmt.where(Task.status.notin_(["Completed", "Cancelled"]))
    tasks = (await
     db.execute(stmt.order_by(Task.due_date.asc().nulls_last(),
     Task.id))).scalars().all()
    out, access_by_project = [], {}
    for t in tasks:
        if t.project_id not in access_by_project:
            access_by_project[t.project_id] = await
     resolve_project_access(db, user, t.project_id)
        out.append(svc.task_dict(await svc.load_task(db, t.id),
     access_by_project[t.project_id]))
    return out

@router.get("/health-summary", summary="Task Health Score totals (Admin)")
async def task_health_summary(
    _: User = Depends(require_roles(RoleEnum.ADMIN)),
    db: AsyncSession = Depends(get_db),
):
    tasks = (await db.execute(
        select(Task).where(Task.deleted_at.is_(None),
     Task.status.notin_(["Completed", "Cancelled"]))
    )).scalars().all()
    counts = {"Green": 0, "Yellow": 0, "Red": 0}
    at_risk = []
    for t in tasks:
        score = t.health_score or "Green"
        counts[score] = counts.get(score, 0) + 1
        if score in ("Yellow", "Red"):
            at_risk.append({"id": t.id, "title": t.title, "project_id":
     t.project_id,
                            "health_score": score, "health_reason":
     t.health_reason,
                            "due_date": t.due_date, "status": t.status})
    return {"summary": counts, "total_tasks": len(tasks), "at_risk_tasks":
     at_risk}

@router.post("/compute-health", summary="Recompute Task Health Scores
     (Admin)")
async def compute_health(_: User = Depends(require_roles(RoleEnum.ADMIN))):
    from app.tasks import compute_task_health_and_escalate
    return {"job_id": compute_task_health_and_escalate.delay().id}
```

```
@router.get("/{task_id}", response_model=TaskResponse)
async def get_task(task_id: int, user: User = Depends(require_any_role), db:
     AsyncSession = Depends(get_db)):
    task, access = await resolve_task_access(db, user, task_id)
    return svc.task_dict(await svc.load_task(db, task.id), access)

@router.patch("/{task_id}", response_model=TaskResponse, summary="Edit task
     details (Project Manager)")
async def update_task(
    task_id: int,
    data: TaskUpdate,
    user: User = Depends(require_any_role),
    db: AsyncSession = Depends(get_db),
):
    task, access = await resolve_task_access(db, user, task_id)
    access.require_project_manager()
    return svc.task_dict(await svc.update_task(db, access, task, data),
     access)

@router.patch("/{task_id}/assign", response_model=TaskMutationResponse,
              summary="Assign or unassign a Member (Project Manager)")
async def assign_task(
    task_id: int,
    data: TaskAssign,
    user: User = Depends(require_any_role),
    github: GitHubClient | None = Depends(get_github_client),
    db: AsyncSession = Depends(get_db),
):
    task, access = await resolve_task_access(db, user, task_id)
    access.require_project_manager()
    task, warnings = await svc.assign_task(db, access, task,
     data.assigned_to_user_id, require_github(github))
    return {"task": svc.task_dict(task, access), "warnings": warnings}

@router.patch("/{task_id}/status", response_model=TaskResponse,
     summary="Change status — assignee or Project Manager")
async def update_task_status(
    task_id: int,
    data: TaskStatusUpdate,
    user: User = Depends(require_any_role),
    github: GitHubClient | None = Depends(get_github_client),
    db: AsyncSession = Depends(get_db),
):
    task, access = await resolve_task_access(db, user, task_id)
    return svc.task_dict(await svc.set_status(db, access, task, data.status,
     github), access)

@router.delete("/{task_id}", status_code=status.HTTP_204_NO_CONTENT,
               summary="Delete a task and close its issue as not planned
     (Project Manager)")
async def delete_task(
    task_id: int,
    user: User = Depends(require_any_role),
    github: GitHubClient | None = Depends(get_github_client),
    db: AsyncSession = Depends(get_db),
):
    task, access = await resolve_task_access(db, user, task_id)
    access.require_project_manager()
    await svc.delete_task(db, access, task, github)

@router.get("/{task_id}/progress-doc", response_model=DocumentResponse,
            summary="The task's AI-written plan and checklist, read from the
     repository")
async def get_task_progress_doc(
    task_id: int,
    user: User = Depends(require_any_role),
    github: GitHubClient | None = Depends(get_github_client),
    db: AsyncSession = Depends(get_db),
```

```
):
    from app.services import ai_jobs
    task, access = await resolve_task_access(db, user, task_id)
    if not task.task_progress_doc:
        return {"kind": "task", "url": None, "content": None}
    content = await ai_jobs.read_document(require_github(github),
     access.project, ai_jobs.task_doc_path(task.id))
    return {"kind": "task", "url": task.task_progress_doc, "content":
     content}

async def _discussion_access(db: AsyncSession, user: User, task_id: int):
    task, access = await resolve_task_access(db, user, task_id)
    if not access.may_discuss(task):
        raise InsufficientPermissionsException()
    return task, access

@router.get("/{task_id}/discussions",
     response_model=list[DiscussionResponse],
            summary="The task's discussion — assignee, assigner, Project
     Managers, owning Manager")
async def list_discussions(task_id: int, user: User =
     Depends(require_any_role), db: AsyncSession = Depends(get_db)):
    task, _ = await _discussion_access(db, user, task_id)
    return await svc.list_discussions(db, task)

@router.post("/{task_id}/discussions", response_model=DiscussionResponse,
     status_code=status.HTTP_201_CREATED,
             summary="Post a message or a reply; the other participants are
     notified")
async def post_discussion(task_id: int, data: DiscussionCreate, user: User =
     Depends(require_any_role),
                          db: AsyncSession = Depends(get_db)):
    task, access = await _discussion_access(db, user, task_id)
    return await svc.post_discussion(db, access, task, data.message,
     data.parent_id)
```

backend/app/api/v1/notifications.py

```
import jwt as pyjwt
from fastapi import APIRouter, Depends, Query, WebSocket,
     WebSocketDisconnect, status
from sqlalchemy import func, select, update
from sqlalchemy.ext.asyncio import AsyncSession

from app.core.permissions import require_any_role
from app.core.security import decode_token
from app.database import async_session_factory, get_db
from app.exceptions import NotFoundException
from app.models.project import Notification
from app.models.user import User
from app.schemas.project import NotificationList, NotificationResponse
from app.services.realtime import hub

router = APIRouter(prefix="/notifications", tags=["Notifications"],
     dependencies=[Depends(require_any_role)])
ws_router = APIRouter(tags=["Notifications"])

def _mine(user: User):
    return (Notification.user_id == user.id,
     Notification.deleted_at.is_(None))

async def _unread(db: AsyncSession, user: User) -> int:
    return (await db.execute(
        select(func.count(Notification.id)).where(*_mine(user),
     Notification.is_read.is_(False))
    )).scalar_one()
```

```
@router.get("", response_model=NotificationList, summary="My notifications,
     newest first, and the unread count")
async def list_notifications(
    unread_only: bool = False,
    limit: int = Query(30, ge=1, le=100),
    user: User = Depends(require_any_role),
    db: AsyncSession = Depends(get_db),
):
    stmt = select(Notification).where(*_mine(user))
    if unread_only:
        stmt = stmt.where(Notification.is_read.is_(False))
    items = (await
     db.execute(stmt.order_by(Notification.id.desc()).limit(limit))).scalars(
     ).all()
    return {"items": items, "unread": await _unread(db, user)}

@router.get("/unread-count", summary="How many of my notifications are
     unread")
async def unread_count(user: User = Depends(require_any_role), db:
     AsyncSession = Depends(get_db)):
    return {"unread": await _unread(db, user)}

@router.patch("/{notification_id}/read",
     response_model=NotificationResponse, summary="Mark one read")
async def mark_read(notification_id: int, user: User =
     Depends(require_any_role), db: AsyncSession = Depends(get_db)):
    note = (await db.execute(
        select(Notification).where(Notification.id == notification_id,
     *_mine(user))
    )).scalar_one_or_none()
    if note is None:
        raise NotFoundException("Notification")
    note.is_read = True
    await db.commit()
    await db.refresh(note)
    return note

@router.post("/read-all", summary="Mark all of mine read")
async def mark_all_read(user: User = Depends(require_any_role), db:
     AsyncSession = Depends(get_db)):
    result = await db.execute(
        update(Notification).where(*_mine(user),
     Notification.is_read.is_(False)).values(is_read=True)
    )
    await db.commit()
    return {"updated": result.rowcount}

@ws_router.websocket("/ws")
async def live_events(websocket: WebSocket, token: str = ""):
    try:
        payload = decode_token(token)
        if payload.get("type") != "access":
            raise pyjwt.InvalidTokenError("not an access token")
        user_id = int(payload["sub"])
    except (pyjwt.PyJWTError, KeyError, TypeError, ValueError):
        await websocket.close(code=status.WS_1008_POLICY_VIOLATION)
        return
    async with async_session_factory() as db:
        user = await db.get(User, user_id)
        if user is None or user.deleted_at is not None or user.status ==
     "Blocked":
            await websocket.close(code=status.WS_1008_POLICY_VIOLATION)
            return

    await hub.connect(user_id, websocket)
    try:
        while True:
            await websocket.receive_text()
```

```
    except WebSocketDisconnect:
        hub.disconnect(user_id, websocket)
```

backend/app/api/v1/settings.py

```
import asyncio
from typing import Literal

from fastapi import APIRouter, Depends
from pydantic import BaseModel, Field, field_validator
from sqlalchemy.ext.asyncio import AsyncSession

from app.core.permissions import RoleEnum, require_roles
from app.database import get_db
from app.services import mail_service
from app.services.github_service import (
    OWNER_PATTERN, GitHubClient, GitHubError, get_github_client,
)
from app.services.system_settings import resolve_setting, setting_source,
     store_setting

router = APIRouter(prefix="/admin/settings", tags=["Configuration"],
                   dependencies=[Depends(require_roles(RoleEnum.ADMIN))])

GITHUB_FIELDS = ("owner", "committer_name", "committer_email")

class GitHubSettings(BaseModel):
    owner: str
    committer_name: str
    committer_email: str
    token_set: bool
    token_source: str | None

class GitHubSettingsUpdate(BaseModel):
    owner: str | None = Field(None, max_length=39)
    committer_name: str | None = Field(None, max_length=100)
    committer_email: str | None = Field(None, max_length=255)
    token: str | None = Field(None, max_length=500)

    @field_validator("owner")
    @classmethod
    def _owner(cls, value: str | None) -> str | None:
        if value is None:
            return None
        value = value.strip().removeprefix("https://github.com/").strip("/")
        if value and not OWNER_PATTERN.match(value):
            raise ValueError("must be a GitHub organization login, e.g.
     acme-corp")
        return value

    @field_validator("committer_email")
    @classmethod
    def _email(cls, value: str | None) -> str | None:
        if value in (None, ""):
            return value
        if "@" not in value or value.startswith("@") or value.endswith("@"):
            raise ValueError("must be an email address")
        return value.strip()

class ConnectionCheck(BaseModel):
    name: str
    ok: bool
    detail: str

class GitHubTestResult(BaseModel):
    ok: bool
    checks: list[ConnectionCheck]
```

```
async def _github_settings(db: AsyncSession) -> dict:
    values = {f: await resolve_setting(db, "github", f) for f in
     GITHUB_FIELDS}
    source = await setting_source(db, "github", "token")
    return {**values, "token_set": source is not None, "token_source":
     source}

@router.get("/github", response_model=GitHubSettings, summary="GitHub
     settings (token never returned)")
async def get_github_settings(db: AsyncSession = Depends(get_db)):
    return await _github_settings(db)

@router.put("/github", response_model=GitHubSettings, summary="Save GitHub
     settings (token is write-only)")
async def put_github_settings(data: GitHubSettingsUpdate, db: AsyncSession =
     Depends(get_db)):
    for field, value in data.model_dump(exclude_unset=True).items():
        if field == "token" and value is None:
            continue
        await store_setting(db, "github", field, value.strip() if
     isinstance(value, str) else value)
    await db.commit()
    return await _github_settings(db)

@router.post("/github/test", response_model=GitHubTestResult, summary="Test
     the saved GitHub settings")
async def test_github_settings(
    github: GitHubClient | None = Depends(get_github_client),
    db: AsyncSession = Depends(get_db),
):
    owner = await resolve_setting(db, "github", "owner")
    checks = []
    if github is None:
        checks.append(ConnectionCheck(name="Token", ok=False, detail="No
     token is set"))
    else:
        try:
            login = await github.authenticated_login()
            checks.append(ConnectionCheck(name="Token", ok=True,
     detail=f"Authenticates as {login}"))
        except GitHubError as exc:
            checks.append(ConnectionCheck(name="Token", ok=False,
     detail=exc.detail))
    if not owner:
        checks.append(ConnectionCheck(name="Organization", ok=False,
     detail="No organization is set"))
    elif github is not None and checks[0].ok:
        try:
            org = await github.organization(owner)
            checks.append(ConnectionCheck(
                name="Organization", ok=True,
                detail=f"{org.get('name') or org.get('login', owner)} is
     visible to the token"))
        except GitHubError as exc:
            detail = (f"'{owner}' is not an organization this token can see
     — personal accounts are "
                      "not supported" if exc.github_status == 404 else
     exc.detail)
            checks.append(ConnectionCheck(name="Organization", ok=False,
     detail=detail))
    else:
        checks.append(ConnectionCheck(name="Organization", ok=False,
     detail="Not checked — the token failed"))
    return {"ok": all(c.ok for c in checks), "checks": checks}

MAIL_FIELDS = ("host", "port", "from_address", "username", "security")

class MailSettings(BaseModel):
```

```
    host: str
    port: str
    from_address: str
    username: str
    security: str
    password_set: bool
    password_source: str | None

class MailSettingsUpdate(BaseModel):
    host: str | None = Field(None, max_length=255)
    port: int | None = Field(None, ge=1, le=65535)
    from_address: str | None = Field(None, max_length=255)
    username: str | None = Field(None, max_length=255)
    password: str | None = Field(None, max_length=500)
    security: Literal["none", "starttls", "ssl"] | None = None

    @field_validator("from_address")
    @classmethod
    def _from(cls, value: str | None) -> str | None:
        if value in (None, ""):
            return value
        if "@" not in value:
            raise ValueError("must be an email address")
        return value.strip()

async def _mail_settings(db: AsyncSession) -> dict:
    values = {f: await resolve_setting(db, "mail", f) for f in MAIL_FIELDS}
    values["security"] = values["security"] or "none"
    source = await setting_source(db, "mail", "password")
    return {**values, "password_set": source is not None, "password_source":
     source}

@router.get("/mail", response_model=MailSettings, summary="Mail settings
     (password never returned)")
async def get_mail_settings(db: AsyncSession = Depends(get_db)):
    return await _mail_settings(db)

@router.put("/mail", response_model=MailSettings, summary="Save mail
     settings (password is write-only)")
async def put_mail_settings(data: MailSettingsUpdate, db: AsyncSession =
     Depends(get_db)):
    for field, value in data.model_dump(exclude_unset=True).items():
        if field == "password" and value is None:
            continue
        await store_setting(db, "mail", field, str(value).strip() if value
     is not None else None)
    await db.commit()
    return await _mail_settings(db)

@router.post("/mail/test", response_model=GitHubTestResult, summary="Test
     the saved mail server")
async def test_mail_settings(db: AsyncSession = Depends(get_db)):
    config = await mail_service.mail_config(db)
    checks = await asyncio.to_thread(mail_service.check, config)
    return {"ok": all(c["ok"] for c in checks), "checks": checks}
```

backend/app/api/v1/workforce.py

```
from fastapi import APIRouter, Depends, File, Response, UploadFile, status
from sqlalchemy.ext.asyncio import AsyncSession

from app.core.permissions import require_admin
from app.database import get_db
from app.services import workforce as wf

router = APIRouter(prefix="/admin/workforce", tags=["Workforce"],
     dependencies=[Depends(require_admin)])
```

```
@router.get("", summary="The tabs and their fields")
async def catalogue():
    return wf.catalogue()

@router.get("/{entity}", summary="Every record of one tab")
async def list_records(entity: str, db: AsyncSession = Depends(get_db)):
    return await wf.list_rows(db, entity)

@router.post("/{entity}", status_code=status.HTTP_201_CREATED, summary="Add
    a record")
async def create_record(entity: str, data: dict, db: AsyncSession =
    Depends(get_db)):
    row_id = await wf.create_row(db, entity, data)
    await db.commit()
    return {"id": row_id}

@router.put("/{entity}/{row_id}", summary="Edit a record (fields left out
    are unchanged)")
async def update_record(entity: str, row_id: int, data: dict, db:
    AsyncSession = Depends(get_db)):
    await wf.update_row(db, entity, row_id, data)
    await db.commit()
    return {"id": row_id}

@router.delete("/{entity}/{row_id}", status_code=status.HTTP_204_NO_CONTENT,
    summary="Delete a record")
async def delete_record(entity: str, row_id: int, db: AsyncSession =
    Depends(get_db)):
    await wf.delete_row(db, entity, row_id)
    await db.commit()

@router.get("/{entity}/template", summary="An empty CSV with this tab's
    columns",
           response_class=Response, responses={200: {"content":
    {"text/csv": {}}}})
async def csv_template(entity: str):
    return Response(wf.template(entity), media_type="text/csv; charset=utf-
    8",
                    headers={"Content-Disposition": f'attachment;
    filename="{entity}-template.csv"'})

@router.post("/{entity}/bulk", summary="Bulk upload a CSV — good rows are
    added, bad rows are reported")
async def bulk_upload(entity: str, file: UploadFile = File(...), db:
    AsyncSession = Depends(get_db)):
    return await wf.bulk_upload(db, entity, await file.read())
```

backend/app/api/v1/admin.py

```
import datetime
from typing import Any, Dict, List, Optional
from fastapi import APIRouter, Depends, HTTPException, Query, status
from pydantic import BaseModel
from sqlalchemy import func, inspect, select, text
from sqlalchemy.ext.asyncio import AsyncSession

from app.core.security import hash_password
from app.database import get_db
from app.dependencies import get_current_user
from app.models.attendance import Attendance, TimeZone
from app.models.employee import (
    Department,
    Designation,
    Employee,
    EmployeeDetail,
    WorkSchedule,
)
from app.models.hrms import Holiday, Leave
```

```
from app.models.productivity import ProductivityScore
from app.models.task import (
    Task,
    TaskComment,
    TaskDetail,
    TaskTimeLog,
    TaskType,
)
from app.models.user import Profile, Role, User, UserRole
from app.models.project import (
    Project,
    ProjectSourceFile,
    ProjectAssignment,
    TaskGroup,
    GanttChart,
    TaskDiscussion,
    Notification,
)

router = APIRouter()

MODEL_REGISTRY: Dict[str, Dict[str, Any]] = {
    "users": {"model": User, "display_name": "Users", "category": "Core Auth
     & Users"},
    "profiles": {"model": Profile, "display_name": "Profiles", "category":
     "Core Auth & Users"},
    "roles": {"model": Role, "display_name": "Roles", "category": "Core Auth
     & Users"},
    "user_roles": {"model": UserRole, "display_name": "User Roles",
     "category": "Core Auth & Users"},
    "employees": {"model": Employee, "display_name": "Employees",
     "category": "HR & Workforce"},
    "employee_details": {"model": EmployeeDetail, "display_name": "Employee
     Details", "category": "HR & Workforce"},
    "departments": {"model": Department, "display_name": "Departments",
     "category": "HR & Workforce"},
    "designations": {"model": Designation, "display_name": "Designations",
     "category": "HR & Workforce"},
    "work_schedules": {"model": WorkSchedule, "display_name": "Work
     Schedules", "category": "HR & Workforce"},
    "attendance": {"model": Attendance, "display_name": "Attendance
     Records", "category": "Attendance & Schedules"},
    "time_zones": {"model": TimeZone, "display_name": "Time Zones",
     "category": "Attendance & Schedules"},
    "leaves": {"model": Leave, "display_name": "Leave Requests", "category":
     "Attendance & Schedules"},
    "holidays": {"model": Holiday, "display_name": "Holidays", "category":
     "Attendance & Schedules"},
    "tasks": {"model": Task, "display_name": "Tasks", "category": "Tasks &
     Projects"},
    "task_types": {"model": TaskType, "display_name": "Task Types",
     "category": "Tasks & Projects"},
    "task_details": {"model": TaskDetail, "display_name": "Task Details",
     "category": "Tasks & Projects"},
    "task_comments": {"model": TaskComment, "display_name": "Task Comments",
     "category": "Tasks & Projects"},
    "task_time_logs": {"model": TaskTimeLog, "display_name": "Task Time
     Logs", "category": "Tasks & Projects"},
    "projects": {"model": Project, "display_name": "Projects", "category":
     "Tasks & Projects"},
    "project_source_files": {"model": ProjectSourceFile, "display_name":
     "Project Source Files", "category": "Tasks & Projects"},
    "project_assignments": {"model": ProjectAssignment, "display_name":
     "Project Assignments", "category": "Tasks & Projects"},
    "task_groups": {"model": TaskGroup, "display_name": "Task Groups",
     "category": "Tasks & Projects"},
    "gantt_charts": {"model": GanttChart, "display_name": "Gantt Charts",
     "category": "Tasks & Projects"},
```

```
    "task_discussions": {"model": TaskDiscussion, "display_name": "Task
     Discussions", "category": "Tasks & Projects"},
    "notifications": {"model": Notification, "display_name":
     "Notifications", "category": "Tasks & Projects"},
    "productivity_scores": {"model": ProductivityScore, "display_name":
     "Productivity Scores", "category": "AI & System"},
}

def require_admin(user: User = Depends(get_current_user)) -> User:
    user_roles = [ur.role.role_name for ur in user.user_roles if ur.role]
    if "Admin" not in user_roles:
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="Access denied. Admin role required.",
        )
    return user

def serialize_value(val: Any) -> Any:
    if isinstance(val, (datetime.datetime, datetime.date, datetime.time)):
        return val.isoformat()
    return val

def parse_field_value(col_name: str, col_type: str, raw_value: Any) -> Any:
    if raw_value is None or raw_value == "":
        return None

    type_str = str(col_type).upper()

    if col_name == "password" and isinstance(raw_value, str) and not
     raw_value.startswith("$2b$"):
        return hash_password(raw_value)

    if "INT" in type_str:
        return int(raw_value)
    elif "FLOAT" in type_str or "NUMERIC" in type_str or "DECIMAL" in
     type_str:
        return float(raw_value)
    elif "BOOL" in type_str:
        if isinstance(raw_value, bool):
            return raw_value
        return str(raw_value).lower() in ("true", "1", "t", "yes")
    elif "DATETIME" in type_str or "TIMESTAMP" in type_str:
        if isinstance(raw_value, str):
            return datetime.datetime.fromisoformat(raw_value.replace("Z",
     "+00:00"))
    elif "DATE" in type_str:
        if isinstance(raw_value, str):
            return datetime.date.fromisoformat(raw_value[:10])
    elif "TIME" in type_str:
        if isinstance(raw_value, str):
            return datetime.time.fromisoformat(raw_value)

    return raw_value

@router.get("/tables", summary="Get metadata for all database tables")
async def list_tables(
    db: AsyncSession = Depends(get_db),
    admin: User = Depends(require_admin),
):
    table_list = []

    for table_key, meta in MODEL_REGISTRY.items():
        model_cls = meta["model"]
        mapper = inspect(model_cls)

        cnt_stmt = select(func.count()).select_from(model_cls)
        try:
            res = await db.execute(cnt_stmt)
```

```
            total_records = res.scalar_one()
        except Exception:
            total_records = 0

        columns = []
        pk_name = "id"
        for col in mapper.columns:
            if col.primary_key:
                pk_name = col.name
            columns.append({
                "name": col.name,
                "type": str(col.type),
                "nullable": col.nullable,
                "primary_key": col.primary_key,
                "default": str(col.default) if col.default else None,
            })

        table_list.append({
            "name": table_key,
            "display_name": meta["display_name"],
            "category": meta["category"],
            "total_records": total_records,
            "primary_key": pk_name,
            "columns": columns,
        })

    return {"status": "success", "tables": table_list}

@router.get("/tables/{table_name}", summary="Get records for a specific
     table")
async def get_table_records(
    table_name: str,
    page: int = Query(1, ge=1),
    page_size: int = Query(20, ge=1, le=100),
    search: Optional[str] = Query(None),
    sort_by: Optional[str] = Query(None),
    sort_order: str = Query("asc", pattern="^(asc|desc)$"),
    db: AsyncSession = Depends(get_db),
    admin: User = Depends(require_admin),
):
    if table_name not in MODEL_REGISTRY:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Table '{table_name}' not found in Admin Registry.",
        )

    meta = MODEL_REGISTRY[table_name]
    model_cls = meta["model"]
    mapper = inspect(model_cls)

    columns = []
    pk_col = None
    for col in mapper.columns:
        if col.primary_key:
            pk_col = col
        columns.append({
            "name": col.name,
            "type": str(col.type),
            "nullable": col.nullable,
            "primary_key": col.primary_key,
        })

    pk_name = pk_col.name if pk_col is not None else "id"

    stmt = select(model_cls)

    if search and search.strip():
        search_term = f"%{search.strip()}%"
```

```
        search_filters = []
        for col in mapper.columns:
            if "VARCHAR" in str(col.type).upper() or "TEXT" in
     str(col.type).upper():
                search_filters.append(col.ilike(search_term))
        if search_filters:
            stmt = stmt.where(func.or_(*search_filters))

    count_stmt = select(func.count()).select_from(stmt.subquery())
    count_res = await db.execute(count_stmt)
    total_records = count_res.scalar_one()

    if sort_by and hasattr(model_cls, sort_by):
        sort_attr = getattr(model_cls, sort_by)
        stmt = stmt.order_by(sort_attr.desc() if sort_order == "desc" else
     sort_attr.asc())
    elif pk_col is not None:
        stmt = stmt.order_by(pk_col.desc() if sort_order == "desc" else
     pk_col.asc())

    offset = (page - 1) * page_size
    stmt = stmt.offset(offset).limit(page_size)

    result = await db.execute(stmt)
    records_raw = result.scalars().all()

    records = []
    for item in records_raw:
        row_dict = {}
        for col in mapper.columns:
            val = getattr(item, col.name, None)
            row_dict[col.name] = serialize_value(val)
        records.append(row_dict)

    total_pages = (total_records + page_size - 1) // page_size if
     total_records > 0 else 1

    return {
        "status": "success",
        "table_name": table_name,
        "display_name": meta["display_name"],
        "category": meta["category"],
        "primary_key": pk_name,
        "total": total_records,
        "page": page,
        "page_size": page_size,
        "total_pages": total_pages,
        "columns": columns,
        "records": records,
    }

@router.post("/tables/{table_name}", summary="Create a new record in a
     table")
async def create_table_record(
    table_name: str,
    payload: Dict[str, Any],
    db: AsyncSession = Depends(get_db),
    admin: User = Depends(require_admin),
):
    if table_name not in MODEL_REGISTRY:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Table '{table_name}' not found in Admin Registry.",
        )

    model_cls = MODEL_REGISTRY[table_name]["model"]
    mapper = inspect(model_cls)
```

```
    kw_args = {}
    for col in mapper.columns:
        if col.name in payload and payload[col.name] is not None:
            kw_args[col.name] = parse_field_value(col.name, str(col.type),
     payload[col.name])

    try:
        new_instance = model_cls(**kw_args)
        db.add(new_instance)
        await db.commit()
        await db.refresh(new_instance)

        result_dict = {}
        for col in mapper.columns:
            result_dict[col.name] = serialize_value(getattr(new_instance,
     col.name, None))

        return {"status": "success", "message": "Record created
     successfully", "record": result_dict}
    except Exception as e:
        await db.rollback()
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail=f"Failed to create record: {str(e)}",
        )

@router.put("/tables/{table_name}/{record_id}", summary="Update a record in
     a table")
async def update_table_record(
    table_name: str,
    record_id: str,
    payload: Dict[str, Any],
    db: AsyncSession = Depends(get_db),
    admin: User = Depends(require_admin),
):
    if table_name not in MODEL_REGISTRY:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Table '{table_name}' not found in Admin Registry.",
        )

    model_cls = MODEL_REGISTRY[table_name]["model"]
    mapper = inspect(model_cls)

    pk_col = None
    for col in mapper.columns:
        if col.primary_key:
            pk_col = col
            break

    if pk_col is None:
        raise HTTPException(status_code=400, detail="Table lacks primary key
     definition")

    pk_val = parse_field_value(pk_col.name, str(pk_col.type), record_id)

    stmt = select(model_cls).where(pk_col == pk_val)
    res = await db.execute(stmt)
    instance = res.scalar_one_or_none()

    if not instance:
        raise HTTPException(status_code=404, detail="Record not found")

    try:
        for col in mapper.columns:
            if col.name in payload and not col.primary_key:
                val = parse_field_value(col.name, str(col.type),
     payload[col.name])
```

```
                setattr(instance, col.name, val)

        await db.commit()
        await db.refresh(instance)

        result_dict = {}
        for col in mapper.columns:
            result_dict[col.name] = serialize_value(getattr(instance,
    col.name, None))

        return {"status": "success", "message": "Record updated
    successfully", "record": result_dict}
    except Exception as e:
        await db.rollback()
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail=f"Failed to update record: {str(e)}",
        )

@router.delete("/tables/{table_name}/{record_id}", summary="Delete a record
    from a table")
async def delete_table_record(
    table_name: str,
    record_id: str,
    db: AsyncSession = Depends(get_db),
    admin: User = Depends(require_admin),
):
    if table_name not in MODEL_REGISTRY:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Table '{table_name}' not found in Admin Registry.",
        )

    model_cls = MODEL_REGISTRY[table_name]["model"]
    mapper = inspect(model_cls)

    pk_col = None
    for col in mapper.columns:
        if col.primary_key:
            pk_col = col
            break

    if pk_col is None:
        raise HTTPException(status_code=400, detail="Table lacks primary key
    definition")

    pk_val = parse_field_value(pk_col.name, str(pk_col.type), record_id)

    stmt = select(model_cls).where(pk_col == pk_val)
    res = await db.execute(stmt)
    instance = res.scalar_one_or_none()

    if not instance:
        raise HTTPException(status_code=404, detail="Record not found")

    try:
        if hasattr(instance, "deleted_at") and getattr(instance,
    "deleted_at") is None:
            setattr(instance, "deleted_at",
    datetime.datetime.now(datetime.timezone.utc))
            message = "Record soft-deleted successfully"
        else:
            await db.delete(instance)
            message = "Record permanently deleted successfully"

        await db.commit()
        return {"status": "success", "message": message}
    except Exception as e:
```

```
        await db.rollback()
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail=f"Failed to delete record: {str(e)}",
        )
```

backend/app/api/v1/services.py

```
import asyncio
import logging
import smtplib
import time
from datetime import datetime, timezone
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart
from typing import Optional

from fastapi import APIRouter, Depends, UploadFile, File, Form,
     HTTPException
from fastapi.responses import JSONResponse
from pydantic import BaseModel

from app.config import get_settings
from app.core.permissions import require_admin, require_any_role

settings = get_settings()
logger = logging.getLogger("trace.services")

router = APIRouter(prefix="/services", tags=["Services"])

class MailRequest(BaseModel):
    to: str
    subject: str
    body: str

class LLMChatRequest(BaseModel):
    message: str
    model: Optional[str] = None

class FileUploadResponse(BaseModel):
    status: str
    task_id: Optional[str] = None
    filename: Optional[str] = None
    message: str

@router.get("/health/all", dependencies=[Depends(require_admin)])
async def all_service_health():
    results = {}

    results["api"] = {"status": "healthy", "uptime": True}

    results["database"] = await _check_db_health()

    results["redis"] = await _check_redis_health()

    results["celery_worker"] = _check_celery_worker_health()

    results["celery_beat"] = _check_celery_beat_health()

    results["mailpit"] = _check_mailpit_health()

    results["minio"] = _check_minio_health()

    results["llm"] = _check_llm_config()

    results["websocket"] = {"status": "healthy", "endpoint": "/api/v1/ws"}

    return {
```

```
        "timestamp": datetime.now(timezone.utc).isoformat(),
        "services": results,
    }

@router.get("/health/api", dependencies=[Depends(require_admin)])
async def api_health():
    return {"status": "healthy", "service": "api", "app": settings.APP_NAME,
     "version": settings.APP_VERSION}

@router.get("/health/database", dependencies=[Depends(require_admin)])
async def database_health():
    return await _check_db_health()

@router.get("/health/redis", dependencies=[Depends(require_admin)])
async def redis_health():
    return await _check_redis_health()

@router.get("/health/celery-worker", dependencies=[Depends(require_admin)])
async def celery_worker_health():
    return _check_celery_worker_health()

@router.get("/health/celery-beat", dependencies=[Depends(require_admin)])
async def celery_beat_health():
    return _check_celery_beat_health()

@router.get("/health/mailpit", dependencies=[Depends(require_admin)])
async def mailpit_health():
    return _check_mailpit_health()

@router.get("/health/minio", dependencies=[Depends(require_admin)])
async def minio_health():
    return _check_minio_health()

@router.get("/health/llm", dependencies=[Depends(require_admin)])
async def llm_health():
    return _check_llm_config()

async def _check_db_health():
    try:
        from app.database import async_session_factory
        async with async_session_factory() as db:
            from sqlalchemy import text
            result = await db.execute(text("SELECT 1"))
            row = result.scalar()
            if row == 1:
                ver = await db.execute(text("SELECT version()"))
                version = ver.scalar()
                return {
                    "status": "healthy",
                    "service": "database",
                    "version": version.split(",")[0] if version else
     "unknown",
                    "connection": settings.DATABASE_URL.split("@")[-
     1].split("/")[0] if "@" in settings.DATABASE_URL else "unknown",
                }
    except Exception as e:
        logger.error(f"DB health check failed: {e}")
        return {"status": "unhealthy", "service": "database", "error":
     str(e)}

async def _check_redis_health():
    try:
        import redis.asyncio as aioredis
        r = aioredis.from_url(settings.REDIS_URL, decode_responses=True)
        pong = await r.ping()
        info = await r.info("server")
        await r.aclose()
        return {
```

```
            "status": "healthy" if pong else "unhealthy",
            "service": "redis",
            "version": info.get("redis_version", "unknown"),
            "url": settings.REDIS_URL.split("@")[-1] if "@" in
     settings.REDIS_URL else settings.REDIS_URL,
        }
    except Exception as e:
        logger.error(f"Redis health check failed: {e}")
        return {"status": "unhealthy", "service": "redis", "error": str(e)}

def _check_celery_worker_health():
    try:
        from app.celery_app import celery_app
        inspector = celery_app.control.inspect(timeout=3.0)
        active = inspector.active()
        stats = inspector.stats()
        if active is not None:
            worker_names = list(active.keys())
            active_tasks = sum(len(v) for v in active.values())
            return {
                "status": "healthy",
                "service": "celery_worker",
                "workers": worker_names,
                "active_tasks": active_tasks,
                "pool": stats[worker_names[0]].get("pool",
     {}).get("implementation", "unknown") if stats and worker_names else
     "unknown",
            }
        return {"status": "unhealthy", "service": "celery_worker", "error":
     "No workers responding"}
    except Exception as e:
        logger.error(f"Celery worker health check failed: {e}")
        return {"status": "unhealthy", "service": "celery_worker", "error":
     str(e)}

def _check_celery_beat_health():
    try:
        from app.celery_app import celery_app
        schedule = celery_app.conf.beat_schedule
        task_names = list(schedule.keys()) if schedule else []
        inspector = celery_app.control.inspect(timeout=3.0)
        active = inspector.active()
        import redis
        r = redis.from_url(settings.REDIS_URL)
        r.ping()
        r.close()
        return {
            "status": "healthy",
            "service": "celery_beat",
            "scheduled_tasks": task_names,
            "task_count": len(task_names),
        }
    except Exception as e:
        logger.error(f"Celery beat health check failed: {e}")
        return {"status": "unhealthy", "service": "celery_beat", "error":
     str(e)}

def _check_mailpit_health():
    try:
        sock = smtplib.SMTP(settings.SMTP_HOST, settings.SMTP_PORT,
     timeout=5)
        sock.ehlo()
        sock.quit()
        return {
            "status": "healthy",
            "service": "mailpit",
            "smtp": f"{settings.SMTP_HOST}:{settings.SMTP_PORT}",
            "ui_url": f"http://localhost:8025",
```

```
        }
    except Exception as e:
        logger.error(f"Mailpit health check failed: {e}")
        return {"status": "unhealthy", "service": "mailpit", "error":
     str(e)}

def _check_minio_health():
    try:
        from minio import Minio
        client = Minio(
            settings.MINIO_ENDPOINT,
            access_key=settings.MINIO_ACCESS_KEY,
            secret_key=settings.MINIO_SECRET_KEY,
            secure=settings.MINIO_SECURE,
        )
        if not client.bucket_exists(settings.MINIO_BUCKET):
            client.make_bucket(settings.MINIO_BUCKET)
        buckets = [b.name for b in client.list_buckets()]
        return {
            "status": "healthy",
            "service": "minio",
            "endpoint": settings.MINIO_ENDPOINT,
            "bucket": settings.MINIO_BUCKET,
            "buckets": buckets,
            "console_url": f"http://localhost:9001",
        }
    except Exception as e:
        logger.error(f"MinIO health check failed: {e}")
        return {"status": "unhealthy", "service": "minio", "error": str(e)}

def _check_llm_config():
    configured = bool(settings.OPENAI_API_KEY)
    return {
        "status": "configured" if configured else "not_configured",
        "service": "llm",
        "model": settings.OPENAI_MODEL,
        "base_url": settings.OPENAI_BASE_URL or "https://api.openai.com/v1",
        "note": "LLM health cannot be actively checked — use the test
     chatbox instead.",
    }

def _job_outcome(meta: dict) -> dict:
    result, state = meta.get("result"), meta.get("status")
    outcome, detail = state, None
    if state == "SUCCESS" and isinstance(result, dict) and
     result.get("status") in ("failed", "error", "skipped"):
        outcome = result["status"].upper()
        detail = result.get("error") or result.get("reason")
    elif state == "FAILURE":
        detail = (result or {}).get("exc_message") if isinstance(result,
     dict) else str(result)
        if isinstance(detail, list):
            detail = "; ".join(map(str, detail))
    name = (meta.get("name") or "").removeprefix("app.tasks.") or "unknown"
    return {"id": meta.get("task_id"), "name": name, "state": state,
     "outcome": outcome,
            "detail": detail, "finished_at": meta.get("date_done")}

@router.get("/jobs/recent", dependencies=[Depends(require_admin)])
async def recent_jobs(limit: int = 30):
    import json as _json
    import redis.asyncio as aioredis

    client = aioredis.Redis.from_url(settings.REDIS_URL)
    try:
        metas = []
        async for key in client.scan_iter(match="celery-task-meta-*",
     count=500):
```

```
            raw = await client.get(key)
            if raw:
                try:
                    metas.append(_json.loads(raw))
                except ValueError:
                    continue
    finally:
        await client.aclose()
    metas.sort(key=lambda m: m.get("date_done") or "", reverse=True)
    return {"jobs": [_job_outcome(m) for m in metas[:max(1, min(limit,
     100))]]}

@router.get("/ws/status", dependencies=[Depends(require_admin)])
async def ws_status():
    from app.services.realtime import hub
    return {"online_users": hub.online(), "status": "active"}

@router.post("/mail/send", dependencies=[Depends(require_admin)])
async def send_test_mail(request: MailRequest):
    from app.tasks import send_mail_task
    task = send_mail_task.delay(request.to, request.subject, request.body)
    return {
        "status": "queued",
        "task_id": task.id,
        "message": f"Mail to {request.to} queued via Celery",
    }

@router.post("/minio/upload", dependencies=[Depends(require_admin)])
async def upload_test_file(file: UploadFile = File(...)):
    content = await file.read()
    import base64
    encoded = base64.b64encode(content).decode("utf-8")

    from app.tasks import upload_file_task
    task = upload_file_task.delay(file.filename, encoded, file.content_type)
    return {
        "status": "queued",
        "task_id": task.id,
        "filename": file.filename,
        "message": f"File '{file.filename}' upload queued via Celery",
    }

@router.post("/llm/chat", dependencies=[Depends(require_admin)])
async def llm_chat(request: LLMChatRequest):
    from app.tasks import llm_chat_task
    task = llm_chat_task.delay(request.message, request.model)
    return {
        "status": "queued",
        "task_id": task.id,
        "message": "LLM request queued via Celery",
    }

@router.get("/task/{task_id}/result",
     dependencies=[Depends(require_any_role)])
async def get_task_result(task_id: str):
    from app.celery_app import celery_app
    result = celery_app.AsyncResult(task_id)
    response = {
        "task_id": task_id,
        "status": result.status,
    }
    if result.ready():
        if result.successful():
            response["result"] = result.result
        else:
            response["error"] = str(result.result)
    return response
```

backend/app/api/v1/attendance.py

```
from fastapi import APIRouter, Depends, Query
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from app.core.permissions import require_admin
from app.database import get_db
from app.dependencies import get_current_user
from app.exceptions import NotFoundException
from app.models.employee import Employee
from app.models.user import User
from app.schemas.attendance import AttendanceResponse,
     AttendanceListResponse

router = APIRouter(prefix="/attendance", tags=["Attendance"],
     dependencies=[Depends(require_admin)])

async def _get_employee(user: User, db: AsyncSession) -> Employee:
    result = await db.execute(
        select(Employee).where(
            Employee.user_id == user.id, Employee.deleted_at.is_(None)
        )
    )
    employee = result.scalar_one_or_none()
    if employee is None:
        employee = Employee(
            user_id=user.id,
            employee_code=f"EMP-{user.id:03d}",
            status="Active",
        )
        db.add(employee)
        await db.commit()
        await db.refresh(employee)
    return employee

@router.get("/history", response_model=AttendanceListResponse, summary="My
     attendance history")
async def attendance_history(
    page: int = Query(1, ge=1),
    per_page: int = Query(20, ge=1, le=100),
    current_user: User = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),
):
    from sqlalchemy import func
    from app.models.attendance import Attendance

    employee = await _get_employee(current_user, db)
    offset = (page - 1) * per_page

    count = (await db.execute(
        select(func.count(Attendance.id)).where(
            Attendance.employee_id == employee.id,
            Attendance.deleted_at.is_(None),
        )
    )).scalar() or 0

    result = await db.execute(
        select(Attendance)
        .where(Attendance.employee_id == employee.id,
     Attendance.deleted_at.is_(None))
        .order_by(Attendance.attendance_date.desc())
        .offset(offset)
        .limit(per_page)
    )
    records = result.scalars().all()

    return AttendanceListResponse(
```

```
        records=[AttendanceResponse.model_validate(r) for r in records],
        total=count,
        page=page,
        per_page=per_page,
    )

@router.get("/all", summary="Manager view — all employees attendance")
async def all_employees_attendance(
    page: int = Query(1, ge=1),
    per_page: int = Query(30, ge=1, le=100),
    date_filter: str | None = None,
    current_user: User = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),
):
    from sqlalchemy import func, Date as SQLDate, cast
    from app.models.attendance import Attendance

    filters = [Attendance.deleted_at.is_(None)]
    if date_filter:
        from datetime import date as dt_date
        try:
            d = dt_date.fromisoformat(date_filter)
            filters.append(Attendance.attendance_date == d)
        except ValueError:
            pass

    offset = (page - 1) * per_page
    count = (await
     db.execute(select(func.count(Attendance.id)).where(*filters))).scalar()
     or 0

    result = await db.execute(
        select(Attendance, Employee)
        .join(Employee, Employee.id == Attendance.employee_id)
        .where(*filters)
        .order_by(Attendance.attendance_date.desc(),
     Attendance.time_in.desc())
        .offset(offset).limit(per_page)
    )
    rows = result.all()

    records = []
    for att, emp in rows:
        records.append({
            "id": att.id,
            "employee_id": att.employee_id,
            "employee_code": emp.employee_code,
            "attendance_date": att.attendance_date.isoformat() if
     att.attendance_date else None,
            "check_in": att.time_in.isoformat() if att.time_in else None,
            "check_out": att.time_out.isoformat() if att.time_out else None,
            "total_hours": float(att.total_hours) if att.total_hours else
     None,
            "location": att.location,
            "location_status": att.location_status,
        })

    return {"records": records, "total": count, "page": page, "per_page":
     per_page}

@router.get("/summary/daily", summary="Today's attendance summary")
async def daily_summary(
    current_user: User = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),
):
    from sqlalchemy import func, cast, Date as SQLDate
    from datetime import date, datetime, timezone
    from app.models.attendance import Attendance
```

```
    today = date.today()
    total_emp = (await db.execute(
        select(func.count(Employee.id)).where(Employee.deleted_at.is_(None),
     Employee.status == "Active")
    )).scalar() or 0

    present = (await db.execute(
        select(func.count(Attendance.id)).where(
            Attendance.attendance_date == today,
            Attendance.deleted_at.is_(None),
        )
    )).scalar() or 0

    rate = round(present / total_emp * 100, 1) if total_emp > 0 else 0
    return {
        "date": today.isoformat(),
        "total_employees": total_emp,
        "present": present,
        "absent": max(0, total_emp - present),
        "attendance_rate": rate,
    }
```

backend/app/api/v1/hrms.py

```
import logging
from datetime import date
from typing import Optional

from fastapi import APIRouter, Depends, HTTPException
from pydantic import BaseModel
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy.orm import selectinload

from app.core.permissions import require_admin
from app.database import get_db
from app.models.employee import WorkSchedule
from app.models.hrms import Leave, Holiday

logger = logging.getLogger("trace.hrms")
router = APIRouter(prefix="/hrms", tags=["HRMS"],
     dependencies=[Depends(require_admin)])

class LeaveCreate(BaseModel):
    employee_id: int
    leave_type: str
    start_date: date
    end_date: date
    reason: Optional[str] = None

class LeaveStatusUpdate(BaseModel):
    status: str
    approved_by: Optional[int] = None

class HolidayCreate(BaseModel):
    name: str
    holiday_date: date
    description: Optional[str] = None
    type: str = "Public"

class ShiftCreate(BaseModel):
    schedule_name: str
    start_time: str
    end_time: str
    break_minutes: int = 0

@router.get("/leaves")
```

```
async def get_leaves(
    employee_id: Optional[int] = None,
    db: AsyncSession = Depends(get_db)
):
    query =
     select(Leave).options(selectinload(Leave.employee)).order_by(Leave.creat
     ed_at.desc())
    if employee_id:
        query = query.where(Leave.employee_id == employee_id)
    result = await db.execute(query)
    leaves = result.scalars().all()

    return [
        {
            "id": l.id,
            "employee_id": l.employee_id,
            "employee_code": l.employee.employee_code if l.employee else
     "N/A",
            "leave_type": l.leave_type,
            "start_date": l.start_date.isoformat(),
            "end_date": l.end_date.isoformat(),
            "reason": l.reason,
            "status": l.status,
            "approved_by": l.approved_by,
            "created_at": l.created_at.isoformat() if l.created_at else
     None,
        }
        for l in leaves
    ]

@router.post("/leaves")
async def create_leave(
    data: LeaveCreate,
    db: AsyncSession = Depends(get_db)
):
    leave = Leave(
        employee_id=data.employee_id,
        leave_type=data.leave_type,
        start_date=data.start_date,
        end_date=data.end_date,
        reason=data.reason,
        status="Pending",
    )
    db.add(leave)
    await db.commit()
    await db.refresh(leave)
    return {"status": "success", "message": "Leave application submitted",
     "leave_id": leave.id}

@router.patch("/leaves/{leave_id}/status")
async def update_leave_status(
    leave_id: int,
    data: LeaveStatusUpdate,
    db: AsyncSession = Depends(get_db)
):
    result = await db.execute(select(Leave).where(Leave.id == leave_id))
    leave = result.scalar_one_or_none()
    if not leave:
        raise HTTPException(status_code=404, detail="Leave record not
     found")

    leave.status = data.status
    if data.approved_by:
        leave.approved_by = data.approved_by
    await db.commit()
    return {"status": "success", "message": f"Leave status updated to
     {data.status}"}
```

```
@router.get("/holidays")
async def get_holidays(db: AsyncSession = Depends(get_db)):
    result = await
     db.execute(select(Holiday).order_by(Holiday.holiday_date.asc()))
    holidays = result.scalars().all()
    return [
        {
            "id": h.id,
            "name": h.name,
            "holiday_date": h.holiday_date.isoformat(),
            "description": h.description,
            "type": h.type,
        }
        for h in holidays
    ]

@router.post("/holidays")
async def create_holiday(data: HolidayCreate, db: AsyncSession =
     Depends(get_db)):
    holiday = Holiday(
        name=data.name,
        holiday_date=data.holiday_date,
        description=data.description,
        type=data.type,
    )
    db.add(holiday)
    await db.commit()
    await db.refresh(holiday)
    return {"status": "success", "message": "Holiday added successfully",
     "holiday_id": holiday.id}

@router.put("/holidays/{holiday_id}")
async def update_holiday(holiday_id: int, data: HolidayCreate, db:
     AsyncSession = Depends(get_db)):
    result = await db.execute(select(Holiday).where(Holiday.id ==
     holiday_id))
    holiday = result.scalar_one_or_none()
    if not holiday:
        raise HTTPException(status_code=404, detail="Holiday not found")
    holiday.name = data.name
    holiday.holiday_date = data.holiday_date
    holiday.description = data.description
    holiday.type = data.type
    await db.commit()
    return {"status": "success", "message": "Holiday updated successfully"}

@router.delete("/holidays/{holiday_id}")
async def delete_holiday(holiday_id: int, db: AsyncSession =
     Depends(get_db)):
    result = await db.execute(select(Holiday).where(Holiday.id ==
     holiday_id))
    holiday = result.scalar_one_or_none()
    if not holiday:
        raise HTTPException(status_code=404, detail="Holiday not found")
    await db.delete(holiday)
    await db.commit()
    return {"status": "success", "message": "Holiday deleted"}

@router.delete("/leaves/{leave_id}")
async def delete_leave(leave_id: int, db: AsyncSession = Depends(get_db)):
    result = await db.execute(select(Leave).where(Leave.id == leave_id))
    leave = result.scalar_one_or_none()
    if not leave:
        raise HTTPException(status_code=404, detail="Leave record not
     found")
    await db.delete(leave)
    await db.commit()
    return {"status": "success", "message": "Leave record deleted"}
```

```
@router.get("/shifts")
async def get_shifts(db: AsyncSession = Depends(get_db)):
    result = await
     db.execute(select(WorkSchedule).order_by(WorkSchedule.id.asc()))
    shifts = result.scalars().all()
    return [
        {
            "id": s.id,
            "schedule_name": s.schedule_name,
            "start_time": s.start_time.strftime("%H:%M") if s.start_time
     else "09:00",
            "end_time": s.end_time.strftime("%H:%M") if s.end_time else
     "18:00",
            "break_minutes": s.break_minutes,
        }
        for s in shifts
    ]

@router.post("/shifts")
async def create_shift(data: ShiftCreate, db: AsyncSession =
     Depends(get_db)):
    from datetime import time
    sh_start = time.fromisoformat(data.start_time)
    sh_end = time.fromisoformat(data.end_time)
    shift = WorkSchedule(
        schedule_name=data.schedule_name,
        start_time=sh_start,
        end_time=sh_end,
        break_minutes=data.break_minutes,
    )
    db.add(shift)
    await db.commit()
    await db.refresh(shift)
    return {"status": "success", "message": "Shift created successfully",
     "shift_id": shift.id}

@router.put("/shifts/{shift_id}")
async def update_shift(shift_id: int, data: ShiftCreate, db: AsyncSession =
     Depends(get_db)):
    from datetime import time
    result = await db.execute(select(WorkSchedule).where(WorkSchedule.id ==
     shift_id))
    shift = result.scalar_one_or_none()
    if not shift:
        raise HTTPException(status_code=404, detail="Shift not found")
    shift.schedule_name = data.schedule_name
    shift.start_time = time.fromisoformat(data.start_time)
    shift.end_time = time.fromisoformat(data.end_time)
    shift.break_minutes = data.break_minutes
    await db.commit()
    return {"status": "success", "message": "Shift updated successfully"}

@router.delete("/shifts/{shift_id}")
async def delete_shift(shift_id: int, db: AsyncSession = Depends(get_db)):
    result = await db.execute(select(WorkSchedule).where(WorkSchedule.id ==
     shift_id))
    shift = result.scalar_one_or_none()
    if not shift:
        raise HTTPException(status_code=404, detail="Shift not found")
    await db.delete(shift)
    await db.commit()
    return {"status": "success", "message": "Shift deleted successfully"}

class ShiftAssignRequest(BaseModel):
    shift_id: int

@router.post("/employees/{employee_id}/assign-shift")
```

```
async def assign_shift_to_employee(
    employee_id: int,
    data: ShiftAssignRequest,
    db: AsyncSession = Depends(get_db),
):
    from app.models.employee import Employee
    emp = (await db.execute(select(Employee).where(Employee.id ==
     employee_id, Employee.deleted_at.is_(None)))).scalar_one_or_none()
    if not emp:
        raise HTTPException(status_code=404, detail="Employee not found")
    shift = (await db.execute(select(WorkSchedule).where(WorkSchedule.id ==
     data.shift_id))).scalar_one_or_none()
    if not shift:
        raise HTTPException(status_code=404, detail="Shift not found")
    emp.work_schedule_id = data.shift_id
    await db.commit()
    return {"status": "success", "message": f"Shift '{shift.schedule_name}'
     assigned to employee {employee_id}"}

LEAVE_ENTITLEMENTS = {
    "Casual": 10,
    "Sick": 7,
    "Earned": 15,
    "Unpaid": 0,
}

@router.get("/employees/{employee_id}/leave-balance")
async def get_leave_balance(employee_id: int, year: int = None, db:
     AsyncSession = Depends(get_db)):
    from sqlalchemy import func, extract
    import datetime

    yr = year or datetime.date.today().year
    result = await db.execute(
        select(Leave.leave_type, func.count(Leave.id).label("count"))
        .where(
            Leave.employee_id == employee_id,
            Leave.status == "Approved",
            extract("year", Leave.start_date) == yr,
        )
        .group_by(Leave.leave_type)
    )
    used_map = {row.leave_type: row.count for row in result.all()}

    balance = {}
    for lt, entitlement in LEAVE_ENTITLEMENTS.items():
        used = used_map.get(lt, 0)
        balance[lt] = {"entitlement": entitlement, "used": used,
     "remaining": max(0, entitlement - used)}

    return {"employee_id": employee_id, "year": yr, "balance": balance}

@router.get("/leave-balance/summary")
async def all_employees_leave_balance(year: int = None, db: AsyncSession =
     Depends(get_db)):
    from sqlalchemy import func, extract
    from app.models.employee import Employee
    import datetime

    yr = year or datetime.date.today().year
    result = await db.execute(
        select(Leave.employee_id, Leave.leave_type,
     func.count(Leave.id).label("count"))
        .where(Leave.status == "Approved", extract("year", Leave.start_date)
     == yr)
        .group_by(Leave.employee_id, Leave.leave_type)
    )
    rows = result.all()
```

```
    emp_map: dict[int, dict] = {}
    for row in rows:
        if row.employee_id not in emp_map:
            emp_map[row.employee_id] = {}
        emp_map[row.employee_id][row.leave_type] = row.count

    emps = (await
     db.execute(select(Employee).where(Employee.deleted_at.is_(None)))).scala
     rs().all()
    summary = []
    for emp in emps:
        used = emp_map.get(emp.id, {})
        b = {}
        for lt, ent in LEAVE_ENTITLEMENTS.items():
            u = used.get(lt, 0)
            b[lt] = {"entitlement": ent, "used": u, "remaining": max(0, ent
     - u)}
        summary.append({"employee_id": emp.id, "employee_code":
     emp.employee_code, "balance": b})

    return {"year": yr, "entitlements": LEAVE_ENTITLEMENTS, "employees":
     summary}
```

backend/app/api/v1/productivity.py

```
from datetime import date

from fastapi import APIRouter, Depends, Query
from sqlalchemy import func, select
from sqlalchemy.ext.asyncio import AsyncSession

from app.core.permissions import require_admin, require_manager_or_admin
from app.dependencies import get_current_user
from app.database import get_db
from app.models.user import User
from app.models.employee import Employee
from app.models.attendance import Attendance
from app.models.task import Task
from app.models.productivity import ProductivityScore
from app.schemas.productivity import ProductivityScoreResponse
from app.services.scoring_service import ScoringService
from app.services.report_service import ReportService

router = APIRouter(tags=["Productivity & Reports"],
     dependencies=[Depends(require_admin)])

@router.post(
    "/productivity/compute/{employee_id}",
    response_model=ProductivityScoreResponse,
    summary="Compute productivity score",
    description="Trigger the AI scoring engine for a specific employee and
     week. Admin/Manager only.",
)
async def compute_score(
    employee_id: int,
    week_start: date = Query(..., description="Monday of the target week
     (YYYY-MM-DD)"),
    current_user: User = Depends(require_manager_or_admin),
    db: AsyncSession = Depends(get_db),
):
    service = ScoringService(db)
    score = await service.compute_weekly_score(employee_id, week_start)
    return ProductivityScoreResponse.model_validate(score)

@router.post(
    "/productivity/compute-all",
    summary="Compute scores for all active employees",
```

```
    description="Batch-trigger scoring for every active employee.
     Admin/Manager only.",
)
async def compute_all_scores(
    week_start: date = Query(..., description="Monday of the target week"),
    current_user: User = Depends(require_manager_or_admin),
    db: AsyncSession = Depends(get_db),
):
    result = await db.execute(
        select(Employee.id).where(
            Employee.status == "Active", Employee.deleted_at.is_(None)
        )
    )
    employee_ids = [row[0] for row in result.all()]

    service = ScoringService(db)
    computed = []
    errors = []

    for emp_id in employee_ids:
        try:
            score = await service.compute_weekly_score(emp_id, week_start)
            computed.append({
                "employee_id": emp_id,
                "final_score": float(score.final_score) if score.final_score
     else 0,
                "rating": score.rating,
            })
        except Exception as e:
            errors.append({"employee_id": emp_id, "error": str(e)})

    return {
        "week_start": week_start.isoformat(),
        "computed": len(computed),
        "failed": len(errors),
        "results": computed,
        "errors": errors,
    }

@router.get(
    "/productivity/scores",
    summary="List productivity scores",
    description="Admin/Manager only. Filterable by employee and week.",
)
async def list_scores(
    employee_id: int | None = None,
    week_start: date | None = None,
    page: int = Query(1, ge=1),
    per_page: int = Query(20, ge=1, le=100),
    current_user: User = Depends(require_manager_or_admin),
    db: AsyncSession = Depends(get_db),
):
    offset = (page - 1) * per_page
    conditions = [ProductivityScore.deleted_at.is_(None)]

    if employee_id:
        conditions.append(ProductivityScore.employee_id == employee_id)
    if week_start:
        conditions.append(ProductivityScore.week_start == week_start)

    count = (await db.execute(
        select(func.count(ProductivityScore.id)).where(*conditions)
    )).scalar() or 0

    result = await db.execute(
        select(ProductivityScore)
        .where(*conditions)
```

```
        .order_by(ProductivityScore.week_start.desc(),
     ProductivityScore.final_score.desc())
        .offset(offset)
        .limit(per_page)
    )
    scores = result.scalars().all()

    return {
        "scores": [ProductivityScoreResponse.model_validate(s) for s in
     scores],
        "total": count,
        "page": page,
        "per_page": per_page,
    }

@router.get(
    "/reports/weekly-summary",
    summary="Weekly productivity summary",
    description="Admin/Manager only.",
)
async def weekly_summary(
    week_start: date | None = None,
    current_user: User = Depends(require_manager_or_admin),
    db: AsyncSession = Depends(get_db),
):
    service = ReportService(db)
    return await service.generate_weekly_summary(week_start)

@router.get(
    "/reports/attendance-summary",
    summary="Daily attendance summary",
    description="Defaults to today.",
)
async def attendance_summary(
    target_date: date | None = None,
    current_user: User = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),
):
    service = ReportService(db)
    return await service.generate_attendance_summary(target_date)

@router.get(
    "/dashboard/stats",
    summary="Admin dashboard statistics",
    description="Aggregated system-wide metrics for the dashboard.",
)
async def dashboard_stats(
    current_user: User = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),
):
    from datetime import timedelta

    today = date.today()
    week_start = today - timedelta(days=today.weekday())

    total_employees = (await db.execute(
        select(func.count(Employee.id)).where(Employee.deleted_at.is_(None))
    )).scalar() or 0

    active_employees = (await db.execute(
        select(func.count(Employee.id)).where(
            Employee.status == "Active", Employee.deleted_at.is_(None)
        )
    )).scalar() or 0

    present_today = (await db.execute(
        select(func.count(Attendance.id)).where(
            Attendance.attendance_date == today,
```

```
            Attendance.deleted_at.is_(None),
        )
    )).scalar() or 0

    task_pending = (await db.execute(
        select(func.count(Task.id)).where(
            Task.status == "Pending", Task.deleted_at.is_(None)
        )
    )).scalar() or 0

    task_wip = (await db.execute(
        select(func.count(Task.id)).where(
            Task.status == "WIP", Task.deleted_at.is_(None)
        )
    )).scalar() or 0

    task_completed = (await db.execute(
        select(func.count(Task.id)).where(
            Task.status == "Completed", Task.deleted_at.is_(None)
        )
    )).scalar() or 0

    task_delayed = (await db.execute(
        select(func.count(Task.id)).where(
            Task.status == "Overdue", Task.deleted_at.is_(None)
        )
    )).scalar() or 0

    total_tasks = (await db.execute(
        select(func.count(Task.id)).where(Task.deleted_at.is_(None))
    )).scalar() or 0

    avg_score = (await db.execute(
        select(func.avg(ProductivityScore.final_score)).where(
            ProductivityScore.week_start == week_start,
            ProductivityScore.deleted_at.is_(None),
        )
    )).scalar()

    at_risk_count = (await db.execute(
        select(func.count(ProductivityScore.id)).where(
            ProductivityScore.week_start == week_start,
            ProductivityScore.rating == "Risk",
            ProductivityScore.deleted_at.is_(None),
        )
    )).scalar() or 0

    return {
        "date": today.isoformat(),
        "week_start": week_start.isoformat(),
        "employees": {
            "total": total_employees,
            "active": active_employees,
        },
        "attendance": {
            "present_today": present_today,
            "attendance_rate": round(
                (present_today / active_employees * 100), 1
            ) if active_employees > 0 else 0,
        },
        "tasks": {
            "total": total_tasks,
            "pending": task_pending,
            "wip": task_wip,
            "completed": task_completed,
            "delayed": task_delayed,
        },
        "productivity": {
```

```
            "avg_score": round(float(avg_score), 2) if avg_score else None,
            "at_risk_count": at_risk_count,
        },
    }
```

4.1.9 Backend: Background Jobs

backend/app/celery_app.py

```
from celery import Celery
from celery.schedules import crontab

from app.config import get_settings

settings = get_settings()

celery_app = Celery(
    "trace",
    broker=settings.REDIS_URL,
    backend=settings.REDIS_URL,
)

celery_app.conf.update(
    task_serializer="json",
    accept_content=["json"],
    result_serializer="json",
    timezone="Asia/Kolkata",
    enable_utc=True,
    task_track_started=True,
    result_extended=True,
    task_acks_late=True,
    worker_prefetch_multiplier=1,
    imports=["app.tasks"],
)

celery_app.conf.beat_schedule = {
    "mark-overdue-tasks-nightly": {
        "task": "app.tasks.mark_overdue_tasks",
        "schedule": crontab(hour=23, minute=0),
    },
    "task-due-reminders-daily": {
        "task": "app.tasks.task_due_reminders",
        "schedule": crontab(hour=8, minute=0),
    },
    "project-progress-docs-weekly": {
        "task": "app.tasks.update_project_progress_docs",
        "schedule": crontab(day_of_week="monday", hour=7, minute=0),
    },
    "compute-weekly-scores": {
        "task": "app.tasks.compute_all_weekly_scores",
        "schedule": crontab(day_of_week="sunday", hour=23, minute=30),
    },
}

celery_app.autodiscover_tasks(["app"])
```

backend/app/tasks.py

```
import asyncio
import logging
from datetime import date, datetime, timedelta, timezone

from app.celery_app import celery_app

logger = logging.getLogger("trace.tasks")

def _run_async(coro):
    loop = asyncio.new_event_loop()
```

```
    try:
        return loop.run_until_complete(coro)
    finally:
        loop.close()

@celery_app.task(name="app.tasks.mark_overdue_tasks", bind=True,
     max_retries=3)
def mark_overdue_tasks(self):
    async def _mark():
        from app.services.jobs import job_session
        from app.services.project_service import mark_overdue_tasks as mark

        async with job_session() as db:
            return await mark(db, date.today())

    try:
        return _run_async(_mark())
    except Exception as e:
        logger.error(f"[mark_overdue_tasks] Failed: {e}")
        raise self.retry(exc=e, countdown=60)

@celery_app.task(name="app.tasks.task_due_reminders", bind=True,
     max_retries=3)
def task_due_reminders(self):
    async def _remind():
        from app.services.jobs import job_session
        from app.services.project_service import task_due_reminders as
     remind

        async with job_session() as db:
            return await remind(db, date.today())

    try:
        return _run_async(_remind())
    except Exception as e:
        logger.error(f"[task_due_reminders] Failed: {e}")
        raise self.retry(exc=e, countdown=60)

def _ai_job(task, runner: str, *args):
    async def _run():
        from app.services import ai_jobs, ai_service
        from app.services.github_service import get_github_client
        from app.services.jobs import job_session

        async with job_session() as db:
            ai = ai_service.client_from_settings() if
     ai_service.ai_configured() else None
            github = await get_github_client(db)
            return await getattr(ai_jobs, runner)(db, *args,
     job_id=task.request.id, ai=ai, github=github)

    return _run_async(_run())

@celery_app.task(name="app.tasks.evaluate_project", bind=True)
def evaluate_project(self, project_id: int):
    return _ai_job(self, "run_evaluate_project", project_id)

@celery_app.task(name="app.tasks.generate_task_progress_doc", bind=True)
def generate_task_progress_doc(self, task_id: int):
    return _ai_job(self, "run_task_progress_doc", task_id)

@celery_app.task(name="app.tasks.reevaluate_gantt", bind=True)
def reevaluate_gantt(self, project_id: int, reason: str, requested_at: str):
    return _ai_job(self, "run_reevaluate_gantt", project_id, reason,
     requested_at)

@celery_app.task(name="app.tasks.update_project_progress_doc", bind=True)
def update_project_progress_doc(self, project_id: int):
```

```
    return _ai_job(self, "run_progress_report", project_id)

@celery_app.task(name="app.tasks.update_project_progress_docs", bind=True)
def update_project_progress_docs(self):
    async def _ids():
        from sqlalchemy import select
        from app.models.project import Project
        from app.services.jobs import job_session

        async with job_session() as db:
            return list((await db.execute(select(Project.id).where(
                Project.status == "Active",
     Project.deleted_at.is_(None)))).scalars())

    from app.services.jobs import enqueue
    ids = _run_async(_ids())
    for project_id in ids:
        enqueue("update_project_progress_doc", project_id)
    return {"status": "success", "projects": ids}

@celery_app.task(name="app.tasks.suggest_managers", bind=True)
def suggest_managers(self, project_id: int):
    return _ai_job(self, "run_suggest_managers", project_id)

@celery_app.task(name="app.tasks.suggest_assignees", bind=True)
def suggest_assignees(self, project_id: int, title: str, description: str |
     None = None):
    return _ai_job(self, "run_suggest_assignees", project_id, title,
     description)

@celery_app.task(name="app.tasks.suggest_tasks", bind=True)
def suggest_tasks(self, group_id: int):
    return _ai_job(self, "run_suggest_tasks", group_id)

@celery_app.task(name="app.tasks.compute_all_weekly_scores", bind=True,
     max_retries=3)
def compute_all_weekly_scores(self):
    async def _compute():
        from sqlalchemy import select
        from app.models.employee import Employee
        from app.services.jobs import job_session
        from app.services.scoring_service import ScoringService

        today = date.today()
        week_start = today - timedelta(days=today.weekday())

        async with job_session() as db:
            try:
                result = await db.execute(
                    select(Employee.id).where(
                        Employee.status == "Active",
                        Employee.deleted_at.is_(None),
                    )
                )
                employee_ids = [row[0] for row in result.all()]

                service = ScoringService(db)
                computed = 0
                errors = []

                for emp_id in employee_ids:
                    try:
                        await service.compute_weekly_score(emp_id,
     week_start)
                        computed += 1
                    except Exception as e:
                        errors.append({"employee_id": emp_id, "error":
     str(e)})
```

```
                        logger.warning(
                            f"[compute_scores] Failed for employee {emp_id}:
    {e}"
                        )

                await db.commit()
                logger.info(
                    f"[compute_scores] Computed
    {computed}/{len(employee_ids)} scores "
                    f"for week {week_start}"
                )
                return {
                    "status": "success",
                    "week_start": week_start.isoformat(),
                    "total_employees": len(employee_ids),
                    "computed": computed,
                    "errors": errors,
                }
            except Exception as e:
                await db.rollback()
                logger.error(f"[compute_scores] Failed: {e}")
                raise self.retry(exc=e, countdown=120)

    return _run_async(_compute())

@celery_app.task(name="app.tasks.send_mail_task", bind=True, max_retries=3)
def send_mail_task(self, to_email: str, subject: str, body: str):
    async def _config():
        from app.services.jobs import job_session
        from app.services.mail_service import mail_config
        async with job_session() as db:
            return await mail_config(db)

    try:
        from app.services.mail_service import send

        config = _run_async(_config())
        send(config, to_email, subject, body)
        logger.info(f"[send_mail_task] Email sent to {to_email}: {subject}")
        return {"status": "success", "to": to_email, "subject": subject,
     "smtp": f"{config.host}:{config.port}"}
    except Exception as e:
        logger.error(f"[send_mail_task] Failed: {e}")
        raise self.retry(exc=e, countdown=30)

@celery_app.task(name="app.tasks.upload_file_task", bind=True,
     max_retries=3)
def upload_file_task(self, filename: str, encoded_content: str,
     content_type: str):
    try:
        import base64
        import io
        from minio import Minio
        from app.config import get_settings

        settings = get_settings()
        content = base64.b64decode(encoded_content)

        client = Minio(
            settings.MINIO_ENDPOINT,
            access_key=settings.MINIO_ACCESS_KEY,
            secret_key=settings.MINIO_SECRET_KEY,
            secure=settings.MINIO_SECURE,
        )

        if not client.bucket_exists(settings.MINIO_BUCKET):
            client.make_bucket(settings.MINIO_BUCKET)
```

```
        import uuid
        object_name = f"uploads/{uuid.uuid4().hex[:8]}_{filename}"
        data = io.BytesIO(content)

        client.put_object(
            settings.MINIO_BUCKET,
            object_name,
            data,
            length=len(content),
            content_type=content_type or "application/octet-stream",
        )

        logger.info(f"[upload_file_task] Uploaded {object_name} to
    {settings.MINIO_BUCKET}")
        return {
            "status": "success",
            "bucket": settings.MINIO_BUCKET,
            "object_name": object_name,
            "size_bytes": len(content),
            "content_type": content_type,
        }
    except Exception as e:
        logger.error(f"[upload_file_task] Failed: {e}")
        raise self.retry(exc=e, countdown=30)

@celery_app.task(name="app.tasks.llm_chat_task", bind=True, max_retries=2)
def llm_chat_task(self, message: str, model: str = None):
    try:
        from openai import OpenAI

        from app.config import get_settings

        settings = get_settings()
        api_key, base_url, default_model = settings.OPENAI_API_KEY,
    settings.OPENAI_BASE_URL, settings.OPENAI_MODEL
        if not api_key:
            return {"status": "error", "error": "AI is not configured. Set
    OPENAI_API_KEY in .env."}

        client_kwargs = {"api_key": api_key}
        if base_url:
            client_kwargs["base_url"] = base_url

        client = OpenAI(**client_kwargs)
        use_model = model or default_model

        response = client.chat.completions.create(
            model=use_model,
            messages=[
                {"role": "system", "content": "You are a helpful assistant
    for the TRACE task management system. Be concise and professional."},
                {"role": "user", "content": message},
            ],
            max_tokens=1024,
            temperature=0.7,
        )

        reply = response.choices[0].message.content
        logger.info(f"[llm_chat_task] LLM response received (model:
    {use_model})")
        return {
            "status": "success",
            "model": use_model,
            "reply": reply,
            "usage": {
                "prompt_tokens": response.usage.prompt_tokens,
                "completion_tokens": response.usage.completion_tokens,
                "total_tokens": response.usage.total_tokens,
```

```
            },
        }
    except Exception as e:
        logger.error(f"[llm_chat_task] Failed: {e}")
        raise self.retry(exc=e, countdown=15)

@celery_app.task(name="app.tasks.compute_task_health_and_escalate",
     bind=True, max_retries=2)
def compute_task_health_and_escalate(self):
    async def _compute():
        from sqlalchemy import select
        from app.database import async_session_factory
        from app.models.task import Task
        from app.models.employee import Employee

        today = date.today()
        now = datetime.now(timezone.utc)

        async with async_session_factory() as db:
            try:
                res = await db.execute(
                    select(Task).where(
                        Task.status.notin_(["Completed", "Cancelled"]),
                        Task.deleted_at.is_(None)
                    )
                )
                tasks = res.scalars().all()

                updated_count = 0
                for t in tasks:
                    health = "Green"
                    reasons = []

                    if t.due_date:
                        if t.due_date < today:
                            health = "Red"
                            reasons.append(f"Overdue since {t.due_date}")
                        elif t.due_date == today:
                            if health != "Red":
                                health = "Yellow"
                            reasons.append("Due today")

                    t.health_score = health
                    t.health_reason = "; ".join(reasons) if reasons else "On
     track"
                    t.last_health_check = now
                    updated_count += 1

                await db.commit()
                return {
                    "status": "success",
                    "tasks_checked": len(tasks),
                    "tasks_updated": updated_count,
                }
            except Exception as e:
                await db.rollback()
                logger.error(f"[compute_task_health_and_escalate] Failed:
     {e}")
                raise self.retry(exc=e, countdown=60)

    return _run_async(_compute())
```

4.1.10 Frontend: Application Shell

`frontend/src/main.jsx`

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
```

```
import { BrowserRouter } from 'react-router-dom'
import { AuthProvider } from './contexts/AuthContext'
import { ThemeProvider } from './contexts/ThemeContext'
import { LiveProvider } from './contexts/LiveContext'
import { TooltipProvider } from '@/components/ui/tooltip'
import { Toaster } from 'sonner'
import App from './App.jsx'
import './index.css'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>
      <AuthProvider>
        <LiveProvider>
          <ThemeProvider>
            <TooltipProvider>
              <App />
              <Toaster richColors position="top-right" closeButton />
            </TooltipProvider>
          </ThemeProvider>
        </LiveProvider>
      </AuthProvider>
    </BrowserRouter>
  </StrictMode>,
)
```

frontend/src/App.jsx

```
import { Routes, Route, Navigate } from 'react-router-dom'

import GuestOnlyRoute from './components/GuestOnlyRoute'
import RoleRoute from './components/RoleRoute'

import AdminLayout from './layouts/AdminLayout'
import ManagerLayout from './layouts/ManagerLayout'
import EmployeeLayout from './layouts/EmployeeLayout'

import LandingPage from './pages/LandingPage'
import LoginPage from './pages/LoginPage'
import ForbiddenPage from './pages/ForbiddenPage'
import NotFoundPage from './pages/NotFoundPage'

import ControlPanelPage from './pages/admin/ControlPanelPage'
import WorkforcePage from './pages/admin/WorkforcePage'
import MonitorPage from './pages/admin/MonitorPage'
import ConfigurationPage from './pages/admin/ConfigurationPage'
import AdminSettingsPage from './pages/admin/AdminSettingsPage'

import ManagerDashboardPage from './pages/manager/ManagerDashboardPage'
import ManagerSettingsPage from './pages/manager/ManagerSettingsPage'
import ProjectCreatePage from './pages/manager/ProjectCreatePage'
import ManagerProjectDetailPage from './pages/manager/ProjectDetailPage'

import EmployeeDashboardPage from './pages/employee/EmployeeDashboardPage'
import EmployeeSettingsPage from './pages/employee/EmployeeSettingsPage'
import EmployeeProjectDetailPage from './pages/employee/ProjectDetailPage'
import TaskDetailPage from './pages/employee/TaskDetailPage'

function App() {
  return (
    <Routes>
      <Route element={<GuestOnlyRoute />}>
        <Route path="/" element={<LandingPage />} />
        <Route path="/login" element={<LoginPage />} />
      </Route>

      <Route path="/dashboard" element={<Navigate to="/login" replace />} />
```

```
      <Route path="/403" element={<ForbiddenPage />} />

      <Route element={<RoleRoute role="Admin" />}>
        <Route element={<AdminLayout />}>
          <Route path="/admin" element={<Navigate to="/admin/control-panel"
    replace />} />
          <Route path="/admin/control-panel" element={<ControlPanelPage
    />} />
          <Route path="/admin/workforce" element={<WorkforcePage />} />
          <Route path="/admin/monitor" element={<MonitorPage />} />
          <Route path="/admin/configuration" element={<ConfigurationPage />}
    />
          <Route path="/admin/settings" element={<AdminSettingsPage />} />
        </Route>
      </Route>

      <Route element={<RoleRoute role="Manager" />}>
        <Route element={<ManagerLayout />}>
          <Route path="/manager" element={<ManagerDashboardPage />} />
          <Route path="/manager/projects/new" element={<ProjectCreatePage
    />} />
          <Route path="/manager/projects/:id"
    element={<ManagerProjectDetailPage />} />
          <Route path="/manager/tasks/:taskId" element={<TaskDetailPage
    />} />
          <Route path="/manager/settings" element={<ManagerSettingsPage
    />} />
        </Route>
      </Route>

      <Route element={<RoleRoute role="Employee" />}>
        <Route element={<EmployeeLayout />}>
          <Route path="/employee" element={<EmployeeDashboardPage />} />
          <Route path="/employee/projects/:id"
    element={<EmployeeProjectDetailPage />} />
          <Route path="/employee/tasks/:taskId" element={<TaskDetailPage />}
    />
          <Route path="/employee/settings" element={<EmployeeSettingsPage
    />} />
        </Route>
      </Route>

      <Route path="*" element={<NotFoundPage />} />
    </Routes>
  )
}

export default App
```

frontend/src/contexts/AuthContext.jsx

```
import { createContext, useContext, useState, useEffect, useCallback, useRef
    } from 'react'
import { authApi } from '@/api'

const AuthContext = createContext(null)

export function AuthProvider({ children }) {
  const [user, setUser] = useState(() => {
    try {
      const stored = localStorage.getItem('trace_user')
      return stored ? JSON.parse(stored) : null
    } catch {
      return null
    }
  })
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState(null)
```

```
const justAuthenticatedRef = useRef(false)

const isAuthenticated = !!user && !!
   localStorage.getItem('trace_access_token')

const fetchUser = useCallback(async () => {
  const token = localStorage.getItem('trace_access_token')
  if (!token) {
    setUser(null)
    setLoading(false)
    return
  }

  if (justAuthenticatedRef.current) {
    justAuthenticatedRef.current = false
    setLoading(false)
    return
  }

  try {
    const res = await authApi.getMe()
    const userData = res.data
    setUser(userData)
    localStorage.setItem('trace_user', JSON.stringify(userData))
  } catch (err) {
    const status = err.response?.status

    if (status === 401 || status === 403) {
      setUser(null)
      localStorage.removeItem('trace_access_token')
      localStorage.removeItem('trace_refresh_token')
      localStorage.removeItem('trace_user')
    }
  } finally {
    setLoading(false)
  }
}, [])

useEffect(() => {
  fetchUser()
}, [fetchUser])

const login = async (email, password) => {
  setError(null)
  try {
    const res = await authApi.login(email, password)
    const { tokens, user: userData } = res.data

    localStorage.setItem('trace_access_token', tokens.access_token)
    localStorage.setItem('trace_refresh_token', tokens.refresh_token)
    localStorage.setItem('trace_user', JSON.stringify(userData))

    justAuthenticatedRef.current = true
    setUser(userData)
    setLoading(false)
    return userData
  } catch (err) {
    const msg =
      err.response?.data?.detail || err.response?.data?.message || 'Login
   failed'
    setError(msg)
    throw new Error(msg)
  }
}

const register = async (data) => {
  setError(null)
```

```
    try {
      const res = await authApi.register(data)
      const { tokens, user: userData } = res.data

      localStorage.setItem('trace_access_token', tokens.access_token)
      localStorage.setItem('trace_refresh_token', tokens.refresh_token)
      localStorage.setItem('trace_user', JSON.stringify(userData))

      justAuthenticatedRef.current = true
      setUser(userData)
      setLoading(false)
      return userData
    } catch (err) {
      const msg =
        err.response?.data?.detail || err.response?.data?.message ||
     'Registration failed'
      setError(msg)
      throw new Error(msg)
    }
  }

  const logout = () => {
    localStorage.removeItem('trace_access_token')
    localStorage.removeItem('trace_refresh_token')
    localStorage.removeItem('trace_user')
    justAuthenticatedRef.current = false
    setUser(null)
    setError(null)
  }

  const updateUser = (newData) => {
    const updated = { ...user, ...newData }
    setUser(updated)
    localStorage.setItem('trace_user', JSON.stringify(updated))
  }

  const value = {
    user,
    loading,
    error,
    isAuthenticated,
    login,
    register,
    logout,
    fetchUser,
    updateUser,
    setError,
  }

  return <AuthContext.Provider
     value={value}>{children}</AuthContext.Provider>
}

export function useAuth() {
  const ctx = useContext(AuthContext)
  if (!ctx) throw new Error('useAuth must be used within AuthProvider')
  return ctx
}
```

frontend/src/contexts/LiveContext.jsx

```
import { createContext, useCallback, useContext, useEffect, useRef } from
     'react'
import { useAuth } from '@/contexts/AuthContext'

const LiveContext = createContext({ subscribe: () => () => {} })

export function LiveProvider({ children }) {
```

```
  const { isAuthenticated } = useAuth()
  const listeners = useRef(new Set())

  useEffect(() => {
    if (!isAuthenticated) return undefined
    let ws = null
    let retry = null
    let delay = 2000
    let closed = false

    const connect = () => {
      const token = localStorage.getItem('trace_access_token')
      if (!token || closed) return
      const proto = window.location.protocol === 'https:' ? 'wss' : 'ws'
      ws = new WebSocket(`${proto}://${window.location.host}/api/v1/ws?
     token=${encodeURIComponent(token)}`)
      ws.onopen = () => { delay = 2000 }
      ws.onmessage = (msg) => {
        let event
        try { event = JSON.parse(msg.data) } catch { return }
        listeners.current.forEach((fn) => fn(event))
      }
      ws.onclose = () => {
        if (closed) return
        retry = setTimeout(connect, delay)
        delay = Math.min(delay * 2, 60000)
      }
    }
    connect()
    return () => {
      closed = true
      clearTimeout(retry)
      ws?.close()
    }
  }, [isAuthenticated])

  const subscribe = useCallback((fn) => {
    listeners.current.add(fn)
    return () => listeners.current.delete(fn)
  }, [])

  return <LiveContext.Provider value={{ subscribe
     }}>{children}</LiveContext.Provider>
}

export function useLiveEvents(handler) {
  const { subscribe } = useContext(LiveContext)
  const ref = useRef(handler)
  useEffect(() => { ref.current = handler })
  useEffect(() => subscribe((event) => ref.current(event)), [subscribe])
}
```

frontend/src/contexts/ThemeContext.jsx

```
import { createContext, useContext, useState, useEffect } from 'react'

const ThemeContext = createContext(null)

export function ThemeProvider({ children }) {
  const [mode, setMode] = useState(() => {
    return localStorage.getItem('trace_mode') || 'dark'
  })

  useEffect(() => {
    localStorage.setItem('trace_mode', mode)
    if (mode === 'dark') {
      document.documentElement.classList.add('dark')
    } else {
```

```
      document.documentElement.classList.remove('dark')
    }
  }, [mode])

  const toggleMode = () => {
    setMode((prev) => (prev === 'dark' ? 'light' : 'dark'))
  }

  const value = {
    mode,
    setMode,
    toggleMode,
  }

  return <ThemeContext.Provider
     value={value}>{children}</ThemeContext.Provider>
}

export function useTheme() {
  const ctx = useContext(ThemeContext)
  if (!ctx) throw new Error('useTheme must be used within ThemeProvider')
  return ctx
}
```

frontend/src/api/client.js

```
import axios from 'axios'

const api = axios.create({
  baseURL: '/api/v1',
  headers: { 'Content-Type': 'application/json' },
  timeout: 15000,
})

api.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem('trace_access_token')
    if (token) {
      config.headers.Authorization = `Bearer ${token}`
    }
    return config
  },
  (error) => Promise.reject(error),
)

let isRefreshing = false
let failedQueue = []

const processQueue = (error, token = null) => {
  failedQueue.forEach((prom) => {
    if (error) prom.reject(error)
    else prom.resolve(token)
  })
  failedQueue = []
}

api.interceptors.response.use(
  (response) => response,
  async (error) => {
    const originalRequest = error.config

    if (error.response?.status === 401 && !originalRequest._retry) {
      const refreshToken = localStorage.getItem('trace_refresh_token')

      if (!refreshToken) {
        localStorage.removeItem('trace_access_token')
        localStorage.removeItem('trace_refresh_token')
        localStorage.removeItem('trace_user')
```

```
        window.location.href = '/login'
        return Promise.reject(error)
      }

      if (isRefreshing) {
        return new Promise((resolve, reject) => {
          failedQueue.push({ resolve, reject })
        })
          .then((token) => {
            originalRequest.headers.Authorization = `Bearer ${token}`
            return api(originalRequest)
          })
          .catch((err) => Promise.reject(err))
      }

      originalRequest._retry = true
      isRefreshing = true

      try {
        const res = await axios.post('/api/v1/auth/refresh', {
          refresh_token: refreshToken,
        })
        const { access_token, refresh_token: newRefresh } = res.data
        localStorage.setItem('trace_access_token', access_token)
        localStorage.setItem('trace_refresh_token', newRefresh)
        api.defaults.headers.Authorization = `Bearer ${access_token}`
        processQueue(null, access_token)
        originalRequest.headers.Authorization = `Bearer ${access_token}`
        return api(originalRequest)
      } catch (refreshError) {
        processQueue(refreshError, null)
        localStorage.removeItem('trace_access_token')
        localStorage.removeItem('trace_refresh_token')
        localStorage.removeItem('trace_user')
        window.location.href = '/login'
        return Promise.reject(refreshError)
      } finally {
        isRefreshing = false
      }
    }

    return Promise.reject(error)
  },
)

export default api
```

frontend/src/api/index.js

```
export { default as api } from './client'
export { authApi } from './auth'
export { tasksApi } from './tasks'
export { projectsApi } from './projects'
export { adminApi } from './admin'
export { settingsApi } from './settings'
export { notificationsApi } from './notifications'
export { workforceApi } from './workforce'
```

frontend/src/api/auth.js

```
import api from './client'

export const authApi = {
  login: (email, password) =>
    api.post('/auth/login', { email, password }),

  register: (data) =>
    api.post('/auth/register', data),
```

```
  refresh: (refreshToken) =>
    api.post('/auth/refresh', { refresh_token: refreshToken }),

  getMe: () =>
    api.get('/users/me'),

  updateProfile: (userId, data) =>
    api.patch(`/users/${userId}`, data),
}
```

frontend/src/api/projects.js

```
import api from './client'

export const projectsApi = {
  list: () => api.get('/projects'),
  get: (id) => api.get(`/projects/${id}`),
  create: (data) => api.post('/projects', data),
  update: (id, data) => api.patch(`/projects/${id}`, data),
  remove: (id) => api.delete(`/projects/${id}`),
  activity: (limit = 15) => api.get('/projects/activity', { params:
     { limit } }),
  report: (start, end, projectId = null) => api.get('/projects/report', {
    params: { start, end, ...(projectId ? { project_id: projectId } : {}) },
     responseType: 'blob',
  }),

  uploadSourceFiles: (id, files) => {
    const form = new FormData()
    files.forEach((f) => form.append('files', f))
    return api.post(`/projects/${id}/source-files`, form, {
      headers: { 'Content-Type': 'multipart/form-data' },
      timeout: 120000,
    })
  },
  deleteSourceFile: (id, fileId) => api.delete(`/projects/${id}/source-
     files/${fileId}`),

  team: (id) => api.get(`/projects/${id}/assignments`),
  candidates: (id) => api.get(`/projects/${id}/candidates`),
  assignProjectManager: (id, userId) =>
     api.post(`/projects/${id}/assignments`, { user_id: userId }),
  removeProjectManager: (id, userId) =>
     api.delete(`/projects/${id}/assignments/${userId}`),

  taskGroups: (id) => api.get(`/projects/${id}/task-groups`),
  createTaskGroup: (id, data) => api.post(`/projects/${id}/task-groups`,
     data),
  updateTaskGroup: (id, groupId, data) => api.patch(`/projects/${id}/task-
     groups/${groupId}`, data),
  deleteTaskGroup: (id, groupId) => api.delete(`/projects/${id}/task-
     groups/${groupId}`),

  tasks: (id, params = {}) => api.get(`/projects/${id}/tasks`, { params }),
  createTask: (id, data) => api.post(`/projects/${id}/tasks`, data),

  tree: (id) => api.get(`/projects/${id}/tree`),
  gantt: (id, version) => api.get(`/projects/${id}/gantt`, { params: version
     ? { version } : {} }),
  ganttVersions: (id) => api.get(`/projects/${id}/gantt/versions`),
  reevaluateGantt: (id) => api.post(`/projects/${id}/gantt/reevaluate`),

  evaluate: (id) => api.post(`/projects/${id}/evaluate`),
  evaluateStatus: (id) => api.get(`/projects/${id}/evaluate/status`),
  suggestManagers: (id) => api.post(`/projects/${id}/suggest-managers`),
  suggestAssignees: (id, title, description) =>
```

```
    api.post(`/projects/${id}/tasks/suggest-assignees`, { title, description
      }),
  suggestTasks: (id, groupId) => api.post(`/projects/${id}/task-groups/$
    {groupId}/suggest-tasks`),
  refreshProgressReport: (id) => api.post(`/projects/${id}/progress-
    report`),

  document: (id, kind) => api.get(`/projects/${id}/documents/${kind}`),
  saveProjectDoc: (id, content) => api.put(`/projects/${id}/documents/doc`,
    { content }),
}

export default projectsApi
```

frontend/src/api/tasks.js

```
import api from './client'

export const tasksApi = {
  mine: (includeClosed = false) => api.get('/tasks/mine', { params:
    { include_closed: includeClosed } }),
  get: (id) => api.get(`/tasks/${id}`),
  update: (id, data) => api.patch(`/tasks/${id}`, data),
  assign: (id, userId) => api.patch(`/tasks/${id}/assign`,
    { assigned_to_user_id: userId }),
  updateStatus: (id, status) => api.patch(`/tasks/${id}/status`,
    { status }),
  remove: (id) => api.delete(`/tasks/${id}`),
  progressDoc: (id) => api.get(`/tasks/${id}/progress-doc`),
  discussions: (id) => api.get(`/tasks/${id}/discussions`),
  postDiscussion: (id, message, parentId = null) =>
    api.post(`/tasks/${id}/discussions`, { message, parent_id: parentId }),
}

export default tasksApi
```

frontend/src/api/notifications.js

```
import api from './client'

export const notificationsApi = {
  list: (params = {}) => api.get('/notifications', { params }),
  unreadCount: () => api.get('/notifications/unread-count'),
  markRead: (id) => api.patch(`/notifications/${id}/read`),
  markAllRead: () => api.post('/notifications/read-all'),
}

export default notificationsApi
```

frontend/src/api/settings.js

```
import api from './client'

export const settingsApi = {
  github: () => api.get('/admin/settings/github'),
  saveGithub: (data) => api.put('/admin/settings/github', data),
  testGithub: () => api.post('/admin/settings/github/test'),
  mail: () => api.get('/admin/settings/mail'),
  saveMail: (data) => api.put('/admin/settings/mail', data),
  testMail: () => api.post('/admin/settings/mail/test'),
}

export default settingsApi
```

frontend/src/api/workforce.js

```
import api from './client'

export const workforceApi = {
```

```
  catalogue: () => api.get('/admin/workforce'),
  list: (tab) => api.get(`/admin/workforce/${tab}`),
  create: (tab, data) => api.post(`/admin/workforce/${tab}`, data),
  update: (tab, id, data) => api.put(`/admin/workforce/${tab}/${id}`, data),
  remove: (tab, id) => api.delete(`/admin/workforce/${tab}/${id}`),
  template: (tab) => api.get(`/admin/workforce/${tab}/template`,
     { responseType: 'blob' }),
  bulk: (tab, file) => {
    const form = new FormData()
    form.append('file', file)
    return api.post(`/admin/workforce/${tab}/bulk`, form, { headers:
     { 'Content-Type': 'multipart/form-data' } })
  },
}

export default workforceApi
```

frontend/src/api/admin.js

```
import api from './client'

export const adminApi = {
  getTables: () => api.get('/admin/tables'),

  getTableRecords: (tableName, params = {}) =>
    api.get(`/admin/tables/${tableName}`, { params }),

  createRecord: (tableName, data) =>
    api.post(`/admin/tables/${tableName}`, data),

  updateRecord: (tableName, recordId, data) =>
    api.put(`/admin/tables/${tableName}/${recordId}`, data),

  deleteRecord: (tableName, recordId) =>
    api.delete(`/admin/tables/${tableName}/${recordId}`),
}

export default adminApi
```

frontend/src/api/services.js

```
import api from './client'

export const servicesApi = {
  getAllHealth: () => api.get('/services/health/all'),
  getApiHealth: () => api.get('/services/health/api'),
  getDbHealth: () => api.get('/services/health/database'),
  getRedisHealth: () => api.get('/services/health/redis'),
  getCeleryWorkerHealth: () => api.get('/services/health/celery-worker'),
  getCeleryBeatHealth: () => api.get('/services/health/celery-beat'),
  getMailpitHealth: () => api.get('/services/health/mailpit'),
  getMinioHealth: () => api.get('/services/health/minio'),
  getLlmHealth: () => api.get('/services/health/llm'),
  recentJobs: (limit = 30) => api.get('/services/jobs/recent', { params:
     { limit } }),
  getWsStatus: () => api.get('/services/ws/status'),

  sendMail: (to, subject, body) =>
    api.post('/services/mail/send', { to, subject, body }),

  uploadFile: (file) => {
    const formData = new FormData()
    formData.append('file', file)
    return api.post('/services/minio/upload', formData, {
      headers: { 'Content-Type': 'multipart/form-data' },
    })
  },
```

```
  llmChat: (message, model) =>
    api.post('/services/llm/chat', { message, model }),

  getTaskResult: (taskId) =>
    api.get(`/services/task/${taskId}/result`),
}

export default servicesApi
```

frontend/src/lib/jobs.js

```
import { servicesApi } from '@/api/services'

export function waitForJob(jobId, { interval = 1500, timeout = 180000 } =
    {}) {
  const started = Date.now()
  return new Promise((resolve, reject) => {
    const tick = async () => {
      try {
        const { data } = await servicesApi.getTaskResult(jobId)
        if (data.status === 'SUCCESS') return resolve(data.result)
        if (data.status === 'FAILURE') return reject(new Error(data.error ||
    'The job failed'))
        if (Date.now() - started > timeout) {
          return reject(new Error('Still running after 3 minutes — is the
    Celery worker up?'))
        }
        setTimeout(tick, interval)
      } catch (err) {
        reject(err)
      }
    }
    tick()
  })
}
```

frontend/src/lib/project.js

```
export function apiError(err, fallback = 'Something went wrong') {
  const data = err?.response?.data
  const first = data?.errors?.[0]
  if (first) {
    const field = (first.field || '').replace(/^body → /, '')
    return `${field ? `${field}: ` : ''}${(first.message || 'invalid
     value').replace(/^Value error, /, '')}`
  }
  if (typeof data?.detail === 'string') return data.detail
  return fallback
}

export function formatDate(value) {
  if (!value) return '—'
  return new Date(value).toLocaleDateString(undefined, { day: 'numeric',
     month: 'short', year: 'numeric' })
}

export function formatDateTime(value) {
  if (!value) return '—'
  return new Date(value).toLocaleString(undefined, { day: 'numeric', month:
     'short', hour: 'numeric', minute: '2-digit' })
}

export function initials(name) {
  if (!name) return '?'
  return name.split(' ').map((w) => w[0]).join('').slice(0, 2).toUpperCase()
}

export function isOverdue(task) {
```

```
  if (!task?.due_date || ['Completed', 'Cancelled'].includes(task.status))
     return false
  return task.status === 'Overdue' || new Date(task.due_date) < new Date(new
     Date().toDateString())
}

export const STATUS_TONE = {
  Pending: 'bg-muted-foreground',
  'In Progress': 'bg-chart-2',
  'In Review': 'bg-chart-4',
  Completed: 'bg-primary',
  Blocked: 'bg-destructive',
  Overdue: 'bg-destructive',
  Cancelled: 'bg-border',
}

export function statusTone(status, overdue = false) {
  return overdue ? STATUS_TONE.Overdue : STATUS_TONE[status] ||
     STATUS_TONE.Pending
}

export function saveBlob(blob, filename) {
  const url = URL.createObjectURL(blob)
  const a = document.createElement('a')
  a.href = url
  a.download = filename
  document.body.appendChild(a)
  a.click()
  a.remove()
  URL.revokeObjectURL(url)
}

export async function blobApiError(err, fallback) {
  const data = err.response?.data
  if (data instanceof Blob) {
    try {
      const detail = JSON.parse(await data.text()).detail
      if (typeof detail === 'string') return detail
    } catch
  }
  return apiError(err, fallback)
}
```

4.1.11 Frontend: Layouts and Shared Components

`frontend/src/layouts/Sidebar.jsx`

```
import { NavLink, useLocation } from 'react-router-dom'
import {
  Database,
  Users,
  Activity,
  Settings2,
  Shield,
  LogOut,
  Settings,
} from 'lucide-react'
import {
  Sidebar,
  SidebarContent,
  SidebarFooter,
  SidebarGroup,
  SidebarGroupLabel,
  SidebarGroupContent,
  SidebarHeader,
  SidebarMenu,
  SidebarMenuButton,
  SidebarMenuItem,
```

```
  SidebarRail,
  SidebarSeparator,
} from '@/components/ui/sidebar'
import { Avatar, AvatarFallback } from '@/components/ui/avatar'
import { useAuth } from '@/contexts/AuthContext'

const ADMIN_NAV = [
  { key: 'control-panel', label: 'Control Panel', path: '/admin/control-
     panel', icon: Database },
  { key: 'workforce',     label: 'Workforce',      path: '/admin/workforce',
     icon: Users },
  { key: 'monitor',       label: 'Monitor',         path: '/admin/monitor',
     icon: Activity },
  { key: 'configuration', label: 'Configuration',   path:
     '/admin/configuration', icon: Settings2 },
]

function getInitials(name) {
  if (!name) return 'A'
  return name
    .split(' ')
    .map((w) => w[0])
    .join('')
    .slice(0, 2)
    .toUpperCase()
}

export default function AdminSidebar() {
  const location = useLocation()
  const { user, logout } = useAuth()

  const userName = user?.name || user?.username || 'Admin'

  return (
    <Sidebar collapsible="icon">
      <SidebarHeader>
        <SidebarMenu>
          <SidebarMenuItem>
            <SidebarMenuButton size="lg" className="pointer-events-none
    select-none">
              <div className="flex aspect-square size-8 items-center
    justify-center rounded-lg bg-primary text-primary-foreground text-sm
    font-bold">
                T
              </div>
              <div className="grid flex-1 text-left text-xs leading-tight">
                <span className="text-sm font-semibold tracking-
    tight">TRACE</span>
                <span className="text-[10px] text-muted-foreground flex
    items-center gap-1">
                  <Shield className="size-2.5" /> Admin
                </span>
              </div>
            </SidebarMenuButton>
          </SidebarMenuItem>
        </SidebarMenu>
      </SidebarHeader>

      <SidebarSeparator />

      <SidebarContent>
        <SidebarGroup>
          <SidebarGroupLabel>Administration</SidebarGroupLabel>
          <SidebarGroupContent>
            <SidebarMenu>
              {ADMIN_NAV.map((item) => {
                const isActive = location.pathname.startsWith(item.path)
                const Icon = item.icon
```

```
                return (
                  <SidebarMenuItem key={item.key}>
                    <SidebarMenuButton
                      render={<NavLink to={item.path} />}
                      isActive={isActive}
                      tooltip={item.label}
                    >
                      <Icon />
                      <span>{item.label}</span>
                    </SidebarMenuButton>
                  </SidebarMenuItem>
                )
              })}
            </SidebarMenu>
          </SidebarGroupContent>
        </SidebarGroup>
      </SidebarContent>

      <SidebarSeparator />

      <SidebarFooter>
        <SidebarMenu>
          <SidebarMenuItem>
            <SidebarMenuButton
              render={<NavLink to="/admin/settings" />}
              isActive={location.pathname === '/admin/settings'}
              tooltip="Profile"
            >
              <Settings />
              <span>Profile</span>
            </SidebarMenuButton>
          </SidebarMenuItem>

          <SidebarMenuItem>
            <SidebarMenuButton size="lg" onClick={logout} tooltip="Logout">
              <Avatar className="size-6 rounded-lg">
                <AvatarFallback className="rounded-lg text-xs bg-primary/20
    text-primary">
                  {getInitials(userName)}
                </AvatarFallback>
              </Avatar>
              <div className="grid flex-1 text-left text-xs leading-tight">
                <span className="truncate font-medium">{userName}</span>
                <span className="truncate text-muted-foreground text-
    [10px]">Admin</span>
              </div>
              <LogOut className="size-4 ml-auto text-muted-foreground" />
            </SidebarMenuButton>
          </SidebarMenuItem>
        </SidebarMenu>
      </SidebarFooter>

      <SidebarRail />
    </Sidebar>
  )
}
```

frontend/src/layouts/AdminLayout.jsx

```
import { Outlet } from 'react-router-dom'
import { SidebarProvider, SidebarInset } from '@/components/ui/sidebar'
import AdminSidebar from './Sidebar'
import AdminHeader from './AdminHeader'

export default function AdminLayout() {
  return (
    <SidebarProvider defaultOpen={true}>
      <AdminSidebar />
```

```
      <SidebarInset>
        <AdminHeader />
        <main className="flex-1 p-6 overflow-auto">
          <Outlet />
        </main>
      </SidebarInset>
    </SidebarProvider>
  )
}
```

frontend/src/layouts/AdminHeader.jsx

```
import { useState } from 'react'
import { Sun, Moon, MonitorCheck } from 'lucide-react'
import { SidebarTrigger } from '@/components/ui/sidebar'
import { Button } from '@/components/ui/button'
import { Separator } from '@/components/ui/separator'
import NotificationBell from '@/components/NotificationBell'
import { useTheme } from '@/contexts/ThemeContext'
import ServiceMonitor from '@/components/ServiceMonitor'

export default function AdminHeader() {
  const [monitorOpen, setMonitorOpen] = useState(false)
  const { mode, toggleMode } = useTheme()

  return (
    <>
      <header
        id="admin-header"
        className="sticky top-0 z-30 flex h-14 shrink-0 items-center gap-2
    border-b bg-background px-4"
      >
        <SidebarTrigger className="-ml-1" />
        <Separator orientation="vertical" className="mr-2 h-4" />

        <span className="text-sm font-medium text-muted-foreground">
          Admin Panel
        </span>

        <div className="ml-auto flex items-center gap-1">
        <NotificationBell id="admin-btn-notifications" />

          <Button
            id="admin-btn-monitor"
            variant="ghost"
            size="icon-sm"
            onClick={() => setMonitorOpen(true)}
            title="Service Monitor"
          >
            <MonitorCheck />
            <span className="sr-only">Service Monitor</span>
          </Button>

          <Button
            id="admin-btn-theme"
            variant="ghost"
            size="icon-sm"
            onClick={toggleMode}
            title={mode === 'dark' ? 'Switch to light mode' : 'Switch to
    dark mode'}
          >
            {mode === 'dark' ? <Sun /> : <Moon />}
            <span className="sr-only">Toggle theme</span>
          </Button>
        </div>
      </header>
```

```
      <ServiceMonitor open={monitorOpen} onClose={() =>
     setMonitorOpen(false)} />
    </>
  )
}
```

frontend/src/layouts/ManagerLayout.jsx

```
import { Outlet } from 'react-router-dom'
import ManagerHeader from './ManagerHeader'

export default function ManagerLayout() {
  return (
    <div className="min-h-screen flex flex-col bg-background">
      <ManagerHeader />
      <main className="flex-1 p-6 overflow-auto max-w-[1600px] mx-auto w-
     full">
        <Outlet />
      </main>
    </div>
  )
}
```

frontend/src/layouts/ManagerHeader.jsx

```
import { useNavigate } from 'react-router-dom'
import { Sun, Moon, ChevronDown, LogOut, Settings } from 'lucide-react'
import { Button } from '@/components/ui/button'
import {
  DropdownMenu,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuSeparator,
  DropdownMenuTrigger,
} from '@/components/ui/dropdown-menu'
import { Avatar, AvatarFallback } from '@/components/ui/avatar'
import ProjectSelector from '@/components/ProjectSelector'
import NotificationBell from '@/components/NotificationBell'
import { useTheme } from '@/contexts/ThemeContext'
import { useAuth } from '@/contexts/AuthContext'

function getInitials(name) {
  if (!name) return 'M'
  return name.split(' ').map((w) => w[0]).join('').slice(0, 2).toUpperCase()
}

export default function ManagerHeader() {
  const { mode, toggleMode } = useTheme()
  const { user, logout } = useAuth()
  const navigate = useNavigate()

  const userName = user?.name || user?.username || 'Manager'

  return (
    <header
      id="manager-header"
      className="sticky top-0 z-30 flex h-14 shrink-0 items-center gap-3
     border-b bg-background px-6"
    >
      <div className="flex items-center gap-2 mr-4">
        <div className="flex size-7 items-center justify-center rounded-md
     bg-primary text-primary-foreground text-xs font-bold">
          T
        </div>
        <span className="font-semibold tracking-tight text-sm">TRACE</span>
      </div>

      <ProjectSelector base="/manager" />
```

```
      <div className="ml-auto flex items-center gap-1">
        <NotificationBell id="manager-btn-notifications" />

        <Button
          id="manager-btn-theme"
          variant="ghost"
          size="icon-sm"
          onClick={toggleMode}
          title={mode === 'dark' ? 'Light mode' : 'Dark mode'}
        >
          {mode === 'dark' ? <Sun /> : <Moon />}
          <span className="sr-only">Toggle theme</span>
        </Button>

        <DropdownMenu>
          <DropdownMenuTrigger render={<Button id="manager-user-menu"
    variant="ghost" size="sm" className="gap-2 px-2" />}>
            <Avatar className="size-6 rounded-full">
              <AvatarFallback className="text-xs bg-primary/20 text-
    primary">
                {getInitials(userName)}
              </AvatarFallback>
            </Avatar>
            <span className="text-xs font-medium hidden
    sm:inline">{userName}</span>
            <ChevronDown className="size-3 text-muted-foreground" />
          </DropdownMenuTrigger>
          <DropdownMenuContent align="end" className="w-40">
            <DropdownMenuItem onClick={() => navigate('/manager/settings')}>
              <Settings className="size-3.5" />
              Profile
            </DropdownMenuItem>
            <DropdownMenuSeparator />
            <DropdownMenuItem
              className="text-destructive focus:text-destructive"
              onClick={() => { logout(); navigate('/login', { replace:
    true }) }}
            >
              <LogOut className="size-3.5" />
              Sign out
            </DropdownMenuItem>
          </DropdownMenuContent>
        </DropdownMenu>
      </div>
    </header>
  )
}
```

frontend/src/layouts/EmployeeLayout.jsx

```
import { Outlet } from 'react-router-dom'
import EmployeeHeader from './EmployeeHeader'

export default function EmployeeLayout() {
  return (
    <div className="min-h-screen flex flex-col bg-background">
      <EmployeeHeader />
      <main className="flex-1 p-6 overflow-auto max-w-[1600px] mx-auto w-
     full">
        <Outlet />
      </main>
    </div>
  )
}
```

frontend/src/layouts/EmployeeHeader.jsx

```
import { useNavigate } from 'react-router-dom'
import { Sun, Moon, ChevronDown, LogOut, Settings } from 'lucide-react'
import { Button } from '@/components/ui/button'
import {
  DropdownMenu,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuSeparator,
  DropdownMenuTrigger,
} from '@/components/ui/dropdown-menu'
import { Avatar, AvatarFallback } from '@/components/ui/avatar'
import ProjectSelector from '@/components/ProjectSelector'
import NotificationBell from '@/components/NotificationBell'
import { useTheme } from '@/contexts/ThemeContext'
import { useAuth } from '@/contexts/AuthContext'

function getInitials(name) {
  if (!name) return 'E'
  return name.split(' ').map((w) => w[0]).join('').slice(0, 2).toUpperCase()
}

export default function EmployeeHeader() {
  const { mode, toggleMode } = useTheme()
  const { user, logout } = useAuth()
  const navigate = useNavigate()

  const userName = user?.name || user?.username || 'Employee'

  return (
    <header
      id="employee-header"
      className="sticky top-0 z-30 flex h-14 shrink-0 items-center gap-3
    border-b bg-background px-6"
    >
      <div className="flex items-center gap-2 mr-4">
        <div className="flex size-7 items-center justify-center rounded-md
    bg-primary text-primary-foreground text-xs font-bold">
          T
        </div>
        <span className="font-semibold tracking-tight text-sm">TRACE</span>
      </div>

      <ProjectSelector base="/employee" />

      <div className="ml-auto flex items-center gap-1">
        <NotificationBell id="employee-btn-notifications" />

        <Button
          id="employee-btn-theme"
          variant="ghost"
          size="icon-sm"
          onClick={toggleMode}
          title={mode === 'dark' ? 'Light mode' : 'Dark mode'}
        >
          {mode === 'dark' ? <Sun /> : <Moon />}
          <span className="sr-only">Toggle theme</span>
        </Button>

        <DropdownMenu>
          <DropdownMenuTrigger render={<Button id="employee-user-menu"
    variant="ghost" size="sm" className="gap-2 px-2" />}>
            <Avatar className="size-6 rounded-full">
              <AvatarFallback className="text-xs bg-primary/20 text-
    primary">
                {getInitials(userName)}
              </AvatarFallback>
```

```
              </Avatar>
              <span className="text-xs font-medium hidden
    sm:inline">{userName}</span>
              <ChevronDown className="size-3 text-muted-foreground" />
            </DropdownMenuTrigger>
            <DropdownMenuContent align="end" className="w-40">
              <DropdownMenuItem onClick={() =>
    navigate('/employee/settings')}>
              <Settings className="size-3.5" />
              Profile
            </DropdownMenuItem>
            <DropdownMenuSeparator />
            <DropdownMenuItem
              className="text-destructive focus:text-destructive"
              onClick={() => { logout(); navigate('/login', { replace:
    true }) }}
            >
              <LogOut className="size-3.5" />
              Sign out
            </DropdownMenuItem>
          </DropdownMenuContent>
        </DropdownMenu>
      </div>
    </header>
  )
}
```

frontend/src/components/ProtectedRoute.jsx

```
import { Navigate, useLocation } from 'react-router-dom'
import { useAuth } from '@/contexts/AuthContext'

export default function ProtectedRoute({ children }) {
  const { isAuthenticated, loading } = useAuth()
  const location = useLocation()

  const hasStoredAuth =
    !!localStorage.getItem('trace_access_token') &&
    !!localStorage.getItem('trace_user')

  if (loading && !hasStoredAuth) {
    return (
      <div className="min-h-screen flex items-center justify-center bg-
    background">
        <div className="flex flex-col items-center gap-3">
          <div className="w-10 h-10 border-2 border-violet-400/20 border-t-
    violet-400/70 rounded-full animate-spin" />
          <span className="text-[13px] text-white/30">Loading…</span>
        </div>
      </div>
    )
  }

  if (!loading && !isAuthenticated) {
    return <Navigate to="/login" state={{ from: location }} replace />
  }

  return children
}
```

frontend/src/components/RoleRoute.jsx

```
import { Navigate, Outlet } from 'react-router-dom'
import { useAuth } from '@/contexts/AuthContext'

export default function RoleRoute({ role, children }) {
  const { user, isAuthenticated, loading } = useAuth()
```

```
  if (loading) return null

  if (!isAuthenticated) {
    return <Navigate to="/login" replace />
  }

  const allowed = Array.isArray(role) ? role : [role]
  const userRoles = user?.roles?.map((r) => r.role_name || r) ?? []

  const hasRole = allowed.some((r) => userRoles.includes(r))

  if (!hasRole) {
    return <Navigate to="/403" replace />
  }

  return children ?? <Outlet />
}
```

frontend/src/components/GuestOnlyRoute.jsx

```
import { Navigate, Outlet } from 'react-router-dom'
import { useAuth } from '@/contexts/AuthContext'

export function getRoleBase(user) {
  const roles = user?.roles?.map((r) => r.role_name || r) ?? []
  if (roles.includes('Admin')) return '/admin'
  if (roles.includes('Manager')) return '/manager'
  if (roles.includes('Employee')) return '/employee'
  return '/login'
}

export default function GuestOnlyRoute() {
  const { user, isAuthenticated, loading } = useAuth()

  if (loading) return null

  if (isAuthenticated) {
    return <Navigate to={getRoleBase(user)} replace />
  }

  return <Outlet />
}
```

frontend/src/components/NotificationBell.jsx

```
import { useCallback, useEffect, useState } from 'react'
import { useNavigate } from 'react-router-dom'
import { Bell, CheckCheck } from 'lucide-react'
import { toast } from 'sonner'
import { Button } from '@/components/ui/button'
import { Popover, PopoverContent, PopoverTrigger } from
    '@/components/ui/popover'
import { ScrollArea } from '@/components/ui/scroll-area'
import { notificationsApi } from '@/api/notifications'
import { useLiveEvents } from '@/contexts/LiveContext'
import { formatDateTime } from '@/lib/project'

export default function NotificationBell({ id }) {
  const navigate = useNavigate()
  const [open, setOpen] = useState(false)
  const [inbox, setInbox] = useState({ items: [], unread: 0 })

  const load = useCallback(async () => {
    try {
      const { data } = await notificationsApi.list({ limit: 20 })
      setInbox(data)
      return data
    } catch {
```

```
    return null
  }
}, [])

useEffect(() => { load() }, [load])

useLiveEvents(async (event) => {
  if (event.type !== 'notification') return
  const data = await load()
  const newest = data?.items[0]
  if (newest && !newest.is_read) {
    toast(newest.title, {
      description: newest.body,
      action: newest.link ? { label: 'Open', onClick: () =>
   navigate(newest.link) } : undefined,
    })
  }
})

const openNote = async (note) => {
  setOpen(false)
  if (!note.is_read) {
    await notificationsApi.markRead(note.id).catch(() => {})
    load()
  }
  if (note.link) navigate(note.link)
}

const readAll = async () => {
  await notificationsApi.markAllRead().catch(() => {})
  load()
}

return (
  <Popover open={open} onOpenChange={setOpen}>
    <PopoverTrigger render={<Button id={id} variant="ghost" size="icon-sm"
   className="relative" />}>
      <Bell />
      {inbox.unread > 0 && (
        <span className="absolute -top-0.5 -right-0.5 flex min-w-4 h-4
   items-center justify-center rounded-full bg-primary px-1 text-[10px]
   font-medium text-primary-foreground">
          {inbox.unread > 9 ? '9+' : inbox.unread}
        </span>
      )}
      <span className="sr-only">Notifications</span>
    </PopoverTrigger>
    <PopoverContent align="end" className="w-80 gap-2 p-0">
      <div className="flex items-center justify-between border-b px-3 py-
   2">
        <span className="text-sm font-medium">Notifications</span>
        <Button variant="ghost" size="sm" onClick={readAll}
   disabled={inbox.unread === 0}>
          <CheckCheck /> Mark all read
        </Button>
      </div>
      {inbox.items.length === 0 ? (
        <p className="px-3 pb-4 pt-2 text-sm text-muted-
   foreground">Nothing yet.</p>
      ) : (
        <ScrollArea className="max-h-96">
          <ul className="pb-1">
            {inbox.items.map((n) => (
              <li key={n.id}>
                <button type="button" onClick={() => openNote(n)}
                  className="flex w-full gap-2 px-3 py-2 text-left
   hover:bg-muted/60">
```

```
                  <span className={`mt-1.5 size-2 shrink-0 rounded-full $
    {n.is_read ? 'bg-transparent' : 'bg-primary'}`} />
                  <span className="min-w-0 space-y-0.5">
                    <span className="block truncate text-sm font-
    medium">{n.title}</span>
                    {n.body && <span className="line-clamp-2 block text-xs
    text-muted-foreground">{n.body}</span>}
                    <span className="block text-[11px] text-muted-
    foreground">{formatDateTime(n.created_at)}</span>
                  </span>
                </button>
              </li>
            ))}
          </ul>
        </ScrollArea>
      )}
    </PopoverContent>
  </Popover>
 )
}
```

frontend/src/components/ProjectSelector.jsx

```
import { useEffect, useState } from 'react'
import { useLocation, useMatch, useNavigate } from 'react-router-dom'
import { Check, ChevronDown, FolderKanban, LayoutGrid } from 'lucide-react'
import { Button } from '@/components/ui/button'
import {
  DropdownMenu,
  DropdownMenuContent,
  DropdownMenuGroup,
  DropdownMenuItem,
  DropdownMenuLabel,
  DropdownMenuSeparator,
  DropdownMenuTrigger,
} from '@/components/ui/dropdown-menu'
import { projectsApi } from '@/api/projects'

export default function ProjectSelector({ base }) {
  const navigate = useNavigate()
  const location = useLocation()
  const match = useMatch(`${base}/projects/:id`)
  const selectedId = match && /^\d+$/.test(match.params.id) ?
     Number(match.params.id) : null
  const [projects, setProjects] = useState([])

  useEffect(() => {
    projectsApi.list().then((res) => setProjects(res.data)).catch(() =>
     setProjects([]))
  }, [location.pathname])

  const selected = projects.find((p) => p.id === selectedId)

  return (
    <DropdownMenu>
      <DropdownMenuTrigger
        render={
          <Button
            id={`${base.slice(1)}-project-selector`}
            variant="outline"
            size="sm"
            className="gap-2 min-w-[180px] max-w-[280px] justify-between"
          />
        }
      >
        <span className="flex items-center gap-1.5 truncate">
          {selected ? <FolderKanban className="size-3.5 shrink-0" /> :
     <LayoutGrid className="size-3.5 shrink-0" />}
```

```
          <span className="truncate">{selected ? selected.name : 'All
    projects'}</span>
        </span>
        <ChevronDown className="size-3.5 shrink-0 text-muted-foreground" />
      </DropdownMenuTrigger>
      <DropdownMenuContent align="start" className="w-72">
        <DropdownMenuItem onClick={() => navigate(base)}>
          <LayoutGrid className="size-3.5" />
          <span className="flex-1">All projects</span>
          {!selectedId && <Check className="size-3.5" />}
        </DropdownMenuItem>
        {projects.length > 0 && <DropdownMenuSeparator />}
        {projects.length > 0 && (
          <DropdownMenuGroup>
            <DropdownMenuLabel>Projects</DropdownMenuLabel>
            {projects.map((p) => (
              <DropdownMenuItem key={p.id} onClick={() => navigate(`$
    {base}/projects/${p.id}`)}>
                <FolderKanban className="size-3.5" />
                <span className="flex-1 truncate">{p.name}</span>
                {p.my_role && (
                  <span className="text-[10px] text-muted-foreground">
                    {p.my_role === 'Project Manager' ? 'PM' : 'Member'}
                  </span>
                )}
                {p.id === selectedId && <Check className="size-3.5" />}
              </DropdownMenuItem>
            ))}
          </DropdownMenuGroup>
        )}
      </DropdownMenuContent>
    </DropdownMenu>
  )
}
```

frontend/src/components/Markdown.jsx

```
import ReactMarkdown from 'react-markdown'
import remarkGfm from 'remark-gfm'

const COMPONENTS = {
  h1: (p) => <h1 className="mb-3 mt-2 text-xl font-semibold tracking-tight"
     {...p} />,
  h2: (p) => <h2 className="mb-2 mt-6 border-b pb-1 text-lg font-semibold"
     {...p} />,
  h3: (p) => <h3 className="mb-2 mt-4 font-semibold" {...p} />,
  p: (p) => <p className="my-2 leading-relaxed" {...p} />,
  ul: ({ className, ...p }) => (
    <ul className={`my-2 space-y-1 ${className?.includes('contains-task-
     list') ? 'list-none pl-1' : 'list-disc pl-5'}`} {...p} />
  ),
  ol: (p) => <ol className="my-2 list-decimal space-y-1 pl-5" {...p} />,
  input: (p) => <input {...p} disabled className="mr-2 translate-y-0.5
     accent-primary" />,
  a: (p) => <a className="text-primary underline-offset-2 hover:underline"
     target="_blank" rel="noopener noreferrer" {...p} />,
  code: (p) => <code className="rounded bg-muted px-1 py-0.5 font-mono text-
     xs" {...p} />,
  pre: (p) => <pre className="my-3 overflow-x-auto rounded-lg bg-muted p-3
     text-xs" {...p} />,
  blockquote: (p) => <blockquote className="my-3 border-l-2 pl-3 text-muted-
     foreground" {...p} />,
  table: (p) => <div className="my-3 overflow-x-auto"><table className="w-
     full text-left text-xs" {...p} /></div>,
  th: (p) => <th className="border-b px-2 py-1 font-medium" {...p} />,
  td: (p) => <td className="border-b px-2 py-1" {...p} />,
  hr: () => <hr className="my-4" />,
}
```

```
export default function Markdown({ children }) {
  return (
    <div className="min-w-0 text-sm text-foreground">
      <ReactMarkdown remarkPlugins={[remarkGfm]}
     components={COMPONENTS}>{children || ''}</ReactMarkdown>
    </div>
  )
}
```

frontend/src/components/ServiceMonitor.jsx

```
import { useState, useEffect, useRef, useCallback } from 'react'
import { X, RefreshCw, Send, Upload, ExternalLink, CheckCircle, AlertCircle,
     AlertTriangle, MessageSquare, Mail, Database, Cloud, Brain, Clock,
     Server, MemoryStick, Activity } from 'lucide-react'
import { Button } from '@/components/ui/button'
import { Input } from '@/components/ui/input'
import { Textarea } from '@/components/ui/textarea'
import { Badge } from '@/components/ui/badge'
import { Card, CardContent, CardHeader, CardTitle } from
     '@/components/ui/card'
import { Separator } from '@/components/ui/separator'
import servicesApi from '@/api/services'

function StatusBadge({ status }) {
  const map = {
    healthy: { variant: 'default', label: 'Healthy', Icon: CheckCircle },
    unhealthy: { variant: 'destructive', label: 'Unhealthy', Icon:
     AlertCircle },
    configured: { variant: 'secondary', label: 'Configured', Icon:
     CheckCircle },
    not_configured: { variant: 'outline', label: 'Not Configured', Icon:
     AlertTriangle },
    loading: { variant: 'secondary', label: 'Checking…', Icon: RefreshCw },
  }
  const s = map[status] || map.loading
  return (
    <Badge variant={s.variant} className="gap-1">
      <s.Icon className={`size-3 ${status === 'loading' ? 'animate-spin' :
     ''}`} />
      {s.label}
    </Badge>
  )
}

function HealthCard({ title, Icon, status, children, details }) {
  return (
    <Card>
      <CardHeader className="flex flex-row items-center justify-between
     space-y-0 pb-2">
        <div className="flex items-center gap-2">
          <Icon className="size-4 text-muted-foreground" />
          <CardTitle className="text-sm font-medium">{title}</CardTitle>
        </div>
        <StatusBadge status={status} />
      </CardHeader>
      <CardContent className="space-y-3">
        {details && Object.keys(details).length > 0 && (
          <div className="flex flex-wrap gap-x-4 gap-y-1 text-xs text-muted-
     foreground">
            {Object.entries(details).map(([k, v]) => (
              <span key={k}><span className="opacity-60">{k}:</span>
     {String(v)}</span>
            ))}
          </div>
        )}
        {children}
```

```
      </CardContent>
    </Card>
  )
}

function LiveSocketCard({ status }) {
  const [online, setOnline] = useState(null)
  useEffect(() => {
    servicesApi.getWsStatus().then((r) =>
     setOnline(r.data.online_users)).catch(() => setOnline(null))
  }, [])
  return (
    <HealthCard title="Live events (WebSocket)" Icon={MessageSquare}
     status={status || 'loading'}>
      <p className="text-xs text-muted-foreground">
        Notifications and discussions reach open browsers over
     <code>/api/v1/ws</code>.
        {online !== null && <> Users connected to this API: <strong
     className="text-foreground">{online}</strong>.</>}
      </p>
    </HealthCard>
  )
}

function MailCard({ data }) {
  const [form, setForm] = useState({ to: 'test@trace.com', subject: 'TRACE
     Test Mail', body: 'Hello from TRACE Service Monitor!' })
  const [sending, setSending] = useState(false)
  const [result, setResult] = useState(null)

  const send = async () => {
    setSending(true); setResult({ ok: true, msg: 'Queuing mail…' })
    try {
      const res = await servicesApi.sendMail(form.to, form.subject,
     form.body)
      const taskId = res.data.task_id
      let attempts = 0
      const poll = setInterval(async () => {
        attempts++
        try {
          const r = await servicesApi.getTaskResult(taskId)
          if (r.data.status === 'SUCCESS') { clearInterval(poll);
     setResult({ ok: true, msg: `✅ Mail sent to ${form.to}!` });
     setSending(false) }
          else if (r.data.status === 'FAILURE' || attempts > 20)
     { clearInterval(poll); setResult({ ok: false, msg: `Failed` });
     setSending(false) }
        } catch { if (attempts > 20) { clearInterval(poll);
     setSending(false) } }
      }, 1500)
    } catch (e) { setResult({ ok: false, msg: e.message });
     setSending(false) }
  }

  return (
    <HealthCard title="Mailpit (SMTP)" Icon={Mail} status={data?.status ||
     'loading'} details={data?.smtp ? { smtp: data.smtp } : {}}>
      <div className="space-y-1.5">
        <Input value={form.to} onChange={e => setForm(p => ({ ...p, to:
     e.target.value }))} placeholder="To email" className="h-7 text-xs" />
        <Input value={form.subject} onChange={e => setForm(p => ({ ...p,
     subject: e.target.value }))} placeholder="Subject" className="h-7 text-
     xs" />
        <Textarea value={form.body} onChange={e => setForm(p => ({ ...p,
     body: e.target.value }))} placeholder="Body" rows={2} className="text-
     xs" />
      </div>
      <div className="flex items-center gap-2 flex-wrap">
```

```
        <Button size="sm" variant="outline" onClick={send}
     disabled={sending}><Send className="size-3" />{sending ? 'Sending…' :
     'Send Mail'}</Button>
        <a href="http://localhost:8025" target="_blank" rel="noopener
     noreferrer"><Button size="sm" variant="outline"><ExternalLink
     className="size-3" />Mailpit UI</Button></a>
        {result && <span className={`text-xs ${result.ok ? 'text-primary' :
     'text-destructive'}`}>{result.msg}</span>}
      </div>
    </HealthCard>
  )
}

function MinioCard({ data }) {
  const [uploading, setUploading] = useState(false)
  const [result, setResult] = useState(null)
  const fileRef = useRef(null)

  const upload = async (e) => {
    const file = e.target.files?.[0]; if (!file) return
    setUploading(true); setResult({ ok: true, msg: 'Queuing upload…' })
    try {
      const res = await servicesApi.uploadFile(file)
      const taskId = res.data.task_id
      let attempts = 0
      const poll = setInterval(async () => {
        attempts++
        try {
          const r = await servicesApi.getTaskResult(taskId)
          if (r.data.status === 'SUCCESS') { clearInterval(poll);
     setResult({ ok: true, msg: `✅ Uploaded: ${r.data.result?.object_name ||
     file.name}` }); setUploading(false) }
          else if (r.data.status === 'FAILURE' || attempts > 20)
     { clearInterval(poll); setResult({ ok: false, msg: 'Failed' });
     setUploading(false) }
        } catch { if (attempts > 20) { clearInterval(poll);
     setUploading(false) } }
      }, 1500)
    } catch (err) { setResult({ ok: false, msg: err.message });
     setUploading(false) }
    if (fileRef.current) fileRef.current.value = ''
  }

  return (
    <HealthCard title="MinIO (Object Storage)" Icon={Cloud}
     status={data?.status || 'loading'} details={data?.bucket ? { bucket:
     data.bucket } : {}}>
      <div className="flex items-center gap-2 flex-wrap">
        <label><Button size="sm" variant="outline" asChild><span
     className="cursor-pointer"><Upload className="size-3" />{uploading ?
     'Uploading…' : 'Test Upload'}<input ref={fileRef} type="file"
     className="sr-only" onChange={upload} disabled={uploading}
     /></span></Button></label>
        <a href="http://localhost:9001" target="_blank" rel="noopener
     noreferrer"><Button size="sm" variant="outline"><ExternalLink
     className="size-3" />MinIO Console</Button></a>
        {result && <span className={`text-xs ${result.ok ? 'text-primary' :
     'text-destructive'}`}>{result.msg}</span>}
      </div>
    </HealthCard>
  )
}

function LlmCard({ data }) {
  const [input, setInput] = useState('')
  const [msgs, setMsgs] = useState([])
  const [loading, setLoading] = useState(false)
  const scrollRef = useRef(null)
```

```
  useEffect(() => { scrollRef.current?.scrollIntoView({ behavior:
     'smooth' }) }, [msgs])

  const send = async () => {
    if (!input.trim()) return
    const userMsg = input.trim(); setMsgs(prev => [...prev, { role: 'user',
     content: userMsg }]); setInput(''); setLoading(true)
    try {
      const res = await servicesApi.llmChat(userMsg)
      const taskId = res.data.task_id; let attempts = 0
      const poll = setInterval(async () => {
        attempts++
        try {
          const r = await servicesApi.getTaskResult(taskId)
          if (r.data.status === 'SUCCESS') {
            clearInterval(poll)
            const result = r.data.result
            setMsgs(prev => [...prev, result.status === 'success' ? { role:
     'assistant', content: result.reply, model: result.model } : { role:
     'error', content: result.error || 'LLM error' }])
            setLoading(false)
          } else if (r.data.status === 'FAILURE' || attempts > 30)
     { clearInterval(poll); setLoading(false) }
        } catch { if (attempts > 30) { clearInterval(poll);
     setLoading(false) } }
      }, 2000)
    } catch (e) { setMsgs(prev => [...prev, { role: 'error', content:
     e.message }]); setLoading(false) }
  }

  return (
    <HealthCard title="LLM (OpenAI SDK)" Icon={Brain} status={data?.status
     || 'loading'} details={{ model: data?.model || '—' }}>
      <div className="h-32 overflow-y-auto rounded-md bg-muted/50 p-2 text-
     xs space-y-2">
        {msgs.length === 0 && <p className="text-muted-foreground
     italic">Send a message to test the LLM…</p>}
        {msgs.map((m, i) => (
          <div key={i}>
            <span className="font-medium">{m.role === 'user' ? 'You' :
     m.role === 'error' ? '⚠️' : `🤖 ${m.model || 'AI'}`}: </span>
            {m.content}
          </div>
        ))}
        {loading && <div className="text-muted-foreground animate-pulse">⏳
     Thinking via Celery…</div>}
        <div ref={scrollRef} />
      </div>
      <div className="flex gap-2">
        <Input value={input} onChange={e => setInput(e.target.value)}
     onKeyDown={e => e.key === 'Enter' && send()} placeholder="Ask the LLM
     anything…" disabled={loading} className="h-8 text-xs" />
        <Button size="icon-sm" onClick={send} disabled={loading}><Send
     className="size-3" /></Button>
      </div>
    </HealthCard>
  )
}

export default function ServiceMonitor({ open, onClose }) {
  const [health, setHealth] = useState(null)
  const [loading, setLoading] = useState(false)

  const fetchHealth = useCallback(async () => {
    setLoading(true)
    try { const res = await servicesApi.getAllHealth();
     setHealth(res.data.services) } catch {}
    setLoading(false)
```

```
  }, [])

  useEffect(() => { if (open) fetchHealth() }, [open, fetchHealth])
  useEffect(() => { if (!open) return; const i = setInterval(fetchHealth,
     30000); return () => clearInterval(i) }, [open, fetchHealth])

  if (!open) return null
  const h = health || {}

  return (
    <div className="fixed inset-0 z-50 flex flex-col bg-background/95
     backdrop-blur-xl">
      <div className="flex items-center justify-between px-6 py-4 border-b">
        <div className="flex items-center gap-3">
          <Activity className="size-5 text-muted-foreground" />
          <div>
            <h1 className="text-base font-semibold">Service Monitor</h1>
            <p className="text-xs text-muted-foreground">Infrastructure
     health & testing dashboard</p>
          </div>
        </div>
        <div className="flex items-center gap-2">
          <Button variant="outline" size="sm" onClick={fetchHealth}
     disabled={loading}>
            <RefreshCw className={`size-4 ${loading ? 'animate-spin' : ''}`}
     /> Refresh
          </Button>
          <Button variant="ghost" size="icon" onClick={onClose}><X
     className="size-4" /></Button>
        </div>
      </div>

      <div className="flex-1 overflow-y-auto p-6">
        <div className="grid grid-cols-1 md:grid-cols-2 xl:grid-cols-3 gap-4
     max-w-[1400px] mx-auto">
          <HealthCard title="API Server" Icon={Server} status={h.api?.status
     || 'loading'} details={h.api ? { app: 'TRACE', uptime: '✓' } : {}} />
          <HealthCard title="PostgreSQL" Icon={Database}
     status={h.database?.status || 'loading'} details={h.database?.status ===
     'healthy' ? { version: h.database.version } : {}}>
            <a href="http://localhost:5050" target="_blank" rel="noopener
     noreferrer"><Button size="sm" variant="outline"><ExternalLink
     className="size-3" />pgAdmin 4</Button></a>
          </HealthCard>
          <HealthCard title="Redis" Icon={MemoryStick}
     status={h.redis?.status || 'loading'} details={h.redis?.version ?
     { version: h.redis.version } : {}} />
          <HealthCard title="Celery Worker" Icon={Clock}
     status={h.celery_worker?.status || 'loading'}
     details={h.celery_worker?.status === 'healthy' ? { active:
     h.celery_worker.active_tasks } : {}} />
          <HealthCard title="Celery Beat" Icon={Clock}
     status={h.celery_beat?.status || 'loading'}
     details={h.celery_beat?.task_count != null ? { tasks:
     h.celery_beat.task_count } : {}} />
          <LiveSocketCard status={h.websocket?.status} />
          <MailCard data={h.mailpit} />
          <MinioCard data={h.minio} />
          <LlmCard data={h.llm} />
        </div>
      </div>
    </div>
  )
}
```

frontend/src/components/ProjectTreeGraph.jsx

```
import { useCallback, useEffect, useMemo, useState } from 'react'
```

```
import { Link } from 'react-router-dom'
import {
  Background, Handle, Panel, Position, ReactFlow, ReactFlowProvider,
    useReactFlow,
} from '@xyflow/react'
import '@xyflow/react/dist/base.css'
import { ExternalLink, Maximize, Network, RefreshCw, X, ZoomIn, ZoomOut }
    from 'lucide-react'
import { toast } from 'sonner'
import { Avatar, AvatarFallback } from '@/components/ui/avatar'
import { Button } from '@/components/ui/button'
import { Card, CardContent, CardHeader, CardTitle } from
    '@/components/ui/card'
import { Progress } from '@/components/ui/progress'
import { Skeleton } from '@/components/ui/skeleton'
import NotYetAvailable from '@/components/project/NotYetAvailable'
import { ProjectStatusBadge, StatusLegend, TaskStatusBadge } from
    '@/components/project/status'
import { projectsApi } from '@/api/projects'
import { useTheme } from '@/contexts/ThemeContext'
import { apiError, formatDate, initials, statusTone } from '@/lib/project'

const COLUMN = 280
const GROUP_Y = 150
const TASK_Y = 290
const TASK_STEP = 92
const TASK_INDENT = 32

const hidden = '!size-1 !min-w-0 !border-0 !bg-transparent'

function ProjectNode({ data }) {
  return (
    <div className="w-64 rounded-lg border-2 border-primary bg-card p-3
    text-card-foreground shadow-sm">
      <p className="truncate font-semibold"
    title={data.name}>{data.name}</p>
      <div className="mt-2 flex items-center justify-between gap-2">
        <ProjectStatusBadge status={data.status} />
        <span className="text-xs text-muted-
    foreground">{data.completion_percentage}% complete</span>
      </div>
      <Handle type="source" position={Position.Bottom} className={hidden}
    isConnectable={false} />
    </div>
  )
}

function GroupNode({ data }) {
  return (
    <div className="w-60 overflow-hidden rounded-lg border bg-card text-
    card-foreground shadow-sm">
      <Handle type="target" position={Position.Top} className={hidden}
    isConnectable={false} />
      <div className={`h-1.5 ${statusTone(data.status)}`} />
      <div className="space-y-2 p-3">
        <p className="truncate text-sm font-medium"
    title={data.name}>{data.name}</p>
        <Progress value={data.completion_percentage} />
        <p className="text-xs text-muted-foreground">
          {data.status} · {data.tasks.length} task{data.tasks.length === 1 ?
    '' : 's'} · {data.completion_percentage}%
        </p>
      </div>
      <Handle type="source" position={Position.Bottom} className={hidden}
    isConnectable={false}
        style={{ left: 12 }} />
    </div>
  )
```

```
}

function TaskNode({ data }) {
  const { task, selected } = data
  return (
    <button type="button"
      className={`flex w-56 cursor-pointer overflow-hidden rounded-md border
     bg-card text-left text-card-foreground shadow-sm transition-colors
     hover:bg-accent ${selected ? 'ring-2 ring-ring' : ''} ${task.status ===
     'Cancelled' ? 'opacity-60' : ''}`}>
      <Handle type="target" position={Position.Left} className={hidden}
     isConnectable={false} />
      <span className={`w-1.5 shrink-0 ${statusTone(task.status,
     task.overdue)}`} />
      <span className="min-w-0 flex-1 space-y-1.5 p-2.5">
        <span className="block truncate text-sm"
     title={task.title}>{task.title}</span>
        <span className="flex items-center gap-1.5 text-xs text-muted-
     foreground">
          <Avatar className="size-5"><AvatarFallback className="text-
     [10px]">
            {task.assignee ? initials(task.assignee.name) : '–'}
          </AvatarFallback></Avatar>
          <span className="truncate">{task.assignee?.name ||
     'Unassigned'}</span>
          {task.overdue && <span className="ml-auto shrink-0 text-
     destructive">Overdue</span>}
        </span>
      </span>
    </button>
  )
}

const NODE_TYPES = { project: ProjectNode, group: GroupNode, task:
     TaskNode }

function layout(tree) {
  const nodes = []
  const edges = []
  const n = tree.groups.length
  const width = Math.max(n, 1) * COLUMN
  const edge = { type: 'smoothstep', style: { stroke: 'var(--muted-
     foreground)', strokeWidth: 1.25 } }

  nodes.push({ id: 'p', type: 'project', position: { x: width / 2 - 128, y:
     0 }, data: tree.project })
  tree.groups.forEach((g, i) => {
    const x = i * COLUMN
    nodes.push({ id: `g${g.id}`, type: 'group', position: { x, y: GROUP_Y },
     data: g })
    edges.push({ id: `p-g${g.id}`, source: 'p', target: `g$
     {g.id}`, ...edge })
    g.tasks.forEach((t, j) => {
      nodes.push({
        id: `t${t.id}`, type: 'task', position: { x: x + TASK_INDENT, y:
     TASK_Y + j * TASK_STEP },
        data: { task: t },
      })
      edges.push({ id: `g${g.id}-t${t.id}`, source: `g${g.id}`, target: `t$
     {t.id}`, ...edge })
    })
  })
  return { nodes, edges }
}

function ZoomControls() {
  const { zoomIn, zoomOut, fitView } = useReactFlow()
  return (
```

```
    <Panel position="bottom-left" className="flex flex-col gap-1">
      <Button size="icon-sm" variant="outline" aria-label="Zoom in"
     onClick={() => zoomIn()}><ZoomIn /></Button>
      <Button size="icon-sm" variant="outline" aria-label="Zoom out"
     onClick={() => zoomOut()}><ZoomOut /></Button>
      <Button size="icon-sm" variant="outline" aria-label="Fit to view"
     onClick={() => fitView({ padding: 0.15 })}><Maximize /></Button>
    </Panel>
  )
}

function TaskPanel({ task, taskPath, onClose }) {
  return (
    <Panel position="top-right" className="w-72">
      <Card size="sm" className="shadow-md">
        <CardHeader className="flex flex-row items-start justify-between
     gap-2">
          <CardTitle className="text-sm
     leading-snug">{task.title}</CardTitle>
          <Button size="icon-xs" variant="ghost" aria-label="Close"
     onClick={onClose}><X /></Button>
        </CardHeader>
        <CardContent className="space-y-2 text-sm">
          <div className="flex flex-wrap items-center gap-2">
            <TaskStatusBadge status={task.status} />
            {task.overdue && <span className="text-xs text-
     destructive">Overdue</span>}
          </div>
          <p className="text-muted-foreground">
            {task.assignee?.name || 'Unassigned'} · due
     {formatDate(task.due_date)}
          </p>
          <div className="flex flex-wrap gap-2">
            {taskPath && (
              <Button size="sm" nativeButton={false} render={<Link
     to={taskPath(task.id)} />}>Open task</Button>
            )}
            {task.github_issue_url && (
              <Button size="sm" variant="outline" nativeButton={false}
                render={<a href={task.github_issue_url} target="_blank"
     rel="noopener noreferrer" />}>
                GitHub issue <ExternalLink />
              </Button>
            )}
          </div>
        </CardContent>
      </Card>
    </Panel>
  )
}

function Graph({ tree, taskPath }) {
  const { mode } = useTheme()
  const { nodes, edges } = useMemo(() => layout(tree), [tree])
  const [selected, setSelected] = useState(null)
  const task = selected && tree.groups.flatMap((g) => g.tasks).find((t) =>
     t.id === selected)

  return (
    <ReactFlow
      nodes={nodes.map((n) => (n.id === `t${selected}` ? { ...n, data:
     { ...n.data, selected: true } } : n))}
      edges={edges}
      nodeTypes={NODE_TYPES}
      colorMode={mode}
      fitView
      fitViewOptions={{ padding: 0.15 }}
      minZoom={0.2}
```

```
      maxZoom={1.75}
      nodesDraggable={false}
      nodesConnectable={false}
      elementsSelectable={false}
      proOptions={{ hideAttribution: true }}
      onNodeClick={(_, node) => node.type === 'task' &&
     setSelected(node.data.task.id)}
      onPaneClick={() => setSelected(null)}
    >
      <Background gap={20} size={1} color="var(--border)" />
      <ZoomControls />
      {task && <TaskPanel task={task} taskPath={taskPath} onClose={() =>
     setSelected(null)} />}
    </ReactFlow>
  )
}

export default function ProjectTreeGraph({ projectId, taskPath }) {
  const [tree, setTree] = useState(null)

  const load = useCallback(async () => {
    try {
      setTree((await projectsApi.tree(projectId)).data)
    } catch (err) {
      toast.error(apiError(err, 'Could not load the project tree'))
    }
  }, [projectId])

  useEffect(() => { load() }, [load])

  if (!tree) return <Skeleton className="h-[600px]" />
  if (!tree.groups.length) {
    return (
      <NotYetAvailable icon={Network} title="Nothing to draw yet">
        The tree appears once the project has task groups.
      </NotYetAvailable>
    )
  }

  return (
    <div id="project-tree" className="space-y-3">
      <div className="flex flex-wrap items-center justify-between gap-2">
        <StatusLegend />
        <Button variant="outline" size="sm" onClick={load}><RefreshCw />
     Refresh</Button>
      </div>
      <div className="h-[600px] overflow-hidden rounded-lg border
     bg-muted/30">
        <ReactFlowProvider><Graph tree={tree} taskPath={taskPath}
     /></ReactFlowProvider>
      </div>
      <p className="text-xs text-muted-foreground">Drag to pan, scroll or
     pinch to zoom. Select a task for its details.</p>
    </div>
  )
}
```

frontend/src/components/GanttChart.jsx

```
import { useCallback, useEffect, useId, useMemo, useState } from 'react'
import {
  ChartGantt, ChevronDown, ChevronRight, ChevronsDownUp, ChevronsUpDown,
     Loader2, RefreshCw, Sparkles,
} from 'lucide-react'
import { toast } from 'sonner'
import { Badge } from '@/components/ui/badge'
import { Button } from '@/components/ui/button'
```

```
import { NativeSelect, NativeSelectOption } from '@/components/ui/native-
    select'
import { Skeleton } from '@/components/ui/skeleton'
import NotYetAvailable from '@/components/project/NotYetAvailable'
import { StatusLegend } from '@/components/project/status'
import { projectsApi } from '@/api/projects'
import { apiError, formatDate, statusTone } from '@/lib/project'
import { waitForJob } from '@/lib/jobs'

const LABEL_W = 220
const BAR_ROW = 44
const TASK_ROW = 34
const DAY = 86400000

const REASONS = {
  initial: 'Initial plan',
  task_status_change: 'A task's status changed',
  project_doc_updated: 'The project document was edited',
  manual: 'Regenerated on request',
}

const utc = (iso) => {
  const [y, m, d] = iso.split('-').map(Number)
  return Date.UTC(y, m - 1, d)
}

function scaleFor(days) {
  if (days <= 45) return 28
  if (days <= 120) return 12
  if (days <= 400) return 6
  return 3
}

function months(start, days, px) {
  const out = []
  const first = new Date(start)
  let cursor = Date.UTC(first.getUTCFullYear(), first.getUTCMonth(), 1)
  const end = start + (days - 1) * DAY
  while (cursor <= end) {
    const d = new Date(cursor)
    const next = Date.UTC(d.getUTCFullYear(), d.getUTCMonth() + 1, 1)
    const from = Math.max(cursor, start)
    const to = Math.min(next - DAY, end)
    out.push({
      key: cursor,
      label: d.toLocaleDateString(undefined, { month: 'short', year:
     'numeric', timeZone: 'UTC' }),
      left: ((from - start) / DAY) * px,
      width: ((to - from) / DAY + 1) * px,
    })
    cursor = next
  }
  return out
}

function isLate(task, generatedAt) {
  if (!task.due || ['Completed', 'Cancelled'].includes(task.status)) return
     false
  const asOf = generatedAt ? new Date(generatedAt).getTime() : Date.now()
  return task.status === 'Overdue' || utc(task.due) + DAY <= asOf
}

function Chart({ data, generatedAt, expanded, toggle }) {
  const markerId = useId()
  const [today] = useState(() => Date.now())
  const start = utc(data.range.start)
  const days = (utc(data.range.end) - start) / DAY + 1
  const px = scaleFor(days)
```

```
const width = days * px
const x = (iso) => ((utc(iso) - start) / DAY) * px

const bars = useMemo(() => [...data.bars].sort((a, b) => a.sequence -
   b.sequence), [data.bars])
const rows = []
let y = 0
const barY = {}
for (const bar of bars) {
  barY[bar.task_group_id] = y + BAR_ROW / 2
  rows.push({ kind: 'bar', bar, top: y })
  y += BAR_ROW
  if (expanded.has(bar.task_group_id)) {
    for (const task of bar.tasks) {
      rows.push({ kind: 'task', task, top: y })
      y += TASK_ROW
    }
  }
}
const height = y
const todayX = today >= start && today < start + days * DAY ? ((today -
   start) / DAY) * px : null
const byId = Object.fromEntries(bars.map((b) => [b.task_group_id, b]))

const arrow = ({ from, to }) => {
  const a = byId[from]
  const b = byId[to]
  const x1 = x(a.end) + px
  const y1 = barY[from]
  const x2 = x(b.start)
  const y2 = barY[to]
  if (x2 - x1 >= 12) {
    const mid = x1 + (x2 - x1) / 2
    return `M ${x1} ${y1} H ${mid} V ${y2} H ${x2}`
  }
  const between = y2 > y1 ? y2 - BAR_ROW / 2 : y2 + BAR_ROW / 2
  return `M ${x1} ${y1} h 6 V ${between} H ${x2 - 6} V ${y2} H ${x2}`
}

return (
  <div className="overflow-x-auto rounded-lg border">
    <div className="relative" style={{ width: LABEL_W + width }}>
      <div className="flex border-b bg-muted/40">
        <div className="sticky left-0 z-20 shrink-0 border-r bg-muted px-3
   py-2 text-xs font-medium text-muted-foreground"
          style={{ width: LABEL_W }}>
          Task group
        </div>
        <div className="relative h-9" style={{ width }}>
          {months(start, days, px).map((m) => (
            <div key={m.key} className="absolute top-0 h-full truncate
   border-l px-2 py-2 text-xs text-muted-foreground"
              style={{ left: m.left, width: m.width }}>
              {m.label}
            </div>
          ))}
        </div>
      </div>
      {data.milestones.length > 0 && (
        <div className="flex border-b">
          <div className="sticky left-0 z-20 shrink-0 border-r bg-card px-
   3 py-1.5 text-xs text-muted-foreground"
            style={{ width: LABEL_W }}>
            Milestones
          </div>
          <div className="relative h-8" style={{ width }}>
            {data.milestones.map((m) => (
```

```
            <div key={`${m.name}-${m.date}`} className="absolute top-1.5
flex items-center gap-1.5 text-xs whitespace-nowrap"
              style={{ left: x(m.date) + px / 2 - 5 }} title={`${m.name}
· ${formatDate(m.date)}`}>
              <span className="size-2.5 rotate-45 bg-foreground" />
              <span className="text-muted-foreground">{m.name}</span>
            </div>
          ))}
        </div>
      </div>
    )}

    <div className="relative">
      {rows.map((row) => (
        <div key={row.kind === 'bar' ? `b${row.bar.task_group_id}` : `t$
{row.task.task_id}`}
          className={`flex border-b last:border-b-0 ${row.kind ===
'task' ? 'bg-muted/20' : ''}`}
          style={{ height: row.kind === 'bar' ? BAR_ROW : TASK_ROW }}>
          <div className="sticky left-0 z-10 flex shrink-0 items-center
gap-1 border-r bg-card px-2 text-sm"
            style={{ width: LABEL_W }}>
            {row.kind === 'bar' ? (
              <>
                <Button size="icon-xs" variant="ghost" disabled={!
row.bar.tasks.length}
                  aria-label={expanded.has(row.bar.task_group_id) ?
'Collapse' : 'Expand'}
                  onClick={() => toggle(row.bar.task_group_id)}>
                  {expanded.has(row.bar.task_group_id) ? <ChevronDown />
: <ChevronRight />}
                </Button>
                <span className="truncate font-medium"
title={row.bar.name}>{row.bar.name}</span>
              </>
            ) : (
              <span className="flex min-w-0 items-center gap-2 pl-7
text-xs">
                <span className={`size-2 shrink-0 rounded-full $
{statusTone(row.task.status, isLate(row.task, generatedAt))}`} />
                <span className="truncate"
title={row.task.title}>{row.task.title}</span>
              </span>
            )}
          </div>
          <div className="relative" style={{ width }}>
            {row.kind === 'bar' ? (
              <div
                className={`absolute top-2.5 h-6 overflow-hidden
rounded-md border bg-muted ${row.bar.status === 'Blocked' ? 'border-
destructive' : ''}`}
                style={{ left: x(row.bar.start), width: x(row.bar.end) -
x(row.bar.start) + px }}
                title={`${row.bar.name}: ${formatDate(row.bar.start)} –
${formatDate(row.bar.end)} · ${row.bar.completion}% · $
{row.bar.status}`}>
                <div className="h-full bg-primary/70" style={{ width: `$
{row.bar.completion}%` }} />
                <span className="absolute inset-0 flex items-center px-2
text-xs font-medium">
                  {row.bar.completion}%
                </span>
              </div>
            ) : row.task.due ? (
              <div className="absolute top-1/2 flex -translate-y-1/2
items-center gap-1.5 whitespace-nowrap text-xs text-muted-foreground"
                style={{ left: x(row.task.due) + px / 2 - 5 }}
```

```
                    title={`${row.task.title} · due $
    {formatDate(row.task.due)} · ${row.task.status}`}>
                    <span className={`size-2.5 rotate-45 $
    {statusTone(row.task.status, isLate(row.task, generatedAt))}`} />
                    {row.task.assignee || 'Unassigned'}
                  </div>
                ) : (
                  <span className="absolute left-2 top-1/2 -translate-y-1/2
    text-xs text-muted-foreground">No due date</span>
                )}
              </div>
            </div>
          ))}

          <div className="pointer-events-none absolute top-0" style={{ left:
    LABEL_W, width, height }}>
            {todayX !== null && (
              <div className="absolute top-0 h-full w-px bg-destructive"
    style={{ left: todayX }} title="Today" />
            )}
            <svg className="absolute inset-0 text-muted-foreground"
    width={width} height={height}>
              <defs>
                <marker id={markerId} viewBox="0 0 6 6" refX="6" refY="3"
    markerWidth="6" markerHeight="6" orient="auto">
                  <path d="M0,0 L6,3 L0,6 z" fill="currentColor" />
                </marker>
              </defs>
              {data.dependencies.filter((d) => byId[d.from] &&
    byId[d.to]).map((d) => (
                <path key={`${d.from}-${d.to}`} d={arrow(d)} fill="none"
    stroke="currentColor" strokeWidth="1.25"
                  markerEnd={`url(#${markerId})`} />
              ))}
            </svg>
          </div>
        </div>
      </div>
    </div>
  )
}

export default function GanttChart({ projectId, canReevaluate = false }) {
  const [chart, setChart] = useState(undefined)
  const [versions, setVersions] = useState([])
  const [expanded, setExpanded] = useState(new Set())
  const [replanning, setReplanning] = useState(false)

  const load = useCallback(async (version) => {
    try {
      const [{ data }, { data: history }] = await Promise.all([
        projectsApi.gantt(projectId, version),
    projectsApi.ganttVersions(projectId),
      ])
      setChart(data)
      setVersions(history)
    } catch (err) {
      toast.error(apiError(err, 'Could not load the Gantt chart'))
      setChart((c) => (c === undefined ? null : c))
    }
  }, [projectId])

  useEffect(() => { load() }, [load])

  const toggle = (id) => setExpanded((prev) => {
    const next = new Set(prev)
    if (next.has(id)) next.delete(id)
    else next.add(id)
```

```
  return next
})

const replan = async () => {
  setReplanning(true)
  try {
    const { data } = await projectsApi.reevaluateGantt(projectId)
    const result = await waitForJob(data.job_id)
    if (result.status === 'success') toast.success(`Gantt re-planned —
   version ${result.version}`)
    else toast.info(result.reason || 'Nothing to re-plan')
    load()
  } catch (err) {
    toast.error(apiError(err, err.message || 'Could not re-plan the Gantt
   chart'))
  } finally {
    setReplanning(false)
  }
}
const replanButton = canReevaluate && (
  <Button variant="outline" size="sm" onClick={replan}
   disabled={replanning}>
    {replanning ? <Loader2 className="animate-spin" /> : <Sparkles />} Re-
   plan now
  </Button>
)

if (chart === undefined) return <Skeleton className="h-80" />
if (chart === null) {
  return (
    <div className="space-y-3">
      <NotYetAvailable icon={ChartGantt} title="No Gantt chart yet">
        The chart is generated when the project is evaluated, then
   regenerated as work moves.
        {canReevaluate && ' If you have planned task groups by hand, re-
   plan to draw them now.'}
      </NotYetAvailable>
      {replanButton && <div className="flex justify-
   center">{replanButton}</div>}
    </div>
  )
}

const data = chart.chart_data
const allOpen = data.bars.every((b) => !b.tasks.length ||
   expanded.has(b.task_group_id))
const isLatest = chart.version === chart.latest_version

return (
  <div id="project-gantt" className="space-y-3">
    <div className="flex flex-wrap items-center justify-between gap-2">
      <div className="flex flex-wrap items-center gap-2 text-sm">
        <Badge variant={isLatest ? 'default' : 'outline'}>Version
   {chart.version}</Badge>
        <span>{REASONS[chart.generated_reason] || chart.generated_reason
   || 'Generated'}</span>
        <span className="text-muted-foreground">·
   {formatDate(chart.generated_at)}</span>
        {!isLatest && <span className="text-muted-foreground">· viewing
   history</span>}
      </div>
      <div className="flex flex-wrap items-center gap-2">
        {versions.length > 1 && (
          <NativeSelect aria-label="Version" value={chart.version}
            onChange={(e) => load(Number(e.target.value))}>
            {versions.map((v) => (
              <NativeSelectOption key={v.version} value={v.version}>
```

```
                  v{v.version} — {REASONS[v.generated_reason] ||
    v.generated_reason}
                </NativeSelectOption>
              ))}
            </NativeSelect>
          )}
          <Button variant="outline" size="sm"
            onClick={() => setExpanded(allOpen ? new Set() : new
    Set(data.bars.map((b) => b.task_group_id)))}>
            {allOpen ? <ChevronsDownUp /> : <ChevronsUpDown />} {allOpen ?
    'Collapse all' : 'Expand all'}
          </Button>
          {replanButton}
          <Button variant="outline" size="icon-sm" aria-label="Refresh"
    onClick={() => load()}><RefreshCw /></Button>
        </div>
      </div>

      <Chart data={data} generatedAt={chart.generated_at}
    expanded={expanded} toggle={toggle} />

      <div className="flex flex-wrap items-center justify-between gap-2">
        <StatusLegend />
        <p className="text-xs text-muted-foreground">
          Bars show each task group’s planned dates and completion; expand
    one for its tasks’ due dates.
          Arrows are dependencies; the red line is today.
        </p>
      </div>
    </div>
  )
}
```

4.1.12 Frontend: Project Components

frontend/src/components/project/status.jsx

```
import { Badge } from '@/components/ui/badge'
import { STATUS_TONE } from '@/lib/project'

const TASK_VARIANT = {
  Pending: 'outline',
  'In Progress': 'secondary',
  'In Review': 'secondary',
  Completed: 'default',
  Blocked: 'destructive',
  Overdue: 'destructive',
  Cancelled: 'ghost',
}

const PROJECT_VARIANT = {
  Draft: 'outline',
  Evaluating: 'secondary',
  Active: 'default',
  Completed: 'secondary',
  Archived: 'ghost',
}

export function TaskStatusBadge({ status }) {
  return <Badge variant={TASK_VARIANT[status] || 'outline'}>{status}</Badge>
}

export function ProjectStatusBadge({ status }) {
  return <Badge variant={PROJECT_VARIANT[status] ||
     'outline'}>{status}</Badge>
}

export function RoleBadge({ role }) {
```

```
  if (!role) return null
  return <Badge variant={role === 'Project Manager' ? 'default' :
     'secondary'}>{role}</Badge>
}

const LEGEND = ['Pending', 'In Progress', 'In Review', 'Completed',
     'Blocked', 'Overdue']

export function StatusLegend({ className = '' }) {
  return (
    <div className={`flex flex-wrap items-center gap-x-4 gap-y-1 text-xs
     text-muted-foreground ${className}`}>
      {LEGEND.map((s) => (
        <span key={s} className="inline-flex items-center gap-1.5">
          <span className={`size-2.5 rounded-full ${STATUS_TONE[s]}`} /> {s}
        </span>
      ))}
    </div>
  )
}
```

frontend/src/components/project/ProjectOverview.jsx

```
import { useRef, useState } from 'react'
import { ExternalLink, FileText, GitBranch, Loader2, Trash2, Upload } from
     'lucide-react'
import { toast } from 'sonner'
import { Button } from '@/components/ui/button'
import { Card, CardAction, CardContent, CardDescription, CardHeader,
     CardTitle } from '@/components/ui/card'
import { Progress } from '@/components/ui/progress'
import { projectsApi } from '@/api/projects'
import { ProjectStatusBadge } from './status'
import EvaluationCard from './EvaluationCard'
import { apiError, formatDate } from '@/lib/project'

const DOCUMENTS = [
  ['project_initial_doc', 'Initial project document', 'What the AI first
     understood from the source files. Never edited.'],
  ['project_doc', 'Project document', 'The living document the Project
     Manager keeps current.'],
  ['project_progress_doc', 'Progress document', 'Rewritten weekly from every
     task's progress.'],
  ['project_report_doc', 'Project report', 'The summary a Manager reads.'],
]

export default function ProjectOverview({ project, isOwner, onChange }) {
  const fileInput = useRef(null)
  const [uploading, setUploading] = useState(false)
  const s = project.stats

  const upload = async (list) => {
    const files = Array.from(list || [])
    if (!files.length) return
    setUploading(true)
    try {
      const { data } = await projectsApi.uploadSourceFiles(project.id,
     files)
      if (data.uploaded.length) toast.success(`${data.uploaded.length}
     file(s) uploaded`)
      data.rejected.forEach((r) => toast.warning(`${r.file_name} was not
     uploaded: ${r.reason}`))
      onChange()
    } catch (err) {
      toast.error(apiError(err, 'Upload failed'))
    } finally {
      setUploading(false)
    }
```

```
}

const removeFile = async (file) => {
  try {
    await projectsApi.deleteSourceFile(project.id, file.id)
    toast.success(`${file.file_name} removed`)
    onChange()
  } catch (err) {
    toast.error(apiError(err, 'Could not remove the file'))
  }
}

return (
  <div className="grid gap-4 lg:grid-cols-3">
    <EvaluationCard project={project} isOwner={isOwner}
   onChange={onChange} />
    <Card className="lg:col-span-2">
      <CardHeader>
        <CardTitle className="text-base">Progress</CardTitle>
        <CardDescription>
          {s.completed_tasks} of {s.total_tasks} tasks complete across
   {s.task_groups} task group(s)
        </CardDescription>
        <CardAction><ProjectStatusBadge status={project.status}
   /></CardAction>
      </CardHeader>
      <CardContent className="space-y-4">
        <div className="space-y-1">
          <div className="flex justify-between text-xs text-muted-
   foreground">
            <span>Completion</span><span>{s.completion_percentage}%</span>
          </div>
          <Progress value={s.completion_percentage} />
        </div>
        <dl className="grid grid-cols-2 gap-4 text-sm sm:grid-cols-4">
          <div><dt
   className="text-muted-foreground">Start</dt><dd>{formatDate(project.star
   t_date)}</dd></div>
          <div><dt
   className="text-muted-foreground">End</dt><dd>{formatDate(project.end_da
   te)}</dd></div>
          <div>
            <dt className="text-muted-foreground">Overdue tasks</dt>
            <dd className={s.overdue_tasks ? 'text-destructive font-
   medium' : ''}>{s.overdue_tasks}</dd>
          </div>
          <div><dt className="text-muted-foreground">Last
   evaluated</dt><dd>{formatDate(project.last_evaluated_at)}</dd></div>
        </dl>
        {project.github_repo && (
          <a href={`https://github.com/${project.github_repo}`}
   target="_blank" rel="noopener noreferrer"
            className="inline-flex items-center gap-1.5 text-sm text-
   primary hover:underline">
            <GitBranch className="size-4" /> {project.github_repo} ·
   {project.github_default_branch}
            <ExternalLink className="size-3" />
          </a>
        )}
      </CardContent>
    </Card>

    <Card>
      <CardHeader>
        <CardTitle className="text-base">Documents</CardTitle>
        <CardDescription>Markdown files in the project
   repository</CardDescription>
      </CardHeader>
```

```
   <CardContent className="space-y-3">
     {DOCUMENTS.map(([key, label, hint]) => (
       <div key={key} className="text-sm">
         {project.documents[key] ? (
           <a href={project.documents[key]} target="_blank"
rel="noopener noreferrer"
             className="inline-flex items-center gap-1.5 font-medium
text-primary hover:underline">
             <FileText className="size-4" /> {label} <ExternalLink
className="size-3" />
           </a>
         ) : (
           <p className="inline-flex items-center gap-1.5 font-medium
text-muted-foreground">
             <FileText className="size-4" /> {label}
           </p>
         )}
         <p className="text-xs text-muted-foreground">
           {project.documents[key] ? hint : 'Not generated yet —
created when the project is evaluated.'}
         </p>
       </div>
     ))}
   </CardContent>
 </Card>

 <Card className="lg:col-span-3">
   <CardHeader>
     <CardTitle className="text-base">Source files</CardTitle>
     <CardDescription>Requirement documents the AI evaluates. Stored in
object storage, not in the repository.</CardDescription>
     {isOwner && (
       <CardAction>
         <Button size="sm" variant="outline" onClick={() =>
fileInput.current?.click()} disabled={uploading}>
           {uploading ? <Loader2 className="animate-spin" /> :
<Upload />} Add files
         </Button>
         <input ref={fileInput} type="file" multiple
accept=".pdf,.docx,.md,.markdown,.txt" className="hidden"
           onChange={(e) => { upload(e.target.files); e.target.value =
'' }} />
       </CardAction>
     )}
   </CardHeader>
   <CardContent>
     {project.source_files.length === 0 ? (
       <p className="text-sm text-muted-foreground">No source files
yet.</p>
     ) : (
       <ul className="divide-y rounded-lg border">
         {project.source_files.map((f) => (
           <li key={f.id} className="flex items-center gap-3 px-3 py-2
text-sm">
             <FileText className="size-4 text-muted-foreground" />
             <span className="flex-1 truncate">{f.file_name}</span>
             <span className="text-xs text-muted-foreground">
               {f.extracted_chars.toLocaleString()} characters
extracted · {formatDate(f.uploaded_at)}
             </span>
             {isOwner && (
               <Button variant="ghost" size="icon-sm" aria-
label={`Remove ${f.file_name}`} onClick={() => removeFile(f)}>
                 <Trash2 />
               </Button>
             )}
           </li>
         ))}
```

```
            </ul>
          )}
        </CardContent>
      </Card>
    </div>
  )
}
```

frontend/src/components/project/EvaluationCard.jsx

```
import { useEffect, useState } from 'react'
import { AlertTriangle, Loader2, Sparkles } from 'lucide-react'
import { toast } from 'sonner'
import { Alert, AlertDescription, AlertTitle } from '@/components/ui/alert'
import { Button } from '@/components/ui/button'
import { Card, CardAction, CardContent, CardDescription, CardHeader,
    CardTitle } from '@/components/ui/card'
import { projectsApi } from '@/api/projects'
import { apiError, formatDate } from '@/lib/project'

const JOB_LABELS = {
  evaluate_project: 'Evaluation failed',
  reevaluate_gantt: 'Gantt re-evaluation',
  update_project_progress_doc: 'Progress report',
}

export default function EvaluationCard({ project, isOwner, onChange }) {
  const [starting, setStarting] = useState(false)
  const running = project.status === 'Evaluating'
  const errors = Object.entries(project.ai_errors || {})
  const hasSources = project.source_files.some((f) => f.extracted_chars > 0)
  const hasTasks = project.stats.total_tasks > 0

  useEffect(() => {
    if (!running) return undefined
    const timer = setInterval(async () => {
      try {
        const { data } = await projectsApi.evaluateStatus(project.id)
        if (data.state === 'running') return
        clearInterval(timer)
        if (data.state === 'failed') toast.error(data.error)
        else toast.success('Project evaluated — documents, task groups and
   Gantt chart are ready')
        onChange()
      } catch {
        clearInterval(timer)
      }
    }, 2000)
    return () => clearInterval(timer)
  }, [running, project.id, onChange])

  const evaluate = async () => {
    setStarting(true)
    try {
      await projectsApi.evaluate(project.id)
      toast.info('Evaluation started — this takes a minute or two')
      onChange()
    } catch (err) {
      toast.error(apiError(err, 'Could not start the evaluation'))
    } finally {
      setStarting(false)
    }
  }

  let hint = project.last_evaluated_at
    ? `Last evaluated ${formatDate(project.last_evaluated_at)}.`
    : 'The AI reads the source files and writes the project document, task
     groups and the first Gantt chart.'
```

```
  if (isOwner && !hasSources) hint = 'Upload at least one requirement
     document to evaluate the project.'
  if (isOwner && hasTasks) hint = 'The project has tasks now, so its plan is
     kept current by editing the project document instead.'

  return (
    <Card className="lg:col-span-3">
      <CardHeader>
        <CardTitle className="text-base">AI evaluation</CardTitle>
        <CardDescription>{running ? 'Evaluating — reading the source files
     and planning the work…' : hint}</CardDescription>
        {isOwner && (
          <CardAction>
            <Button size="sm" onClick={evaluate} disabled={running ||
     starting || !hasSources || hasTasks}>
              {running || starting ? <Loader2 className="animate-spin" /> :
     <Sparkles />}
              {running ? 'Evaluating…' : project.last_evaluated_at ?
     'Evaluate again' : 'Evaluate project'}
            </Button>
          </CardAction>
        )}
      </CardHeader>
      {errors.length > 0 && (
        <CardContent className="space-y-2">
          {errors.map(([job, e]) => (
            <Alert key={job} variant={job === 'evaluate_project' ?
     'destructive' : 'default'}>
              <AlertTriangle />
              <AlertTitle>{JOB_LABELS[job] || job}</AlertTitle>
              <AlertDescription>{e.message}
     ({formatDate(e.at)})</AlertDescription>
            </Alert>
          ))}
        </CardContent>
      )}
    </Card>
  )
}
```

frontend/src/components/project/TaskBoard.jsx

```
import { useCallback, useEffect, useState } from 'react'
import { useNavigate } from 'react-router-dom'
import { AlertTriangle, ExternalLink, FolderPlus, Loader2, Plus, Sparkles,
     Trash2 } from 'lucide-react'
import { toast } from 'sonner'
import { Button } from '@/components/ui/button'
import { Card, CardAction, CardContent, CardDescription, CardHeader,
     CardTitle } from '@/components/ui/card'
import {
  Dialog, DialogContent, DialogDescription, DialogFooter, DialogHeader,
     DialogTitle,
} from '@/components/ui/dialog'
import { Input } from '@/components/ui/input'
import { Label } from '@/components/ui/label'
import { Textarea } from '@/components/ui/textarea'
import { Progress } from '@/components/ui/progress'
import { NativeSelect, NativeSelectOption } from '@/components/ui/native-
     select'
import { Empty, EmptyDescription, EmptyHeader, EmptyTitle } from
     '@/components/ui/empty'
import { projectsApi } from '@/api/projects'
import { TaskStatusBadge } from './status'
import PeopleSuggestions from './PeopleSuggestions'
import { apiError, formatDate, isOverdue } from '@/lib/project'
import { waitForJob } from '@/lib/jobs'
```

```
function useAiJob() {
  const [busy, setBusy] = useState(false)
  const run = async (start) => {
    setBusy(true)
    try {
      const { data } = await start()
      return await waitForJob(data.job_id)
    } catch (err) {
      toast.error(apiError(err, err.message || 'The AI request failed'))
      return null
    } finally {
      setBusy(false)
    }
  }
  return { run, busy }
}

function GroupDialog({ projectId, open, onOpenChange, onCreated }) {
  const [form, setForm] = useState({ name: '', description: '',
     estimated_duration_days: '' })
  const [busy, setBusy] = useState(false)

  const submit = async () => {
    setBusy(true)
    try {
      await projectsApi.createTaskGroup(projectId, {
        name: form.name.trim(),
        description: form.description.trim() || null,
        estimated_duration_days: form.estimated_duration_days ?
    Number(form.estimated_duration_days) : null,
      })
      toast.success('Task group created')
      setForm({ name: '', description: '', estimated_duration_days: '' })
      onOpenChange(false)
      onCreated()
    } catch (err) {
      toast.error(apiError(err, 'Could not create the group'))
    } finally {
      setBusy(false)
    }
  }

  return (
    <Dialog open={open} onOpenChange={onOpenChange}>
      <DialogContent className="max-w-md">
        <DialogHeader>
          <DialogTitle>New task group</DialogTitle>
          <DialogDescription>A group is one module on the Gantt
    chart.</DialogDescription>
        </DialogHeader>
        <div className="space-y-3">
          <div className="space-y-2">
            <Label htmlFor="g-name">Name</Label>
            <Input id="g-name" autoFocus value={form.name} onChange={(e) =>
    setForm({ ...form, name: e.target.value })} />
          </div>
          <div className="space-y-2">
            <Label htmlFor="g-desc">Description</Label>
            <Textarea id="g-desc" rows={2} value={form.description}
              onChange={(e) => setForm({ ...form, description:
    e.target.value })} />
          </div>
          <div className="space-y-2">
            <Label htmlFor="g-days">Estimated duration (days)</Label>
            <Input id="g-days" type="number" min="1"
    value={form.estimated_duration_days}
              onChange={(e) => setForm({ ...form, estimated_duration_days:
    e.target.value })} />
```

```
          </div>
        </div>
        <DialogFooter>
          <Button variant="outline" onClick={() =>
    onOpenChange(false)}>Cancel</Button>
          <Button onClick={submit} disabled={busy || !form.name.trim()}>
            {busy && <Loader2 className="animate-spin" />} Create group
          </Button>
        </DialogFooter>
      </DialogContent>
    </Dialog>
  )
}

function TaskDialog({ projectId, groups, defaultGroupId, open, onOpenChange,
    onCreated }) {
  const empty = { title: '', description: '', task_group_id: '', priority:
    'Medium', due_date: '', assigned_to_user_id: '' }
  const [form, setForm] = useState(empty)
  const [candidates, setCandidates] = useState([])
  const [busy, setBusy] = useState(false)
  const [ideas, setIdeas] = useState(null)
  const [people, setPeople] = useState(null)
  const ideaJob = useAiJob()
  const peopleJob = useAiJob()

  useEffect(() => {
    if (!open) return
    setForm({ ...empty, task_group_id: defaultGroupId ?
     String(defaultGroupId) : '' })
    setIdeas(null)
    setPeople(null)
    projectsApi.candidates(projectId).then((r) =>
     setCandidates(r.data)).catch(() => setCandidates([]))
  }, [open, projectId, defaultGroupId])

  const assignee = candidates.find((c) => String(c.id) ===
    form.assigned_to_user_id)

  const suggestTasks = async () => {
    const result = await ideaJob.run(() =>
     projectsApi.suggestTasks(projectId, form.task_group_id))
    if (result) setIdeas(result.tasks)
  }
  const suggestPeople = async () => {
    const result = await peopleJob.run(() =>
      projectsApi.suggestAssignees(projectId, form.title.trim(),
     form.description.trim() || null))
    if (result) setPeople(result.suggestions)
  }
  const applyIdea = (idea) => {
    setForm((f) => ({ ...f, title: idea.title, description: idea.description
     || '', priority: idea.priority }))
    setPeople(null)
  }

  const submit = async () => {
    setBusy(true)
    try {
      const { data } = await projectsApi.createTask(projectId, {
        title: form.title.trim(),
        description: form.description.trim() || null,
        task_group_id: Number(form.task_group_id),
        priority: form.priority,
        due_date: form.due_date || null,
        assigned_to_user_id: form.assigned_to_user_id ?
     Number(form.assigned_to_user_id) : null,
      })
```

```
      toast.success(`Task created as GitHub issue #$
    {data.task.github_issue_id}`)
      data.warnings.forEach((w) => toast.warning(w))
      onOpenChange(false)
      onCreated()
    } catch (err) {
      toast.error(apiError(err, 'Could not create the task'))
    } finally {
      setBusy(false)
    }
  }

  return (
    <Dialog open={open} onOpenChange={onOpenChange}>
      <DialogContent className="max-h-[90vh] max-w-lg overflow-y-auto">
        <DialogHeader>
          <DialogTitle>New task</DialogTitle>
          <DialogDescription>Creating the task opens a GitHub issue in the
    project repository.</DialogDescription>
        </DialogHeader>
        <div className="space-y-3">
          <div className="space-y-2">
            <Label htmlFor="t-title">Title</Label>
            <Input id="t-title" autoFocus value={form.title} onChange={(e)
    => setForm({ ...form, title: e.target.value })} />
          </div>
          <div className="space-y-2">
            <Label htmlFor="t-desc">Description</Label>
            <Textarea id="t-desc" rows={3} value={form.description}
    placeholder="Becomes the issue body"
              onChange={(e) => setForm({ ...form, description:
    e.target.value })} />
          </div>
          <div className="grid gap-3 sm:grid-cols-2">
            <div className="space-y-2">
              <Label htmlFor="t-group">Task group</Label>
              <NativeSelect id="t-group" className="w-full"
    value={form.task_group_id}
                onChange={(e) => setForm({ ...form, task_group_id:
    e.target.value })}>
                <NativeSelectOption value="">Choose…</NativeSelectOption>
                {groups.map((g) => <NativeSelectOption key={g.id}
    value={g.id}>{g.name}</NativeSelectOption>)}
              </NativeSelect>
            </div>
            <div className="space-y-2">
              <Label htmlFor="t-priority">Priority</Label>
              <NativeSelect id="t-priority" className="w-full"
    value={form.priority}
                onChange={(e) => setForm({ ...form, priority: e.target.value
    })}>
                {['Low', 'Medium', 'High', 'Critical'].map((p) =>
    <NativeSelectOption key={p} value={p}>{p}</NativeSelectOption>)}
              </NativeSelect>
            </div>
            <div className="space-y-2">
              <Label htmlFor="t-due">Due date</Label>
              <Input id="t-due" type="date" value={form.due_date}
    onChange={(e) => setForm({ ...form, due_date: e.target.value })} />
            </div>
            <div className="space-y-2">
              <Label htmlFor="t-assignee">Assignee</Label>
              <NativeSelect id="t-assignee" className="w-full"
    value={form.assigned_to_user_id}
                onChange={(e) => setForm({ ...form, assigned_to_user_id:
    e.target.value })}>
                <NativeSelectOption value="">Unassigned</NativeSelectOption>
```

```
            {candidates.map((c) => <NativeSelectOption key={c.id}
    value={c.id}>{c.name}</NativeSelectOption>)}
          </NativeSelect>
        </div>
      </div>
      <div className="flex flex-wrap gap-2">
        <Button type="button" size="sm" variant="outline"
    onClick={suggestTasks}
          disabled={!form.task_group_id || ideaJob.busy}>
          {ideaJob.busy ? <Loader2 className="animate-spin" /> :
    <Sparkles />} Suggest tasks for this group
        </Button>
        <Button type="button" size="sm" variant="outline"
    onClick={suggestPeople}
          disabled={form.title.trim().length < 2 || peopleJob.busy}>
          {peopleJob.busy ? <Loader2 className="animate-spin" /> :
    <Sparkles />} Suggest assignees
        </Button>
      </div>
      {ideas && (
        <ul className="space-y-1.5">
          {ideas.map((idea) => (
            <li key={idea.title} className="flex items-start gap-2
    rounded-lg border p-2 text-sm">
              <span className="min-w-0 flex-1">
                <span className="font-medium">{idea.title}</span>
                <span className="block text-xs text-muted-foreground">
                  {idea.priority}{idea.estimated_hours ? ` · ~$
    {idea.estimated_hours} h` : ''}
                  {idea.description ? ` · ${idea.description}` : ''}
                </span>
              </span>
              <Button type="button" size="sm" variant="ghost"
    onClick={() => applyIdea(idea)}>Use</Button>
            </li>
          ))}
        </ul>
      )}
      {people && (
        <PeopleSuggestions suggestions={people} actionLabel="Pick"
          onPick={(u) => setForm((f) => ({ ...f, assigned_to_user_id:
    String(u.id) }))} />
      )}
      {assignee && !assignee.github_username && (
        <p className="flex items-start gap-1.5 text-xs text-muted-
    foreground">
          <AlertTriangle className="mt-0.5 size-3.5 shrink-0" />
          {assignee.name} has no GitHub username, so the issue will be
    created unassigned.
          They can add one under Profile.
        </p>
      )}
    </div>
    <DialogFooter>
      <Button variant="outline" onClick={() =>
    onOpenChange(false)}>Cancel</Button>
      <Button onClick={submit} disabled={busy ||
    form.title.trim().length < 2 || !form.task_group_id}>
        {busy && <Loader2 className="animate-spin" />} Create task
      </Button>
    </DialogFooter>
  </DialogContent>
 </Dialog>
 )
}

export default function TaskBoard({ projectId, canManage = false, mine =
    false, taskPath = null, onChange }) {
```

```
const navigate = useNavigate()
const [groups, setGroups] = useState(null)
const [tasks, setTasks] = useState([])
const [groupOpen, setGroupOpen] = useState(false)
const [taskOpen, setTaskOpen] = useState(false)
const [taskGroup, setTaskGroup] = useState(null)

const load = useCallback(async () => {
  try {
    const [g, t] = await Promise.all([projectsApi.taskGroups(projectId),
   projectsApi.tasks(projectId, mine ? { mine: true } : {})])
    setGroups(g.data)
    setTasks(t.data)
  } catch (err) {
    toast.error(apiError(err, 'Could not load tasks'))
    setGroups([])
  }
}, [projectId, mine])

useEffect(() => { load() }, [load])

const changed = () => { load(); onChange?.() }

const deleteGroup = async (group) => {
  try {
    await projectsApi.deleteTaskGroup(projectId, group.id)
    toast.success(`${group.name} deleted`)
    changed()
  } catch (err) {
    toast.error(apiError(err, 'Could not delete the group'))
  }
}

if (groups === null) return <p className="text-sm text-muted-
   foreground">Loading tasks…</p>

const visibleGroups = mine ? groups.filter((g) => tasks.some((t) =>
   t.task_group_id === g.id)) : groups

return (
  <div className="space-y-4">
    {canManage && (
      <div className="flex flex-wrap gap-2">
        <Button onClick={() => { setTaskGroup(null); setTaskOpen(true) }}
   disabled={groups.length === 0}>
          <Plus /> New task
        </Button>
        <Button variant="outline" onClick={() =>
   setGroupOpen(true)}><FolderPlus /> New task group</Button>
      </div>
    )}

    {visibleGroups.length === 0 && (
      <Empty className="border border-dashed">
        <EmptyHeader>
          <EmptyTitle>{mine ? 'No tasks assigned to you here' : 'No task
   groups yet'}</EmptyTitle>
          <EmptyDescription>
            {mine ? 'When the Project Manager assigns you a task it
   appears here.'
              : canManage ? 'Create a task group first — each group is a
   module on the Gantt chart.'
                : 'The Project Manager has not planned any work yet.'}
          </EmptyDescription>
        </EmptyHeader>
      </Empty>
    )}
```

```
 {visibleGroups.map((g) => {
   const groupTasks = tasks.filter((t) => t.task_group_id === g.id)
   return (
     <Card key={g.id}>
       <CardHeader>
         <CardTitle className="text-base">{g.name}</CardTitle>
         <CardDescription>
           {g.completed_tasks}/{g.total_tasks} complete · {g.status}
           {g.estimated_duration_days ? ` · ~${
{g.estimated_duration_days} days` : ''}
         </CardDescription>
         {canManage && (
           <CardAction className="flex gap-1">
             <Button size="sm" variant="ghost" onClick={() =>
{ setTaskGroup(g.id); setTaskOpen(true) }}>
               <Plus /> Task
             </Button>
             {groupTasks.length === 0 && (
               <Button size="icon-sm" variant="ghost" aria-
label={`Delete ${g.name}`} onClick={() => deleteGroup(g)}>
                 <Trash2 />
               </Button>
             )}
           </CardAction>
         )}
       </CardHeader>
       <CardContent className="space-y-3">
         <Progress value={g.completion_percentage} />
         {groupTasks.length === 0 ? (
           <p className="text-sm text-muted-foreground">No tasks in
this group yet.</p>
         ) : (
           <ul className="divide-y rounded-lg border">
             {groupTasks.map((t) => (
               <li
                 key={t.id}
                 className={`flex flex-wrap items-center gap-3 px-3 py-
2 text-sm ${taskPath ? 'cursor-pointer hover:bg-muted/40' : ''}`}
                 onClick={taskPath ? () => navigate(taskPath(t.id)) :
undefined}
               >
                 <span className="min-w-0 flex-1">
                   <span className="font-medium">{t.title}</span>
                   <span className="block text-xs text-muted-
foreground">
                     {t.assigned_to?.name || 'Unassigned'} ·
{t.priority}
                     {t.due_date && <> · due <span
className={isOverdue(t) ? 'text-destructive' :
''}>{formatDate(t.due_date)}</span></>}
                   </span>
                 </span>
                 {t.github_issue_url && (
                   <a href={t.github_issue_url} target="_blank"
rel="noopener noreferrer" onClick={(e) => e.stopPropagation()}
                     className="inline-flex items-center gap-1 text-xs
text-muted-foreground hover:text-foreground">
                     #{t.github_issue_id} <ExternalLink
className="size-3" />
                   </a>
                 )}
                 <TaskStatusBadge status={t.status} />
               </li>
             ))}
           </ul>
         )}
       </CardContent>
     </Card>
```

```
        )
      })}

      {canManage && (
        <>
          <GroupDialog projectId={projectId} open={groupOpen}
    onOpenChange={setGroupOpen} onCreated={changed} />
          <TaskDialog projectId={projectId} groups={groups}
    defaultGroupId={taskGroup}
            open={taskOpen} onOpenChange={setTaskOpen}
    onCreated={changed} />
        </>
      )}
    </div>
  )
}
```

frontend/src/components/project/TaskDiscussion.jsx

```
import { useCallback, useEffect, useState } from 'react'
import { Loader2, MessageSquare, Reply, Send, X } from 'lucide-react'
import { toast } from 'sonner'
import { Avatar, AvatarFallback } from '@/components/ui/avatar'
import { Button } from '@/components/ui/button'
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from
    '@/components/ui/card'
import { Textarea } from '@/components/ui/textarea'
import { tasksApi } from '@/api/tasks'
import { useLiveEvents } from '@/contexts/LiveContext'
import { apiError, formatDateTime, initials } from '@/lib/project'

function Message({ m, onReply }) {
  return (
    <div className="flex gap-3">
      <Avatar className="size-7"><AvatarFallback className="text-
    xs">{initials(m.author?.name)}</AvatarFallback></Avatar>
      <div className="min-w-0 flex-1">
        <p className="text-xs text-muted-foreground">
          <span className="font-medium text-foreground">{m.author?.name ||
    'Someone'}</span> · {formatDateTime(m.created_at)}
        </p>
        <p className="whitespace-pre-wrap text-sm">{m.message}</p>
        {onReply && (
          <Button variant="ghost" size="xs" className="-ml-2" onClick={() =>
    onReply(m)}><Reply /> Reply</Button>
        )}
      </div>
    </div>
  )
}

export default function TaskDiscussion({ taskId }) {
  const [messages, setMessages] = useState(null)
  const [draft, setDraft] = useState('')
  const [replyTo, setReplyTo] = useState(null)
  const [sending, setSending] = useState(false)

  const load = useCallback(async () => {
    try {
      setMessages((await tasksApi.discussions(taskId)).data)
    } catch (err) {
      toast.error(apiError(err, 'Could not load the discussion'))
    }
  }, [taskId])

  useEffect(() => { load() }, [load])
  useLiveEvents((event) => {
```

```
  if (event.type === 'discussion' && String(event.task_id) ===
   String(taskId)) load()
})

const send = async () => {
  if (!draft.trim()) return
  setSending(true)
  try {
    await tasksApi.postDiscussion(taskId, draft.trim(), replyTo?.id ??
   null)
    setDraft('')
    setReplyTo(null)
    load()
  } catch (err) {
    toast.error(apiError(err, 'Could not post the message'))
  } finally {
    setSending(false)
  }
}

const roots = (messages || []).filter((m) => !m.parent_id)
const repliesOf = (id) => (messages || []).filter((m) => m.parent_id ===
   id)

return (
  <Card id="task-discussion">
    <CardHeader>
      <CardTitle className="flex items-center gap-2 text-
   base"><MessageSquare className="size-4" /> Discussion</CardTitle>
      <CardDescription>Between the assignee, the assigner, the Project
   Managers and the project’s Manager.</CardDescription>
    </CardHeader>
    <CardContent className="space-y-4">
      {messages === null ? (
        <Loader2 className="size-4 animate-spin text-muted-foreground" />
      ) : roots.length === 0 ? (
        <p className="text-sm text-muted-foreground">No messages yet.</p>
      ) : (
        roots.map((m) => (
          <div key={m.id} className="space-y-3">
            <Message m={m} onReply={setReplyTo} />
            {repliesOf(m.id).length > 0 && (
              <div className="ml-10 space-y-3 border-l pl-4">
                {repliesOf(m.id).map((r) => <Message key={r.id} m={r}
   onReply={() => setReplyTo(m)} />)}
              </div>
            )}
          </div>
        ))
      )}

      <div className="space-y-2 border-t pt-4">
        {replyTo && (
          <p className="flex items-center gap-1 text-xs text-muted-
   foreground">
            Replying to {replyTo.author?.name}
            <Button variant="ghost" size="icon-xs" onClick={() =>
   setReplyTo(null)}><X /><span className="sr-only">Cancel
   reply</span></Button>
          </p>
        )}
        <Textarea value={draft} onChange={(e) => setDraft(e.target.value)}
   rows={3} maxLength={5000}
          placeholder="Write a message…" aria-label="Message"
          onKeyDown={(e) => { if (e.key === 'Enter' && (e.metaKey ||
   e.ctrlKey)) send() }} />
        <div className="flex justify-end">
```

```
            <Button size="sm" onClick={send} disabled={sending || !
    draft.trim()}>
              {sending ? <Loader2 className="animate-spin" /> : <Send />}
    Send
            </Button>
          </div>
        </div>
      </CardContent>
    </Card>
  )
}
```

frontend/src/components/project/TeamPanel.jsx

```
import { useCallback, useEffect, useState } from 'react'
import { Loader2, Sparkles, UserMinus, UserPlus } from 'lucide-react'
import { toast } from 'sonner'
import { Button } from '@/components/ui/button'
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from
    '@/components/ui/card'
import { Avatar, AvatarFallback } from '@/components/ui/avatar'
import { NativeSelect, NativeSelectOption } from '@/components/ui/native-
    select'
import { projectsApi } from '@/api/projects'
import { RoleBadge } from './status'
import PeopleSuggestions from './PeopleSuggestions'
import { apiError, initials } from '@/lib/project'
import { waitForJob } from '@/lib/jobs'

export default function TeamPanel({ projectId, isOwner, onChange }) {
  const [team, setTeam] = useState([])
  const [candidates, setCandidates] = useState([])
  const [pick, setPick] = useState('')
  const [busy, setBusy] = useState(false)
  const [suggesting, setSuggesting] = useState(false)
  const [suggestions, setSuggestions] = useState(null)

  const load = useCallback(async () => {
    try {
      const { data } = await projectsApi.team(projectId)
      setTeam(data)
      if (isOwner) setCandidates((await
    projectsApi.candidates(projectId)).data)
    } catch (err) {
      toast.error(apiError(err, 'Could not load the team'))
    }
  }, [projectId, isOwner])

  useEffect(() => { load() }, [load])

  const pmIds = new Set(team.filter((m) => m.project_role === 'Project
     Manager').map((m) => m.user.id))
  const choices = candidates.filter((c) => !pmIds.has(c.id))

  const suggest = async () => {
    setSuggesting(true)
    try {
      const { data } = await projectsApi.suggestManagers(projectId)
      setSuggestions((await waitForJob(data.job_id)).suggestions)
    } catch (err) {
      toast.error(apiError(err, err.message || 'Could not get suggestions'))
    } finally {
      setSuggesting(false)
    }
  }

  const appoint = async (userId = Number(pick)) => {
    setBusy(true)
```

```
  try {
    const { data } = await projectsApi.assignProjectManager(projectId,
   userId)
    toast.success('Project Manager assigned — invited to the GitHub
   repository')
    data.warnings.forEach((w) => toast.warning(w))
    setPick('')
    setSuggestions((list) => list?.filter((s) => s.user.id !== userId) ??
   null)
    await load()
    onChange?.()
  } catch (err) {
    toast.error(apiError(err, 'Could not assign'))
  } finally {
    setBusy(false)
  }
}

const remove = async (member) => {
  try {
    const { data } = await projectsApi.removeProjectManager(projectId,
   member.user.id)
    toast.success(data.project_role === 'Member'
      ? `${member.user.name} stays on the project as a Member — they still
   hold tasks`
      : `${member.user.name} removed from the project and its GitHub
   repository`)
    data.warnings.forEach((w) => toast.warning(w))
    await load()
    onChange?.()
  } catch (err) {
    toast.error(apiError(err, 'Could not remove'))
  }
}

return (
  <div className="grid gap-4 lg:grid-cols-3">
    <Card className="lg:col-span-2">
      <CardHeader>
        <CardTitle className="text-base">Team</CardTitle>
        <CardDescription>
          Project Managers run the project. Members join when they are
   first assigned a task.
        </CardDescription>
      </CardHeader>
      <CardContent>
        {team.length === 0 ? (
          <p className="text-sm text-muted-foreground">Nobody is on this
   project yet.</p>
        ) : (
          <ul className="divide-y">
            {team.map((m) => (
              <li key={m.user.id} className="flex items-center gap-3 py-
   2.5">
                <Avatar className="size-8"><AvatarFallback
   className="text-xs">{initials(m.user.name)}</AvatarFallback></Avatar>
                <div className="min-w-0 flex-1">
                  <p className="truncate text-sm font-
   medium">{m.user.name}</p>
                  <p className="truncate text-xs text-muted-foreground">
                    {m.user.email} · {m.user.github_username ? `@$
   {m.user.github_username}` : 'no GitHub username'}
                    {' · '}{m.open_tasks} open task(s)
                  </p>
                </div>
                <RoleBadge role={m.project_role} />
                {isOwner && m.project_role === 'Project Manager' && (
```

```
                  <Button variant="ghost" size="icon-sm" aria-
    label={`Remove ${m.user.name}`} onClick={() => remove(m)}>
                    <UserMinus />
                  </Button>
                )}
              </li>
            ))}
          </ul>
        )}
      </CardContent>
    </Card>

    {isOwner && (
      <Card className="content-start">
        <CardHeader>
          <CardTitle className="text-base">Assign a Project
    Manager</CardTitle>
          <CardDescription>Any Employee. They can then create tasks and
    assign Members.</CardDescription>
        </CardHeader>
        <CardContent className="space-y-3">
          <NativeSelect value={pick} onChange={(e) =>
    setPick(e.target.value)} className="w-full">
            <NativeSelectOption value="">Choose an
    Employee…</NativeSelectOption>
            {choices.map((c) => (
              <NativeSelectOption key={c.id} value={c.id}>{c.name} —
    {c.email}</NativeSelectOption>
            ))}
          </NativeSelect>
          <Button className="w-full" disabled={!pick || busy} onClick={()
    => appoint()}>
            {busy ? <Loader2 className="animate-spin" /> : <UserPlus />}
    Assign
          </Button>
          <Button className="w-full" variant="outline"
    disabled={suggesting} onClick={suggest}>
            {suggesting ? <Loader2 className="animate-spin" /> : <Sparkles
    />}
            {suggesting ? 'Ranking candidates…' : 'Suggest with AI'}
          </Button>
          {suggestions && (
            <div className="space-y-2 pt-1">
              <p className="text-xs text-muted-foreground">
                Ranked on delivery record and task history. Suggestions
    only — you choose.
              </p>
              <PeopleSuggestions suggestions={suggestions}
    actionLabel="Assign" busyId={busy ? 0 : null}
                onPick={(u) => appoint(u.id)} />
            </div>
          )}
        </CardContent>
      </Card>
    )}
  </div>
 )
}
```

frontend/src/components/project/PeopleSuggestions.jsx

```
import { Sparkles } from 'lucide-react'
import { Avatar, AvatarFallback } from '@/components/ui/avatar'
import { Badge } from '@/components/ui/badge'
import { Button } from '@/components/ui/button'
import { initials } from '@/lib/project'

function statLine(stats) {
```

```
  const parts = [`${stats.tasks_completed} task(s) done`]
  if (stats.on_time_rate != null) parts.push(`$
     {Math.round(stats.on_time_rate * 100)}% on time`)
  parts.push(`${stats.open_tasks} open`)
  if (stats.projects_managed) parts.push(`managed ${stats.projects_managed}
     project(s)`)
  return parts.join(' · ')
}

export default function PeopleSuggestions({ suggestions, actionLabel,
     onPick, busyId }) {
  if (!suggestions.length) {
    return <p className="text-sm text-muted-foreground">The AI found no
     suitable candidates.</p>
  }
  return (
    <ul className="space-y-2">
      {suggestions.map((s) => (
        <li key={s.user.id} className="space-y-1.5 rounded-lg border p-3">
          <div className="flex items-center gap-2">
            <Avatar className="size-7"><AvatarFallback className="text-
     xs">{initials(s.user.name)}</AvatarFallback></Avatar>
            <div className="min-w-0 flex-1">
              <p className="truncate text-sm font-medium">{s.user.name}</p>
              <p className="truncate text-xs text-muted-
     foreground">{statLine(s.stats)}</p>
            </div>
            <Badge variant="secondary">{Math.round(s.score * 100)}</Badge>
            <Button size="sm" variant="outline" disabled={busyId != null}
     onClick={() => onPick(s.user)}>
              {actionLabel}
            </Button>
          </div>
          <p className="flex gap-1.5 text-xs text-muted-foreground">
            <Sparkles className="mt-0.5 size-3 shrink-0" /> {s.reasoning}
          </p>
        </li>
      ))}
    </ul>
  )
}
```

frontend/src/components/project/ProjectDocEditor.jsx

```
import { useCallback, useEffect, useState } from 'react'
import { ExternalLink, FileText, Info, Loader2, Save } from 'lucide-react'
import { toast } from 'sonner'
import { Alert, AlertDescription } from '@/components/ui/alert'
import { Button } from '@/components/ui/button'
import { Card, CardAction, CardContent, CardHeader, CardTitle } from
     '@/components/ui/card'
import { Skeleton } from '@/components/ui/skeleton'
import { Tabs, TabsContent, TabsList, TabsTrigger } from
     '@/components/ui/tabs'
import { Textarea } from '@/components/ui/textarea'
import Markdown from '@/components/Markdown'
import NotYetAvailable from './NotYetAvailable'
import { projectsApi } from '@/api/projects'
import { apiError } from '@/lib/project'

export default function ProjectDocEditor({ projectId }) {
  const [doc, setDoc] = useState(null)
  const [draft, setDraft] = useState('')
  const [saving, setSaving] = useState(false)

  const load = useCallback(async () => {
    try {
      const { data } = await projectsApi.document(projectId, 'doc')
```

```
    setDoc(data)
    setDraft(data.content || '')
  } catch (err) {
    toast.error(apiError(err, 'Could not load the project document'))
  }
}, [projectId])

useEffect(() => { load() }, [load])

if (!doc) return <Skeleton className="h-96" />
if (!doc.url) {
  return (
    <NotYetAvailable icon={FileText} title="No project document yet">
      The AI writes it when the Manager evaluates the project. You will
   edit it here, and saving it
      will re-plan the Gantt chart.
    </NotYetAvailable>
  )
}

const dirty = draft.trim() !== (doc.content || '').trim()

const save = async () => {
  setSaving(true)
  try {
    const { data } = await projectsApi.saveProjectDoc(projectId, draft)
    setDoc(data)
    setDraft(data.content)
    toast.success(data.changed
      ? 'Saved to the repository — the Gantt chart will be re-planned in
   about half a minute'
      : 'No changes to save')
  } catch (err) {
    toast.error(apiError(err, 'Could not save the project document'))
  } finally {
    setSaving(false)
  }
}

return (
  <Card id="project-doc">
    <CardHeader>
      <CardTitle className="text-base">Project document</CardTitle>
      <CardAction className="flex gap-2">
        <Button size="sm" variant="ghost" nativeButton={false}
          render={<a href={doc.url} target="_blank" rel="noopener
   noreferrer" />}>
          History on GitHub <ExternalLink />
        </Button>
        <Button size="sm" onClick={save} disabled={!dirty || saving}>
          {saving ? <Loader2 className="animate-spin" /> : <Save />} Save
        </Button>
      </CardAction>
    </CardHeader>
    <CardContent className="space-y-3">
      <Alert>
        <Info />
        <AlertDescription>
          Keep this current as features shift. Saving a change commits it
   to the repository and makes the AI
          re-plan the Gantt chart — it may re-estimate unfinished task
   groups and add groups for new features.
        </AlertDescription>
      </Alert>
      <Tabs defaultValue="edit">
        <TabsList>
          <TabsTrigger value="edit">Edit</TabsTrigger>
          <TabsTrigger value="preview">Preview</TabsTrigger>
```

```
          </TabsList>
          <TabsContent value="edit">
            <Textarea value={draft} onChange={(e) =>
    setDraft(e.target.value)} rows={24}
              className="font-mono text-xs" aria-label="Project document
    markdown" />
          </TabsContent>
          <TabsContent value="preview" className="rounded-lg border p-
    4"><Markdown>{draft}</Markdown></TabsContent>
        </Tabs>
      </CardContent>
    </Card>
  )
}
```

frontend/src/components/project/ProgressReport.jsx

```
import { useCallback, useEffect, useState } from 'react'
import { ExternalLink, FileBarChart, Loader2, RefreshCw, Sparkles } from
    'lucide-react'
import { toast } from 'sonner'
import { Button } from '@/components/ui/button'
import { Card, CardAction, CardContent, CardHeader, CardTitle } from
    '@/components/ui/card'
import { Skeleton } from '@/components/ui/skeleton'
import { Tabs, TabsContent, TabsList, TabsTrigger } from
    '@/components/ui/tabs'
import Markdown from '@/components/Markdown'
import NotYetAvailable from './NotYetAvailable'
import { projectsApi } from '@/api/projects'
import { apiError } from '@/lib/project'
import { waitForJob } from '@/lib/jobs'

function DocumentPanel({ doc, title }) {
  if (!doc) return <Skeleton className="h-64" />
  if (!doc.content) {
    return (
      <NotYetAvailable icon={FileBarChart} title={`No ${title.toLowerCase()}
    yet`}>
        It is written every Monday for active projects, or now with “Rewrite
    now”.
      </NotYetAvailable>
    )
  }
  return (
    <Card>
      <CardHeader>
        <CardTitle className="text-base">{title}</CardTitle>
        <CardAction>
          <Button size="sm" variant="ghost" nativeButton={false}
            render={<a href={doc.url} target="_blank" rel="noopener
    noreferrer" />}>
            On GitHub <ExternalLink />
          </Button>
        </CardAction>
      </CardHeader>
      <CardContent><Markdown>{doc.content}</Markdown></CardContent>
    </Card>
  )
}

export default function ProgressReport({ projectId, canRefresh }) {
  const [docs, setDocs] = useState({ report: null, progress: null })
  const [busy, setBusy] = useState(false)

  const load = useCallback(async () => {
    try {
      const [report, progress] = await Promise.all(
```

```
        ['report', 'progress'].map((k) => projectsApi.document(projectId,
    k)))
      setDocs({ report: report.data, progress: progress.data })
    } catch (err) {
      toast.error(apiError(err, 'Could not load the report'))
    }
  }, [projectId])

  useEffect(() => { load() }, [load])

  const rewrite = async () => {
    setBusy(true)
    try {
      const { data } = await projectsApi.refreshProgressReport(projectId)
      const result = await waitForJob(data.job_id)
      if (result.status === 'failed') toast.error(result.error)
      else toast.success('Progress document and report rewritten')
      load()
    } catch (err) {
      toast.error(apiError(err, err.message || 'Could not rewrite the
    report'))
    } finally {
      setBusy(false)
    }
  }

  return (
    <div id="progress-report" className="space-y-3">
      <div className="flex flex-wrap items-center justify-between gap-2">
        <p className="text-sm text-muted-foreground">Written by the AI from
    every task’s status and progress document.</p>
        <div className="flex gap-2">
          {canRefresh && (
            <Button size="sm" onClick={rewrite} disabled={busy}>
              {busy ? <Loader2 className="animate-spin" /> : <Sparkles />}
    {busy ? 'Writing…' : 'Rewrite now'}
            </Button>
          )}
          <Button size="sm" variant="outline" onClick={load}><RefreshCw />
    Refresh</Button>
        </div>
      </div>
      <Tabs defaultValue="report">
        <TabsList>
          <TabsTrigger value="report">Report</TabsTrigger>
          <TabsTrigger value="progress">Progress document</TabsTrigger>
        </TabsList>
        <TabsContent value="report"><DocumentPanel doc={docs.report}
    title="Project report" /></TabsContent>
        <TabsContent value="progress"><DocumentPanel doc={docs.progress}
    title="Progress document" /></TabsContent>
      </Tabs>
    </div>
  )
}
```

frontend/src/components/project/ReportDownload.jsx

```
import { useState } from 'react'
import { Download, Loader2 } from 'lucide-react'
import { toast } from 'sonner'
import { Button } from '@/components/ui/button'
import {
  Dialog, DialogContent, DialogDescription, DialogFooter, DialogHeader,
    DialogTitle,
} from '@/components/ui/dialog'
import { Input } from '@/components/ui/input'
import { Label } from '@/components/ui/label'
```

```
import { NativeSelect, NativeSelectOption } from '@/components/ui/native-
    select'
import { projectsApi } from '@/api/projects'
import { blobApiError, saveBlob } from '@/lib/project'

const iso = (d) => d.toISOString().slice(0, 10)

export default function ReportDownload({ projects = [] }) {
  const today = new Date()
  const [open, setOpen] = useState(false)
  const [range, setRange] = useState({
    start: iso(new Date(today.getFullYear(), today.getMonth(), 1)), end:
    iso(today),
  })
  const [projectId, setProjectId] = useState('')
  const [busy, setBusy] = useState(false)

  const download = async () => {
    setBusy(true)
    try {
      const { data } = await projectsApi.report(range.start, range.end,
    projectId || null)
      saveBlob(data, `trace-report-${range.start}-to-${range.end}.csv`)
      setOpen(false)
    } catch (err) {
      toast.error(await blobApiError(err, 'Could not build the report'))
    } finally {
      setBusy(false)
    }
  }

  return (
    <>
      <Button id="btn-download-report" variant="outline" onClick={() =>
    setOpen(true)}>
        <Download /> Download report
      </Button>
      <Dialog open={open} onOpenChange={setOpen}>
        <DialogContent>
          <DialogHeader>
            <DialogTitle>Download report</DialogTitle>
            <DialogDescription>
              Every task created, due or completed in the range, grouped by
    project and task group, with its
              status today and its group’s completion. A CSV file — it opens
    in any spreadsheet.
            </DialogDescription>
          </DialogHeader>
          <div className="grid gap-4 sm:grid-cols-2">
            <div className="space-y-2">
              <Label htmlFor="report-start">From</Label>
              <Input id="report-start" type="date" value={range.start}
    max={range.end}
                onChange={(e) => setRange({ ...range, start:
    e.target.value })} />
            </div>
            <div className="space-y-2">
              <Label htmlFor="report-end">To</Label>
              <Input id="report-end" type="date" value={range.end}
    min={range.start}
                onChange={(e) => setRange({ ...range, end:
    e.target.value })} />
            </div>
            <div className="space-y-2 sm:col-span-2">
              <Label htmlFor="report-project">Project</Label>
              <NativeSelect id="report-project" className="w-full"
    value={projectId}
                onChange={(e) => setProjectId(e.target.value)}>
```

```
              <NativeSelectOption value="">All my
    projects</NativeSelectOption>
              {projects.map((p) => <NativeSelectOption key={p.id}
    value={p.id}>{p.name}</NativeSelectOption>)}
            </NativeSelect>
          </div>
        </div>
        <DialogFooter>
          <Button variant="outline" onClick={() =>
    setOpen(false)}>Cancel</Button>
          <Button onClick={download} disabled={busy || !range.start || !
    range.end || range.end < range.start}>
            {busy ? <Loader2 className="animate-spin" /> : <Download />}
    Download CSV
          </Button>
        </DialogFooter>
      </DialogContent>
    </Dialog>
  </>
 )
}
```

frontend/src/components/project/NotYetAvailable.jsx

```
import { Empty, EmptyDescription, EmptyHeader, EmptyMedia, EmptyTitle } from
    '@/components/ui/empty'

export default function NotYetAvailable({ icon: Icon, title, children }) {
  return (
    <Empty className="border border-dashed">
      <EmptyHeader>
        {Icon && <EmptyMedia variant="icon"><Icon /></EmptyMedia>}
        <EmptyTitle>{title}</EmptyTitle>
        <EmptyDescription>{children}</EmptyDescription>
      </EmptyHeader>
    </Empty>
  )
}
```

4.1.13 Frontend: Pages

frontend/src/pages/LandingPage.jsx

```
import { useNavigate } from 'react-router-dom'
import {
  ArrowRight,
  Zap,
  GitBranch,
  BarChart3,
  ShieldCheck,
  Bot,
  FolderKanban,
  CheckCircle2,
  Users,
  FileText,
  Sparkles,
  Cpu,
  Bell,
} from 'lucide-react'
import { Button } from '@/components/ui/button'
import { Badge } from '@/components/ui/badge'
import { useTheme } from '@/contexts/ThemeContext'
import { Sun, Moon } from 'lucide-react'

const HOW_IT_WORKS = [
  {
    step: '01',
    icon: FileText,
```

```
    title: 'Manager uploads requirements',
    body: 'Drop in any combination of PDFs, Word docs, or markdown files.
     TRACE extracts the intent.',
  },
  {
    step: '02',
    icon: Bot,
    title: 'AI plans the project',
    body: 'The AI authors a living project document, generates task groups,
     assigns a Gantt chart, and ranks candidate Project Managers — all in one
     pipeline run.',
  },
  {
    step: '03',
    icon: GitBranch,
    title: 'The team delivers',
    body: 'Every task is a GitHub issue. Whoever does the work updates its
     status; completing it closes the issue and moves the Gantt.',
  },
]

const FEATURES = [
  {
    icon: Sparkles,
    title: 'AI Project Evaluation',
    body: 'Upload requirement documents and receive a structured project
     plan, task breakdown, and Gantt chart — generated end-to-end.',
  },
  {
    icon: FolderKanban,
    title: 'Living Project Documents',
    body: 'The project doc starts as what the AI understood. The PM refines
     it. Every edit triggers Gantt re-evaluation.',
  },
  {
    icon: CheckCircle2,
    title: 'Tasks Are GitHub Issues',
    body: 'Each project gets its own public repository. Creating a task
     opens an issue; completing it closes the issue.',
  },
  {
    icon: Users,
    title: 'AI-Ranked Assignee Suggestions',
    body: 'The AI scores employees on task throughput, on-time rate, and
     past subject matter — and shows its reasoning.',
  },
  {
    icon: BarChart3,
    title: 'Auto-Maintained Gantt',
    body: 'The chart regenerates on every status change. Each version is
     stamped with the reason it moved.',
  },
  {
    icon: Bell,
    title: 'Smart Notifications',
    body: 'Assignment, due-date, and completion events reach the right
     people in-app and via email — no polling required.',
  },
  {
    icon: ShieldCheck,
    title: 'Server-Enforced Authority',
    body: 'Only a task's assignee or its Project Manager can change its
     status — checked by the API, not just hidden in the UI.',
  },
  {
    icon: Cpu,
    title: 'Three-Role Panel Separation',
```

```
    body: 'Admin, Manager, and Employee each live in a fully isolated panel.
     Cross-panel access returns a 403 — never a silent redirect.',
  },
]

export default function LandingPage() {
  const navigate = useNavigate()
  const { mode, toggleMode } = useTheme()

  return (
    <div className="min-h-screen flex flex-col bg-background text-
     foreground">

      <nav className="sticky top-0 z-50 border-b bg-background/80 backdrop-
     blur supports-[backdrop-filter]:bg-background/60">
        <div className="mx-auto max-w-6xl px-6 flex h-14 items-center
     justify-between">
          <div className="flex items-center gap-2">
            <div className="flex size-7 items-center justify-center rounded-
     md bg-primary text-primary-foreground text-xs font-bold">
              T
            </div>
            <span className="font-semibold tracking-tight">TRACE</span>
          </div>

          <div className="flex items-center gap-2">
            <Button
              id="landing-btn-theme"
              variant="ghost"
              size="icon-sm"
              onClick={toggleMode}
              title={mode === 'dark' ? 'Light mode' : 'Dark mode'}
            >
              {mode === 'dark' ? <Sun className="size-4" /> : <Moon
     className="size-4" />}
            </Button>
            <Button
              id="landing-btn-sign-in"
              variant="outline"
              size="sm"
              onClick={() => navigate('/login')}
            >
              Sign in
            </Button>
            <Button
              id="landing-btn-get-started"
              size="sm"
              onClick={() => navigate('/login')}
            >
              Get started <ArrowRight className="size-4" />
            </Button>
          </div>
        </div>
      </nav>

      <section className="relative flex flex-col items-center justify-center
     text-center px-6 py-28 overflow-hidden">
        <div className="pointer-events-none absolute inset-0 overflow-
     hidden" aria-hidden>
          <div className="absolute -top-32 left-1/2 -translate-x-1/2 size-
     [600px] rounded-full bg-primary/10 blur-3xl" />
          <div className="absolute top-20 right-0 size-72 rounded-full bg-
     primary/5 blur-2xl" />
        </div>

        <div className="relative max-w-3xl space-y-6">
          <Badge variant="secondary" className="gap-1.5 px-3 py-1 text-xs">
            <Zap className="size-3" />
```

```
        AI-Driven Project Management
      </Badge>

      <h1 className="text-5xl md:text-6xl font-bold tracking-tight
leading-[1.1]">
        From requirements<br />
        to{' '}
        <span className="text-primary">delivered tasks</span>
        ,<br />
        automatically.
      </h1>

      <p className="text-lg text-muted-foreground max-w-2xl mx-auto
leading-relaxed">
        TRACE turns a folder of requirement documents into a structured
project plan, a
        living Gantt chart, and GitHub issues — then keeps them current
as code ships.
      </p>

      <div className="flex flex-col sm:flex-row items-center justify-
center gap-3 pt-2">
        <Button
          id="hero-btn-get-started"
          size="lg"
          className="gap-2 px-8"
          onClick={() => navigate('/login')}
        >
          Get started free
          <ArrowRight className="size-4" />
        </Button>
        <Button
          id="hero-btn-learn-more"
          size="lg"
          variant="outline"
          className="gap-2 px-8"
          onClick={() => document.getElementById('how-it-
works')?.scrollIntoView({ behavior: 'smooth' })}
        >
          See how it works
        </Button>
      </div>
    </div>
  </section>

  <section className="bg-muted/40 border-y px-6 py-20">
    <div className="mx-auto max-w-4xl text-center space-y-4">
      <h2 className="text-3xl font-bold tracking-tight">The problem with
project handoffs</h2>
      <p className="text-muted-foreground text-lg max-w-2xl mx-auto
leading-relaxed">
        Requirements live in PDFs. Plans live in spreadsheets. Tasks
live in Jira.
        Nobody knows if the code matches any of them — until it ships
and it doesn't.
      </p>
      <p className="text-muted-foreground text-base max-w-xl mx-auto">
        TRACE puts all three in one pipeline, connected by AI and
anchored to the same
        GitHub repository the team is pushing to.
      </p>
    </div>
  </section>

  <section id="how-it-works" className="px-6 py-24">
    <div className="mx-auto max-w-5xl">
      <div className="text-center space-y-3 mb-16">
```

```
        <h2 className="text-3xl font-bold tracking-tight">How TRACE
works</h2>
        <p className="text-muted-foreground">Three steps from brief to
verified delivery.</p>
      </div>

      <div className="grid md:grid-cols-3 gap-8">
        {HOW_IT_WORKS.map(({ step, icon: Icon, title, body }) => (
          <div
            key={step}
            className="relative flex flex-col items-start gap-4 rounded-
xl border bg-card p-6 shadow-sm hover:shadow-md transition-shadow"
          >
            <div className="flex items-center gap-3">
              <span className="text-4xl font-black text-primary/20
leading-none">{step}</span>
              <div className="flex size-9 items-center justify-center
rounded-lg bg-primary/10">
                <Icon className="size-4 text-primary" />
              </div>
            </div>
            <div className="space-y-1">
              <h3 className="font-semibold text-base">{title}</h3>
              <p className="text-sm text-muted-foreground leading-
relaxed">{body}</p>
            </div>
          </div>
        ))}
      </div>
    </div>
  </section>

  <section className="bg-muted/40 border-y px-6 py-24">
    <div className="mx-auto max-w-5xl">
      <div className="text-center space-y-3 mb-16">
        <h2 className="text-3xl font-bold tracking-tight">Everything in
one platform</h2>
        <p className="text-muted-foreground">
          From raw requirements to delivered, tracked work.
        </p>
      </div>

      <div className="grid sm:grid-cols-2 lg:grid-cols-4 gap-5">
        {FEATURES.map(({ icon: Icon, title, body }) => (
          <div
            key={title}
            className="flex flex-col gap-3 rounded-xl border bg-card p-5
hover:shadow-md transition-shadow"
          >
            <div className="flex size-9 items-center justify-center
rounded-lg bg-primary/10">
              <Icon className="size-4 text-primary" />
            </div>
            <h3 className="font-semibold text-sm">{title}</h3>
            <p className="text-xs text-muted-foreground leading-
relaxed">{body}</p>
          </div>
        ))}
      </div>
    </div>
  </section>

  <section className="px-6 py-24 text-center">
    <div className="mx-auto max-w-2xl space-y-6">
      <h2 className="text-3xl font-bold tracking-tight">
        Ready to automate your project lifecycle?
      </h2>
      <p className="text-muted-foreground text-lg">
```

```
            Sign in to your TRACE workspace and start turning requirements
    into results.
          </p>
          <Button
            id="cta-btn-get-started"
            size="lg"
            className="gap-2 px-10"
            onClick={() => navigate('/login')}
          >
            Get started
            <ArrowRight className="size-4" />
          </Button>
        </div>
      </section>

      <footer className="border-t bg-muted/30 px-6 py-8">
        <div className="mx-auto max-w-6xl flex flex-col sm:flex-row items-
    center justify-between gap-4 text-sm text-muted-foreground">
          <div className="flex items-center gap-2">
            <div className="flex size-5 items-center justify-center rounded
    bg-primary text-primary-foreground text-[10px] font-bold">
              T
            </div>
            <span className="font-medium text-foreground">TRACE</span>
            <span>— Task and Resource Attendance and Comprehensive
    Evaluation</span>
          </div>
          <p>© {new Date().getFullYear()} TRACE. All rights reserved.</p>
        </div>
      </footer>
    </div>
  )
}
```

frontend/src/pages/LoginPage.jsx

```
import { useState, useEffect } from 'react'
import { useNavigate, useLocation } from 'react-router-dom'
import { Eye, EyeOff, ArrowRight, Mail, Lock, AlertCircle } from 'lucide-
    react'
import { getRoleBase } from '@/components/GuestOnlyRoute'
import { Button } from '@/components/ui/button'
import { Input } from '@/components/ui/input'
import { Label } from '@/components/ui/label'
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from
    '@/components/ui/card'
import { Alert, AlertDescription } from '@/components/ui/alert'
import { Checkbox } from '@/components/ui/checkbox'
import { useAuth } from '@/contexts/AuthContext'

export default function LoginPage() {
  const [email, setEmail] = useState('')
  const [password, setPassword] = useState('')
  const [showPassword, setShowPassword] = useState(false)
  const [isLoading, setIsLoading] = useState(false)
  const [loginError, setLoginError] = useState(null)

  const { login, isAuthenticated, user, error: authError, setError } =
    useAuth()
  const navigate = useNavigate()
  const location = useLocation()

  useEffect(() => {
    if (isAuthenticated && user) {
      const from = location.state?.from?.pathname
      const base = getRoleBase(user)
      navigate(from || base, { replace: true })
    }
```

```
}, [isAuthenticated, user, navigate, location])

useEffect(() => {
  setError(null)
}, [setError])

const handleSubmit = async (e) => {
  e.preventDefault()
  setIsLoading(true)
  setLoginError(null)
  try {
    const userData = await login(email, password)
    const from = location.state?.from?.pathname
    const base = getRoleBase(userData)
    navigate(from || base, { replace: true })
  } catch (err) {
    setLoginError(err.message || 'Invalid email or password')
  } finally {
    setIsLoading(false)
  }
}

return (
  <div className="min-h-screen flex items-center justify-center bg-
   background px-4">
    <div className="w-full max-w-sm">
      <div className="text-center mb-8">
        <div className="inline-flex size-14 items-center justify-center
   rounded-xl bg-primary text-primary-foreground text-xl font-bold mb-4">
          T
        </div>
        <h1 className="text-2xl font-bold tracking-tight">Welcome to
   TRACE</h1>
        <p className="text-sm text-muted-foreground mt-1">Sign in to your
   TRACE panel</p>
      </div>

      {(loginError || authError) && (
        <Alert variant="destructive" className="mb-4">
          <AlertCircle className="size-4" />
          <AlertDescription>{loginError || authError}</AlertDescription>
        </Alert>
      )}

      <Card>
        <CardContent className="pt-6">
          <form onSubmit={handleSubmit} className="space-y-4">
            <div className="space-y-2">
              <Label htmlFor="login-email">Email Address</Label>
              <div className="relative">
                <Mail className="absolute left-2.5 top-1/2 -translate-y-
   1/2 size-4 text-muted-foreground" />
                <Input
                  id="login-email"
                  type="email"
                  placeholder="you@trace.com"
                  value={email}
                  onChange={(e) => setEmail(e.target.value)}
                  required
                  className="pl-9"
                />
              </div>
            </div>

            <div className="space-y-2">
              <Label htmlFor="login-password">Password</Label>
              <div className="relative">
```

```
                <Lock className="absolute left-2.5 top-1/2 -translate-y-
    1/2 size-4 text-muted-foreground" />
                <Input
                  id="login-password"
                  type={showPassword ? 'text' : 'password'}
                  placeholder="••••••••"
                  value={password}
                  onChange={(e) => setPassword(e.target.value)}
                  required
                  className="pl-9 pr-10"
                />
                <button
                  type="button"
                  onClick={() => setShowPassword(!showPassword)}
                  className="absolute right-2.5 top-1/2 -translate-y-1/2
    text-muted-foreground hover:text-foreground transition-colors"
                >
                  {showPassword ? <EyeOff className="size-4" /> : <Eye
    className="size-4" />}
                </button>
              </div>
            </div>

            <div className="flex items-center justify-between">
              <div className="flex items-center gap-2">
                <Checkbox id="remember" />
                <Label htmlFor="remember" className="text-xs font-normal
    cursor-pointer">
                  Remember me
                </Label>
              </div>
              <Button variant="link" size="xs" type="button"
    className="px-0 text-xs h-auto">
                Forgot password?
              </Button>
            </div>

            <Button
              id="btn-login"
              type="submit"
              disabled={isLoading}
              className="w-full"
            >
              {isLoading ? (
                <span className="size-4 border-2 border-primary-
    foreground/30 border-t-primary-foreground rounded-full animate-spin" />
              ) : (
                <>
                  Sign In <ArrowRight className="size-4" />
                </>
              )}
            </Button>
          </form>
        </CardContent>
      </Card>

      <p className="text-center text-xs text-muted-foreground mt-6">
        TRACE — AI-Powered Project Management
      </p>
    </div>
  </div>
  )
}
```

frontend/src/pages/ForbiddenPage.jsx

```
import { useNavigate } from 'react-router-dom'
import { ShieldX, ArrowLeft, LogOut } from 'lucide-react'
```

```
import { Button } from '@/components/ui/button'
import { useAuth } from '@/contexts/AuthContext'
import { getRoleBase } from '@/components/GuestOnlyRoute'

export default function ForbiddenPage() {
  const navigate = useNavigate()
  const { user, logout } = useAuth()

  const userRoles = user?.roles?.map((r) => r.role_name || r) ?? []
  const base = getRoleBase(user)

  return (
    <div
      id="forbidden-page"
      className="min-h-screen flex items-center justify-center bg-
    background"
    >
      <div className="text-center max-w-md px-6 space-y-6">
        <div className="inline-flex size-20 items-center justify-center
    rounded-2xl bg-destructive/10 text-destructive">
          <ShieldX className="size-10" />
        </div>

        <div className="space-y-2">
          <h1 className="text-4xl font-bold tracking-tight">403</h1>
          <p className="text-xl font-semibold">Access Forbidden</p>
          <p className="text-sm text-muted-foreground leading-relaxed">
            Your account does not have permission to view this panel.
            {userRoles.length > 0 && (
              <>
                {' '}
                You are logged in as{' '}
                <span className="font-medium text-foreground">
                  {userRoles.join(', ')}
                </span>
                .
              </>
            )}
          </p>
        </div>

        {user && (
          <div className="rounded-lg border bg-muted/40 p-4 text-left space-
    y-1">
            <p className="text-xs text-muted-foreground">Logged in as</p>
            <p className="text-sm font-medium">{user.name || user.username
    || user.email}</p>
            <p className="text-xs text-muted-foreground">{user.email}</p>
            <p className="text-xs text-muted-foreground">
              Roles: {userRoles.length > 0 ? userRoles.join(', ') : 'None
    assigned'}
            </p>
          </div>
        )}

        <div className="flex flex-col gap-2">
          <Button id="btn-go-to-panel" onClick={() => navigate(base,
    { replace: true })}>
            <ArrowLeft className="size-4" />
            Go to my panel
          </Button>
          <Button
            id="btn-forbidden-logout"
            variant="outline"
            onClick={() => {
              logout()
              navigate('/login', { replace: true })
            }}
```

```
          >
            <LogOut className="size-4" />
            Sign out
          </Button>
        </div>
      </div>
    </div>
  )
}
```

frontend/src/pages/NotFoundPage.jsx

```
import { Link } from 'react-router-dom'
import { Home, SearchX } from 'lucide-react'
import { Button } from '@/components/ui/button'

export default function NotFoundPage() {
  return (
    <div className="min-h-screen flex items-center justify-center bg-
    background px-4">
      <div className="text-center">
        <SearchX className="size-16 text-muted-foreground/30 mx-auto mb-
    6" />
        <h1 className="text-6xl font-bold text-muted-foreground/20 mb-
    2">404</h1>
        <p className="text-base text-muted-foreground mb-1">Page Not
    Found</p>
        <p className="text-sm text-muted-foreground/60 mb-8">
          The page you&apos;re looking for doesn&apos;t exist or has been
    moved.
        </p>
        <Link to="/dashboard">
          <Button>
            <Home className="size-4" /> Back to Dashboard
          </Button>
        </Link>
      </div>
    </div>
  )
}
```

frontend/src/pages/admin/ControlPanelPage.jsx

```
import { useState, useEffect, useCallback } from 'react'
import { toast } from 'sonner'
import {
  Database,
  Search,
  Plus,
  Edit2,
  Trash2,
  RefreshCw,
  Server,
  Layers,
  Table as TableIcon,
  ChevronLeft,
  ChevronRight,
  ShieldAlert,
  SlidersHorizontal,
  CheckCircle2,
  XCircle,
  Key,
} from 'lucide-react'
import { Card, CardContent, CardHeader, CardTitle, CardDescription } from
    '@/components/ui/card'
import { Button } from '@/components/ui/button'
import { Input } from '@/components/ui/input'
import { Label } from '@/components/ui/label'
```

```
import { Badge } from '@/components/ui/badge'
import { Dialog, DialogContent, DialogHeader, DialogTitle, DialogFooter,
     DialogDescription } from '@/components/ui/dialog'
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow }
     from '@/components/ui/table'
import { NativeSelect } from '@/components/ui/native-select'
import { adminApi } from '@/api'

export default function AdminPanelPage() {
  const [tables, setTables] = useState([])
  const [selectedTable, setSelectedTable] = useState(null)
  const [records, setRecords] = useState([])
  const [tableMeta, setTableMeta] = useState(null)
  const [loadingTables, setLoadingTables] = useState(true)
  const [loadingRecords, setLoadingRecords] = useState(false)

  const [searchTerm, setSearchTerm] = useState('')
  const [page, setPage] = useState(1)
  const [pageSize, setPageSize] = useState(20)
  const [totalRecords, setTotalRecords] = useState(0)
  const [totalPages, setTotalPages] = useState(1)
  const [sortBy, setSortBy] = useState('')
  const [sortOrder, setSortOrder] = useState('asc')

  const [isModalOpen, setIsModalOpen] = useState(false)
  const [editingRecord, setEditingRecord] = useState(null)
  const [formData, setFormData] = useState({})
  const [submitting, setSubmitting] = useState(false)

  const [deleteConfirmOpen, setDeleteConfirmOpen] = useState(false)
  const [recordToDelete, setRecordToDelete] = useState(null)

  const fetchTables = useCallback(async () => {
    setLoadingTables(true)
    try {
      const res = await adminApi.getTables()
      const fetchedTables = res.data?.tables || []
      setTables(fetchedTables)
      if (fetchedTables.length > 0 && !selectedTable) {
        setSelectedTable(fetchedTables[0].name)
      }
    } catch (err) {
      toast.error(err.response?.data?.detail || 'Failed to fetch database
     tables')
    } finally {
      setLoadingTables(false)
    }
  }, [selectedTable])

  useEffect(() => {
    fetchTables()
  }, [fetchTables])

  const fetchTableRecords = useCallback(async () => {
    if (!selectedTable) return
    setLoadingRecords(true)
    try {
      const res = await adminApi.getTableRecords(selectedTable, {
        page,
        page_size: pageSize,
        search: searchTerm,
        sort_by: sortBy || undefined,
        sort_order: sortOrder,
      })
      setRecords(res.data?.records || [])
      setTotalRecords(res.data?.total || 0)
      setTotalPages(res.data?.total_pages || 1)
      setTableMeta({
```

```
      columns: res.data?.columns || [],
      primaryKey: res.data?.primary_key || 'id',
      displayName: res.data?.display_name || selectedTable,
      category: res.data?.category || 'General',
    })
  } catch (err) {
    toast.error(err.response?.data?.detail || `Failed to fetch records for
   ${selectedTable}`)
  } finally {
    setLoadingRecords(false)
  }
}, [selectedTable, page, pageSize, searchTerm, sortBy, sortOrder])

useEffect(() => {
  fetchTableRecords()
}, [fetchTableRecords])

const categories = Array.from(new Set(tables.map((t) => t.category)))
const totalDbRecords = tables.reduce((sum, t) => sum + (t.total_records ||
   0), 0)

const handleSelectTable = (tblName) => {
  setSelectedTable(tblName)
  setPage(1)
  setSearchTerm('')
  setSortBy('')
}

const handleOpenCreateModal = () => {
  setEditingRecord(null)
  const initialForm = {}
  if (tableMeta?.columns) {
    tableMeta.columns.forEach((col) => {
      if (!col.primary_key && col.name !== 'created_at' && col.name !==
   'updated_at' && col.name !== 'deleted_at') {
        initialForm[col.name] = ''
      }
    })
  }
  setFormData(initialForm)
  setIsModalOpen(true)
}

const handleOpenEditModal = (rec) => {
  setEditingRecord(rec)
  const formValues = {}
  if (tableMeta?.columns) {
    tableMeta.columns.forEach((col) => {
      formValues[col.name] = rec[col.name] !== null && rec[col.name] !==
   undefined ? String(rec[col.name]) : ''
    })
  }
  setFormData(formValues)
  setIsModalOpen(true)
}

const handleSubmitForm = async (e) => {
  e.preventDefault()
  setSubmitting(true)
  try {
    if (editingRecord) {
      const pk = tableMeta?.primaryKey || 'id'
      const recId = editingRecord[pk]
      await adminApi.updateRecord(selectedTable, recId, formData)
      toast.success('Record updated successfully!')
    } else {
      await adminApi.createRecord(selectedTable, formData)
      toast.success('Record created successfully!')
```

```
    }
    setIsModalOpen(false)
    fetchTableRecords()
    fetchTables()
  } catch (err) {
    toast.error(err.response?.data?.detail || 'Operation failed')
  } finally {
    setSubmitting(false)
  }
}

const handleConfirmDelete = async () => {
  if (!recordToDelete || !tableMeta) return
  const pk = tableMeta.primaryKey || 'id'
  const recId = recordToDelete[pk]
  try {
    await adminApi.deleteRecord(selectedTable, recId)
    toast.success('Record deleted successfully')
    setDeleteConfirmOpen(false)
    setRecordToDelete(null)
    fetchTableRecords()
    fetchTables()
  } catch (err) {
    toast.error(err.response?.data?.detail || 'Failed to delete record')
  }
}

return (
  <div className="space-y-6">
    <div className="flex flex-col md:flex-row md:items-center justify-
   between gap-4 bg-card border border-border p-6 rounded-xl shadow-sm">
      <div className="space-y-1">
        <div className="flex items-center gap-2">
          <div className="p-2 bg-primary/10 text-primary rounded-lg">
            <Server className="size-6" />
          </div>
          <h1 className="text-2xl font-bold tracking-tight">Database Admin
   Panel</h1>
          <Badge variant="outline" className="border-primary/40 text-
   primary font-mono text-xs">
            Admin
          </Badge>
        </div>
        <p className="text-sm text-muted-foreground">
          Direct real-time CRUD access across all {tables.length} system
   database tables.
        </p>
      </div>

      <div className="flex items-center gap-4 text-xs">
        <div className="bg-muted/50 border border-border p-3 rounded-lg
   flex items-center gap-3">
          <TableIcon className="size-4 text-muted-foreground" />
          <div>
            <div className="text-muted-foreground">Total Tables</div>
            <div className="font-semibold text-sm">{tables.length}</div>
          </div>
        </div>

        <div className="bg-muted/50 border border-border p-3 rounded-lg
   flex items-center gap-3">
          <Layers className="size-4 text-muted-foreground" />
          <div>
            <div className="text-muted-foreground">Total Records</div>
            <div className="font-semibold text-
   sm">{totalDbRecords.toLocaleString()}</div>
          </div>
        </div>
```

```
   </div>
 </div>

 <div className="grid grid-cols-1 lg:grid-cols-12 gap-6">
   <Card className="lg:col-span-3 border-border shadow-sm h-fit">
     <CardHeader className="pb-3 border-b border-border">
       <CardTitle className="text-sm font-semibold flex items-center
justify-between">
         <span>Database Tables</span>
         <Button variant="ghost" size="icon" className="size-7"
onClick={fetchTables} title="Refresh tables list">
           <RefreshCw className={`size-3.5 ${loadingTables ? 'animate-
spin' : ''}`} />
         </Button>
       </CardTitle>
       <CardDescription className="text-xs">Select a table to inspect
or modify</CardDescription>
     </CardHeader>
     <CardContent className="p-2 space-y-4 max-h-[700px] overflow-y-
auto">
       {categories.map((cat) => {
         const categoryTables = tables.filter((t) => t.category ===
cat)
         return (
           <div key={cat} className="space-y-1">
             <div className="px-2 py-1 text-[11px] font-semibold text-
muted-foreground uppercase tracking-wider">
               {cat}
             </div>
             {categoryTables.map((t) => {
               const isSelected = selectedTable === t.name
               return (
                 <button
                   key={t.name}
                   onClick={() => handleSelectTable(t.name)}
                   className={`w-full flex items-center justify-between
px-3 py-2 text-xs rounded-lg transition-colors text-left font-medium ${
                     isSelected
                       ? 'bg-primary text-primary-foreground font-
semibold shadow-sm'
                       : 'hover:bg-accent hover:text-accent-foreground
text-foreground'
                   }`}
                 >
                   <span className="truncate">{t.display_name}</span>
                   <Badge
                     variant={isSelected ? 'secondary' : 'outline'}
                     className="ml-2 text-[10px] px-1.5 py-0 h-4 font-
mono"
                   >
                     {t.total_records}
                   </Badge>
                 </button>
               )
             })}
           </div>
         )
       })}
     </CardContent>
   </Card>

   <Card className="lg:col-span-9 border-border shadow-sm flex flex-
col">
     <CardHeader className="pb-4 border-b border-border">
       <div className="flex flex-col sm:flex-row sm:items-center
justify-between gap-4">
         <div>
           <div className="flex items-center gap-2">
```

```
              <CardTitle className="text-lg font-bold">
                {tableMeta?.displayName || selectedTable || 'Select
Table'}
              </CardTitle>
              <Badge variant="secondary" className="font-mono text-xs">
                {selectedTable}
              </Badge>
            </div>
            <CardDescription className="text-xs mt-1">
              {totalRecords} record{totalRecords === 1 ? '' : 's'} in
database table
            </CardDescription>
          </div>

          <div className="flex items-center gap-2">
            <Button size="sm" onClick={handleOpenCreateModal}
disabled={!selectedTable}>
              <Plus className="size-4 mr-1.5" /> Add Record
            </Button>
            <Button variant="outline" size="sm"
onClick={fetchTableRecords} disabled={loadingRecords}>
              <RefreshCw className={`size-4 mr-1 ${loadingRecords ?
'animate-spin' : ''}`} /> Refresh
            </Button>
          </div>
        </div>

        <div className="mt-4 flex flex-col sm:flex-row items-center gap-
3">
          <div className="relative flex-1 w-full">
            <Search className="absolute left-2.5 top-2.5 size-4 text-
muted-foreground" />
            <Input
              placeholder={`Search ${tableMeta?.displayName ||
'records'}...`}
              value={searchTerm}
              onChange={(e) => {
                setSearchTerm(e.target.value)
                setPage(1)
              }}
              className="pl-9 h-9 text-xs"
            />
          </div>

          <div className="flex items-center gap-2 w-full sm:w-auto text-
xs">
            <span className="text-muted-foreground
shrink-0">Rows:</span>
            <NativeSelect
              value={pageSize}
              onChange={(e) => {
                setPageSize(Number(e.target.value))
                setPage(1)
              }}
              className="h-9 text-xs w-20"
            >
              <option value={10}>10</option>
              <option value={20}>20</option>
              <option value={50}>50</option>
              <option value={100}>100</option>
            </NativeSelect>
          </div>
        </div>
      </CardHeader>

      <CardContent className="p-0 flex-1 overflow-x-auto">
        {loadingRecords ? (
```

```
          <div className="p-12 text-center text-muted-foreground space-
y-2">
            <RefreshCw className="size-6 animate-spin mx-auto text-
primary" />
            <p className="text-sm">Fetching table records...</p>
          </div>
        ) : records.length === 0 ? (
          <div className="p-12 text-center text-muted-foreground space-
y-2">
            <Database className="size-8 mx-auto opacity-40" />
            <p className="text-sm font-medium">No records found</p>
            <p className="text-xs text-muted-foreground">
              {searchTerm ? 'Try adjusting your search criteria.' :
'This table is currently empty.'}
            </p>
          </div>
        ) : (
          <Table>
            <TableHeader className="bg-muted/40">
              <TableRow>
                {tableMeta?.columns?.map((col) => (
                  <TableHead key={col.name} className="text-xs font-
semibold whitespace-nowrap">
                    <div className="flex items-center gap-1">
                      {col.primary_key && <Key className="size-3 text-
amber-500" />}
                      <span>{col.name}</span>
                    </div>
                  </TableHead>
                ))}
                <TableHead className="text-xs font-semibold text-right
sticky right-0 bg-muted/40 pr-4">
                  Actions
                </TableHead>
              </TableRow>
            </TableHeader>
            <TableBody>
              {records.map((rec, i) => (
                <TableRow key={rec[tableMeta?.primaryKey] || i}
className="hover:bg-muted/20 text-xs">
                  {tableMeta?.columns?.map((col) => {
                    const val = rec[col.name]
                    const isPk = col.primary_key
                    return (
                      <TableCell key={col.name} className="max-w-[200px]
truncate font-mono text-[11px]">
                        {isPk ? (
                          <Badge variant="outline" className="font-mono
text-[10px] border-amber-500/40 text-amber-600 bg-amber-500/5">
                            {val}
                          </Badge>
                        ) : val === null || val === undefined ? (
                          <span className="text-muted-foreground/40
italic">null</span>
                        ) : typeof val === 'boolean' ? (
                          val ? (
                            <Badge className="bg-emerald-500/10 text-
emerald-600 border-emerald-500/20 text-[10px]">true</Badge>
                          ) : (
                            <Badge variant="outline" className="text-
muted-foreground text-[10px]">false</Badge>
                          )
                        ) : (
                          <span>{String(val)}</span>
                        )}
                      </TableCell>
                    )
                  })}
```

```
                  <TableCell className="text-right whitespace-nowrap
sticky right-0 bg-card pr-4">
                    <div className="flex items-center justify-end gap-
1">
                      <Button variant="ghost" size="icon"
className="size-7 text-muted-foreground hover:text-foreground"
onClick={() => handleOpenEditModal(rec)} title="Edit Record">
                        <Edit2 className="size-3.5" />
                      </Button>
                      <Button
                        variant="ghost"
                        size="icon"
                        className="size-7 text-destructive hover:bg-
destructive/10"
                        onClick={() => {
                          setRecordToDelete(rec)
                          setDeleteConfirmOpen(true)
                        }}
                        title="Delete Record"
                      >
                        <Trash2 className="size-3.5" />
                      </Button>
                    </div>
                  </TableCell>
                </TableRow>
              ))}
            </TableBody>
          </Table>
        )}
      </CardContent>

      <div className="p-4 border-t border-border flex items-center
justify-between gap-4 text-xs">
        <span className="text-muted-foreground">
          Page {page} of {totalPages} ({totalRecords} records)
        </span>

        <div className="flex items-center gap-2">
          <Button
            variant="outline"
            size="sm"
            onClick={() => setPage((p) => Math.max(1, p - 1))}
            disabled={page <= 1 || loadingRecords}
          >
            <ChevronLeft className="size-4 mr-1" /> Previous
          </Button>
          <Button
            variant="outline"
            size="sm"
            onClick={() => setPage((p) => Math.min(totalPages, p + 1))}
            disabled={page >= totalPages || loadingRecords}
          >
            Next <ChevronRight className="size-4 ml-1" />
          </Button>
        </div>
      </div>
    </Card>
  </div>

  <Dialog open={isModalOpen} onOpenChange={setIsModalOpen}>
    <DialogContent className="max-w-2xl max-h-[85vh] overflow-y-auto">
      <DialogHeader>
        <DialogTitle className="flex items-center gap-2">
          {editingRecord ? <Edit2 className="size-5 text-primary" /> :
<Plus className="size-5 text-primary" />}
          <span>{editingRecord ? `Edit ${tableMeta?.displayName} Record`
: `Add Record to ${tableMeta?.displayName}`}</span>
```

```
        </DialogTitle>
        <DialogDescription>
          {editingRecord
            ? `Updating PK (${tableMeta?.primaryKey}): $
{editingRecord[tableMeta?.primaryKey]}`
            : 'Fill in the field values below to create a new database
row.'}
        </DialogDescription>
      </DialogHeader>

      <form onSubmit={handleSubmitForm} className="space-y-4 py-2">
        <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
          {tableMeta?.columns?.map((col) => {
            const isPk = col.primary_key
            const isAutoTimestamp = col.name === 'created_at' ||
col.name === 'updated_at'

            if (isPk && !editingRecord) {
              return null
            }

            return (
              <div key={col.name} className="space-y-1.5">
                <Label className="text-xs flex items-center justify-
between">
                  <span className="font-semibold">
                    {col.name}
                    {!col.nullable && !isPk && <span className="text-
destructive ml-1">*</span>}
                  </span>
                  <span className="text-[10px] text-muted-foreground
font-mono">{col.type}</span>
                </Label>

                {col.name === 'password' ? (
                  <Input
                    type="password"
                    placeholder="Set or update password"
                    value={formData[col.name] || ''}
                    onChange={(e) => setFormData({ ...formData,
[col.name]: e.target.value })}
                    className="text-xs"
                  />
                ) : (
                  <Input
                    disabled={isPk || isAutoTimestamp}
                    placeholder={col.nullable ? 'Optional' : 'Required'}
                    value={formData[col.name] ?? ''}
                    onChange={(e) => setFormData({ ...formData,
[col.name]: e.target.value })}
                    className="text-xs font-mono"
                  />
                )}
              </div>
            )
          })}
        </div>

        <DialogFooter className="pt-4 border-t border-border">
          <Button type="button" variant="outline" onClick={() =>
setIsModalOpen(false)}>
            Cancel
          </Button>
          <Button type="submit" disabled={submitting}>
            {submitting ? <RefreshCw className="size-4 animate-spin mr-
1.5" /> : null}
            {editingRecord ? 'Save Changes' : 'Create Record'}
          </Button>
```

```
            </DialogFooter>
          </form>
        </DialogContent>
      </Dialog>

      <Dialog open={deleteConfirmOpen} onOpenChange={setDeleteConfirmOpen}>
        <DialogContent className="max-w-md">
          <DialogHeader>
            <DialogTitle className="flex items-center gap-2 text-
    destructive">
              <ShieldAlert className="size-5" /> Confirm Record Deletion
            </DialogTitle>
            <DialogDescription className="pt-2">
              Are you sure you want to delete this record from table <strong
    className="text-foreground font-mono">{selectedTable}</strong>?
            </DialogDescription>
          </DialogHeader>

          {recordToDelete && (
            <div className="bg-muted/50 p-3 rounded-lg border border-border
    text-xs font-mono max-h-36 overflow-y-auto space-y-1">
              <div><strong className="text-foreground font-
    semibold">{tableMeta?.primaryKey}:</strong>
    {recordToDelete[tableMeta?.primaryKey]}</div>
              {Object.entries(recordToDelete).slice(0, 5).map(([k, v]) => (
                <div key={k} className="truncate"><span className="text-
    muted-foreground">{k}:</span> {String(v)}</div>
              ))}
            </div>
          )}

          <DialogFooter className="pt-4">
            <Button variant="outline" onClick={() =>
    setDeleteConfirmOpen(false)}>
              Cancel
            </Button>
            <Button variant="destructive" onClick={handleConfirmDelete}>
              Confirm Delete
            </Button>
          </DialogFooter>
        </DialogContent>
      </Dialog>
    </div>
  )
}
```

frontend/src/pages/admin/WorkforcePage.jsx

```
import { useCallback, useEffect, useMemo, useRef, useState } from 'react'
import { Download, Loader2, Pencil, Plus, RefreshCw, Search, Trash2,
    Upload } from 'lucide-react'
import { toast } from 'sonner'
import { Badge } from '@/components/ui/badge'
import { Button } from '@/components/ui/button'
import { Card, CardContent } from '@/components/ui/card'
import {
  Dialog, DialogContent, DialogDescription, DialogFooter, DialogHeader,
    DialogTitle,
} from '@/components/ui/dialog'
import {
  AlertDialog, AlertDialogAction, AlertDialogCancel, AlertDialogContent,
    AlertDialogDescription,
  AlertDialogFooter, AlertDialogHeader, AlertDialogTitle,
} from '@/components/ui/alert-dialog'
import { Input } from '@/components/ui/input'
import { Label } from '@/components/ui/label'
import { NativeSelect, NativeSelectOption } from '@/components/ui/native-
    select'
```

```
import { Skeleton } from '@/components/ui/skeleton'
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow }
    from '@/components/ui/table'
import { Tabs, TabsContent, TabsList, TabsTrigger } from
    '@/components/ui/tabs'
import { workforceApi } from '@/api/workforce'
import { apiError, blobApiError, saveBlob } from '@/lib/project'

const INPUT_TYPE = { email: 'email', password: 'password', date: 'date',
    time: 'time', int: 'number' }

function RecordDialog({ tab, record, onClose, onSaved }) {
  const editing = Boolean(record?.id)
  const fields = tab.fields.filter((f) => !f.read_only && !(editing &&
    f.create_only))
  const [form, setForm] = useState(() =>
    Object.fromEntries(fields.map((f) => [f.name, record?.[f.name] ??
    (f.type === 'select' && f.required ? f.options[0] : '')])))
  const [saving, setSaving] = useState(false)

  const save = async (e) => {
    e.preventDefault()
    setSaving(true)
    try {
      if (editing) await workforceApi.update(tab.key, record.id, form)
      else await workforceApi.create(tab.key, form)
      toast.success(editing ? 'Saved' : 'Added')
      onSaved()
    } catch (err) {
      toast.error(apiError(err, 'Could not save'))
    } finally {
      setSaving(false)
    }
  }

  return (
    <Dialog open onOpenChange={(open) => !open && onClose()}>
      <DialogContent className="sm:max-w-lg">
        <form onSubmit={save} className="space-y-4">
          <DialogHeader>
            <DialogTitle>{editing ? 'Edit' : 'Add'} —
    {tab.label}</DialogTitle>
            <DialogDescription>Fields marked * are
    required.</DialogDescription>
          </DialogHeader>
          <div className="grid max-h-[60vh] gap-3 overflow-y-auto sm:grid-
    cols-2">
            {fields.map((f) => (
              <div key={f.name} className="space-y-1.5">
                <Label htmlFor={`wf-${f.name}`}>{f.label}{f.required && '
    *'}</Label>
                {f.type === 'select' ? (
                  <NativeSelect id={`wf-${f.name}`} className="w-full"
    value={form[f.name] ?? ''}
                    onChange={(e) => setForm({ ...form, [f.name]:
    e.target.value })}>
                    {!f.required && <NativeSelectOption
    value="">—</NativeSelectOption>}
                    {f.options.map((o) => <NativeSelectOption key={o}
    value={o}>{o}</NativeSelectOption>)}
                  </NativeSelect>
                ) : (
                  <Input id={`wf-${f.name}`} type={INPUT_TYPE[f.type] ||
    'text'} value={form[f.name] ?? ''}
                    required={f.required} autoComplete="off"
                    onChange={(e) => setForm({ ...form, [f.name]:
    e.target.value })} />
                )}
```

```
              </div>
            ))}
          </div>
          <DialogFooter>
            <Button type="button" variant="outline"
    onClick={onClose}>Cancel</Button>
            <Button type="submit" disabled={saving}>{saving && <Loader2
    className="animate-spin" />} Save</Button>
          </DialogFooter>
        </form>
      </DialogContent>
    </Dialog>
  )
}

function BulkResult({ result, onClose }) {
  return (
    <Dialog open onOpenChange={(open) => !open && onClose()}>
      <DialogContent className="sm:max-w-lg">
        <DialogHeader>
          <DialogTitle>Bulk upload</DialogTitle>
          <DialogDescription>
            {result.created} row{result.created === 1 ? '' : 's'} added
            {result.rejected.length > 0 && `, ${result.rejected.length}
    rejected — fix them and upload those rows again`}.
          </DialogDescription>
        </DialogHeader>
        {result.ignored_columns?.length > 0 && (
          <p className="text-sm text-muted-foreground">Ignored columns:
    {result.ignored_columns.join(', ')}</p>
        )}
        {result.rejected.length > 0 && (
          <div className="max-h-72 overflow-y-auto rounded-md border">
            <Table>
              <TableHeader><TableRow><TableHead
    className="w-16">Line</TableHead><TableHead>Reason</TableHead></TableRow
    ></TableHeader>
              <TableBody>
                {result.rejected.map((r) => (
                  <TableRow
    key={r.line}><TableCell>{r.line}</TableCell><TableCell
    className="whitespace-normal">{r.error}</TableCell></TableRow>
                ))}
              </TableBody>
            </Table>
          </div>
        )}
        <DialogFooter><Button onClick={onClose}>Done</Button></DialogFooter>
      </DialogContent>
    </Dialog>
  )
}

function WorkforceTab({ tab }) {
  const [rows, setRows] = useState(null)
  const [query, setQuery] = useState('')
  const [editing, setEditing] = useState(null)
  const [deleting, setDeleting] = useState(null)
  const [bulk, setBulk] = useState(null)
  const [uploading, setUploading] = useState(false)
  const fileRef = useRef(null)
  const columns = tab.fields.filter((f) => f.list)

  const load = useCallback(async () => {
    try {
      setRows((await workforceApi.list(tab.key)).data)
    } catch (err) {
```

```
    toast.error(apiError(err, `Could not load $
   {tab.label.toLowerCase()}`))
    setRows([])
  }
}, [tab])

useEffect(() => { load() }, [load])

const shown = useMemo(() => {
  const q = query.trim().toLowerCase()
  if (!q || !rows) return rows
  return rows.filter((r) => columns.some((c) => String(r[c.name] ??
   '').toLowerCase().includes(q)))
}, [rows, query, columns])

const template = async () => {
  try {
    saveBlob((await workforceApi.template(tab.key)).data, `${tab.key}-
   template.csv`)
  } catch (err) {
    toast.error(await blobApiError(err, 'Could not download the
   template'))
  }
}

const upload = async (file) => {
  if (!file) return
  setUploading(true)
  try {
    setBulk((await workforceApi.bulk(tab.key, file)).data)
    load()
  } catch (err) {
    toast.error(apiError(err, 'Could not read the file'))
  } finally {
    setUploading(false)
    if (fileRef.current) fileRef.current.value = ''
  }
}

const remove = async () => {
  try {
    await workforceApi.remove(tab.key, deleting.id)
    toast.success('Deleted')
    load()
  } catch (err) {
    toast.error(apiError(err, 'Could not delete'))
  } finally {
    setDeleting(null)
  }
}

return (
  <Card>
    <CardContent className="space-y-4">
      <div className="flex flex-wrap items-center gap-2">
        <div className="relative w-full sm:w-64">
          <Search className="absolute left-2.5 top-1/2 size-4 -translate-
   y-1/2 text-muted-foreground" />
          <Input value={query} onChange={(e) => setQuery(e.target.value)}
   placeholder="Filter…" className="pl-8" />
        </div>
        <div className="ml-auto flex flex-wrap gap-2">
          <Button variant="outline" size="icon" onClick={load} aria-
   label="Refresh"><RefreshCw /></Button>
          <Button variant="outline" onClick={template}><Download /> CSV
   template</Button>
          <Button variant="outline" onClick={() =>
   fileRef.current?.click()} disabled={uploading}>
```

```
          {uploading ? <Loader2 className="animate-spin" /> : <Upload
/>} Bulk upload
        </Button>
        <input ref={fileRef} type="file" accept=".csv,text/csv"
className="hidden"
          onChange={(e) => upload(e.target.files?.[0])} />
        <Button onClick={() => setEditing({})}><Plus /> Add</Button>
      </div>
    </div>

    {shown === null ? (
      <div className="space-y-2"><Skeleton className="h-9" /><Skeleton
className="h-9" /><Skeleton className="h-9" /></div>
    ) : shown.length === 0 ? (
      <p className="py-8 text-center text-sm text-muted-foreground">
        {rows.length ? 'Nothing matches the filter.' : `No $
{tab.label.toLowerCase()} yet — add one, or bulk upload a CSV.`}
      </p>
    ) : (
      <div className="overflow-x-auto rounded-md border">
        <Table>
          <TableHeader>
            <TableRow>
              {columns.map((c) => <TableHead
key={c.name}>{c.label}</TableHead>)}
              <TableHead className="w-24" />
            </TableRow>
          </TableHeader>
          <TableBody>
            {shown.map((r) => (
              <TableRow key={r.id}>
                {columns.map((c) => (
                  <TableCell key={c.name}>
                    {c.name === 'status' && r[c.name]
                      ? <Badge variant={['Active',
'Approved'].includes(r[c.name]) ? 'secondary' :
'outline'}>{r[c.name]}</Badge>
                      : (r[c.name] ?? '—')}
                  </TableCell>
                ))}
                <TableCell className="text-right">
                  <Button variant="ghost" size="icon-sm" onClick={() =>
setEditing(r)} aria-label="Edit"><Pencil /></Button>
                  <Button variant="ghost" size="icon-sm" onClick={() =>
setDeleting(r)} aria-label="Delete"><Trash2 /></Button>
                </TableCell>
              </TableRow>
            ))}
          </TableBody>
        </Table>
      </div>
    )}
    <p className="text-xs text-muted-foreground">
      Bulk upload takes a UTF-8 CSV with the template's columns;
references use codes and names
      (an employee's code, a department's name). Good rows are added and
every bad row is listed with its reason.
    </p>
  </CardContent>

  {editing && <RecordDialog tab={tab} record={editing} onClose={() =>
setEditing(null)}
    onSaved={() => { setEditing(null); load() }} />}
  {bulk && <BulkResult result={bulk} onClose={() => setBulk(null)} />}
  <AlertDialog open={Boolean(deleting)} onOpenChange={(open) => !open &&
setDeleting(null)}>
    <AlertDialogContent>
      <AlertDialogHeader>
```

```
            <AlertDialogTitle>Delete this record?</AlertDialogTitle>
            <AlertDialogDescription>
              {['employees', 'managers'].includes(tab.key)
                ? 'The account is deactivated and removed from the list; its
    history stays.'
                : 'This cannot be undone.'}
            </AlertDialogDescription>
          </AlertDialogHeader>
          <AlertDialogFooter>
            <AlertDialogCancel>Cancel</AlertDialogCancel>
            <AlertDialogAction onClick={remove}>Delete</AlertDialogAction>
          </AlertDialogFooter>
        </AlertDialogContent>
      </AlertDialog>
    </Card>
  )
}

export default function WorkforcePage() {
  const [tabs, setTabs] = useState(null)

  useEffect(() => {
    workforceApi.catalogue().then((r) => setTabs(r.data)).catch((err) => {
      toast.error(apiError(err, 'Could not load Workforce'))
      setTabs([])
    })
  }, [])

  return (
    <div id="workforce-page" className="space-y-6 max-w-[1400px]">
      <div>
        <h1 className="text-2xl font-bold tracking-tight">Workforce</h1>
        <p className="text-sm text-muted-foreground">
          The people and HR records TRACE plans with — kept here by the
    Admin, typically bulk-imported.
        </p>
      </div>
      {tabs === null ? <Skeleton className="h-96" /> : tabs.length > 0 && (
        <Tabs defaultValue={tabs[0].key}>
          <TabsList className="flex-wrap h-auto">
            {tabs.map((t) => <TabsTrigger key={t.key}
    value={t.key}>{t.label}</TabsTrigger>)}
          </TabsList>
          {tabs.map((t) => <TabsContent key={t.key}
    value={t.key}><WorkforceTab tab={t} /></TabsContent>)}
        </Tabs>
      )}
    </div>
  )
}
```

frontend/src/pages/admin/MonitorPage.jsx

```
import { useState, useEffect, useCallback } from 'react'
import {
  RefreshCw,
  CheckCircle,
  AlertCircle,
  AlertTriangle,
  Database,
  Cloud,
  Brain,
  Server,
  Activity,
  Mail,
  MessageSquare,
  Clock,
  ListChecks,
```

```
} from 'lucide-react'
import { Button } from '@/components/ui/button'
import { Badge } from '@/components/ui/badge'
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from
     '@/components/ui/card'
import { Table, TableBody, TableCell, TableHead, TableHeader, TableRow }
     from '@/components/ui/table'
import { formatDateTime } from '@/lib/project'
import servicesApi from '@/api/services'

function StatusBadge({ status }) {
  const map = {
    healthy: { variant: 'default', label: 'Healthy', Icon: CheckCircle },
    unhealthy: { variant: 'destructive', label: 'Unhealthy', Icon:
     AlertCircle },
    configured: { variant: 'secondary', label: 'Configured', Icon:
     CheckCircle },
    not_configured: { variant: 'outline', label: 'Not Configured', Icon:
     AlertTriangle },
    loading: { variant: 'secondary', label: 'Checking…', Icon: RefreshCw },
  }
  const s = map[status] || map.loading
  return (
    <Badge variant={s.variant} className="gap-1">
      <s.Icon className={`size-3 ${status === 'loading' ? 'animate-spin' :
     ''}`} />
      {s.label}
    </Badge>
  )
}

const SERVICE_CARDS = [
  { key: 'api',      label: 'API',             Icon: Server },
  { key: 'database', label: 'PostgreSQL',       Icon: Database },
  { key: 'redis',    label: 'Redis',            Icon: Activity },
  { key: 'celery',   label: 'Celery Worker',    Icon: Activity },
  { key: 'beat',     label: 'Celery Beat',      Icon: Clock },
  { key: 'minio',    label: 'MinIO',            Icon: Cloud },
  { key: 'mailpit',  label: 'Mailpit',          Icon: Mail },
  { key: 'llm',      label: 'LLM',              Icon: Brain },
  { key: 'websocket',label: 'WebSocket',        Icon: MessageSquare },
]

export default function MonitorPage() {
  const [statuses, setStatuses] = useState(
    Object.fromEntries(SERVICE_CARDS.map((s) => [s.key, 'loading']))
  )
  const [lastChecked, setLastChecked] = useState(null)
  const [checking, setChecking] = useState(false)

  const runChecks = useCallback(async () => {
    setChecking(true)
    try {
      const s = (await servicesApi.getAllHealth()).data.services || {}
      setStatuses({
        api:       s.api?.status ?? 'unhealthy',
        database:  s.database?.status ?? 'unhealthy',
        redis:     s.redis?.status ?? 'unhealthy',
        celery:    s.celery_worker?.status ?? 'unhealthy',
        beat:      s.celery_beat?.status ?? 'unhealthy',
        minio:     s.minio?.status ?? 'unhealthy',
        mailpit:   s.mailpit?.status ?? 'unhealthy',
        llm:       s.llm?.status ?? 'not_configured',
        websocket: s.websocket?.status ?? 'unhealthy',
      })
    } catch {
      setStatuses(Object.fromEntries(SERVICE_CARDS.map((s) => [s.key,
     'unhealthy'])))
```

```
    } finally {
      setChecking(false)
      setLastChecked(new Date())
    }
  }, [])

  useEffect(() => { runChecks() }, [runChecks])

  const healthy = Object.values(statuses).filter((s) => s === 'healthy' || s
     === 'configured').length

  return (
    <div id="monitor-page" className="space-y-6 max-w-[1400px]">
      <div className="flex items-center justify-between">
        <div>
          <h1 className="text-2xl font-bold tracking-tight">Monitor</h1>
          <p className="text-sm text-muted-foreground">
            Server and service health — {healthy}/{SERVICE_CARDS.length}
     healthy
            {lastChecked && (
              <span className="ml-2 text-xs">
                · Last checked {lastChecked.toLocaleTimeString()}
              </span>
            )}
          </p>
        </div>
        <Button
          id="btn-monitor-refresh"
          variant="outline"
          size="icon"
          onClick={runChecks}
          disabled={checking}
        >
          <RefreshCw className={checking ? 'animate-spin' : ''} />
        </Button>
      </div>

      <div className="grid grid-cols-2 md:grid-cols-4 gap-4">
        {SERVICE_CARDS.map(({ key, label, Icon }) => (
          <Card key={key} className="transition-shadow hover:shadow-md">
            <CardHeader className="pb-2 pt-4 px-4">
              <div className="flex items-center justify-between">
                <div className="flex items-center gap-2">
                  <Icon className="size-4 text-muted-foreground" />
                  <CardTitle className="text-sm
     font-medium">{label}</CardTitle>
                </div>
              </div>
            </CardHeader>
            <CardContent className="pb-4 px-4">
              <StatusBadge status={statuses[key]} />
            </CardContent>
          </Card>
        ))}
      </div>

      <RecentJobs key={lastChecked?.getTime()} />
    </div>
  )
}

const OUTCOME_VARIANT = { SUCCESS: 'secondary', FAILED: 'destructive',
     FAILURE: 'destructive', ERROR: 'destructive' }

function RecentJobs() {
  const [jobs, setJobs] = useState(null)
  useEffect(() => {
```

```
    servicesApi.recentJobs(30).then((r) => setJobs(r.data.jobs)).catch(() =>
     setJobs([]))
  }, [])
  return (
    <Card id="recent-jobs">
      <CardHeader>
        <CardTitle className="flex items-center gap-2 text-base"><ListChecks
     className="size-4" /> Recent jobs</CardTitle>
        <CardDescription>The last 30 finished Celery jobs (results are kept
     for a day).</CardDescription>
      </CardHeader>
      <CardContent>
        {jobs === null ? <p className="text-sm text-muted-
     foreground">Loading…</p> : jobs.length === 0 ? (
          <p className="text-sm text-muted-foreground">No finished jobs yet
     — or the worker has not run since results expired.</p>
        ) : (
          <div className="overflow-x-auto rounded-md border">
            <Table>
              <TableHeader>

     <TableRow><TableHead>Job</TableHead><TableHead>Outcome</TableHead><Table
     Head>Finished</TableHead><TableHead>Detail</TableHead></TableRow>
              </TableHeader>
              <TableBody>
                {jobs.map((j) => (
                  <TableRow key={j.id}>
                    <TableCell className="font-mono
     text-xs">{j.name}</TableCell>
                    <TableCell><Badge variant={OUTCOME_VARIANT[j.outcome] ||
     'outline'}>{j.outcome}</Badge></TableCell>
                    <TableCell className="text-
     xs">{formatDateTime(j.finished_at)}</TableCell>
                    <TableCell className="max-w-md truncate text-xs text-
     muted-foreground" title={j.detail || ''}>{j.detail || '—'}</TableCell>
                  </TableRow>
                ))}
              </TableBody>
            </Table>
          </div>
        )}
      </CardContent>
    </Card>
  )
}
```

frontend/src/pages/admin/ConfigurationPage.jsx

```
import { useCallback, useEffect, useState } from 'react'
import { Brain, CheckCircle, GitBranch, KeyRound, Loader2, Mail, PlugZap,
     RefreshCw, Save, XCircle } from 'lucide-react'
import { toast } from 'sonner'
import { Alert, AlertDescription, AlertTitle } from '@/components/ui/alert'
import { Badge } from '@/components/ui/badge'
import { Button } from '@/components/ui/button'
import { Card, CardContent, CardDescription, CardFooter, CardHeader,
     CardTitle } from '@/components/ui/card'
import { Input } from '@/components/ui/input'
import { Label } from '@/components/ui/label'
import { NativeSelect, NativeSelectOption } from '@/components/ui/native-
     select'
import { Skeleton } from '@/components/ui/skeleton'
import { settingsApi } from '@/api/settings'
import servicesApi from '@/api/services'
import { apiError } from '@/lib/project'

const TOKEN_SOURCE = { database: 'Set here', env: 'Set in .env' }
```

```
function TestResult({ result, okNote }) {
  if (!result) return null
  return (
    <Alert variant={result.ok ? 'default' : 'destructive'}>
      {result.ok ? <CheckCircle /> : <XCircle />}
      <AlertTitle>{result.ok ? 'Connection works' : 'Connection
     failed'}</AlertTitle>
      <AlertDescription>
        <ul className="space-y-1">
          {result.checks.map((c) => (
            <li key={c.name}><span className="font-medium">{c.name}:</span>
     {c.detail}</li>
          ))}
        </ul>
        {result.ok && okNote && <p className="mt-1">{okNote}</p>}
      </AlertDescription>
    </Alert>
  )
}

const MAIL_FIELDS = ['host', 'port', 'security', 'username', 'from_address']

function MailSection() {
  const [saved, setSaved] = useState(null)
  const [form, setForm] = useState(Object.fromEntries(MAIL_FIELDS.map((f) =>
     [f, '']))
  const [password, setPassword] = useState('')
  const [saving, setSaving] = useState(false)
  const [testing, setTesting] = useState(false)
  const [result, setResult] = useState(null)

  const load = useCallback(async () => {
    try {
      const { data } = await settingsApi.mail()
      setSaved(data)
      setForm(Object.fromEntries(MAIL_FIELDS.map((f) => [f, data[f] ??
     '']))
    } catch (err) {
      toast.error(apiError(err, 'Could not load mail settings'))
    }
  }, [])

  useEffect(() => { load() }, [load])

  const save = async (extra = {}) => {
    setSaving(true)
    try {
      const payload = { ...form, port: form.port ? Number(form.port) : null,
     ...extra }
      if (password) payload.password = password
      const { data } = await settingsApi.saveMail(payload)
      setSaved(data)
      setPassword('')
      setResult(null)
      toast.success('Mail settings saved — the next email uses them')
    } catch (err) {
      toast.error(apiError(err, 'Could not save'))
    } finally {
      setSaving(false)
    }
  }

  const test = async () => {
    setTesting(true)
    try {
      setResult((await settingsApi.testMail()).data)
    } catch (err) {
      toast.error(apiError(err, 'Could not run the test'))
```

```
  } finally {
    setTesting(false)
  }
}

const dirty = saved && (password || MAIL_FIELDS.some((k) => String(form[k]
   ?? '') !== String(saved[k] ?? '')))
const set = (key) => (e) => setForm({ ...form, [key]: e.target.value })

return (
  <Card id="config-mail">
    <CardHeader>
      <div className="flex items-center gap-2">
        <Mail className="size-4 text-muted-foreground" />
        <CardTitle className="text-base">Mail</CardTitle>
        {saved && <Badge variant={saved.host ? 'secondary' :
   'outline'}>{saved.host ? 'Configured' : 'Not configured'}</Badge>}
      </div>
      <CardDescription>
        The SMTP server for notification emails. Empty fields fall back to
   <code className="font-mono text-xs">.env</code> (Mailpit locally).
      </CardDescription>
    </CardHeader>
    {!saved ? (
      <CardContent className="space-y-3"><Skeleton className="h-9 w-
   full" /><Skeleton className="h-9 w-2/3" /></CardContent>
    ) : (
      <CardContent className="space-y-4">
        <div className="grid gap-4 sm:grid-cols-2">
          <div className="space-y-2">
            <Label htmlFor="mail-host">SMTP host</Label>
            <Input id="mail-host" value={form.host}
   placeholder="smtp.acme.com" onChange={set('host')} />
          </div>
          <div className="grid grid-cols-2 gap-4">
            <div className="space-y-2">
              <Label htmlFor="mail-port">Port</Label>
              <Input id="mail-port" type="number" min={1} max={65535}
   value={form.port} onChange={set('port')} />
            </div>
            <div className="space-y-2">
              <Label htmlFor="mail-security">Security</Label>
              <NativeSelect id="mail-security" className="w-full"
   value={form.security} onChange={set('security')}>
                <NativeSelectOption value="none">None</NativeSelectOption>
                <NativeSelectOption
   value="starttls">STARTTLS</NativeSelectOption>
                <NativeSelectOption
   value="ssl">SSL/TLS</NativeSelectOption>
              </NativeSelect>
            </div>
          </div>
          <div className="space-y-2">
            <Label htmlFor="mail-user">Username</Label>
            <Input id="mail-user" value={form.username} placeholder="Empty
   — no login" autoComplete="off" onChange={set('username')} />
          </div>
          <div className="space-y-2">
            <div className="flex items-center justify-between">
              <Label htmlFor="mail-password">Password</Label>
              <span className="text-xs text-muted-
   foreground">{saved.password_set ? TOKEN_SOURCE[saved.password_source] :
   'Not set'}</span>
            </div>
            <Input id="mail-password" type="password" autoComplete="off"
   value={password}
              placeholder={saved.password_set ? '•••••••••••• (unchanged)'
   : ''} onChange={(e) => setPassword(e.target.value)} />
```

```
            <p className="text-xs text-muted-foreground">Stored encrypted
    and never shown again.</p>
          </div>
          <div className="space-y-2">
            <Label htmlFor="mail-from">From address</Label>
            <Input id="mail-from" type="email" value={form.from_address}
    placeholder="noreply@acme.com" onChange={set('from_address')} />
          </div>
        </div>
        <TestResult result={result} okNote="The test only connects;
    nothing was sent." />
      </CardContent>
    )}
    <CardFooter className="flex-wrap justify-between gap-2 border-t">
      <Button variant="outline" onClick={test} disabled={!saved || testing
    || dirty}
        title={dirty ? 'Save first — the test uses the saved settings' :
    undefined}>
        {testing ? <Loader2 className="animate-spin" /> : <PlugZap />}
    Test connection
      </Button>
      <div className="flex gap-2">
        {saved?.password_source === 'database' && (
          <Button variant="ghost" onClick={() => save({ password: '' })}
    disabled={saving}><KeyRound /> Clear saved password</Button>
        )}
        <Button id="btn-save-mail" onClick={() => save()} disabled={!dirty
    || saving}>
          {saving ? <Loader2 className="animate-spin" /> : <Save />} Save
        </Button>
      </div>
    </CardFooter>
  </Card>
 )
}

function AiSection() {
  const [llm, setLlm] = useState(null)
  useEffect(() => { servicesApi.getLlmHealth().then((r) =>
     setLlm(r.data)).catch(() => setLlm({})) }, [])
  return (
    <Card id="config-ai">
      <CardHeader>
        <div className="flex items-center gap-2">
          <Brain className="size-4 text-muted-foreground" />
          <CardTitle className="text-base">AI</CardTitle>
          {llm && <Badge variant={llm.status === 'configured' ?
    'secondary' : 'outline'}>
            {llm.status === 'configured' ? 'Configured' : 'Not
    configured'}</Badge>}
        </div>
        <CardDescription>
          Fixed in <code className="font-mono text-xs">.env</code> (<code
    className="font-mono text-xs">OPENAI_API_KEY</code>,{' '}
          <code className="font-mono text-xs">OPENAI_MODEL</code>, <code
    className="font-mono text-xs">OPENAI_BASE_URL</code>) — not changeable
    here.
        </CardDescription>
      </CardHeader>
      <CardContent className="grid gap-2 text-sm sm:grid-cols-2">
        {!llm ? <Skeleton className="h-5 w-48" /> : (
          <>
            <p><span className="text-muted-foreground">Model:</span>
    {llm.model || '—'}</p>
            <p className="truncate"><span className="text-muted-
    foreground">Endpoint:</span> {llm.base_url || '—'}</p>
          </>
        )}
```

```
      </CardContent>
    </Card>
  )
}

function GitHubSection() {
  const [saved, setSaved] = useState(null)
  const [form, setForm] = useState({ owner: '', committer_name: '',
     committer_email: '' })
  const [token, setToken] = useState('')
  const [saving, setSaving] = useState(false)
  const [testing, setTesting] = useState(false)
  const [result, setResult] = useState(null)

  const load = useCallback(async () => {
    try {
      const { data } = await settingsApi.github()
      setSaved(data)
      setForm({ owner: data.owner, committer_name: data.committer_name,
     committer_email: data.committer_email })
    } catch (err) {
      toast.error(apiError(err, 'Could not load GitHub settings'))
    }
  }, [])

  useEffect(() => { load() }, [load])

  const save = async (extra = {}) => {
    setSaving(true)
    try {
      const payload = { ...form, ...extra }
      if (token) payload.token = token
      const { data } = await settingsApi.saveGithub(payload)
      setSaved(data)
      setToken('')
      setResult(null)
      toast.success('GitHub settings saved')
    } catch (err) {
      toast.error(apiError(err, 'Could not save'))
    } finally {
      setSaving(false)
    }
  }

  const test = async () => {
    setTesting(true)
    try {
      const { data } = await settingsApi.testGithub()
      setResult(data)
    } catch (err) {
      toast.error(apiError(err, 'Could not run the test'))
    } finally {
      setTesting(false)
    }
  }

  const dirty = saved && (token || Object.keys(form).some((k) => form[k] !==
     saved[k]))

  return (
    <Card id="config-github">
      <CardHeader>
        <div className="flex items-center gap-2">
          <GitBranch className="size-4 text-muted-foreground" />
          <CardTitle className="text-base">GitHub</CardTitle>
          {saved && (
            <Badge variant={saved.token_set && saved.owner ? 'secondary' :
     'outline'}>
```

```
          {saved.token_set && saved.owner ? 'Configured' : 'Not
configured'}
        </Badge>
      )}
    </div>
    <CardDescription>
      TRACE creates every project's repository — always public — under
this organization, opens
      each task as an issue there and invites project members as
collaborators.
    </CardDescription>
  </CardHeader>

  {!saved ? (
    <CardContent className="space-y-3">
      <Skeleton className="h-9 w-full" /><Skeleton className="h-9 w-
full" /><Skeleton className="h-9 w-2/3" />
    </CardContent>
  ) : (
    <CardContent className="space-y-4">
      <div className="grid gap-4 sm:grid-cols-2">
        <div className="space-y-2">
          <Label htmlFor="gh-owner">Organization</Label>
          <Input id="gh-owner" value={form.owner} placeholder="acme-
corp"
            onChange={(e) => setForm({ ...form, owner:
e.target.value })} />
          <p className="text-xs text-muted-foreground">The organization
login. Personal accounts are not supported.</p>
        </div>
        <div className="space-y-2">
          <div className="flex items-center justify-between">
            <Label htmlFor="gh-token">Token</Label>
            <span className="text-xs text-muted-foreground">
              {saved.token_set ? TOKEN_SOURCE[saved.token_source] : 'Not
set'}
            </span>
          </div>
          <Input id="gh-token" type="password" autoComplete="off"
value={token}
            placeholder={saved.token_set ? '•••••••••••• (unchanged)' :
'github_pat_…'}
            onChange={(e) => setToken(e.target.value)} />
          <p className="text-xs text-muted-foreground">
            Fine-grained, for this organization only: Administration,
Contents and Issues read/write,
            Metadata read. Stored encrypted and never shown again.
          </p>
        </div>
        <div className="space-y-2">
          <Label htmlFor="gh-cname">Committer name</Label>
          <Input id="gh-cname" value={form.committer_name}
placeholder="TRACE"
            onChange={(e) => setForm({ ...form, committer_name:
e.target.value })} />
        </div>
        <div className="space-y-2">
          <Label htmlFor="gh-cemail">Committer email</Label>
          <Input id="gh-cemail" type="email"
value={form.committer_email} placeholder="trace@acme.com"
            onChange={(e) => setForm({ ...form, committer_email:
e.target.value })} />
        </div>
      </div>

      <TestResult result={result}
        okNote="Permission to create repositories is confirmed by the
first project a Manager creates." />
```

```
        </CardContent>
      )}

      <CardFooter className="flex-wrap justify-between gap-2 border-t">
        <Button variant="outline" onClick={test} disabled={!saved || testing
    || dirty}
          title={dirty ? 'Save first — the test uses the saved settings' :
    undefined}>
          {testing ? <Loader2 className="animate-spin" /> : <PlugZap />}
    Test connection
        </Button>
        <div className="flex gap-2">
          {saved?.token_source === 'database' && (
            <Button variant="ghost" onClick={() => save({ token: '' })}
    disabled={saving}>
              <KeyRound /> Clear saved token
            </Button>
          )}
          <Button id="btn-save-github" onClick={() => save()} disabled={!
    dirty || saving}>
            {saving ? <Loader2 className="animate-spin" /> : <Save />} Save
          </Button>
        </div>
      </CardFooter>
    </Card>
  )
}

export default function ConfigurationPage() {
  const [key, setKey] = useState(0)

  return (
    <div id="configuration-page" className="space-y-6 max-w-[1400px]">
      <div className="flex items-center justify-between">
        <div>
          <h1 className="text-2xl font-bold
    tracking-tight">Configuration</h1>
          <p className="text-sm text-muted-foreground">
            Runtime settings stored in the database — a saved value wins
    over <code className="font-mono text-xs">.env</code>. No redeploy
    required.
          </p>
        </div>
        <Button variant="outline" size="icon" aria-label="Refresh"
    onClick={() => setKey((k) => k + 1)}>
          <RefreshCw />
        </Button>
      </div>

      <GitHubSection key={`gh-${key}`} />
      <MailSection key={`mail-${key}`} />
      <AiSection key={`ai-${key}`} />
    </div>
  )
}
```

frontend/src/pages/admin/AdminSettingsPage.jsx

```
import { useState, useEffect } from 'react'
import { Settings as SettingsIcon, User, BellRing, ShieldCheck, Palette as
    PaletteIcon, Save as SaveIcon, Sun as SunIcon, Moon as MoonIcon,
    CheckCircle, AlertTriangle, Pencil } from 'lucide-react'
import { Card, CardContent, CardHeader, CardTitle } from
    '@/components/ui/card'
import { Badge } from '@/components/ui/badge'
import { Button } from '@/components/ui/button'
import { Input } from '@/components/ui/input'
import { Label } from '@/components/ui/label'
```

```
import { Switch } from '@/components/ui/switch'
import { Alert, AlertDescription } from '@/components/ui/alert'
import { Avatar, AvatarFallback } from '@/components/ui/avatar'
import { Separator } from '@/components/ui/separator'
import { Tabs, TabsContent, TabsList, TabsTrigger } from
    '@/components/ui/tabs'
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue }
    from '@/components/ui/select'
import { useAuth } from '@/contexts/AuthContext'
import { useTheme } from '@/contexts/ThemeContext'
import { authApi } from '@/api'

export default function SettingsPage() {
  const { user, updateUser } = useAuth()
  const { mode, toggleMode } = useTheme()
  const [saving, setSaving] = useState(false)
  const [msg, setMsg] = useState(null)
  const [profileForm, setProfileForm] = useState({ name: '', phone: '',
     gender: '' })
  const [notifications, setNotifications] = useState({
    email_task: true, email_attendance: true, email_reports: false,
    push_task: true, push_attendance: false,
  })

  useEffect(() => {
    if (user) setProfileForm({ name: user.name || '', phone: user.phone ||
     '', gender: user.gender || '' })
  }, [user])

  const handleSaveProfile = async () => {
    setSaving(true); setMsg(null)
    try {
      const res = await authApi.updateProfile(user.id, {
        name: profileForm.name || undefined,
        phone: profileForm.phone || undefined,
        gender: profileForm.gender || undefined,
      })
      updateUser(res.data)
      setMsg({ type: 'success', text: 'Profile updated successfully!' })
    } catch (err) {
      setMsg({ type: 'error', text: err.response?.data?.detail || 'Failed to
     update profile' })
    } finally { setSaving(false) }
  }

  const handleSaveNotifications = () => {
    localStorage.setItem('trace_notifications',
     JSON.stringify(notifications))
    setMsg({ type: 'success', text: 'Notification preferences saved!' })
  }

  const initials = (user?.name || 'U').split(' ').map(w =>
     w[0]).join('').slice(0, 2).toUpperCase()

  return (
    <div id="settings-page" className="space-y-6 max-w-[900px]">
      <div>
        <h1 className="text-2xl font-bold tracking-tight flex items-center
     gap-2">
          <SettingsIcon className="size-5 text-muted-foreground" /> Settings
        </h1>
        <p className="text-sm text-muted-foreground">Manage your account and
     preferences</p>
      </div>

      {msg && (
        <Alert variant={msg.type === 'success' ? 'default' : 'destructive'}>
```

```
     {msg.type === 'success' ? <CheckCircle className="size-4" /> :
<AlertTriangle className="size-4" />}
     <AlertDescription>{msg.text}</AlertDescription>
   </Alert>
 )}

 <Tabs defaultValue="profile" onValueChange={() => setMsg(null)}>
   <TabsList>
     <TabsTrigger value="profile"><User className="size-3 mr-1"
/>Profile</TabsTrigger>
     <TabsTrigger value="notifications"><BellRing className="size-3 mr-
1" />Notifications</TabsTrigger>
     <TabsTrigger value="security"><ShieldCheck className="size-3 mr-1"
/>Security</TabsTrigger>
     <TabsTrigger value="appearance"><PaletteIcon className="size-3 mr-
1" />Appearance</TabsTrigger>
   </TabsList>

   <TabsContent value="profile">
     <Card>
       <CardHeader><CardTitle className="text-sm">Profile
Information</CardTitle></CardHeader>
       <CardContent className="space-y-6">
         <div className="flex items-center gap-4 p-4 rounded-lg bg-
muted/50">
           <Avatar className="size-12">
             <AvatarFallback>{initials}</AvatarFallback>
           </Avatar>
           <div className="flex-1">
             <div className="font-medium">{user?.name || '—'}</div>
             <div className="text-xs text-muted-
foreground">{user?.email}</div>
             <div className="flex items-center gap-1.5 mt-1 flex-wrap">
               {user?.roles?.map((r) => <Badge key={r}
variant="secondary">{r}</Badge>)}
               <Badge variant={user?.is_verified ? 'default' :
'secondary'}>
                 {user?.is_verified ? 'Verified' : 'Unverified'}
               </Badge>
             </div>
           </div>
         </div>

         <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
           <div className="space-y-2">
             <Label htmlFor="s-name">Full Name</Label>
             <Input id="s-name" value={profileForm.name} onChange={e =>
setProfileForm({ ...profileForm, name: e.target.value })} />
           </div>
           <div className="space-y-2">
             <Label htmlFor="s-phone">Phone</Label>
             <Input id="s-phone" type="tel" placeholder="+91-
9876543210" value={profileForm.phone} onChange={e =>
setProfileForm({ ...profileForm, phone: e.target.value })} />
           </div>
           <div className="space-y-2">
             <Label htmlFor="s-email">Email</Label>
             <Input id="s-email" type="email" value={user?.email || ''}
disabled />
           </div>
           <div className="space-y-2">
             <Label>Gender</Label>
             <Select value={profileForm.gender} onValueChange={v =>
setProfileForm({ ...profileForm, gender: v })}>
               <SelectTrigger><SelectValue placeholder="Prefer not to
say" /></SelectTrigger>
               <SelectContent>
                 <SelectItem value="">Prefer not to say</SelectItem>
```

```
                  <SelectItem value="Male">Male</SelectItem>
                  <SelectItem value="Female">Female</SelectItem>
                  <SelectItem value="Non-binary">Non-binary</SelectItem>
                </SelectContent>
              </Select>
            </div>
          </div>

          <Button onClick={handleSaveProfile} disabled={saving}>
            {saving ? <span className="size-4 border-2 border-primary-
foreground/30 border-t-primary-foreground rounded-full animate-spin"
/> : <SaveIcon className="size-4" />}
            Save Changes
          </Button>
        </CardContent>
      </Card>
    </TabsContent>

    <TabsContent value="notifications">
      <Card>
        <CardHeader><CardTitle className="text-sm">Notification
Preferences</CardTitle></CardHeader>
        <CardContent className="space-y-6">
          <div className="space-y-4">
            <p className="text-xs font-medium text-muted-foreground
uppercase tracking-wider">Email Notifications</p>
            {[
              { key: 'email_task', label: 'Task assignments &
updates' },
              { key: 'email_attendance', label: 'Attendance
reminders' },
              { key: 'email_reports', label: 'Weekly productivity
reports' },
            ].map(item => (
              <div key={item.key} className="flex items-center justify-
between">
                <Label className="font-normal">{item.label}</Label>
                <Switch checked={notifications[item.key]}
onCheckedChange={v => setNotifications({ ...notifications, [item.key]: v
})} />
              </div>
            ))}
          </div>
          <Separator />
          <div className="space-y-4">
            <p className="text-xs font-medium text-muted-foreground
uppercase tracking-wider">Push Notifications</p>
            {[
              { key: 'push_task', label: 'Task deadline reminders' },
              { key: 'push_attendance', label: 'Check-in/out alerts' },
            ].map(item => (
              <div key={item.key} className="flex items-center justify-
between">
                <Label className="font-normal">{item.label}</Label>
                <Switch checked={notifications[item.key]}
onCheckedChange={v => setNotifications({ ...notifications, [item.key]: v
})} />
              </div>
            ))}
          </div>
          <Button onClick={handleSaveNotifications}><SaveIcon
className="size-4" />Save Preferences</Button>
        </CardContent>
      </Card>
    </TabsContent>

    <TabsContent value="security">
      <Card>
```

```
            <CardHeader><CardTitle className="text-sm">Security
    Settings</CardTitle></CardHeader>
            <CardContent className="space-y-3">
              {[
                { title: 'Password', sub: 'Last changed: Never', action:
    <Button variant="outline" size="sm"><Pencil className="size-3"
    />Change</Button> },
                { title: 'Two-Factor Authentication', sub: 'Add an extra
    layer of security', action: <Badge variant="secondary">Not
    Enabled</Badge> },
                { title: 'Active Sessions', sub: '1 active session on this
    device', action: <Badge>Current</Badge> },
              ].map(item => (
                <div key={item.title} className="flex items-center justify-
    between p-4 rounded-lg bg-muted/50">
                  <div>
                    <div className="text-sm font-medium">{item.title}</div>
                    <div className="text-xs text-muted-foreground mt-
    0.5">{item.sub}</div>
                  </div>
                  {item.action}
                </div>
              ))}
            </CardContent>
          </Card>
        </TabsContent>

        <TabsContent value="appearance">
          <Card>
            <CardHeader><CardTitle
    className="text-sm">Appearance</CardTitle></CardHeader>
            <CardContent className="space-y-6">
              <div>
                <p className="text-xs font-medium text-muted-foreground
    uppercase tracking-wider mb-3">Theme Mode</p>
                <div className="flex gap-3">
                  {[{ key: 'dark', label: 'Dark', Icon: MoonIcon }, { key:
    'light', label: 'Light', Icon: SunIcon }].map(({ key, label, Icon }) =>
    (
                    <Button
                      key={key}
                      variant={mode === key ? 'default' : 'outline'}
                      onClick={toggleMode}
                      className="flex items-center gap-2"
                    >
                      <Icon className="size-4" /> {label}
                      {mode === key && <CheckCircle className="size-3" />}
                    </Button>
                  ))}
                </div>
              </div>
            </CardContent>
          </Card>
        </TabsContent>
      </Tabs>
    </div>
  )
}
```

frontend/src/pages/manager/ManagerDashboardPage.jsx

```
import { useCallback, useEffect, useState } from 'react'
import { useNavigate } from 'react-router-dom'
import { AlertTriangle, FolderKanban, Plus, RefreshCw } from 'lucide-react'
import { toast } from 'sonner'
import { Button } from '@/components/ui/button'
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from
     '@/components/ui/card'
```

```
import { Avatar, AvatarFallback } from '@/components/ui/avatar'
import { Progress } from '@/components/ui/progress'
import { Skeleton } from '@/components/ui/skeleton'
import { Empty, EmptyContent, EmptyDescription, EmptyHeader, EmptyMedia,
     EmptyTitle } from '@/components/ui/empty'
import { useAuth } from '@/contexts/AuthContext'
import { projectsApi } from '@/api/projects'
import { ProjectStatusBadge, TaskStatusBadge } from
     '@/components/project/status'
import ReportDownload from '@/components/project/ReportDownload'
import { apiError, formatDate, initials } from '@/lib/project'

export default function ManagerDashboardPage() {
  const { user } = useAuth()
  const navigate = useNavigate()
  const [projects, setProjects] = useState(null)
  const [activity, setActivity] = useState([])

  const refresh = useCallback(async () => {
    try {
      const [p, a] = await Promise.all([projectsApi.list(),
     projectsApi.activity()])
      setProjects(p.data)
      setActivity(a.data)
    } catch (err) {
      toast.error(apiError(err, 'Could not load projects'))
      setProjects([])
    }
  }, [])

  useEffect(() => { refresh() }, [refresh])

  const loading = projects === null
  const list = projects || []
  const totalTasks = list.reduce((n, p) => n + p.stats.total_tasks, 0)
  const doneTasks = list.reduce((n, p) => n + p.stats.completed_tasks, 0)
  const stats = [
    { label: 'Projects', value: list.length },
    { label: 'Active', value: list.filter((p) => p.status ===
     'Active').length },
    { label: 'Completed', value: list.filter((p) => p.status ===
     'Completed').length },
    { label: 'Overall completion', value: totalTasks ? `$
     {Math.round((doneTasks / totalTasks) * 100)}%` : '—' },
    { label: 'At risk', value: list.filter((p) => p.at_risk).length, warn:
     true },
  ]

  return (
    <div id="manager-dashboard-page" className="space-y-6 max-w-[1400px]">
      <div className="flex flex-wrap items-center justify-between gap-3">
        <div>
          <h1 className="text-2xl font-bold tracking-tight">Welcome,
     {user?.name || 'Manager'}</h1>
          <p className="text-sm text-muted-foreground">All your projects at
     a glance</p>
        </div>
        <div className="flex items-center gap-2">
          <Button variant="outline" size="icon" onClick={refresh} aria-
     label="Refresh"><RefreshCw /></Button>
          <ReportDownload projects={list} />
          <Button id="btn-new-project" onClick={() =>
     navigate('/manager/projects/new')}>
            <Plus /> New project
          </Button>
        </div>
      </div>
```

```
 <div className="grid grid-cols-2 md:grid-cols-5 gap-4">
   {stats.map((s) => (
     <Card key={s.label}>
       <CardHeader className="pb-1">
         <CardDescription>{s.label}</CardDescription>
       </CardHeader>
       <CardContent>
         {loading ? <Skeleton className="h-8 w-12" /> : (
           <p className={`text-2xl font-bold ${s.warn && s.value ?
'text-destructive' : ''}`}>{s.value}</p>
         )}
       </CardContent>
     </Card>
   ))}
 </div>

 {!loading && list.length === 0 ? (
   <Empty className="border border-dashed">
     <EmptyHeader>
       <EmptyMedia variant="icon"><FolderKanban /></EmptyMedia>
       <EmptyTitle>No projects yet</EmptyTitle>
       <EmptyDescription>
         Create a project, upload its requirement documents and assign
a Project Manager.
       </EmptyDescription>
     </EmptyHeader>
     <EmptyContent>
       <Button id="btn-create-first-project" onClick={() =>
navigate('/manager/projects/new')}>
         <Plus /> Create a project
       </Button>
     </EmptyContent>
   </Empty>
 ) : (
   <div className="grid gap-6 lg:grid-cols-3">
     <div className="lg:col-span-2 grid gap-4 sm:grid-cols-2 content-
start">
       {loading && [0, 1].map((i) => <Skeleton key={i} className="h-40"
/>)}
       {list.map((p) => (
         <Card
           key={p.id}
           className="cursor-pointer transition-colors
hover:bg-muted/40"
           onClick={() => navigate(`/manager/projects/${p.id}`)}
         >
           <CardHeader>
             <div className="flex items-start justify-between gap-2">
               <CardTitle className="text-base">{p.name}</CardTitle>
               <ProjectStatusBadge status={p.status} />
             </div>
             <CardDescription className="line-clamp-2">{p.description
|| p.github_repo}</CardDescription>
           </CardHeader>
           <CardContent className="space-y-3">
             <div className="space-y-1">
               <div className="flex justify-between text-xs text-muted-
foreground">
                 <span>{p.stats.completed_tasks}/{p.stats.total_tasks}
tasks</span>
                 <span>{p.stats.completion_percentage}%</span>
               </div>
               <Progress value={p.stats.completion_percentage} />
             </div>
             <div className="flex items-center justify-between">
               <div className="flex -space-x-2">
                 {p.project_managers.length === 0 && (
```

```
                      <span className="text-xs text-muted-foreground">No
    Project Manager yet</span>
                    )}
                    {p.project_managers.map((pm) => (
                      <Avatar key={pm.id} className="size-6 border-2
    border-background" title={pm.name}>
                        <AvatarFallback className="text-
    [10px]">{initials(pm.name)}</AvatarFallback>
                      </Avatar>
                    ))}
                  </div>
                  {p.at_risk && (
                    <span className="flex items-center gap-1 text-xs text-
    destructive">
                      <AlertTriangle className="size-3.5" /> At risk
                    </span>
                  )}
                </div>
              </CardContent>
            </Card>
          ))}
        </div>

        <Card className="content-start">
          <CardHeader>
            <CardTitle className="text-base">Recent activity</CardTitle>
            <CardDescription>Latest task changes across your
    projects</CardDescription>
          </CardHeader>
          <CardContent className="space-y-3">
            {activity.length === 0 && <p className="text-sm text-muted-
    foreground">No task activity yet.</p>}
            {activity.map((a) => (
              <div key={a.task_id} className="flex items-start justify-
    between gap-2 text-sm">
                <div className="min-w-0">
                  <p className="truncate font-medium">{a.task_title}</p>
                  <p className="truncate text-xs text-muted-foreground">
                    {a.project_name} · {a.assigned_to?.name ||
    'Unassigned'} · {formatDate(a.updated_at)}
                  </p>
                </div>
                <TaskStatusBadge status={a.status} />
              </div>
            ))}
          </CardContent>
        </Card>
      </div>
    ))}
  </div>
 )
}
```

frontend/src/pages/manager/ProjectCreatePage.jsx

```
import { useRef, useState } from 'react'
import { useNavigate } from 'react-router-dom'
import { ArrowLeft, ArrowRight, Check, FileText, Globe, Loader2, Upload, X }
    from 'lucide-react'
import { toast } from 'sonner'
import { Alert, AlertDescription, AlertTitle } from '@/components/ui/alert'
import { Button } from '@/components/ui/button'
import { Card, CardContent, CardDescription, CardFooter, CardHeader,
    CardTitle } from '@/components/ui/card'
import { Input } from '@/components/ui/input'
import { Label } from '@/components/ui/label'
import { Textarea } from '@/components/ui/textarea'
import { projectsApi } from '@/api/projects'
```

```
import { apiError } from '@/lib/project'

const STEPS = ['Details', 'Source files', 'Review']
const ACCEPT = '.pdf,.docx,.md,.markdown,.txt'
function repoSlug(name) {
  return name.toLowerCase().replace(/[^a-z0-9]+/g, '-').replace(/^-+|-+$/g,
     '').slice(0, 100).replace(/-+$/, '')
}

function PublicNotice() {
  return (
    <Alert>
      <Globe />
      <AlertTitle>The project's GitHub repository is public</AlertTitle>
      <AlertDescription>
        Issues, generated documents — including the AI's summary of these
     requirements — and commit
        history are world-readable. The files themselves stay private. Never
     put secrets in them.
      </AlertDescription>
    </Alert>
  )
}

function formatSize(bytes) {
  return bytes > 1024 * 1024 ? `${(bytes / 1024 / 1024).toFixed(1)} MB` : `$
     {Math.max(1, Math.round(bytes / 1024))} KB`
}

export default function ProjectCreatePage() {
  const navigate = useNavigate()
  const fileInput = useRef(null)
  const [step, setStep] = useState(0)
  const [form, setForm] = useState({ name: '', description: '' })
  const [files, setFiles] = useState([])
  const [dragging, setDragging] = useState(false)
  const [submitting, setSubmitting] = useState(false)

  const slug = repoSlug(form.name)
  const detailsValid = form.name.trim().length >= 2 && slug !== ''

  const addFiles = (list) => {
    const incoming = Array.from(list || [])
    setFiles((prev) => [...prev, ...incoming.filter((f) => !prev.some((p) =>
     p.name === f.name && p.size === f.size))])
  }

  const submit = async () => {
    setSubmitting(true)
    try {
      const { data: project } = await projectsApi.create({
        name: form.name.trim(),
        description: form.description.trim() || null,
      })
      if (files.length) {
        const { data: upload } = await
     projectsApi.uploadSourceFiles(project.id, files)
        upload.rejected.forEach((r) => toast.warning(`${r.file_name} was not
     uploaded: ${r.reason}`))
      }
      toast.success(`Project "${project.name}" created`)
      navigate(`/manager/projects/${project.id}`, { replace: true })
    } catch (err) {
      toast.error(apiError(err, 'Could not create the project'))
      setSubmitting(false)
    }
  }
```

```
return (
  <div id="project-create-page" className="space-y-6 max-w-3xl mx-auto">
    <div>
      <Button variant="ghost" size="sm" onClick={() =>
   navigate('/manager')} className="-ml-2 mb-2">
        <ArrowLeft /> Back
      </Button>
      <h1 className="text-2xl font-bold tracking-tight">New project</h1>
      <p className="text-sm text-muted-foreground">
        Describe the project, attach its requirement documents, then
   assign a Project Manager.
      </p>
    </div>

    <ol className="flex items-center gap-2 text-sm">
      {STEPS.map((label, i) => (
        <li key={label} className="flex items-center gap-2">
          <span className={`flex size-6 items-center justify-center
   rounded-full border text-xs font-medium ${
            i < step ? 'bg-primary text-primary-foreground border-primary'
              : i === step ? 'border-primary text-primary' : 'text-muted-
   foreground'}`}>
            {i < step ? <Check className="size-3.5" /> : i + 1}
          </span>
          <span className={i === step ? 'font-medium' : 'text-muted-
   foreground'}>{label}</span>
          {i < STEPS.length - 1 && <span className="mx-1 h-px w-8 bg-
   border" />}
        </li>
      ))}
    </ol>

    <Card>
      {step === 0 && (
        <>
          <CardHeader>
            <CardTitle>Project details</CardTitle>
            <CardDescription>
              TRACE creates a GitHub repository for the project: tasks
   become its issues, and generated
              documents are committed to it.
            </CardDescription>
          </CardHeader>
          <CardContent className="space-y-4">
            <div className="space-y-2">
              <Label htmlFor="p-name">Name</Label>
              <Input id="p-name" value={form.name} autoFocus
                onChange={(e) => setForm({ ...form, name:
   e.target.value })} placeholder="Storefront relaunch" />
            </div>
            <div className="space-y-2">
              <Label htmlFor="p-desc">Description</Label>
              <Textarea id="p-desc" rows={3} value={form.description}
                onChange={(e) => setForm({ ...form, description:
   e.target.value })}
                placeholder="What is being built, and for whom" />
            </div>
            <p className="text-xs text-muted-foreground">
              {form.name.trim() && !slug
                ? 'The name needs at least one letter or digit to name its
   repository.'
                : <>Repository: <span className="font-mono">{slug ||
   'project-name'}</span> in the company's GitHub organization</>}
            </p>
          </CardContent>
        </>
      )}
```

```
   {step === 1 && (
     <>
       <CardHeader>
         <CardTitle>Source files</CardTitle>
         <CardDescription>
           Requirement documents — PDF, Word (.docx), Markdown or plain
text, up to 20 MB each.
           Their text is extracted for AI evaluation. You can add more
later.
         </CardDescription>
       </CardHeader>
       <CardContent className="space-y-4">
         <PublicNotice />
         <button
           type="button"
           onClick={() => fileInput.current?.click()}
           onDragOver={(e) => { e.preventDefault();
setDragging(true) }}
           onDragLeave={() => setDragging(false)}
           onDrop={(e) => { e.preventDefault(); setDragging(false);
addFiles(e.dataTransfer.files) }}
           className={`flex w-full flex-col items-center justify-center
gap-2 rounded-lg border-2 border-dashed p-8 text-sm transition-colors ${
             dragging ? 'border-primary bg-muted' : 'border-border
hover:bg-muted/50'}`}
         >
           <Upload className="size-6 text-muted-foreground" />
           <span className="font-medium">Drop files here, or click to
choose</span>
           <span className="text-xs text-muted-
foreground">{ACCEPT.replaceAll(',', ' ')}</span>
         </button>
         <input ref={fileInput} type="file" multiple accept={ACCEPT}
className="hidden"
           onChange={(e) => { addFiles(e.target.files); e.target.value
= '' }} />
         {files.length > 0 && (
           <ul className="divide-y rounded-lg border">
             {files.map((f) => (
               <li key={`${f.name}-${f.size}`} className="flex items-
center gap-3 px-3 py-2 text-sm">
                 <FileText className="size-4 text-muted-foreground" />
                 <span className="flex-1 truncate">{f.name}</span>
                 <span className="text-xs text-muted-
foreground">{formatSize(f.size)}</span>
                 <Button variant="ghost" size="icon-sm" aria-
label={`Remove ${f.name}`}
                   onClick={() => setFiles(files.filter((x) => x !==
f))}><X /></Button>
               </li>
             ))}
           </ul>
         )}
       </CardContent>
     </>
   )}

   {step === 2 && (
     <>
       <CardHeader>
         <CardTitle>Review</CardTitle>
         <CardDescription>The project starts as a
Draft.</CardDescription>
       </CardHeader>
       <CardContent className="space-y-4">
         <dl className="grid grid-cols-[140px_1fr] gap-y-2 text-sm">
           <dt
className="text-muted-foreground">Name</dt><dd>{form.name}</dd>
```

```
                <dt
    className="text-muted-foreground">Description</dt><dd>{form.description
    || '—'}</dd>
                <dt className="text-muted-foreground">Repository</dt>
                <dd><span className="font-mono">{slug}</span> · public ·
    created by TRACE</dd>
                <dt className="text-muted-foreground">Source files</dt>
                <dd>{files.length ? files.map((f) => f.name).join(', ') :
    'None yet'}</dd>
              </dl>
              <PublicNotice />
            </CardContent>
          </>
        )}

        <CardFooter className="justify-between border-t">
          <Button variant="outline" onClick={() => setStep(step - 1)}
    disabled={step === 0 || submitting}>
            <ArrowLeft /> Back
          </Button>
          {step < 2 ? (
            <Button onClick={() => setStep(step + 1)} disabled={step === 0
    && !detailsValid}>
              Next <ArrowRight />
            </Button>
          ) : (
            <Button id="btn-create-project" onClick={submit}
    disabled={submitting}>
              {submitting ? <Loader2 className="animate-spin" /> : <Check
    />} Create project
            </Button>
          )}
        </CardFooter>
      </Card>
    </div>
  )
}
```

frontend/src/pages/manager/ProjectDetailPage.jsx

```
import { useCallback, useEffect, useState } from 'react'
import { useNavigate, useParams } from 'react-router-dom'
import { RefreshCw } from 'lucide-react'
import { toast } from 'sonner'
import { Button } from '@/components/ui/button'
import { Tabs, TabsContent, TabsList, TabsTrigger } from
     '@/components/ui/tabs'
import { Skeleton } from '@/components/ui/skeleton'
import { projectsApi } from '@/api/projects'
import ProjectOverview from '@/components/project/ProjectOverview'
import TaskBoard from '@/components/project/TaskBoard'
import TeamPanel from '@/components/project/TeamPanel'
import ProgressReport from '@/components/project/ProgressReport'
import ProjectTreeGraph from '@/components/ProjectTreeGraph'
import GanttChart from '@/components/GanttChart'
import { ProjectStatusBadge } from '@/components/project/status'
import { apiError } from '@/lib/project'

const taskPath = (taskId) => `/manager/tasks/${taskId}`

export default function ManagerProjectDetailPage() {
  const { id } = useParams()
  const navigate = useNavigate()
  const [project, setProject] = useState(null)

  const load = useCallback(async () => {
    try {
      setProject((await projectsApi.get(id)).data)
```

```
    } catch (err) {
      toast.error(apiError(err, 'Could not load the project'))
      if ([403, 404].includes(err.response?.status)) navigate('/manager',
     { replace: true })
    }
  }, [id, navigate])

  useEffect(() => { setProject(null); load() }, [load])

  if (!project) {
    return <div className="space-y-4 max-w-[1400px]"><Skeleton className="h-
     10 w-72" /><Skeleton className="h-64" /></div>
  }

  return (
    <div id="manager-project-page" className="space-y-6 max-w-[1400px]">
      <div className="flex flex-wrap items-start justify-between gap-3">
        <div className="min-w-0">
          <div className="flex items-center gap-2">
            <h1 className="text-2xl font-bold tracking-tight
     truncate">{project.name}</h1>
            <ProjectStatusBadge status={project.status} />
          </div>
          <p className="text-sm text-muted-foreground">{project.description
     || project.github_repo}</p>
        </div>
        <Button variant="outline" size="icon" onClick={load} aria-
     label="Refresh"><RefreshCw /></Button>
      </div>

      <Tabs defaultValue="overview">
        <TabsList>
          <TabsTrigger value="overview">Overview</TabsTrigger>
          <TabsTrigger value="tasks">Tasks</TabsTrigger>
          <TabsTrigger value="tree">Tree</TabsTrigger>
          <TabsTrigger value="gantt">Gantt</TabsTrigger>
          <TabsTrigger value="report">Progress report</TabsTrigger>
          <TabsTrigger value="team">Team</TabsTrigger>
        </TabsList>
        <TabsContent value="overview"><ProjectOverview project={project}
     isOwner onChange={load} /></TabsContent>
        <TabsContent value="tasks"><TaskBoard projectId={project.id}
     taskPath={taskPath} /></TabsContent>
        <TabsContent value="tree"><ProjectTreeGraph projectId={project.id}
     taskPath={taskPath} /></TabsContent>
        <TabsContent value="gantt"><GanttChart projectId={project.id}
     /></TabsContent>
        <TabsContent value="report"><ProgressReport projectId={project.id}
     canRefresh /></TabsContent>
        <TabsContent value="team"><TeamPanel projectId={project.id} isOwner
     onChange={load} /></TabsContent>
      </Tabs>
    </div>
  )
}
```

frontend/src/pages/manager/ManagerSettingsPage.jsx

```
import { useState, useEffect } from 'react'
import { Settings as SettingsIcon, User, BellRing, ShieldCheck, Palette as
     PaletteIcon, Save as SaveIcon, Sun as SunIcon, Moon as MoonIcon,
     CheckCircle, AlertTriangle, Pencil } from 'lucide-react'
import { Card, CardContent, CardHeader, CardTitle } from
     '@/components/ui/card'
import { Badge } from '@/components/ui/badge'
import { Button } from '@/components/ui/button'
import { Input } from '@/components/ui/input'
import { Label } from '@/components/ui/label'
```

```
import { Switch } from '@/components/ui/switch'
import { Alert, AlertDescription } from '@/components/ui/alert'
import { Avatar, AvatarFallback } from '@/components/ui/avatar'
import { Separator } from '@/components/ui/separator'
import { Tabs, TabsContent, TabsList, TabsTrigger } from
    '@/components/ui/tabs'
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue }
    from '@/components/ui/select'
import { useAuth } from '@/contexts/AuthContext'
import { useTheme } from '@/contexts/ThemeContext'
import { authApi } from '@/api'

export default function SettingsPage() {
  const { user, updateUser } = useAuth()
  const { mode, toggleMode } = useTheme()
  const [saving, setSaving] = useState(false)
  const [msg, setMsg] = useState(null)
  const [profileForm, setProfileForm] = useState({ name: '', phone: '',
     gender: '' })
  const [notifications, setNotifications] = useState({
    email_task: true, email_attendance: true, email_reports: false,
    push_task: true, push_attendance: false,
  })

  useEffect(() => {
    if (user) setProfileForm({ name: user.name || '', phone: user.phone ||
     '', gender: user.gender || '' })
  }, [user])

  const handleSaveProfile = async () => {
    setSaving(true); setMsg(null)
    try {
      const res = await authApi.updateProfile(user.id, {
        name: profileForm.name || undefined,
        phone: profileForm.phone || undefined,
        gender: profileForm.gender || undefined,
      })
      updateUser(res.data)
      setMsg({ type: 'success', text: 'Profile updated successfully!' })
    } catch (err) {
      setMsg({ type: 'error', text: err.response?.data?.detail || 'Failed to
     update profile' })
    } finally { setSaving(false) }
  }

  const handleSaveNotifications = () => {
    localStorage.setItem('trace_notifications',
     JSON.stringify(notifications))
    setMsg({ type: 'success', text: 'Notification preferences saved!' })
  }

  const initials = (user?.name || 'U').split(' ').map(w =>
     w[0]).join('').slice(0, 2).toUpperCase()

  return (
    <div id="settings-page" className="space-y-6 max-w-[900px]">
      <div>
        <h1 className="text-2xl font-bold tracking-tight flex items-center
     gap-2">
          <SettingsIcon className="size-5 text-muted-foreground" /> Settings
        </h1>
        <p className="text-sm text-muted-foreground">Manage your account and
     preferences</p>
      </div>

      {msg && (
        <Alert variant={msg.type === 'success' ? 'default' : 'destructive'}>
```

```
     {msg.type === 'success' ? <CheckCircle className="size-4" /> :
<AlertTriangle className="size-4" />}
     <AlertDescription>{msg.text}</AlertDescription>
   </Alert>
 )}

 <Tabs defaultValue="profile" onValueChange={() => setMsg(null)}>
   <TabsList>
     <TabsTrigger value="profile"><User className="size-3 mr-1"
/>Profile</TabsTrigger>
     <TabsTrigger value="notifications"><BellRing className="size-3 mr-
1" />Notifications</TabsTrigger>
     <TabsTrigger value="security"><ShieldCheck className="size-3 mr-1"
/>Security</TabsTrigger>
     <TabsTrigger value="appearance"><PaletteIcon className="size-3 mr-
1" />Appearance</TabsTrigger>
   </TabsList>

   <TabsContent value="profile">
     <Card>
       <CardHeader><CardTitle className="text-sm">Profile
Information</CardTitle></CardHeader>
       <CardContent className="space-y-6">
         <div className="flex items-center gap-4 p-4 rounded-lg bg-
muted/50">
           <Avatar className="size-12">
             <AvatarFallback>{initials}</AvatarFallback>
           </Avatar>
           <div className="flex-1">
             <div className="font-medium">{user?.name || '—'}</div>
             <div className="text-xs text-muted-
foreground">{user?.email}</div>
             <div className="flex items-center gap-1.5 mt-1 flex-wrap">
               {user?.roles?.map((r) => <Badge key={r}
variant="secondary">{r}</Badge>)}
               <Badge variant={user?.is_verified ? 'default' :
'secondary'}>
                 {user?.is_verified ? 'Verified' : 'Unverified'}
               </Badge>
             </div>
           </div>
         </div>

         <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
           <div className="space-y-2">
             <Label htmlFor="s-name">Full Name</Label>
             <Input id="s-name" value={profileForm.name} onChange={e =>
setProfileForm({ ...profileForm, name: e.target.value })} />
           </div>
           <div className="space-y-2">
             <Label htmlFor="s-phone">Phone</Label>
             <Input id="s-phone" type="tel" placeholder="+91-
9876543210" value={profileForm.phone} onChange={e =>
setProfileForm({ ...profileForm, phone: e.target.value })} />
           </div>
           <div className="space-y-2">
             <Label htmlFor="s-email">Email</Label>
             <Input id="s-email" type="email" value={user?.email || ''}
disabled />
           </div>
           <div className="space-y-2">
             <Label>Gender</Label>
             <Select value={profileForm.gender} onValueChange={v =>
setProfileForm({ ...profileForm, gender: v })}>
               <SelectTrigger><SelectValue placeholder="Prefer not to
say" /></SelectTrigger>
               <SelectContent>
                 <SelectItem value="">Prefer not to say</SelectItem>
```

```
                  <SelectItem value="Male">Male</SelectItem>
                  <SelectItem value="Female">Female</SelectItem>
                  <SelectItem value="Non-binary">Non-binary</SelectItem>
                </SelectContent>
              </Select>
            </div>
          </div>

          <Button onClick={handleSaveProfile} disabled={saving}>
            {saving ? <span className="size-4 border-2 border-primary-
foreground/30 border-t-primary-foreground rounded-full animate-spin"
/> : <SaveIcon className="size-4" />}
            Save Changes
          </Button>
        </CardContent>
      </Card>
    </TabsContent>

    <TabsContent value="notifications">
      <Card>
        <CardHeader><CardTitle className="text-sm">Notification
Preferences</CardTitle></CardHeader>
        <CardContent className="space-y-6">
          <div className="space-y-4">
            <p className="text-xs font-medium text-muted-foreground
uppercase tracking-wider">Email Notifications</p>
            {[
              { key: 'email_task', label: 'Task assignments &
updates' },
              { key: 'email_attendance', label: 'Attendance
reminders' },
              { key: 'email_reports', label: 'Weekly productivity
reports' },
            ].map(item => (
              <div key={item.key} className="flex items-center justify-
between">
                <Label className="font-normal">{item.label}</Label>
                <Switch checked={notifications[item.key]}
onCheckedChange={v => setNotifications({ ...notifications, [item.key]: v
})} />
              </div>
            ))}
          </div>
          <Separator />
          <div className="space-y-4">
            <p className="text-xs font-medium text-muted-foreground
uppercase tracking-wider">Push Notifications</p>
            {[
              { key: 'push_task', label: 'Task deadline reminders' },
              { key: 'push_attendance', label: 'Check-in/out alerts' },
            ].map(item => (
              <div key={item.key} className="flex items-center justify-
between">
                <Label className="font-normal">{item.label}</Label>
                <Switch checked={notifications[item.key]}
onCheckedChange={v => setNotifications({ ...notifications, [item.key]: v
})} />
              </div>
            ))}
          </div>
          <Button onClick={handleSaveNotifications}><SaveIcon
className="size-4" />Save Preferences</Button>
        </CardContent>
      </Card>
    </TabsContent>

    <TabsContent value="security">
      <Card>
```

```
            <CardHeader><CardTitle className="text-sm">Security
      Settings</CardTitle></CardHeader>
            <CardContent className="space-y-3">
              {[
                { title: 'Password', sub: 'Last changed: Never', action:
      <Button variant="outline" size="sm"><Pencil className="size-3"
      />Change</Button> },
                { title: 'Two-Factor Authentication', sub: 'Add an extra
      layer of security', action: <Badge variant="secondary">Not
      Enabled</Badge> },
                { title: 'Active Sessions', sub: '1 active session on this
      device', action: <Badge>Current</Badge> },
              ].map(item => (
                <div key={item.title} className="flex items-center justify-
      between p-4 rounded-lg bg-muted/50">
                  <div>
                    <div className="text-sm font-medium">{item.title}</div>
                    <div className="text-xs text-muted-foreground mt-
      0.5">{item.sub}</div>
                  </div>
                  {item.action}
                </div>
              ))}
            </CardContent>
          </Card>
        </TabsContent>

        <TabsContent value="appearance">
          <Card>
            <CardHeader><CardTitle
      className="text-sm">Appearance</CardTitle></CardHeader>
            <CardContent className="space-y-6">
              <div>
                <p className="text-xs font-medium text-muted-foreground
      uppercase tracking-wider mb-3">Theme Mode</p>
                <div className="flex gap-3">
                  {[{ key: 'dark', label: 'Dark', Icon: MoonIcon }, { key:
      'light', label: 'Light', Icon: SunIcon }].map(({ key, label, Icon }) =>
      (
                    <Button
                      key={key}
                      variant={mode === key ? 'default' : 'outline'}
                      onClick={toggleMode}
                      className="flex items-center gap-2"
                    >
                      <Icon className="size-4" /> {label}
                      {mode === key && <CheckCircle className="size-3" />}
                    </Button>
                  ))}
                </div>
              </div>
            </CardContent>
          </Card>
        </TabsContent>
      </Tabs>
    </div>
  )
}
```

frontend/src/pages/employee/EmployeeDashboardPage.jsx

```
import { useCallback, useEffect, useState } from 'react'
import { useNavigate } from 'react-router-dom'
import { AlertTriangle, Briefcase, RefreshCw } from 'lucide-react'
import { toast } from 'sonner'
import { Button } from '@/components/ui/button'
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from
     '@/components/ui/card'
```

```
import { Alert, AlertDescription, AlertTitle } from '@/components/ui/alert'
import { Progress } from '@/components/ui/progress'
import { Skeleton } from '@/components/ui/skeleton'
import { Empty, EmptyDescription, EmptyHeader, EmptyMedia, EmptyTitle } from
    '@/components/ui/empty'
import { useAuth } from '@/contexts/AuthContext'
import { projectsApi } from '@/api/projects'
import ReportDownload from '@/components/project/ReportDownload'
import { tasksApi } from '@/api/tasks'
import { RoleBadge, TaskStatusBadge } from '@/components/project/status'
import { apiError, formatDate, isOverdue } from '@/lib/project'

function dueBucket(task) {
  if (isOverdue(task)) return 'Overdue'
  if (!task.due_date) return 'No due date'
  const days = (new Date(task.due_date) - new Date(new
     Date().toDateString())) / 86400000
  return days <= 7 ? 'Due this week' : 'Later'
}

const BUCKETS = ['Overdue', 'Due this week', 'Later', 'No due date']

export default function EmployeeDashboardPage() {
  const { user } = useAuth()
  const navigate = useNavigate()
  const [projects, setProjects] = useState(null)
  const [tasks, setTasks] = useState([])

  const refresh = useCallback(async () => {
    try {
      const [p, t] = await Promise.all([projectsApi.list(),
    tasksApi.mine()])
      setProjects(p.data)
      setTasks(t.data)
    } catch (err) {
      toast.error(apiError(err, 'Could not load your work'))
      setProjects([])
    }
  }, [])

  useEffect(() => { refresh() }, [refresh])

  const loading = projects === null
  const overdue = tasks.filter(isOverdue)

  return (
    <div id="employee-dashboard-page" className="space-y-6 max-w-[1400px]">
      <div className="flex items-center justify-between">
        <div>
          <h1 className="text-2xl font-bold tracking-tight">Hello,
    {user?.name || 'there'}</h1>
          <p className="text-sm text-muted-foreground">Your projects and
    open tasks</p>
        </div>
        <div className="flex items-center gap-2">
          <Button variant="outline" size="icon" onClick={refresh} aria-
    label="Refresh"><RefreshCw /></Button>
          <ReportDownload projects={projects || []} />
        </div>
      </div>

      {overdue.length > 0 && (
        <Alert variant="destructive">
          <AlertTriangle />
          <AlertTitle>{overdue.length} overdue task{overdue.length > 1 ? 's'
    : ''}</AlertTitle>
          <AlertDescription>{overdue.map((t) => t.title).join(',
    ')}</AlertDescription>
```

```
    </Alert>
  )}

  {!loading && projects.length === 0 ? (
    <Empty className="border border-dashed">
      <EmptyHeader>
        <EmptyMedia variant="icon"><Briefcase /></EmptyMedia>
        <EmptyTitle>You are not on a project yet</EmptyTitle>
        <EmptyDescription>
          A Manager makes you a Project Manager, or a Project Manager
assigns you a task — then it appears here.
        </EmptyDescription>
      </EmptyHeader>
    </Empty>
  ) : (
    <div className="grid gap-6 lg:grid-cols-3">
      <div className="space-y-4 lg:col-span-2">
        <h2 className="text-lg font-semibold">My tasks</h2>
        {!loading && tasks.length === 0 && <p className="text-sm text-
muted-foreground">No open tasks assigned to you.</p>}
        {BUCKETS.map((bucket) => {
          const items = tasks.filter((t) => dueBucket(t) === bucket)
          if (!items.length) return null
          return (
            <Card key={bucket}>
              <CardHeader>
                <CardTitle className={`text-base ${bucket ===
'Overdue' ? 'text-destructive' : ''}`}>{bucket}</CardTitle>
              </CardHeader>
              <CardContent>
                <ul className="divide-y rounded-lg border">
                  {items.map((t) => (
                    <li key={t.id} className="flex cursor-pointer items-
center gap-3 px-3 py-2 text-sm hover:bg-muted/40"
                      onClick={() => navigate(`/employee/tasks/$
{t.id}`)}>
                      <span className="min-w-0 flex-1">
                        <span className="font-medium">{t.title}</span>
                        <span className="block text-xs text-muted-
foreground">
                          {t.project_name} · {t.task_group_name} · due
{formatDate(t.due_date)}
                        </span>
                      </span>
                      <TaskStatusBadge status={t.status} />
                    </li>
                  ))}
                </ul>
              </CardContent>
            </Card>
          )
        })}
      </div>

      <div className="space-y-4">
        <h2 className="text-lg font-semibold">My projects</h2>
        {loading && <Skeleton className="h-32" />}
        {(projects || []).map((p) => (
          <Card key={p.id} className="cursor-pointer transition-colors
hover:bg-muted/40"
            onClick={() => navigate(`/employee/projects/${p.id}`)}>
            <CardHeader>
              <CardTitle className="text-base">{p.name}</CardTitle>
              <CardDescription><RoleBadge role={p.my_role}
/></CardDescription>
            </CardHeader>
            <CardContent className="space-y-1">
```

```
                <div className="flex justify-between text-xs text-muted-
    foreground">
                  <span>{p.stats.completed_tasks}/{p.stats.total_tasks}
    tasks</span>
                  <span>{p.stats.completion_percentage}%</span>
                </div>
                <Progress value={p.stats.completion_percentage} />
              </CardContent>
            </Card>
          ))}
        </div>
      </div>
    ))}
  </div>
 )
}
```

frontend/src/pages/employee/ProjectDetailPage.jsx

```
import { useCallback, useEffect, useState } from 'react'
import { useNavigate, useParams } from 'react-router-dom'
import { RefreshCw } from 'lucide-react'
import { toast } from 'sonner'
import { Button } from '@/components/ui/button'
import { Tabs, TabsContent, TabsList, TabsTrigger } from
    '@/components/ui/tabs'
import { Skeleton } from '@/components/ui/skeleton'
import { projectsApi } from '@/api/projects'
import ProjectOverview from '@/components/project/ProjectOverview'
import TaskBoard from '@/components/project/TaskBoard'
import TeamPanel from '@/components/project/TeamPanel'
import ProjectDocEditor from '@/components/project/ProjectDocEditor'
import ProjectTreeGraph from '@/components/ProjectTreeGraph'
import GanttChart from '@/components/GanttChart'
import { ProjectStatusBadge, RoleBadge } from '@/components/project/status'
import { apiError } from '@/lib/project'

const taskPath = (taskId) => `/employee/tasks/${taskId}`

export default function EmployeeProjectDetailPage() {
  const { id } = useParams()
  const navigate = useNavigate()
  const [project, setProject] = useState(null)

  const load = useCallback(async () => {
    try {
      setProject((await projectsApi.get(id)).data)
    } catch (err) {
      toast.error(apiError(err, 'Could not load the project'))
      if ([403, 404].includes(err.response?.status)) navigate('/employee', {
     replace: true })
    }
  }, [id, navigate])

  useEffect(() => { setProject(null); load() }, [load])

  if (!project) {
    return <div className="space-y-4 max-w-[1400px]"><Skeleton className="h-
     10 w-72" /><Skeleton className="h-64" /></div>
  }

  const isPM = project.my_role === 'Project Manager'
  const tree = <ProjectTreeGraph projectId={project.id}
     taskPath={taskPath} />
  const gantt = <GanttChart projectId={project.id} canReevaluate={isPM} />

  return (
    <div id="employee-project-page" className="space-y-6 max-w-[1400px]">
```

```
      <div className="flex flex-wrap items-start justify-between gap-3">
        <div className="min-w-0">
          <div className="flex flex-wrap items-center gap-2">
            <h1 className="text-2xl font-bold tracking-tight
    truncate">{project.name}</h1>
            <ProjectStatusBadge status={project.status} />
            <RoleBadge role={project.my_role} />
          </div>
          <p className="text-sm text-muted-foreground">{project.description
    || project.github_repo}</p>
        </div>
        <Button variant="outline" size="icon" onClick={load} aria-
    label="Refresh"><RefreshCw /></Button>
      </div>

      {isPM ? (
        <Tabs defaultValue="tasks">
          <TabsList>
            <TabsTrigger value="overview">Overview</TabsTrigger>
            <TabsTrigger value="tasks">Tasks</TabsTrigger>
            <TabsTrigger value="tree">Tree</TabsTrigger>
            <TabsTrigger value="gantt">Gantt</TabsTrigger>
            <TabsTrigger value="doc">Project doc</TabsTrigger>
            <TabsTrigger value="team">Team</TabsTrigger>
          </TabsList>
          <TabsContent value="overview"><ProjectOverview project={project}
    isOwner={false} onChange={load} /></TabsContent>
          <TabsContent value="tasks"><TaskBoard projectId={project.id}
    canManage taskPath={taskPath} onChange={load} /></TabsContent>
          <TabsContent value="tree">{tree}</TabsContent>
          <TabsContent value="gantt">{gantt}</TabsContent>
          <TabsContent value="doc"><ProjectDocEditor
    projectId={project.id} /></TabsContent>
          <TabsContent value="team"><TeamPanel projectId={project.id}
    isOwner={false} /></TabsContent>
        </Tabs>
      ) : (
        <Tabs defaultValue="mine">
          <TabsList>
            <TabsTrigger value="overview">Overview</TabsTrigger>
            <TabsTrigger value="mine">My tasks</TabsTrigger>
            <TabsTrigger value="tree">Tree</TabsTrigger>
            <TabsTrigger value="gantt">Gantt</TabsTrigger>
          </TabsList>
          <TabsContent value="overview"><ProjectOverview project={project}
    isOwner={false} onChange={load} /></TabsContent>
          <TabsContent value="mine"><TaskBoard projectId={project.id} mine
    taskPath={taskPath} /></TabsContent>
          <TabsContent value="tree">{tree}</TabsContent>
          <TabsContent value="gantt">{gantt}</TabsContent>
        </Tabs>
      )}
    </div>
  )
}
```

frontend/src/pages/employee/TaskDetailPage.jsx

```
import { useCallback, useEffect, useState } from 'react'
import { Link, useLocation, useNavigate, useParams } from 'react-router-dom'
import { ArrowLeft, CheckCircle2, ExternalLink, FileText,
    GitCommitHorizontal, Loader2 } from 'lucide-react'
import { toast } from 'sonner'
import { Button } from '@/components/ui/button'
import { Card, CardAction, CardContent, CardDescription, CardHeader,
    CardTitle } from '@/components/ui/card'
import { Skeleton } from '@/components/ui/skeleton'
```

```
import { NativeSelect, NativeSelectOption } from '@/components/ui/native-
    select'
import {
  AlertDialog, AlertDialogAction, AlertDialogCancel, AlertDialogContent,
    AlertDialogDescription,
  AlertDialogFooter, AlertDialogHeader, AlertDialogTitle,
} from '@/components/ui/alert-dialog'
import { tasksApi } from '@/api/tasks'
import { projectsApi } from '@/api/projects'
import { TaskStatusBadge } from '@/components/project/status'
import Markdown from '@/components/Markdown'
import TaskDiscussion from '@/components/project/TaskDiscussion'
import { apiError, formatDate, isOverdue } from '@/lib/project'

export default function TaskDetailPage() {
  const { taskId } = useParams()
  const navigate = useNavigate()
  const panel = useLocation().pathname.startsWith('/manager') ? '/manager' :
    '/employee'
  const [task, setTask] = useState(null)
  const [candidates, setCandidates] = useState([])
  const [busy, setBusy] = useState(null)
  const [confirmComplete, setConfirmComplete] = useState(false)
  const [progressDoc, setProgressDoc] = useState(null)

  const load = useCallback(async () => {
    try {
      const { data } = await tasksApi.get(taskId)
      setTask(data)
      if (data.task_progress_doc) tasksApi.progressDoc(data.id).then((r) =>
    setProgressDoc(r.data)).catch(() => {})
      if (data.can_edit) setCandidates((await
    projectsApi.candidates(data.project_id)).data)
    } catch (err) {
      toast.error(apiError(err, 'Could not load the task'))
      if ([403, 404].includes(err.response?.status)) navigate(panel,
    { replace: true })
    }
  }, [taskId, navigate, panel])

  useEffect(() => { load() }, [load])

  const setStatus = async (status) => {
    setBusy(status)
    try {
      setTask((await tasksApi.updateStatus(task.id, status)).data)
      toast.success(`Status set to ${status}`)
    } catch (err) {
      toast.error(apiError(err, 'Could not change the status'))
    } finally {
      setBusy(null)
      setConfirmComplete(false)
    }
  }

  const reassign = async (value) => {
    try {
      const { data } = await tasksApi.assign(task.id, value ?
    Number(value) : null)
      setTask(data.task)
      toast.success(value ? `Assigned to ${data.task.assigned_to.name}` :
    'Task unassigned')
      data.warnings.forEach((w) => toast.warning(w))
    } catch (err) {
      toast.error(apiError(err, 'Could not reassign'))
    }
  }
```

```
if (!task) return <div className="space-y-4 max-w-4xl"><Skeleton
   className="h-10 w-80" /><Skeleton className="h-64" /></div>

const isPM = task.can_edit
const issueRef = task.github_issue_id ? `#${task.github_issue_id}` : null

return (
  <div id="task-detail-page" className="space-y-6 max-w-4xl">
    <div>
      <Button variant="ghost" size="sm" className="-ml-2 mb-2"
   nativeButton={false} render={<Link to={`${panel}/projects/$
   {task.project_id}`} />}>
        <ArrowLeft /> {task.project_name}
      </Button>
      <div className="flex flex-wrap items-center gap-2">
        <h1 className="text-2xl font-bold
   tracking-tight">{task.title}</h1>
        <TaskStatusBadge status={task.status} />
      </div>
      <p className="text-sm text-muted-foreground">
        {task.task_group_name} · {task.priority} priority · due{' '}
        <span className={isOverdue(task) ? 'text-destructive' :
   ''}>{formatDate(task.due_date)}</span>
      </p>
    </div>

    <div className="grid gap-4 md:grid-cols-3">
      <div className="space-y-4 md:col-span-2">
        <Card>
          <CardHeader><CardTitle
   className="text-base">Description</CardTitle></CardHeader>
          <CardContent>
            <p className="whitespace-pre-wrap text-sm">{task.description
   || 'No description.'}</p>
          </CardContent>
        </Card>

        <Card>
          <CardHeader>
            <CardTitle className="flex items-center gap-2 text-
   base"><GitCommitHorizontal className="size-4" /> GitHub
   issue</CardTitle>
            <CardDescription>This task is an issue in the project's
   repository.</CardDescription>
          </CardHeader>
          <CardContent className="space-y-3 text-sm">
            <p>
              Reference {issueRef ? <strong>{issueRef}</strong> : 'the
   issue number'} in your commit messages so GitHub
              links the work to it. When you are done, mark the task
   complete here — that closes the issue.
            </p>
            {issueRef && (
              <pre className="rounded-md bg-muted px-3 py-2 text-xs">git
   commit -m "Add coupon field to cart ({issueRef})"</pre>
            )}
            {task.github_issue_url && (
              <a href={task.github_issue_url} target="_blank"
   rel="noopener noreferrer"
                 className="inline-flex items-center gap-1.5 text-primary
   hover:underline">
                 Open GitHub issue {issueRef} <ExternalLink
   className="size-3" />
              </a>
            )}
          </CardContent>
        </Card>
```

```
      <Card>
        <CardHeader>
          <CardTitle className="flex items-center gap-2 text-
base"><FileText className="size-4" /> Progress document</CardTitle>
          {task.task_progress_doc && (
            <CardAction>
              <Button size="sm" variant="ghost" nativeButton={false}
                render={<a href={task.task_progress_doc} target="_blank"
rel="noopener noreferrer" />}>
                On GitHub <ExternalLink />
              </Button>
            </CardAction>
          )}
        </CardHeader>
        <CardContent className="text-sm">
          {progressDoc?.content ? (
            <Markdown>{progressDoc.content}</Markdown>
          ) : task.progress_doc_error ? (
            <p className="text-destructive">The AI could not write the
plan: {task.progress_doc_error}</p>
          ) : (
            <p className="text-muted-foreground">
              {task.task_progress_doc ? 'Loading…' : 'Being written —
the AI drafts the steps and checklist when a task is created. Refresh in
a minute.'}
            </p>
          )}
        </CardContent>
      </Card>

      {task.can_discuss && <TaskDiscussion taskId={task.id} />}
    </div>

    <div className="space-y-4">
      <Card>
        <CardHeader><CardTitle
className="text-base">Status</CardTitle></CardHeader>
        <CardContent className="space-y-3 text-sm">
          {task.completed_by && (
            <p className="flex items-center gap-1.5">
              <CheckCircle2 className="size-4" /> Completed by
{task.completed_by} · {formatDate(task.completed_at)}
            </p>
          )}
          {task.allowed_statuses.length > 0 ? (
            <div className="flex flex-wrap gap-2">
              {task.allowed_statuses.map((s) => (
                <Button key={s} size="sm" variant={s === 'Completed' ?
'default' : 'outline'} disabled={!!busy}
                  onClick={() => (s === 'Completed' ?
setConfirmComplete(true) : setStatus(s))}>
                  {busy === s && <Loader2 className="animate-spin" />}
                  {s === 'Completed' ? 'Mark complete' : s}
                </Button>
              ))}
            </div>
          ) : (
            <p className="text-muted-foreground">You can view this task
but not change its status.</p>
          )}
        </CardContent>
      </Card>

      <Card>
        <CardHeader><CardTitle
className="text-base">People</CardTitle></CardHeader>
        <CardContent className="space-y-3 text-sm">
          <div>
```

```
              <p className="text-xs text-muted-foreground">Assignee</p>
              {isPM ? (
                <NativeSelect className="mt-1 w-full"
    value={task.assigned_to?.id || ''} onChange={(e) =>
    reassign(e.target.value)}>
                  <NativeSelectOption
    value="">Unassigned</NativeSelectOption>
                  {candidates.map((c) => <NativeSelectOption key={c.id}
    value={c.id}>{c.name}</NativeSelectOption>)}
                </NativeSelect>
              ) : (
                <p>{task.assigned_to?.name || 'Unassigned'}</p>
              )}
            </div>
            <div>
              <p className="text-xs text-muted-foreground">Assigned by</p>
              <p>{task.assigned_by?.name || '—'}</p>
            </div>
          </CardContent>
        </Card>
      </div>
    </div>

    <AlertDialog open={confirmComplete} onOpenChange={setConfirmComplete}>
      <AlertDialogContent>
        <AlertDialogHeader>
          <AlertDialogTitle>Mark this task complete?</AlertDialogTitle>
          <AlertDialogDescription>
            This records the task as completed and closes GitHub issue
    {issueRef}. It can be reopened
            later by setting another status.
          </AlertDialogDescription>
        </AlertDialogHeader>
        <AlertDialogFooter>
          <AlertDialogCancel>Cancel</AlertDialogCancel>
          <AlertDialogAction onClick={() => setStatus('Completed')}>Mark
    complete</AlertDialogAction>
        </AlertDialogFooter>
      </AlertDialogContent>
    </AlertDialog>
  </div>
  )
}
```

frontend/src/pages/employee/EmployeeSettingsPage.jsx

```
import { useState, useEffect } from 'react'
import { Settings as SettingsIcon, User, BellRing, ShieldCheck, Palette as
    PaletteIcon, Save as SaveIcon, Sun as SunIcon, Moon as MoonIcon,
    CheckCircle, AlertTriangle, Pencil } from 'lucide-react'
import { Card, CardContent, CardHeader, CardTitle } from
    '@/components/ui/card'
import { Badge } from '@/components/ui/badge'
import { Button } from '@/components/ui/button'
import { Input } from '@/components/ui/input'
import { Label } from '@/components/ui/label'
import { Switch } from '@/components/ui/switch'
import { Alert, AlertDescription } from '@/components/ui/alert'
import { Avatar, AvatarFallback } from '@/components/ui/avatar'
import { Separator } from '@/components/ui/separator'
import { Tabs, TabsContent, TabsList, TabsTrigger } from
    '@/components/ui/tabs'
import { Select, SelectContent, SelectItem, SelectTrigger, SelectValue }
    from '@/components/ui/select'
import { useAuth } from '@/contexts/AuthContext'
import { useTheme } from '@/contexts/ThemeContext'
import { authApi } from '@/api'
```

```
export default function SettingsPage() {
  const { user, updateUser } = useAuth()
  const { mode, toggleMode } = useTheme()
  const [saving, setSaving] = useState(false)
  const [msg, setMsg] = useState(null)
  const [profileForm, setProfileForm] = useState({ name: '', phone: '',
     gender: '', github_username: '' })
  const [notifications, setNotifications] = useState({
    email_task: true, email_attendance: true, email_reports: false,
    push_task: true, push_attendance: false,
  })

  useEffect(() => {
    if (user) setProfileForm({ name: user.name || '', phone: user.phone ||
     '', gender: user.gender || '', github_username: user.github_username ||
     '' })
  }, [user])

  const handleSaveProfile = async () => {
    setSaving(true); setMsg(null)
    try {
      const res = await authApi.updateProfile(user.id, {
        name: profileForm.name || undefined,
        phone: profileForm.phone || undefined,
        gender: profileForm.gender || undefined,
        github_username: profileForm.github_username.trim(),
      })
      updateUser(res.data)
      setMsg({ type: 'success', text: 'Profile updated successfully!' })
    } catch (err) {
      setMsg({ type: 'error', text: err.response?.data?.detail || 'Failed to
     update profile' })
    } finally { setSaving(false) }
  }

  const handleSaveNotifications = () => {
    localStorage.setItem('trace_notifications',
     JSON.stringify(notifications))
    setMsg({ type: 'success', text: 'Notification preferences saved!' })
  }

  const initials = (user?.name || 'U').split(' ').map(w =>
     w[0]).join('').slice(0, 2).toUpperCase()

  return (
    <div id="settings-page" className="space-y-6 max-w-[900px]">
      <div>
        <h1 className="text-2xl font-bold tracking-tight flex items-center
     gap-2">
          <SettingsIcon className="size-5 text-muted-foreground" /> Settings
        </h1>
        <p className="text-sm text-muted-foreground">Manage your account and
     preferences</p>
      </div>

      {msg && (
        <Alert variant={msg.type === 'success' ? 'default' : 'destructive'}>
          {msg.type === 'success' ? <CheckCircle className="size-4" /> :
     <AlertTriangle className="size-4" />}
          <AlertDescription>{msg.text}</AlertDescription>
        </Alert>
      )}

      <Tabs defaultValue="profile" onValueChange={() => setMsg(null)}>
        <TabsList>
          <TabsTrigger value="profile"><User className="size-3 mr-1"
     />Profile</TabsTrigger>
```

```
     <TabsTrigger value="notifications"><BellRing className="size-3 mr-
1" />Notifications</TabsTrigger>
     <TabsTrigger value="security"><ShieldCheck className="size-3 mr-1"
/>Security</TabsTrigger>
     <TabsTrigger value="appearance"><PaletteIcon className="size-3 mr-
1" />Appearance</TabsTrigger>
   </TabsList>

   <TabsContent value="profile">
     <Card>
       <CardHeader><CardTitle className="text-sm">Profile
Information</CardTitle></CardHeader>
       <CardContent className="space-y-6">
         <div className="flex items-center gap-4 p-4 rounded-lg bg-
muted/50">
           <Avatar className="size-12">
             <AvatarFallback>{initials}</AvatarFallback>
           </Avatar>
           <div className="flex-1">
             <div className="font-medium">{user?.name || '—'}</div>
             <div className="text-xs text-muted-
foreground">{user?.email}</div>
             <div className="flex items-center gap-1.5 mt-1 flex-wrap">
               {user?.roles?.map((r) => <Badge key={r}
variant="secondary">{r}</Badge>)}
               <Badge variant={user?.is_verified ? 'default' :
'secondary'}>
                 {user?.is_verified ? 'Verified' : 'Unverified'}
               </Badge>
             </div>
           </div>
         </div>

         <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
           <div className="space-y-2">
             <Label htmlFor="s-name">Full Name</Label>
             <Input id="s-name" value={profileForm.name} onChange={e =>
setProfileForm({ ...profileForm, name: e.target.value })} />
           </div>
           <div className="space-y-2">
             <Label htmlFor="s-phone">Phone</Label>
             <Input id="s-phone" type="tel" placeholder="+91-
9876543210" value={profileForm.phone} onChange={e =>
setProfileForm({ ...profileForm, phone: e.target.value })} />
           </div>
           <div className="space-y-2">
             <Label htmlFor="s-github">GitHub username</Label>
             <Input id="s-github" placeholder="octocat"
value={profileForm.github_username} onChange={e =>
setProfileForm({ ...profileForm, github_username: e.target.value })} />
             <p className="text-xs text-muted-foreground">Needed so the
GitHub issues for your tasks can be assigned to you.</p>
           </div>
           <div className="space-y-2">
             <Label htmlFor="s-email">Email</Label>
             <Input id="s-email" type="email" value={user?.email || ''}
disabled />
           </div>
           <div className="space-y-2">
             <Label>Gender</Label>
             <Select value={profileForm.gender} onValueChange={v =>
setProfileForm({ ...profileForm, gender: v })}>
               <SelectTrigger><SelectValue placeholder="Prefer not to
say" /></SelectTrigger>
               <SelectContent>
                 <SelectItem value="">Prefer not to say</SelectItem>
                 <SelectItem value="Male">Male</SelectItem>
                 <SelectItem value="Female">Female</SelectItem>
```

```
                  <SelectItem value="Non-binary">Non-binary</SelectItem>
                </SelectContent>
              </Select>
            </div>
          </div>

          <Button onClick={handleSaveProfile} disabled={saving}>
            {saving ? <span className="size-4 border-2 border-primary-
foreground/30 border-t-primary-foreground rounded-full animate-spin"
/> : <SaveIcon className="size-4" />}
            Save Changes
          </Button>
        </CardContent>
      </Card>
    </TabsContent>

    <TabsContent value="notifications">
      <Card>
        <CardHeader><CardTitle className="text-sm">Notification
Preferences</CardTitle></CardHeader>
        <CardContent className="space-y-6">
          <div className="space-y-4">
            <p className="text-xs font-medium text-muted-foreground
uppercase tracking-wider">Email Notifications</p>
            {[
              { key: 'email_task', label: 'Task assignments &
updates' },
              { key: 'email_attendance', label: 'Attendance
reminders' },
              { key: 'email_reports', label: 'Weekly productivity
reports' },
            ].map(item => (
              <div key={item.key} className="flex items-center justify-
between">
                <Label className="font-normal">{item.label}</Label>
                <Switch checked={notifications[item.key]}
onCheckedChange={v => setNotifications({ ...notifications, [item.key]: v
})} />
              </div>
            ))}
          </div>
          <Separator />
          <div className="space-y-4">
            <p className="text-xs font-medium text-muted-foreground
uppercase tracking-wider">Push Notifications</p>
            {[
              { key: 'push_task', label: 'Task deadline reminders' },
              { key: 'push_attendance', label: 'Check-in/out alerts' },
            ].map(item => (
              <div key={item.key} className="flex items-center justify-
between">
                <Label className="font-normal">{item.label}</Label>
                <Switch checked={notifications[item.key]}
onCheckedChange={v => setNotifications({ ...notifications, [item.key]: v
})} />
              </div>
            ))}
          </div>
          <Button onClick={handleSaveNotifications}><SaveIcon
className="size-4" />Save Preferences</Button>
        </CardContent>
      </Card>
    </TabsContent>

    <TabsContent value="security">
      <Card>
        <CardHeader><CardTitle className="text-sm">Security
Settings</CardTitle></CardHeader>
```

```
            <CardContent className="space-y-3">
              {[
                { title: 'Password', sub: 'Last changed: Never', action:
    <Button variant="outline" size="sm"><Pencil className="size-3"
    />Change</Button> },
                { title: 'Two-Factor Authentication', sub: 'Add an extra
    layer of security', action: <Badge variant="secondary">Not
    Enabled</Badge> },
                { title: 'Active Sessions', sub: '1 active session on this
    device', action: <Badge>Current</Badge> },
              ].map(item => (
                <div key={item.title} className="flex items-center justify-
    between p-4 rounded-lg bg-muted/50">
                  <div>
                    <div className="text-sm font-medium">{item.title}</div>
                    <div className="text-xs text-muted-foreground mt-
    0.5">{item.sub}</div>
                  </div>
                  {item.action}
                </div>
              ))}
            </CardContent>
          </Card>
        </TabsContent>

        <TabsContent value="appearance">
          <Card>
            <CardHeader><CardTitle
    className="text-sm">Appearance</CardTitle></CardHeader>
            <CardContent className="space-y-6">
              <div>
                <p className="text-xs font-medium text-muted-foreground
    uppercase tracking-wider mb-3">Theme Mode</p>
                <div className="flex gap-3">
                  {[{ key: 'dark', label: 'Dark', Icon: MoonIcon }, { key:
    'light', label: 'Light', Icon: SunIcon }].map(({ key, label, Icon }) =>
    (
                    <Button
                      key={key}
                      variant={mode === key ? 'default' : 'outline'}
                      onClick={toggleMode}
                      className="flex items-center gap-2"
                    >
                      <Icon className="size-4" /> {label}
                      {mode === key && <CheckCircle className="size-3" />}
                    </Button>
                  ))}
                </div>
              </div>
            </CardContent>
          </Card>
        </TabsContent>
      </Tabs>
    </div>
  )
}
```

5. STANDARDIZATION OF THE CODING

A fixed set of coding standards was followed throughout TRACE, so that every module looks and behaves the same way and a new developer can predict where a rule lives. The standards are written down in the project's conventions document and checked by automated tests where possible.

5.1 Naming and Layout Conventions

Table 37: Coding conventions

Item	Convention
Python names	`snake_case` for functions and variables, `PascalCase` for classes, `UPPER_CASE` for constants (PEP 8)
JavaScript names	`camelCase` for functions and variables, `PascalCase` for React components, one component per file
Database	Table names plural and `snake_case`; foreign keys named `<entity>_id`; every table has `id`, `created_at`, `updated_at`, `deleted_at`
API	Versioned under `/api/v1`; nouns for resources (`/projects/{id}/tasks`), a verb only for actions (`/evaluate`, `/reevaluate`)
Files	One router per resource in `api/v1/`, business rules in `services/`, schemas in `schemas/`, models in `models/`
Frontend styling	Only the shared component library and design tokens; no hard-coded colours; one icon library
Configuration	One environment file (`.env`) for the whole stack; nothing duplicated
Schema changes	Only through Alembic migrations, one concern per migration

5.2 Code Efficiency

- **Asynchronous I/O throughout the backend.** FastAPI routes, SQLAlchemy sessions (asyncpg driver) and the HTTP client for GitHub are all asynchronous, so one server process serves many requests while others wait on the database or on GitHub.
- **Slow work leaves the request.** Every AI call, every Gantt re-plan, every e-mail and every scheduled sweep runs in a Celery worker; the API only queues the job and returns its id. A request never waits for the AI model.
- **Debouncing.** A burst of task completions produces one Gantt re-plan, not one per completion: each trigger is delayed 30 seconds, and a job that finds a newer version already exists does nothing.

- **Deterministic where possible, AI only where needed.** Rescheduling after a status change is plain date arithmetic; the AI is consulted only when the scope changes (the project document is edited). This saves cost and removes a failure mode.
- **Extract once, read many times.** Text is extracted from uploaded files once, at upload, and stored; the AI pipeline never re-parses PDFs or Word files.
- **Bounded AI inputs.** Source text sent to the AI is capped at 60,000 characters, task documents at 4,000 characters each and candidate lists at 25 people, keeping calls fast and cheap.
- **Eager loading.** Related rows (roles of a user, the project of a task) are loaded in the same query with `selectinload`, avoiding the "N + 1 queries" problem.
- **Content-free live events.** WebSocket events only say "refetch", so the server does not serialise data twice, and the socket needs no permission logic of its own.
- **Soft delete with indexed keys.** Rows are marked deleted instead of being removed, and every lookup is by primary or foreign key.
- **Production frontend build.** In production the React application is served as a minified, pre-built bundle.

5.3 Error Handling

Errors are handled consistently at every layer:

- **One exception hierarchy.** All expected errors are subclasses of `TraceException` (an HTTP exception carrying a status code and a readable message): invalid credentials (401), expired or malformed token (401), insufficient permissions (403), not found (404), already exists (409), plus specific ones such as *AI not configured* (503) and *GitHub not configured* (503). FastAPI turns them into a JSON response `{"detail": "…"}`, which the frontend shows in a toast.
- **Meaningful status codes.** 400/422 for invalid input, 401 for authentication, 403 for authorisation, 404 for missing records, 409 for conflicts (repository name clash, evaluation already running, group still holding tasks), 502 when GitHub refuses, 503 when a required service is not configured or reachable.
- **Explicit rollback.** Where an external call follows a database write (a GitHub issue after a task row), the service rolls the transaction back itself on failure, rather than relying on the caller.
- **Warnings instead of failures where appropriate.** Problems that should not block the main change — a missing GitHub username, an invitation GitHub

refused, a pending invitation — are returned as a `warnings` list and shown to the user, while the change itself succeeds.

- **Visible AI failures.** A failed AI job records its reason where people will see it: `projects.ai_errors` (per job, with time and the step that failed) or `tasks.progress_doc_error`. It never leaves stale data looking fresh.
- **Retry on invalid AI output.** An AI answer that fails validation is sent back with the errors for correction, up to two retries.
- **No secret in an error.** Configuration errors are raised in a way that never echoes the values of settings; a changed encryption key gives a clear "enter it again" message instead of a crash.
- **Frontend handling.** A central HTTP client refreshes an expired token automatically and retries the request once; a shared helper turns any error response into a readable message; every page has loading, empty and error states.
- **Logging.** Background jobs log failures with the step and the project id, so problems can be traced in the worker log and on the Admin Monitor page.

5.4 Parameters Calling and Passing

- **Dependency injection.** FastAPI injects the database session, the current user, the GitHub client and the project access object into each route as parameters (`Depends(...)`). Routes therefore never create these themselves, and tests replace them with fakes.
- **Typed parameters.** Every Python function declares the types of its parameters and return value. Path parameters (`project_id: int`), query parameters (`start: date`, `end: date`) and request bodies (Pydantic models) are converted and validated by FastAPI before the function runs.
- **Keyword-only arguments** are used for optional behaviour (for example `notify(..., task=..., exclude=actor)`, `run_evaluate_project(db, project_id, *, job_id, ai, github)`), which makes calls self-describing and prevents arguments in the wrong order.
- **Pass by object reference.** Python passes references to objects; services receive the ORM objects they need (a `Task`, a `ProjectAccess`) rather than re-loading them, and modify them within the caller's transaction. The caller commits.
- **Jobs receive ids, not objects.** Background jobs are queued by name with plain arguments (ids, strings, ISO dates), because objects cannot cross process boundaries; the job loads fresh rows in its own session.

- **Frontend props.** React components receive data and callbacks as props (for example a task board receives the project, its tasks and an `onChanged` callback) and never fetch permissions on their own.

5.5 Validation Checks

Table 38: Validation checks

Where	Checks
Login and registration	E-mail format, required fields, unique e-mail and username, minimum password length; public registration cannot choose a privileged role
Project	Name 2–255 characters; the name must yield a valid repository slug; editable status limited to Draft, Active, Completed, Archived
Source files	Type must be PDF, DOCX, MD or TXT; size at most 20 MB; file must be readable; rejected files are listed with the reason
Task group	Name 1–255 characters; duration at least 1 day; must belong to the project
Task	Title 2–255 characters; priority one of Low, Medium, High, Critical; estimated hours 0–999; group must belong to the project; assignee must be an active Employee
Task status	One of the seven statuses; the caller must be allowed to set it (assignee or Project Manager); Overdue is refused for everyone
Project document	1–200,000 characters; an unchanged save commits nothing
Discussion	Message 1–5,000 characters; a reply must belong to the same task; caller must be allowed to discuss
Report download	Start and end dates required; end not before start
Settings	Organization name and e-mail formats; mail port range; secrets write-only
Bulk upload	UTF-8 CSV; known column names; each value coerced to its type (integer, date, time, e-mail, one of a list); required fields present; references (department, employee code) must exist
AI output	Must match its schema: lengths, ranges, unique group names, dependencies only on earlier groups, suggested people only from the candidate list
Gantt version	Range covers every bar and milestone; one bar per group; start ≤ end; completion 0–100; dependencies between existing bars and never to itself
Frontend forms	Required fields and formats are checked before submission; the server repeats every check

6. TESTING

Testing is the process of executing the software with the intent of finding errors, and of showing that it meets its requirements. TRACE was tested continuously: every build increment added its tests, and the complete suite was run before an increment was considered finished.

6.1 Testing Techniques and Testing Strategies Used

6.1.1 Testing Techniques

Black-box testing

Black-box testing examines the functionality of the system without looking at its internal structure; test cases are derived from the specification. In TRACE almost every test calls the real HTTP API as a client would and checks only the response and the visible effects (the status code, the returned data, the GitHub issue created, the notification sent). Techniques used:

- **Equivalence partitioning** — for example, file uploads were tested with one representative of each class: supported and readable, supported but unreadable, unsupported type, over the size limit.
- **Boundary value analysis** — the 20 MB upload limit, 1 and 20 task groups, 1 and 365 days, 2-character titles, the end date equal to and before the start date of a report.
- **Decision-table testing** — the *role matrix*: every combination of caller (owner Manager, other Manager, Admin, Project Manager, Member, outsider, anonymous) and project operation, with the expected answer (allowed, 403, 401).
- **State-transition testing** — task status changes in both directions (completing and re-opening, cancelling), project states through evaluation success and failure.
- **Use-case testing** — end-to-end flows such as *Manager → Project Manager → task → GitHub issue*.
- **Error guessing** — GitHub refusing an invitation, an assignee whose invitation is still pending, a repository that is already gone, an AI that keeps returning invalid JSON, a changed encryption key.

White-box testing

White-box (structural) testing uses knowledge of the internal code. It was used for the pure functions where every branch matters:

- the **scheduling rule** — groups with and without dependencies, with a hand-planned span, completed groups, and unfinished groups whose end has passed (the slip branch);
- the **AI retry loop** — success on the first attempt, success after a retry, failure after the last retry;
- the **status-authority function** — every branch of `may_set_status` for assignee, Project Manager and others;
- the **Gantt schema validator** — each rule that can reject a chart;
- the **repository slug** function — letters, digits, punctuation, long names, names without letters or digits.

Coverage criteria applied were **statement coverage** and **branch (decision) coverage** of these functions.

Grey-box testing

Many tests combine both views: they drive the system through its public API (black box) but then inspect the database or the fake GitHub's recorded state (white box) — for example, checking that after a failed evaluation *no* task group from the failed run remains in the database.

6.1.2 Testing Strategies

Table 39: Levels of testing applied

Level	How it was applied in TRACE
Unit testing	Individual functions and endpoints in isolation, with GitHub, the AI model, MinIO and Redis replaced by fakes that behave like the real services
Integration testing	Routes, services, the ORM and a real PostgreSQL test database working together; the real GitHub client code runs against a fake GitHub transport
System testing	Complete flows across modules through the HTTP API: project creation to issue, evaluation to Gantt, completion to notification and re-plan
Regression testing	The full suite was run after every change; bugs that were found received a test that pins the fix
Security testing	A static audit that every route declares an authorisation guard; role matrices; checks that secrets are never returned and are encrypted at rest; production pre-flight refusals
Migration testing	An empty database is migrated to the latest revision and compared with the models (no drift); only one migration head may exist
Live acceptance testing	An opt-in test that runs the real code path against real GitHub (create repository, name clash, commit documents, assign and close issues, archive)
User acceptance testing	Walk-through of each panel's flows by the users of each role

6.2 Testing Plan Used

Table 40: Test plan

Item	Plan
Objective	Verify every functional requirement (FR-01 … FR-30) and the security rules before an increment is accepted
Scope	All backend routes, services, background job bodies, schemas and migrations; frontend verified by building and by walk-through
Test environment	Linux; PostgreSQL and Redis in Docker; a dedicated test database `trace_test` built by Alembic migrations; Python virtual environment
Tools	pytest, pytest-asyncio, HTTPX asynchronous test client, a fake GitHub on `httpx.MockTransport`, a scripted fake AI, a fake object store
Isolation	Autouse fixtures block real GitHub and MinIO in every test, record every queued job and every live event, and pin an empty AI key, so no test reaches a real model, queue or worker
Test data	A shared "world" fixture per test: an Admin, two Managers, a Project Manager, Members and an outsider, with a project and repository
Entry criteria	Code of the increment complete; database migrated to head
Exit criteria	Whole suite passes; frontend build passes; the increment's "done when" condition demonstrated
Responsibility	The developer writes the tests together with the code of each increment
Schedule	Continuous during every build phase, followed by the dedicated testing activity (J) of the project plan

6.3 Test Reports

The suite contains **218 test functions**; the role-matrix and other parametrised tests expand them to several hundred test cases. At the Phase 5 hand-off the full run reported **374 passed and 2 skipped** (the two skipped tests are the opt-in live GitHub tests), and the notification, discussion and administration tests were added after that. The tables below report representative cases from the suite.

6.3.1 Unit Test Cases

Authentication

Table 41: Unit tests — authentication

Test Id	Test case	Expected output	Actual output	Result
UT-01	Log in with a valid e-mail and password	200; access and refresh tokens and roles returned	Tokens returned	Pass

Test Id	Test case	Expected output	Actual output	Result
UT-02	Log in with a wrong password	401 "Invalid email or password"	401 returned	Pass
UT-03	Log in with an e-mail that does not exist	401, same message (no account enumeration)	401 returned	Pass
UT-04	Log in with an empty body	422 validation error	422 returned	Pass
UT-05	Refresh with a valid refresh token	New access token	Token issued	Pass
UT-06	Refresh with an invalid token	401	401 returned	Pass
UT-07	Register without a role	Account created as Employee	Employee created	Pass
UT-08	Anonymous registration asking for Admin	403 — privileged roles need an Admin	403 returned	Pass
UT-09	Admin registers a Manager	201	Manager created	Pass
UT-10	Register with an e-mail already used	409 already exists	409 returned	Pass

Projects and GitHub

Table 42: Unit tests — projects and repositories

Test Id	Test case	Expected output	Actual output	Result
UT-11	Admin or Project Manager tries to create a project	403	403 returned	Pass
UT-12	Manager creates "Mobile App: v2 (Beta)!"	Public repo `acme/mobile-app-v2-beta` created with issues enabled; project Draft	As expected	Pass
UT-13	Project name with no letters or digits	422 — no valid repository name	422 returned	Pass
UT-14	Repository name already exists	409 naming the repository; nothing created	409, no row	Pass
UT-15	No GitHub token or organization configured	503; no project	503 returned	Pass
UT-16	Token lacks the Administration permission	502 with a precise message	Message returned	Pass
UT-17	Upload PDF, DOCX, MD and an unsupported file	Supported files stored and text extracted; unsupported reported	As expected	Pass
UT-18	A non-owner uploads a source file	403	403 returned	Pass
UT-19	Delete a project with open tasks	409; project kept	409 returned	Pass

Test Id	Test case	Expected output	Actual output	Result
UT-20	Delete a project; archiving the repo fails	Project kept, error returned	As expected	Pass
UT-21	Managers list projects	Each sees only their own	As expected	Pass

Team and assignment

Table 43: Unit tests — team and assignment

Test Id	Test case	Expected output	Actual output	Result
UT-22	Appoint a Project Manager	PM row created; invited to the repo with Write	As expected	Pass
UT-23	Appoint a PM with no GitHub username	Appointed, with a warning	Warning returned	Pass
UT-24	GitHub refuses the invitation	Appointment still succeeds, with a warning	As expected	Pass
UT-25	Appoint a Manager or Admin as PM	422 — only Employees	422 returned	Pass
UT-26	Assign a task to a pending invitee	Issue left unassigned, "invitation pending" warning	As expected	Pass
UT-27	Remove a PM who holds no tasks	Access revoked, pending invitation withdrawn	As expected	Pass
UT-28	Remove a PM who holds tasks	Stays as Member, keeps repository access	As expected	Pass

Tasks and status authority

Table 44: Unit tests — tasks and status authority

Test Id	Test case	Expected output	Actual output	Result
UT-29	Member tries to create a task	403	403 returned	Pass
UT-30	Create a task; GitHub fails	Task rolled back; no row	No row	Pass
UT-31	Create a task in another project's group	422	422 returned	Pass
UT-32	Assignee sets own task to Completed	200; completed_by = Assignee; issue closed	As expected	Pass
UT-33	Assignee tries Cancelled or Overdue	403	403 returned	Pass
UT-34	Member changes someone else's task	403	403 returned	Pass
UT-35	Manager or Admin tries to complete a task	403	403 returned	Pass

Test Id	Test case	Expected output	Actual output	Result
UT-36	Status sent through the generic edit endpoint	Ignored — status unchanged	Unchanged	Pass
UT-37	PM completes a task	Issue closed; group roll-up updated	As expected	Pass
UT-38	`allowed_statuses` for each role	Exactly the statuses `may_set_status` allows	Match	Pass
UT-39	Delete a task	Soft-deleted; issue closed as not planned	As expected	Pass
UT-40	Delete a group that holds tasks	409	409 returned	Pass

AI pipeline, Gantt and scheduling

Table 45: Unit tests — AI pipeline and Gantt

Test Id	Test case	Expected output	Actual output	Result
UT-41	AI returns invalid JSON once, then valid	Errors sent back; valid result accepted	As expected	Pass
UT-42	AI returns invalid JSON three times	`AIOutputInvalid` raised; nothing saved	As expected	Pass
UT-43	Plan with repeated names or a dependency on a later group	Rejected by the schema	Rejected	Pass
UT-44	Suggestion naming a non-candidate user	Rejected by the schema	Rejected	Pass
UT-45	Evaluate while running / without text / with tasks	409 each	409 returned	Pass
UT-46	Evaluate without AI or GitHub configured	503	503 returned	Pass
UT-47	Scheduling with dependencies and a late group	Dependants start after; slip stretches to today	As expected	Pass
UT-48	Save an inconsistent Gantt chart	Rejected; nothing persisted	As expected	Pass
UT-49	Versions are appended	New version number each time; old kept	As expected	Pass
UT-50	No Gantt yet	`null`, not an error	`null`	Pass

Settings, notifications and administration

Table 46: Unit tests — settings, notifications and administration

Test Id	Test case	Expected output	Actual output	Result
UT-51	Save the GitHub token	Stored encrypted; never returned — only "set"	As expected	Pass
UT-52	Omit the token on save / send empty	Omitted keeps it; empty clears it	As expected	Pass
UT-53	Encryption key changed after saving	Clear 503 asking to re-enter the secret	As expected	Pass
UT-54	Generic Control Panel CRUD on `system_settings`	Table not available	Not listed	Pass
UT-55	Test connection with a bad token	Failed check with GitHub's message	As expected	Pass
UT-56	Mail body containing HTML	Escaped in the e-mail	Escaped	Pass
UT-57	Bulk upload with good and bad rows	Good rows added; bad rows reported with line numbers	As expected	Pass
UT-58	Notifications without a login	401	401 returned	Pass
UT-59	WebSocket with a bad token	Connection refused	Refused	Pass
UT-60	Report with end before start	422	422 returned	Pass

6.3.2 System Test Cases

Table 47: System tests

Test Id	Scenario	Steps and expected outcome	Actual outcome	Result
ST-01	Manager to issue	Manager creates project → appoints PM → PM creates group and task with assignee → GitHub issue #1 exists, titled as the task, assigned to the Member's GitHub account, body contains the description	As expected	Pass
ST-02	Evaluate Project	Upload source text → evaluate → project document committed twice, task groups created by AI, Gantt v1 with reason *initial*, project Active, owner notified	As expected	Pass
ST-03	Failed evaluation	AI fails at planning → project back to Draft, `ai_errors` names the step, no groups from the failed run remain, owner notified of failure	As expected	Pass

Test Id	Scenario	Steps and expected outcome	Actual outcome	Result
ST-04	Completion flow	Assignee completes task → issue closed, group roll-up updated, PMs notified live and by e-mail, Gantt re-plan queued with 30 s delay	As expected	Pass
ST-05	Burst of completions	Several completions in quick succession → exactly one new Gantt version	One version	Pass
ST-06	Project document edit	PM saves a changed document → AI revises durations / adds groups → new version *project_doc_updated*; unchanged save commits nothing	As expected	Pass
ST-07	AI failure on re-plan	AI call fails during a document re-plan → chart still re-planned, `ai_errors` records the failure	As expected	Pass
ST-08	Overdue sweep	Past-due open tasks → Overdue, assignee and PMs notified, one re-plan per project	As expected	Pass
ST-09	Due reminders	Only tasks due tomorrow produce a reminder	As expected	Pass
ST-10	Discussion	Assignee posts, PM replies → both see the thread; an outsider Member and the Admin get 403; participants notified	As expected	Pass
ST-11	Weekly progress report	Job reads task documents → progress and report documents committed, URLs stored	As expected	Pass
ST-12	Report download	Manager downloads CSV for a date range → only their projects, tasks created, due or completed in range	As expected	Pass
ST-13	Role matrix	Every project route × {owner, other Manager, Admin, PM, Member, outsider, anonymous} → expected allow / 403 / 401	All as expected	Pass
ST-14	Route guard audit	Every registered route declares a guard; only `/health` and login / refresh / register are public	As expected	Pass
ST-15	Fresh installation	Empty database migrated to head; no difference from the models; application start does not create tables	As expected	Pass
ST-16	Production pre-flight	Production refuses weak JWT secret, debug mode, missing encryption key and seed passwords, without printing secrets	As expected	Pass
ST-17	Live GitHub (opt-in)	Real organization: connection test, repository creation, name clash, document commit, issue assign / close, archive on delete	Passed	Pass

6.4 Debugging and Code Improvement

Debugging was driven by failing tests and by review of each increment. The most instructive defects found, and the improvements made, were:

Table 48: Defects found and improvements made

No.	Defect or weakness found	Correction / improvement
1	A legacy generic task-update endpoint accepted a `status` field, so any signed-in user could mark any task complete	Removed; status got its own endpoint that enforces status authority in one place
2	Public registration trusted a caller-supplied role, so anyone could register as Admin	Public registration now always creates an Employee; privileged accounts need an Admin token
3	Several service endpoints were reachable without authentication	Every route now declares a guard, and a static audit test fails if one does not
4	A fresh database could not reach the latest migration, because some tables had been created at start-up by `create_all` instead of by a migration	A catch-up migration was inserted and `create_all` removed; a test now migrates an empty database and checks for drift
5	Creating a task flushed the row before calling GitHub; on a GitHub error the row could survive	The service rolls back explicitly on failure; the test harness was changed to behave like the real session
6	Reading `created_at` of a new row raised an asynchronous lazy-load error	Rows are refreshed after insert before being returned
7	The attendance log endpoint crashed on columns that no longer existed	Fixed; found by the new role matrix
8	The generic Control Panel exposed the settings table, which would have returned the token and stored it unencrypted	Table removed from the generic CRUD; a dedicated settings API is the only writer
9	A malformed encryption key produced an unexplained server error	Now a clear 503 naming the key; production refuses to start with it
10	GitHub refuses a pending invitee as issue assignee	The service retries unassigned and warns; the fake GitHub enforces the same rule
11	Background jobs shared the web server's connection pool across event loops	Each job uses its own session without pooling
12	The old chat WebSocket was an open, unauthenticated broadcast room	Removed; the new socket requires a valid token and carries no data
13	E-mail bodies inserted text into HTML unescaped	Escaped

No.	Defect or weakness found	Correction / improvement
14	The production pre-flight, as first written, printed part of a secret in its error	Moved so that errors never echo settings; a test with canary values pins it
15	An AI call on every Gantt re-plan added cost and a failure mode	Re-plans after status changes became deterministic; the AI is consulted only when the project document changes

7. SYSTEM SECURITY MEASURES

Security was treated as a requirement of every increment, not a final step. This chapter describes how TRACE protects its data and how user profiles and access rights are created and enforced.

7.1 Database and Data Security

Protection of credentials and secrets

- **Password hashing.** User passwords are hashed with **bcrypt** using 12 rounds of salting. The plain password is never stored or logged, and a hash cannot be reversed; login compares the entered password against the hash.
- **Encryption of stored secrets.** Secrets entered by the Admin — the GitHub token and the SMTP password — are encrypted with **Fernet** (authenticated symmetric encryption) before they are written to `system_settings`. The encryption key lives only in the server's `.env` file, never in the database. A reader of the database, a backup or the Control Panel therefore sees only ciphertext.
- **Write-only secrets.** No API ever returns a secret. The Configuration screen shows only whether a token is set and where it comes from (*Not set / Set here / Set in .env*). The settings table is deliberately excluded from the Admin's generic table editor, which would otherwise be able to read or overwrite it unencrypted.
- **One configuration file.** Database, Redis, MinIO, AI and JWT credentials are kept in a single `.env` file on the server, excluded from version control; a template without values is provided instead.

Protection of data in the database

- **No direct access for end users.** Users reach data only through the API; the application connects to PostgreSQL with a single role whose password is known only to the server.
- **SQL injection prevention.** All queries are built with the SQLAlchemy ORM and executed with bound parameters; user input is never concatenated into SQL.
- **Input validation.** Every request body, path and query parameter is validated by Pydantic schemas before it reaches the business logic (types, lengths, ranges, allowed values).
- **Referential integrity and constraints** (Section 3.3) prevent inconsistent data even if a bug slipped past the application.

- **Soft deletion.** Records are marked deleted instead of being erased, preserving history and references; deleted rows are excluded from every query.
- **Atomic changes.** Each request runs in one transaction; a change that also needs GitHub (a task and its issue, a project and its repository) is either completed fully or rolled back.
- **Schema control.** The schema is changed only by reviewed, versioned migrations; a test fails if the models and the migrations disagree.

Protection of files and generated documents

- Uploaded requirement files are stored privately in MinIO and are served only through the authorised API.
- Generated documents live in the project's GitHub repository, which has its own history (`git log`) — nothing can be changed without a trace. Repositories are public by an explicit, stated design decision of this educational project, and the upload screen warns users not to put confidential material in requirement documents.

Protection in transit and at the edge

- In production, all traffic uses **HTTPS**; the application is published through a Cloudflare tunnel, so the server opens no inbound port and the API binds only to the local interface.
- The frontend is served as a production build (not the development server) in production.
- **Production pre-flight.** The server refuses to start in production with a weak JWT secret, debug mode on, a missing public URL, a missing or invalid encryption key, or any seeded account still using its published demo password — and the refusal never prints the settings values. Demo accounts and demo data are not seeded in production. A command is provided to change a seeded password.

Protection of external accounts

- TRACE uses one **fine-grained GitHub token** scoped to the company Organization with only the permissions it needs (Administration, Contents, Issues read/write; Metadata read), not a classic full-access token, so a leaked token exposes only TRACE's repositories.
- The token cannot delete repositories; deleting a project **archives** its repository instead.
- Collaborators are invited with *Write* permission per repository, and access is revoked when a Project Manager leaves a project.

- The AI key, model and endpoint are fixed in `.env` and cannot be changed by any user through the application.

7.2 Creation of User Profiles and Access Rights

Creation of user profiles

- The first **Admin** account is created by the installation seed (in local development) or by the operator, and its password must be changed before production will start.
- **Admin and Manager accounts** can only be created by an Admin — through the Workforce screen or the registration endpoint called with an Admin token.
- **Public self-registration** always creates an **Employee** account; any requested privileged role is refused.
- Each user maintains their own profile — name, contact details, password and **GitHub username** — through the Settings page; a user can never edit another user's profile.
- The Admin maintains the workforce record (employee code, department, designation, shift, reporting manager, status) for each employee, individually or by CSV bulk upload. Setting a user *Inactive* or *Blocked* stops them from being chosen for projects.

Authentication

- Login returns a short-lived **access token** and a longer-lived **refresh token**, both JWTs signed with the server secret and carrying their type, so one cannot be used as the other.
- The browser sends the access token with every request; when it expires, the client uses the refresh token once to obtain a new one, and signs the user out if that fails.
- The WebSocket connection is authenticated with the same token; an invalid token is refused.

Access rights

Access is controlled in **two layers** on the server; the interface only reflects the result.

1. **Platform role on every route.** Every API route declares which of Admin, Manager and Employee may call it. An automated audit fails the build if any route has no guard; the only public routes are the health check and login, token refresh and registration.
2. **Project role on every project and task operation.** The caller's role in the specific project is resolved from `project_assignments`: the owning Manager administers the project; a Project Manager runs its structure; a

Member works on their own tasks; any other Employee gets 403. Managers see only the projects they own; the Admin can read every project but change none of them through the project screens.

Table 49: Summary of access rights by role

Role	Rights
Admin	All administration: Control Panel, Workforce, Monitor, Configuration. Reads every project; takes no part in project delivery
Manager	Creates projects and uploads their documents; evaluates them; appoints and removes Project Managers; reads progress reports; discusses tasks of own projects; downloads reports. No HR functions
Project Manager	Edits the living project document; creates task groups and tasks; assigns Members; sets any task status (including completing on someone's behalf and cancelling); requests re-plans and suggestions
Member	Sees the project, tree and Gantt; works on their own tasks; sets their own task's status, including Completed; discusses their tasks
System	Sets *Overdue* (nightly sweep) — no person can

Status authority is enforced in one function on the server, and API responses carry the permissions (`can_edit`, `can_discuss`, `allowed_statuses`) derived from the same check, so a user can never be shown an action the server would refuse, and a crafted request for such an action is refused anyway.

A user who opens another role's panel sees a **403 page** that names their actual role, rather than being silently redirected, so a wrongly provisioned account is noticed immediately.

8. REPORTS

TRACE produces the reports below. Sample layouts follow.

Table 50: Reports generated by the system

Report	Trigger	Recipient / where
Project report (CSV) for a date range	On demand — *Download report* on the dashboard	Manager (own projects), Employee (projects they are on)
Project progress document	Weekly (Mondays 07:00) and on demand	Manager and team — Progress Report tab; repository file
Project report document	Weekly (Mondays 07:00) and on demand	Manager — Progress Report tab; repository file
Project document (initial and living)	Evaluate Project; edits by the PM	Everyone on the project — Overview; repository files
Task progress document	On task creation	Assignee and team — task page; repository file
Gantt chart version	Evaluation and every re-plan	Everyone on the project — Gantt tab, with version history
Dashboard statistics	Live	Manager and Employee dashboards
Notifications and e-mails	Assignment, due, overdue, completion, evaluation, discussion	The people concerned
Service health and job outcomes	Live	Admin — Monitor page

8.1 Project Report (CSV Download)

The dashboard's **Download report** action asks for a start date and an end date (and, optionally, one project) and downloads a CSV file named `trace-report-<start>-to-<end>.csv`. A task is included if it was created, due or completed within the range; each row shows its status as of today. The file opens in any spreadsheet program.

Table 51: Columns of the CSV project report

Column	Meaning	Example
Project	Project name	Storefront
Project status	Draft, Active, Completed, Archived	Active
Task group	Module the task belongs to	Backend Foundation
Group completion %	Completed ÷ non-cancelled tasks of the group	66.67
Task	Task title	Cart with coupons

Column	Meaning	Example
Status	Status of the task today	In Progress
Priority	Low, Medium, High, Critical	High
Assignee	Name of the Member	Dan K.
Created	Date the task was created	2026-09-05
Due	Due date	2026-09-20
Completed	Completion date, if completed	(empty)
Completed by	Assignee or Project Manager	(empty)
GitHub issue	Link to the task's issue	https://github.com/…/issues/3

Sample of the downloaded file:

```
Project,Project status,Task group,Group completion
%,Task,Status,Priority,Assignee,Created,Due,Completed,Completed by,GitHub issue
Storefront,Active,Backend Foundation,66.67,Auth API,Completed,High,Asha R.,2026-09-
01,2026-09-10,2026-09-09,Assignee,https://github.com/…/issues/1
Storefront,Active,Backend Foundation,66.67,Product catalogue API,Completed,Medium,Dan
K.,2026-09-02,2026-09-14,2026-09-15,Project Manager,https://github.com/…/issues/2
Storefront,Active,Backend Foundation,66.67,Cart with coupons,In Progress,High,Dan
K.,2026-09-05,2026-09-20,,,https://github.com/…/issues/3
Storefront,Active,Checkout,0.00,Payment gateway,Pending,Critical,,2026-09-12,2026-09-
30,,,https://github.com/…/issues/4
```

(Sample data for illustration.)

8.2 Project Progress Document and Report Document

Both are Markdown files written by the AI every week into the project's repository (`docs/project_progress_doc.md` and `docs/project_report_doc.md`) and rendered on the *Progress Report* tab with a **Rewrite now** button. Sample layout of the report document:

```
# Storefront — Weekly Report (week of 21 Sep 2026)

## Summary
Overall completion 42%. Backend Foundation is nearly done; Checkout has not started.
The project is on track for its planned end date of 30 Nov 2026.

## Progress by task group
| Task group          | Completion | Status      | Planned end |
|---------------------|-----------:|-------------|-------------|
| Backend Foundation  |   66.7 %   | In Progress | 25 Sep 2026 |
| Checkout            |    0.0 %   | Pending     | 20 Oct 2026 |

## Completed this week
- Auth API (Asha R.)
- Product catalogue API (Dan K., one day late)

## Risks and blockers
```

```
- Cart with coupons depends on the pricing rules still under discussion.

## Next week
- Finish the cart; start the payment gateway integration.
```

8.3 Task Progress Document

Written by the AI when a task is created (`docs/tasks/task-<id>.md`) and shown on the task page with its checklist:

```
# Task 3 — Cart with coupons

## Steps
- [ ] 1. Cart model — items, quantities, totals per user
- [ ] 2. Coupon rules — percentage and fixed discounts, expiry
- [ ] 3. API endpoints — add, remove, apply coupon, view cart
- [ ] 4. Tests — totals, invalid and expired coupons

## Acceptance criteria
- A user can add and remove items and see the correct total.
- An expired or unknown coupon is refused with a clear message.
```

8.4 Gantt Chart

The Gantt tab shows one bar per task group with its completion, dependency arrows, a *today* line and the project's end milestone; a bar expands to show its tasks as markers on their due dates. A version picker shows every earlier version with its number, date and reason (*initial*, *task_status_change*, *project_doc_updated*, *manual*).

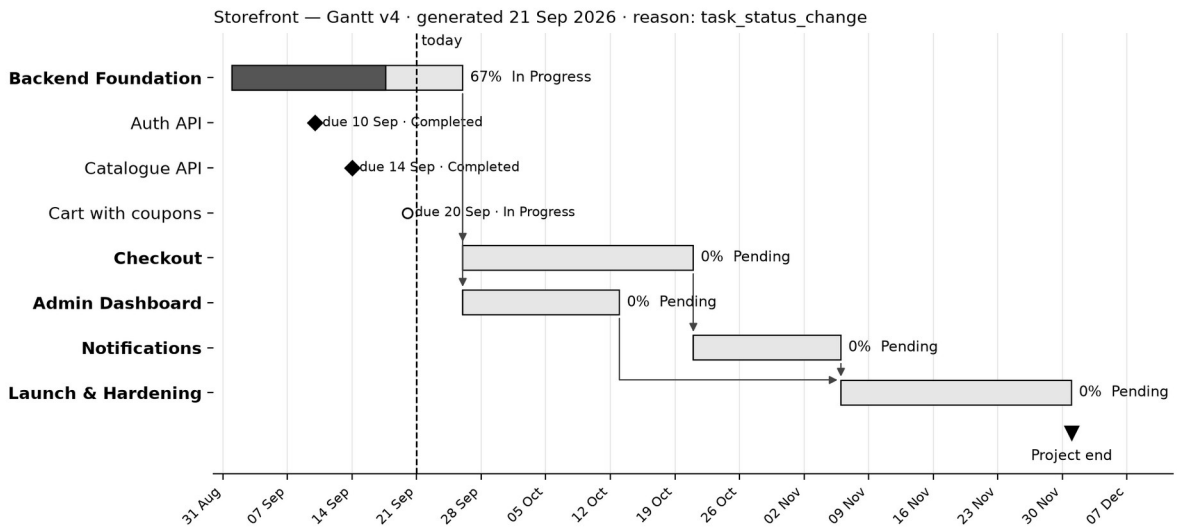


Figure 23: Sample layout — Gantt chart of a project as shown in TRACE

8.5 Dashboard Statistics

Table 52: Manager dashboard — statistics shown

Item	Content
Stat row	Total, active and completed projects; overall completion; number of at-risk projects
Project cards	Name, status, progress bar, Project Managers, open and overdue task counts
At-risk projects	Projects with overdue tasks or past their end date
Recent activity	Latest task changes across the Manager's projects

Table 53: Employee dashboard — statistics shown

Item	Content
Progress	Completion across the projects the Employee is on
My tasks	Open tasks grouped by due state: overdue, due soon, later, no date
Overdue callout	Count and list of overdue tasks
Notifications	Most recent notifications

8.6 Notification E-mail

```
Subject: New task: Cart with coupons

You were assigned 'Cart with coupons' in Storefront.

Open in TRACE: https://<trace-host>/employee/tasks/3
```

9. OUTPUT

This chapter shows the screens of TRACE as each type of user sees them, taken from the running application with its sample data. The main screen of each type of user is in Section 3.6; every other screen follows here, grouped by user — Visitor, Admin, Manager and Employee. The Settings screens are the same for every user and are shown once, for the Employee. Where a screen is longer than the browser window, its first part is shown.

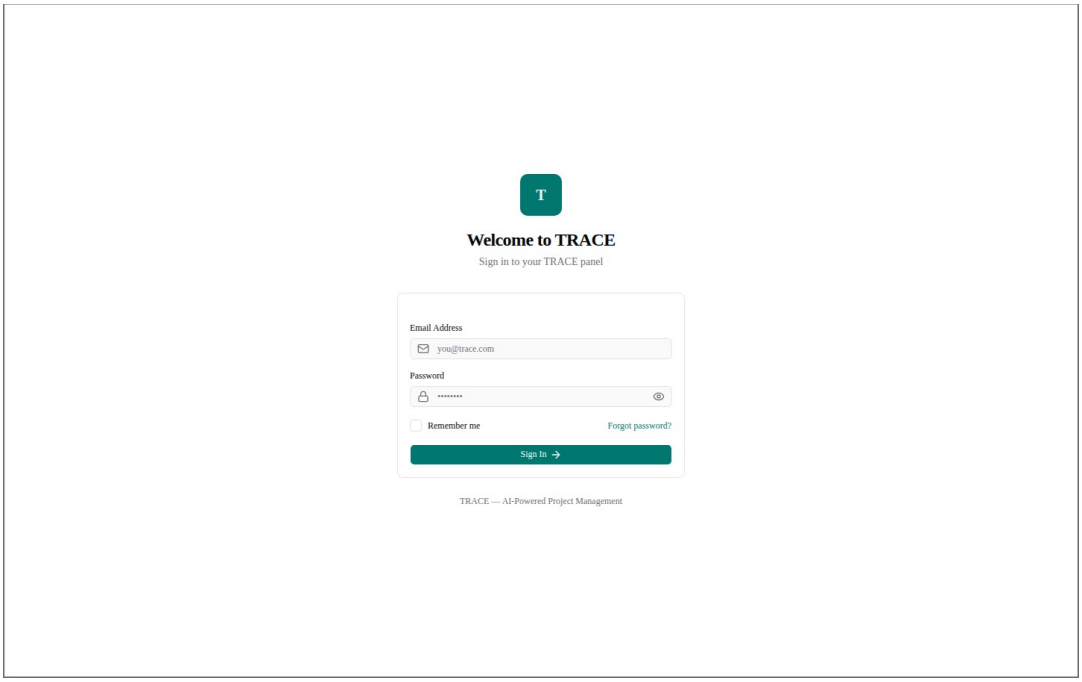


Figure 24: Visitor — Sign In

Sign-in with e-mail and password; each role is taken to its own panel.

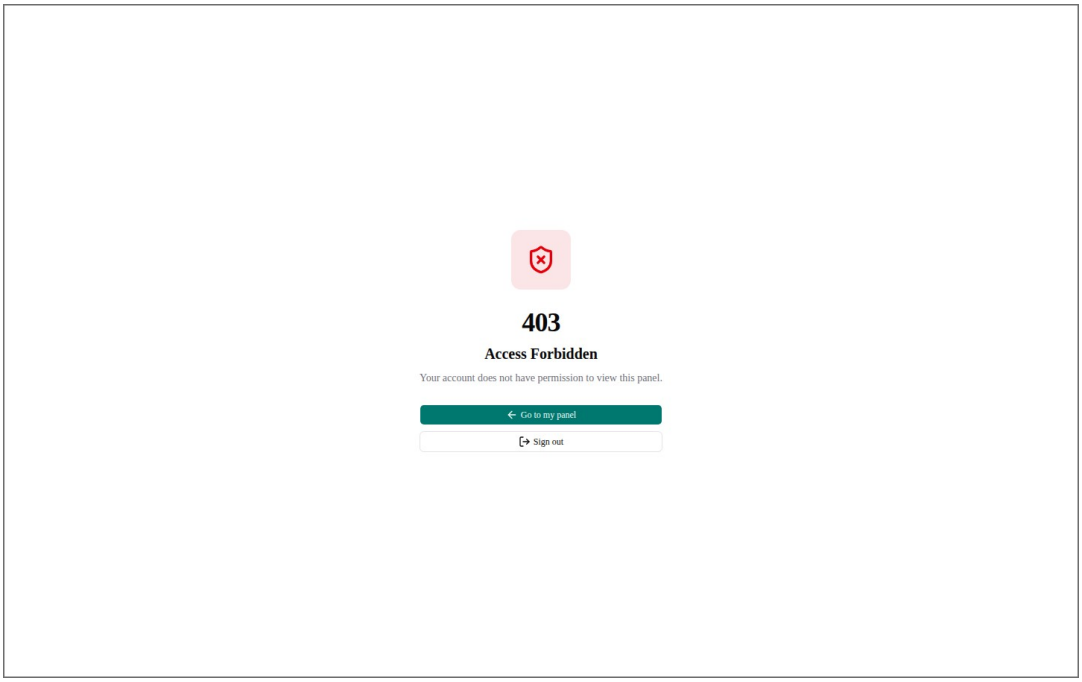


Figure 25: Visitor — Access Forbidden (403)

Shown when an account opens a panel its role may not use, with the way back to its own.

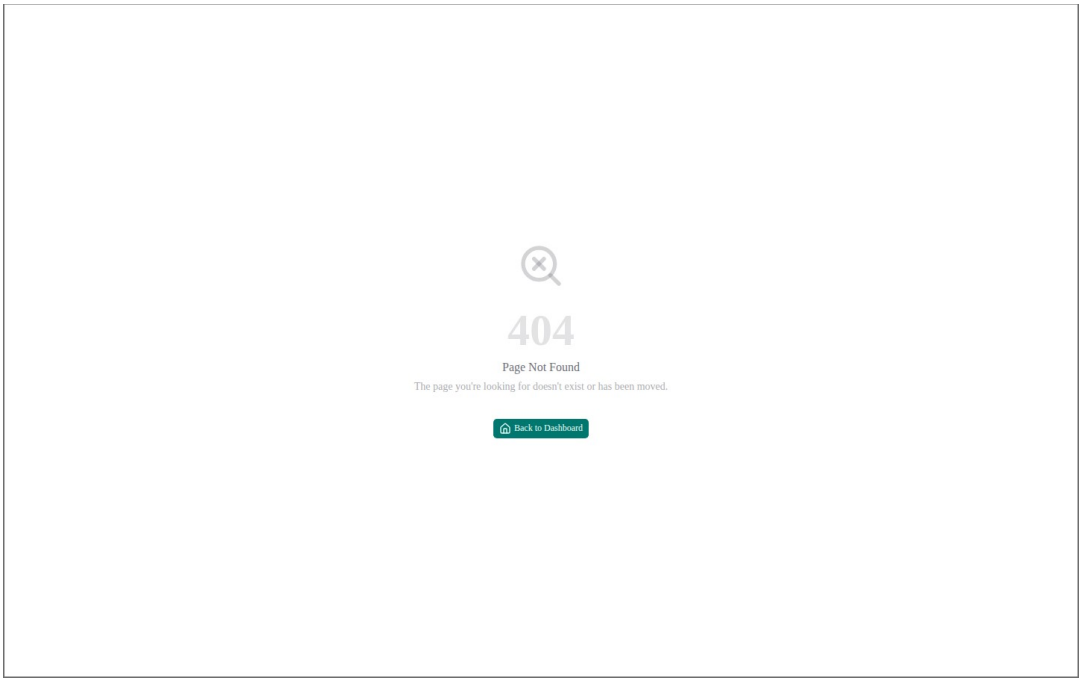


Figure 26: Visitor — Page Not Found (404)

Shown for any address the application does not have.

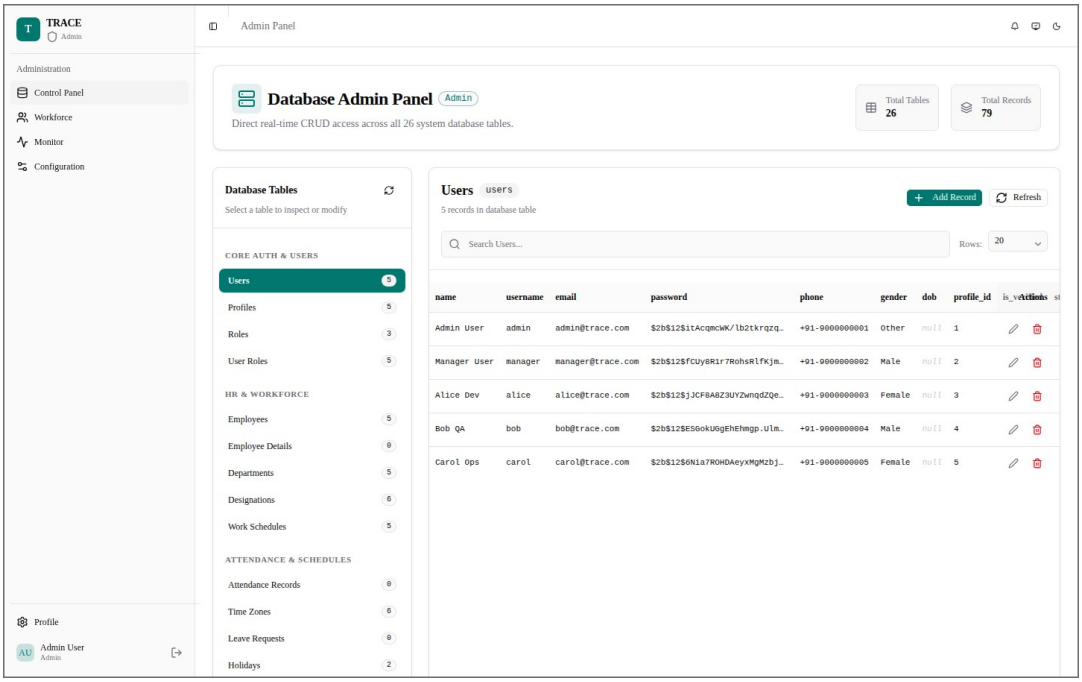


Figure 27: Admin — Control Panel — Users

The rows of the users table, with search, paging, and edit and delete on every row.

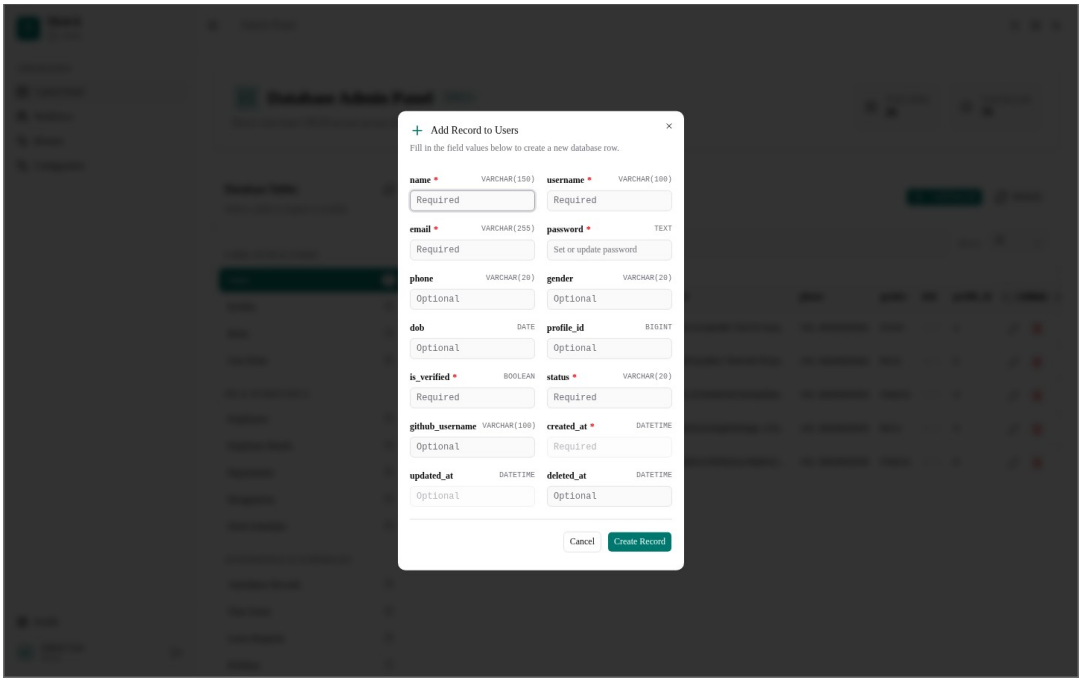


Figure 28: Admin — Control Panel — Add Record

A new row for any table; the form is built from the table's columns, types and required fields.

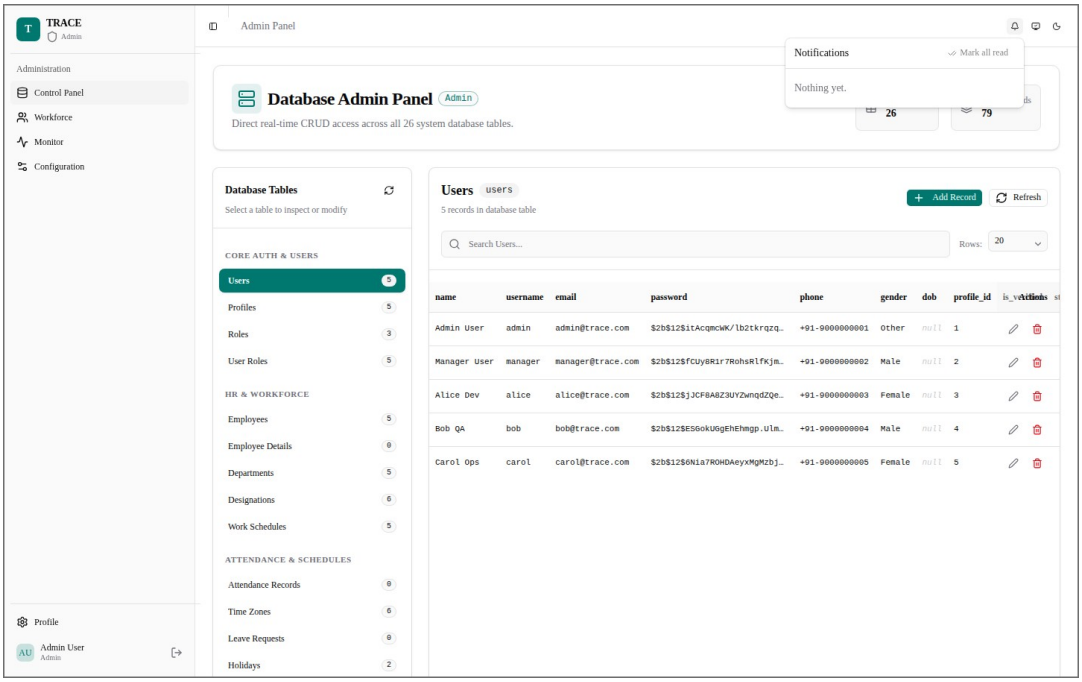


Figure 29: Admin — Notifications

The notification bell's list, with mark-all-read.

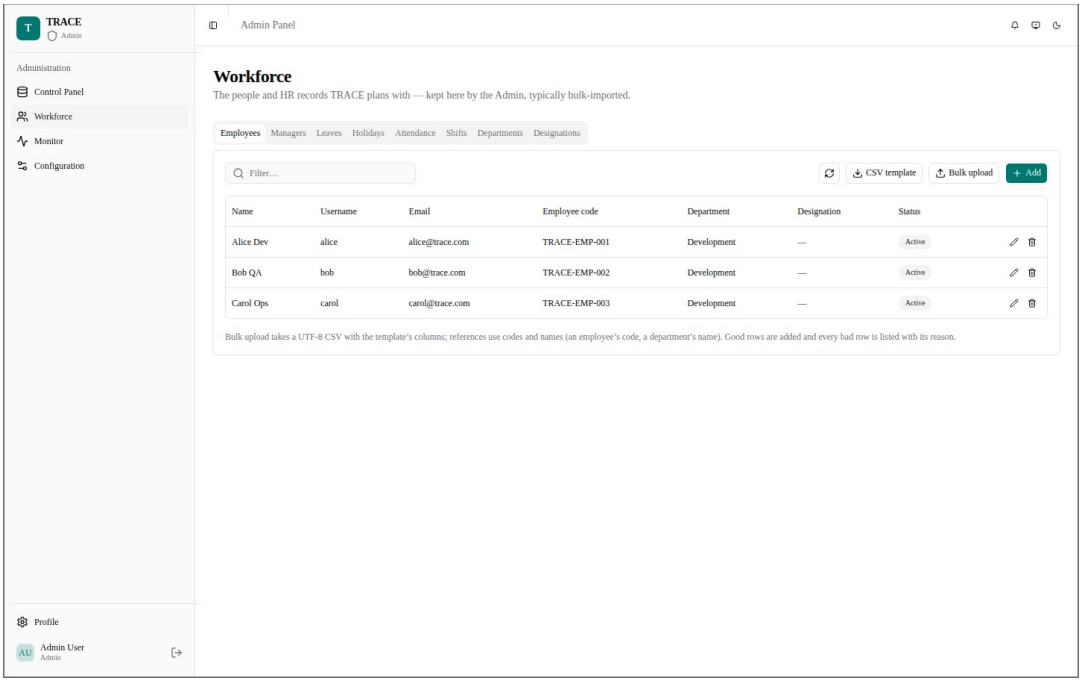


Figure 30: Admin — Workforce — Employees

Employee records with code and department; added one by one or bulk-uploaded from a CSV.

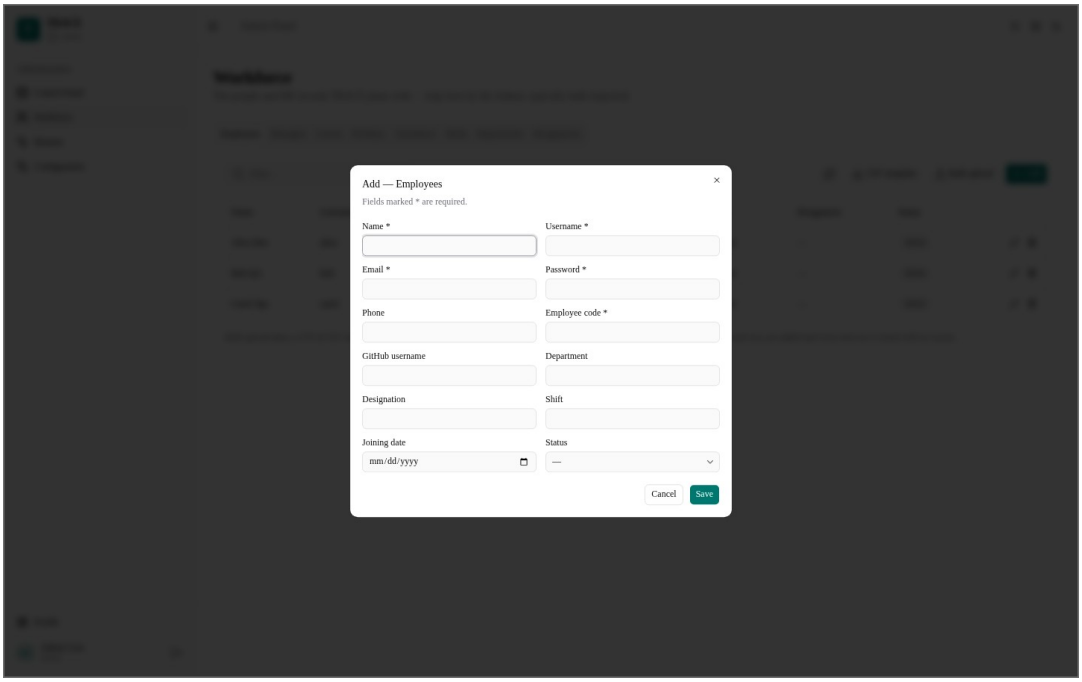


Figure 31: Admin — Workforce — Add Employee

The employee form: account details, employee code, department, designation, shift and joining date.

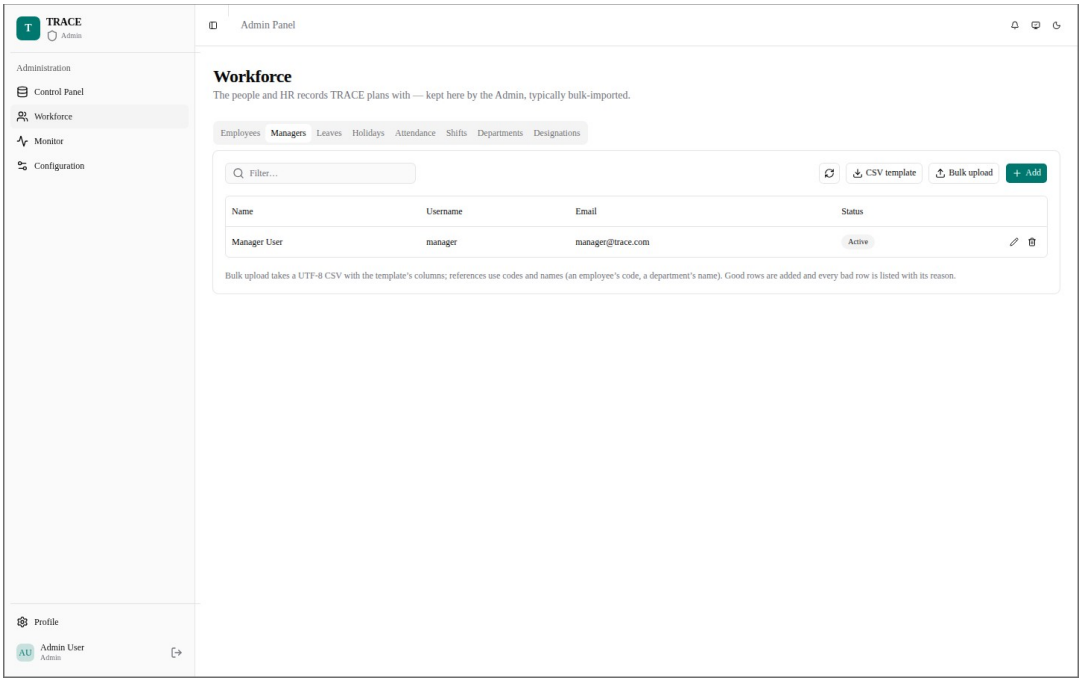


Figure 32: Admin — Workforce — Managers

The Manager accounts of the organisation.

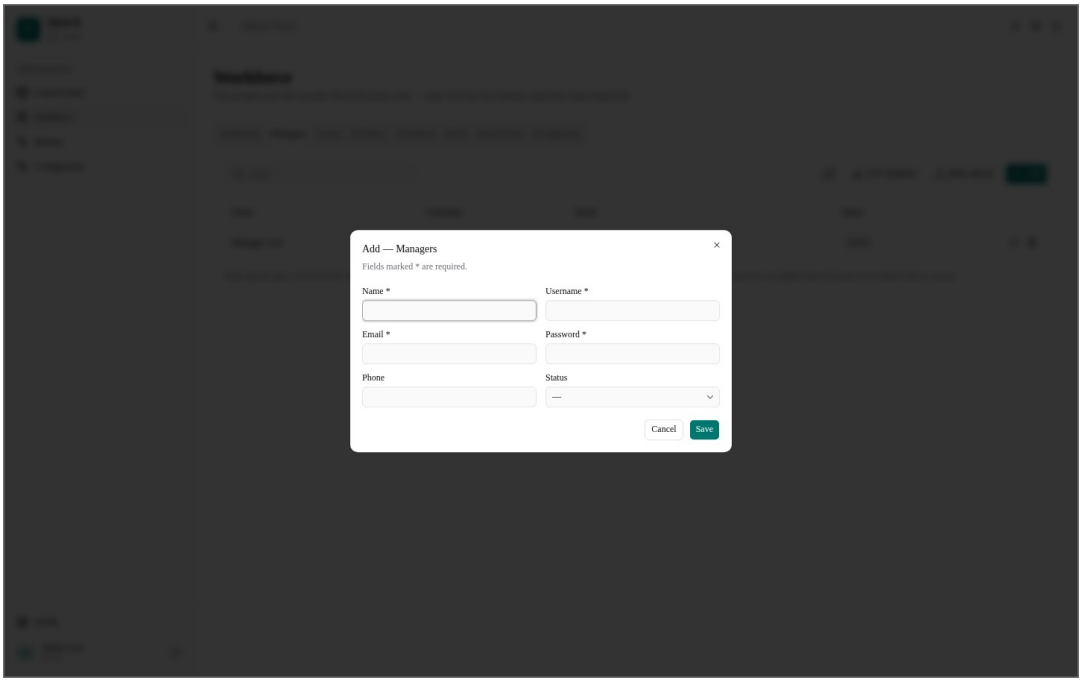


Figure 33: Admin — Workforce — Add Manager

A new Manager: name, username, e-mail, password, phone and status.

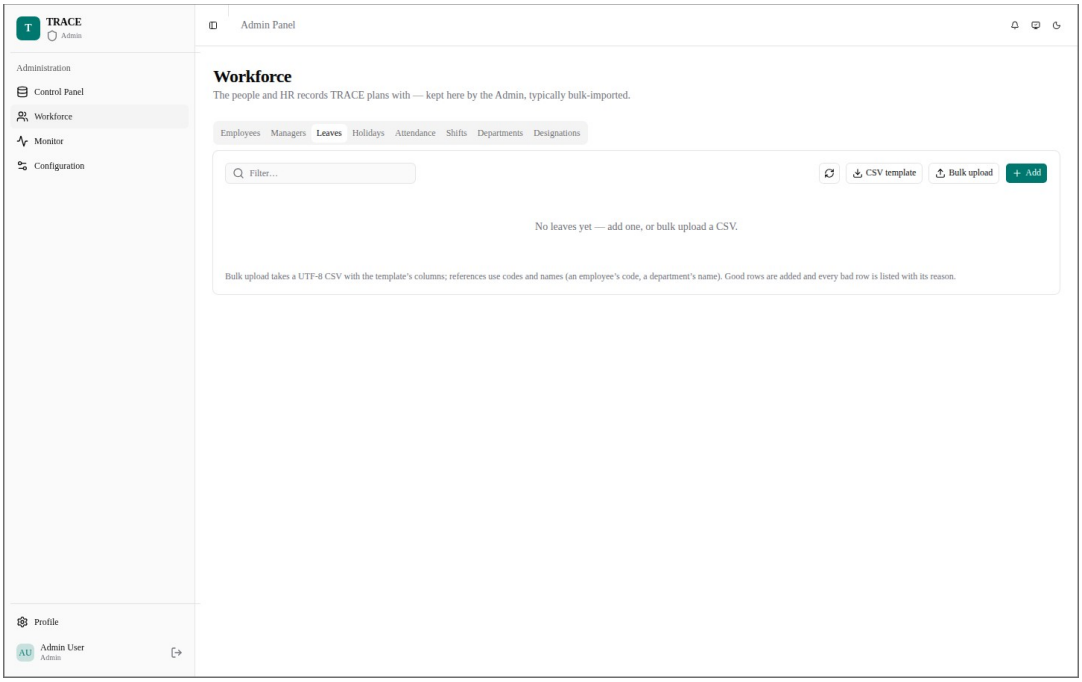


Figure 34: Admin — Workforce — Leaves

Leave records of the employees, added here or bulk-uploaded.

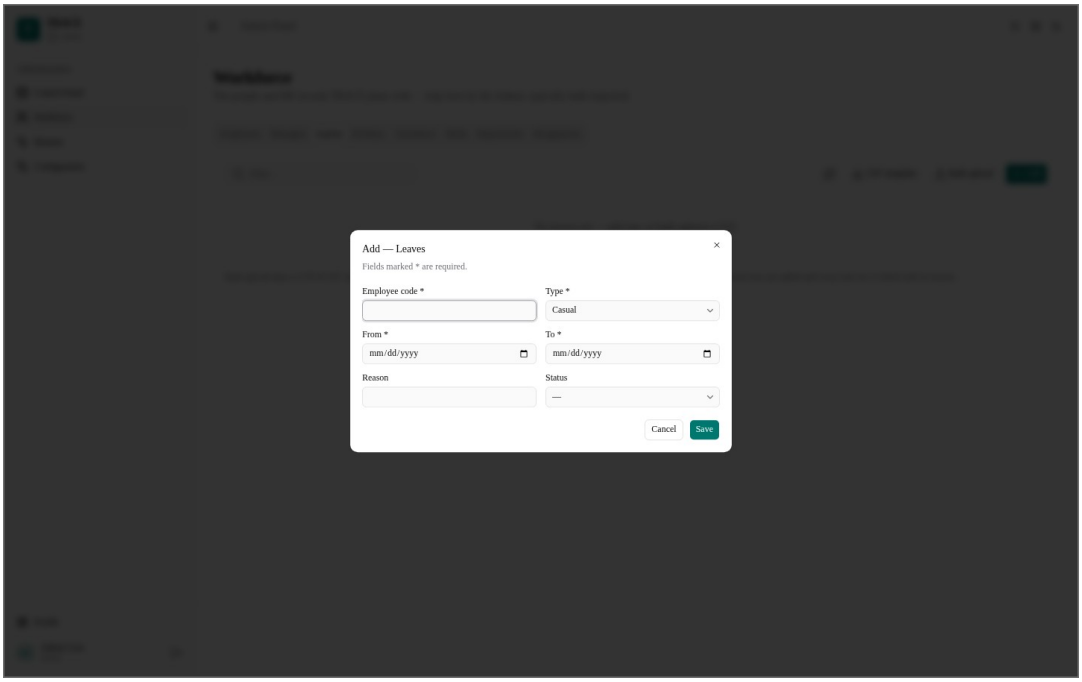


Figure 35: Admin — Workforce — Add Leave

A leave by employee code, with its type, dates, reason and status.

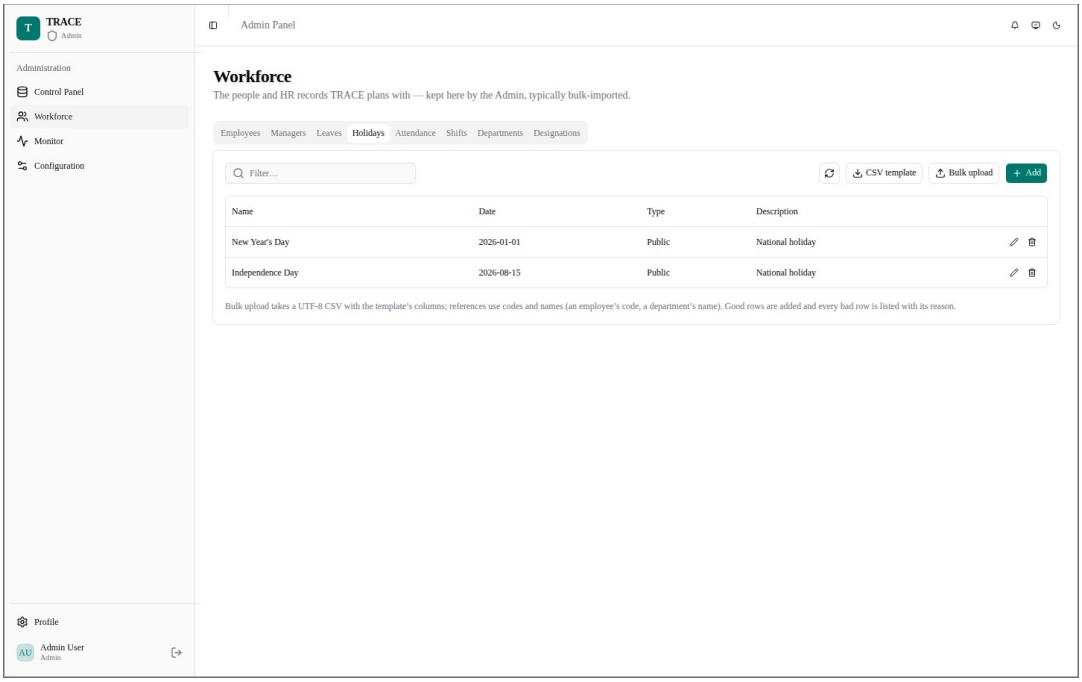


Figure 36: Admin — Workforce — Holidays

The organisation's holidays, with date, type and description.

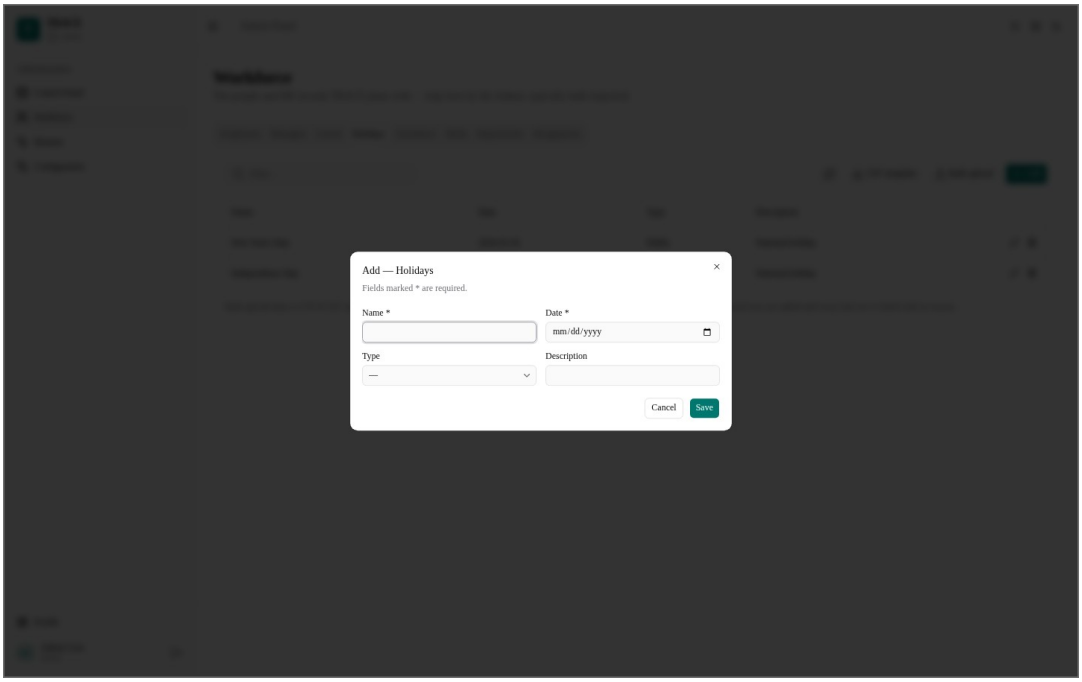


Figure 37: Admin — Workforce — Add Holiday

A new holiday: name, date, type and description.

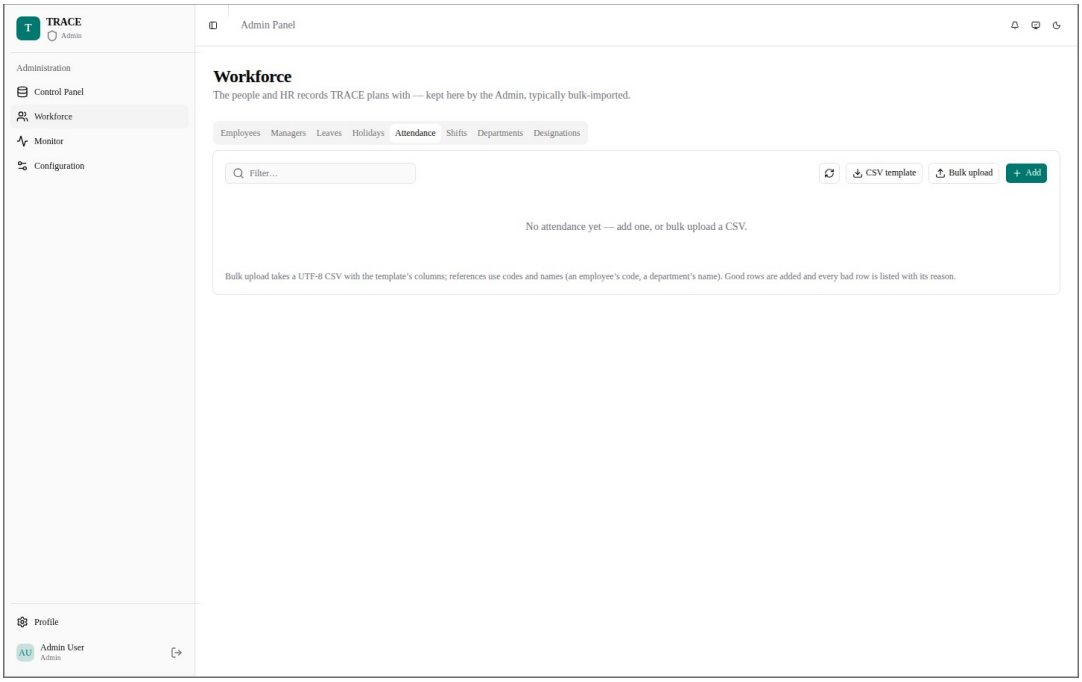


Figure 38: Admin — Workforce — Attendance

Attendance records maintained by the Admin.

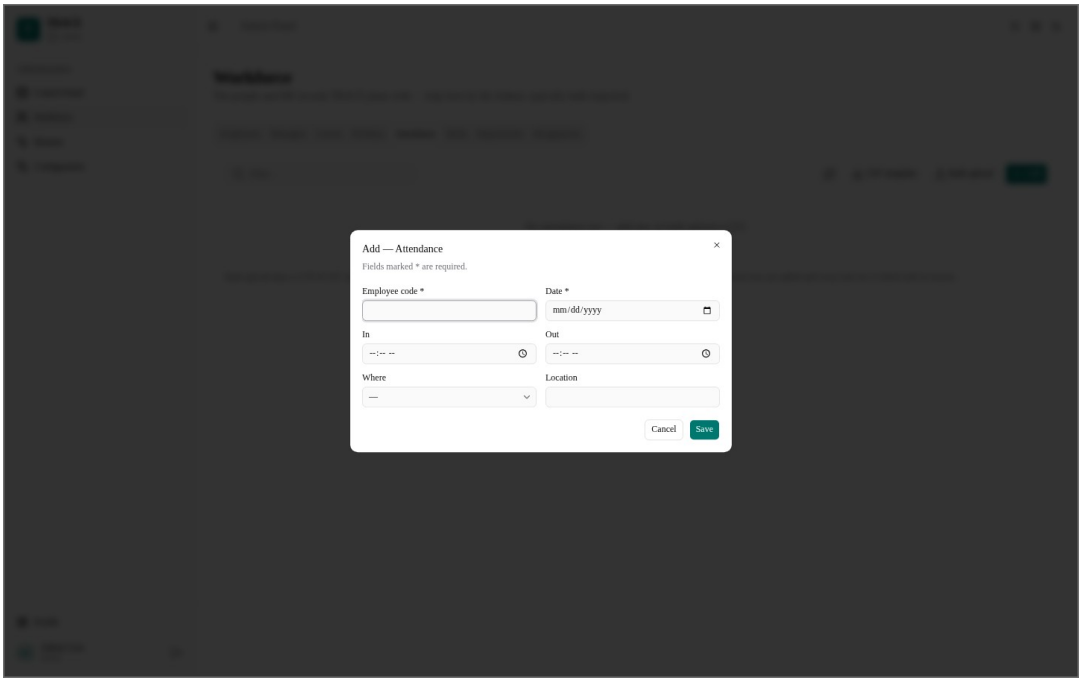


Figure 39: Admin — Workforce — Add Attendance

An attendance record: employee code, date, in and out times, and where it was worked.

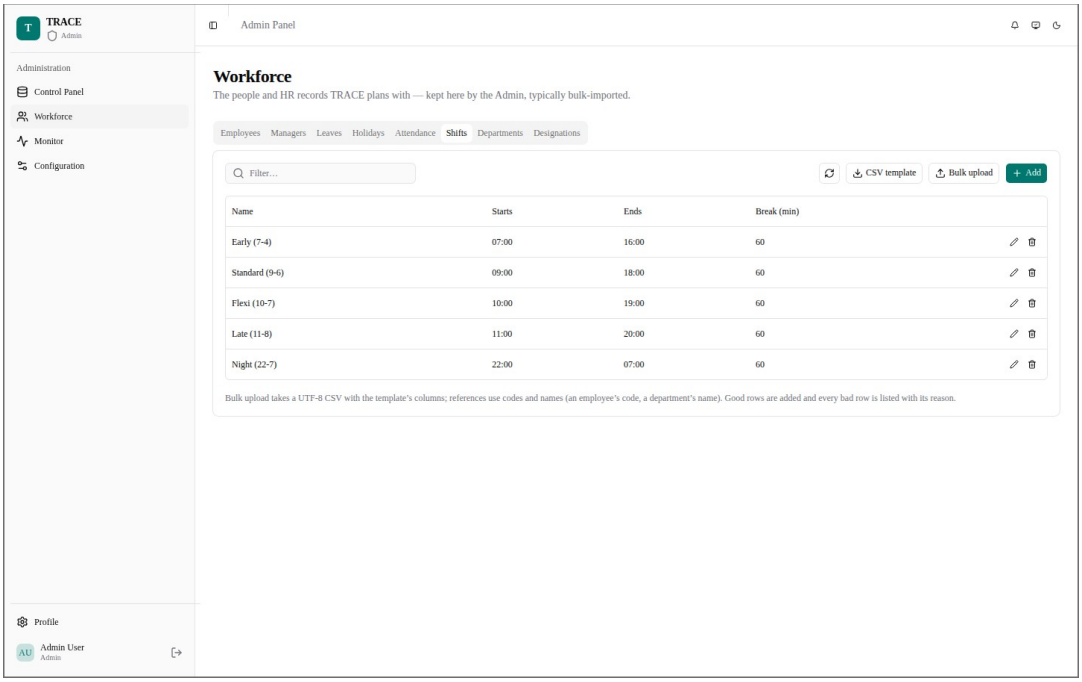


Figure 40: Admin — Workforce — Shifts

The work shifts with their start and end times and break length.

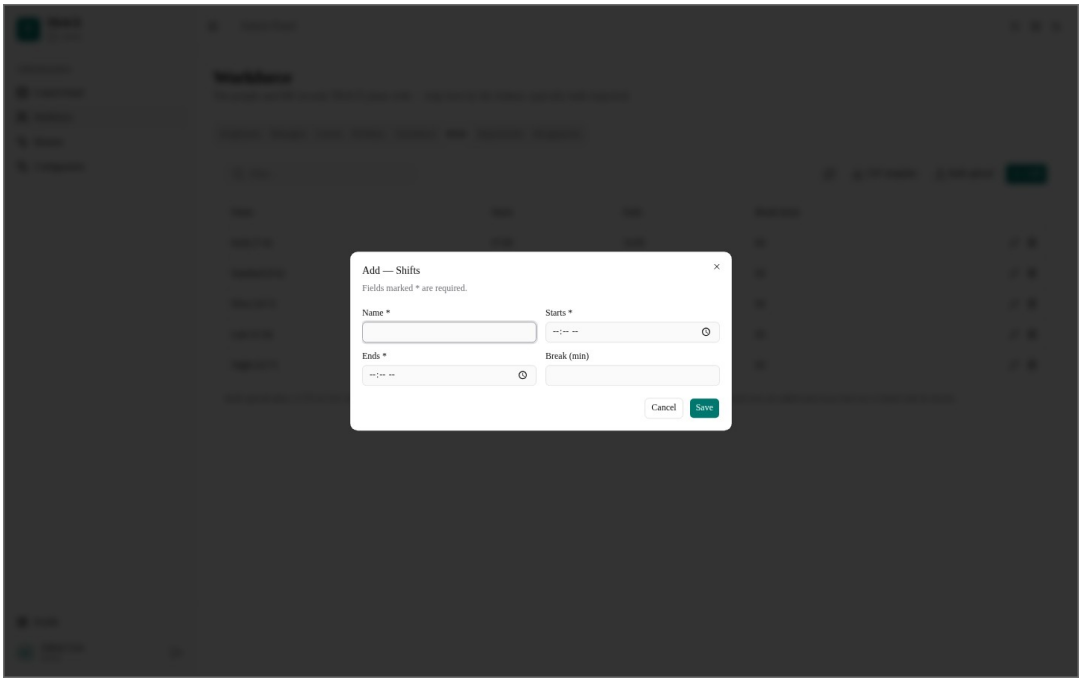


Figure 41: Admin — Workforce — Add Shift

A new shift: name, start, end and break in minutes.

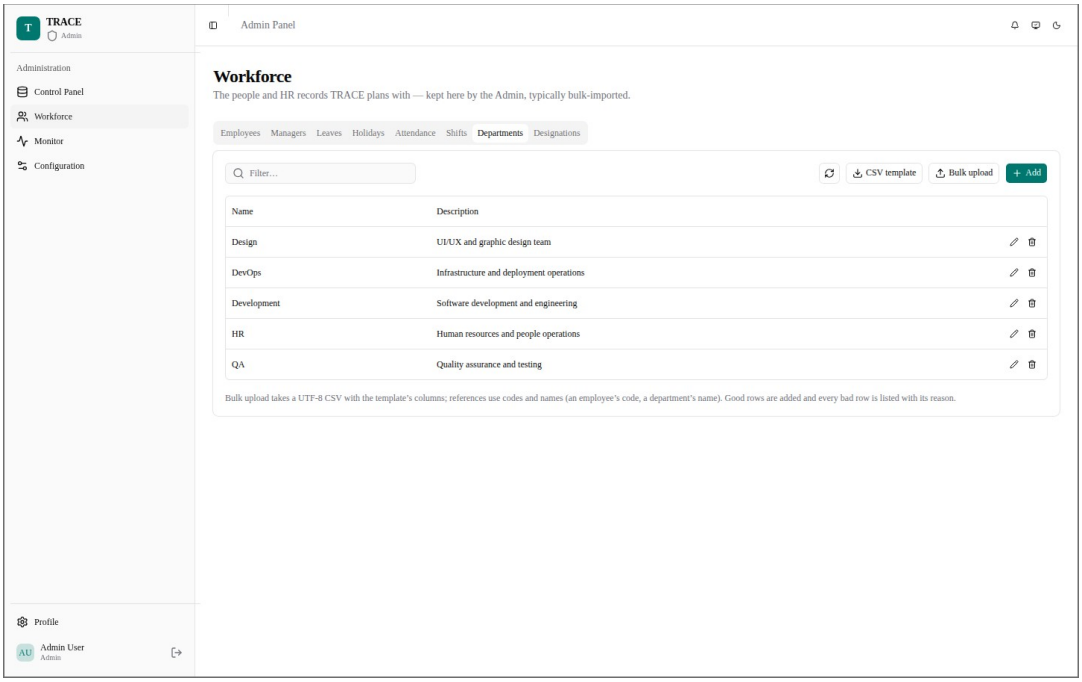


Figure 42: Admin — Workforce — Departments

The departments of the organisation.

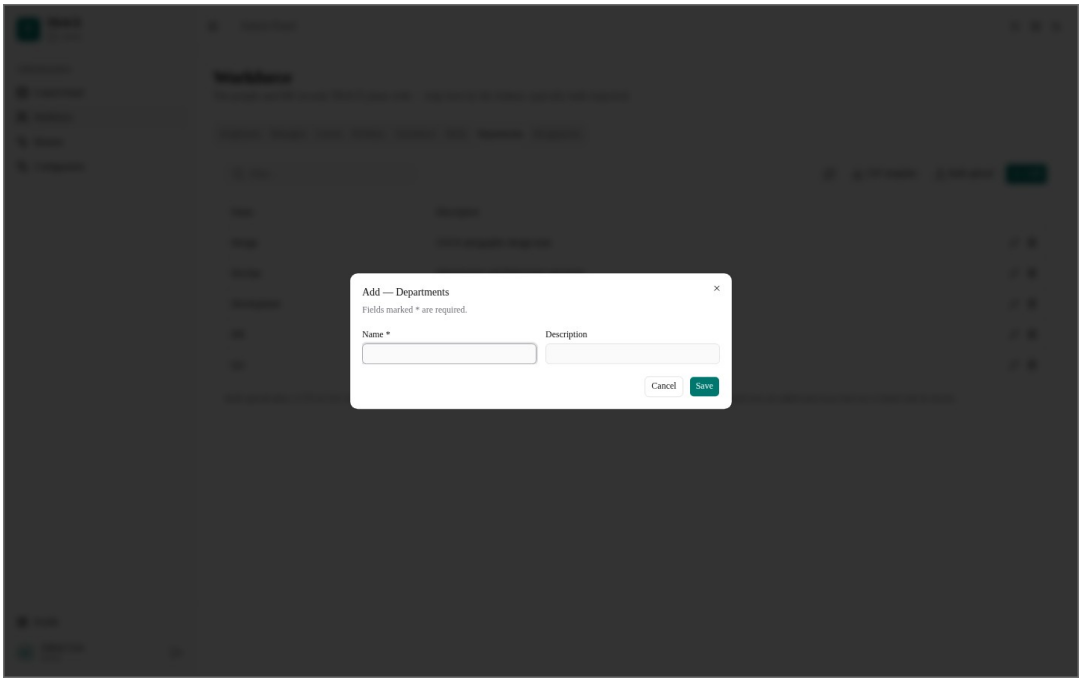


Figure 43: Admin — Workforce — Add Department

A new department with its description.

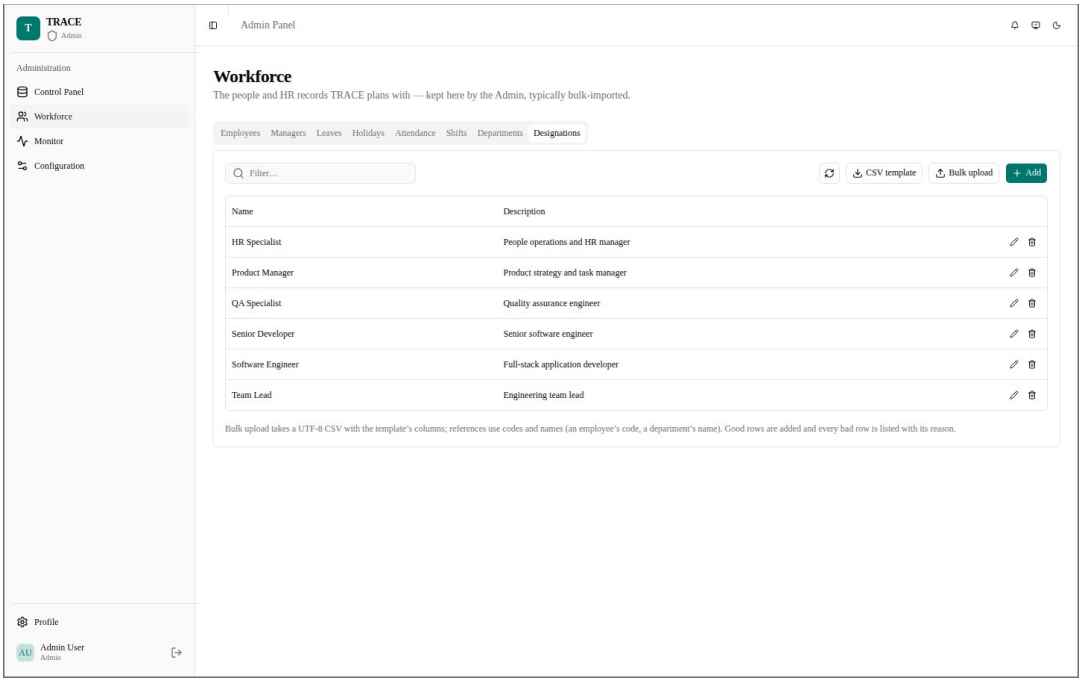


Figure 44: Admin — Workforce — Designations

The job titles employees can hold.

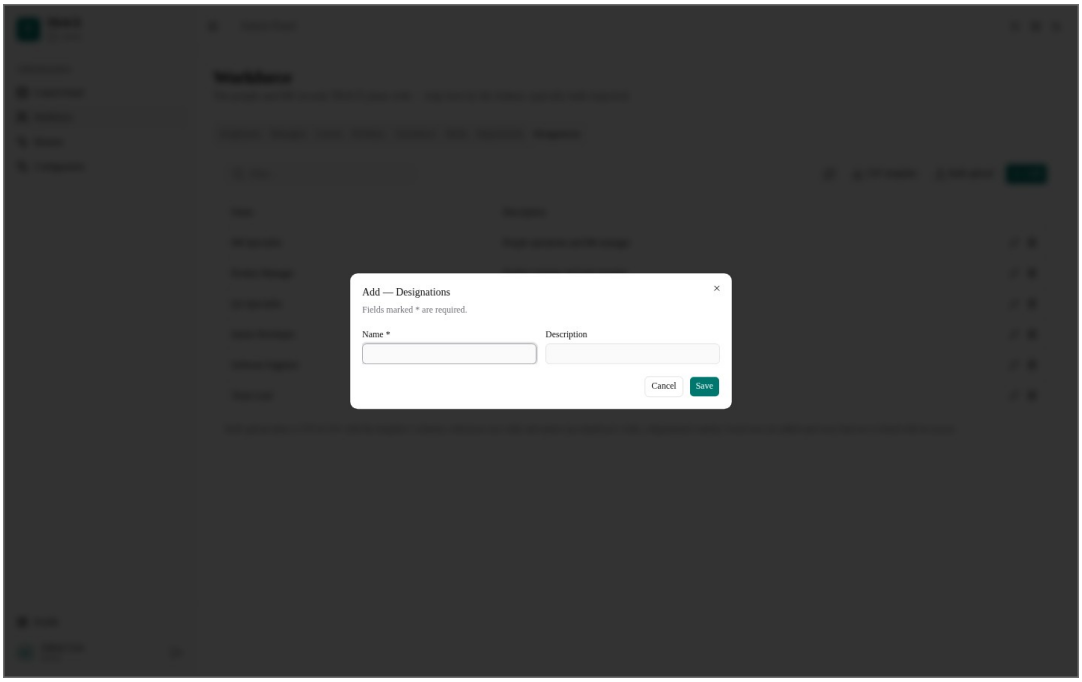


Figure 45: Admin — Workforce — Add Designation

A new designation with its description.

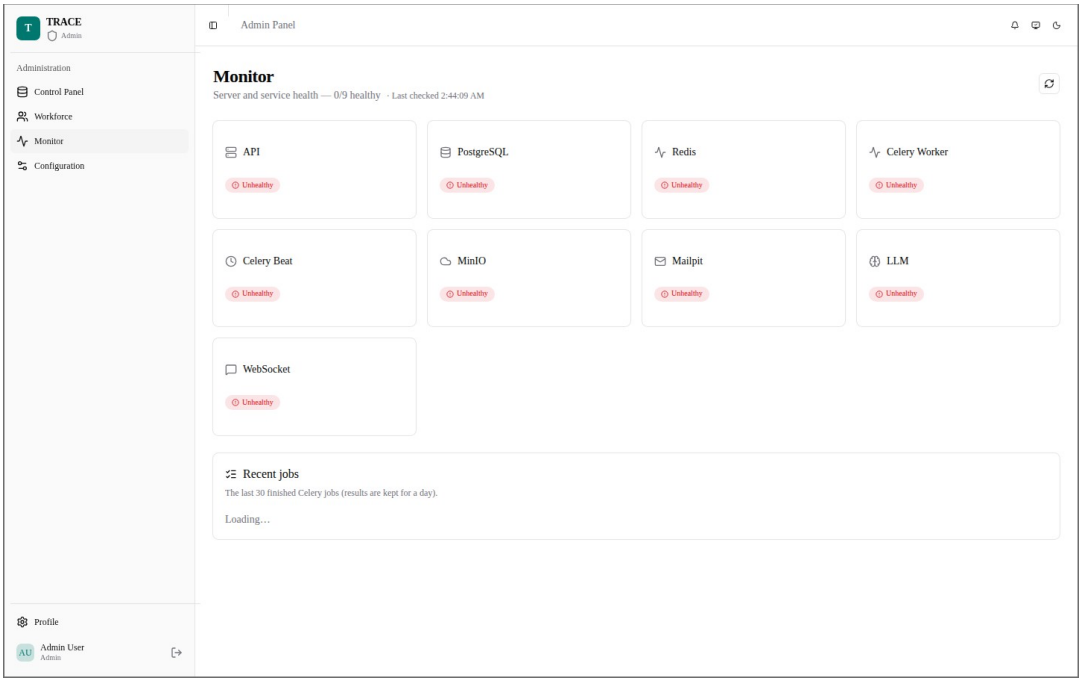


Figure 46: Admin — Monitor

The health of every service — API, PostgreSQL, Redis, Celery, MinIO, mail, LLM, WebSocket — and recent jobs.

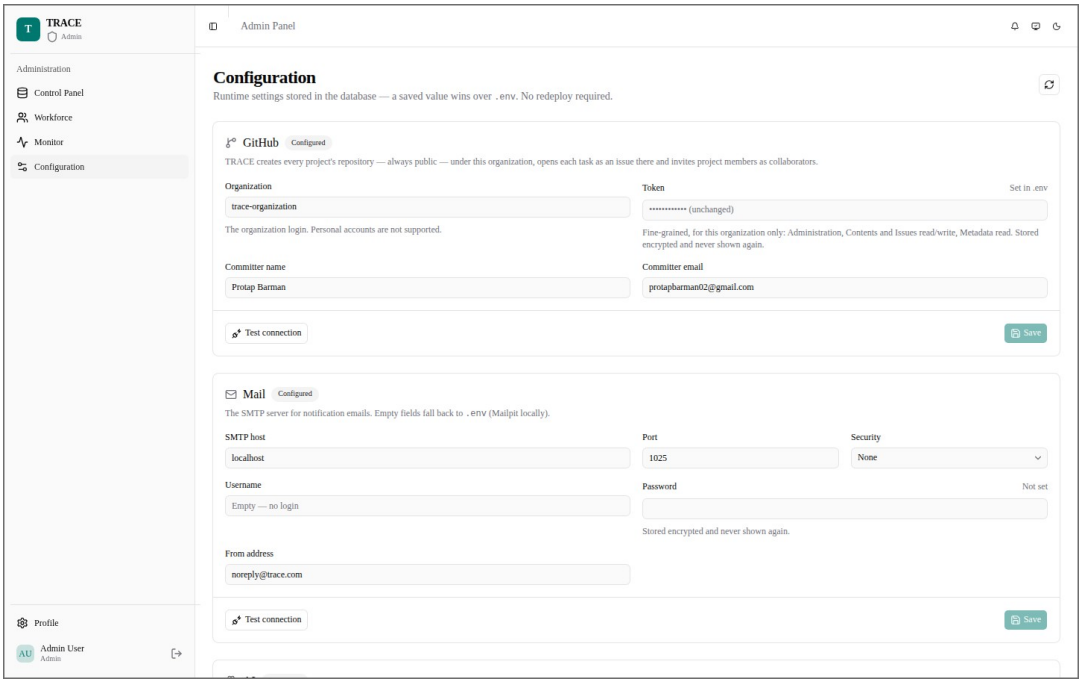


Figure 47: Admin — Configuration

GitHub and mail settings kept in the database; a saved value wins over .env, with a connection test.

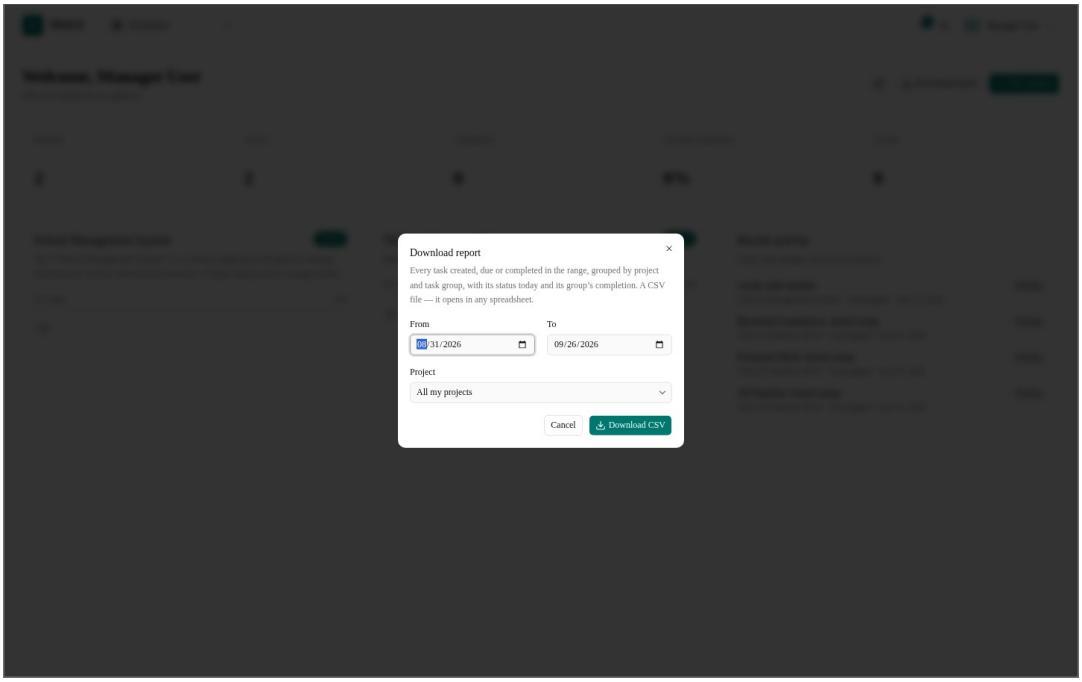


Figure 48: Manager — Download Report

A CSV of every task created, due or completed in a date range, grouped by project and task group.

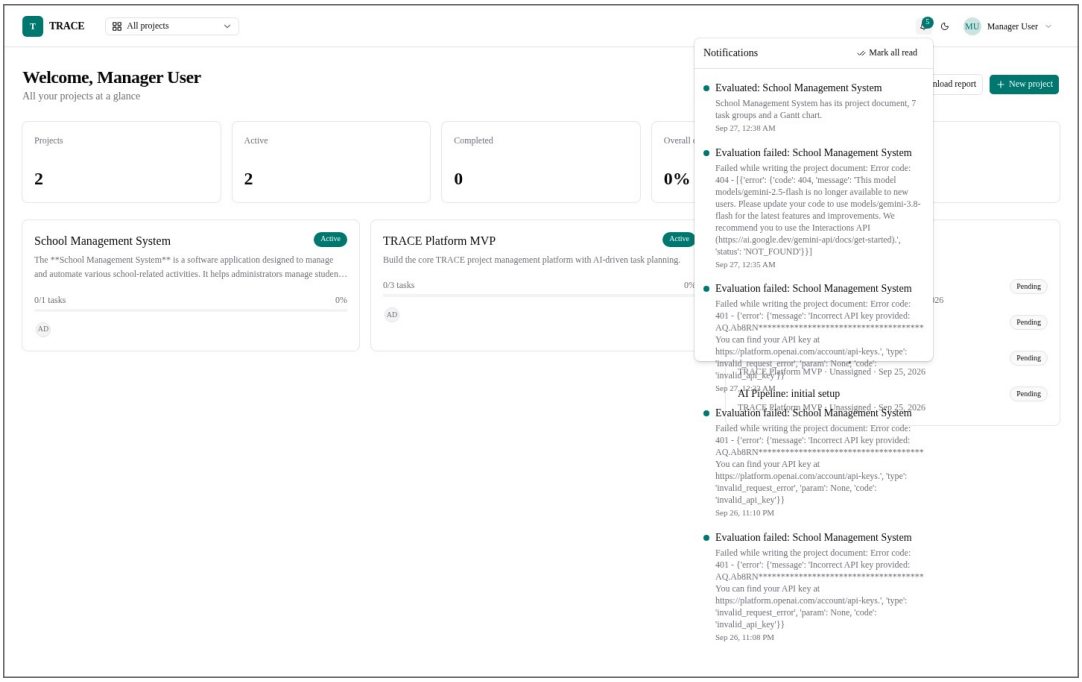


Figure 49: Manager — Notifications

Evaluation results and failures, each with its reason, in the notification list.

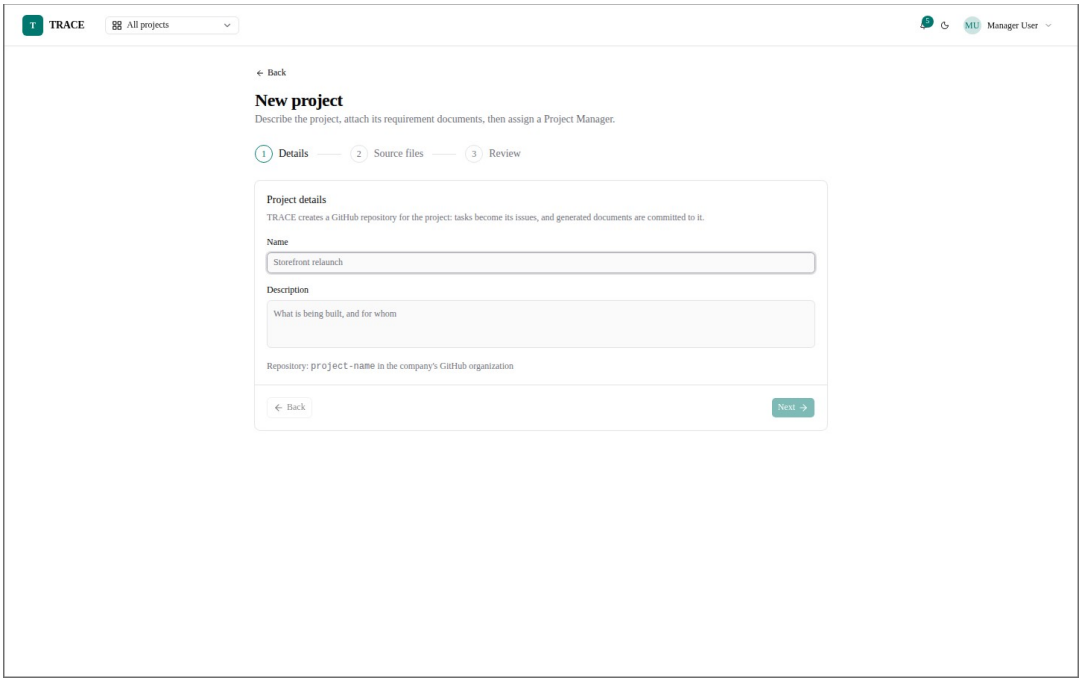


Figure 50: Manager — New Project

The three-step project wizard: details, source files, then review; a GitHub repository is created for it.

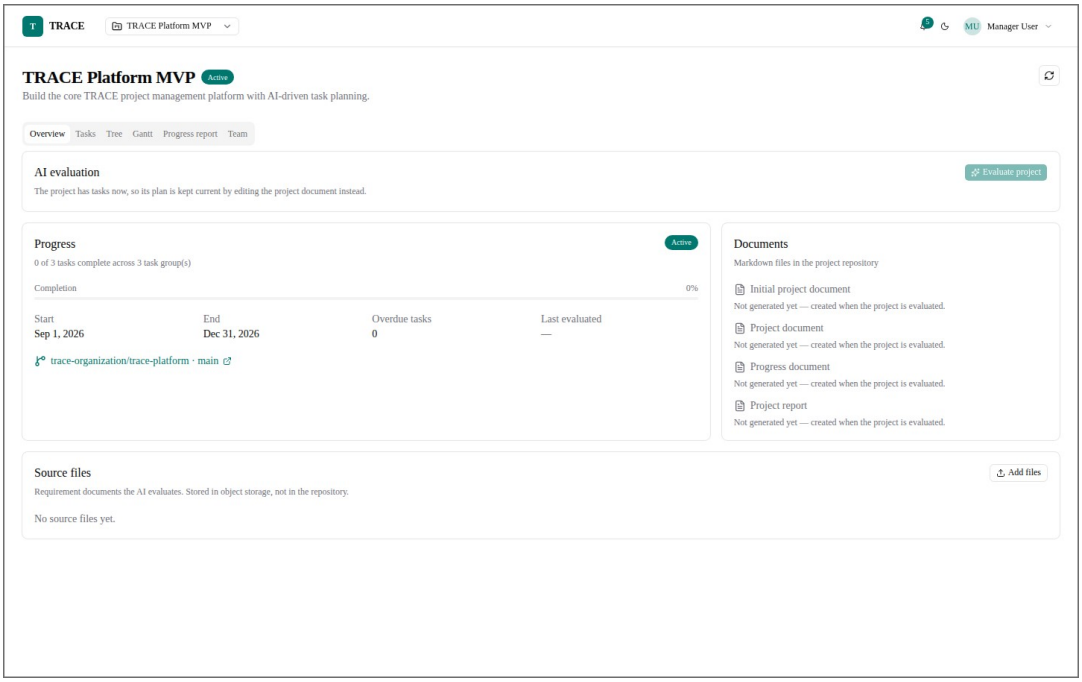


Figure 51: Manager — Project Overview (Not Yet Evaluated)

Progress, dates, repository and documents of a project that has not been evaluated by the AI.

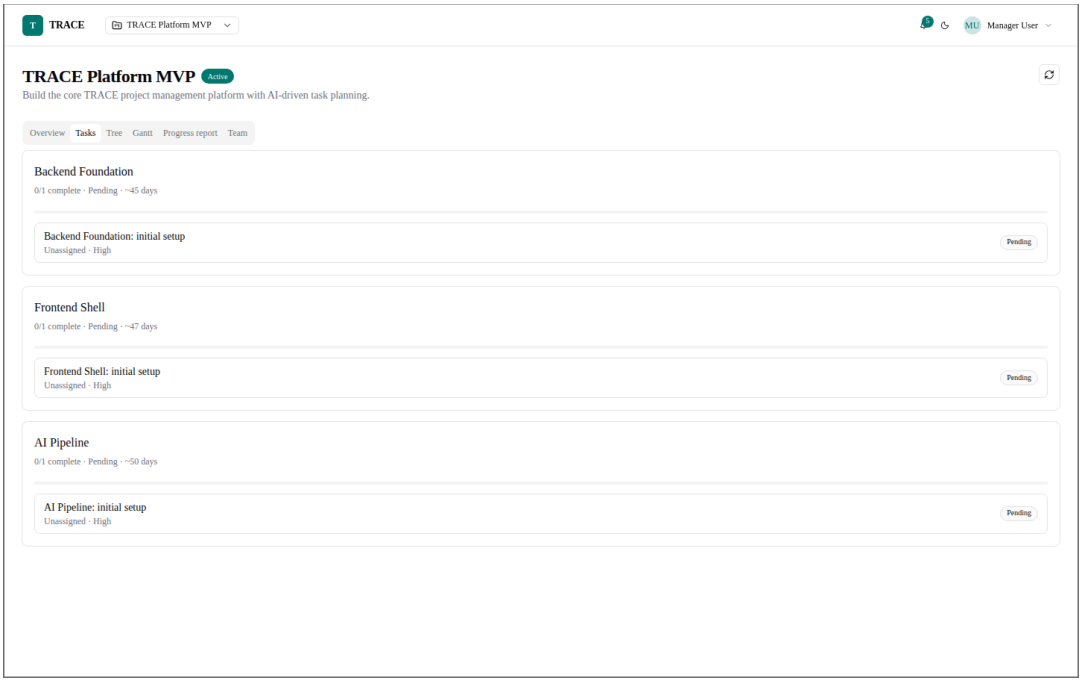


Figure 52: Manager — Project Tasks

The project's task groups and their tasks, with status, priority and estimated days.

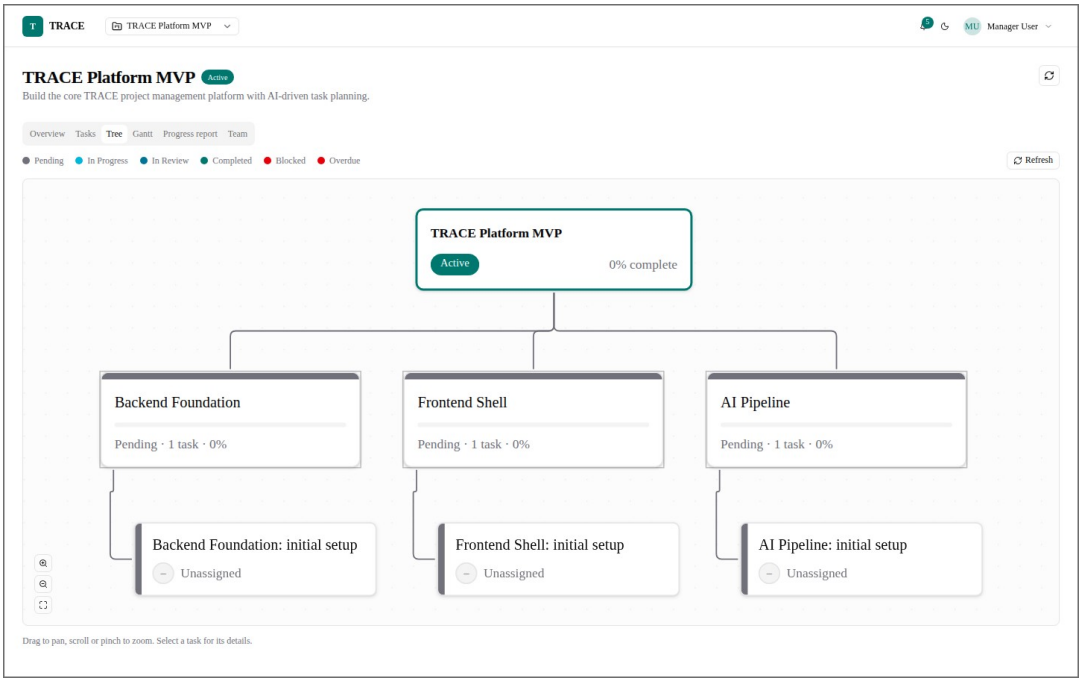


Figure 53: Manager — Project Tree

The project as a tree: project, task groups, then tasks, coloured by status.

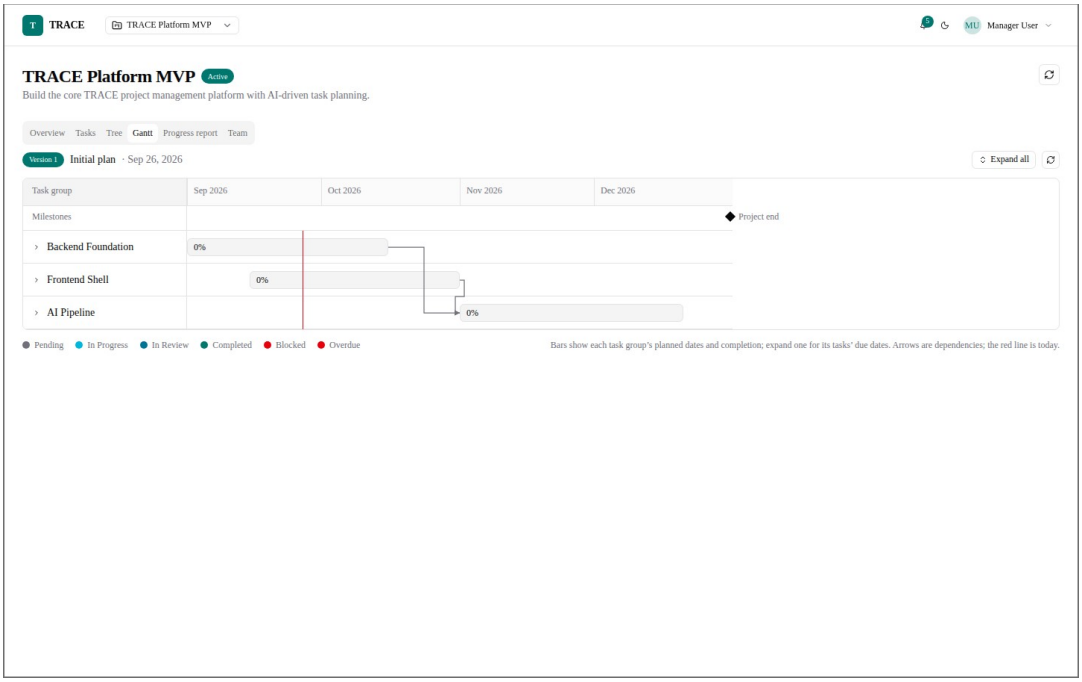


Figure 54: Manager — Gantt Chart

Planned dates and completion of each task group, with dependencies and today's line.

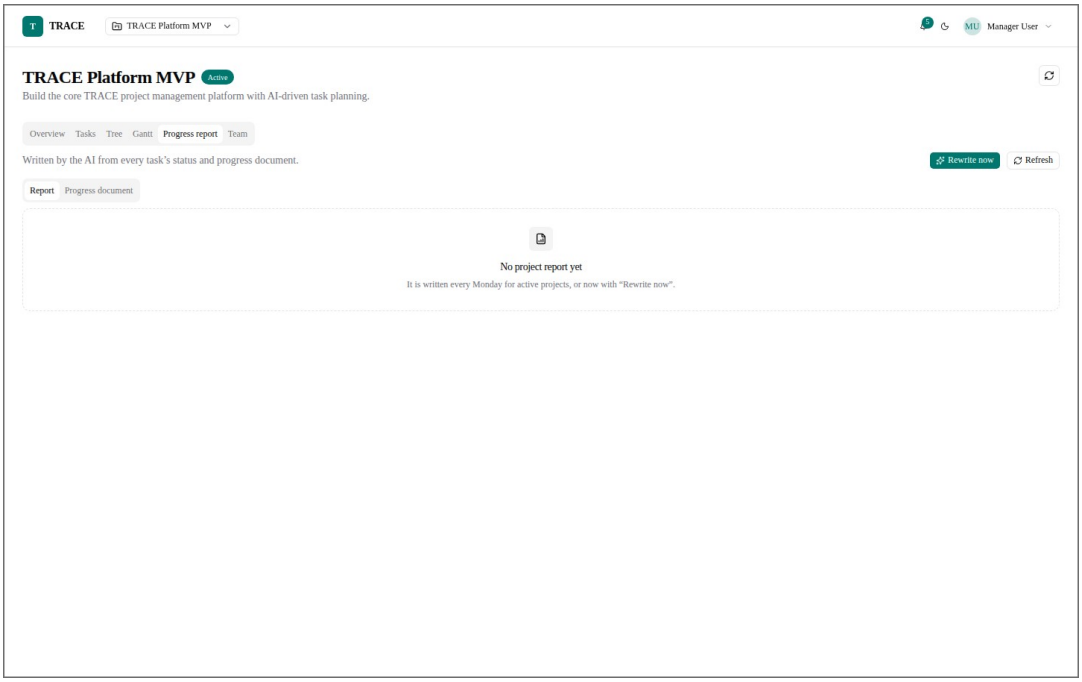


Figure 55: Manager — Progress Report

The weekly project report the AI writes from every task's status, or on demand with Rewrite now.

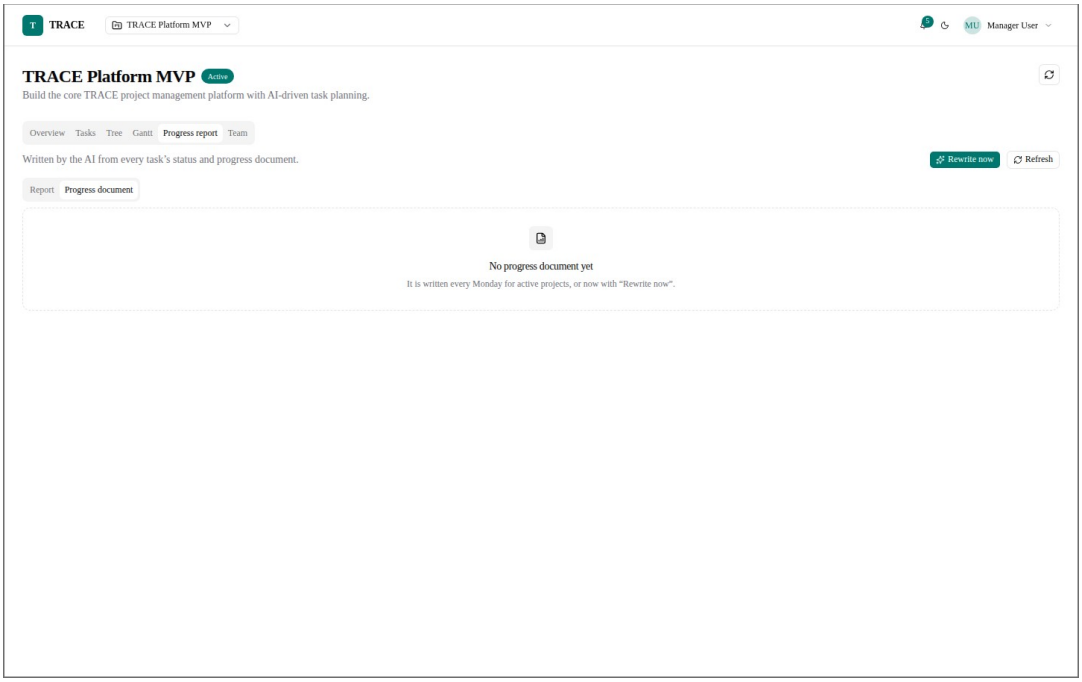


Figure 56: Manager — Progress Document

The project's progress document, kept alongside the report.

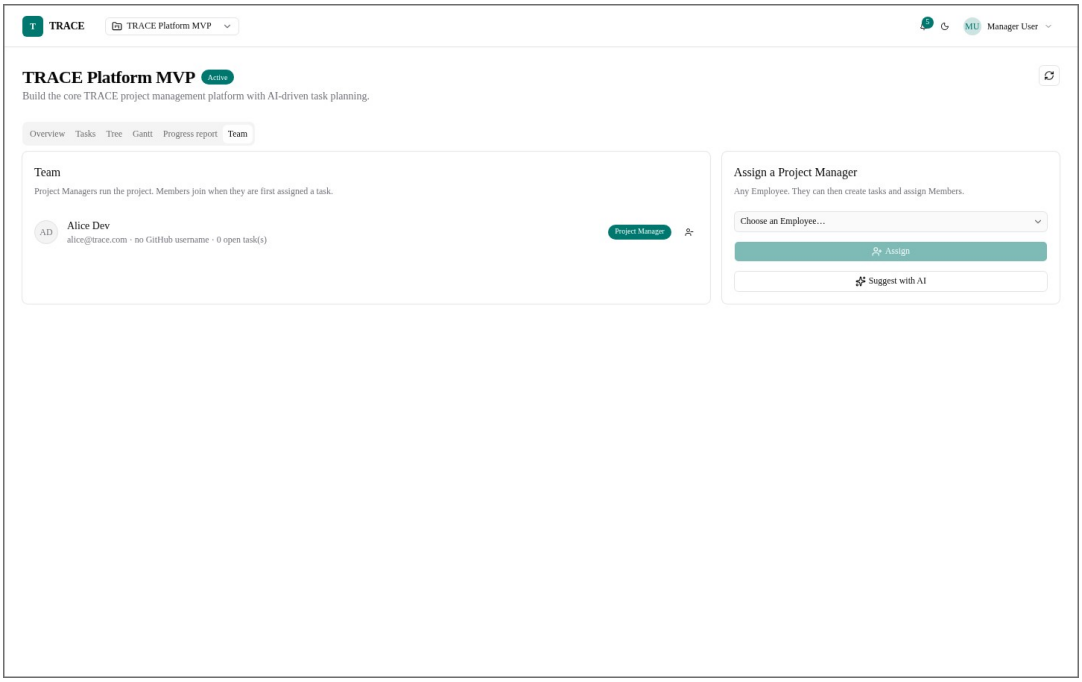


Figure 57: Manager — Team

The project's people, and the assignment of a Project Manager with AI suggestions.

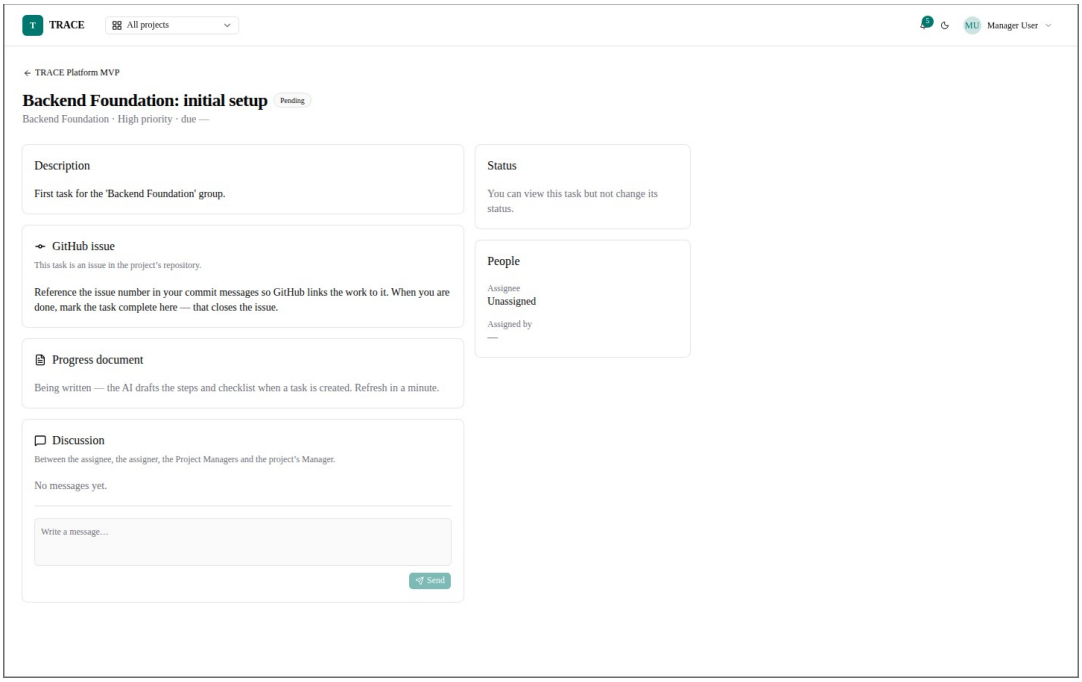


Figure 58: Manager — Task Detail (Manager)

A task as the Manager sees it: read-only status, its GitHub issue, progress document and discussion.

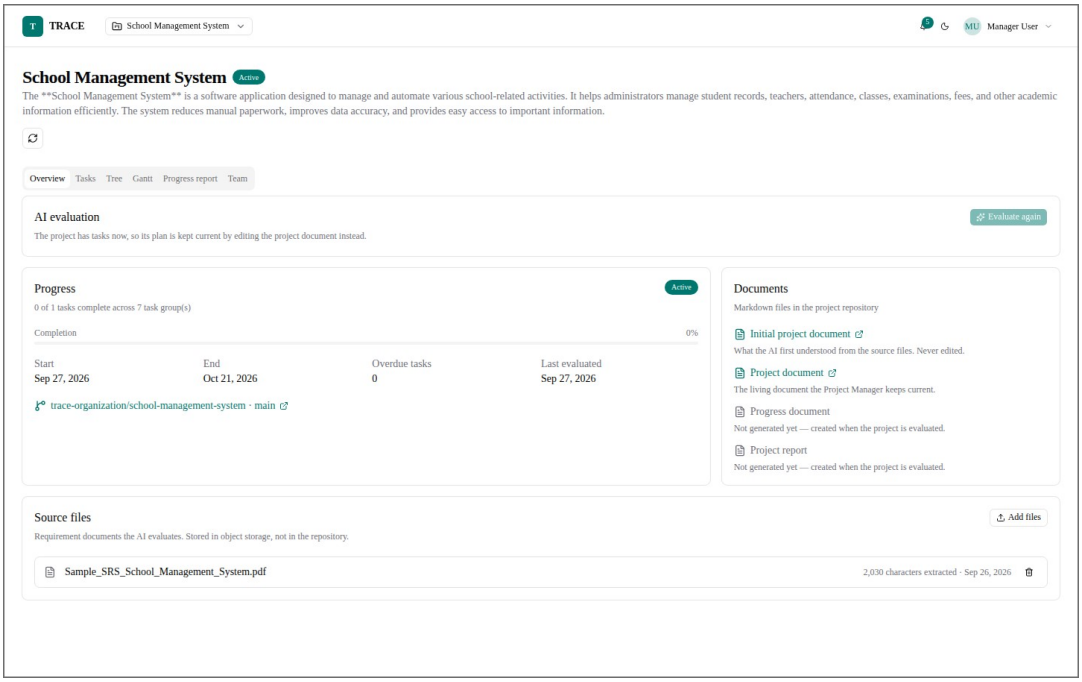


Figure 59: Manager — Project Overview (Evaluated)

An evaluated project: seven task groups, the generated documents and the uploaded source file.

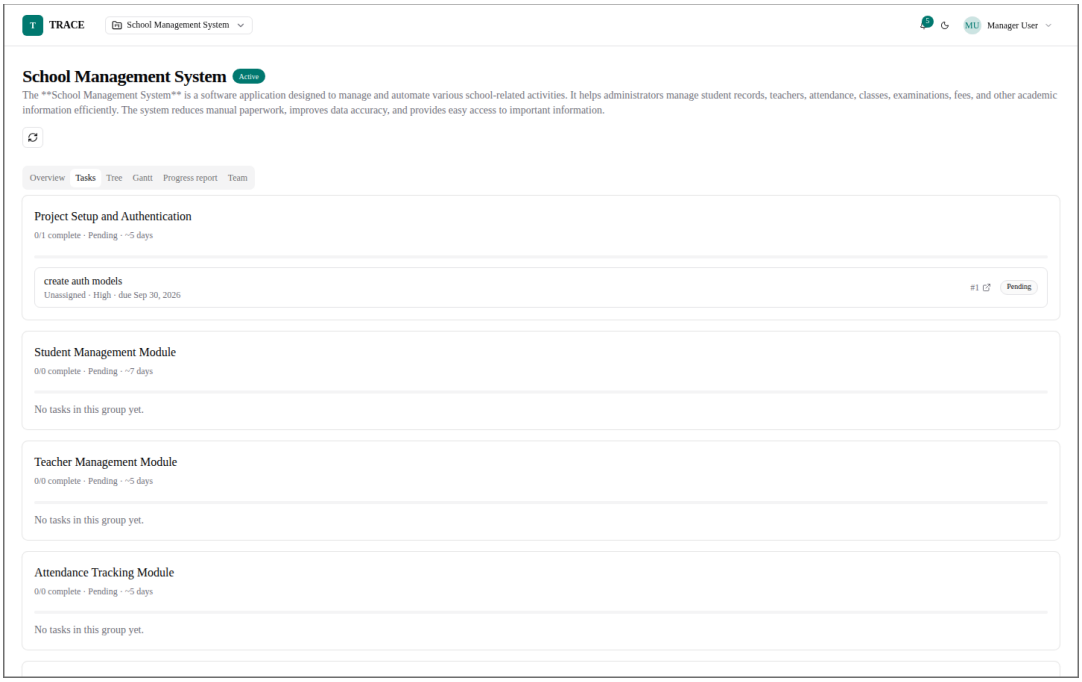


Figure 60: Manager — Project Tasks (Evaluated)

The task groups the AI created from the requirement documents.

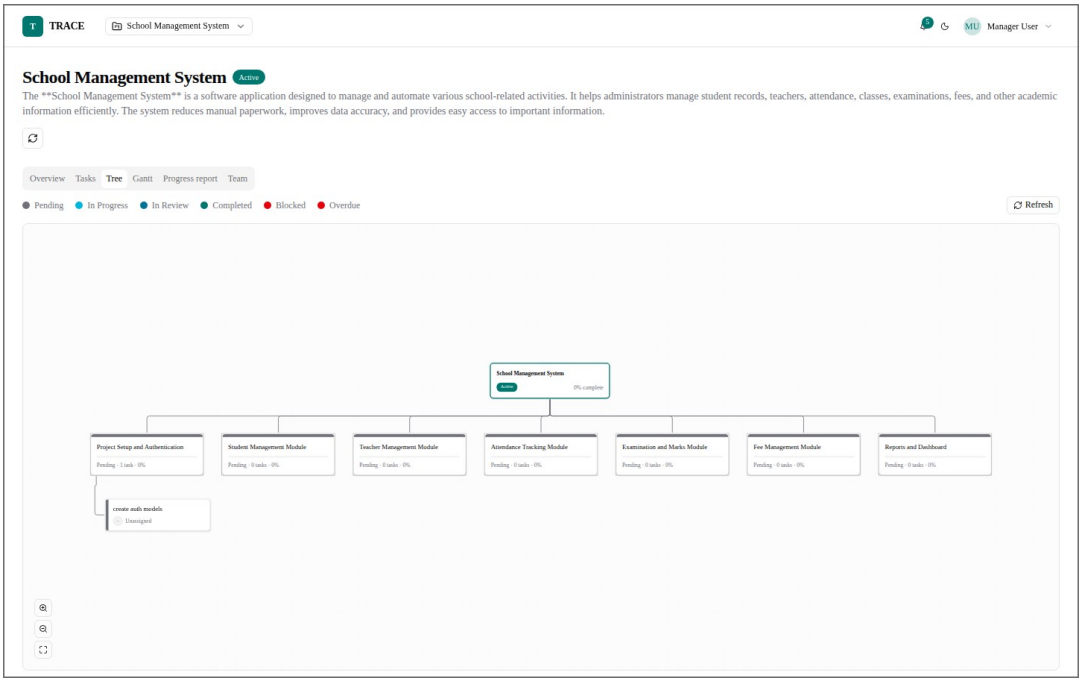


Figure 61: Manager — Project Tree (Evaluated)

The tree of the evaluated project with all its task groups.

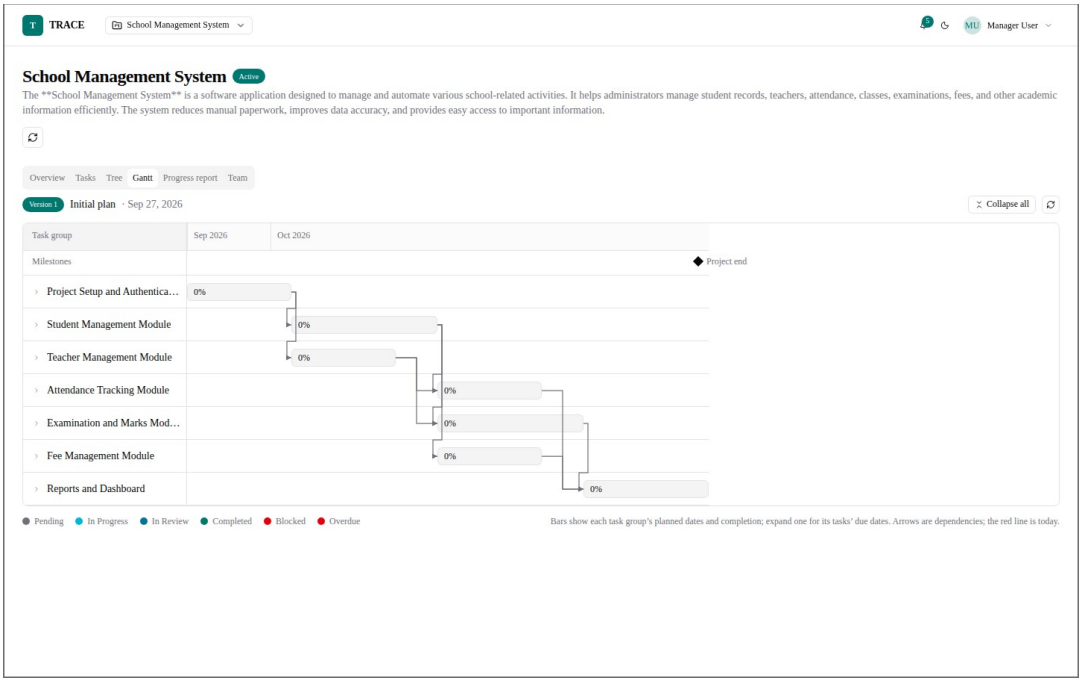


Figure 62: Manager — Gantt Chart (Evaluated)

The AI's first plan: task groups laid out in order of their dependencies.

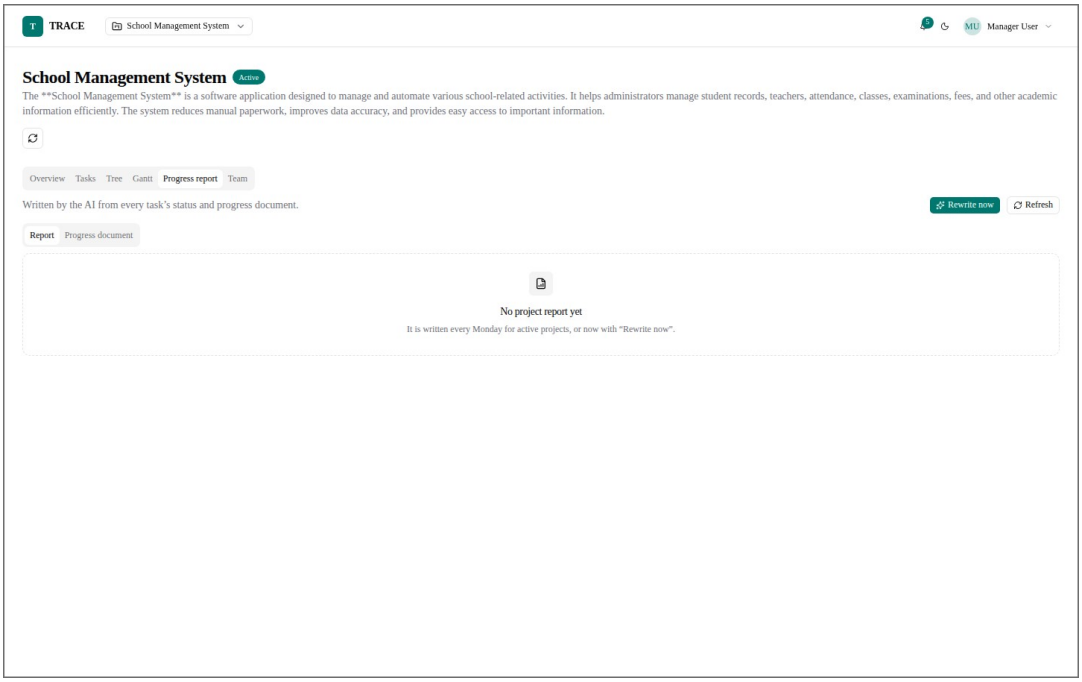


Figure 63: Manager — Progress Report (Evaluated)

The report tab of the evaluated project before its first weekly report.

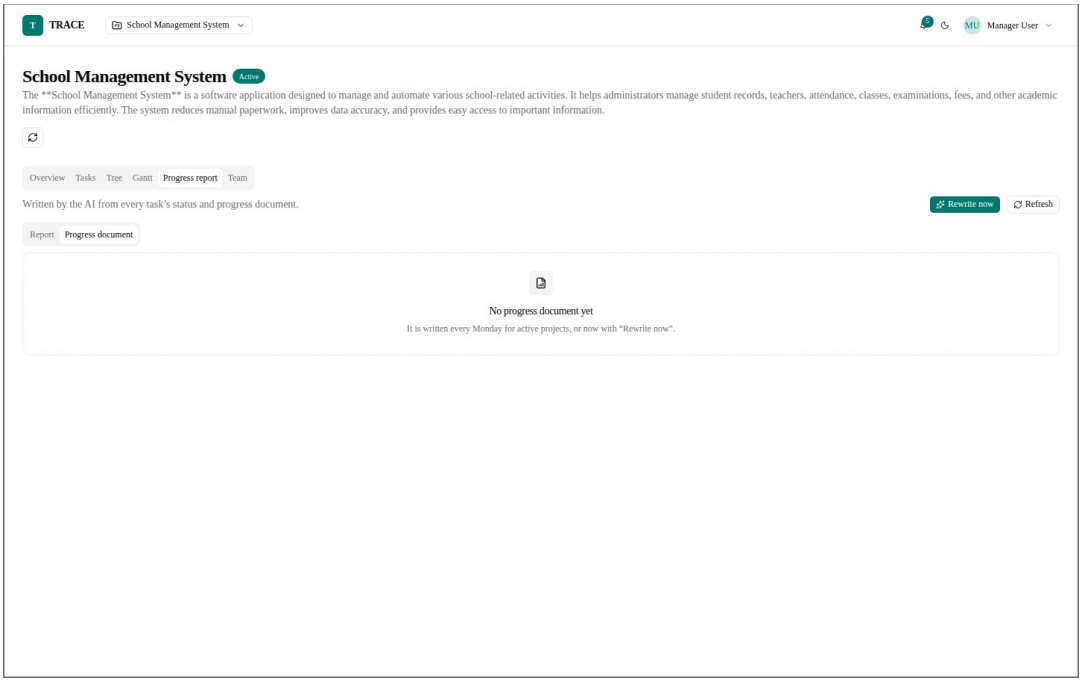


Figure 64: Manager — Progress Document (Evaluated)

The progress document tab of the evaluated project.

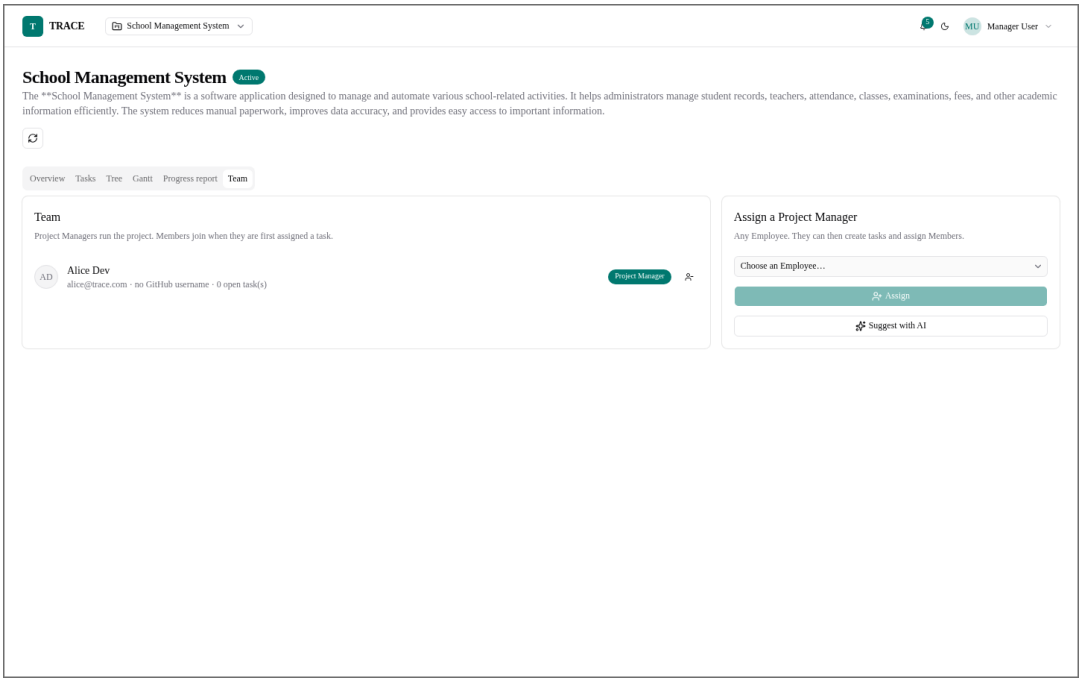


Figure 65: Manager — Team (Evaluated)

The evaluated project's Project Manager, with the option to assign another.

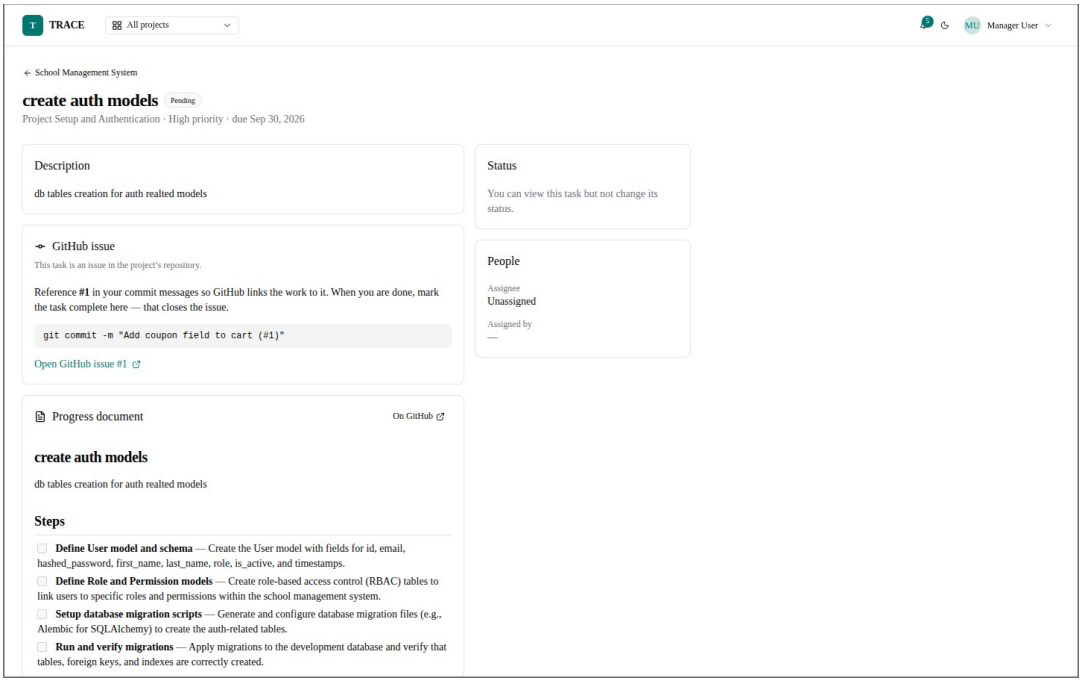


Figure 66: Manager — Task Detail with Plan

A task whose GitHub issue exists and whose progress document holds the AI's steps.

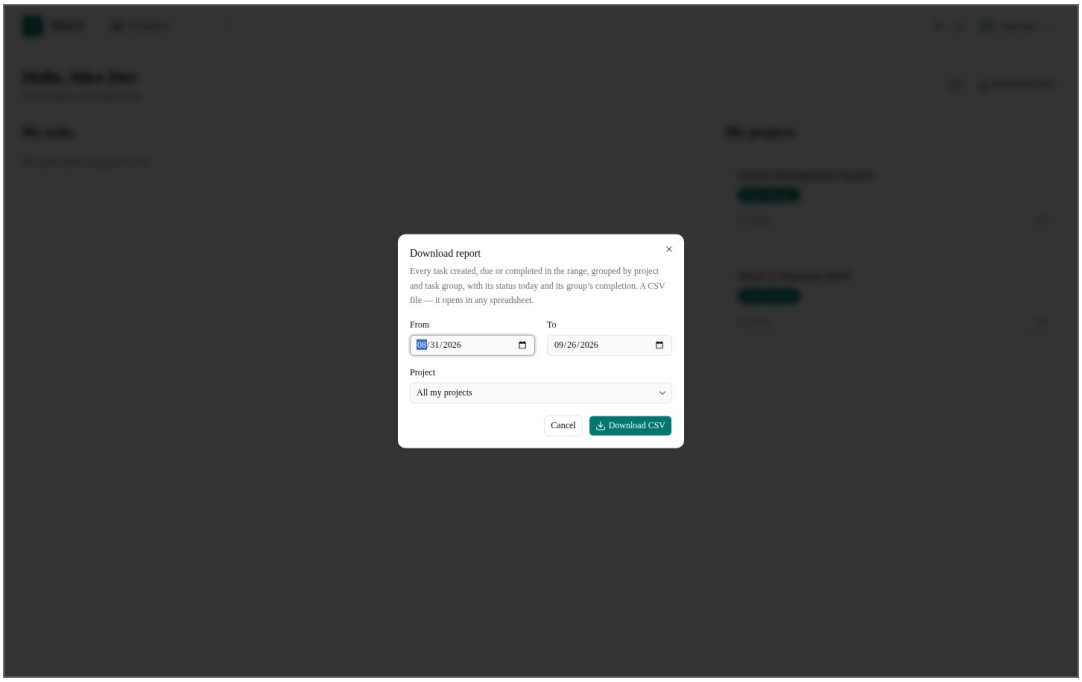


Figure 67: Employee — Download Report

The same CSV report, limited to the employee's own projects.

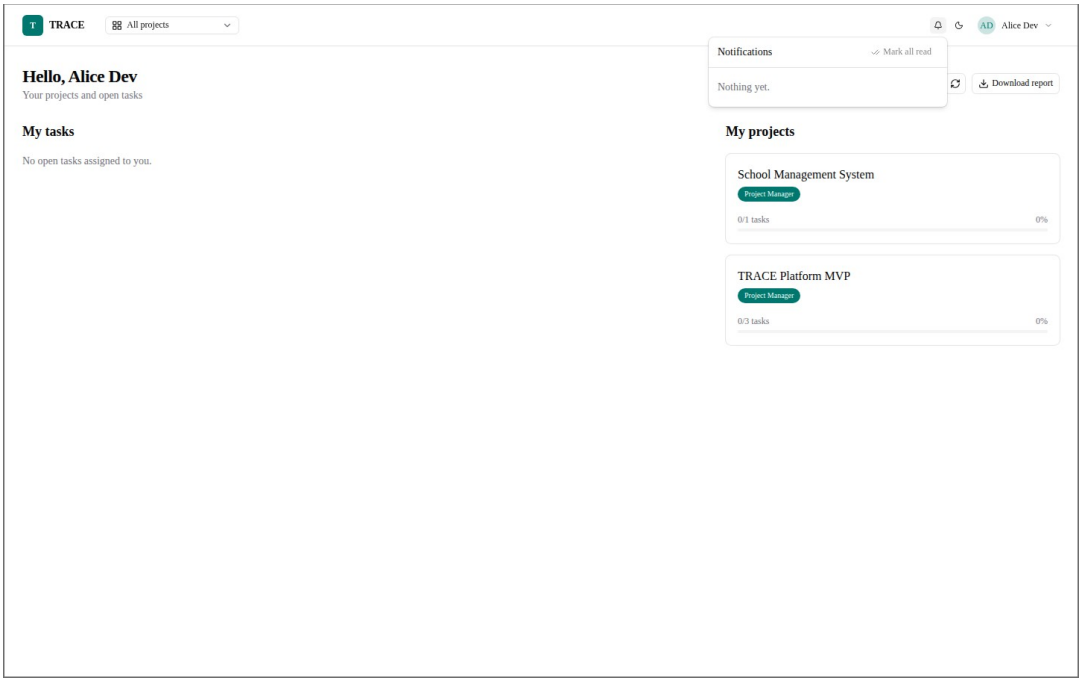


Figure 68: Employee — Notifications

The employee's notification list.

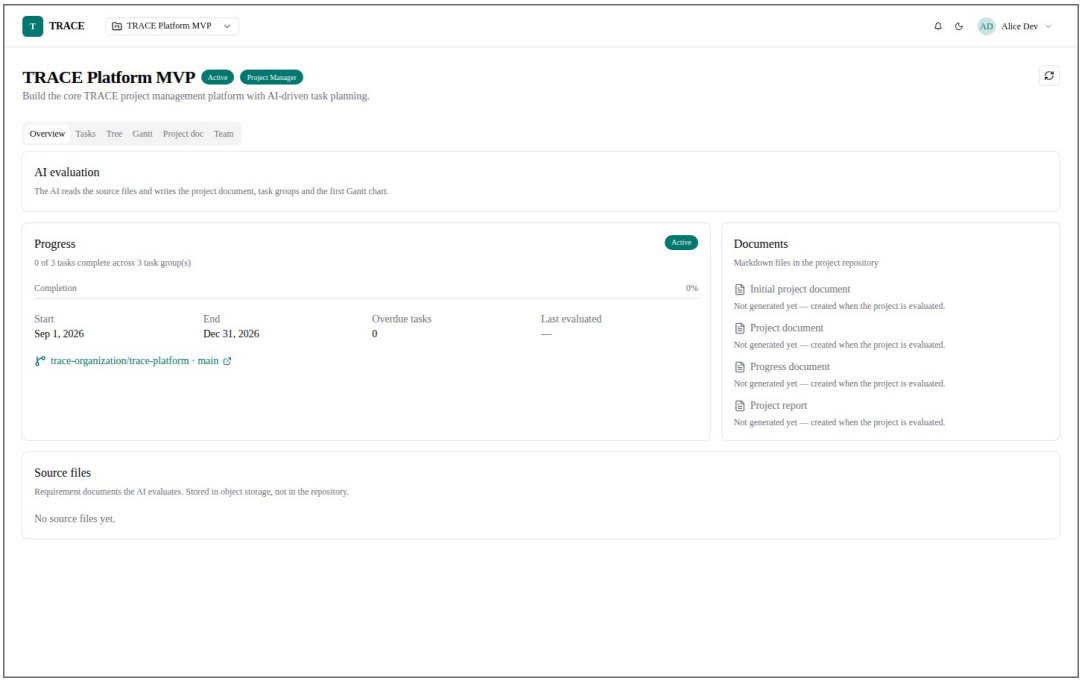


Figure 69: Employee — Project Overview

A project the employee runs as Project Manager, before AI evaluation.

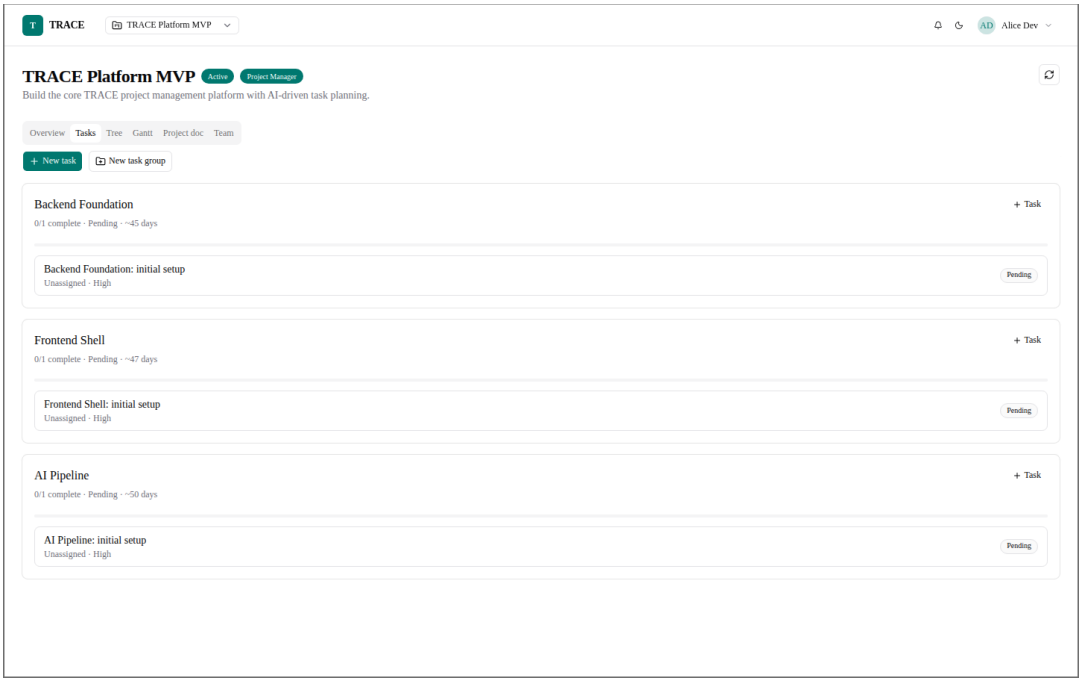


Figure 70: Employee — Project Tasks

The Project Manager's task board, with New task and New task group.

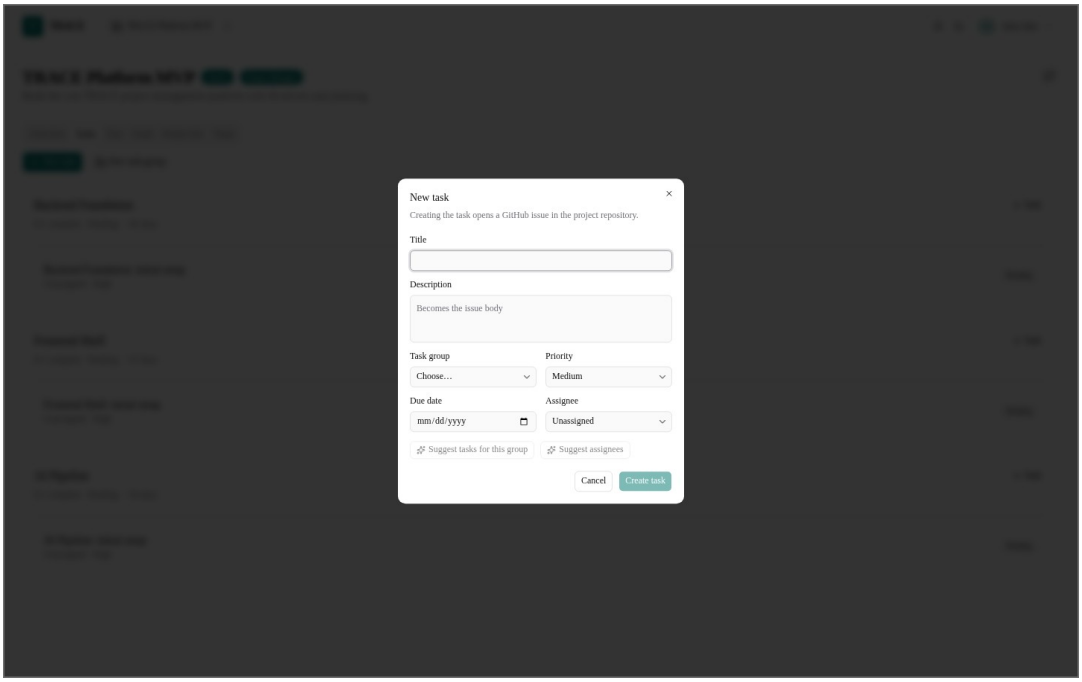


Figure 71: Employee — New Task

A new task opens a GitHub issue; the AI can suggest tasks for a group and suggest assignees.

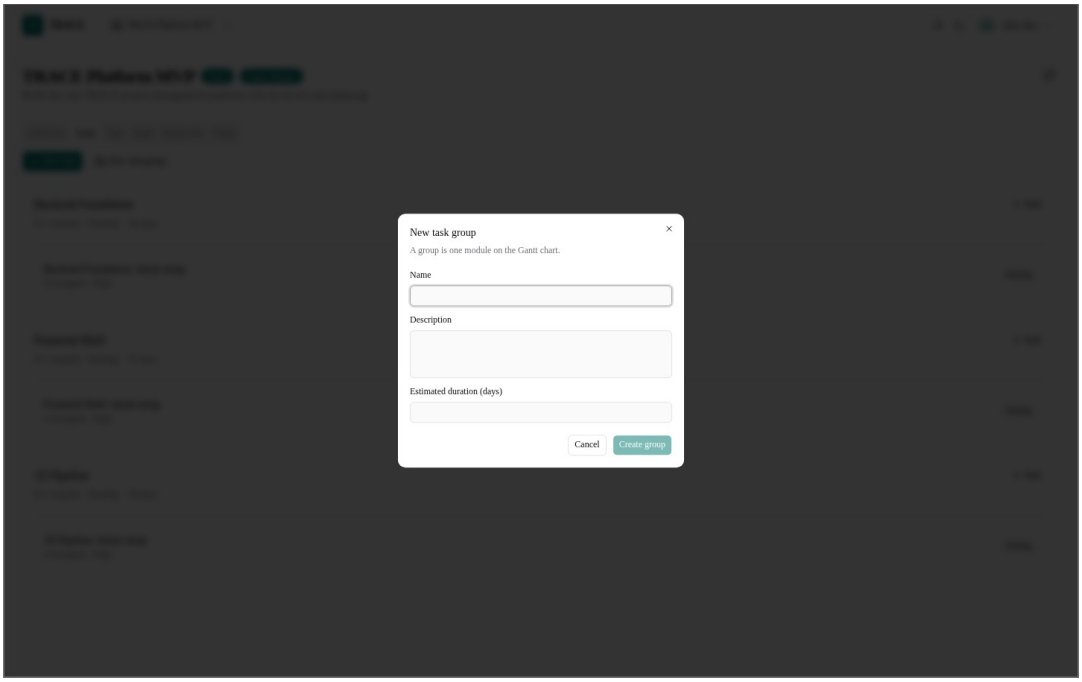


Figure 72: Employee — New Task Group

A task group is one module on the Gantt chart, with an estimated duration.

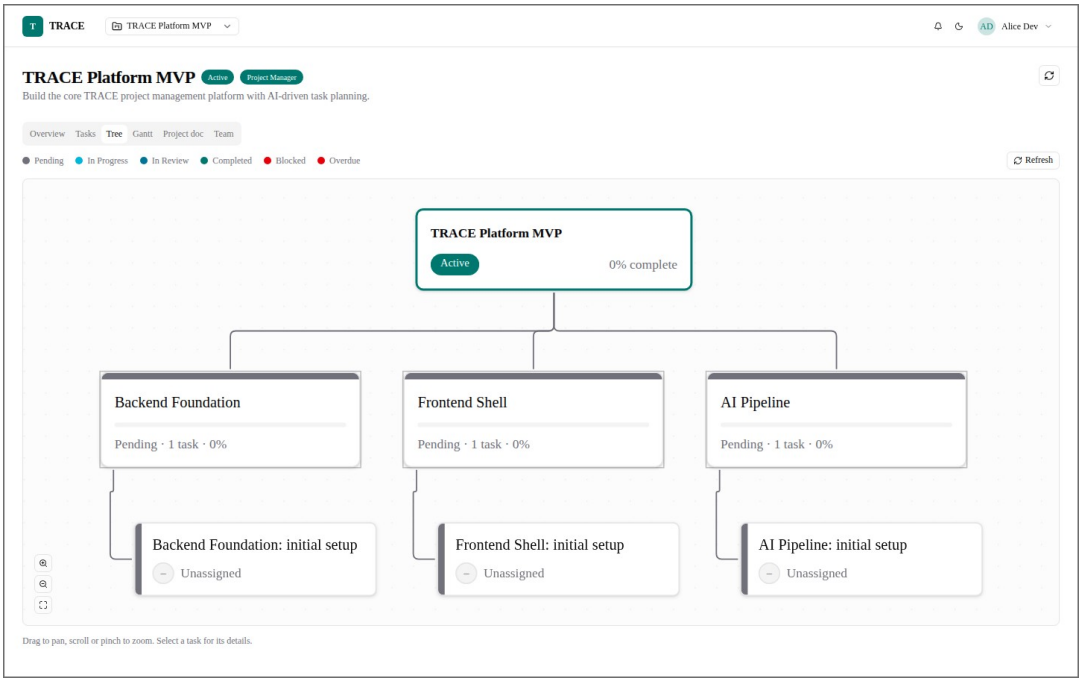


Figure 73: Employee — Project Tree

The project tree as the Project Manager sees it.

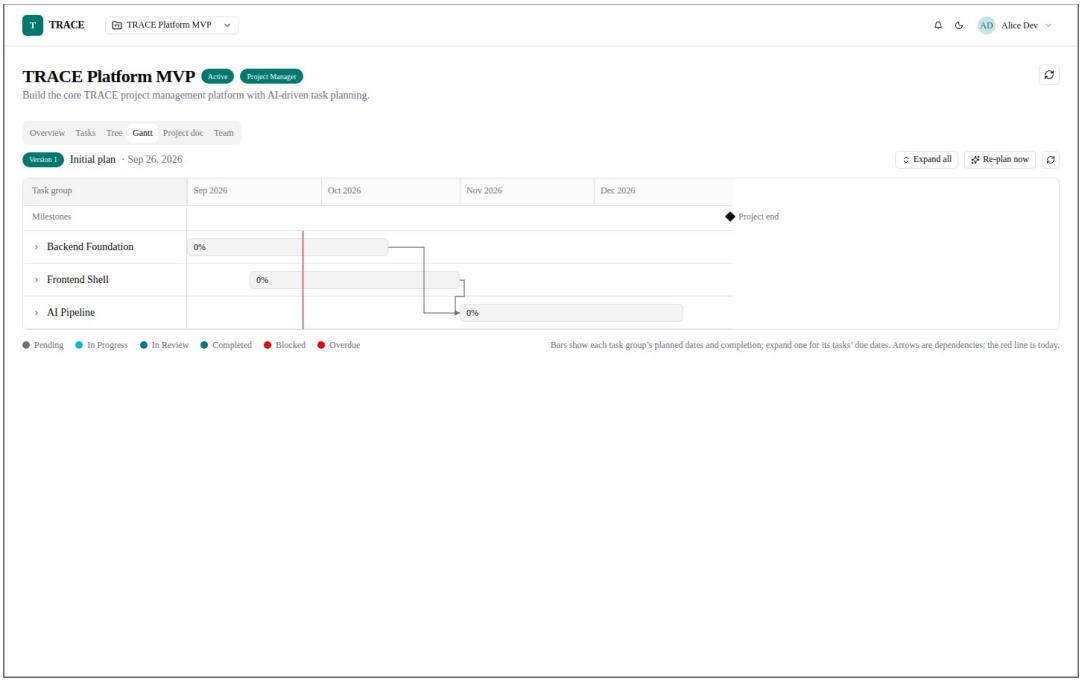


Figure 74: Employee — Gantt Chart

The Gantt chart, with Re-plan now for the Project Manager.

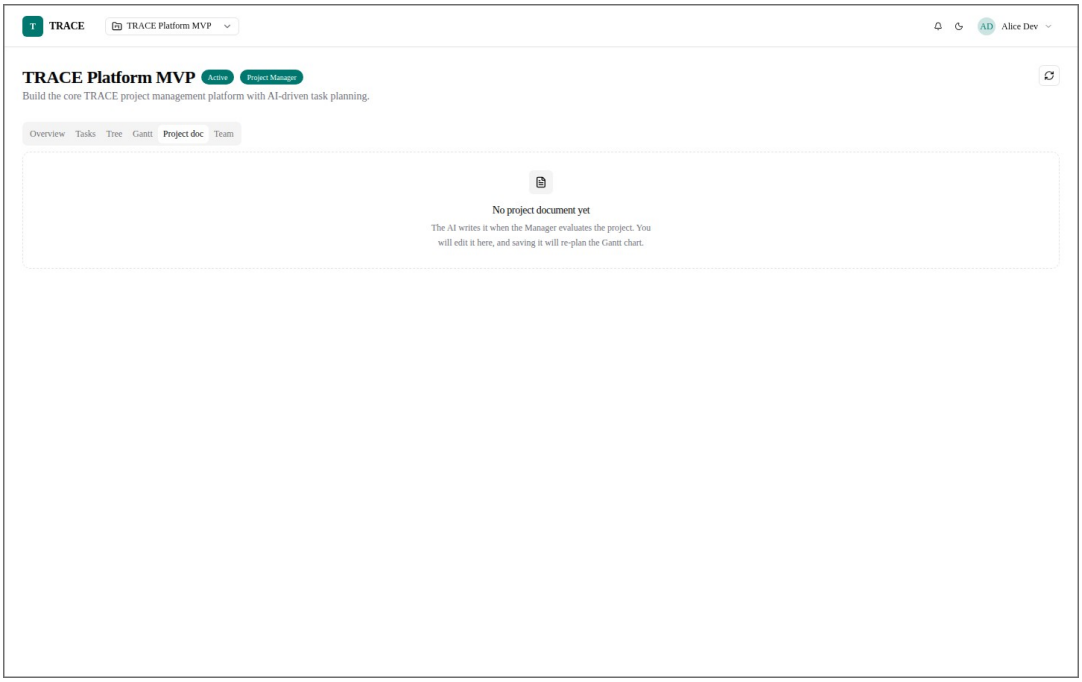


Figure 75: Employee — Project Document (Empty)

Before evaluation there is no project document; the AI writes it when the project is evaluated.

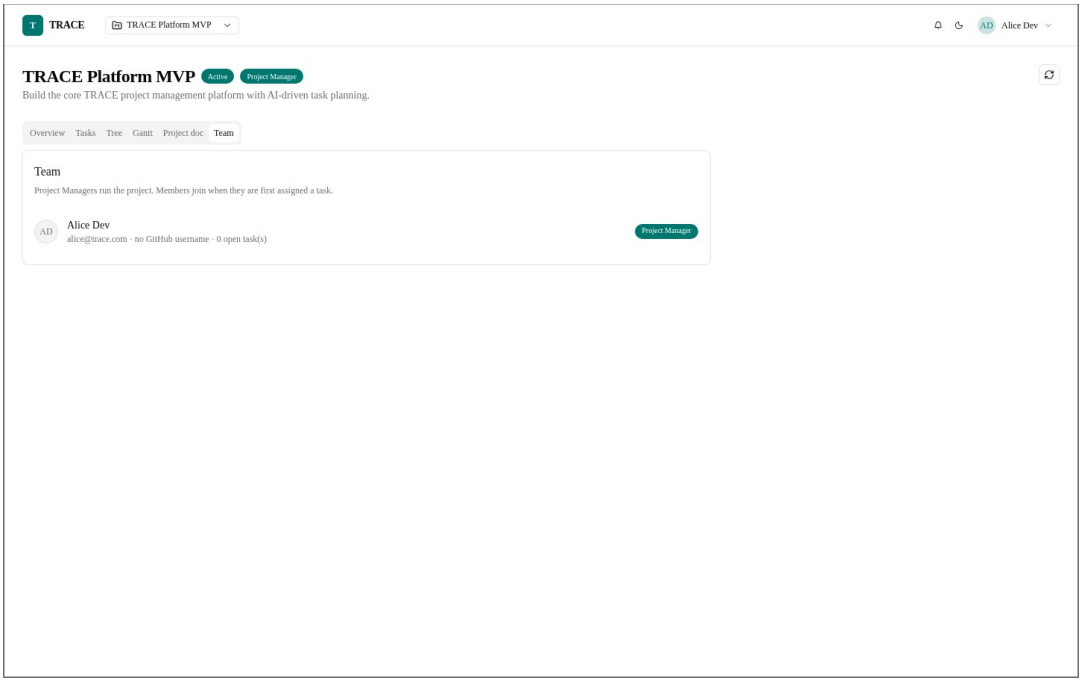


Figure 76: Employee — Team

The team of the project as the Project Manager sees it.

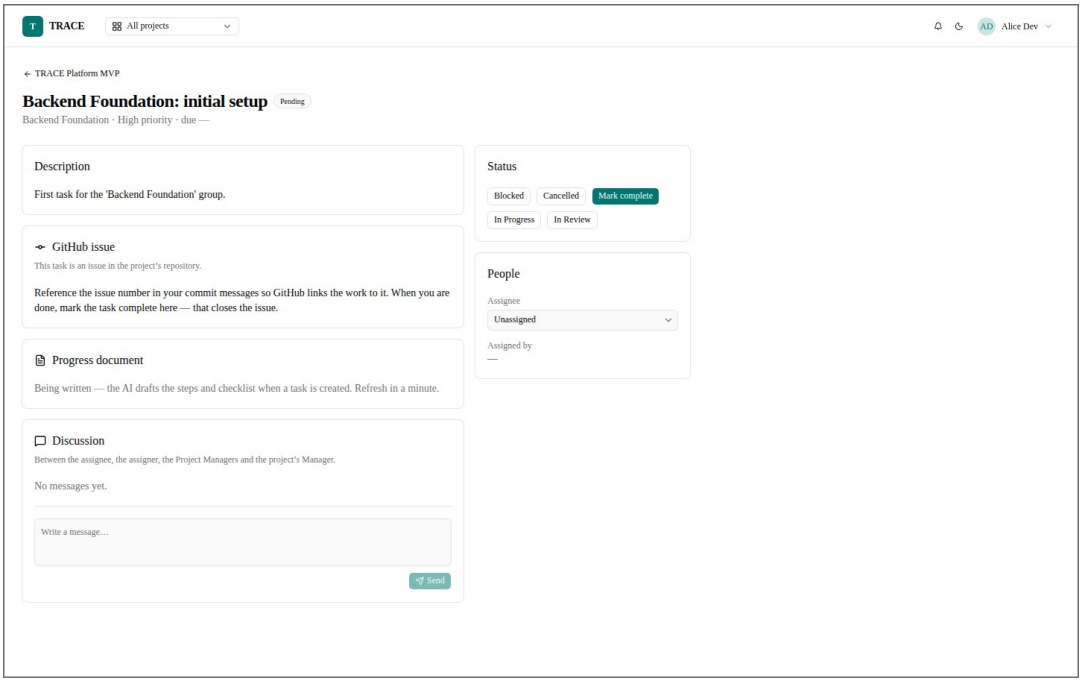


Figure 77: Employee — Task Detail (Project Manager)

A task with the status controls — Blocked, Cancelled, Mark complete, In Progress, In Review — and its assignee.

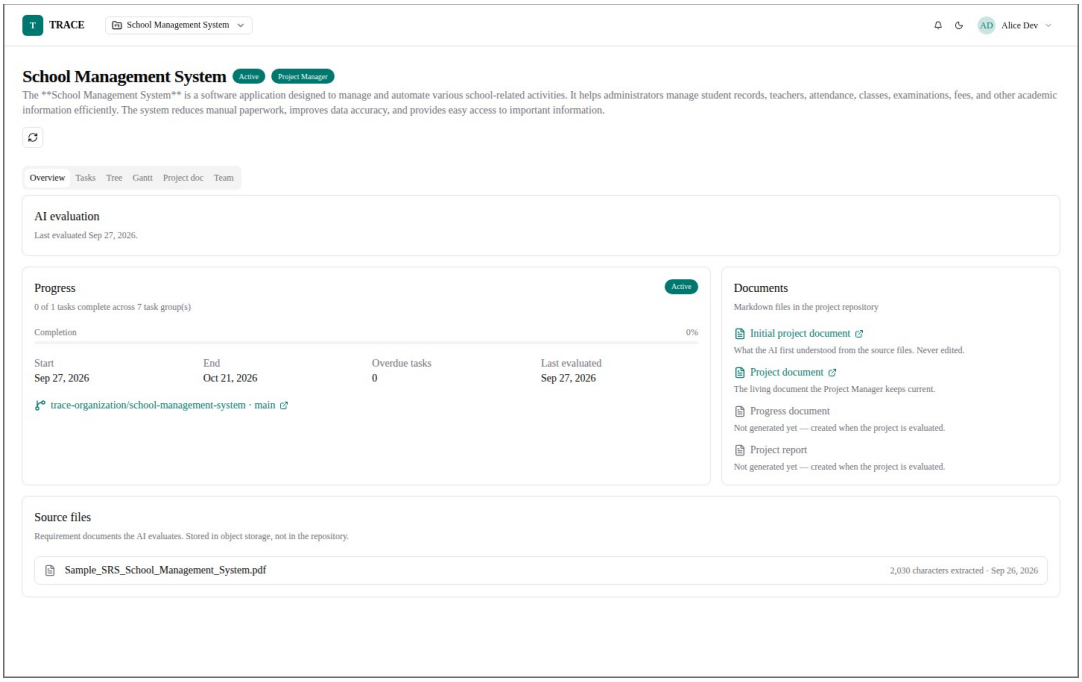


Figure 78: Employee — Project Overview (Evaluated)

An evaluated project with its documents linked in the project's GitHub repository.

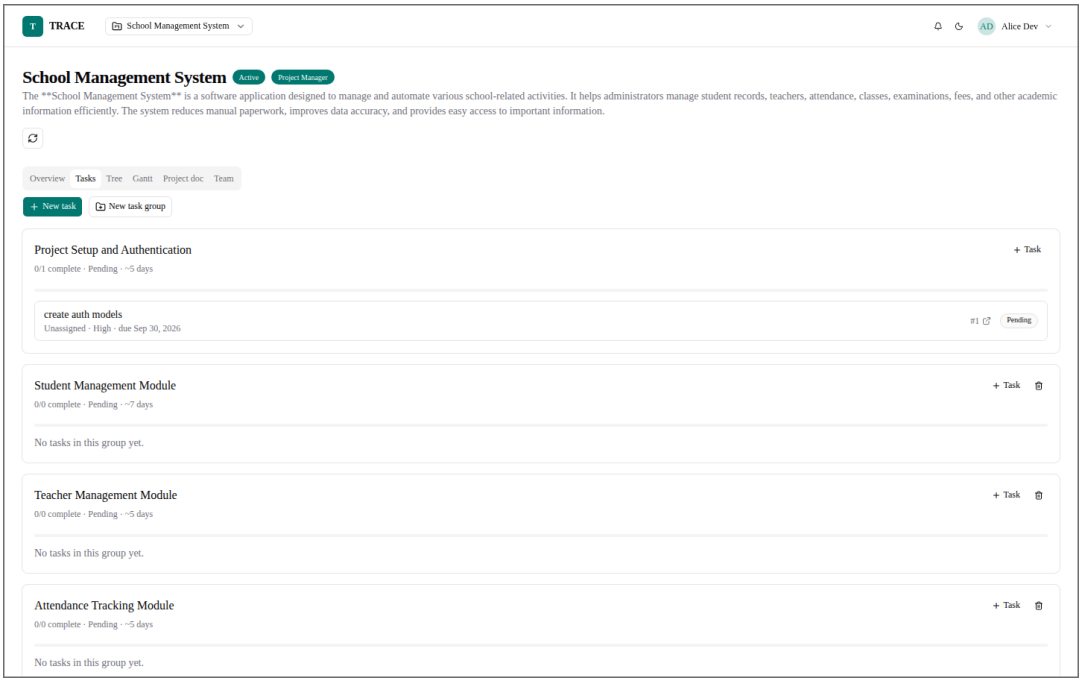


Figure 79: Employee — Project Tasks (Evaluated)

The task groups of the evaluated project, with the first task and its issue number.

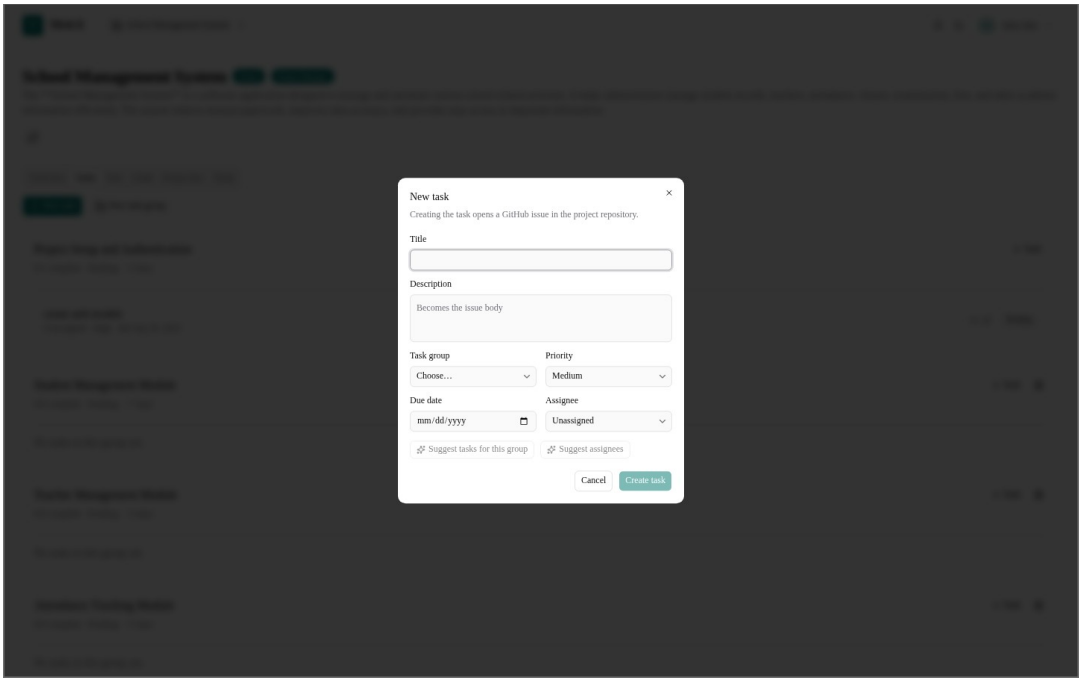


Figure 80: Employee — New Task (Evaluated)
Creating a task in one of the AI-made task groups.

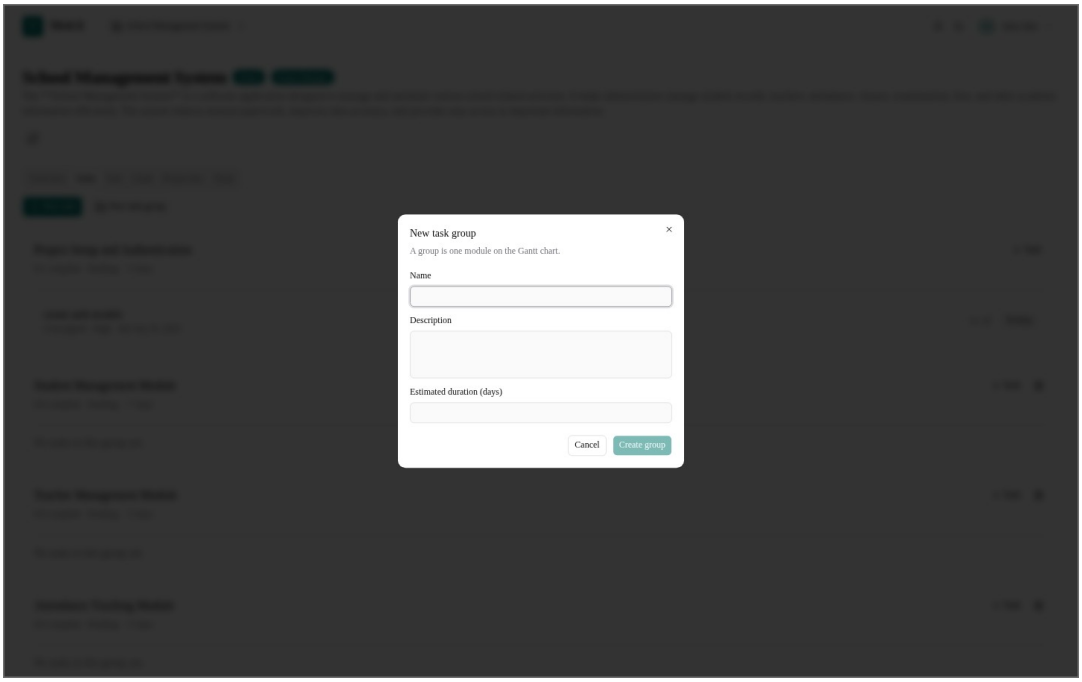


Figure 81: Employee — New Task Group (Evaluated)
Adding a task group to an evaluated project.

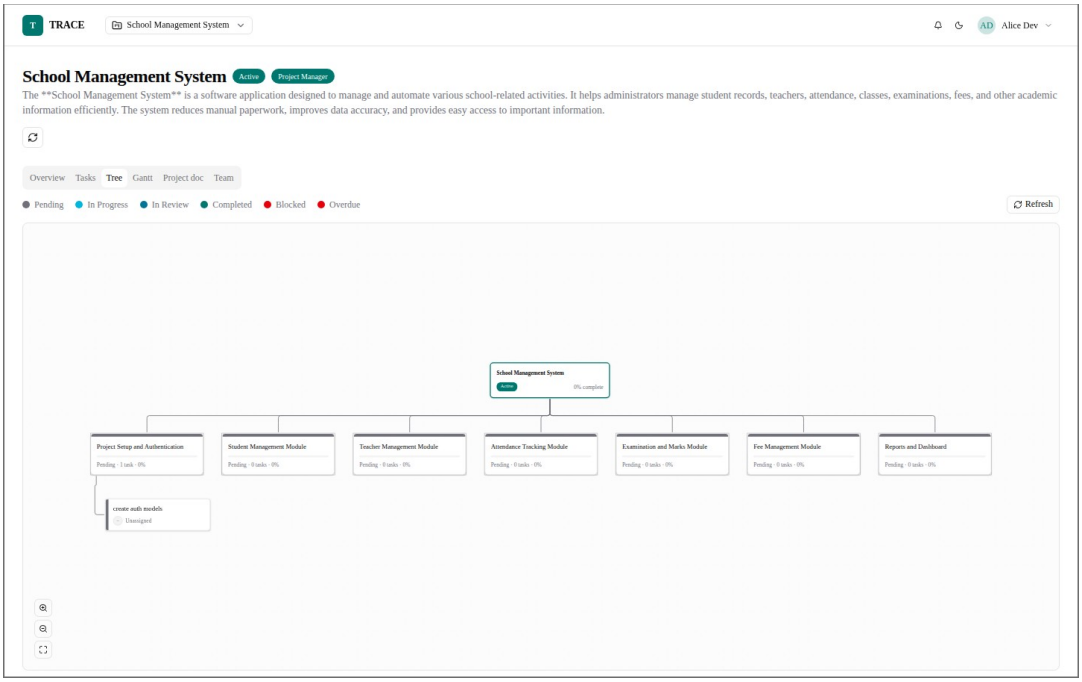


Figure 82: Employee — Project Tree (Evaluated)

The tree of the evaluated project.

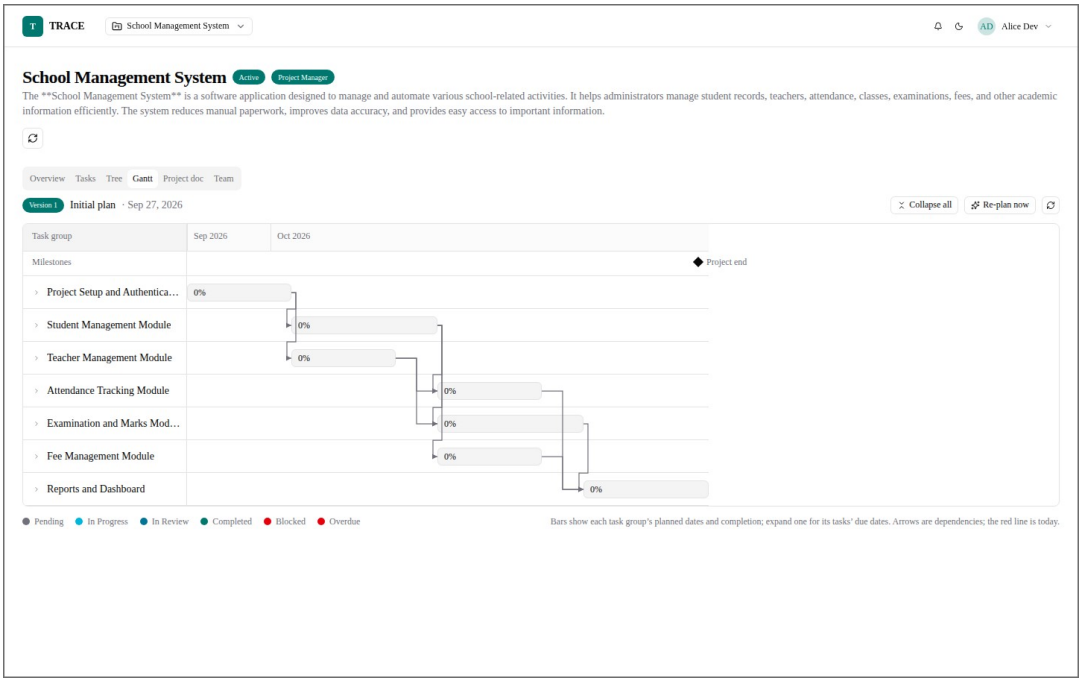


Figure 83: Employee — Gantt Chart (Evaluated)

The evaluated project's plan, with Re-plan now.

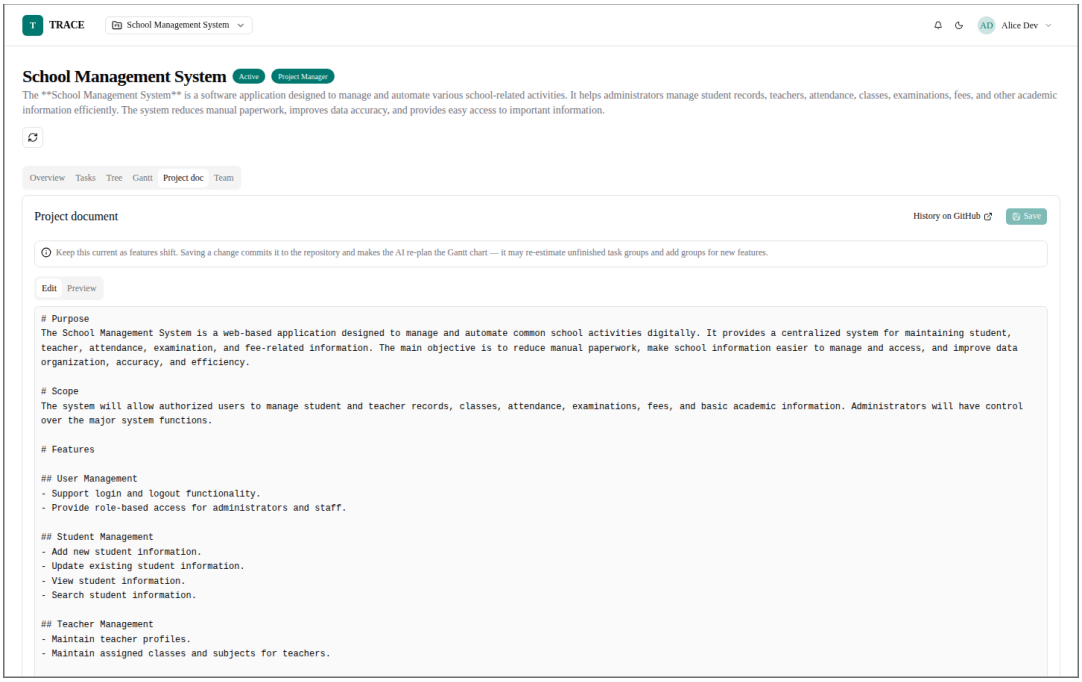


Figure 84: Employee — Project Document — Edit

The Markdown editor; saving commits the document to GitHub and has the AI re-plan the Gantt chart.

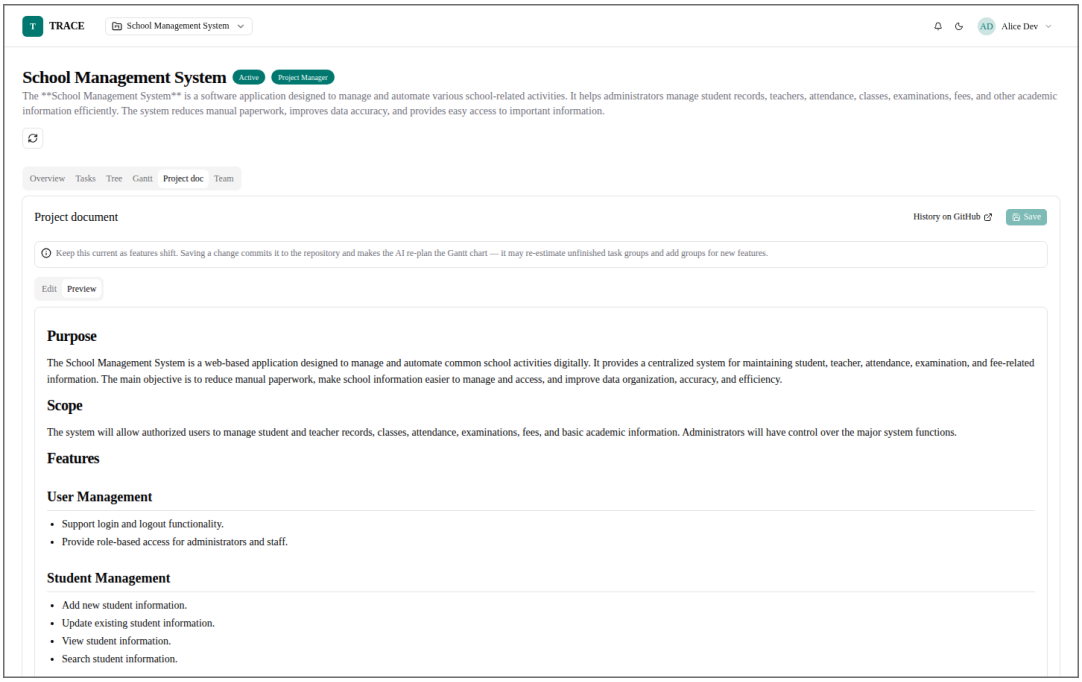


Figure 85: Employee — Project Document — Preview

The same document rendered, as it reads in the repository.

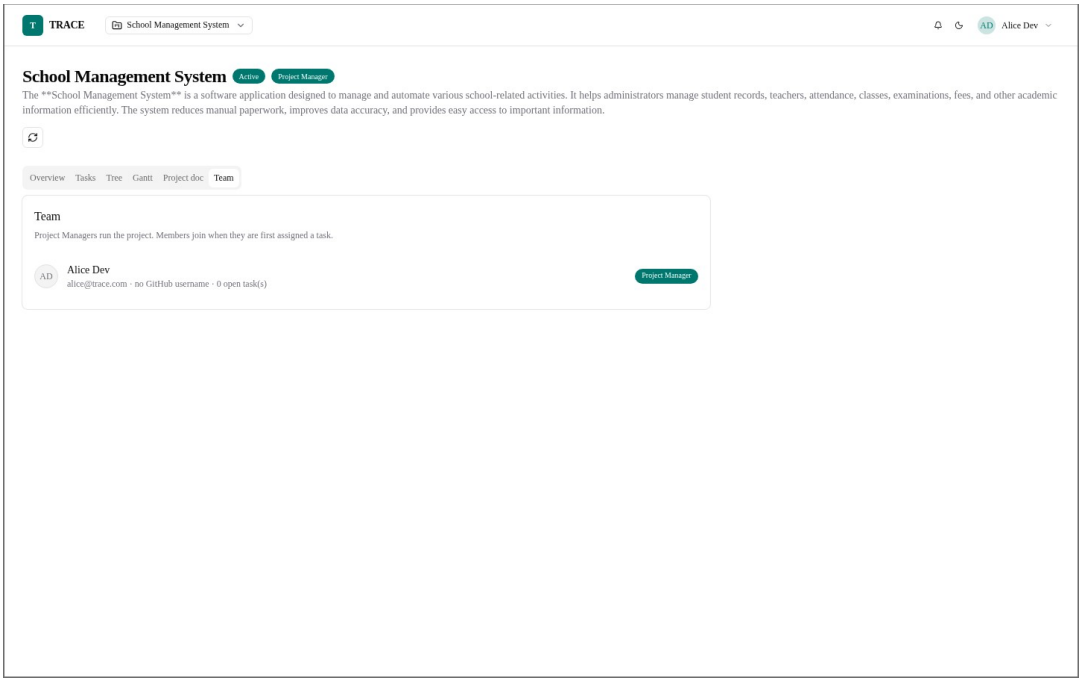


Figure 86: Employee — Team (Evaluated)

The team of the evaluated project.

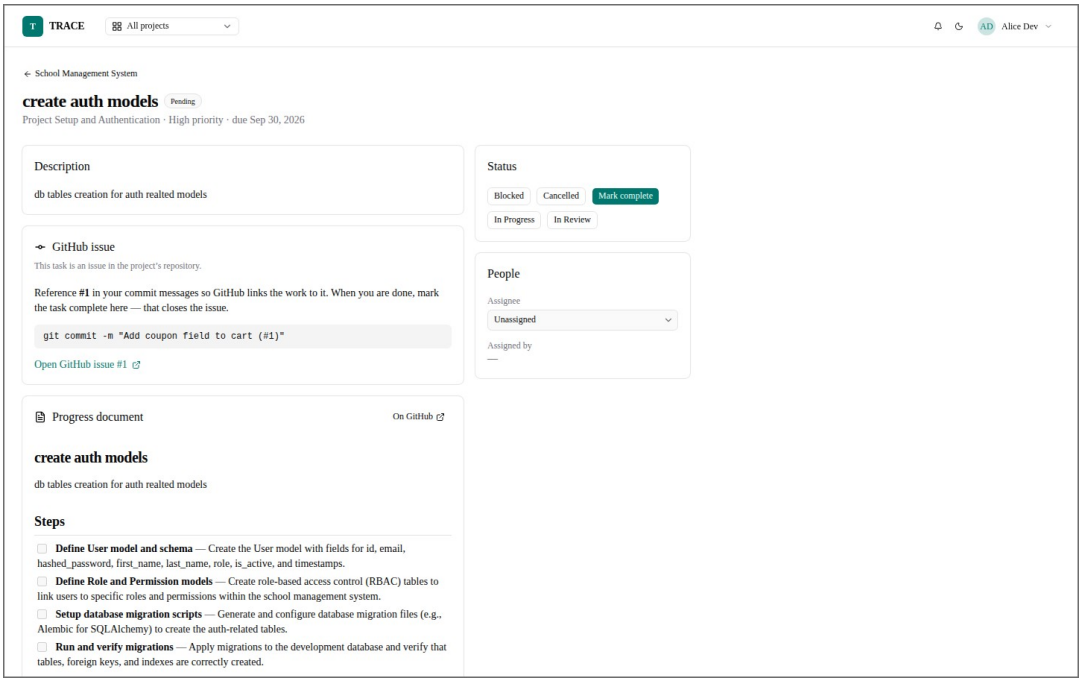


Figure 87: Employee — Task Detail with Plan

The task's GitHub issue, the commit message to use, and the AI-written steps in its progress document.

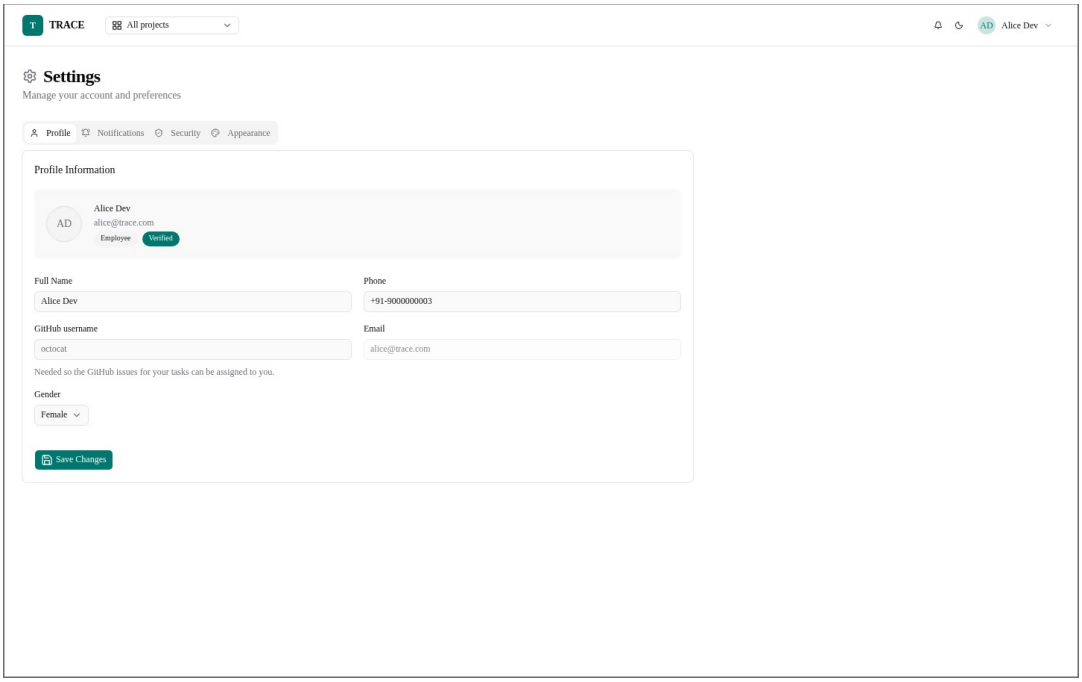


Figure 88: Employee — Settings — Profile

Name, phone, gender and the GitHub username that tasks' issues are assigned to.

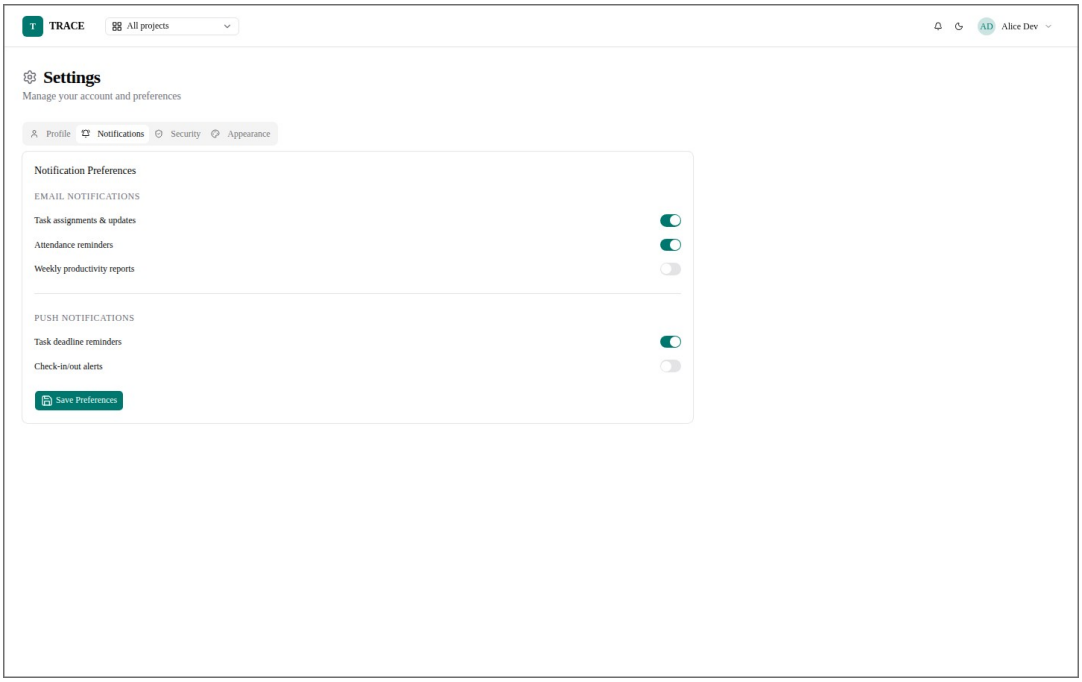


Figure 89: Employee — Settings — Notifications

Which e-mail and push notifications the user receives.

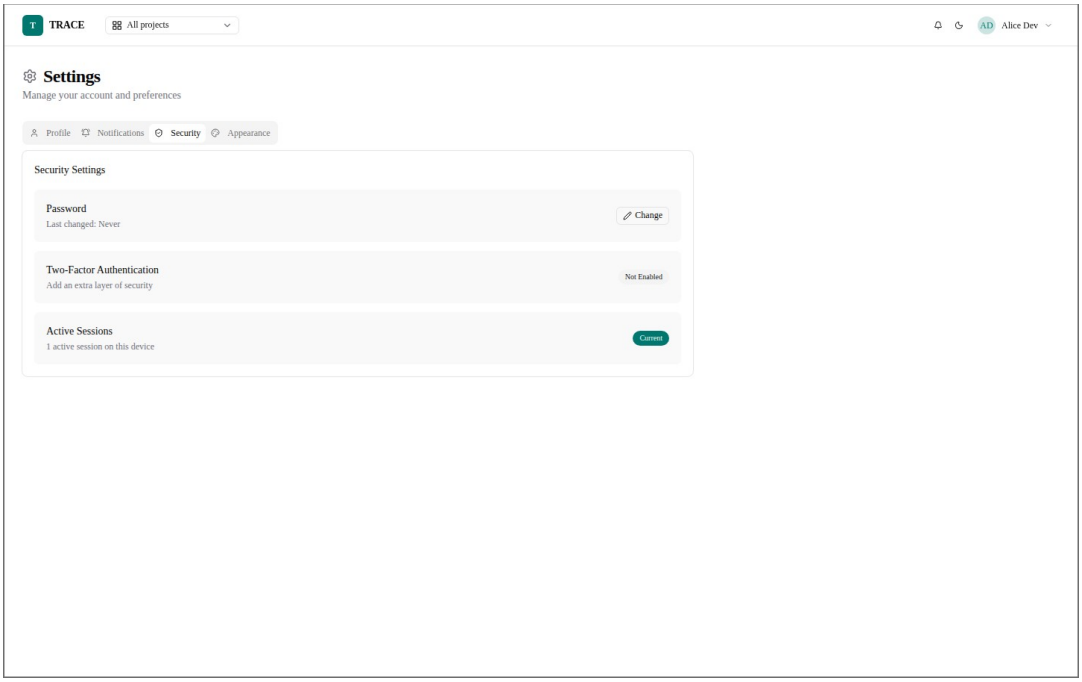


Figure 90: Employee — Settings — Security
Password, two-factor authentication and active sessions.

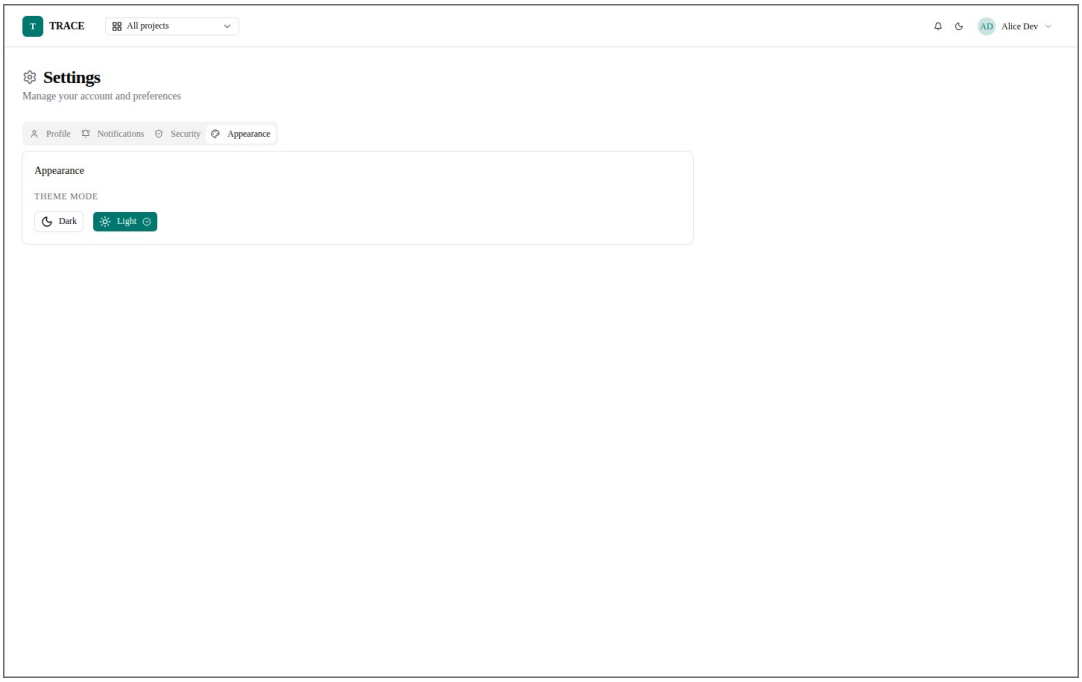


Figure 91: Employee — Settings — Appearance
Light or dark theme.

10. FUTURE SCOPE AND FURTHER ENHANCEMENT OF THE PROJECT

10.1 Limitations of the Present System

- **Public repositories only.** Every repository TRACE creates is public, so generated documents (including the AI's summary of the requirements) and issues are world-readable. This suits an educational deployment but not confidential client work.
- **One GitHub Organization per installation**, accessed with one token; personal GitHub accounts are not supported.
- **Dependence on external services.** Task creation needs GitHub, and the AI features need an AI provider; when either is unavailable those actions are refused (the rest of the system keeps working).
- **AI quality depends on the input.** The plan is only as good as the requirement documents; scanned PDFs without a text layer cannot be read (no OCR).
- **The task checklist is written once.** A task's progress document is not ticked off automatically as work proceeds; the task's status is the record.
- **No branch strategy.** Documents are committed to the default branch; per-task branches are not created.
- **Desktop-first interface.** The screens are designed for desktop browsers; there is no mobile application.
- **Members are not removed from projects**, and there is no *resend invitation* action; assigning a task again re-sends the invitation.
- **Two-factor authentication and session management** are not yet available.

10.2 Future Enhancements

- **Private repositories and per-project visibility**, with GitHub branch protection.
- **GitHub App integration** instead of a personal token — short-lived, per-installation credentials not tied to one person, and finer permissions.
- **Branch per task.** Create a branch named from the issue number when a task starts, and link pull requests to tasks.
- **Critical-path overlay on the Gantt chart**, with projected completion dates computed from dependencies.
- **Cross-project workload view** to detect people over-allocated across several projects at once, and use it in assignee ranking.

- **Scope-drift detection.** Compare the living project document and the work actually done against the immutable initial document, and warn when the project drifts from its original brief.
- **Chat-operations in task discussions** — for example `@trace create task …` to create a task in natural language.
- **Reusable project templates** — a standard task-group structure instantiated in one click.
- **Contribution heat-map** of commits and completions per person over time.
- **Multimodal intake** — whiteboard photos and voice notes as source documents, with OCR for scanned PDFs.
- **AI fairness audit** — periodic checks that suggestion scores show no systematic bias.
- **Explainable dispute queue** — if the AI is ever allowed to decide things, a way to contest its verdicts with a human override.
- **Mobile application** (React Native) and a responsive pass over all panels.
- **Two-factor authentication**, session management and single sign-on.
- **Integrations** with Slack or Microsoft Teams for notifications, and with calendars for due dates.
- **Export formats** — PDF and Excel versions of the project report, alongside CSV.
- **Multi-tenant SaaS deployment**, each company with isolated data and its own GitHub Organization.

11. BIBLIOGRAPHY

Books

- Pressman, Roger S. and Maxim, Bruce R. — *Software Engineering: A Practitioner's Approach*, McGraw-Hill Education.
- Sommerville, Ian — *Software Engineering*, Pearson Education.
- Mall, Rajib — *Fundamentals of Software Engineering*, PHI Learning.
- Elmasri, Ramez and Navathe, Shamkant B. — *Fundamentals of Database Systems*, Pearson Education.
- Silberschatz, Abraham, Korth, Henry F. and Sudarshan, S. — *Database System Concepts*, McGraw-Hill Education.
- Lutz, Mark — *Learning Python*, O'Reilly Media.
- Banks, Alex and Porcello, Eve — *Learning React*, O'Reilly Media.

Standards

- IEEE Std 830-1998 — *IEEE Recommended Practice for Software Requirements Specifications*.
- RFC 7519 — *JSON Web Token (JWT)*, Internet Engineering Task Force.
- RFC 6455 — *The WebSocket Protocol*, Internet Engineering Task Force.

Official documentation

- FastAPI — https://fastapi.tiangolo.com
- Pydantic — https://docs.pydantic.dev
- SQLAlchemy 2.0 — https://docs.sqlalchemy.org
- Alembic — https://alembic.sqlalchemy.org
- PostgreSQL — https://www.postgresql.org/docs/
- Celery — https://docs.celeryq.dev
- Redis — https://redis.io/docs/
- React — https://react.dev
- Vite — https://vite.dev
- Tailwind CSS — https://tailwindcss.com/docs
- shadcn/ui — https://ui.shadcn.com
- React Flow (@xyflow/react) — https://reactflow.dev
- GitHub REST API — https://docs.github.com/en/rest
- OpenAI API reference — https://platform.openai.com/docs
- MinIO — https://min.io/docs
- Docker and Docker Compose — https://docs.docker.com

- Cryptography (Fernet) — https://cryptography.io
- pytest — https://docs.pytest.org

Other resources

- IGNOU MCA project guidelines (MCSP-232).
- Stack Overflow — https://stackoverflow.com
- MDN Web Docs — https://developer.mozilla.org

APPENDICES

Appendix A — API Endpoint Reference

All endpoints are under `/api/v1`. The *Guard* column is the platform role checked on the route; project routes also check the project role.

Table 54: Principal API endpoints

Method	Path	Purpose	Guard
POST	/auth/login	Sign in; returns tokens and roles	Public
POST	/auth/refresh	New access token from a refresh token	Public
POST	/auth/register	Register (Employee only, unless called by an Admin)	Public
GET	/projects	Projects visible to the caller, with statistics	Any role
POST	/projects	Create a project and its repository	Manager
GET	/projects/report	CSV report for a date range	Manager, Employee
GET / PATCH / DELETE	/projects/{id}	Detail, edit, delete (archives repository)	Any / owner
POST / DELETE	/projects/{id}/source-files[/{fid}]	Upload with text extraction; remove	Owner
GET / POST / DELETE	/projects/{id}/assignments[/{uid}]	Team; appoint or remove a Project Manager	Owner
GET / POST / PATCH / DELETE	/projects/{id}/task-groups[/{gid}]	Task groups	Project Manager
GET / POST	/projects/{id}/tasks	List; create task and GitHub issue	Project Manager to create
GET	/tasks/mine	My open tasks across projects	Employee
GET / PATCH / DELETE	/tasks/{id}	Detail with permissions; edit details; delete	PM to change

Method	Path	Purpose	Guard
PATCH	/tasks/{id}/assign	Assign / unassign	Project Manager
PATCH	/tasks/{id}/status	Change status (status authority)	Assignee or PM
GET / POST	/tasks/{id}/discussions	Discussion thread	Participants
GET	/projects/{id}/tree	Project tree graph	Project users
GET	/projects/{id}/gantt[?version=N], /gantt/versions	Gantt and its history	Project users
POST	/projects/{id}/gantt/reevaluate	Re-plan now	Project Manager
POST / GET	/projects/{id}/evaluate[/status]	Evaluate Project; its state	Owner / project users
POST	/projects/{id}/suggest-managers	AI-ranked PM candidates	Owner
POST	/projects/{id}/tasks/suggest-assignees	AI-ranked assignees	Project Manager
POST	/projects/{id}/task-groups/{gid}/suggest-tasks	AI task suggestions	Project Manager
POST	/projects/{id}/progress-report	Rewrite progress and report documents now	Owner or PM
GET / PUT	/projects/{id}/documents/{kind}	Read a project document; save the living one	Project users / PM
GET	/tasks/{id}/progress-doc	The task's plan and checklist	Project users
GET / PATCH / POST	/notifications, /{id}/read, /read-all	Notification centre	Any role
WS	/ws?token=…	Live "refetch" events	Valid token
GET / PUT / POST	/admin/settings/github, /mail, …/test	Configuration and connection tests	Admin
—	/admin/tables, /workforce/{entity}[/bulk]	Control Panel CRUD; Workforce CRUD and bulk upload	Admin
GET	/services/health/*, /services/jobs/recent	Monitor	Admin
GET	/services/task/{job_id}/result	Poll a background job	Any role

Appendix B — Installation and Running

Prerequisites: Docker with Docker Compose, Python 3.11+, Node.js 20+, a GitHub Organization with a fine-grained token, and an OpenAI-compatible API key.

1. Copy `.env.example` to `.env` at the repository root and fill in the values (database, JWT secret, encryption key, AI key and model, GitHub organization and token, SMTP).

2. Install the backend dependencies into `backend/.venv` (`pip install -r requirements.txt`) and the frontend dependencies (`npm install` in `frontend/`).
3. Start everything with `./start.sh start` — this starts PostgreSQL, Redis, MinIO, Mailpit and pgAdmin in Docker, runs the migrations and seeds, and starts the API, the Celery worker and beat, and the frontend.
4. Open http://localhost:5173 and sign in as the Admin; set the GitHub organization and token in *Configuration* and press *Test connection*.

Table 55: Commands

Command	Purpose
`./start.sh start` / `stop` / `restart`	Start, stop or restart the whole stack
`./start.sh migrate`	Apply database migrations
`./start.sh status`	Show what is running
`./start.sh set-password <email>`	Set a user's password
`.venv/bin/python -m pytest -q` (in `backend/`)	Run the automated test suite

Table 56: Service addresses (local)

Service	Address
Frontend	http://localhost:5173
API and interactive API documentation	http://localhost:8000, http://localhost:8000/docs
pgAdmin	http://localhost:5050
Mailpit (development mail)	http://localhost:8025
MinIO console	http://localhost:9001

Table 57: Main environment settings (.env)

Key	Meaning
`APP_ENV`	`local` or `production`
`POSTGRES_DB`, `POSTGRES_USER`, `POSTGRES_PASSWORD`, `DB_PORT`	Database
`REDIS_PORT`	Redis (Celery broker, live events)
`MINIO_ROOT_USER`, `MINIO_ROOT_PASSWORD`, `MINIO_BUCKET`	Object storage

Key	Meaning
JWT_SECRET_KEY	Token signing secret
SETTINGS_ENCRYPTION_KEY	Fernet key for stored secrets
OPENAI_API_KEY, OPENAI_MODEL, OPENAI_BASE_URL	AI model (fixed here; not changeable in the application)
GITHUB_OWNER, GITHUB_TOKEN, GITHUB_COMMITTER_NAME, GITHUB_COMMITTER_EMAIL	GitHub fallback values (Configuration overrides them)
SMTP_FROM and SMTP settings	Outgoing mail

Appendix C — Scheduled Jobs

Table 58: Celery beat schedule

Job	When	Purpose
mark_overdue_tasks	Every night, 23:00	Open past-due tasks → Overdue; notify; re-plan affected Gantt charts
task_due_reminders	Every day, 08:00	Remind assignees and PMs of tasks due tomorrow
update_project_progress_docs	Mondays, 07:00	AI rewrites every active project's progress and report documents
compute_all_weekly_scores	Sundays, 23:30	Task-based performance scores used as evidence in AI ranking

GLOSSARY

Table 59: Glossary of terms

Term	Meaning
Access token	Short-lived signed token sent with every request to prove who the user is
Alembic	Tool that applies versioned changes (migrations) to the database schema
API	Application Programming Interface — the set of HTTP endpoints the frontend calls
ASGI	Asynchronous Server Gateway Interface — the standard FastAPI runs on
Assignee	The Member a task is assigned to; the person doing it
bcrypt	Slow, salted password-hashing algorithm
Celery	Distributed task queue that runs background jobs
Celery beat	Scheduler that queues jobs at fixed times
Critical path	The longest chain of dependent activities; it fixes the project's duration
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values — a plain-text table format
Debounce	Collapsing a burst of triggers into a single action
DFD	Data Flow Diagram
Docker Compose	Tool that runs several containers (database, Redis, MinIO …) together
ER diagram	Entity Relationship diagram
Evaluate Project	The AI action that turns source documents into a project document, task groups and a Gantt chart
FastAPI	Python web framework used for the backend
Fernet	Authenticated symmetric encryption used for stored secrets
Fine-grained token	A GitHub token limited to chosen repositories and permissions
Gantt chart	Bar chart of activities on a calendar
GitHub issue	A work item in a GitHub repository; in TRACE, one per task
GitHub Organization	Shared GitHub account owning the repositories TRACE creates
JSON	JavaScript Object Notation — the data format of the API
JSON mode	AI model option that forces the reply to be a JSON object
JSONB	PostgreSQL binary JSON column type
JWT	JSON Web Token (RFC 7519)
LLM	Large Language Model — the AI model used for planning and writing
Markdown	Plain-text formatting language used for the generated documents
Member	Employee assigned at least one task in a project
Migration	A versioned, reversible change to the database schema

Term	Meaning
MinIO	S3-compatible object store for uploaded files
ORM	Object-Relational Mapping — working with tables as classes (SQLAlchemy)
PERT	Programme Evaluation and Review Technique
Project document	The AI-written description of the project; an immutable initial copy and a living copy
Project Manager	Employee appointed to run one project
Pydantic	Python library that validates data against declared schemas
Re-plan	Regeneration of the Gantt chart as a new version
Redis	In-memory data store used as the job broker and for live events
Refresh token	Longer-lived token used only to obtain a new access token
REST	Representational State Transfer — style of HTTP API
Slug	URL-safe form of a name, used as the repository name
Soft delete	Marking a row deleted (`deleted_at`) instead of removing it
SPA	Single-Page Application
SRS	Software Requirement Specification
Status authority	The rule deciding who may set a task's status
Task group	A module of work in a project; one Gantt bar
Vite	Build tool and development server for the frontend
WebSocket	Persistent two-way connection used for live events